

TẬP ĐOÀN CÔNG NGHIỆP - VIỄN THÔNG QUÂN ĐỘI

BÁO CÁO MINI-PROJECT

Nghiên cứu Feature Flag và xây dựng hệ thống quản lý phát hành tính năng an toàn

Bùi Đình Bảo

dinhbao.itech@gmail.com - 0975 742 607

Chương trình Viettel Digital Talent 2026

Lĩnh vực: Software Engineer

Mentor:	Vũ Tuấn Đạt
Đơn vị:	Viettel Networks

Lời cảm ơn

Trước hết, em xin gửi lời cảm ơn chân thành đến Ban Tổ chức chương trình Viettel Digital Talent 2026 đã tạo điều kiện để em được tham gia học tập, rèn luyện và phát triển trong một môi trường chuyên nghiệp.

Em xin cảm ơn các anh mentor đã trực tiếp giảng dạy và chia sẻ kiến thức trong các buổi training của chương trình. Mỗi buổi đào tạo với một chủ đề và góc nhìn khác nhau đã mang lại cho em nhiều kiến thức, kinh nghiệm thực tế và những định hướng có giá trị trong quá trình học tập và hoàn thiện bản thân.

Em cũng xin cảm ơn mentor phụ trách Mini Project cùng Ban Tổ chức đã cung cấp định hướng, yêu cầu và những góp ý cần thiết trong quá trình thực hiện. Sự hỗ trợ của các anh đã giúp em xác định rõ mục tiêu, chủ động hơn trong quá trình nghiên cứu và hoàn thành sản phẩm.

Bên cạnh đó, em xin cảm ơn các bạn học viên trong chương trình đã cùng trao đổi, chia sẻ và hỗ trợ nhau trong suốt quá trình tham gia Viettel Digital Talent 2026.

Mặc dù đã dành nhiều thời gian và nỗ lực để hoàn thành báo cáo và sản phẩm, do kinh nghiệm còn hạn chế, nội dung thực hiện khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ Ban Tổ chức và các anh/chị mentor để có thể tiếp tục cải thiện và hoàn thiện bản thân.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 7 năm 2026

Học viên thực hiện

Bùi Đình Bảo

Lời mở đầu

Trong giai đoạn đầu của ngành công nghiệp phần mềm, các hệ thống thường được phát triển và phát hành theo những phiên bản lớn, với chu kỳ kéo dài từ vài tuần đến vài tháng. Mỗi lần phát hành thường bao gồm nhiều thay đổi được đóng gói và triển khai cùng nhau. Trong mô hình này, việc đưa mã nguồn lên môi trường production và việc cung cấp tính năng cho người dùng gần như là một hoạt động thống nhất: khi phiên bản mới được triển khai, các tính năng bên trong phiên bản đó cũng đồng thời được phát hành.

Cách tiếp cận trên tương đối phù hợp với những hệ thống có tốc độ thay đổi thấp. Tuy nhiên, khi quy mô phần mềm, số lượng người dùng và tần suất cập nhật ngày càng tăng, việc phát hành nhiều thay đổi cùng lúc làm gia tăng đáng kể rủi ro vận hành. Một lỗi trong tính năng mới có thể ảnh hưởng đến toàn bộ người dùng, trong khi quá trình khắc phục thường yêu cầu rollback phiên bản hoặc thực hiện một lần triển khai mới. Bên cạnh đó, việc duy trì các nhánh phát triển tồn tại trong thời gian dài còn làm phát sinh xung đột mã nguồn, tăng chi phí tích hợp và khiến các thay đổi khó được kiểm chứng sớm.

Sự phát triển của các phương pháp Agile, Continuous Integration, Continuous Delivery và trunk-based development đã thúc đẩy các nhóm phát triển tích hợp những thay đổi nhỏ thường xuyên hơn, đồng thời duy trì mã nguồn ở trạng thái có thể triển khai. Tuy nhiên, không phải mọi phần mã đã được tích hợp đều sẵn sàng để cung cấp ngay cho người dùng. Từ đó hình thành nhu cầu đưa mã nguồn của tính năng mới vào cùng phiên bản với hệ thống hiện tại nhưng vẫn có khả năng che giấu, vô hiệu hóa hoặc chỉ kích hoạt tính năng trong những điều kiện nhất định.

Ở dạng sơ khai, nhu cầu này có thể được giải quyết bằng các biến điều kiện, tham số dòng lệnh, biến môi trường hoặc tệp cấu hình. Những cơ chế bật, tắt hành vi phần mềm như vậy đã tồn tại từ trước khi thuật ngữ Feature Flag được sử dụng phổ biến. Khi hệ thống phát triển lớn

hơn, các cấu hình đơn giản dần được tổ chức thành một cơ chế có cấu trúc, cho phép lựa chọn giữa những luồng xử lý khác nhau tại thời điểm chạy. Cơ chế đó được gọi bằng nhiều tên như Feature Toggle, Feature Flag, Feature Switch hoặc Feature Gate. Pete Hodgson mô tả Feature Toggle là tập hợp các kỹ thuật giúp đội ngũ cung cấp chức năng mới nhanh chóng nhưng vẫn kiểm soát được mức độ an toàn; kỹ thuật này cho phép thay đổi hành vi hệ thống mà không cần sửa đổi và triển khai lại mã nguồn. [1]

Sự phổ biến của Feature Flag gắn liền với một thay đổi quan trọng trong tư duy phát triển phần mềm: deploy không đồng nghĩa với release. Deploy là quá trình đưa một phiên bản mã nguồn vào môi trường vận hành, trong khi release là quyết định cho phép người dùng thực sự tiếp cận một tính năng. Khi hai hoạt động này được tách biệt, đội phát triển có thể triển khai mã nguồn sớm nhưng giữ tính năng ở trạng thái tắt, kiểm thử tính năng trên production với một nhóm nội bộ, phát hành cho một tỷ lệ nhỏ người dùng và chỉ mở rộng phạm vi khi đã có đủ độ tin cậy.

Từ một công tắc bật hoặc tắt đơn giản, Feature Flag dần được sử dụng cho nhiều mục đích khác nhau. Release Flag hỗ trợ tích hợp mã chưa sẵn sàng mà không làm ảnh hưởng đến người dùng; Experiment Flag phục vụ A/B testing và đánh giá tác động của các phương án sản phẩm; Permission Flag giới hạn tính năng theo nhóm người dùng hoặc quyền truy cập; Ops Flag và Kill Switch giúp đội vận hành nhanh chóng vô hiệu hóa một chức năng khi phát hiện lỗi, suy giảm hiệu năng hoặc áp lực tải bất thường. Việc phân loại này cho thấy Feature Flag không chỉ là một câu lệnh điều kiện trong mã nguồn mà còn là một công cụ hỗ trợ quản lý phát hành, thử nghiệm sản phẩm và vận hành hệ thống. [1]

Tuy nhiên, khi số lượng flag, môi trường và điều kiện đánh giá tăng lên, việc quản lý bằng biến môi trường hoặc tệp cấu hình riêng lẻ trở nên khó kiểm soát. Trạng thái flag có thể không nhất quán giữa các máy chủ; người quản trị khó xác định ai đã thay đổi cấu hình; các ứng dụng phải tự cài đặt logic đánh giá; và những flag đã hết mục đích có thể tiếp tục tồn tại, tạo ra technical debt. Nghiên cứu thực nghiệm về việc sử dụng Feature Toggle trong doanh nghiệp cũng ghi nhận vai trò của các công cụ quản lý chuyên dụng, giá trị mặc định, metadata và nhật ký thay đổi đối với quá trình vận hành flag an toàn.

Những hạn chế trên tạo ra nhu cầu xây dựng một nền tảng Feature Flag tập trung. Thay vì để từng ứng dụng tự lưu trữ cấu hình và tự quyết định cách đánh giá flag, nền tảng cung cấp một nguồn cấu hình thống nhất, giao diện quản trị, cơ chế đánh giá có quy tắc, API tích hợp

và lịch sử thay đổi. Mô hình này giúp quá trình phát hành trở nên có kiểm soát, có khả năng truy vết và dễ mở rộng hơn khi số lượng ứng dụng hoặc đội phát triển tăng lên.

Xuất phát từ bối cảnh đó, đề tài tập trung nghiên cứu và xây dựng một nền tảng quản lý Feature Flag dành cho ứng dụng web. Hệ thống bao gồm dashboard quản trị để tạo và cấu hình flag; rule engine để đánh giá theo người dùng, vai trò và tỷ lệ rollout; evaluation API để các ứng dụng xác định trạng thái tính năng; audit log để ghi nhận thay đổi; cùng demo app nhằm minh họa quá trình tích hợp và sử dụng Feature Flag trong thực tế.

Việc thực hiện đề tài không chỉ nhằm xây dựng một chức năng bật hoặc tắt tính năng. Mục tiêu quan trọng hơn là nghiên cứu cách thiết kế một hệ thống có thể đưa ra quyết định nhất quán, xác định rõ thứ tự ưu tiên giữa các quy tắc, xử lý an toàn khi thiếu dữ liệu hoặc xảy ra lỗi, đồng thời cân bằng giữa tính linh hoạt, hiệu năng, khả năng kiểm soát và mức độ phức tạp của kiến trúc. Thông qua quá trình phân tích, thiết kế, triển khai và kiểm thử, đề tài hướng đến việc làm rõ giá trị của Feature Flag trong vòng đời phát triển phần mềm hiện đại, đồng thời thể hiện khả năng giải quyết vấn đề, tư duy thiết kế và khả năng giải thích các quyết định công nghệ được sử dụng trong hệ thống.

Tóm tắt nội dung và đóng góp

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Mục lục

1 Giới thiệu **1**

1.1 Đặt vấn đề 1

1.2 Mục tiêu của báo cáo 2

1.3 Câu hỏi nghiên cứu 2

1.4 Phạm vi triển khai 3

2 Nội dung **5**

2.1 Tổng quan về Feature Flag 5

2.1.1 Khái niệm Feature Flag 5

2.1.2 Deploy và Release 8

2.1.3 Vai trò của Feature Flag trong vòng đời phần mềm 11

2.1.4 Các loại Feature Flag phổ biến 14

2.1.5 Các chiến lược phát hành tính năng 18

2.1.6 Lợi ích và rủi ro của Feature Flag 22

2.2 Khảo sát các nền tảng Feature Flag hiện có 28

2.2.1 Nền tảng LaunchDarkly 28

2.2.2 Nền tảng Unleash 31

2.2.3 So sánh các giải pháp hiện có 35

2.2.4 Định hướng của hệ thống đề xuất 42

2.3 Phân tích bài toán và yêu cầu hệ thống 48

2.3.1 Các đối tượng sử dụng hệ thống 48

2.3.2 Các ca sử dụng chính 50

2.3.3 Yêu cầu chức năng mức MVP 53

2.3.4 Yêu cầu phi chức năng 55

2.3.5 Các yêu cầu khuyến khích 57
2.4 Kiến trúc tổng thể của hệ thống 59
2.4.1 Các thành phần chính 59
2.4.2 Control plane và data plane 62
2.4.3 Luồng quản trị Feature Flag 64
2.4.4 Luồng đánh giá Feature Flag 66
2.4.5 Luồng tích hợp ứng dụng khách 68
2.4.6 Lựa chọn công nghệ 71
2.5 Thiết kế dữ liệu 73
2.5.1 Mô hình dữ liệu tổng thể 73
2.5.2 Project và environment 79
2.5.3 Feature Flag và cấu hình theo environment 83
2.5.4 Rule và tham số rule 89
2.5.5 User context mẫu 94
2.5.6 Flag group và group kill switch 100
2.5.7 Audit log và lịch sử cấu hình 104
2.5.8 Ràng buộc toàn vẹn dữ liệu 112
2.6 Thiết kế Rule Evaluation Engine 119
2.6.1 Evaluation request và context 119
2.6.2 Evaluation snapshot 125
2.6.3 Thứ tự ưu tiên khi đánh giá 131
2.6.4 Đánh giá trạng thái toàn cục 139
2.6.5 Targeting theo người dùng 144
2.6.6 Targeting theo vai trò 150
2.6.7 Rollout theo phần trăm người dùng 155
2.6.8 Thực thi rule theo độ ưu tiên 162
2.6.9 Variant và kết quả đánh giá 170
2.6.10 Reason code 176
2.6.11 Default value và error fallback 182
2.6.12 Khả năng mở rộng Rule Evaluation Engine 189
2.7 Thiết kế API 200
2.7.1 Nguyên tắc thiết kế API 200

2.7.2 API quản lý project và environment . . . 205
2.7.3 API quản lý Feature Flag . . . 211
2.7.4 API quản lý rule . . . 220
2.7.5 Evaluation API . . . 227
2.7.6 API audit log . . . 234
2.7.7 API thống kê evaluation . . . 239
2.7.8 Validation dữ liệu đầu vào . . . 244
2.7.9 Mô hình phản hồi lỗi . . . 249
2.7.10 Tài liệu hóa API . . . 254
2.8 Hiệu năng, nhất quán và bảo mật . . . 260
2.8.1 Caching kết quả evaluation . . . 260
2.8.2 Thiết kế cache key và thời gian tồn tại . . . 262
2.8.3 Cache invalidation . . . 264
2.8.4 Tính nhất quán của cấu hình . . . 265
2.8.5 Trust boundary của evaluation context . . . 267
2.8.6 Bảo vệ Management API . . . 269
2.8.7 Phân quyền quản trị . . . 271
2.8.8 Auditability và truy vết thay đổi . . . 273
2.9 Phương pháp kiểm thử và đánh giá . . . 274
2.9.1 Unit test cho Rule Evaluation Engine . . . 274
2.9.2 Integration test cho API . . . 277
2.9.3 Kiểm thử validation và xử lý lỗi . . . 279
2.9.4 Kiểm thử stable percentage rollout . . . 282
2.9.5 Kiểm thử cache và invalidation . . . 284
2.9.6 Đánh giá Dashboard . . . 286
2.9.7 Thiết kế kịch bản demo . . . 288

3 Kết quả thực hiện và đánh giá 291

3.1 Môi trường và cấu trúc triển khai . . . 291
3.1.1 Cấu trúc mã nguồn . . . 291
3.1.2 Môi trường phát triển . . . 293
3.1.3 Database migration và seed data . . . 294

3.1.4 Triển khai bằng Docker Compose 296
3.1.5 Quy trình khởi chạy hệ thống 297
3.2 Kết quả xây dựng Backend API 299
3.2.1 Quản lý project và environment 299
3.2.2 Quản lý Feature Flag 301
3.2.3 Quản lý rule và flag group 302
3.2.4 Evaluation API 304
3.2.5 Audit log và lịch sử cấu hình 307
3.2.6 Validation và xử lý lỗi 309
3.2.7 Phân quyền người dùng quản trị 310
3.2.8 Cache và evaluation metrics 312
3.3 Kết quả xây dựng Frontend Dashboard 314
3.3.1 Màn hình quản lý project 314
3.3.2 Màn hình quản lý Feature Flag 315
3.3.3 Màn hình tạo và chỉnh sửa flag 317
3.3.4 Màn hình cấu hình rule 318
3.3.5 Màn hình quản lý flag group 320
3.3.6 Màn hình audit log 321
3.3.7 Màn hình thống kê evaluation 323
3.3.8 Hiển thị quyền và trạng thái 324
3.4 Kết quả xây dựng Demo Application và SDK 325
3.4.1 Cấu trúc Demo Application 325
3.4.2 JavaScript SDK 327
3.4.3 Tích hợp SDK với Demo Application 329
3.4.4 User context trong Demo Application 330
3.4.5 Hành vi fallback 332
3.5 Các kịch bản minh họa 333
3.5.1 Bật và tắt tính năng toàn cục 333
3.5.2 Bật tính năng cho người dùng cụ thể 335
3.5.3 Bật tính năng theo vai trò 336
3.5.4 Rollout theo phần trăm người dùng 338
3.5.5 Cấu hình theo environment 339

3.5.6 Thứ tự ưu tiên của rule . . . 341
3.5.7 Flag kill switch . . . 342
3.5.8 Group kill switch . . . 343
3.5.9 Audit log và versioning . . . 345
3.5.10 Evaluation metrics . . . 346
3.6 Kết quả kiểm thử . . . 348
3.6.1 Kết quả unit test Rule Evaluation Engine . . . 348
3.6.2 Kết quả kiểm thử API . . . 349
3.6.3 Kết quả kiểm thử validation và error handling . . . 351
3.6.4 Kết quả kiểm thử stable rollout . . . 352
3.6.5 Kết quả kiểm thử cache . . . 353
3.6.6 Kết quả kiểm thử phân quyền . . . 355
3.6.7 Kết quả kiểm thử triển khai . . . 356
3.7 Đánh giá kết quả thực hiện . . . 358
3.7.1 Mức độ đáp ứng yêu cầu MVP . . . 358
3.7.2 Mức độ đáp ứng yêu cầu khuyến khích . . . 360
3.7.3 Đánh giá Rule Evaluation Engine . . . 361
3.7.4 Đánh giá Backend API . . . 363
3.7.5 Đánh giá Dashboard . . . 364
3.7.6 Đánh giá Demo Application . . . 366
3.7.7 So sánh với các nền tảng hiện có . . . 367
3.7.8 Giá trị và tính mới của sản phẩm . . . 369
3.7.9 Hạn chế của hệ thống . . . 370

4 Kết luận **373**

4.1 Tổng kết kết quả đạt được . . . 373
4.1.1 Kết quả nghiên cứu . . . 373
4.1.2 Kết quả xây dựng hệ thống . . . 374
4.1.3 Kết quả kiểm thử và demo . . . 376
4.2 Các đóng góp chính . . . 377
4.2.1 Đóng góp về nghiên cứu . . . 377
4.2.2 Đóng góp về thiết kế hệ thống . . . 378

4.2.3 Đóng góp về triển khai sản phẩm . . . 380
4.3 Bài học kinh nghiệm . . . 381
4.3.1 Bài học về phân tích bài toán . . . 381
4.3.2 Bài học về thiết kế hệ thống . . . 383
4.3.3 Bài học về triển khai và kiểm thử . . . 384
4.4 Hạn chế . . . 386
4.4.1 Hạn chế về kiến trúc . . . 386
4.4.2 Hạn chế về chức năng . . . 387
4.4.3 Hạn chế về bảo mật và vận hành . . . 388
4.5 Hướng phát triển . . . 390
4.6 Mở rộng Rule Evaluation Engine . . . 390
4.6.1 Targeting theo thuộc tính . . . 390
4.6.2 Targeting theo organization . . . 391
4.6.3 Targeting theo thời gian và vị trí . . . 391
4.6.4 Hỗ trợ nhiều variant và A/B testing . . . 391
4.7 Mở rộng khả năng tích hợp . . . 392
4.7.1 SDK cho nhiều ngôn ngữ . . . 392
4.7.2 Webhook khi cấu hình thay đổi . . . 392
4.7.3 Đồng bộ cấu hình theo thời gian thực . . . 392
4.7.4 Tương thích OpenFeature . . . 393
4.8 Mở rộng quản trị vòng đời flag . . . 393
4.8.1 Approval workflow . . . 393
4.8.2 Lập lịch bật và tắt flag . . . 394
4.8.3 Phát hiện stale flag . . . 394
4.8.4 Nhắc nhở và tự động dọn dẹp flag . . . 394
4.9 Mở rộng khả năng vận hành . . . 395
4.9.1 High availability . . . 395
4.9.2 Distributed cache . . . 395
4.9.3 Monitoring và alerting . . . 395
4.9.4 Triển khai trên Kubernetes . . . 396

Tài liệu tham khảo **397**

Danh sách hình vẽ

2.1 Sơ đồ thực thể liên kết của hệ thống Feature Flag 78

Danh sách bảng

2.1 So sánh deploy và release 9
2.2 Vòng đời của các loại Feature Flag phổ biến 17
2.3 So sánh mô hình triển khai của LaunchDarkly và Unleash 36
2.4 So sánh khả năng targeting và rollout 38
2.5 So sánh SDK và khả năng tích hợp 39
2.6 So sánh audit log và phân quyền 40
2.7 Tổng hợp so sánh LaunchDarkly và Unleash 41
2.8 Các chức năng được kế thừa trong hệ thống đề xuất 44
2.9 Các chức năng được đơn giản hóa trong MiniProject 45
2.10 Quan hệ giữa các thực thể chính 76
2.11 Các thuộc tính của thực thể Project 79
2.12 Các thuộc tính của thực thể Environment 80
2.13 Các thuộc tính của thực thể FeatureFlag 84
2.14 Ý nghĩa trạng thái vòng đời của Feature Flag 85
2.15 Các thuộc tính của thực thể FlagConfig 86
2.16 Tổ hợp trạng thái cấu hình và serving mode 87
2.17 Ví dụ cấu hình một Feature Flag theo environment 88
2.18 Các thuộc tính của thực thể FlagRule 90
2.19 Các loại rule được hỗ trợ 90
2.20 Validation tham số theo loại rule 94
2.21 Các thuộc tính của thực thể SampleUser 95
2.22 Các thuộc tính của thực thể FlagGroup 100
2.23 Các thuộc tính của thực thể AuditLogEntry 105
2.24 Vai trò của audit log và versioning 112

2.25 Các ràng buộc duy nhất trong mô hình dữ liệu 113
2.26 Tương tác giữa trạng thái và serving mode 143
2.27 Quan hệ giữa mã lý do và matched rule 175
2.28 Reason code cho trạng thái hệ thống 178

Danh sách đoạn mã

2.1 Điều khiển hành vi ứng dụng bằng Feature Flag 7
2.2 Tách deploy khỏi release bằng Feature Flag 10
2.3 Sử dụng Feature Flag để phản ứng nhanh khi xảy ra sự cố 13
2.4 Ví dụ ngữ cảnh đánh giá theo vai trò 20
2.5 Khôi phục hành vi dự phòng bằng Feature Flag 24
2.6 Chỉ mục bảo đảm một environment mặc định cho mỗi project 81
2.7 Ràng buộc khóa environment duy nhất trong project 82
2.8 Ràng buộc khóa Feature Flag duy nhất trong project 84
2.9 Ràng buộc một cấu hình cho mỗi flag trong một environment 86
2.10 Ràng buộc priority duy nhất trong một cấu hình flag 91
2.11 Tham số của rule USER_ALLOWLIST 92
2.12 Tham số của rule ROLE_TARGETING 92
2.13 Tham số của rule PERCENTAGE_ROLLOUT 92
2.14 Ràng buộc targeting key duy nhất trong project 95
2.15 Evaluation context chứa user ID và organization ID 96
2.16 Danh sách role trong user context mẫu 97
2.17 Thuộc tính mở rộng của user context mẫu 98
2.18 Ràng buộc khóa group duy nhất trong project 100
2.19 Thông tin actor và request trong audit log 107
2.20 Snapshot trước và sau khi cập nhật rule 108
2.21 Hàm ngăn cập nhật và xóa audit log 109
2.22 Trigger bảo đảm audit log có tính chất append-only 110
2.23 Tăng phiên bản cấu hình sau khi thay đổi 111
2.24 Ràng buộc khóa Feature Flag duy nhất trong project 113

2.25 Biểu thức kiểm tra định dạng khóa nghiệp vụ 114
2.26 Ràng buộc một environment mặc định trong mỗi project 115
2.27 Tham số percentage rollout hợp lệ 116
2.28 Các tham số percentage rollout không hợp lệ 116
2.29 Ràng buộc giá trị percentage trong tham số JSONB 116
2.30 Hàm ngăn thay đổi bản ghi audit log 117
2.31 Trigger bảo vệ bảng audit log 117
2.32 Cấu trúc evaluation request 119
2.33 Cấu trúc khái quát của evaluation snapshot 126
2.34 Truy xuất rule theo thứ tự ưu tiên 129
2.35 Thứ tự ưu tiên của Rule Evaluation Engine 131
2.36 Kết quả khi feature flag đã được archive 133
2.37 Kết quả khi cấu hình flag bị disabled 133
2.38 Kết quả khi group kill switch được kích hoạt 134
2.39 Kết quả khi flag kill switch được kích hoạt 134
2.40 Kết quả khi feature flag được bật toàn cục 135
2.41 Ví dụ tham số của user allowlist rule 136
2.42 Kết quả khi user allowlist khớp 136
2.43 Kết quả khi role targeting khớp 137
2.44 Kết quả khi percentage rollout khớp 138
2.45 Kết quả mặc định khi không có rule nào khớp 138
2.46 Cấu trúc trạng thái toàn cục của flag config 139
2.47 Kết quả khi flag config ở trạng thái disabled 139
2.48 Kết quả khi serving mode là global on 140
2.49 Xử lý serving mode targeted 141
2.50 Cấu trúc user allowlist rule 144
2.51 Kết quả khi user allowlist rule khớp 145
2.52 So khớp targeting key với user allowlist 146
2.53 So khớp user ID với user allowlist 147
2.54 Xử lý user allowlist khi thiếu định danh 148
2.55 Kết quả khi thiếu định danh và không có rule nào khớp 149
2.56 Role trong evaluation context 150

2.57 Cấu trúc role targeting rule . . . 150
2.58 So khớp role theo nguyên tắc OR . . . 152
2.59 Kết quả khi role targeting khớp . . . 152
2.60 Quan hệ ưu tiên giữa user targeting và role targeting . . . 153
2.61 Xử lý khi evaluation context không chứa role . . . 154
2.62 Evaluation context không chứa role . . . 155
2.63 Cấu trúc percentage rollout rule . . . 156
2.64 Tạo giá trị băm ổn định bằng SHA-256 . . . 157
2.65 Chuỗi đầu vào của stable hash . . . 158
2.66 Chuyển đổi SHA-256 digest thành bucket phần trăm . . . 159
2.67 Danh sách rule trước khi sắp xếp . . . 163
2.68 Danh sách rule sau khi sắp xếp . . . 163
2.69 Sắp xếp rule trước khi thực thi . . . 164
2.70 Dừng tại rule đầu tiên khớp . . . 165
2.71 Hai rule có cùng độ ưu tiên . . . 166
2.72 Sắp xếp ổn định khi rule có cùng priority . . . 167
2.73 Bỏ qua rule khi không đủ context . . . 168
2.74 Xử lý rule không hợp lệ trong quá trình thực thi . . . 169
2.75 Variant bật khi user allowlist khớp . . . 171
2.76 Variant tắt khi flag bị disabled . . . 171
2.77 Variant mặc định khi không có rule nào khớp . . . 172
2.78 Variant mặc định khi evaluation xảy ra lỗi . . . 172
2.79 Cấu trúc EvaluationResult . . . 173
2.80 Định nghĩa kiểu EvaluationResult . . . 173
2.81 EvaluationResult với metadata nội bộ . . . 176
2.82 Tập reason code của Evaluation API . . . 177
2.83 Kết quả evaluation với targeting reason code . . . 178
2.84 Kết quả default off . . . 179
2.85 Kết quả khi không tìm thấy feature flag . . . 180
2.86 Kết quả fallback khi evaluation xảy ra lỗi . . . 180
2.87 Hai kết quả tắt với nguyên nhân khác nhau . . . 181
2.88 Kiểm thử reason code của Rule Evaluation Engine . . . 182

2.89 Kết quả fallback mặc định . . . 182
2.90 Fallback khi không tìm thấy feature flag . . . 184
2.91 Xử lý rule khi context không đầy đủ . . . 185
2.92 Kết quả khi context không đầy đủ và không có rule khớp . . . 185
2.93 Fallback giữa cache và database . . . 186
2.94 Fallback khi không thể truy xuất snapshot . . . 187
2.95 Xử lý lỗi nội bộ của Evaluation Engine . . . 188
2.96 Cấu trúc rule dựa trên thuộc tính . . . 190
2.97 Đánh giá rule dựa trên thuộc tính . . . 191
2.98 Organization ID trong evaluation context . . . 192
2.99 Cấu trúc organization targeting rule . . . 192
2.100 So khớp organization ID . . . 192
2.101 Cấu trúc rule theo thời gian . . . 193
2.102 Đánh giá rule theo khoảng thời gian . . . 194
2.103 Cấu trúc rule theo vị trí . . . 194
2.104 Cấu hình feature flag nhiều variant . . . 195
2.105 EvaluationResult cho flag nhiều variant . . . 196
2.106 Interface của rule evaluator . . . 197
2.107 Các strategy đánh giá rule . . . 197
2.108 Registry của các rule evaluator . . . 198
2.109 Thực thi rule thông qua evaluator strategy . . . 199

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Trong quá trình phát triển phần mềm, việc đưa một tính năng mới vào hệ thống luôn đi kèm với rủi ro. Mặc dù tính năng đã được kiểm thử trước khi triển khai, các vấn đề về logic nghiệp vụ, hiệu năng, khả năng tương thích hoặc hành vi của người dùng vẫn có thể chỉ xuất hiện trên môi trường thực tế. Nếu tính năng được phát hành đồng thời cho toàn bộ người dùng, phạm vi ảnh hưởng của sự cố có thể lớn, trong khi việc khắc phục thường đòi hỏi rollback phiên bản hoặc thực hiện một lần triển khai mới.

Bên cạnh đó, nhu cầu phát hành phần mềm hiện nay không còn giới hạn ở hai trạng thái bật hoặc tắt cho toàn hệ thống. Đội phát triển có thể cần cung cấp tính năng cho một số người dùng cụ thể, một nhóm có vai trò nhất định hoặc một tỷ lệ người dùng được lựa chọn ổn định. Do đó, bài toán đặt ra là xây dựng một cơ chế cho phép kiểm soát việc phát hành tính năng độc lập với quá trình triển khai mã nguồn, đồng thời bảo đảm quyết định bật hoặc tắt tính năng được thực hiện nhất quán, có thể truy vết và dễ dàng quản lý.

Feature Flag là một giải pháp phù hợp cho bài toán này. Tuy nhiên, khi số lượng flag và quy tắc tăng lên, việc sử dụng các biến cấu hình rời rạc hoặc cài đặt logic trực tiếp trong từng ứng dụng có thể dẫn đến cấu hình không nhất quán, khó kiểm soát lịch sử thay đổi và khó mở rộng. Vì vậy, đề tài lựa chọn nghiên cứu và xây dựng một nền tảng quản lý Feature Flag tập trung cho ứng dụng web.

1.2 Mục tiêu của báo cáo

Báo cáo hướng đến hai nhóm mục tiêu chính: mục tiêu nghiên cứu và mục tiêu triển khai.

Về nghiên cứu, báo cáo làm rõ bản chất của Feature Flag, sự khác biệt giữa deploy và release, các loại flag phổ biến và vai trò của Feature Flag trong quản lý phát hành phần mềm. Báo cáo đồng thời phân tích các chiến lược đánh giá như bật toàn cục, nhắm mục tiêu theo người dùng, theo vai trò và rollout theo tỷ lệ phần trăm.

Về triển khai, đề tài xây dựng một hệ thống có khả năng quản lý project, feature flag và các rule tương ứng; tiếp nhận evaluation context từ ứng dụng tích hợp; đánh giá trạng thái flag theo một thứ tự ưu tiên xác định; trả về kết quả có lý do rõ ràng; và ghi nhận các thay đổi cấu hình thông qua audit log. Hệ thống cũng cung cấp dashboard quản trị và demo app để chứng minh khả năng ứng dụng trong thực tế. Các thành phần này phù hợp với yêu cầu về backend API, frontend dashboard, evaluation engine, audit log và ứng dụng minh họa của mini project.

Ngoài việc bảo đảm hệ thống có thể hoạt động, báo cáo còn tập trung giải thích các quyết định thiết kế và công nghệ, bao gồm cách tổ chức kiến trúc, mô hình dữ liệu, thứ tự đánh giá rule, cơ chế rollout ổn định, xử lý lỗi và khả năng mở rộng. Đây cũng là cơ sở để thể hiện khả năng giải quyết vấn đề, tư duy thiết kế và mức độ hiểu những nội dung đã triển khai.

1.3 Câu hỏi nghiên cứu

Báo cáo tập trung giải quyết các câu hỏi chính sau:

1. Feature Flag giải quyết những hạn chế nào của quy trình triển khai và phát hành phần mềm truyền thống?
2. Làm thế nào để tách quá trình deploy mã nguồn khỏi quyết định release tính năng cho người dùng?
3. Một nền tảng Feature Flag cần những thành phần nào để hỗ trợ quản lý cấu hình, đánh giá rule, tích hợp với ứng dụng và truy vết thay đổi?
4. Làm thế nào để xác định thứ tự ưu tiên giữa các trạng thái và rule nhằm bảo đảm kết quả đánh giá rõ ràng, nhất quán và có thể dự đoán?
5. Làm thế nào để rollout tính năng theo tỷ lệ phần trăm mà vẫn giữ kết quả ổn định đối

với cùng một người dùng?

6. Hệ thống nên xử lý như thế nào khi feature flag không tồn tại, dữ liệu context bị thiếu hoặc xảy ra lỗi trong quá trình đánh giá?
7. Kiến trúc và công nghệ nào phù hợp với quy mô của mini project, đồng thời vẫn tạo điều kiện để hệ thống được mở rộng trong tương lai?

1.4 Phạm vi triển khai

Trong phạm vi báo cáo, đề tài triển khai một nền tảng Feature Flag phục vụ các ứng dụng web, gồm bốn thành phần chính:

- Backend API chịu trách nhiệm quản lý dữ liệu, xử lý nghiệp vụ và cung cấp evaluation endpoint.
- Frontend dashboard hỗ trợ quản trị project, flag, cấu hình rule và audit log.
- Evaluation engine thực hiện việc đánh giá flag dựa trên trạng thái cấu hình và context của request.
- Demo app tích hợp evaluation API để minh họa việc hiển thị hoặc ẩn tính năng theo kết quả đánh giá.

Các chiến lược được hỗ trợ bao gồm bật hoặc tắt toàn cục, nhắm mục tiêu theo user ID, theo role và rollout theo phần trăm người dùng. Hệ thống lưu trữ project, môi trường, feature flag, cấu hình, rule, dữ liệu người dùng mẫu và audit log trong cơ sở dữ liệu. Ngoài ra, hệ thống thực hiện validation dữ liệu đầu vào, xử lý lỗi cơ bản, cung cấp dữ liệu mẫu và tài liệu hướng dẫn cài đặt.

Dề tài tập trung vào việc chứng minh đầy đủ luồng hoạt động của Feature Flag, từ khâu tạo và cấu hình flag đến khi ứng dụng gửi context và nhận kết quả đánh giá. Các vấn đề ở quy mô hệ thống phân tán lớn như đồng bộ cấu hình đa vùng, streaming update theo thời gian thực, độ sẵn sàng đa trung tâm dữ liệu, phân tích thử nghiệm thống kê chuyên sâu và quản lý hàng triệu request mỗi giây không thuộc phạm vi triển khai chính.

Các nội dung nâng cao như caching, SDK JavaScript, phân quyền, versioning cấu hình, thống kê lượt đánh giá, kill switch theo nhóm và Docker Compose được xem là phần mở rộng nhằm

đánh giá khả năng phát triển tiếp của hệ thống, thay vì điều kiện bắt buộc để hoàn thành luồng chức năng cốt lõi.

Chương 2

Nội dung

2.1 Tổng quan về Feature Flag

2.1.1 Khái niệm Feature Flag

Định nghĩa Feature Flag

Feature Flag, còn được gọi là Feature Toggle hoặc Feature Switch, là một kỹ thuật phát triển phần mềm cho phép hệ thống quyết định tại thời điểm chạy liệu một tính năng hoặc một nhánh xử lý có được kích hoạt hay không. Quyết định này được thực hiện dựa trên cấu hình bên ngoài mã nguồn và ngữ cảnh của đối tượng đang sử dụng hệ thống, thay vì được cố định trong quá trình biên dịch hoặc triển khai ứng dụng [1, 2].

Có thể xem Feature Flag như một công tắc logic nằm giữa mã nguồn đã được triển khai và người dùng cuối. Mã của tính năng mới có thể đã tồn tại trong môi trường chạy, nhưng tính năng chỉ được cung cấp cho người dùng khi flag tương ứng được đánh giá là bật. Nhờ đó, đội phát triển có thể thay đổi trạng thái phát hành của tính năng mà không cần chỉnh sửa mã nguồn hoặc triển khai lại toàn bộ ứng dụng.

Đặc điểm cốt lõi của Feature Flag là tách biệt hai hoạt động thường bị xem là đồng nhất: *deploy* và *release*. Deploy là quá trình đưa mã nguồn lên môi trường chạy, trong khi release là quá trình cho phép một nhóm người dùng thực sự truy cập và sử dụng tính năng. Sự tách biệt này giúp đội phát triển triển khai mã thường xuyên hơn nhưng vẫn kiểm soát được thời điểm, phạm vi và điều kiện phát hành tính năng [3].

Các thành phần cơ bản của Feature Flag

Một Feature Flag thường bao gồm các thành phần cơ bản sau:

- **Khóa định danh của flag:** là giá trị duy nhất dùng để tham chiếu đến flag trong mã nguồn và trong các yêu cầu đánh giá, chẳng hạn như `new-checkout-page` hoặc `recommendation-engine`.
- **Trạng thái và cấu hình mặc định:** xác định flag đang được bật, tắt hoặc ngừng sử dụng. Giá trị mặc định được sử dụng khi không có quy tắc nào phù hợp hoặc khi hệ thống không thể đánh giá đầy đủ ngữ cảnh.
- **Quy tắc nhắm mục tiêu:** mô tả các điều kiện để xác định đối tượng được sử dụng tính năng. Quy tắc có thể dựa trên định danh người dùng, vai trò, tổ chức, thuộc tính ngữ cảnh hoặc các tiêu chí nghiệp vụ khác [4].
- **Chiến lược phân phối:** xác định cách tính năng được phát hành, chẳng hạn như bật toàn cục, bật cho một tập người dùng cụ thể hoặc rollout theo tỷ lệ phần trăm [5].
- **Ngữ cảnh đánh giá:** là tập hợp thông tin mô tả đối tượng đang yêu cầu sử dụng tính năng, ví dụ như `targetingKey`, mã người dùng, vai trò và các thuộc tính bổ sung. Ngữ cảnh này được truyền vào bộ máy đánh giá để đối chiếu với các quy tắc đã cấu hình [6].
- **Giá trị trả về:** là kết quả cuối cùng sau quá trình đánh giá, thường bao gồm trạng thái bật hoặc tắt, giá trị biến thể và lý do dẫn đến quyết định.

Trong một nền tảng quản lý Feature Flag, các thành phần trên thường được tổ chức theo project và environment. Cùng một flag có thể tồn tại trên nhiều môi trường như development, staging và production, nhưng có cấu hình và quy tắc khác nhau. Cách tổ chức này cho phép đội phát triển kiểm thử tính năng ở môi trường nội bộ trước khi mở rộng phạm vi phát hành trên môi trường production [7].

Cơ chế quyết định hành vi tại thời điểm chạy

Feature Flag điều khiển hành vi của phần mềm thông qua quá trình đánh giá tại thời điểm chạy. Khi ứng dụng cần thực hiện một chức năng được kiểm soát bởi flag, ứng dụng gửi khóa của flag cùng với ngữ cảnh người dùng đến bộ máy đánh giá. Bộ máy này tải cấu hình hiện tại của flag, kiểm tra trạng thái tổng quát và lần lượt đối chiếu ngữ cảnh với các quy tắc đã được

thiết lập.

Quá trình đánh giá có thể được biểu diễn khái quát như sau:

1. Xác định flag theo project, environment và khóa định danh.
2. Kiểm tra flag có tồn tại và có đang ở trạng thái cho phép phục vụ hay không.
3. Kiểm tra các cơ chế vô hiệu hóa ưu tiên cao như trạng thái tắt hoặc kill switch.
4. Nếu flag được bật toàn cục, trả về kết quả bật mà không cần đánh giá thêm.
5. Nếu flag sử dụng chế độ nhắm mục tiêu, lần lượt đánh giá các quy tắc theo thứ tự ưu tiên.
6. Nếu một quy tắc phù hợp, trả về giá trị được cấu hình cho quy tắc đó.
7. Nếu không có quy tắc nào phù hợp, trả về giá trị mặc định của flag.

Ví dụ, một tính năng có thể được cấu hình bật cho người dùng có vai trò `beta-customer`. Khi người dùng gửi yêu cầu, ứng dụng cung cấp vai trò này trong ngữ cảnh đánh giá. Nếu quy tắc vai trò được thỏa mãn, hệ thống trả về trạng thái bật và ứng dụng thực thi nhánh xử lý mới. Ngược lại, ứng dụng tiếp tục sử dụng hành vi cũ.

Đoạn mã 2.1: Điều khiển hành vi ứng dụng bằng Feature Flag

```
const result = await featureFlagClient.evaluate(
    "new-checkout-page",
    userContext
);

if (result.enabled) {
    return showNewCheckout();
}

return showCurrentCheckout();
```

Đối với rollout theo phần trăm, hệ thống thường sử dụng hàm băm ổn định trên khóa định danh của đối tượng và khóa của flag để gán đối tượng vào một nhóm cố định. Cơ chế này giúp cùng một người dùng nhận được kết quả nhất quán giữa nhiều lần truy cập, đồng thời cho phép

đội phát triển tăng hoặc giảm tỷ lệ rollout mà không cần lưu trạng thái riêng cho từng người dùng [5].

Như vậy, Feature Flag không thay đổi mã nguồn tại thời điểm chạy mà thay đổi nhánh xử lý được lựa chọn trong mã nguồn đã triển khai. Hiệu quả của kỹ thuật này phụ thuộc vào việc thiết kế quy tắc rõ ràng, xác định thứ tự ưu tiên nhất quán và cung cấp giá trị mặc định an toàn khi quá trình đánh giá xảy ra lỗi.

2.1.2 Deploy và Release

Khái niệm Deploy

Deploy là quá trình đưa một phiên bản phần mềm từ môi trường phát triển hoặc kiểm thử lên một môi trường chạy cụ thể, chẳng hạn như staging hoặc production. Quá trình này có thể bao gồm biên dịch mã nguồn, đóng gói ứng dụng, cập nhật cơ sở dữ liệu, cấu hình hạ tầng, khởi động dịch vụ và kiểm tra trạng thái hoạt động sau triển khai.

Mục tiêu của deploy là làm cho mã nguồn mới có mặt và có khả năng thực thi trên môi trường đích. Tuy nhiên, việc mã nguồn đã được deploy không đồng nghĩa với việc tất cả chức năng chứa trong phiên bản đó phải được cung cấp ngay cho người dùng cuối. Một tính năng có thể đã tồn tại trong hệ thống production nhưng vẫn được ẩn, vô hiệu hóa hoặc chỉ cho phép một nhóm người dùng nhất định truy cập.

Trong quy trình phát triển hiện đại, deploy thường được tự động hóa thông qua các pipeline tích hợp và phân phối liên tục. Việc triển khai thường xuyên với các thay đổi nhỏ giúp giảm độ phức tạp của mỗi lần cập nhật, rút ngắn vòng phản hồi và hạn chế rủi ro so với việc tích lũy nhiều thay đổi trong một lần triển khai lớn [3].

Khái niệm Release

Release là quá trình đưa một tính năng hoặc một thay đổi trong phần mềm đến người sử dụng thực tế. Hoạt động này xác định thời điểm, phạm vi và đối tượng được phép truy cập chức năng, thay vì chỉ xác định mã nguồn đang được cài đặt ở môi trường nào.

Một release có thể được thực hiện cho toàn bộ người dùng hoặc giới hạn theo một nhóm cụ thể, chẳng hạn như người dùng nội bộ, khách hàng thử nghiệm, người dùng thuộc một vai trò nhất định hoặc một tỷ lệ phần trăm người dùng. Vì vậy, release mang tính chất nghiệp vụ và

vận hành nhiều hơn deploy. Nó liên quan trực tiếp đến trải nghiệm người dùng, mức độ rủi ro được chấp nhận và chiến lược cung cấp tính năng của tổ chức.

Release cũng không nhất thiết yêu cầu một lần deploy mới. Khi mã nguồn của tính năng đã tồn tại trên production và được kiểm soát bằng Feature Flag, đội phát triển có thể release tính năng bằng cách thay đổi cấu hình của flag. Tương tự, tính năng có thể được tạm dừng hoặc thu hẹp phạm vi cung cấp mà không cần loại bỏ mã nguồn khỏi môi trường chạy.

Sự khác biệt giữa Deploy và Release

Deploy và release thường diễn ra gần nhau trong các quy trình phát triển truyền thống nên dễ bị xem là cùng một hoạt động. Tuy nhiên, hai khái niệm này giải quyết hai vấn đề khác nhau. Deploy trả lời câu hỏi mã nguồn đã được đưa lên môi trường chạy hay chưa, trong khi release trả lời câu hỏi người dùng đã được phép sử dụng tính năng hay chưa.

Sự khác biệt giữa deploy và release được thể hiện trong Bảng 2.1.

Bảng 2.1: So sánh deploy và release

Tiêu chí	**Deploy**	**Release**
Mục tiêu	Đưa mã nguồn và thành phần phần mềm lên môi trường chạy	Cung cấp tính năng cho người dùng thực tế
Đối tượng tác động	Hạ tầng, dịch vụ, cơ sở dữ liệu và phiên bản ứng dụng	Người dùng, nhóm người dùng hoặc phân khúc khách hàng
Cách thực hiện	Pipeline CI/CD, script triển khai hoặc công cụ quản lý hạ tầng	Thay đổi cấu hình, bật flag hoặc điều chỉnh quy tắc nhắm mục tiêu
Phạm vi	Thường áp dụng cho toàn bộ phiên bản ứng dụng trên môi trường đích	Có thể áp dụng cho toàn bộ hoặc một phần người dùng
Khả năng đảo ngược	Có thể yêu cầu rollback phiên bản hoặc deploy lại	Có thể tắt tính năng bằng thay đổi cấu hình
Thời điểm	Khi phiên bản phần mềm được đưa lên môi trường chạy	Khi tổ chức quyết định cho phép người dùng truy cập tính năng

Nếu deploy và release bị ràng buộc chặt với nhau, mọi lần đưa mã mới lên production đều đồng thời làm thay đổi hành vi của toàn bộ hệ thống. Điều này làm tăng phạm vi ảnh hưởng khi xảy ra lỗi và khiến đội phát triển phải phụ thuộc vào các khung thời gian phát hành cố định. Ngược lại, khi hai hoạt động được tách biệt, việc triển khai kỹ thuật có thể diễn ra thường xuyên, còn quyết định phát hành được thực hiện độc lập dựa trên mức độ sẵn sàng và rủi ro nghiệp vụ.

Tách Deploy khỏi Release bằng Feature Flag

Feature Flag cung cấp một lớp điều khiển nằm giữa mã nguồn đã được deploy và hành vi được cung cấp cho người dùng. Mã của tính năng mới được triển khai cùng ứng dụng nhưng được đặt phía sau một flag. Khi flag ở trạng thái tắt, ứng dụng tiếp tục thực hiện hành vi hiện tại. Khi flag được bật, ứng dụng chuyển sang nhánh xử lý mới mà không cần deploy lại [1].

Quy trình tách deploy khỏi release có thể được mô tả qua các bước sau:

1. Đội phát triển triển khai mã nguồn chứa cả hành vi hiện tại và hành vi mới.
2. Tính năng mới được bảo vệ bởi một Feature Flag và mặc định chưa được cung cấp rộng rãi.
3. Tính năng được kiểm tra trên production với người dùng nội bộ hoặc một nhóm giới hạn.
4. Phạm vi release được mở rộng theo user, role hoặc tỷ lệ phần trăm người dùng.
5. Nếu phát hiện sự cố, flag được tắt để khôi phục hành vi cũ mà không cần rollback toàn bộ phiên bản.
6. Khi tính năng đã ổn định và được phát hành hoàn toàn, flag cùng nhánh mã cũ có thể được loại bỏ.

Ví dụ, mã nguồn của một giao diện thanh toán mới có thể được deploy lên production trong khi người dùng vẫn nhìn thấy giao diện hiện tại:

Đoạn mã 2.2: Tách deploy khỏi release bằng Feature Flag

```
if (featureFlags.isEnabled("new-checkout", userContext)) {
    return renderNewCheckout();
}

return renderCurrentCheckout();
```

Sau khi hoàn tất kiểm thử, đội phát triển có thể bật flag cho nhân viên nội bộ, tiếp tục mở rộng cho khách hàng thử nghiệm và sau đó tăng dần tỷ lệ rollout. Cách tiếp cận này cho phép kiểm soát phạm vi ảnh hưởng của thay đổi, thu thập phản hồi sớm và giảm rủi ro khi phát hành [7, 8].

Việc tách deploy khỏi release cũng hỗ trợ mô hình phát triển liên tục. Các thay đổi chưa hoàn

thiện có thể được tích hợp vào nhánh mã chính nhưng chưa được hiển thị cho người dùng. Nhờ đó, đội phát triển giảm nhu cầu duy trì các nhánh tính năng tồn tại trong thời gian dài, hạn chế xung đột khi hợp nhất mã và có thể triển khai các thay đổi nhỏ thường xuyên hơn [3].

Tuy nhiên, Feature Flag không loại bỏ hoàn toàn nhu cầu rollback. Nếu lỗi nằm trong phần mã dùng chung, quá trình khởi động ứng dụng, thay đổi cơ sở dữ liệu hoặc hạ tầng, việc tắt flag có thể không đủ để khôi phục hệ thống. Do đó, Feature Flag nên được xem là cơ chế kiểm soát release và giảm phạm vi ảnh hưởng, không phải là phương án thay thế cho toàn bộ chiến lược rollback và khôi phục sự cố.

2.1.3 Vai trò của Feature Flag trong vòng đời phần mềm

Hỗ trợ phát triển và tích hợp liên tục

Feature Flag hỗ trợ quá trình phát triển và tích hợp liên tục bằng cách cho phép mã nguồn của tính năng chưa hoàn thiện được tích hợp sớm vào nhánh mã chính nhưng chưa được cung cấp cho người dùng. Thay vì duy trì các nhánh tính năng trong thời gian dài, đội phát triển có thể bảo vệ phần mã mới bằng một flag và giữ flag ở trạng thái tắt cho đến khi tính năng đạt mức độ sẵn sàng cần thiết.

Cách tiếp cận này giúp giảm xung đột khi hợp nhất mã, rút ngắn thời gian tồn tại của các nhánh phát triển và khuyến khích việc tích hợp các thay đổi nhỏ, thường xuyên. Các thành viên trong đội có thể cùng làm việc trên một phiên bản mã nguồn thống nhất, trong khi trạng thái phát hành của từng tính năng vẫn được kiểm soát độc lập [3, 1].

Feature Flag cũng hỗ trợ kiểm thử theo từng giai đoạn trong pipeline phát triển. Một tính năng có thể được bật trên môi trường development, tiếp tục được kiểm tra trên staging và vẫn giữ trạng thái tắt trên production. Cùng một phiên bản mã nguồn có thể được triển khai qua nhiều môi trường nhưng sử dụng cấu hình flag khác nhau, từ đó giảm sự khác biệt giữa các phiên bản phần mềm được kiểm thử và được vận hành thực tế.

Tuy nhiên, việc sử dụng Feature Flag trong phát triển liên tục đòi hỏi các nhánh xử lý cũ và mới đều phải được kiểm thử. Flag cần có mục đích, người chịu trách nhiệm và thời điểm loại bỏ rõ ràng để tránh tích lũy mã không còn cần thiết trong hệ thống [8, 9].

Giảm rủi ro khi phát hành

Trong mô hình phát hành truyền thống, một tính năng mới thường được cung cấp đồng thời cho toàn bộ người dùng ngay sau khi deploy. Nếu tính năng chứa lỗi, phạm vi ảnh hưởng có thể lớn và đội vận hành phải rollback phiên bản hoặc thực hiện một lần triển khai khẩn cấp. Feature Flag giảm rủi ro này bằng cách cho phép kiểm soát phạm vi release độc lập với quá trình deploy.

Tính năng có thể được phát hành trước cho một nhóm nhỏ, chẳng hạn như nhân viên nội bộ, khách hàng thử nghiệm hoặc người dùng thuộc một vai trò cụ thể. Sau khi quan sát kết quả và xác nhận tính ổn định, đội phát triển có thể mở rộng phạm vi phát hành theo từng giai đoạn. Nếu xuất hiện lỗi, phạm vi ảnh hưởng được giới hạn trong nhóm người dùng đã được bật tính năng.

Rollout theo phần trăm là một chiến lược phổ biến để thực hiện phát hành từng bước. Hệ thống có thể bắt đầu với một tỷ lệ nhỏ người dùng, sau đó tăng dần từ 5%, 20%, 50% đến 100%. Việc sử dụng cơ chế phân nhóm ổn định giúp một người dùng tiếp tục nhận cùng một kết quả qua nhiều lần truy cập, hạn chế trải nghiệm thay đổi ngẫu nhiên giữa hành vi cũ và mới [5].

Feature Flag không loại bỏ lỗi khỏi phần mềm, nhưng giúp giảm xác suất và mức độ ảnh hưởng của lỗi trong quá trình phát hành. Nhờ khả năng kiểm soát đối tượng và phạm vi tiếp cận, đội phát triển có thể đưa ra quyết định mở rộng hoặc thu hẹp release dựa trên dữ liệu quan sát được thay vì phát hành toàn bộ trong một lần.

Hỗ trợ thử nghiệm trên môi trường thực tế

Một số vấn đề chỉ xuất hiện trong điều kiện vận hành thực tế, nơi hệ thống phải xử lý dữ liệu, lưu lượng truy cập, cấu hình hạ tầng và hành vi người dùng đa dạng hơn môi trường kiểm thử. Feature Flag cho phép đội phát triển kiểm tra tính năng trên production nhưng giới hạn quyền truy cập cho một nhóm đối tượng được lựa chọn.

Ví dụ, một tính năng mới có thể chỉ được bật cho tài khoản nội bộ hoặc nhóm khách hàng beta. Nhóm này sử dụng cùng hạ tầng và dữ liệu vận hành như người dùng thông thường nhưng được phép truy cập hành vi mới. Qua đó, đội phát triển có thể phát hiện các vấn đề về hiệu năng, khả năng tương thích, trải nghiệm người dùng hoặc logic nghiệp vụ trước khi phát hành rộng rãi.

Feature Flag cũng là cơ sở để thực hiện các thử nghiệm so sánh. Những nhóm người dùng khác nhau có thể được phân phối đến các biến thể khác nhau của một tính năng, sau đó hệ thống thu thập các chỉ số để đánh giá hiệu quả. Khác với việc chỉ bật hoặc tắt tính năng, experiment flag thường cần quản lý biến thể, phân nhóm ổn định và dữ liệu đo lường để bảo đảm kết quả thử nghiệm có ý nghĩa [1].

Việc thử nghiệm trên production cần được kiểm soát cẩn thận. Đội phát triển phải xác định rõ đối tượng tham gia, tiêu chí thành công, thời gian thử nghiệm và phương án dừng. Các dữ liệu nhạy cảm không nên được đưa trực tiếp vào cấu hình flag, đồng thời các thay đổi cấu hình cần được ghi nhận để có thể truy vết khi kết quả thử nghiệm bất thường [8].

Tăng khả năng phản ứng khi xảy ra sự cố

Feature Flag giúp rút ngắn thời gian phản ứng khi một tính năng gây lỗi trên môi trường production. Thay vì phải sửa mã nguồn, xây dựng lại ứng dụng và thực hiện một lần deploy mới, đội vận hành có thể tắt flag để ngừng cung cấp tính năng và đưa người dùng trở lại hành vi ổn định trước đó.

Đối với các tính năng có nguy cơ ảnh hưởng lớn đến hệ thống, flag có thể được sử dụng như một kill switch. Kill switch là cơ chế ưu tiên cao cho phép vô hiệu hóa nhanh một tính năng hoặc một nhóm tính năng khi xuất hiện sự cố về hiệu năng, bảo mật, dữ liệu hoặc phụ thuộc bên ngoài. Cơ chế này đặc biệt hữu ích khi thời gian phục hồi dịch vụ là yếu tố quan trọng.

Ví dụ, nếu một dịch vụ gợi ý sản phẩm mới làm tăng thời gian phản hồi của hệ thống, đội vận hành có thể tắt flag tương ứng để ứng dụng bỏ qua dịch vụ này và sử dụng hành vi dự phòng:

Đoạn mã 2.3: Sử dụng Feature Flag để phản ứng nhanh khi xảy ra sự cố

```
if (featureFlags.isEnabled("product-recommendation")) {
    return getRecommendedProducts(userContext);
}

return getPopularProducts();
```

Khả năng tắt nhanh giúp giảm thời gian gián đoạn và phạm vi ảnh hưởng, nhưng chỉ hiệu quả khi nhánh xử lý dự phòng vẫn hoạt động đúng. Vì vậy, hành vi khi flag tắt cần được thiết kế và kiểm thử như một phần chính thức của hệ thống, không nên được xem là nhánh mã tạm thời.

Mọi thao tác bật, tắt hoặc thay đổi quy tắc trong tình huống sự cố cũng cần được ghi lại trong audit log, bao gồm người thực hiện, thời điểm, đối tượng bị thay đổi và cấu hình trước và sau thao tác. Thông tin này hỗ trợ phân tích nguyên nhân, truy vết quyết định và đánh giá quá trình xử lý sau sự cố.

Feature Flag chủ yếu xử lý sự cố liên quan đến hành vi được đặt phía sau flag. Nếu lỗi nằm trong phần mã khởi động, cơ sở dữ liệu, hạ tầng hoặc thành phần dùng chung, việc tắt flag có thể không đủ để phục hồi hệ thống. Do đó, Feature Flag cần được kết hợp với giám sát, cảnh báo, rollback phiên bản và các cơ chế phục hồi khác trong chiến lược vận hành tổng thể.

2.1.4 Các loại Feature Flag phổ biến

Feature Flag có thể được phân loại theo mục đích sử dụng, thời gian tồn tại và đối tượng chịu trách nhiệm quản lý. Một cách phân loại phổ biến gồm release flag, experiment flag, ops flag và permission flag [1]. Việc xác định đúng loại flag giúp đội phát triển thiết kế quy tắc đánh giá, cơ chế giám sát và kế hoạch loại bỏ phù hợp.

Release Flag

Release flag được sử dụng để kiểm soát việc phát hành một tính năng mới sau khi mã nguồn của tính năng đã được deploy. Loại flag này cho phép đội phát triển tích hợp mã chưa hoàn thiện hoặc chưa sẵn sàng phát hành rộng rãi vào nhánh mã chính mà không làm thay đổi trải nghiệm của người dùng hiện tại.

Release flag thường có hai trạng thái cơ bản: tắt để tiếp tục sử dụng hành vi hiện tại và bật để sử dụng hành vi mới. Tuy nhiên, quá trình phát hành có thể được thực hiện theo nhiều giai đoạn, chẳng hạn như bật trước cho đội phát triển, người dùng nội bộ, khách hàng beta và sau đó mở rộng dần đến toàn bộ người dùng.

Ví dụ, một giao diện thanh toán mới có thể được triển khai lên production nhưng được đặt phía sau flag `new-checkout`. Khi flag chưa được bật, người dùng tiếp tục sử dụng giao diện cũ. Sau khi tính năng được kiểm thử đầy đủ, flag được bật cho toàn bộ người dùng và nhánh xử lý cũ có thể được loại bỏ.

Release flag thường là loại flag có thời gian tồn tại ngắn hoặc trung hạn. Sau khi tính năng đã được phát hành hoàn toàn và ổn định, flag cùng các nhánh mã không còn cần thiết nên được loại bỏ để giảm độ phức tạp của hệ thống [8].

Experiment Flag

Experiment flag được sử dụng để phân phối các biến thể khác nhau của một tính năng cho những nhóm người dùng khác nhau nhằm đo lường và so sánh kết quả. Loại flag này thường được áp dụng trong thử nghiệm A/B, kiểm chứng giả thuyết sản phẩm hoặc đánh giá tác động của một thay đổi đối với hành vi người dùng.

Khác với release flag, kết quả của experiment flag không nhất thiết chỉ gồm hai trạng thái bật và tắt. Một flag có thể trả về nhiều biến thể, chẳng hạn như `control`, `variant-a` và `variant-b`. Mỗi biến thể đại diện cho một hành vi hoặc giao diện khác nhau.

Việc phân nhóm người dùng cần bảo đảm tính ổn định. Cùng một người dùng nên tiếp tục nhận cùng một biến thể trong suốt thời gian thử nghiệm, trừ khi cấu hình được thay đổi có chủ đích. Điều này thường được thực hiện bằng cách áp dụng hàm băm ổn định trên khóa định danh của người dùng và khóa của flag.

Kết quả của experiment flag cần được kết hợp với dữ liệu đo lường, chẳng hạn như tỷ lệ chuyển đổi, thời gian sử dụng, số lần tương tác hoặc tỷ lệ lỗi. Chỉ riêng kết quả đánh giá flag không đủ để xác định biến thể nào hiệu quả hơn. Vì vậy, loại flag này thường gắn với hệ thống phân tích và quy trình đánh giá thống kê [1].

Experiment flag thường có thời gian tồn tại giới hạn theo thời gian thử nghiệm. Sau khi thu thập đủ dữ liệu và xác định được biến thể phù hợp, thử nghiệm cần được kết thúc, biến thể được lựa chọn trở thành hành vi chính và các nhánh mã còn lại nên được loại bỏ.

Ops và Kill-Switch Flag

Ops flag được sử dụng để kiểm soát hành vi vận hành của hệ thống, đặc biệt đối với các chức năng có ảnh hưởng đến hiệu năng, độ ổn định hoặc các dịch vụ phụ thuộc. Loại flag này cho phép đội vận hành thay đổi cách hệ thống hoạt động mà không cần sửa mã nguồn và deploy lại ứng dụng.

Ví dụ, một ops flag có thể được sử dụng để bật hoặc tắt tác vụ chạy nền, cơ chế gửi thông báo, dịch vụ gợi ý sản phẩm hoặc một tích hợp với hệ thống bên ngoài động mà không cần sửa mã nguồn và deploy lại ứng dụng.

Ví dụ, một ops flag có thể được sử dụng để bật hoặc tắt tác. Khi thành phần phụ thuộc gặp lỗi hoặc gây quá tải, đội vận hành có thể tạm thời vô hiệu hóa chức năng liên quan để duy trì

hoạt động của các phần quan trọng hơn.

Kill-switch flag là một dạng ops flag có mức ưu tiên cao, được thiết kế để tắt nhanh một tính năng hoặc một nhóm tính năng khi xảy ra sự cố. Khi kill switch được kích hoạt, quyết định này thường được thực hiện trước các quy tắc nhắm mục tiêu khác. Do đó, ngay cả khi người dùng thỏa mãn user allowlist, role targeting hoặc percentage rollout, tính năng vẫn bị tắt.

Một kill switch hiệu quả cần có các đặc điểm sau:

- Có thể được kích hoạt nhanh thông qua dashboard hoặc API quản trị.
- Có thứ tự ưu tiên rõ ràng trong quá trình đánh giá.
- Có hành vi dự phòng an toàn khi tính năng bị vô hiệu hóa.
- Có audit log ghi nhận người thực hiện, thời điểm và thay đổi cấu hình.
- Được kiểm thử định kỳ để bảo đảm cơ chế tắt khẩn cấp vẫn hoạt động.

Ops flag và kill-switch flag có thể tồn tại lâu hơn release flag nếu chúng đóng vai trò như một cơ chế kiểm soát vận hành lâu dài. Tuy nhiên, các flag không còn gắn với rủi ro thực tế hoặc không còn được sử dụng vẫn cần được rà soát và loại bỏ.

Permission Flag

Permission flag được sử dụng để xác định người dùng hoặc nhóm người dùng nào có quyền truy cập một tính năng. Quyết định đánh giá thường dựa trên vai trò, gói dịch vụ, loại tài khoản, tổ chức hoặc các thuộc tính nghiệp vụ khác.

Ví dụ, một tính năng quản trị nâng cao có thể chỉ được bật cho người dùng có vai trò `admin`, trong khi một chức năng thử nghiệm có thể chỉ dành cho nhóm `beta-customer`. Tương tự, một tính năng trả phí có thể được bật cho khách hàng thuộc gói `premium` và tắt đối với người dùng gói miễn phí.

Permission flag có điểm tương đồng với cơ chế phân quyền, nhưng không nên được xem là phương án thay thế hoàn toàn cho authorization. Feature Flag chủ yếu quyết định tính năng có được cung cấp hay không, trong khi hệ thống phân quyền phải bảo vệ tài nguyên và kiểm tra quyền truy cập tại phía máy chủ. Việc chỉ ẩn giao diện bằng flag mà không kiểm tra quyền tại backend có thể tạo ra rủi ro bảo mật.

Permission flag thường có thời gian tồn tại dài vì nó có thể phản ánh chính sách sản phẩm hoặc mô hình kinh doanh ổn định. Do đó, loại flag này cần được đặt tên rõ ràng, quản lý quyền thay đổi chặt chẽ và kiểm thử như một phần của logic nghiệp vụ.

Vòng đời và thời gian tồn tại của từng loại Flag

Mỗi loại Feature Flag có mục đích và thời gian tồn tại khác nhau. Việc áp dụng cùng một quy trình quản lý cho mọi loại flag có thể dẫn đến tình trạng các flag tạm thời tồn tại quá lâu hoặc các flag dài hạn bị loại bỏ không phù hợp.

Vòng đời tổng quát của một Feature Flag thường gồm các giai đoạn sau:

1. **Tạo flag:** xác định mục đích, người chịu trách nhiệm, giá trị mặc định và phạm vi sử dụng.
2. **Tích hợp:** thêm điểm kiểm tra flag vào mã nguồn và cấu hình các quy tắc đánh giá.
3. **Triển khai và vận hành:** sử dụng flag để kiểm soát phát hành, thử nghiệm, quyền truy cập hoặc hành vi vận hành.
4. **Ổn định:** theo dõi kết quả, xử lý sự cố và điều chỉnh phạm vi áp dụng.
5. **Kết thúc:** lưu trữ, vô hiệu hóa hoặc loại bỏ flag cùng các nhánh mã không còn cần thiết.

Bảng 2.2 trình bày đặc điểm vòng đời của các loại flag phổ biến.

Bảng 2.2: Vòng đời của các loại Feature Flag phổ biến

Loại flag	Mục đích chính	Thời gian tồn tại	Điều kiện kết thúc
Release flag	Kiểm soát quá trình phát hành tính năng mới	Ngắn hoặc trung hạn	Tính năng đã phát hành hoàn toàn và hành vi cũ không còn cần thiết
Experiment flag	So sánh các biến thể và kiểm chứng giả thuyết	Ngắn hạn	Thử nghiệm kết thúc và đã lựa chọn được biến thể chính thức
Ops/Kill-switch flag	Kiểm soát rủi ro vận hành và phản ứng sự cố	Trung hoặc dài hạn	Rủi ro không còn tồn tại hoặc cơ chế kiểm soát đã được thay thế
Permission flag	Kiểm soát quyền truy cập theo vai trò hoặc chính sách sản phẩm	Dài hạn	Chính sách quyền hoặc mô hình sản phẩm thay đổi

Release flag và experiment flag thường được xem là các flag tạm thời. Nếu không được loại bỏ sau khi hoàn thành mục đích, chúng tạo ra nhiều nhánh xử lý dư thừa, làm tăng số trường hợp

cần kiểm thử và gây khó khăn khi bảo trì. Tình trạng này được gọi là *feature flag debt* [9].

Ngược lại, ops flag và permission flag có thể được duy trì lâu dài nếu chúng tiếp tục mang lại giá trị vận hành hoặc nghiệp vụ. Tuy nhiên, flag dài hạn vẫn cần được rà soát định kỳ để xác nhận rằng cấu hình, người sở hữu và mục đích sử dụng còn phù hợp.

Để quản lý vòng đời hiệu quả, mỗi flag nên có metadata mô tả loại flag, người chịu trách nhiệm, ngày tạo, thời điểm dự kiến rà soát và trạng thái vòng đời. Hệ thống cũng nên hỗ trợ lưu trữ flag thay vì xóa trực tiếp, ghi nhận lịch sử thay đổi và cảnh báo các flag đã tồn tại vượt quá thời gian dự kiến [8].

2.1.5 Các chiến lược phát hành tính năng

Feature Flag cho phép đội phát triển kiểm soát phạm vi phát hành tính năng theo nhiều chiến lược khác nhau. Thay vì chỉ bật hoặc tắt tính năng cho toàn bộ hệ thống, flag có thể được đánh giá dựa trên danh tính, vai trò hoặc nhóm phân phối của người dùng. Việc lựa chọn chiến lược phù hợp phụ thuộc vào mục tiêu phát hành, mức độ rủi ro và loại thông tin có trong ngữ cảnh đánh giá.

Bật hoặc tắt toàn cục

Bật hoặc tắt toàn cục là chiến lược đơn giản nhất, trong đó một tính năng được cung cấp cho toàn bộ người dùng hoặc bị vô hiệu hóa hoàn toàn. Kết quả đánh giá không phụ thuộc vào danh tính, vai trò hay thuộc tính của người dùng.

Khi flag được bật toàn cục, mọi yêu cầu đánh giá hợp lệ đều nhận kết quả bật. Chiến lược này phù hợp khi tính năng đã được kiểm thử đầy đủ và sẵn sàng phát hành rộng rãi. Ngược lại, trạng thái tắt toàn cục được sử dụng khi tính năng chưa sẵn sàng, đã kết thúc hoặc cần được vô hiệu hóa nhanh do xảy ra sự cố.

Bật hoặc tắt toàn cục thường được sử dụng trong các trường hợp sau:

- Phát hành hoàn toàn một tính năng đã ổn định.
- Ẩn một tính năng chưa hoàn thiện nhưng mã nguồn đã được deploy.
- Tạm dừng một tính năng gây lỗi hoặc ảnh hưởng đến hiệu năng.
- Kiểm soát các chức năng vận hành như tác vụ nền hoặc tích hợp bên ngoài.

Do không cần xử lý nhiều điều kiện, chiến lược này có chi phí đánh giá thấp và hành vi dễ dự đoán. Tuy nhiên, nó không giới hạn được phạm vi ảnh hưởng khi tính năng được bật, vì toàn bộ người dùng sẽ nhận cùng một kết quả.

Targeting theo người dùng

Targeting theo người dùng cho phép bật tính năng cho một tập hợp đối tượng cụ thể dựa trên khóa định danh của họ. Khóa này có thể là mã người dùng, mã khách hàng hoặc một `targetingKey` ổn định do ứng dụng tích hợp cung cấp trong ngữ cảnh đánh giá [6].

Chiến lược này thường được triển khai dưới dạng danh sách cho phép. Hệ thống so sánh khóa định danh trong ngữ cảnh với danh sách đã được cấu hình cho flag. Nếu tìm thấy giá trị phù hợp, tính năng được bật; nếu không, hệ thống tiếp tục đánh giá quy tắc tiếp theo hoặc trả về giá trị mặc định.

Targeting theo người dùng phù hợp trong các trường hợp như:

- Bật tính năng cho thành viên đội phát triển hoặc kiểm thử.
- Cho phép một số khách hàng tham gia chương trình beta.
- Kiểm tra lỗi với một tài khoản cụ thể trên production.
- Cung cấp tính năng sớm cho một nhóm khách hàng chiến lược.

Ví dụ, một flag có thể được cấu hình bật cho các khóa `user-001`, `user-024` và `user-105`. Khi một trong các đối tượng này gửi yêu cầu, hệ thống trả về trạng thái bật bất kể phần lớn người dùng còn lại đang nhận trạng thái tắt.

Khóa dùng để targeting cần ổn định và duy nhất trong phạm vi phù hợp. Không nên phụ thuộc vào các giá trị có thể thay đổi thường xuyên, chẳng hạn như tên hiển thị hoặc địa chỉ email chưa được chuẩn hóa. Đồng thời, dữ liệu định danh nhạy cảm không nên được đưa trực tiếp vào cấu hình nếu không thực sự cần thiết [8].

Targeting theo vai trò

Targeting theo vai trò quyết định trạng thái tính năng dựa trên vai trò nghiệp vụ của người dùng, chẳng hạn như `admin`, `beta-customer` hoặc `regular-customer`. Ứng dụng tích hợp cung cấp danh sách vai trò trong ngữ cảnh đánh giá, sau đó hệ thống kiểm tra xem ít nhất một vai

trò có phù hợp với quy tắc đã cấu hình hay không.

Chiến lược này hữu ích khi quyền sử dụng tính năng gắn với một nhóm người dùng thay vì từng cá nhân. So với targeting theo người dùng, việc cấu hình theo vai trò dễ quản lý hơn khi số lượng người dùng lớn và thành viên của nhóm thường xuyên thay đổi.

Ví dụ, một tính năng quản trị mới có thể được bật cho vai trò `admin`, trong khi một giao diện thử nghiệm được bật cho vai trò `beta-customer`. Khi vai trò của người dùng thay đổi trong hệ thống nguồn, kết quả đánh giá flag cũng thay đổi tương ứng mà không cần cập nhật từng định danh người dùng trong cấu hình.

Một quy tắc targeting theo vai trò có thể được mô tả như sau:

Đoạn mã 2.4: Ví dụ ngữ cảnh đánh giá theo vai trò

```
const context = {
    targetingKey: "user-024",
    roles: ["beta-customer"]
};

const result = await featureFlagClient.evaluate(
    "new-product-page",
    context
);
```

Cần phân biệt targeting theo vai trò với cơ chế phân quyền bảo mật. Vai trò trong Feature Flag được sử dụng để quyết định cách phát hành tính năng, nhưng backend vẫn phải kiểm tra quyền truy cập đối với các tài nguyên và thao tác nhạy cảm. Việc flag trả về trạng thái bật không tự động chứng minh rằng người dùng có quyền thực hiện mọi hành động liên quan.

Rollout theo phần trăm người dùng

Rollout theo phần trăm cho phép phát hành tính năng cho một tỷ lệ xác định trong tổng số người dùng đủ điều kiện. Chiến lược này giúp đội phát triển mở rộng phạm vi phát hành từng bước thay vì bật tính năng đồng thời cho toàn bộ hệ thống.

Một quy trình rollout có thể bắt đầu với 5% người dùng, sau đó tăng lên 20%, 50% và cuối cùng là 100%. Sau mỗi giai đoạn, đội phát triển theo dõi tỷ lệ lỗi, thời gian phản hồi, phản hồi

của người dùng và các chỉ số nghiệp vụ trước khi quyết định tiếp tục mở rộng.

Để bảo đảm kết quả ổn định, hệ thống không nên lựa chọn ngẫu nhiên lại người dùng trong mỗi lần đánh giá. Thay vào đó, một hàm băm ổn định được áp dụng trên khóa của flag và khóa định danh của đối tượng. Giá trị băm sau đó được ánh xạ vào một khoảng bucket cố định. Tính năng được bật nếu bucket của người dùng nằm trong tỷ lệ rollout đã cấu hình [5].

Cơ chế này có thể được biểu diễn khái quát như sau:

$$b = H(flagKey, targetingKey) \bmod 10000$$

Trong đó H là hàm băm ổn định và b là bucket của người dùng trong khoảng từ 0 đến 9999. Với tỷ lệ rollout $p\%$, tính năng được bật khi:

$$b < p \times 100$$

Ví dụ, với rollout 20%, các bucket từ 0 đến 1999 nhận trạng thái bật. Do cùng một đầu vào tạo ra cùng một giá trị băm, một người dùng tiếp tục nhận cùng một kết quả giữa nhiều lần truy cập, miễn là khóa định danh, khóa flag và thuật toán phân bucket không thay đổi.

Rollout theo phần trăm giúp giới hạn phạm vi ảnh hưởng nhưng không bảo đảm rằng tỷ lệ quan sát được trong một tập người dùng nhỏ luôn chính xác tuyệt đối. Tỷ lệ cấu hình thể hiện tỷ lệ bucket mục tiêu; khi số lượng người dùng đủ lớn, tỷ lệ thực tế thường tiến gần đến giá trị này.

Kết hợp nhiều chiến lược Targeting

Trong thực tế, một flag thường cần kết hợp nhiều chiến lược để đáp ứng quy trình phát hành theo từng giai đoạn. Ví dụ, tính năng có thể được bật cho một số tài khoản kiểm thử cụ thể, tiếp tục bật cho toàn bộ khách hàng beta và sau đó rollout cho 20% người dùng còn lại.

Khi nhiều quy tắc cùng tồn tại, hệ thống cần xác định thứ tự đánh giá rõ ràng. Một thứ tự phổ biến có thể gồm:

1. Kiểm tra trạng thái vòng đời và trạng thái bật hoặc tắt của flag.
2. Kiểm tra kill switch hoặc các cơ chế vô hiệu hóa ưu tiên cao.

3. Trả về trạng thái bật nếu flag đang ở chế độ bật toàn cục.
4. Kiểm tra danh sách người dùng được cho phép.
5. Kiểm tra các quy tắc targeting theo vai trò.
6. Kiểm tra rollout theo phần trăm.
7. Trả về giá trị mặc định nếu không có quy tắc nào phù hợp.

Với cách tổ chức này, quy tắc cụ thể thường được đánh giá trước quy tắc có phạm vi rộng hơn. Người dùng nằm trong allowlist có thể được bật tính năng ngay cả khi bucket phần trăm của họ không thuộc nhóm rollout. Tương tự, người dùng có vai trò beta có thể được truy cập tính năng trước khi tính năng được phát hành cho người dùng thông thường.

Ví dụ, một chiến lược phát hành kết hợp có thể được cấu hình như sau:

- Bật cho các tài khoản nội bộ được xác định bằng `targetingKey`.
- Bật cho người dùng có vai trò `beta-customer`.
- Bật cho 10% người dùng còn lại bằng stable hashing.
- Tắt mặc định đối với các đối tượng không thỏa mãn bất kỳ điều kiện nào.

Các quy tắc cùng loại cũng cần có mức độ ưu tiên xác định. Bộ máy đánh giá thường sắp xếp quy tắc theo trường `priority` và dừng tại quy tắc đầu tiên phù hợp [4]. Vì vậy, thay đổi thứ tự quy tắc có thể làm thay đổi kết quả ngay cả khi nội dung từng quy tắc không đổi.

Khi kết hợp nhiều chiến lược, hệ thống nên trả về không chỉ trạng thái bật hoặc tắt mà còn cả lý do đánh giá, chẳng hạn như `USER_MATCH`, `ROLE_MATCH`, `PERCENTAGE_MATCH` hoặc `DEFAULT_OFF`. Thông tin này giúp giải thích quyết định, hỗ trợ kiểm thử và rút ngắn thời gian phân tích khi hành vi thực tế không đúng như mong đợi [10].

2.1.6 Lợi ích và rủi ro của Feature Flag

Feature Flag mang lại khả năng kiểm soát linh hoạt đối với quá trình phát hành và vận hành phần mềm. Tuy nhiên, việc bổ sung các điểm rẽ nhánh động vào hệ thống cũng làm tăng độ phức tạp của mã nguồn, cấu hình và quy trình quản trị. Vì vậy, Feature Flag chỉ phát huy hiệu quả khi được sử dụng có mục đích, có cơ chế giám sát và có kế hoạch quản lý vòng đời rõ ràng [8, 9].

Tăng tốc độ phát hành

Feature Flag giúp tăng tốc độ phát hành bằng cách tách quá trình deploy mã nguồn khỏi quyết định release tính năng. Đội phát triển có thể tích hợp và triển khai mã của tính năng mới lên môi trường production trong khi vẫn giữ tính năng ở trạng thái tắt đối với người dùng cuối. Khi tính năng đã sẵn sàng, việc phát hành được thực hiện bằng cách thay đổi cấu hình flag thay vì xây dựng và triển khai một phiên bản mới [1, 3].

Cơ chế này cho phép các thay đổi nhỏ được tích hợp thường xuyên vào nhánh mã chính, giảm thời gian tồn tại của các nhánh phát triển dài hạn và hạn chế xung đột khi hợp nhất mã. Nhiều tính năng có thể được triển khai trong cùng một phiên bản ứng dụng nhưng được phát hành ở các thời điểm khác nhau, tùy theo mức độ hoàn thiện và yêu cầu nghiệp vụ.

Feature Flag cũng giảm sự phụ thuộc giữa đội phát triển và đội sản phẩm. Đội kỹ thuật có thể hoàn tất deploy theo lịch trình kỹ thuật, trong khi đội sản phẩm lựa chọn thời điểm release phù hợp với kế hoạch kinh doanh, hoạt động truyền thông hoặc mức độ sẵn sàng của người dùng.

Tuy nhiên, tốc độ phát hành chỉ được cải thiện khi quy trình cấu hình flag đủ rõ ràng và đáng tin cậy. Nếu việc bật flag yêu cầu nhiều thao tác thủ công, thiếu kiểm thử hoặc không có cơ chế phê duyệt phù hợp, Feature Flag có thể chuyển rủi ro từ deploy sang quá trình quản lý cấu hình.

Kiểm soát phạm vi ảnh hưởng

Feature Flag cho phép giới hạn phạm vi người dùng chịu ảnh hưởng của một thay đổi. Thay vì phát hành đồng thời cho toàn bộ hệ thống, tính năng có thể được bật trước cho tài khoản nội bộ, người dùng beta, một vai trò cụ thể hoặc một tỷ lệ nhỏ người dùng.

Cách tiếp cận này giúp đội phát triển quan sát hành vi thực tế của tính năng trước khi mở rộng phạm vi phát hành. Nếu xuất hiện lỗi, chỉ một nhóm người dùng giới hạn bị ảnh hưởng. Nếu hệ thống hoạt động ổn định, phạm vi rollout có thể được tăng dần đến toàn bộ người dùng.

Các chiến lược targeting còn cho phép tổ chức kiểm soát phạm vi theo nhu cầu nghiệp vụ. Ví dụ, một tính năng có thể chỉ được cung cấp cho khách hàng thuộc một gói dịch vụ, một tổ chức cụ thể hoặc một nhóm có vai trò phù hợp. Nhờ đó, cùng một phiên bản ứng dụng có thể phục vụ nhiều nhóm người dùng với các hành vi khác nhau.

Việc giới hạn phạm vi ảnh hưởng không đồng nghĩa với việc loại bỏ hoàn toàn rủi ro. Các thay

đổi trong mã dùng chung, cơ sở dữ liệu hoặc hạ tầng vẫn có thể ảnh hưởng đến toàn bộ hệ thống, ngay cả khi tính năng được đặt phía sau flag. Do đó, phạm vi bảo vệ của từng flag cần được xác định rõ trong quá trình thiết kế.

Rollback và khôi phục dịch vụ

Khi một tính năng gây lỗi trên production, Feature Flag cho phép vô hiệu hóa nhanh hành vi mới và đưa người dùng trở lại nhánh xử lý ổn định trước đó. Thao tác này thường chỉ yêu cầu thay đổi cấu hình và có thể được thực hiện nhanh hơn so với rollback phiên bản hoặc triển khai một bản sửa lỗi khẩn cấp.

Đối với các thành phần có rủi ro vận hành cao, flag có thể đóng vai trò như một kill switch. Khi kill switch được kích hoạt, tính năng bị tắt bất kể các quy tắc targeting khác có phù hợp hay không. Cơ chế này giúp đội vận hành giảm thời gian phản ứng và giới hạn tác động của sự cố.

Ví dụ, nếu một dịch vụ bên ngoài làm tăng thời gian phản hồi, ứng dụng có thể bỏ qua dịch vụ đó khi flag bị tắt và chuyển sang hành vi dự phòng:

Đoạn mã 2.5: Khôi phục hành vi dự phòng bằng Feature Flag

```
if (featureFlags.isEnabled("external-recommendation-service")) {
    return loadExternalRecommendations(userContext);
}

return loadDefaultRecommendations();
```

Khả năng tắt flag không hoàn toàn tương đương với rollback. Rollback đưa toàn bộ ứng dụng hoặc một thành phần về phiên bản trước, trong khi tắt flag chỉ thay đổi nhánh hành vi được lựa chọn trong phiên bản hiện tại. Nếu lỗi nằm trong quá trình khởi động, migration cơ sở dữ liệu, thư viện dùng chung hoặc đoạn mã được thực thi trước khi đánh giá flag, việc tắt flag có thể không khôi phục được dịch vụ.

Vì vậy, Feature Flag nên được kết hợp với cơ chế rollback phiên bản, giám sát, cảnh báo và kế hoạch khôi phục sự cố. Nhánh xử lý khi flag tắt cũng cần được kiểm thử thường xuyên để bảo đảm vẫn hoạt động khi cần sử dụng.

Độ phức tạp của các nhánh xử lý

Mỗi Feature Flag bổ sung ít nhất một điểm rẽ nhánh vào mã nguồn. Khi nhiều flag cùng tác động đến một luồng xử lý, số lượng tổ hợp trạng thái có thể tăng nhanh. Với n flag nhị phân độc lập, về lý thuyết có thể tồn tại tối đa: 2^n tổ hợp bật và tắt khác nhau.

Không phải mọi tổ hợp đều xuất hiện trong thực tế, nhưng đội phát triển vẫn phải xác định các trạng thái hợp lệ và kiểm thử các trường hợp quan trọng. Nếu hai hoặc nhiều flag phụ thuộc lẫn nhau, hành vi của hệ thống có thể trở nên khó dự đoán và khó truy vết.

Độ phức tạp còn xuất hiện trong cấu hình. Một flag có thể có nhiều quy tắc user targeting, role targeting và percentage rollout với thứ tự ưu tiên khác nhau. Kết quả đánh giá không chỉ phụ thuộc vào trạng thái của flag mà còn phụ thuộc vào ngữ cảnh người dùng, môi trường, thứ tự quy tắc và giá trị mặc định.

Để hạn chế độ phức tạp, điểm kiểm tra flag nên được đặt tại các ranh giới chức năng rõ ràng. Đội phát triển nên tránh kiểm tra cùng một flag tại quá nhiều vị trí hoặc lồng nhiều flag trong cùng một khối điều kiện. Bộ máy đánh giá cũng cần trả về lý do quyết định để hỗ trợ kiểm thử và phân tích lỗi.

Feature Flag Debt

Feature flag debt là khoản nợ kỹ thuật hình thành khi các flag tạm thời không được loại bỏ sau khi đã hoàn thành mục đích. Flag có thể đã được bật cho toàn bộ người dùng nhưng nhánh xử lý cũ vẫn tồn tại, hoặc thử nghiệm đã kết thúc nhưng các biến thể không còn sử dụng vẫn được giữ lại trong mã nguồn và cấu hình [9].

Các flag lỗi thời làm tăng số lượng nhánh cần đọc, kiểm thử và bảo trì. Thành viên mới có thể không biết flag nào còn hiệu lực, flag nào có thể thay đổi và flag nào chỉ còn tồn tại do chưa được dọn dẹp. Theo thời gian, việc xóa flag trở nên khó khăn hơn vì không còn rõ những thành phần nào phụ thuộc vào nó.

Feature flag debt có thể dẫn đến các vấn đề sau:

- Mã nguồn chứa nhiều nhánh xử lý không còn cần thiết.
- Số lượng trường hợp kiểm thử và cấu hình tăng lên.
- Khó xác định trạng thái thực tế của tính năng.

- Tăng nguy cơ bật lại nhầm một hành vi đã lỗi thời.
- Tăng chi phí bảo trì và phân tích sự cố.

Để kiểm soát feature flag debt, mỗi flag nên có loại, người chịu trách nhiệm, ngày tạo và thời điểm dự kiến rà soát. Release flag và experiment flag cần có tiêu chí kết thúc rõ ràng. Khi tính năng đã ổn định, đội phát triển nên loại bỏ nhánh cũ, điểm kiểm tra trong mã nguồn và cấu hình flag không còn sử dụng [8].

Rủi ro từ cấu hình sai

Feature Flag chuyển một phần hành vi của hệ thống từ mã nguồn sang cấu hình có thể thay đổi tại thời điểm chạy. Điều này tạo ra sự linh hoạt nhưng cũng làm tăng rủi ro từ thao tác cấu hình sai.

Một lỗi cấu hình có thể bao gồm bật nhầm flag trên production, đặt sai tỷ lệ rollout, nhập sai định danh người dùng, thay đổi thứ tự ưu tiên của quy tắc hoặc kích hoạt nhầm kill switch. Không giống lỗi mã nguồn phải trải qua quá trình build và deploy, lỗi cấu hình có thể ảnh hưởng đến hệ thống ngay sau khi thay đổi được áp dụng.

Rủi ro cũng có thể xuất hiện do khác biệt cấu hình giữa các environment. Một flag hoạt động đúng trên staging nhưng có thể cho kết quả khác trên production nếu trạng thái, quy tắc hoặc dữ liệu ngữ cảnh không đồng nhất.

Để giảm rủi ro từ cấu hình sai, nền tảng quản lý Feature Flag nên hỗ trợ:

- Validation dữ liệu và giới hạn giá trị hợp lệ.
- Hiển thị rõ environment đang được chỉnh sửa.
- Cảnh báo trước các thay đổi có phạm vi ảnh hưởng lớn.
- Phân quyền đối với thao tác tạo và thay đổi flag.
- Audit log lưu cấu hình trước và sau thay đổi.
- Versioning hoặc lịch sử để khôi phục cấu hình trước đó.
- Quy trình phê duyệt đối với các thay đổi quan trọng.

Hệ thống cũng cần xác định giá trị mặc định an toàn khi không thể tải cấu hình hoặc khi quá trình đánh giá xảy ra lỗi. Việc lựa chọn mặc định bật hay tắt phụ thuộc vào bản chất của tính

năng, nhưng quyết định này phải được xác định có chủ đích thay vì phụ thuộc vào hành vi ngẫu nhiên.

Rủi ro bảo mật và quyền truy cập

Nền tảng Feature Flag có khả năng thay đổi hành vi của ứng dụng production nên các API và dashboard quản trị trở thành mục tiêu nhạy cảm. Nếu quyền truy cập không được kiểm soát, một người dùng không hợp lệ có thể bật tính năng chưa hoàn thiện, tắt chức năng quan trọng hoặc thay đổi phạm vi phát hành.

Các thao tác quản trị cần được bảo vệ bằng cơ chế xác thực và phân quyền. Vai trò quản trị viên có thể được phép tạo, lưu trữ và kích hoạt kill switch, trong khi developer chỉ được chỉnh sửa một số cấu hình và viewer chỉ có quyền xem. Nguyên tắc quyền tối thiểu giúp giới hạn hậu quả nếu một tài khoản bị lộ hoặc sử dụng sai mục đích.

Endpoint evaluation cũng cần được bảo vệ phù hợp với mô hình triển khai. Dữ liệu đầu vào phải được validation để ngăn chặn các giá trị không hợp lệ hoặc payload quá lớn. Giao tiếp giữa client và hệ thống Feature Flag cần sử dụng kênh mã hóa, đặc biệt khi evaluation context chứa thông tin liên quan đến người dùng [11].

Evaluation context và cấu hình targeting không nên chứa dữ liệu cá nhân nhạy cảm nếu không cần thiết. Các thuộc tính như mật khẩu, token truy cập hoặc thông tin định danh trực tiếp không nên được lưu trong rule. Khi cần nhận diện người dùng, hệ thống nên sử dụng một targeting key ổn định với phạm vi và mục đích rõ ràng [8].

Feature Flag cũng không nên được sử dụng như cơ chế authorization duy nhất. Việc flag trả về trạng thái tắt có thể ẩn một nút hoặc một màn hình, nhưng backend vẫn phải kiểm tra quyền trước khi thực hiện thao tác nhạy cảm. Nếu chỉ dựa vào flag ở phía client, người dùng có thể bỏ qua giao diện và gọi trực tiếp API được bảo vệ không đầy đủ.

Cuối cùng, mọi thay đổi quan trọng cần được ghi nhận trong audit log, bao gồm chủ thể thực hiện, thời điểm, hành động, đối tượng và kết quả thay đổi. Audit log hỗ trợ truy vết sự cố và trách nhiệm vận hành, nhưng không nên lưu token, mật khẩu hoặc dữ liệu nhạy cảm không cần thiết [12].

2.2 Khảo sát các nền tảng Feature Flag hiện có

2.2.1 Nền tảng LaunchDarkly

LaunchDarkly là một nền tảng quản lý Feature Flag dưới dạng dịch vụ, cung cấp các công cụ để kiểm soát việc phát hành tính năng, phân phối biến thể và quản trị cấu hình theo từng môi trường. Thay vì gắn quá trình phát hành tính năng trực tiếp với một lần triển khai mã nguồn, ứng dụng tích hợp LaunchDarkly có thể đánh giá trạng thái flag tại thời điểm chạy và lựa chọn hành vi tương ứng [7].

Kiến trúc và mô hình hoạt động

LaunchDarkly tổ chức tài nguyên theo mô hình gồm project, environment và feature flag. Một project đại diện cho một sản phẩm hoặc hệ thống phần mềm, trong khi các environment biểu diễn những môi trường triển khai khác nhau như development, staging và production. Mỗi feature flag có cùng một khóa định danh trong phạm vi project nhưng có thể sở hữu trạng thái, tập luật targeting và cấu hình rollout độc lập giữa các environment.

Nền tảng đóng vai trò là lớp điều khiển tập trung để người quản trị tạo flag, thay đổi trạng thái và cấu hình các luật phân phối. Trong ứng dụng, LaunchDarkly SDK tiếp nhận evaluation context, tải cấu hình tương ứng với environment và thực hiện đánh giá để trả về một variation. Variation có thể là giá trị Boolean hoặc một giá trị đa biến như chuỗi, số hay đối tượng JSON [7].

Luồng hoạt động tổng quát gồm các bước sau:

1. Người quản trị tạo feature flag và khai báo các variation.
2. Cấu hình của flag được thiết lập riêng cho từng environment.
3. Ứng dụng khởi tạo SDK bằng thông tin xác thực của environment.
4. Khi cần sử dụng tính năng, ứng dụng truyền flag key, evaluation context và giá trị mặc định cho SDK.
5. SDK đánh giá các luật theo thứ tự ưu tiên và trả về variation phù hợp.
6. Ứng dụng lựa chọn nhánh xử lý tương ứng với variation nhận được.

Giá trị mặc định được khai báo trong mã nguồn giúp ứng dụng có hành vi dự phòng khi flag

không tồn tại, SDK chưa khởi tạo thành công hoặc quá trình đánh giá gặp lỗi. Cơ chế này làm giảm khả năng sự cố của hệ thống Feature Flag gây gián đoạn trực tiếp cho ứng dụng.

Khả năng targeting và rollout

LaunchDarkly hỗ trợ targeting dựa trên *context*. Context có thể biểu diễn người dùng, thiết bị, tổ chức hoặc một thực thể nghiệp vụ khác và chứa các thuộc tính phục vụ quá trình đánh giá, chẳng hạn như mã định danh, vai trò, gói dịch vụ, khu vực hoặc phiên bản ứng dụng [4].

Một cấu hình flag có thể bao gồm targeting trực tiếp, các rule có điều kiện và default rule. Các rule được đánh giá theo thứ tự từ trên xuống dưới; rule đầu tiên thỏa mãn điều kiện sẽ quyết định variation được trả về. Điều kiện của rule có thể sử dụng nhiều toán tử như so sánh bằng, kiểm tra tập hợp, tiền tố, hậu tố, biểu thức chính quy, giá trị số, thời gian hoặc phiên bản ngữ nghĩa.

Nền tảng còn hỗ trợ segment để tái sử dụng một nhóm đối tượng trong nhiều feature flag. Thay vì lặp lại cùng một danh sách người dùng hoặc cùng một điều kiện trong từng flag, người quản trị có thể tạo segment và tham chiếu segment đó trong các targeting rule.

Đối với percentage rollout, LaunchDarkly phân chia context vào các bucket dựa trên một thuộc tính ổn định, thường là khóa định danh của context. Kết quả phân chia có tính xác định, do đó cùng một context thường tiếp tục nhận cùng một variation khi được đánh giá nhiều lần với cùng cấu hình. Cơ chế này hạn chế tình trạng người dùng liên tục chuyển đổi giữa phiên bản cũ và phiên bản mới [5].

Percentage rollout có thể được áp dụng trong default rule hoặc kết hợp với targeting rule. Ví dụ, hệ thống có thể giới hạn rule cho những người dùng có thuộc tính `role = beta-customer`, sau đó chỉ bật tính năng cho một tỷ lệ nhất định trong nhóm này. Nhờ đó, phạm vi rollout được xác định trên tập context đã thỏa mãn điều kiện thay vì trên toàn bộ người dùng.

Ngoài rollout thủ công, LaunchDarkly còn hỗ trợ các cơ chế phát hành nâng cao như tăng tỷ lệ theo tiến trình và kết hợp rollout với dữ liệu giám sát. Tuy nhiên, đối với một hệ thống Feature Flag cơ bản, targeting theo thuộc tính và percentage rollout ổn định đã đáp ứng phần lớn nhu cầu phát hành có kiểm soát.

SDK, audit log và quản trị

LaunchDarkly cung cấp SDK cho nhiều nền tảng phía máy chủ, trình duyệt, thiết bị di động và hệ thống biên. SDK che giấu phần lớn chi tiết giao tiếp với dịch vụ Feature Flag và cung cấp giao diện đánh giá thống nhất cho ứng dụng. Thông thường, ứng dụng chỉ cần cung cấp flag key, context và giá trị mặc định để nhận variation tương ứng.

SDK phía máy chủ có thể lưu cục bộ cấu hình flag và thực hiện đánh giá trong tiến trình ứng dụng. Cách tiếp cận này giúp giảm số lần gọi mạng trên từng request và cho phép ứng dụng tiếp tục đánh giá bằng cấu hình gần nhất khi kết nối tới dịch vụ quản lý tạm thời bị gián đoạn. SDK cũng có thể gửi các sự kiện đánh giá để nền tảng thống kê mức độ sử dụng flag và phân tích việc phân phối variation.

LaunchDarkly ghi nhận lịch sử thay đổi đối với feature flag và các tài nguyên liên quan. Thông tin lịch sử cho phép xác định đối tượng thực hiện thay đổi, thời điểm thay đổi và nội dung cấu hình trước hoặc sau thao tác. Audit log hỗ trợ điều tra sự cố, kiểm tra trách nhiệm và khôi phục cấu hình khi một thay đổi không phù hợp được áp dụng.

Về quản trị, nền tảng hỗ trợ quản lý thành viên, nhóm và quyền truy cập. Quyền có thể được giới hạn theo tài nguyên hoặc thao tác, chẳng hạn như chỉ cho phép một nhóm thay đổi flag trong staging nhưng yêu cầu quyền cao hơn khi sửa cấu hình production. Cách tổ chức này giảm nguy cơ thay đổi ngoài ý muốn và phù hợp với các hệ thống có nhiều nhóm phát triển cùng sử dụng chung nền tảng.

Dashboard của LaunchDarkly cung cấp giao diện tập trung để quản lý flag, environment, targeting rule, segment và lịch sử thay đổi. Bên cạnh dashboard, tài nguyên còn có thể được quản lý thông qua REST API, cho phép tích hợp việc tạo hoặc cập nhật flag vào các quy trình CI/CD và công cụ nội bộ.

Ưu điểm và hạn chế

Ưu điểm chính của LaunchDarkly là hệ sinh thái Feature Flag tương đối hoàn chỉnh. Nền tảng hỗ trợ nhiều loại variation, targeting linh hoạt, percentage rollout ổn định, quản lý nhiều environment, SDK cho nhiều ngôn ngữ và cơ chế theo dõi lịch sử thay đổi. Các khả năng này giúp LaunchDarkly đáp ứng cả nhu cầu phát hành tính năng đơn giản lẫn những quy trình triển khai có nhiều nhóm và nhiều môi trường.

Việc sử dụng SDK và lưu cấu hình cục bộ giúp giảm độ trễ đánh giá so với mô hình gửi yêu cầu HTTP đến máy chủ Feature Flag trong mọi lần kiểm tra. Khả năng phân quyền, audit log và quản lý cấu hình tập trung cũng giúp giảm rủi ro khi nhiều thành viên cùng thay đổi trạng thái của các tính năng quan trọng.

Tuy nhiên, số lượng chức năng và tùy chọn cấu hình lớn có thể làm tăng độ phức tạp trong quá trình tiếp cận và quản trị. Khi số lượng flag và targeting rule tăng, đội phát triển cần thiết lập quy ước đặt tên, người chịu trách nhiệm, thời hạn sử dụng và quy trình loại bỏ flag để tránh hình thành feature flag debt.

Do được cung cấp dưới dạng dịch vụ quản lý bên ngoài, hệ thống sử dụng LaunchDarkly còn phụ thuộc vào hạ tầng, mô hình định giá và cơ chế tích hợp của nhà cung cấp. Việc truyền context cũng cần được kiểm soát để tránh gửi thông tin nhận dạng hoặc dữ liệu nhạy cảm không cần thiết. Ngoài ra, thay đổi cấu hình phải được phân phối đến các SDK nên có thể tồn tại một khoảng trễ ngắn trước khi tất cả instance của ứng dụng nhận được cùng một phiên bản cấu hình.

Nhìn chung, LaunchDarkly phù hợp với các tổ chức cần một nền tảng Feature Flag hoàn chỉnh, có khả năng mở rộng và quản trị chặt chẽ. Đối với MiniProject, nền tảng này cung cấp mô hình tham khảo quan trọng cho cách tổ chức project, environment, flag, ordered targeting rule, percentage rollout, evaluation SDK và audit log. Tuy nhiên, phạm vi triển khai của MiniProject chỉ lựa chọn các thành phần cốt lõi nhằm bảo đảm hệ thống dễ hiểu, dễ demo và phù hợp với mục tiêu nghiên cứu.

2.2.2 Nền tảng Unleash

Unleash là một nền tảng Feature Flag mã nguồn mở, cho phép tổ chức tự triển khai trên hạ tầng riêng hoặc sử dụng dịch vụ được quản lý. Nền tảng tập trung vào việc tách cấu hình phát hành khỏi mã nguồn ứng dụng, đồng thời cung cấp các activation strategy để quyết định tính năng được bật cho đối tượng nào trong từng môi trường [2].

Kiến trúc và mô hình triển khai

Kiến trúc của Unleash gồm máy chủ quản lý, cơ sở dữ liệu, giao diện quản trị và các SDK được tích hợp vào ứng dụng. Unleash Server đóng vai trò là control plane, chịu trách nhiệm lưu trữ project, environment, feature flag, strategy và các cấu hình quản trị. SDK định kỳ lấy cấu hình

từ máy chủ và thực hiện đánh giá flag trong tiến trình của ứng dụng.

Unleash tổ chức tài nguyên theo mô hình project, environment và feature flag. Một project đại diện cho một sản phẩm hoặc phạm vi nghiệp vụ. Mỗi project có thể sử dụng nhiều environment như development, staging và production. Feature flag được định nghĩa trong project nhưng trạng thái bật, tắt và activation strategy có thể được cấu hình độc lập theo từng environment.

Luồng hoạt động tổng quát của nền tảng gồm các bước sau:

1. Người quản trị tạo feature flag trong một project.
2. Flag được kích hoạt và cấu hình strategy riêng cho từng environment.
3. SDK của ứng dụng xác thực với Unleash Server và tải tập cấu hình phù hợp.
4. Ứng dụng truyền flag key cùng evaluation context cho SDK.
5. SDK lần lượt đánh giá các activation strategy của flag.
6. Nếu có ít nhất một strategy thỏa mãn, flag được xem là đang bật.
7. SDK trả về trạng thái hoặc variant để ứng dụng lựa chọn nhánh xử lý.

Việc đánh giá flag chủ yếu được thực hiện tại SDK thay vì gửi một yêu cầu tới máy chủ trong mỗi lần kiểm tra. Cách tiếp cận này giúp giảm độ trễ, giảm lưu lượng mạng và hạn chế việc Unleash Server trở thành điểm nghẽn của luồng xử lý nghiệp vụ. SDK có thể tiếp tục sử dụng cấu hình gần nhất khi kết nối tới máy chủ tạm thời bị gián đoạn, qua đó ưu tiên tính sẵn sàng của ứng dụng [8].

Đối với ứng dụng chạy phía trình duyệt hoặc thiết bị di động, Unleash có thể sử dụng một thành phần trung gian như Unleash Proxy hoặc Unleash Edge. Thành phần này giới hạn tập flag được cung cấp cho client, thực hiện một phần xử lý ở phía máy chủ và tránh để lộ thông tin xác thực có quyền truy cập trực tiếp vào Unleash Server.

Khả năng tự triển khai là một đặc điểm quan trọng của Unleash. Tổ chức có thể vận hành nền tảng trong hạ tầng nội bộ để chủ động kiểm soát dữ liệu, mạng và quy trình triển khai. Đổi lại, đội vận hành phải tự chịu trách nhiệm về cơ sở dữ liệu, sao lưu, cập nhật phiên bản, giám sát và khả năng mở rộng.

Activation strategy và evaluation context

Trong Unleash, activation strategy mô tả điều kiện mà theo đó một feature flag được kích hoạt. Một flag có thể có nhiều strategy trong cùng environment. Các strategy thường được kết hợp theo quan hệ OR, nghĩa là flag được bật nếu có ít nhất một strategy trả về kết quả phù hợp.

Strategy mặc định cho phép bật flag đối với toàn bộ context. Ngoài ra, Unleash hỗ trợ các chiến lược phổ biến như bật cho một danh sách người dùng, rollout dần theo tỷ lệ, giới hạn theo địa chỉ mạng hoặc dựa trên các thuộc tính tùy chỉnh. Các strategy có thể được bổ sung constraint để giới hạn phạm vi áp dụng theo những thuộc tính cụ thể của context [2].

Evaluation context trong Unleash chứa thông tin mô tả đối tượng đang yêu cầu đánh giá flag. Các trường phổ biến gồm `userId`, `sessionId`, `remoteAddress` và tập `properties` tùy chỉnh. Phần `properties` có thể chứa các thuộc tính như vai trò, organization ID, quốc gia, gói dịch vụ hoặc phiên bản ứng dụng.

Ví dụ, một activation strategy có thể chỉ áp dụng khi thuộc tính `role` có giá trị `beta-customer`. Một strategy khác có thể giới hạn rollout cho những context có `organizationId` thuộc một tập được lựa chọn. Mô hình này tương đồng với khái niệm evaluation context trong các đặc tả Feature Flag chung, trong đó context cung cấp dữ liệu đầu vào để rule engine đưa ra quyết định [6].

Đối với gradual rollout, Unleash sử dụng một giá trị ổn định từ context, chẳng hạn như `userId` hoặc `sessionId`, kết hợp với thông tin của flag để ánh xạ đối tượng vào một bucket. Bucket được so sánh với tỷ lệ rollout đã cấu hình. Nhờ phép ánh xạ có tính xác định, cùng một người dùng thường tiếp tục nhận cùng một kết quả khi tỷ lệ rollout chưa thay đổi.

Việc lựa chọn trường dùng cho rollout có ảnh hưởng trực tiếp đến tính ổn định của kết quả. Nếu sử dụng `userId`, người dùng đã đăng nhập có thể nhận cùng một quyết định trên nhiều phiên làm việc. Nếu sử dụng `sessionId`, quyết định có thể thay đổi giữa các phiên. Khi context không cung cấp giá trị cần thiết, SDK phải sử dụng giá trị dự phòng hoặc trả về trạng thái mặc định theo cấu hình của ứng dụng.

Unleash còn hỗ trợ variant để phân phối nhiều giá trị khác nhau thay vì chỉ trả về trạng thái Boolean. Mỗi variant có thể được gán trọng số, payload và cơ chế phân phối ổn định. Khả năng này phù hợp với thử nghiệm nhiều phiên bản giao diện hoặc cấu hình nghiệp vụ khác nhau.

SDK, audit log và quản trị

Unleash cung cấp SDK cho nhiều ngôn ngữ và nền tảng phổ biến. SDK chịu trách nhiệm đồng bộ cấu hình flag, duy trì bộ nhớ đệm cục bộ, đánh giá activation strategy và trả về kết quả cho ứng dụng. Giao diện sử dụng cơ bản thường gồm flag key, evaluation context và giá trị mặc định.

Giá trị mặc định là thành phần cần thiết để bảo đảm ứng dụng có hành vi xác định khi flag không tồn tại, SDK chưa nhận được cấu hình hoặc quá trình đánh giá gặp lỗi. Theo các thực hành được Unleash khuyến nghị, hệ thống Feature Flag không nên trở thành phụ thuộc đồng bộ bắt buộc đối với từng request nghiệp vụ [8].

SDK có thể gửi các metric tổng hợp về số lần flag được đánh giá là bật hoặc tắt. Unleash Server sử dụng dữ liệu này để hiển thị mức độ sử dụng flag, hỗ trợ phát hiện các flag không còn được đánh giá và cung cấp thông tin cho quá trình quản lý vòng đời.

Unleash ghi nhận các sự kiện thay đổi đối với project, environment, feature flag và activation strategy. Lịch sử này cho phép xác định người thực hiện, thời điểm và loại thao tác đã xảy ra. Audit log hỗ trợ kiểm tra trách nhiệm, điều tra lỗi cấu hình và theo dõi quá trình thay đổi trạng thái flag.

Về quản trị, Unleash hỗ trợ phân chia tài nguyên theo project và environment. Tùy theo phiên bản triển khai, nền tảng có thể cung cấp các cơ chế quản lý người dùng, nhóm, vai trò và quyền truy cập. Quyền thay đổi cấu hình production có thể được giới hạn cho một nhóm nhỏ hơn so với quyền chỉnh sửa development hoặc staging.

Unleash cũng nhấn mạnh việc quản lý vòng đời feature flag. Flag cần có mục đích, người sở hữu, trạng thái và thời điểm dự kiến loại bỏ. Các flag tạm thời nên được dọn dẹp sau khi quá trình rollout hoàn tất để tránh làm tăng số nhánh xử lý và hình thành feature flag debt [8].

Ngoài giao diện quản trị, Unleash cung cấp API để tự động hóa việc tạo flag, thay đổi strategy và tích hợp với quy trình CI/CD. Tuy nhiên, các thao tác quản trị tự động vẫn cần cơ chế xác thực, phân quyền và kiểm soát thay đổi nhằm hạn chế cấu hình sai trên production.

Ưu điểm và hạn chế

Ưu điểm nổi bật của Unleash là mã nguồn mở và hỗ trợ tự triển khai. Đặc điểm này phù hợp với các tổ chức cần kiểm soát dữ liệu, hạ tầng và quy trình vận hành, hoặc không muốn phụ

thuộc hoàn toàn vào một dịch vụ Feature Flag bên ngoài.

Mô hình đánh giá tại SDK giúp giảm độ trễ và duy trì khả năng hoạt động khi kết nối tới máy chủ quản lý bị gián đoạn. Activation strategy và constraint cung cấp cơ chế targeting tương đối linh hoạt, trong khi gradual rollout cho phép phát hành tính năng từng bước với kết quả ổn định giữa các lần đánh giá.

Cách tổ chức theo project, environment và strategy rõ ràng, phù hợp để mở rộng từ các trường hợp bật hoặc tắt đơn giản đến targeting theo thuộc tính và phân phối variant. Việc hỗ trợ SDK đa nền tảng, metric đánh giá, lịch sử thay đổi và quản lý vòng đời giúp Unleash đáp ứng phần lớn yêu cầu của một hệ thống Feature Flag thực tế.

Tuy nhiên, khi lựa chọn tự triển khai, tổ chức phải vận hành thêm một hệ thống có yêu cầu về tính sẵn sàng, bảo mật, sao lưu và giám sát. Việc nâng cấp Unleash Server, SDK và các thành phần trung gian cũng cần được quản lý để tránh lỗi tương thích.

Một số khả năng quản trị nâng cao có thể phụ thuộc vào phiên bản hoặc gói triển khai. So với các nền tảng thương mại được quản lý hoàn toàn, đội phát triển có thể cần tự xây dựng thêm quy trình phê duyệt, tích hợp giám sát hoặc cơ chế quản trị phù hợp với tổ chức.

Ngoài ra, tính linh hoạt của activation strategy có thể làm cấu hình trở nên khó kiểm soát khi số lượng strategy và constraint tăng cao. Nếu không có quy ước về thứ tự, phạm vi, người sở hữu và thời hạn sử dụng, hệ thống vẫn có nguy cơ tích lũy cấu hình cũ và feature flag debt.

Đối với MiniProject, Unleash là mô hình tham khảo phù hợp cho kiến trúc tách biệt giữa control plane và SDK, evaluation context, activation strategy, rollout ổn định, audit log và quản lý theo environment. Tuy nhiên, MiniProject triển khai rule engine đơn giản hơn với thứ tự đánh giá tường minh nhằm bảo đảm kết quả dễ giải thích, dễ kiểm thử và phù hợp với phạm vi trình diễn.

2.2.3 So sánh các giải pháp hiện có

LaunchDarkly và Unleash đều cung cấp các thành phần cốt lõi của một nền tảng Feature Flag, bao gồm quản lý flag theo môi trường, targeting dựa trên context, rollout theo tỷ lệ, SDK tích hợp với ứng dụng và lịch sử thay đổi cấu hình. Tuy nhiên, hai nền tảng khác nhau về mô hình triển khai, cách biểu diễn luật đánh giá, mức độ hoàn thiện của công cụ quản trị và trách nhiệm vận hành hệ thống.

So sánh mô hình triển khai

LaunchDarkly chủ yếu được cung cấp dưới dạng dịch vụ Feature Flag được quản lý. Nhà cung cấp chịu trách nhiệm vận hành hạ tầng, lưu trữ cấu hình, phân phối dữ liệu flag và duy trì tính sẵn sàng của control plane. Ứng dụng kết nối với nền tảng thông qua SDK và thông tin xác thực tương ứng với từng môi trường [7].

Mô hình này giúp tổ chức giảm khối lượng công việc liên quan đến triển khai, giám sát, sao lưu và nâng cấp hệ thống Feature Flag. Đội phát triển có thể tập trung vào cấu hình phát hành và tích hợp flag vào ứng dụng. Tuy nhiên, hệ thống phụ thuộc nhiều hơn vào hạ tầng, chính sách dữ liệu, cơ chế định giá và khả năng cung cấp dịch vụ của nhà cung cấp.

Unleash cung cấp cả dịch vụ được quản lý và phiên bản mã nguồn mở có thể tự triển khai. Trong mô hình self-hosted, tổ chức tự vận hành Unleash Server, cơ sở dữ liệu và các thành phần phân phối cấu hình trên hạ tầng riêng [2].

Khả năng tự triển khai giúp tổ chức chủ động kiểm soát dữ liệu, mạng nội bộ, chính sách bảo mật và vị trí đặt hệ thống. Đổi lại, đội vận hành phải chịu trách nhiệm về tính sẵn sàng, sao lưu, cập nhật phiên bản, giám sát và khả năng mở rộng của nền tảng.

Cả hai giải pháp đều phân tách control plane khỏi quá trình đánh giá flag trong ứng dụng. Control plane chịu trách nhiệm quản lý cấu hình, trong khi SDK tải cấu hình và thực hiện evaluation cục bộ. Cách tiếp cận này hạn chế việc mỗi lần kiểm tra flag đều phải gửi một yêu cầu mạng đồng bộ tới máy chủ quản lý.

Bảng 2.3: So sánh mô hình triển khai của LaunchDarkly và Unleash

Tiêu chí	**LaunchDarkly**	**Unleash**
Hình thức cung cấp	Chủ yếu là dịch vụ được quản lý	Dịch vụ được quản lý hoặc tự triển khai
Trách nhiệm vận hành	Do nhà cung cấp đảm nhiệm	Do nhà cung cấp hoặc tổ chức sử dụng đảm nhiệm
Kiểm soát hạ tầng	Hạn chế hơn do phụ thuộc nền tảng dịch vụ	Cao khi triển khai trên hạ tầng riêng
Kiểm soát dữ liệu	Phụ thuộc vào chính sách của nhà cung cấp	Có thể kiểm soát trực tiếp khi self-hosted
Chi phí vận hành nội bộ	Tương đối thấp	Cao hơn khi tự triển khai
Mức độ phụ thuộc nhà cung cấp	Tương đối cao	Thấp hơn khi sử dụng phiên bản mã nguồn mở

So sánh khả năng targeting

LaunchDarkly biểu diễn logic targeting bằng các rule có thứ tự. Mỗi rule gồm một hoặc nhiều điều kiện dựa trên thuộc tính của evaluation context. Các rule được đánh giá lần lượt từ trên xuống dưới và rule đầu tiên thỏa mãn sẽ quyết định variation được trả về [4].

Thứ tự rule cho phép người quản trị biểu diễn rõ ràng mức độ ưu tiên giữa các trường hợp. Ví dụ, một rule bật tính năng cho nhóm thử nghiệm nội bộ có thể được đặt trước rule rollout theo tỷ lệ cho người dùng thông thường. Nếu không có rule nào phù hợp, hệ thống sử dụng default rule của flag.

LaunchDarkly còn hỗ trợ targeting trực tiếp tới context cụ thể và sử dụng segment để tái sử dụng cùng một tập đối tượng hoặc điều kiện trong nhiều flag. Evaluation context có thể biểu diễn người dùng, tổ chức, thiết bị hoặc một thực thể nghiệp vụ khác.

Unleash sử dụng activation strategy để mô tả điều kiện kích hoạt flag. Một flag có thể có nhiều strategy trong cùng một môi trường. Các strategy thường được kết hợp theo quan hệ OR, nghĩa là flag được bật nếu có ít nhất một strategy phù hợp [2].

Mỗi activation strategy có thể đi kèm các constraint để giới hạn phạm vi áp dụng theo thuộc tính của context. Ví dụ, gradual rollout có thể chỉ áp dụng đối với người dùng có vai trò `beta-customer`, thuộc một tổ chức cụ thể hoặc đang sử dụng một phiên bản ứng dụng nhất định.

So với activation strategy của Unleash, ordered rule của LaunchDarkly thể hiện thứ tự ưu tiên trực tiếp hơn. Mô hình của Unleash có cấu trúc linh hoạt và thuận tiện cho việc kết hợp nhiều chiến lược kích hoạt, nhưng kết quả có thể khó giải thích hơn khi nhiều strategy đồng thời thỏa mãn.

Cả hai nền tảng đều hỗ trợ rollout theo phần trăm bằng cơ chế phân nhóm có tính xác định. Một thuộc tính ổn định của context được sử dụng để ánh xạ đối tượng vào một bucket. Cùng một đối tượng thường tiếp tục nhận cùng một kết quả nếu khóa targeting và cấu hình flag không thay đổi [5].

Cơ chế phân phối ổn định giúp hạn chế tình trạng người dùng liên tục chuyển đổi giữa phiên bản cũ và phiên bản mới. Tuy nhiên, độ ổn định phụ thuộc vào việc ứng dụng cung cấp một targeting key nhất quán trong evaluation context [6].

Bảng 2.4: So sánh khả năng targeting và rollout

Tiêu chí	**LaunchDarkly**	**Unleash**
Đơn vị cấu hình	Targeting rule	Activation strategy và constraint
Cách kết hợp	Đánh giá theo thứ tự; rule đầu tiên phù hợp được sử dụng	Nhiều strategy thường được kết hợp theo quan hệ OR
Targeting trực tiếp	Context, thuộc tính và segment	User ID, context properties, segment và constraint
Thứ tự ưu tiên	Được thể hiện trực tiếp qua thứ tự rule	Không phải thành phần trung tâm của mô hình strategy
Rollout theo tỷ lệ	Percentage rollout với phân phối ổn định	Gradual rollout với phép phân nhóm ổn định
Nhiều biến thể	Hỗ trợ nhiều variation và kiểu dữ liệu	Hỗ trợ variant, trọng số và payload
Khả năng giải thích	Tương đối rõ do rule có thứ tự	Phụ thuộc vào số lượng strategy và constraint

So sánh SDK và khả năng tích hợp

LaunchDarkly và Unleash đều cung cấp SDK cho nhiều ngôn ngữ và nền tảng. SDK chịu trách nhiệm nhận cấu hình flag, duy trì trạng thái cục bộ, tiếp nhận evaluation context và trả về kết quả đánh giá cho ứng dụng.

Đối với các SDK phía máy chủ, quá trình evaluation chủ yếu được thực hiện trong tiến trình ứng dụng. Cách tiếp cận này giảm độ trễ và tránh việc mỗi request nghiệp vụ phải phụ thuộc vào một lời gọi mạng tới control plane.

LaunchDarkly cung cấp hệ sinh thái SDK rộng cho ứng dụng backend, trình duyệt, thiết bị di động và các môi trường thực thi khác. SDK hỗ trợ đánh giá flag Boolean hoặc đa biến, nhận cập nhật cấu hình và gửi event phục vụ thống kê hoặc phân tích việc sử dụng flag.

Unleash SDK thường tải cấu hình từ Unleash Server hoặc một thành phần phân phối trung gian, sau đó áp dụng activation strategy tại ứng dụng. Đối với ứng dụng phía trình duyệt hoặc thiết bị di động, một lớp trung gian có thể được sử dụng để tránh cung cấp trực tiếp thông tin xác thực có quyền truy cập vào máy chủ quản trị.

Cả hai nền tảng đều khuyến nghị ứng dụng xác định giá trị mặc định khi đánh giá flag. Giá trị mặc định được sử dụng khi flag không tồn tại, SDK chưa có cấu hình hoặc quá trình evaluation gặp lỗi. Đây là cơ chế fail-safe giúp giảm nguy cơ hệ thống Feature Flag trở thành điểm lỗi bắt buộc trong luồng nghiệp vụ [8].

Ngoài SDK, cả LaunchDarkly và Unleash đều cung cấp API quản trị để tích hợp với quy trình CI/CD hoặc công cụ nội bộ. API quản trị có thể được sử dụng để tạo flag, cập nhật cấu hình hoặc quản lý môi trường. Quyền quản trị cần được tách biệt với quyền chỉ dùng để đánh giá flag nhằm hạn chế việc ứng dụng có khả năng thay đổi cấu hình production.

Bảng 2.5: So sánh SDK và khả năng tích hợp

Tiêu chí	**LaunchDarkly**	**Unleash**
SDK đa nền tảng	Hệ sinh thái SDK rộng và được quản lý tập trung	Hỗ trợ các ngôn ngữ và nền tảng phổ biến
Vị trí đánh giá	Chủ yếu tại SDK	Chủ yếu tại SDK
Đồng bộ cấu hình	Nhận cập nhật cấu hình từ dịch vụ LaunchDarkly	Lấy cấu hình từ server hoặc lớp phân phối trung gian
Frontend và mobile	Có SDK chuyên biệt	Thường sử dụng proxy, edge hoặc API trung gian
Giá trị mặc định	Được cung cấp khi gọi SDK	Được cung cấp khi gọi SDK
API quản trị	Có	Có
Khả năng tùy chỉnh	Phụ thuộc vào chức năng do nền tảng cung cấp	Cao hơn khi sử dụng phiên bản mã nguồn mở

So sánh audit log và phân quyền

Audit log là thành phần quan trọng trong hệ thống Feature Flag vì thay đổi cấu hình có thể ảnh hưởng trực tiếp đến hành vi của ứng dụng mà không cần triển khai lại mã nguồn. Log cần cho phép xác định actor, thời điểm, hành động, tài nguyên bị thay đổi và nội dung thay đổi [12].

LaunchDarkly cung cấp lịch sử thay đổi và các cơ chế quản trị thành viên, nhóm và quyền truy cập. Quyền có thể được giới hạn theo project, environment, tài nguyên hoặc loại thao tác. Nhờ đó, tổ chức có thể cho phép nhiều thành viên chỉnh sửa cấu hình development nhưng giới hạn quyền thay đổi production cho một nhóm nhỏ hơn.

Khả năng phân quyền chi tiết phù hợp với các tổ chức có nhiều nhóm phát triển, nhiều sản phẩm và yêu cầu kiểm soát thay đổi chặt chẽ. Lịch sử cấu hình còn hỗ trợ điều tra nguyên nhân khi một thay đổi flag gây ra sự cố.

Unleash cũng ghi nhận các sự kiện liên quan đến project, environment, feature flag và activation strategy. Trong mô hình self-hosted, tổ chức có quyền kiểm soát trực tiếp đối với cơ sở dữ liệu chứa lịch sử thay đổi và có thể tích hợp dữ liệu này với hệ thống giám sát hoặc lưu trữ log nội bộ.

Mức độ phân quyền và quản trị của Unleash có thể phụ thuộc vào phiên bản hoặc mô hình triển khai. Các tổ chức sử dụng phiên bản cơ bản có thể phải tự bổ sung quy trình phê duyệt, tích hợp danh tính hoặc các chính sách truy cập chi tiết hơn.

LaunchDarkly có ưu thế về trải nghiệm quản trị và phân quyền được tích hợp sẵn. Unleash có ưu thế về quyền kiểm soát dữ liệu và khả năng tùy chỉnh khi tự triển khai. Trong cả hai trường hợp, audit log không nên ghi token, thông tin xác thực hoặc dữ liệu cá nhân không cần thiết.

Bảng 2.6: So sánh audit log và phân quyền

Tiêu chí	**LaunchDarkly**	**Unleash**
Lịch sử thay đổi	Có lịch sử thay đổi cấu hình và tài nguyên	Có event log cho các thay đổi trong hệ thống
Quản lý thành viên và nhóm	Được tích hợp trong nền tảng	Có hỗ trợ nhưng phụ thuộc phiên bản triển khai
Phân quyền theo tài nguyên	Có thể cấu hình tương đối chi tiết	Có thể cần cấu hình hoặc mở rộng thêm
Phân quyền theo môi trường	Hỗ trợ kiểm soát riêng giữa các environment	Phụ thuộc vào phiên bản và chính sách triển khai
Kiểm soát dữ liệu log	Do dịch vụ quản lý theo chính sách nền tảng	Tổ chức kiểm soát trực tiếp khi self-hosted
Khả năng tùy chỉnh	Giới hạn trong cơ chế nền tảng cung cấp	Cao hơn khi tổ chức quản lý mã nguồn và hạ tầng

So sánh khả năng mở rộng

Khả năng mở rộng của nền tảng Feature Flag không chỉ phụ thuộc vào số lượng flag mà còn phụ thuộc vào số lượng ứng dụng, SDK instance, environment, targeting rule và số lần evaluation.

LaunchDarkly có lợi thế về khả năng mở rộng dưới dạng dịch vụ được quản lý. Việc vận hành control plane, phân phối cấu hình và mở rộng hạ tầng do nhà cung cấp đảm nhiệm. Mô hình này phù hợp với tổ chức muốn mở rộng số lượng ứng dụng mà không phải xây dựng đội vận hành riêng cho nền tảng Feature Flag.

Unleash cho phép tổ chức tự thiết kế kiến trúc mở rộng. Nhiều instance máy chủ hoặc thành phần phân phối có thể được triển khai để phục vụ các nhóm ứng dụng khác nhau. Tuy nhiên, hiệu quả mở rộng phụ thuộc vào cơ sở dữ liệu, kiến trúc mạng, chiến lược cache và năng lực vận hành của tổ chức.

Cả hai nền tảng đều giảm tải cho control plane bằng cách đánh giá flag tại SDK. Số lần evaluation trong ứng dụng không nhất thiết làm tăng tương ứng số request tới máy chủ quản

lý. Đây là đặc điểm quan trọng để hệ thống có thể phục vụ lưu lượng lớn mà không đưa control plane vào đường xử lý đồng bộ của mỗi request.

Việc lưu cấu hình cục bộ và đồng bộ bất đồng bộ tạo ra sự đánh đổi giữa tính sẵn sàng và tính nhất quán. Sau khi cấu hình được thay đổi, các SDK có thể nhận phiên bản mới tại những thời điểm khác nhau. Trong một khoảng thời gian ngắn, các instance ứng dụng có thể đánh giá cùng một flag dựa trên các phiên bản cấu hình khác nhau [8].

LaunchDarkly phù hợp với tổ chức ưu tiên khả năng mở rộng nhanh và giảm trách nhiệm vận hành. Unleash phù hợp với tổ chức cần kiểm soát kiến trúc, dữ liệu và chi phí hạ tầng, đồng thời có đủ năng lực để tự triển khai và giám sát hệ thống.

Bảng 2.7: Tổng hợp so sánh LaunchDarkly và Unleash

Tiêu chí	**LaunchDarkly**	**Unleash**
Mô hình triển khai	Dịch vụ được quản lý	Dịch vụ được quản lý hoặc self-hosted
Mô hình targeting	Ordered targeting rule	Activation strategy và constraint
Rollout theo tỷ lệ	Hỗ trợ percentage rollout ổn định	Hỗ trợ gradual rollout ổn định
Đánh giá tại SDK	Có	Có
SDK và tích hợp	Hệ sinh thái lớn, nhiều tích hợp sẵn có	Linh hoạt và có thể tùy chỉnh khi self-hosted
Audit log	Được tích hợp đầy đủ trong nền tảng	Có event log; mức độ quản trị phụ thuộc phiên bản
Phân quyền	Chi tiết theo thành viên, nhóm và tài nguyên	Linh hoạt nhưng có thể cần cấu hình bổ sung
Kiểm soát dữ liệu	Phụ thuộc vào chính sách của nhà cung cấp	Cao khi triển khai trên hạ tầng riêng
Gánh nặng vận hành	Thấp hơn	Cao hơn trong mô hình self-hosted
Khả năng mở rộng	Do nhà cung cấp quản lý	Phụ thuộc vào kiến trúc và năng lực vận hành
Đối tượng phù hợp	Tổ chức cần nền tảng hoàn chỉnh và giảm công việc vận hành	Tổ chức cần tự chủ hạ tầng, dữ liệu và khả năng tùy chỉnh

Kết quả so sánh cho thấy không có giải pháp phù hợp tuyệt đối cho mọi hệ thống. LaunchDarkly ưu tiên trải nghiệm quản trị, hệ sinh thái SDK và khả năng vận hành dưới dạng dịch vụ. Unleash ưu tiên tính mở, khả năng tự triển khai và quyền kiểm soát đối với hạ tầng.

MiniProject tiếp thu mô hình project, environment và feature flag từ cả hai nền tảng. Rule engine sử dụng thứ tự đánh giá tường minh tương tự targeting rule của LaunchDarkly, đồng

thời áp dụng evaluation context và rollout ổn định tương tự các activation strategy của Unleash. Hệ thống giới hạn các loại rule ở user allowlist, role targeting và percentage rollout nhằm bảo đảm logic dễ hiểu, kết quả dễ giải thích và phù hợp với phạm vi triển khai.

MiniProject cũng triển khai audit log, phân quyền, cache, SDK JavaScript và group kill switch để mô phỏng các khả năng quản trị và mở rộng của một nền tảng thực tế. Tuy nhiên, kiến trúc được đơn giản hóa để tập trung vào các nguyên lý cốt lõi của Feature Flag thay vì tái tạo toàn bộ chức năng của một sản phẩm thương mại.

2.2.4 Định hướng của hệ thống đề xuất

Từ kết quả khảo sát LaunchDarkly và Unleash, hệ thống đề xuất lựa chọn các thành phần cốt lõi cần thiết để minh họa đầy đủ quá trình quản lý và đánh giá Feature Flag. Mục tiêu của MiniProject không phải là tái tạo toàn bộ một nền tảng thương mại, mà là xây dựng một hệ thống có kiến trúc rõ ràng, logic đánh giá dễ giải thích và có thể chứng minh trực tiếp giá trị của Feature Flag thông qua ứng dụng demo.

Hệ thống được định hướng theo nguyên tắc tách biệt giữa lớp quản trị cấu hình và lớp sử dụng flag trong ứng dụng. Dashboard quản trị cho phép tạo và thay đổi cấu hình, trong khi evaluation API tiếp nhận context và trả về quyết định tại thời điểm chạy. Cách tổ chức này phản ánh nguyên lý tách quá trình triển khai mã nguồn khỏi quá trình phát hành tính năng [1].

Các chức năng được kế thừa

Hệ thống kế thừa mô hình phân cấp tài nguyên gồm project, environment và feature flag từ các nền tảng được khảo sát. Project đại diện cho một sản phẩm hoặc hệ thống phần mềm. Mỗi project có thể có nhiều environment như development, staging và production. Một feature flag có cùng định danh trong phạm vi project nhưng có cấu hình hoạt động độc lập theo từng environment [7, 2].

Cấu hình của flag được tách thành trạng thái vòng đời, trạng thái hoạt động và chế độ phục vụ. Trạng thái vòng đời xác định flag còn được sử dụng hay đã được lưu trữ. Trạng thái hoạt động cho biết cấu hình tại một environment có được phép tham gia đánh giá hay không. Chế độ phục vụ xác định flag được bật toàn cục hay được quyết định thông qua các targeting rule.

Evaluation API tiếp nhận context mô tả đối tượng cần đánh giá. Context có thể chứa targeting key, mã người dùng, danh sách vai trò và các thuộc tính mở rộng. Mô hình này kế thừa khái

niệm evaluation context, trong đó dữ liệu về người dùng hoặc thực thể nghiệp vụ được sử dụng làm đầu vào cho rule engine [6].

Các chiến lược phát hành được lựa chọn gồm:

- bật hoặc tắt tính năng toàn cục;
- bật tính năng cho danh sách người dùng cụ thể;
- bật tính năng theo vai trò;
- rollout theo phần trăm đối tượng;
- tắt nhanh một flag hoặc một nhóm flag bằng kill switch.

Đối với percentage rollout, hệ thống sử dụng phép băm ổn định để ánh xạ targeting key vào một bucket xác định. Nhờ đó, cùng một đối tượng thường nhận cùng một kết quả giữa các lần đánh giá nếu cấu hình và targeting key không thay đổi. Cơ chế này được kế thừa từ cách tiếp cận stable assignment của các nền tảng Feature Flag hiện có [5].

Hệ thống cũng kế thừa các cơ chế hỗ trợ quản trị gồm audit log, phân quyền, cache kết quả đánh giá, thống kê số lần evaluation và SDK JavaScript đơn giản. Audit log ghi nhận actor, hành động, thời điểm, đối tượng bị tác động và dữ liệu trước hoặc sau thay đổi. Việc lưu lịch sử cấu hình hỗ trợ kiểm tra trách nhiệm và điều tra sự cố khi một thay đổi flag ảnh hưởng đến ứng dụng [12].

Bảng 2.8: Các chức năng được kế thừa trong hệ thống đề xuất

Nhóm chức năng	**Chức năng được kế thừa**	**Mục đích**
Quản lý tài nguyên	Project, environment, feature flag và flag group	Tổ chức cấu hình theo sản phẩm, môi trường và nhóm chức năng
Evaluation context	Targeting key, user ID, role và thuộc tính mở rộng	Cung cấp dữ liệu đầu vào cho rule engine
Targeting	User allowlist, role targeting và percentage rollout	Giới hạn phạm vi phát hành tính năng
Kiểm soát khẩn cấp	Kill switch cho flag và nhóm flag	Tắt nhanh tính năng khi xảy ra sự cố
Tích hợp ứng dụng	Evaluation API và SDK JavaScript	Cho phép ứng dụng lấy quyết định flag tại thời điểm chạy
Quản trị	Audit log, phân quyền và lịch sử cấu hình	Kiểm soát thay đổi và xác định trách nhiệm
Hiệu năng	Cache kết quả đánh giá	Giảm số lần truy vấn dữ liệu và độ trễ evaluation
Quan sát hệ thống	Thống kê số lần đánh giá flag	Theo dõi mức độ sử dụng của từng flag

Các chức năng được đơn giản hóa

So với LaunchDarkly và Unleash, hệ thống đề xuất giới hạn số loại rule để tập trung vào các trường hợp sử dụng chính của MiniProject. Rule engine chỉ hỗ trợ user allowlist, role targeting và percentage rollout. Các toán tử targeting phức tạp như biểu thức chính quy, so sánh phiên bản ngữ nghĩa, điều kiện thời gian hoặc segment lồng nhau không nằm trong phạm vi triển khai.

Kết quả đánh giá chủ yếu tập trung vào trạng thái bật hoặc tắt. Hệ thống có thể trả về giá trị biểu diễn nhánh `on` hoặc `off`, nhưng không xây dựng cơ chế quản lý nhiều variation và payload phức tạp như các nền tảng thương mại.

SDK JavaScript được thiết kế như một lớp bao đơn giản cho evaluation API. SDK chịu trách nhiệm chuẩn hóa request, truyền context và xử lý giá trị dự phòng. Hệ thống không triển khai đầy đủ nhiều SDK theo ngôn ngữ, cơ chế streaming cấu hình hoặc evaluation cục bộ hoàn toàn trong tiến trình ứng dụng.

Phân quyền được giới hạn ở một số vai trò quản trị cơ bản như administrator, developer và viewer. Các chức năng nâng cao như phân quyền theo biểu thức, quyền tùy chỉnh theo từng thuộc tính tài nguyên, đăng nhập một lần hoặc tích hợp hệ thống danh tính doanh nghiệp

không được triển khai.

Audit log chỉ tập trung vào các thao tác làm thay đổi dữ liệu quản trị. Hệ thống không xây dựng nền tảng phân tích log chuyên biệt, cơ chế lưu trữ dài hạn hoặc tích hợp với hệ thống quản lý sự kiện bảo mật.

Cơ chế cache được triển khai ở mức phù hợp với một ứng dụng đơn lẻ hoặc môi trường demo. Hệ thống không giải quyết toàn bộ các vấn đề về cache phân tán, đồng bộ đa vùng hoặc invalidation giữa số lượng lớn instance như trong kiến trúc production quy mô lớn [13].

Bảng 2.9: Các chức năng được đơn giản hóa trong MiniProject

Thành phần	**Nền tảng hoàn chỉnh**	**MiniProject**
Targeting rule	Nhiều toán tử, segment và điều kiện kết hợp phức tạp	Ba loại rule chính với tham số giới hạn
Variation	Nhiều biến thể và nhiều kiểu payload	Chủ yếu sử dụng kết quả bật hoặc tắt
SDK	Nhiều ngôn ngữ, đồng bộ cấu hình và evaluation cục bộ	SDK JavaScript gọi evaluation API
Phân quyền	Quyền chi tiết theo nhóm, tài nguyên và thao tác	Một số vai trò quản trị cố định
Audit log	Lịch sử đầy đủ và tích hợp quản trị doanh nghiệp	Log bất biến cho các thao tác thay đổi chính
Cache	Cache phân tán và cơ chế đồng bộ quy mô lớn	Cache ở mức ứng dụng với invalidation cơ bản
Thử nghiệm	Experimentation và phân tích thống kê chuyên sâu	Không triển khai nền tảng thử nghiệm hoàn chỉnh

Điểm khác biệt của MiniProject

Điểm khác biệt chính của MiniProject là ưu tiên tính minh bạch của quá trình đánh giá. Rule engine sử dụng thứ tự ưu tiên cố định và trả về reason code tương ứng với nhánh quyết định. Thứ tự đánh giá tổng quát gồm:

1. flag đã được lưu trữ;
2. cấu hình flag bị vô hiệu hóa;
3. group kill switch đang được kích hoạt;
4. kill switch của flag đang được kích hoạt;
5. flag được bật toàn cục;
6. người dùng thuộc user allowlist;

7. vai trò của người dùng phù hợp;
8. người dùng thuộc bucket percentage rollout;
9. không có điều kiện nào phù hợp và sử dụng trạng thái mặc định tắt.

Mỗi kết quả evaluation không chỉ trả về trạng thái bật hoặc tắt mà còn cung cấp reason code như `FLAG_ARCHIVED`, `FLAG_DISABLED`, `GROUP_KILL_SWITCH`, `GLOBAL_ON`, `USER_MATCH`, `ROLE_MATCH`, `PERCENTAGE_MATCH` hoặc `DEFAULT_OFF`. Khi một rule cụ thể được sử dụng, kết quả có thể chứa thêm định danh của rule đã khớp.

Cách thiết kế này giúp người sử dụng giải thích được vì sao một đối tượng nhận hoặc không nhận tính năng. Đây là yêu cầu quan trọng đối với quá trình kiểm thử, trình diễn và điều tra lỗi cấu hình. Mô hình kết quả chi tiết cũng tương đồng với định hướng của Flag Evaluation API, trong đó kết quả có thể bao gồm giá trị, lý do đánh giá và thông tin lỗi [10].

MiniProject tách rõ ba khái niệm vòng đời, trạng thái cấu hình và chế độ phục vụ. Một flag đã được lưu trữ không còn được đánh giá, một cấu hình bị vô hiệu hóa không tham gia phục vụ tại environment tương ứng, còn chế độ `GLOBAL_ON` cho phép bỏ qua các targeting rule. Sự phân tách này giúp hạn chế việc sử dụng một trường trạng thái duy nhất cho nhiều mục đích khác nhau.

Group kill switch cho phép tập hợp nhiều feature flag có liên quan và vô hiệu hóa toàn bộ nhóm bằng một thao tác. Cơ chế này phù hợp với trường hợp một chức năng nghiệp vụ được cấu thành từ nhiều flag riêng lẻ nhưng cần được tắt đồng thời khi xảy ra sự cố.

Audit log được thiết kế theo hướng append-only. Các bản ghi đã tạo không được cập nhật hoặc xóa thông qua luồng nghiệp vụ thông thường. Mỗi bản ghi lưu snapshot trước và sau thay đổi cùng request ID, giúp liên kết thao tác quản trị với request đã tạo ra thay đổi.

Ngoài dashboard quản trị, MiniProject cung cấp một demo app có dữ liệu người dùng và tổ chức mẫu. Demo app cho phép quan sát trực tiếp sự thay đổi giao diện hoặc hành vi khi trạng thái flag, role targeting, user allowlist hoặc tỷ lệ rollout được cập nhật. Nhờ đó, hệ thống không chỉ trình bày API mà còn chứng minh tác động của Feature Flag đối với ứng dụng tích hợp.

Toàn bộ backend, dashboard, demo app, cơ sở dữ liệu và các dịch vụ phụ trợ được tổ chức để có thể khởi chạy bằng Docker Compose. Cách đóng gói này giúp tái tạo môi trường demo thống nhất và giảm số bước cài đặt thủ công [14].

Giới hạn phạm vi triển khai

MiniProject được xây dựng nhằm phục vụ nghiên cứu, trình diễn và đánh giá kiến trúc nên chưa được xem là một nền tảng Feature Flag sẵn sàng cho production ở quy mô lớn.

Hệ thống chưa triển khai kiến trúc đa vùng, cân bằng tải giữa nhiều cụm, sao chép dữ liệu xuyên khu vực hoặc cơ chế chuyển đổi dự phòng tự động. Khả năng chịu lỗi chủ yếu được xem xét ở mức xử lý lỗi API, fallback và cache cục bộ.

SDK hiện chỉ tập trung vào JavaScript và gọi evaluation API từ xa. Hệ thống chưa cung cấp SDK chuyên biệt cho nhiều ngôn ngữ, chưa đồng bộ toàn bộ cấu hình về client và chưa thực hiện evaluation hoàn toàn cục bộ như LaunchDarkly hoặc Unleash.

Các rule được giới hạn ở ba loại chính và chưa hỗ trợ segment dùng chung, rule lồng nhau, điều kiện thời gian, so sánh phiên bản hoặc biểu thức targeting tùy chỉnh. Hệ thống cũng chưa cung cấp công cụ thiết kế rule trực quan cho các biểu thức phức tạp.

MiniProject chưa triển khai hệ thống experimentation đầy đủ. Percentage rollout chỉ được sử dụng để phân phối tính năng ổn định, không bao gồm kiểm định giả thuyết, đo lường chuyển đổi, phân tích độ tin cậy thống kê hoặc tự động lựa chọn biến thể.

Các chức năng bảo mật doanh nghiệp như đăng nhập một lần, xác thực đa yếu tố, tích hợp LDAP, quản lý khóa bí mật chuyên dụng và quy trình phê duyệt nhiều cấp chưa thuộc phạm vi triển khai. Phân quyền hiện tại chỉ nhằm chứng minh việc giới hạn thao tác quản trị theo vai trò.

Thống kê evaluation được sử dụng để minh họa khả năng quan sát cơ bản. Hệ thống chưa được tối ưu để thu thập khối lượng sự kiện rất lớn, chưa cung cấp pipeline phân tích thời gian thực và chưa tích hợp với nền tảng giám sát chuyên dụng.

Cuối cùng, khả năng mở rộng và hiệu năng mới được kiểm chứng trong môi trường phát triển và demo. Hệ thống chưa trải qua kiểm thử tải ở quy mô production, chưa có cam kết về độ trễ hoặc tính sẵn sàng và chưa thay thế được các nền tảng Feature Flag thương mại hoặc mã nguồn mở đã được vận hành rộng rãi.

2.3 Phân tích bài toán và yêu cầu hệ thống

2.3.1 Các đối tượng sử dụng hệ thống

Hệ thống Feature Flag Platform phục vụ nhiều nhóm người dùng với nhu cầu và phạm vi quyền hạn khác nhau. Việc phân tách rõ các đối tượng sử dụng giúp đơn giản hóa quá trình quản trị, tăng tính an toàn khi thay đổi cấu hình và tạo nền tảng để triển khai cơ chế phân quyền theo vai trò (Role-Based Access Control – RBAC). Trong phạm vi mini project, hệ thống định nghĩa ba nhóm người dùng trên dashboard quản trị và một nhóm ứng dụng khách sử dụng API evaluation để tích hợp Feature Flag vào quá trình thực thi của hệ thống. Cách phân chia này phù hợp với mô hình quản lý Feature Flag của nhiều nền tảng hiện nay như LaunchDarkly và Unleash [7, 2].

Quản trị viên

Quản trị viên (Administrator) là đối tượng có quyền cao nhất trong hệ thống. Người dùng thuộc nhóm này chịu trách nhiệm quản lý toàn bộ tài nguyên và cấu hình Feature Flag.

Các chức năng chính của quản trị viên bao gồm:

- Quản lý project và environment.
- Tạo, chỉnh sửa, lưu trữ hoặc khôi phục Feature Flag.
- Cấu hình trạng thái hoạt động của Feature Flag.
- Thiết lập các rule như bật toàn cục, theo người dùng, theo vai trò và rollout theo phần trăm.
- Quản lý nhóm Feature Flag và Kill Switch.
- Quản lý người dùng và phân quyền.
- Theo dõi audit log và thống kê hoạt động của hệ thống.

Quản trị viên có thể thực hiện mọi thao tác thay đổi cấu hình. Mỗi thay đổi đều được ghi nhận vào audit log nhằm phục vụ truy vết và kiểm toán sau này [12].

Nhà phát triển

Nhà phát triển (Developer) là đối tượng trực tiếp tích hợp Feature Flag vào ứng dụng và quản lý cấu hình của các tính năng trong phạm vi được phân quyền.

Các chức năng chính bao gồm:

- Tạo và chỉnh sửa Feature Flag.
- Cấu hình rule evaluation cho từng Feature Flag.
- Kiểm tra trạng thái Feature Flag.
- Theo dõi lịch sử thay đổi của các Feature Flag thuộc phạm vi phụ trách.
- Tích hợp Evaluation API vào ứng dụng thông qua SDK hoặc lời gọi HTTP.

Nhà phát triển không có quyền quản lý người dùng hoặc thay đổi các thiết lập quản trị của toàn hệ thống. Việc giới hạn quyền giúp giảm nguy cơ thay đổi ngoài ý muốn đối với các cấu hình quan trọng [8].

Người xem

Người xem (Viewer) là nhóm người dùng chỉ có quyền truy cập để quan sát trạng thái hệ thống mà không được phép thay đổi cấu hình.

Các chức năng bao gồm:

- Xem danh sách project và Feature Flag.
- Theo dõi trạng thái hoạt động của Feature Flag.
- Xem các rule đã được cấu hình.
- Xem audit log và thống kê evaluate.

Việc tách riêng vai trò người xem giúp các bên liên quan như quản lý dự án, kiểm thử hoặc vận hành có thể theo dõi hệ thống mà không làm ảnh hưởng đến cấu hình đang hoạt động.

Ứng dụng tích hợp Feature Flag

Bên cạnh người dùng truy cập dashboard quản trị, hệ thống còn phục vụ các ứng dụng khách (Client Application) tích hợp Feature Flag thông qua Evaluation API.

Ứng dụng khách gửi yêu cầu evaluate kèm theo Evaluation Context, bao gồm các thông tin như `targetingKey`, vai trò hoặc các thuộc tính cần thiết. Rule engine sẽ đánh giá các rule theo thứ tự ưu tiên và trả về kết quả bật hoặc tắt của Feature Flag cùng với lý do đánh giá. Mô hình này tương đồng với đặc tả của OpenFeature và các nền tảng Feature Flag hiện đại [6, 10].

Trong mini project, ứng dụng tích hợp được hiện thực dưới dạng một demo web application nhằm minh họa việc thay đổi giao diện hoặc chức năng theo trạng thái của Feature Flag mà không cần triển khai lại hệ thống. Điều này thể hiện đúng mục tiêu tách biệt giữa quá trình triển khai (deploy) và phát hành tính năng (release) trong quá trình phát triển phần mềm [1, 3].

2.3.2 Các ca sử dụng chính

Dựa trên yêu cầu của mini project, hệ thống Feature Flag Platform được xây dựng xoay quanh năm nhóm chức năng chính, bao gồm quản lý tài nguyên, quản lý Feature Flag, cấu hình chiến lược rollout, đánh giá Feature Flag và theo dõi hoạt động của hệ thống. Các ca sử dụng này bao quát toàn bộ vòng đời của một Feature Flag, từ khi được tạo ra, cấu hình, sử dụng trong ứng dụng cho đến khi được lưu trữ hoặc ngừng sử dụng. Đồng thời, các chức năng cũng đáp ứng đầy đủ các yêu cầu bắt buộc và phần lớn các yêu cầu khuyến khích của đề tài [7, 8].

Quản lý project và environment

Project là đơn vị quản lý cao nhất trong hệ thống, dùng để phân tách các ứng dụng hoặc dịch vụ khác nhau. Mỗi project có thể bao gồm nhiều environment như Development, Staging hoặc Production nhằm quản lý cấu hình Feature Flag độc lập giữa các môi trường triển khai.

Các chức năng chính bao gồm:

- Tạo, cập nhật và quản lý project.
- Tạo và quản lý các environment thuộc project.
- Thiết lập environment mặc định.
- Phân tách cấu hình Feature Flag theo từng environment.

Việc tách biệt project và environment giúp các thay đổi được kiểm thử trước khi áp dụng lên môi trường production, phù hợp với quy trình phát triển và phát hành phần mềm hiện đại [3].

Quản lý Feature Flag

Đây là chức năng trung tâm của hệ thống, cho phép quản trị viên hoặc nhà phát triển tạo và quản lý các Feature Flag phục vụ quá trình phát hành tính năng.

Các chức năng chính gồm:

- Tạo Feature Flag mới.
- Cập nhật thông tin Feature Flag.
- Bật hoặc vô hiệu hóa Feature Flag.
- Lưu trữ (archive) hoặc khôi phục Feature Flag.
- Quản lý nhóm Feature Flag và Kill Switch.

Việc sử dụng trạng thái lưu trữ thay cho xóa vật lý giúp bảo toàn lịch sử cấu hình và giảm nguy cơ mất dữ liệu, đồng thời phù hợp với thực tiễn quản lý Feature Flag của các nền tảng thương mại [7, 8].

Cấu hình chiến lược rollout

Sau khi tạo Feature Flag, người quản trị có thể cấu hình cách thức phát hành tính năng thông qua các rule evaluation.

Trong phạm vi mini project, hệ thống hỗ trợ các chiến lược rollout sau:

- Bật hoặc tắt toàn cục.
- Cho phép theo danh sách người dùng.
- Cho phép theo vai trò người dùng.
- Rollout theo phần trăm người dùng.
- Thiết lập thứ tự ưu tiên giữa các rule.

Các rule được đánh giá theo một thứ tự xác định nhằm đảm bảo kết quả nhất quán. Đối với rollout theo phần trăm, hệ thống sử dụng cơ chế băm ổn định (stable hashing) để bảo đảm một người dùng luôn nhận được cùng một kết quả khi cấu hình không thay đổi [4, 5].

Đánh giá Feature Flag

Dây là chức năng được các ứng dụng tích hợp sử dụng nhiều nhất trong quá trình vận hành.

Ứng dụng khách gửi yêu cầu tới Evaluation API kèm theo Evaluation Context. Rule engine sẽ lấy cấu hình tương ứng, đánh giá các rule theo thứ tự ưu tiên và trả về kết quả bao gồm trạng thái của Feature Flag, lý do đánh giá và các thông tin liên quan.

Quá trình evaluate bao gồm các bước chính:

1. Tiếp nhận Evaluation Context.
2. Xác định project, environment và Feature Flag.
3. Kiểm tra trạng thái hoạt động của Feature Flag.
4. Thực hiện đánh giá các rule.
5. Trả về kết quả evaluate cho ứng dụng.

Thiết kế này tương thích với mô hình Evaluation API và Evaluation Context được đề xuất trong OpenFeature Specification [6, 10].

Theo dõi thay đổi và thống kê

Để tăng khả năng quản trị và hỗ trợ truy vết, hệ thống ghi nhận các thao tác thay đổi cấu hình cũng như thống kê hoạt động evaluate của Feature Flag.

Các chức năng bao gồm:

- Ghi nhận audit log cho các thao tác tạo, cập nhật, lưu trữ và thay đổi rule.
- Lưu thông tin người thực hiện và thời điểm thay đổi.
- Theo dõi số lần evaluate của từng Feature Flag.
- Hiển thị thống kê phục vụ việc phân tích mức độ sử dụng.

Audit log giúp đảm bảo khả năng truy vết, hỗ trợ kiểm toán và phân tích nguyên nhân khi xảy ra sự cố, trong khi thống kê evaluate cung cấp cơ sở để đánh giá hiệu quả của từng Feature Flag và lập kế hoạch dọn dẹp các Feature Flag không còn được sử dụng [12, 9].

2.3.3 Yêu cầu chức năng mức MVP

Dựa trên mô tả và yêu cầu của mini project, hệ thống được xây dựng theo mô hình gồm bốn thành phần chính: Backend API, Frontend Dashboard, Demo Application và bộ dữ liệu cùng tài liệu hỗ trợ. Các chức năng trong mức MVP tập trung vào việc chứng minh đầy đủ quy trình quản lý và sử dụng Feature Flag từ phía quản trị cho đến ứng dụng tích hợp, đồng thời đáp ứng các yêu cầu kỹ thuật cơ bản về tính đúng đắn, khả năng mở rộng và dễ triển khai [7, 8].

Yêu cầu đối với Backend API

Backend API là thành phần trung tâm của hệ thống, chịu trách nhiệm quản lý dữ liệu, thực hiện đánh giá Feature Flag và cung cấp giao diện lập trình cho các ứng dụng tích hợp.

Các yêu cầu chức năng chính bao gồm:

- Cung cấp API quản lý project và environment.
- Cung cấp API tạo, cập nhật và quản lý Feature Flag.
- Cung cấp API cấu hình rule cho Feature Flag.
- Hỗ trợ các chiến lược rollout gồm:
  - bật hoặc tắt toàn cục;
  - theo người dùng;
  - theo vai trò;
  - theo phần trăm người dùng.
- Cung cấp Evaluation API để ứng dụng khách kiểm tra trạng thái Feature Flag theo Evaluation Context.
- Ghi nhận audit log cho các thao tác thay đổi cấu hình.
- Thực hiện validation dữ liệu đầu vào.
- Trả về mã lỗi và thông báo phù hợp khi xảy ra lỗi.

Evaluation API cần hoạt động theo cơ chế fail-safe, trong đó mọi yêu cầu đều nhận được kết quả xác định ngay cả khi xảy ra lỗi trong quá trình đánh giá, nhằm hạn chế ảnh hưởng đến ứng dụng sử dụng Feature Flag [10, 11].

Yêu cầu đối với Frontend Dashboard

Frontend Dashboard là giao diện quản trị cho phép người dùng thao tác trực tiếp với các tài nguyên của hệ thống.

Các yêu cầu chức năng bao gồm:

- Hiển thị danh sách project.
- Hiển thị danh sách Feature Flag.
- Tạo và chỉnh sửa Feature Flag.
- Cấu hình các rule rollout.
- Hiển thị trạng thái hoạt động của Feature Flag.
- Xem lịch sử thay đổi thông qua audit log.

Giao diện cần trực quan, phản ánh chính xác trạng thái hiện tại của từng Feature Flag và hỗ trợ người quản trị thực hiện các thao tác cấu hình một cách thuận tiện [7].

Yêu cầu đối với Demo Application

Demo Application được xây dựng nhằm minh họa cách một ứng dụng thực tế tích hợp Feature Flag Platform.

Các yêu cầu chức năng gồm:

- Gửi yêu cầu tới Evaluation API.
- Truyền Evaluation Context của người dùng.
- Hiển thị hoặc ẩn tính năng dựa trên kết quả evaluate.
- Minh họa tối thiểu hai kịch bản:
  - bật hoặc tắt tính năng toàn cục;
  - bật tính năng theo vai trò hoặc theo phần trăm người dùng.

Thông qua Demo Application, hệ thống cần chứng minh được khả năng tách biệt giữa quá trình triển khai phần mềm và phát hành tính năng, đồng thời cho phép thay đổi hành vi của ứng dụng mà không cần triển khai lại phiên bản mới [1, 3].

Yêu cầu đối với dữ liệu và tài liệu

Bên cạnh các chức năng của hệ thống, mini project còn yêu cầu chuẩn bị đầy đủ dữ liệu mẫu và tài liệu kỹ thuật nhằm hỗ trợ quá trình cài đặt, vận hành và đánh giá.

Các yêu cầu bao gồm:

- Cơ sở dữ liệu lưu trữ project, environment, Feature Flag, rule, dữ liệu người dùng mẫu và audit log.
- Seed data hoặc script khởi tạo dữ liệu mẫu.
- README hướng dẫn cài đặt và chạy hệ thống.
- Tài liệu mô tả kiến trúc hệ thống.
- Tài liệu thiết kế cơ sở dữ liệu.
- Tài liệu đặc tả API.

Việc chuẩn bị đầy đủ dữ liệu và tài liệu giúp hệ thống có thể được triển khai nhanh chóng, hỗ trợ kiểm thử, đánh giá và tạo nền tảng cho các bước mở rộng trong tương lai [15, 14].

2.3.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống Feature Flag Platform cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo tính ổn định, hiệu quả và khả năng mở rộng trong quá trình vận hành. Các yêu cầu này tập trung vào chất lượng của hệ thống thay vì các chức năng nghiệp vụ cụ thể, đồng thời phản ánh những đặc điểm quan trọng của các nền tảng Feature Flag hiện đại [8, 7].

Tính đúng đắn và xác định của kết quả

Evaluation API phải luôn trả về kết quả nhất quán đối với cùng một cấu hình Feature Flag và cùng một Evaluation Context. Thứ tự đánh giá các rule cần được xác định rõ ràng, tránh các trường hợp phụ thuộc vào thứ tự lưu trữ dữ liệu hoặc điều kiện thực thi.

Đối với rollout theo phần trăm người dùng, hệ thống cần sử dụng cơ chế băm ổn định để bảo đảm cùng một người dùng luôn được phân vào cùng một nhóm khi cấu hình không thay đổi. Điều này giúp hạn chế hiện tượng thay đổi kết quả giữa các lần evaluate và đảm bảo trải nghiệm nhất quán cho người dùng [5, 10].

Hiệu năng evaluation

Evaluation API là thành phần được gọi với tần suất cao trong quá trình vận hành ứng dụng, do đó cần có thời gian phản hồi thấp và không làm ảnh hưởng đến hiệu năng của hệ thống tích hợp.

Quá trình evaluate cần giới hạn số lượng thao tác truy cập cơ sở dữ liệu, thực hiện đánh giá rule theo thứ tự ưu tiên và hỗ trợ cơ chế cache để giảm độ trễ trong các lần truy cập lặp lại. Trong phạm vi mở rộng của mini project, hệ thống có thể sử dụng bộ nhớ trong hoặc Redis để lưu kết quả evaluate phù hợp với các yêu cầu về hiệu năng [13, 8].

Tính nhất quán của cấu hình

Mọi thay đổi đối với Feature Flag cần được phản ánh nhất quán trong toàn bộ hệ thống. Khi cấu hình được cập nhật, các lần evaluate tiếp theo phải sử dụng phiên bản cấu hình mới, đồng thời tránh các trạng thái không đồng bộ giữa dữ liệu lưu trữ và dữ liệu được sử dụng trong quá trình đánh giá.

Hệ thống cũng cần định nghĩa giá trị mặc định khi Feature Flag không tồn tại hoặc xảy ra lỗi trong quá trình evaluate nhằm đảm bảo ứng dụng luôn hoạt động theo cơ chế an toàn (fail-safe) [10, 8].

Khả năng bảo trì và mở rộng

Kiến trúc hệ thống cần được thiết kế theo hướng phân tách rõ các thành phần như Backend API, Rule Engine, Frontend Dashboard và Demo Application nhằm giảm sự phụ thuộc giữa các mô-đun.

Các loại rule cần được thiết kế theo hướng dễ mở rộng để có thể bổ sung các chiến lược rollout mới mà không ảnh hưởng đáng kể đến cơ chế evaluation hiện có. Đồng thời, cấu hình rule và thuộc tính người dùng nên được lưu dưới dạng JSONB nhằm tăng tính linh hoạt khi mở rộng mô hình dữ liệu [15].

Khả năng kiểm thử

Hệ thống cần được xây dựng theo hướng dễ kiểm thử ở cả mức chức năng và mức thành phần.

Rule Engine nên được tách biệt khỏi các thành phần giao tiếp với cơ sở dữ liệu để có thể xây

dựng các bộ kiểm thử độc lập cho từng chiến lược rollout. Đồng thời, Evaluation API cần có dữ liệu mẫu và các kịch bản kiểm thử nhằm xác minh tính đúng đắn của quá trình đánh giá Feature Flag đối với nhiều trường hợp khác nhau [10].

Bảo mật và khả năng truy vết

Các API quản trị cần được bảo vệ bằng cơ chế xác thực và phân quyền phù hợp nhằm ngăn chặn các thao tác thay đổi cấu hình trái phép. Dữ liệu đầu vào phải được kiểm tra tính hợp lệ trước khi xử lý và chỉ những thông tin cần thiết mới được ghi nhận trong quá trình giao tiếp giữa các thành phần của hệ thống [11].

Bên cạnh đó, mọi thao tác thay đổi Feature Flag, rule hoặc cấu hình hệ thống cần được ghi nhận vào audit log với đầy đủ thông tin về thời gian, người thực hiện và nội dung thay đổi. Khả năng truy vết này giúp hỗ trợ điều tra sự cố, kiểm toán hệ thống và xác định nguyên nhân khi xảy ra các thay đổi ngoài mong muốn [12].

2.3.5 Các yêu cầu khuyến khích

Bên cạnh các chức năng trong mức MVP, đề tài còn đề xuất một số yêu cầu khuyến khích nhằm nâng cao hiệu năng, khả năng mở rộng và tính hoàn thiện của hệ thống. Các chức năng này phản ánh nhiều đặc điểm thường gặp trong các nền tảng Feature Flag thương mại, đồng thời tạo nền tảng cho việc phát triển hệ thống theo hướng sẵn sàng vận hành trong môi trường thực tế [7, 8].

Caching kết quả evaluation

Evaluation API thường được gọi với tần suất rất lớn, do đó việc truy cập cơ sở dữ liệu cho mỗi lần evaluate có thể làm tăng độ trễ và giảm khả năng đáp ứng của hệ thống.

Hệ thống có thể bổ sung cơ chế cache bằng bộ nhớ trong hoặc Redis để lưu tạm kết quả evaluate trong một khoảng thời gian phù hợp. Khi cấu hình Feature Flag thay đổi, dữ liệu cache cần được làm mới hoặc loại bỏ nhằm đảm bảo tính nhất quán giữa cấu hình và kết quả đánh giá [13].

JavaScript SDK

Thay vì để ứng dụng khách tự xây dựng các lời gọi HTTP tới Evaluation API, hệ thống có thể cung cấp một JavaScript SDK đóng vai trò là lớp trung gian giữa ứng dụng và Feature Flag Platform.

SDK thực hiện việc đóng gói các thao tác gửi yêu cầu, xử lý lỗi, phân tích kết quả và cung cấp giao diện lập trình thống nhất cho các ứng dụng tích hợp. Điều này giúp giảm chi phí tích hợp và tăng khả năng tái sử dụng mã nguồn [10].

Unit test cho Rule Evaluation Engine

Rule Evaluation Engine là thành phần quyết định kết quả cuối cùng của Feature Flag, do đó cần được kiểm thử độc lập.

Các bộ unit test cần bao phủ đầy đủ các trường hợp như bật toàn cục, theo người dùng, theo vai trò, rollout theo phần trăm, thứ tự ưu tiên giữa các rule và các trường hợp lỗi. Việc kiểm thử giúp phát hiện sớm sai sót trong thuật toán evaluation và hạn chế lỗi hồi quy khi mở rộng hệ thống.

Versioning và lịch sử cấu hình

Mỗi lần thay đổi cấu hình Feature Flag hoặc rule có thể tạo ra một phiên bản mới thay vì ghi đè trực tiếp lên dữ liệu hiện có.

Versioning giúp theo dõi quá trình thay đổi cấu hình theo thời gian, hỗ trợ so sánh giữa các phiên bản và phục hồi về cấu hình trước đó khi cần thiết. Chức năng này bổ sung cho audit log bằng cách lưu giữ toàn bộ trạng thái cấu hình thay vì chỉ ghi nhận sự kiện thay đổi [9].

Phân quyền người dùng quản trị

Hệ thống có thể triển khai cơ chế phân quyền theo vai trò nhằm giới hạn phạm vi thao tác của từng nhóm người dùng.

Trong phạm vi mini project, ba vai trò gồm Administrator, Developer và Viewer được sử dụng để minh họa mô hình phân quyền. Mỗi vai trò được cấp các quyền khác nhau đối với việc quản lý Feature Flag, cấu hình rule và theo dõi hệ thống, góp phần giảm nguy cơ thay đổi cấu hình ngoài ý muốn [11].

Thống kê số lần evaluation

Hệ thống có thể thu thập số liệu về số lần Evaluation API được gọi đối với từng Feature Flag.

Các thống kê này được sử dụng để hiển thị biểu đồ trực quan trên Dashboard, giúp người quản trị đánh giá mức độ sử dụng của từng Feature Flag, xác định các Feature Flag ít được sử dụng và hỗ trợ quá trình dọn dẹp các Feature Flag không còn cần thiết [9].

Group kill switch

Ngoài Kill Switch cho từng Feature Flag, hệ thống có thể hỗ trợ Kill Switch ở cấp nhóm nhằm vô hiệu hóa đồng thời nhiều Feature Flag có liên quan.

Khi Group Kill Switch được kích hoạt, toàn bộ Feature Flag trong nhóm sẽ bị tắt ngay lập tức mà không cần chỉnh sửa từng Feature Flag riêng lẻ. Chức năng này giúp giảm thời gian phản ứng khi xảy ra sự cố trên môi trường production và hỗ trợ quá trình rollback ở phạm vi lớn [1].

Triển khai bằng Docker Compose

Hệ thống có thể được đóng gói dưới dạng nhiều container bao gồm Backend API, Frontend Dashboard, Demo Application, cơ sở dữ liệu và các dịch vụ hỗ trợ khác.

Docker Compose cho phép khởi tạo toàn bộ môi trường chỉ với một lệnh, giúp đơn giản hóa quá trình cài đặt, kiểm thử và trình diễn hệ thống trên nhiều máy tính khác nhau. Đồng thời, việc chuẩn hóa môi trường chạy cũng giúp giảm các khác biệt giữa môi trường phát triển và môi trường triển khai [14].

2.4 Kiến trúc tổng thể của hệ thống

2.4.1 Các thành phần chính

Kiến trúc của hệ thống Feature Flag được xây dựng theo mô hình nhiều thành phần (multi-component architecture), trong đó mỗi thành phần đảm nhiệm một nhóm chức năng riêng biệt và giao tiếp thông qua các API REST. Kiến trúc này giúp tách biệt giữa chức năng quản trị cấu hình, chức năng đánh giá feature flag tại thời điểm chạy và ứng dụng sử dụng feature flag, từ đó tăng khả năng bảo trì, mở rộng và tái sử dụng. Thiết kế này cũng phù hợp với kiến trúc được áp dụng trong các nền tảng Feature Flag hiện đại như LaunchDarkly và Unleash [7, 8].

Trong phạm vi mini project, hệ thống bao gồm sáu thành phần chính: Admin Dashboard, Backend API, Rule Evaluation Engine, Database và Cache, JavaScript SDK và Demo Application. Mỗi thành phần có vai trò riêng nhưng phối hợp để tạo thành một quy trình quản lý và đánh giá Feature Flag hoàn chỉnh.

Admin Dashboard

Admin Dashboard là giao diện web dành cho người quản trị, cho phép quản lý toàn bộ cấu hình của hệ thống Feature Flag. Thông qua Dashboard, người dùng có thể tạo project, environment, feature flag, cấu hình các rule targeting, bật hoặc tắt kill switch, theo dõi audit log và xem thống kê số lần evaluation.

Dashboard không trực tiếp quyết định trạng thái của feature flag mà chỉ đóng vai trò gửi các yêu cầu quản trị đến Backend API. Mọi thay đổi cấu hình đều được lưu trữ trong cơ sở dữ liệu và ghi nhận vào audit log nhằm đảm bảo khả năng truy vết và kiểm toán [12].

Việc tách riêng Dashboard khỏi hệ thống evaluation giúp các ứng dụng sử dụng feature flag không bị ảnh hưởng bởi giao diện quản trị, đồng thời cho phép mở rộng hoặc thay thế Dashboard mà không làm thay đổi cơ chế đánh giá flag.

Backend API

Backend API là trung tâm của toàn bộ hệ thống, chịu trách nhiệm xử lý các yêu cầu từ Dashboard và các ứng dụng tích hợp Feature Flag. Hệ thống được xây dựng theo kiến trúc RESTful API với các nhóm chức năng chính gồm:

- Quản lý project và environment.
- Quản lý feature flag.
- Cấu hình các rule targeting.
- Đánh giá trạng thái feature flag thông qua Evaluation API.
- Ghi nhận audit log.
- Cung cấp dữ liệu thống kê phục vụ Dashboard.

Các API quản trị yêu cầu xác thực và phân quyền, trong khi Evaluation API được tối ưu cho số lượng truy cập lớn với mục tiêu giảm độ trễ trong quá trình đánh giá feature flag. Thiết kế

này phù hợp với khuyến nghị của OpenFeature về việc tách biệt API quản trị và API đánh giá nhằm tối ưu hiệu năng và khả năng mở rộng [10].

Rule Evaluation Engine

Rule Evaluation Engine là thành phần quan trọng nhất của hệ thống, chịu trách nhiệm xác định một feature flag được bật hay tắt đối với từng request cụ thể.

Engine nhận Evaluation Context từ ứng dụng khách, tải cấu hình tương ứng của feature flag và thực hiện đánh giá theo thứ tự ưu tiên đã được xác định trước. Trong mini project, các rule được hỗ trợ gồm bật hoặc tắt toàn cục, targeting theo người dùng, targeting theo vai trò và rollout theo phần trăm người dùng.

Dối với rollout theo phần trăm, hệ thống sử dụng thuật toán băm ổn định (stable hashing) để bảo đảm cùng một người dùng luôn được phân vào cùng một bucket trong các lần đánh giá khác nhau, tránh hiện tượng thay đổi kết quả ngẫu nhiên giữa các request [5]. Evaluation Engine cũng trả về lý do (reason) của kết quả nhằm hỗ trợ việc kiểm thử, giám sát và gỡ lỗi trong quá trình vận hành [10].

Database và cache

Cơ sở dữ liệu lưu trữ toàn bộ thông tin của hệ thống, bao gồm project, environment, feature flag, cấu hình rule, dữ liệu người dùng mẫu, audit log và các thông tin thống kê. PostgreSQL được lựa chọn do hỗ trợ đầy đủ giao dịch, tính nhất quán dữ liệu và kiểu dữ liệu JSONB để lưu trữ linh hoạt các tham số của rule cũng như các bản ghi audit [15].

Để giảm độ trễ của Evaluation API, hệ thống bổ sung tầng cache cho các kết quả đánh giá hoặc cấu hình feature flag thường xuyên được truy cập. Khi cấu hình thay đổi, cache sẽ được làm mới nhằm bảo đảm tính nhất quán giữa dữ liệu lưu trong bộ nhớ và cơ sở dữ liệu. Cơ chế này phù hợp với các khuyến nghị về caching trong các hệ thống Feature Flag hiện đại [13, 8].

JavaScript SDK

JavaScript SDK đóng vai trò là thư viện trung gian giúp các ứng dụng web tích hợp Feature Flag một cách thuận tiện hơn. Thay vì trực tiếp gọi Evaluation API, ứng dụng chỉ cần sử dụng các hàm do SDK cung cấp để gửi Evaluation Context và nhận kết quả đánh giá.

SDK chịu trách nhiệm đóng gói request, xử lý lỗi cơ bản, chuyển đổi dữ liệu giữa ứng dụng và

Backend API, đồng thời cung cấp giao diện lập trình thống nhất cho các ứng dụng sử dụng Feature Flag. Việc xây dựng SDK giúp giảm sự phụ thuộc của ứng dụng vào chi tiết triển khai của API và tạo điều kiện mở rộng sang các ngôn ngữ lập trình khác trong tương lai [10].

Demo Application

Demo Application là ứng dụng web minh họa việc tích hợp Feature Flag Platform vào một hệ thống thực tế. Ứng dụng sử dụng JavaScript SDK để gửi Evaluation Context tới Backend API và thay đổi giao diện hoặc chức năng dựa trên kết quả evaluation.

Trong mini project, Demo Application được sử dụng để minh họa các kịch bản như bật hoặc tắt tính năng toàn cục, bật tính năng theo vai trò người dùng, rollout theo phần trăm và kích hoạt kill switch khi xảy ra sự cố. Thành phần này giúp kiểm chứng tính đúng đắn của Rule Evaluation Engine, đồng thời thể hiện rõ lợi ích của việc tách quá trình triển khai phần mềm khỏi quá trình phát hành tính năng như mô hình Feature Flag đã đề xuất [1, 3].

2.4.2 Control plane và data plane

Một trong những đặc điểm quan trọng của các nền tảng Feature Flag hiện đại là tách biệt giữa *control plane* và *data plane*. Hai thành phần này đảm nhiệm các chức năng khác nhau: control plane chịu trách nhiệm quản lý cấu hình, trong khi data plane tập trung xử lý các yêu cầu đánh giá feature flag tại thời điểm chạy. Cách tiếp cận này giúp giảm sự phụ thuộc giữa hoạt động quản trị và hoạt động vận hành, đồng thời nâng cao hiệu năng, khả năng mở rộng và tính sẵn sàng của hệ thống [7, 8].

Trong mini project, kiến trúc này được áp dụng bằng cách tách các API quản trị khỏi Evaluation API. Dashboard và Backend API thực hiện việc cấu hình feature flag, còn Rule Evaluation Engine phục vụ các yêu cầu đánh giá từ ứng dụng khách.

Vai trò của control plane

Control plane là thành phần chịu trách nhiệm quản lý toàn bộ vòng đời của feature flag và các tài nguyên liên quan. Các thao tác như tạo project, environment, feature flag, cấu hình rule, bật hoặc tắt kill switch, thay đổi rollout, quản lý người dùng mẫu và ghi nhận audit log đều được thực hiện thông qua control plane.

Các thay đổi cấu hình được lưu vào cơ sở dữ liệu và trở thành nguồn dữ liệu chính cho quá

trình evaluation. Control plane không trực tiếp tham gia xử lý các request từ ứng dụng khách trong thời gian chạy mà chỉ cung cấp cơ chế quản trị và cập nhật cấu hình của hệ thống.

Việc tập trung toàn bộ chức năng quản trị vào control plane giúp hệ thống dễ kiểm soát quyền truy cập, ghi nhận lịch sử thay đổi và mở rộng thêm các chức năng như versioning, RBAC hoặc approval workflow trong tương lai [12, 7].

Vai trò của data plane

Data plane chịu trách nhiệm xử lý các yêu cầu đánh giá feature flag từ các ứng dụng sử dụng hệ thống. Thành phần này bao gồm Evaluation API, Rule Evaluation Engine và các cơ chế hỗ trợ như cache nhằm bảo đảm thời gian phản hồi thấp và khả năng phục vụ số lượng lớn request đồng thời.

Đối với mỗi yêu cầu đánh giá, data plane nhận Evaluation Context từ ứng dụng, tải cấu hình tương ứng, thực hiện đánh giá theo thứ tự ưu tiên của các rule và trả về kết quả bao gồm trạng thái feature flag cùng với lý do đánh giá. Toàn bộ quá trình này được thực hiện độc lập với Dashboard nhằm tránh ảnh hưởng đến hiệu năng của các ứng dụng đang hoạt động [10, 6].

Do Evaluation API thường có tần suất truy cập rất lớn, data plane được thiết kế theo hướng tối ưu độ trễ và hạn chế tối đa các xử lý không cần thiết trong quá trình evaluation.

Luồng dữ liệu giữa hai thành phần

Mối quan hệ giữa control plane và data plane được thể hiện thông qua luồng cập nhật cấu hình và luồng đánh giá feature flag.

Đầu tiên, người quản trị thực hiện các thao tác cấu hình trên Admin Dashboard. Các yêu cầu này được gửi tới Backend API, sau đó được kiểm tra tính hợp lệ và lưu vào cơ sở dữ liệu. Nếu hệ thống sử dụng cache, dữ liệu liên quan cũng được cập nhật hoặc làm mới sau khi cấu hình thay đổi.

Khi Demo Application hoặc một ứng dụng khách cần kiểm tra trạng thái của feature flag, JavaScript SDK sẽ gửi Evaluation Context tới Evaluation API. Rule Evaluation Engine đọc cấu hình hiện tại từ cache hoặc cơ sở dữ liệu, thực hiện đánh giá theo các rule đã cấu hình và trả về kết quả cho ứng dụng.

Luồng dữ liệu này bảo đảm rằng các thay đổi được thực hiện thông qua control plane sẽ được

phản ánh tới data plane mà không làm gián đoạn quá trình phục vụ các request evaluation [8, 13].

Lợi ích của việc tách biệt hai plane

Việc tách biệt control plane và data plane mang lại nhiều lợi ích đối với hệ thống Feature Flag.

Thứ nhất, hai thành phần có thể được mở rộng độc lập theo đặc điểm tải của từng loại dịch vụ. Trong thực tế, số lượng request evaluation thường lớn hơn rất nhiều so với số lượng thao tác quản trị, do đó data plane có thể được triển khai với nhiều bản sao hơn mà không cần mở rộng Dashboard hoặc các API quản trị.

Thứ hai, việc tách biệt giúp tăng tính sẵn sàng của hệ thống. Ngay cả khi Dashboard hoặc một số chức năng quản trị gặp sự cố hoặc đang bảo trì, data plane vẫn có thể tiếp tục phục vụ các yêu cầu evaluation dựa trên cấu hình hiện có, đặc biệt khi kết hợp với cơ chế cache.

Thứ ba, kiến trúc này giúp đơn giản hóa việc bảo trì và phát triển hệ thống. Các thay đổi liên quan đến giao diện quản trị, RBAC hoặc audit log không ảnh hưởng đến Rule Evaluation Engine, trong khi các cải tiến về hiệu năng evaluation cũng không làm thay đổi logic quản trị.

Cuối cùng, việc phân tách control plane và data plane phù hợp với kiến trúc của các nền tảng Feature Flag hiện đại, đồng thời tạo tiền đề để mở rộng hệ thống thành các dịch vụ độc lập khi quy mô triển khai tăng lên [7, 8, 3].

2.4.3 Luồng quản trị Feature Flag

Luồng quản trị Feature Flag mô tả toàn bộ quá trình từ khi người quản trị tạo một feature flag mới cho đến khi cấu hình được lưu trữ và sẵn sàng phục vụ quá trình evaluation. Trong kiến trúc của hệ thống, toàn bộ các thao tác quản trị được thực hiện thông qua Admin Dashboard và Backend API, sau đó được lưu vào cơ sở dữ liệu, ghi nhận vào audit log và đồng bộ tới tầng cache nếu cần thiết. Việc tách biệt luồng quản trị khỏi luồng evaluation giúp bảo đảm các thay đổi cấu hình không ảnh hưởng trực tiếp đến các request đang được xử lý [7, 8].

Tạo và chỉnh sửa Feature Flag

Quá trình quản trị bắt đầu khi người quản trị tạo một feature flag mới trên Admin Dashboard. Người dùng cần cung cấp các thông tin cơ bản như project, environment, khóa định danh (flag key), tên hiển thị và mô tả của feature flag. Backend API thực hiện kiểm tra dữ liệu đầu vào,

xác minh tính duy nhất của khóa định danh trong phạm vi project và lưu thông tin vào cơ sở dữ liệu nếu hợp lệ.

Đối với các feature flag đã tồn tại, người quản trị có thể cập nhật thông tin mô tả hoặc thay đổi cấu hình theo nhu cầu triển khai. Các thao tác chỉnh sửa đều được thực hiện thông qua API quản trị và chỉ có hiệu lực sau khi dữ liệu được lưu thành công vào cơ sở dữ liệu.

Việc quản lý feature flag theo khóa định danh ổn định giúp các ứng dụng khách có thể tham chiếu đến cùng một feature flag trong suốt vòng đời của tính năng mà không cần thay đổi mã nguồn [7].

Cấu hình trạng thái và serving mode

Sau khi tạo feature flag, người quản trị tiến hành cấu hình trạng thái hoạt động và chế độ phục vụ của flag.

Trạng thái hoạt động xác định liệu feature flag có được phép tham gia vào quá trình evaluation hay không. Hệ thống hỗ trợ các trạng thái như kích hoạt (enabled), vô hiệu hóa (disabled) và lưu trữ (archived), trong đó archived được sử dụng để loại bỏ các feature flag không còn sử dụng nhưng vẫn giữ lại lịch sử phục vụ việc truy vết.

Serving mode xác định cách Rule Evaluation Engine xử lý feature flag tại thời điểm chạy. Đối với chế độ bật toàn cục (global on), mọi request hợp lệ đều nhận cùng một kết quả mà không cần đánh giá các rule targeting. Đối với chế độ theo rule (targeted), hệ thống sẽ thực hiện quá trình evaluation dựa trên tập các rule đã được cấu hình.

Việc tách biệt trạng thái hoạt động và serving mode giúp người quản trị kiểm soát độc lập giữa vòng đời của feature flag và chiến lược phát hành tính năng [1, 7].

Cấu hình và sắp xếp rule

Khi feature flag hoạt động ở chế độ targeted, người quản trị có thể cấu hình một hoặc nhiều rule để xác định đối tượng được phép sử dụng tính năng.

Trong phạm vi mini project, hệ thống hỗ trợ ba loại rule gồm targeting theo người dùng, targeting theo vai trò và rollout theo phần trăm người dùng. Mỗi rule được gán một giá trị ưu tiên (priority), từ đó Rule Evaluation Engine sẽ đánh giá lần lượt theo thứ tự đã cấu hình và dừng ngay khi tìm thấy rule đầu tiên thỏa mãn điều kiện.

Cách tổ chức này giúp người quản trị dễ dàng xây dựng các chiến lược rollout phức tạp bằng cách kết hợp nhiều rule khác nhau, đồng thời bảo đảm kết quả evaluation luôn xác định và nhất quán đối với cùng một Evaluation Context [4, 5].

Lưu audit log và invalidation cache

Sau mỗi thao tác thay đổi cấu hình, Backend API ghi nhận một bản ghi audit log bao gồm thông tin về người thực hiện, thời điểm thực hiện, đối tượng bị thay đổi, hành động tương ứng và trạng thái trước cũng như sau khi thay đổi. Audit log giúp tăng khả năng truy vết, hỗ trợ kiểm toán và phục vụ việc phân tích khi xảy ra sự cố hoặc cấu hình sai [12].

Nếu hệ thống sử dụng tầng cache để tăng tốc Evaluation API, các thay đổi cấu hình cũng sẽ kích hoạt cơ chế invalidation nhằm loại bỏ dữ liệu đã lỗi thời hoặc cập nhật lại cache tương ứng. Nhờ đó, Rule Evaluation Engine luôn sử dụng cấu hình mới nhất trong quá trình evaluation, đồng thời vẫn duy trì được độ trễ thấp đối với các request thường xuyên được truy cập [13, 8].

Quá trình ghi audit log và invalidation cache được thực hiện ngay sau khi giao dịch cập nhật dữ liệu hoàn tất, bảo đảm tính nhất quán giữa cơ sở dữ liệu, lịch sử thay đổi và dữ liệu được sử dụng trong quá trình evaluation.

2.4.4 Luồng đánh giá Feature Flag

Luồng đánh giá Feature Flag mô tả quá trình hệ thống xác định trạng thái của một feature flag đối với từng request tại thời điểm chạy. Đây là luồng xử lý quan trọng nhất của nền tảng vì được thực hiện với tần suất rất lớn và ảnh hưởng trực tiếp đến hành vi của các ứng dụng tích hợp. Trong mini project, toàn bộ quá trình evaluation được thực hiện bởi Evaluation API và Rule Evaluation Engine, bảo đảm kết quả đánh giá có tính xác định, nhất quán và độ trễ thấp [10, 8].

Tiếp nhận evaluation request

Quá trình evaluation bắt đầu khi ứng dụng khách gửi yêu cầu tới Evaluation API thông qua JavaScript SDK hoặc trực tiếp bằng HTTP. Request bao gồm khóa định danh của project, feature flag và Evaluation Context.

Evaluation Context chứa các thông tin cần thiết để Rule Evaluation Engine xác định đối tượng đang thực hiện request, bao gồm targeting key, vai trò người dùng và các thuộc tính mở rộng

nếu có. Các thông tin này không được lưu trữ cố định trong hệ thống Feature Flag mà được cung cấp bởi ứng dụng tích hợp tại thời điểm đánh giá [6].

Sau khi tiếp nhận request, Backend API thực hiện kiểm tra dữ liệu đầu vào, xác thực tính hợp lệ của request và chuyển yêu cầu sang Rule Evaluation Engine để xử lý.

Truy xuất evaluation snapshot

Trước khi bắt đầu đánh giá, Rule Evaluation Engine truy xuất toàn bộ cấu hình cần thiết của feature flag dưới dạng một evaluation snapshot.

Evaluation snapshot bao gồm các thông tin như trạng thái của feature flag, serving mode, kill switch, danh sách rule đã được sắp xếp theo thứ tự ưu tiên và các tham số phục vụ quá trình evaluation. Snapshot được xây dựng từ cơ sở dữ liệu hoặc tầng cache nếu dữ liệu đã tồn tại.

Việc sử dụng evaluation snapshot giúp toàn bộ quá trình đánh giá sử dụng cùng một phiên bản cấu hình, tránh trường hợp dữ liệu thay đổi giữa chừng làm phát sinh kết quả không nhất quán. Đồng thời, snapshot cũng giảm số lần truy cập cơ sở dữ liệu trong một lần evaluation, góp phần cải thiện hiệu năng của hệ thống [13, 8].

Thực thi Rule Evaluation Engine

Sau khi có evaluation snapshot, Rule Evaluation Engine tiến hành đánh giá feature flag theo thứ tự ưu tiên đã được xác định trước.

Đầu tiên, hệ thống kiểm tra các điều kiện có mức ưu tiên cao như feature flag đã bị lưu trữ, bị vô hiệu hóa hoặc đang được tắt bởi kill switch. Nếu một trong các điều kiện này được thỏa mãn, quá trình evaluation kết thúc ngay và trả về kết quả tương ứng.

Nếu feature flag vẫn hoạt động, Engine tiếp tục kiểm tra serving mode. Đối với chế độ bật toàn cục, hệ thống trả về kết quả bật mà không cần đánh giá các rule targeting. Đối với chế độ targeted, các rule được đánh giá tuần tự theo thứ tự ưu tiên, bao gồm targeting theo người dùng, targeting theo vai trò và rollout theo phần trăm người dùng.

Trong trường hợp rollout theo phần trăm, hệ thống sử dụng thuật toán băm ổn định để ánh xạ targeting key vào một bucket cố định. Cách tiếp cận này bảo đảm cùng một người dùng luôn nhận được cùng một kết quả evaluation giữa các request khác nhau, ngay cả khi hệ thống được triển khai trên nhiều máy chủ [4, 5].

Nếu không có rule nào phù hợp, Rule Evaluation Engine sử dụng giá trị mặc định của feature flag để bảo đảm mọi request đều nhận được một kết quả xác định [10].

Trả về kết quả và reason code

Sau khi hoàn tất quá trình evaluation, Backend API trả về kết quả cho ứng dụng khách.

Kết quả bao gồm tối thiểu trạng thái bật hoặc tắt của feature flag và reason code mô tả nguyên nhân dẫn đến quyết định evaluation. Reason code có thể phản ánh nhiều trường hợp khác nhau như feature flag được bật toàn cục, khớp rule theo người dùng, khớp rule theo vai trò, khớp rollout theo phần trăm, bị vô hiệu hóa, bị lưu trữ hoặc sử dụng giá trị mặc định.

Việc trả về reason code giúp tăng khả năng quan sát của hệ thống, hỗ trợ kiểm thử, phân tích hành vi của Rule Evaluation Engine và rút ngắn thời gian xử lý khi xảy ra sự cố trong môi trường vận hành [10].

Ghi nhận evaluation metric

Sau khi trả về kết quả evaluation, hệ thống thực hiện ghi nhận các thông tin thống kê phục vụ việc giám sát và phân tích hoạt động của feature flag.

Trong mini project, evaluation metric được sử dụng để thống kê số lần evaluation của từng feature flag theo từng environment, từ đó cung cấp dữ liệu cho Dashboard hiển thị biểu đồ và đánh giá mức độ sử dụng của các tính năng. Việc ghi nhận metric được thực hiện theo cơ chế bất đồng bộ hoặc best-effort nhằm tránh làm tăng độ trễ của Evaluation API.

Các metric này không tham gia vào quá trình quyết định kết quả evaluation mà chỉ phục vụ mục đích quan sát hệ thống, hỗ trợ phân tích hiệu năng, đánh giá hiệu quả rollout và đưa ra quyết định quản trị trong các lần cấu hình tiếp theo [7, 8].

2.4.5 Luồng tích hợp ứng dụng khách

Luồng tích hợp ứng dụng khách mô tả cách một hệ thống sử dụng Feature Flag Platform để quyết định hành vi của ứng dụng tại thời điểm chạy. Thay vì lưu trữ cấu hình feature flag trong mã nguồn, ứng dụng gửi Evaluation Context tới hệ thống Feature Flag để nhận kết quả evaluation và thay đổi hành vi tương ứng. Cách tiếp cận này giúp tách biệt quá trình phát triển phần mềm khỏi quá trình phát hành tính năng, đồng thời cho phép thay đổi cấu hình mà không cần triển khai lại ứng dụng [1, 3].

Trong mini project, ứng dụng khách có thể gọi Evaluation API trực tiếp hoặc thông qua JavaScript SDK. Hai cách tiếp cận đều sử dụng cùng một Evaluation API và cùng cơ chế Rule Evaluation Engine.

Khởi tạo evaluation context

Trước khi gửi yêu cầu evaluation, ứng dụng cần xây dựng Evaluation Context mô tả thông tin của đối tượng đang sử dụng hệ thống.

Evaluation Context bao gồm targeting key, danh sách vai trò của người dùng và các thuộc tính mở rộng nếu cần thiết. Các thông tin này được lấy từ cơ sở dữ liệu hoặc hệ thống xác thực của ứng dụng khách và được truyền cùng với request tới Evaluation API.

Evaluation Context không chứa logic đánh giá feature flag mà chỉ cung cấp dữ liệu đầu vào cho Rule Evaluation Engine. Việc chuẩn hóa cấu trúc Evaluation Context giúp nhiều ứng dụng khác nhau có thể sử dụng cùng một Feature Flag Platform mà không phụ thuộc vào chi tiết triển khai nội bộ [6].

Gọi trực tiếp Evaluation API

Trong cách tích hợp trực tiếp, ứng dụng gửi yêu cầu HTTP tới Evaluation API, truyền vào project key, feature flag key và Evaluation Context.

Evaluation API tiếp nhận request, thực hiện quá trình đánh giá thông qua Rule Evaluation Engine và trả về kết quả bao gồm trạng thái feature flag, reason code và các thông tin liên quan.

Cách tiếp cận này đơn giản, dễ triển khai và phù hợp với các ứng dụng hoặc dịch vụ không sử dụng JavaScript hoặc chưa có SDK tương ứng. Đồng thời, ứng dụng cũng có thể chủ động kiểm soát quá trình gửi request và xử lý kết quả evaluation [10].

Gọi thông qua JavaScript SDK

Đối với các ứng dụng web, mini project cung cấp JavaScript SDK nhằm đơn giản hóa quá trình tích hợp với Feature Flag Platform.

SDK đóng vai trò là lớp trung gian giữa ứng dụng và Evaluation API. Thay vì tự xây dựng request HTTP, ứng dụng chỉ cần gọi các hàm do SDK cung cấp. SDK chịu trách nhiệm xây

dựng request, gửi Evaluation Context, xử lý lỗi cơ bản, phân tích kết quả trả về và cung cấp giao diện lập trình thống nhất cho ứng dụng.

Việc sử dụng SDK giúp giảm mã nguồn lặp lại giữa các ứng dụng, hạn chế sai sót trong quá trình tích hợp và tạo điều kiện mở rộng sang các SDK cho những ngôn ngữ lập trình khác trong tương lai [10].

Áp dụng kết quả trong ứng dụng

Sau khi nhận được kết quả evaluation, ứng dụng sử dụng trạng thái của feature flag để quyết định hành vi tại thời điểm chạy.

Nếu feature flag được bật, ứng dụng hiển thị hoặc kích hoạt tính năng mới. Ngược lại, nếu feature flag bị tắt, ứng dụng tiếp tục sử dụng chức năng hiện có hoặc ẩn hoàn toàn tính năng tương ứng. Cách xử lý này cho phép cùng một phiên bản phần mềm phục vụ nhiều nhóm người dùng khác nhau dựa trên cấu hình của feature flag.

Trong mini project, Demo Application minh họa các kịch bản như bật hoặc tắt tính năng toàn cục, phát hành theo vai trò người dùng và rollout theo phần trăm, qua đó thể hiện rõ khả năng điều khiển hành vi của ứng dụng mà không cần triển khai lại hệ thống [1, 4].

Xử lý khi Evaluation API không khả dụng

Trong thực tế, Evaluation API có thể tạm thời không khả dụng do lỗi mạng, sự cố dịch vụ hoặc quá trình bảo trì. Để bảo đảm ứng dụng vẫn hoạt động ổn định, hệ thống cần có cơ chế xử lý lỗi phù hợp.

Trong mini project, nếu Evaluation API không thể trả về kết quả hợp lệ hoặc xảy ra lỗi trong quá trình evaluation, ứng dụng sử dụng giá trị mặc định (default value) của feature flag và tiếp tục hoạt động theo nguyên tắc *fail-closed*. Cách tiếp cận này giúp tránh việc vô tình kích hoạt các tính năng chưa sẵn sàng hoặc gây ảnh hưởng tới người dùng khi hệ thống gặp sự cố.

Việc sử dụng default value và xử lý lỗi rõ ràng là một trong những nguyên tắc quan trọng của các hệ thống Feature Flag nhằm tăng tính sẵn sàng và độ tin cậy của ứng dụng [10, 8].

2.4.6 Lựa chọn công nghệ

Việc lựa chọn công nghệ cho mini project hướng đến ba mục tiêu chính: đáp ứng đầy đủ các yêu cầu của hệ thống Feature Flag, bảo đảm khả năng mở rộng trong tương lai và phù hợp với quy mô triển khai của một nền tảng quản lý cấu hình. Các công nghệ được lựa chọn đều có hệ sinh thái trưởng thành, tài liệu đầy đủ và được sử dụng rộng rãi trong phát triển các hệ thống web hiện đại.

Lựa chọn NestJS cho Backend API

Backend API được xây dựng bằng NestJS trên nền tảng Node.js. NestJS cung cấp kiến trúc theo mô-đun (module-based architecture), hỗ trợ cơ chế Dependency Injection, middleware, guard, interceptor và exception filter, giúp tổ chức mã nguồn rõ ràng và dễ mở rộng.

Đối với hệ thống Feature Flag, Backend API bao gồm nhiều nhóm chức năng như quản lý project, feature flag, rule, audit log, evaluation và thống kê. Việc chia hệ thống thành các module độc lập giúp giảm sự phụ thuộc giữa các thành phần, đồng thời thuận lợi cho việc kiểm thử và bảo trì.

Bên cạnh đó, NestJS tích hợp tốt với TypeScript, hỗ trợ xây dựng REST API, validation dữ liệu, OpenAPI (Swagger) và nhiều thư viện phổ biến khác. Những đặc điểm này phù hợp với yêu cầu xây dựng một Backend API có cấu trúc rõ ràng và dễ phát triển trong tương lai.

Lựa chọn React cho giao diện web

Admin Dashboard và Demo Application được xây dựng bằng React. React sử dụng mô hình giao diện theo component, cho phép chia giao diện thành các thành phần độc lập và có khả năng tái sử dụng cao.

Trong hệ thống Feature Flag, nhiều màn hình như danh sách feature flag, cấu hình rule, audit log và thống kê có cấu trúc tương tự nhau. Việc xây dựng dưới dạng các component giúp giảm trùng lặp mã nguồn, đơn giản hóa quá trình bảo trì và hỗ trợ mở rộng thêm các chức năng mới.

Ngoài ra, React có hệ sinh thái phong phú, khả năng tích hợp thuận lợi với các thư viện quản lý trạng thái, gọi REST API và định tuyến, đáp ứng tốt yêu cầu xây dựng Dashboard quản trị hiện đại.

Lựa chọn PostgreSQL và Prisma

PostgreSQL được lựa chọn làm hệ quản trị cơ sở dữ liệu do hỗ trợ đầy đủ giao dịch ACID, khóa ngoại, chỉ mục, ràng buộc dữ liệu và khả năng xử lý các truy vấn phức tạp.

Đối với hệ thống Feature Flag, nhiều dữ liệu như tham số của rule, thuộc tính người dùng mẫu và audit snapshot có cấu trúc linh hoạt. PostgreSQL hỗ trợ kiểu dữ liệu JSONB, cho phép lưu trữ các cấu trúc dữ liệu bán cấu trúc nhưng vẫn hỗ trợ lập chỉ mục và truy vấn hiệu quả [15].

Prisma được sử dụng làm ORM để ánh xạ giữa cơ sở dữ liệu và mã nguồn TypeScript. Prisma cung cấp kiểu dữ liệu an toàn, hỗ trợ migration, sinh mã tự động và giảm đáng kể lượng câu lệnh SQL phải viết thủ công. Điều này giúp tăng năng suất phát triển đồng thời hạn chế lỗi trong quá trình thao tác với cơ sở dữ liệu.

Lựa chọn cache in-memory hoặc Redis

Evaluation API thường có số lượng request lớn hơn nhiều so với các API quản trị. Vì vậy, việc bổ sung tầng cache giúp giảm số lần truy cập cơ sở dữ liệu và cải thiện thời gian phản hồi.

Trong phạm vi mini project, hệ thống hỗ trợ sử dụng cache in-memory do triển khai đơn giản, không yêu cầu thêm hạ tầng và phù hợp với môi trường chạy một tiến trình.

Đối với các hệ thống triển khai trên nhiều máy chủ hoặc yêu cầu khả năng chia sẻ dữ liệu cache giữa nhiều phiên bản ứng dụng, Redis là lựa chọn phù hợp hơn. Redis cung cấp bộ nhớ dùng chung, hỗ trợ invalidation và đồng bộ cache hiệu quả giữa các instance của Backend API [13].

Việc thiết kế tầng cache theo hướng có thể thay thế giúp hệ thống dễ dàng chuyển từ cache in-memory sang Redis mà không làm thay đổi Rule Evaluation Engine.

Lựa chọn Docker và Docker Compose

Hệ thống được triển khai bằng Docker nhằm chuẩn hóa môi trường thực thi giữa các máy phát triển và môi trường triển khai.

Docker Compose được sử dụng để quản lý toàn bộ các thành phần của hệ thống như Backend API, Frontend Dashboard, Demo Application, PostgreSQL và Redis thông qua một tệp cấu hình duy nhất. Nhờ đó, việc cài đặt và khởi chạy hệ thống chỉ cần một lệnh, giảm đáng kể thời gian cấu hình môi trường và hạn chế sự khác biệt giữa các máy phát triển [14].

Bên cạnh đó, Docker Compose cũng tạo điều kiện thuận lợi cho việc kiểm thử tích hợp, trình diễn sản phẩm và mở rộng hệ thống trong tương lai.

Các phương án thay thế và lý do không lựa chọn

Trong quá trình thiết kế, một số công nghệ khác cũng được xem xét.

Đối với Backend API, Express có ưu điểm đơn giản và nhẹ hơn NestJS. Tuy nhiên, Express không cung cấp sẵn kiến trúc theo mô-đun, cơ chế Dependency Injection và nhiều thành phần hỗ trợ phát triển hệ thống quy mô lớn, khiến việc tổ chức mã nguồn phức tạp hơn khi số lượng chức năng tăng lên.

Đối với giao diện web, Vue và Angular đều có thể đáp ứng yêu cầu của hệ thống. Tuy nhiên, React có cộng đồng sử dụng rộng lớn, hệ sinh thái phong phú và phù hợp với kinh nghiệm phát triển của nhóm thực hiện, giúp rút ngắn thời gian triển khai.

Đối với cơ sở dữ liệu, MySQL có thể đáp ứng các chức năng CRUD cơ bản nhưng khả năng hỗ trợ JSON và các tính năng truy vấn dữ liệu bán cấu trúc không linh hoạt bằng PostgreSQL. SQLite cũng không được lựa chọn do không phù hợp với mô hình máy chủ nhiều người dùng đồng thời.

Đối với tầng cache, chỉ sử dụng cơ sở dữ liệu mà không có cache sẽ làm tăng độ trễ của Evaluation API khi số lượng request lớn. Ngược lại, việc triển khai Redis ngay từ đầu sẽ làm tăng độ phức tạp của hạ tầng đối với một mini project. Vì vậy, hệ thống được thiết kế theo hướng hỗ trợ cache in-memory trong giai đoạn đầu và có thể chuyển sang Redis khi cần mở rộng.

Cuối cùng, Kubernetes không được lựa chọn thay cho Docker Compose do vượt quá phạm vi của mini project. Docker Compose đã đáp ứng đầy đủ yêu cầu triển khai nhiều dịch vụ, đồng thời đơn giản hơn trong quá trình phát triển, kiểm thử và trình diễn hệ thống.

2.5 Thiết kế dữ liệu

2.5.1 Mô hình dữ liệu tổng thể

Mô hình dữ liệu của hệ thống được xây dựng nhằm hỗ trợ hai nhóm nghiệp vụ chính: quản lý cấu hình Feature Flag và đánh giá trạng thái flag tại thời điểm chạy. Dữ liệu được tổ chức theo cấu trúc phân cấp từ project, environment, feature flag đến cấu hình runtime và các rule

targeting. Bên cạnh đó, hệ thống duy trì dữ liệu người dùng mẫu phục vụ kiểm thử và audit log để truy vết các thay đổi cấu hình.

Thiết kế này bảo đảm mỗi Feature Flag có một định danh ổn định trong phạm vi project, đồng thời có thể mang cấu hình khác nhau giữa các environment. Cách tổ chức này phù hợp với mô hình quản lý Feature Flag phổ biến, trong đó định nghĩa flag được dùng chung nhưng trạng thái và chiến lược phát hành có thể được cấu hình độc lập cho từng môi trường [7].

Các thực thể chính

Mô hình dữ liệu gồm các thực thể chính sau:

- **Project**: đại diện cho một sản phẩm hoặc ứng dụng sử dụng nền tảng Feature Flag. Project là phạm vi quản lý cấp cao nhất và chứa các environment, feature flag và người dùng mẫu.
- **Environment**: đại diện cho môi trường triển khai của một project, chẳng hạn như development, staging hoặc production. Mỗi environment có thể chứa cấu hình runtime khác nhau cho cùng một Feature Flag.
- **FeatureFlag**: lưu định nghĩa ổn định của một flag, bao gồm khóa định danh, tên hiển thị, mô tả và trạng thái vòng đời. Thực thể này không trực tiếp lưu trạng thái bật hoặc tắt theo environment.
- **FlagConfig**: lưu cấu hình runtime của một Feature Flag trong một environment cụ thể. Cấu hình bao gồm trạng thái bật hoặc tắt, chế độ phục vụ, kill switch và giá trị variant tương ứng.
- **FlagRule**: biểu diễn một rule targeting thuộc một cấu hình flag. Rule có loại, mức ưu tiên và tập tham số dùng để xác định đối tượng được bật tính năng.
- **SampleUser**: lưu dữ liệu người dùng mẫu phục vụ việc kiểm thử và minh họa Evaluation API. Mỗi người dùng có targeting key, danh sách role và tập thuộc tính mở rộng.
- **AuditLogEntry**: ghi nhận lịch sử các thao tác thay đổi dữ liệu quản trị, bao gồm người thực hiện, hành động, đối tượng bị tác động và trạng thái trước, sau khi thay đổi.

Các thực thể `FlagRule`, `SampleUser` và `AuditLogEntry` sử dụng dữ liệu có cấu trúc linh hoạt như tham số rule, thuộc tính người dùng hoặc snapshot thay đổi. Những trường dữ liệu này

được lưu bằng kiểu `JSONB` của PostgreSQL để vừa duy trì khả năng mở rộng, vừa hỗ trợ truy vấn và lập chỉ mục khi cần thiết [15].

Quan hệ giữa các thực thể

Các thực thể trong hệ thống có quan hệ như sau:

- Một `Project` có nhiều `Environment`.
- Một `Project` có nhiều `FeatureFlag`.
- Một `Project` có nhiều `SampleUser`.
- Một `Project` có nhiều `AuditLogEntry`.
- Một `FeatureFlag` có nhiều `FlagConfig`.
- Một `Environment` có nhiều `FlagConfig`.
- Mỗi `FlagConfig` thuộc đúng một `FeatureFlag` và một `Environment`.
- Một `FlagConfig` có thể có nhiều `FlagRule`.
- Mỗi `FlagRule` chỉ thuộc một `FlagConfig`.

Quan hệ giữa `FeatureFlag` và `Environment` là quan hệ nhiều–nhiều được triển khai thông qua thực thể trung gian `FlagConfig`. Nhờ đó, cùng một flag có thể được bật toàn cục trên môi trường staging nhưng chỉ rollout theo phần trăm người dùng trên môi trường production.

Bảng 2.10 tóm tắt các quan hệ chính trong mô hình dữ liệu.

Bảng 2.10: Quan hệ giữa các thực thể chính

Thực thể nguồn	Thực thể đích	Quan hệ	Ý nghĩa
Project	Environment	Một–nhiều	Một project có thể được triển khai trên nhiều môi trường.
Project	FeatureFlag	Một–nhiều	Các flag được định danh và quản lý trong phạm vi project.
FeatureFlag	FlagConfig	Một–nhiều	Một flag có cấu hình riêng cho từng environment.
Environment	FlagConfig	Một–nhiều	Một environment chứa cấu hình runtime của nhiều flag.
FlagConfig	FlagRule	Một–nhiều	Một cấu hình có thể chứa nhiều rule targeting theo thứ tự ưu tiên.
Project	SampleUser	Một–nhiều	Người dùng mẫu được sử dụng để kiểm thử evaluation trong project.
Project	AuditLogEntry	Một–nhiều	Các thay đổi quản trị trong project được ghi lại để truy vết.

Các khóa ngoại được sử dụng để bảo đảm tính toàn vẹn tham chiếu. Đồng thời, các ràng buộc duy nhất được thiết lập cho những cặp thuộc tính như `projectId-environmentKey`, `projectId-flagKey` và `featureFlagId-environmentId`. Các ràng buộc này ngăn việc tạo trùng environment, flag hoặc cấu hình runtime trong cùng một phạm vi quản lý.

Phân tách định nghĩa flag và cấu hình runtime

Một quyết định quan trọng trong thiết kế dữ liệu là tách thực thể `FeatureFlag` khỏi `FlagConfig`. Trong đó, `FeatureFlag` mô tả danh tính và vòng đời của flag, còn `FlagConfig` mô tả cách flag được phục vụ tại thời điểm chạy trong từng environment.

Thực thể `FeatureFlag` chứa các thông tin tương đối ổn định, bao gồm:

- khóa định danh của flag;
- tên và mô tả;
- project sở hữu;
- trạng thái vòng đời như `ACTIVE` hoặc `ARCHIVED`;
- thời điểm tạo và cập nhật.

Ngược lại, `FlagConfig` chứa các thuộc tính có thể thay đổi thường xuyên trong quá trình phát hành:

- environment áp dụng;
- trạng thái `ENABLED` hoặc `DISABLED`;
- chế độ `GLOBAL_ON` hoặc `TARGETED`;
- trạng thái kill switch;
- giá trị variant khi flag bật hoặc tắt;
- phiên bản cấu hình;
- thời điểm cập nhật gần nhất.

Việc phân tách này mang lại các lợi ích sau:

- Cho phép một flag được cấu hình độc lập trên nhiều environment mà không phải sao chép toàn bộ định nghĩa flag.
- Tránh trộn lẫn trạng thái vòng đời với trạng thái runtime. Một flag đang ở vòng đời `ACTIVE` vẫn có thể bị tắt trên một environment cụ thể.
- Cho phép thay đổi cấu hình production mà không ảnh hưởng đến development hoặc staging.
- Tạo điều kiện thuận lợi cho việc versioning, cache invalidation và lưu lịch sử cấu hình.
- Giúp Evaluation API truy vấn trực tiếp cấu hình tương ứng với bộ ba `projectKey`, `environmentKey` và `flagKey`.

Cách phân tách này cũng phản ánh bản chất của Feature Flag: định nghĩa flag là một phần của hệ thống quản trị, trong khi kết quả evaluation phụ thuộc vào cấu hình runtime và evaluation context của từng request [6, 10].

Ví dụ, flag có khóa `new-checkout` chỉ được định nghĩa một lần trong project thương mại điện tử. Tuy nhiên, flag này có thể được cấu hình như sau:

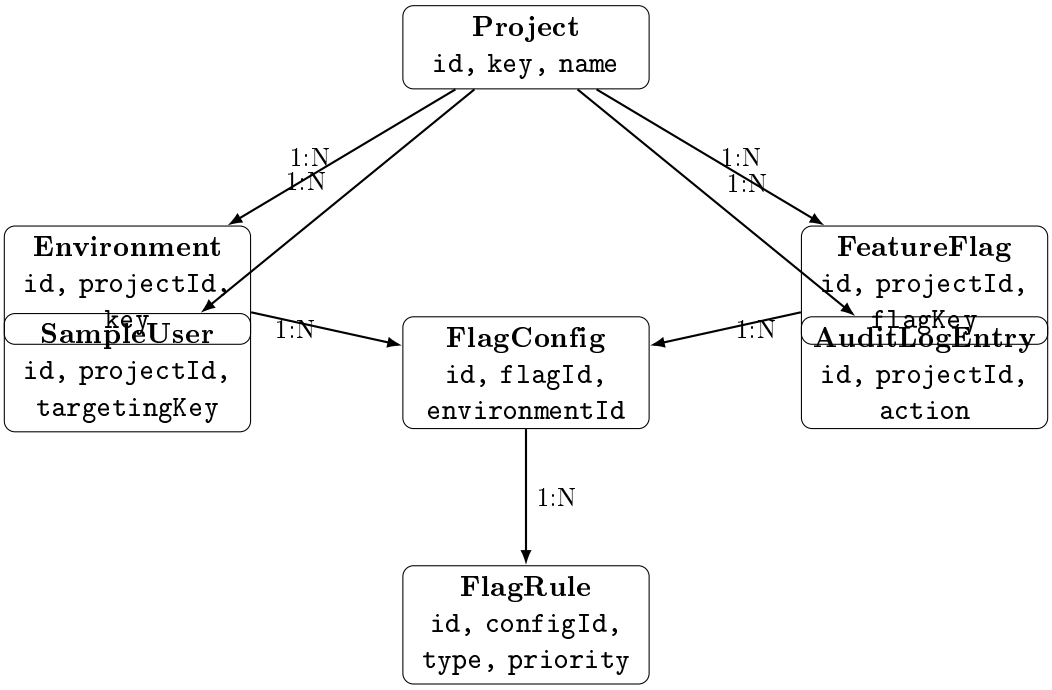
- bật toàn cục trong environment development;
- bật cho role `BETA_CUSTOMER` trong environment staging;

- rollout cho 20% người dùng trong environment production.

Nếu trạng thái bật, tắt và các rule được lưu trực tiếp trong bảng `FeatureFlag`, hệ thống sẽ khó biểu diễn các cấu hình khác nhau giữa các environment. Vì vậy, `FlagConfig` đóng vai trò là thực thể liên kết giữa định nghĩa flag và môi trường triển khai.

Sơ đồ thực thể liên kết

Sơ đồ thực thể liên kết của hệ thống được trình bày trong Hình 2.1. Sơ đồ thể hiện cấu trúc phân cấp từ `Project` đến `Environment` và `FeatureFlag`, cùng với thực thể trung gian `FlagConfig` và các rule targeting tương ứng.



Hình 2.1: Sơ đồ thực thể liên kết của hệ thống Feature Flag

Trong sơ đồ, `FlagConfig` là trung tâm của dữ liệu phục vụ evaluation. Khi nhận một yêu cầu đánh giá, hệ thống xác định project, environment và flag tương ứng, sau đó truy xuất cấu hình runtime cùng danh sách rule được sắp xếp theo độ ưu tiên. Dữ liệu trong `SampleUser` không phải nguồn dữ liệu bắt buộc của Evaluation API mà chủ yếu phục vụ demo, kiểm thử và mô phỏng các evaluation context khác nhau.

2.5.2 Project và environment

Project và environment tạo thành hai cấp quản lý nền tảng của hệ thống. Project xác định phạm vi logic của một ứng dụng hoặc sản phẩm, trong khi environment biểu diễn các môi trường triển khai độc lập bên trong project. Cách phân tách này cho phép cùng một Feature Flag được cấu hình khác nhau giữa development, staging và production mà không cần tạo các flag riêng biệt.

Thực thể Project

Thực thể `Project` đại diện cho một ứng dụng, sản phẩm hoặc miền nghiệp vụ sử dụng nền tảng Feature Flag. Đây là thực thể gốc trong mô hình dữ liệu và cung cấp phạm vi định danh cho environment, Feature Flag, người dùng mẫu và audit log.

Các thuộc tính chính của `Project` được trình bày trong Bảng 2.11.

Bảng 2.11: Các thuộc tính của thực thể Project

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
`id`	UUID	Khóa chính	Định danh nội bộ duy nhất của project.
`key`	Chuỗi	Bắt buộc, duy nhất	Khóa ổn định dùng trong API và quá trình evaluation.
`name`	Chuỗi	Bắt buộc	Tên hiển thị của project trên dashboard quản trị.
`description`	Chuỗi	Có thể rỗng	Mô tả mục đích hoặc phạm vi sử dụng của project.
`createdAt`	Timestamp	Bắt buộc	Thời điểm project được tạo.
`updatedAt`	Timestamp	Bắt buộc	Thời điểm project được cập nhật gần nhất.

Thuộc tính `key` được sử dụng thay cho `id` trong các API công khai vì có ý nghĩa rõ ràng hơn đối với ứng dụng tích hợp và không phụ thuộc vào định danh nội bộ của cơ sở dữ liệu. Giá trị này phải ổn định trong suốt vòng đời project, do đó hệ thống không cho phép tạo hai project có cùng khóa.

Khóa project được kiểm tra theo quy tắc định dạng thống nhất, chỉ bao gồm chữ thường, chữ số và dấu gạch ngang. Việc chuẩn hóa này giúp khóa có thể được sử dụng an toàn trong URL, request payload, cache key và dữ liệu log.

Project có các quan hệ một–nhiều với những thực thể sau:

- `Environment`: các môi trường triển khai thuộc project;
- `FeatureFlag`: các flag được định nghĩa trong project;
- `SampleUser`: dữ liệu người dùng mẫu phục vụ demo và kiểm thử;
- `AuditLogEntry`: lịch sử thay đổi các tài nguyên thuộc project.

Việc đặt Feature Flag và environment trong phạm vi project giúp tránh xung đột khóa giữa các ứng dụng khác nhau. Hai project có thể cùng sử dụng khóa flag `new-checkout`, nhưng mỗi flag vẫn là một tài nguyên độc lập với cấu hình và lịch sử thay đổi riêng.

Thực thể Environment

Thực thể `Environment` đại diện cho một môi trường triển khai cụ thể của project. Các environment phổ biến gồm development, staging và production. Mỗi environment duy trì tập cấu hình runtime độc lập cho các Feature Flag thuộc cùng project.

Các thuộc tính chính của `Environment` được trình bày trong Bảng 2.12.

Bảng 2.12: Các thuộc tính của thực thể Environment

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
`id`	UUID	Khóa chính	Định danh nội bộ duy nhất của environment.
`projectId`	UUID	Khóa ngoại	Tham chiếu đến project sở hữu environment.
`key`	Chuỗi	Bắt buộc	Khóa environment được sử dụng trong Evaluation API.
`name`	Chuỗi	Bắt buộc	Tên hiển thị của environment trên dashboard.
`description`	Chuỗi	Có thể rỗng	Mô tả mục đích sử dụng của environment.
`isDefault`	Boolean	Mặc định là `false`	Xác định environment mặc định của project.
`createdAt`	Timestamp	Bắt buộc	Thời điểm environment được tạo.
`updatedAt`	Timestamp	Bắt buộc	Thời điểm environment được cập nhật gần nhất.

Thuộc tính `projectId` xác định phạm vi sở hữu của environment. Khóa environment chỉ cần duy nhất trong cùng một project, vì vậy các project khác nhau có thể cùng sử dụng các khóa phổ biến như `development`, `staging` hoặc `production`.

Mỗi environment có quan hệ một–nhiều với `FlagConfig`. Một cấu hình runtime chỉ có hiệu lực

trong environment mà nó tham chiếu. Vì vậy, việc thay đổi trạng thái hoặc rule của flag trong staging không làm thay đổi cấu hình của cùng flag trong production.

Ví dụ, cùng một flag `new-checkout` có thể có ba cấu hình độc lập:

- environment `development`: bật toàn cục để đội phát triển kiểm thử;
- environment `staging`: bật theo role để kiểm thử chấp nhận;
- environment `production`: rollout cho một tỷ lệ nhỏ người dùng.

Cơ chế này hỗ trợ quá trình phát hành theo từng giai đoạn và giảm nguy cơ một thay đổi thử nghiệm ảnh hưởng trực tiếp đến môi trường production.

Environment mặc định

Mỗi project có thể xác định một environment mặc định thông qua thuộc tính `isDefault`. Environment mặc định được sử dụng khi request evaluation không truyền rõ `environmentKey`.

Luồng lựa chọn environment được thực hiện theo thứ tự sau:

1. Nếu request chứa `environmentKey`, hệ thống tìm environment có khóa tương ứng trong project.
2. Nếu request không chứa `environmentKey`, hệ thống sử dụng environment có `isDefault = true`.
3. Nếu không tìm thấy environment được yêu cầu hoặc project chưa có environment mặc định, Evaluation API trả về lỗi hoặc giá trị fallback an toàn tùy theo hợp đồng API.

Environment mặc định giúp đơn giản hóa quá trình tích hợp trong các trường hợp ứng dụng khách chỉ sử dụng một môi trường hoặc đã được triển khai với cấu hình cố định. Tuy nhiên, đối với các hệ thống có nhiều môi trường, việc truyền rõ `environmentKey` vẫn được khuyến nghị nhằm tránh đánh giá nhầm cấu hình.

Tại một thời điểm, mỗi project chỉ được phép có tối đa một environment mặc định. Ràng buộc này được bảo đảm ở mức cơ sở dữ liệu bằng chỉ mục duy nhất có điều kiện trên `projectId` đối với các bản ghi có `isDefault = true`.

Ví dụ về chỉ mục trong PostgreSQL:

Đoạn mã 2.6: Chỉ mục bảo đảm một environment mặc định cho mỗi project

```
CREATE UNIQUE INDEX unique_default_environment_per_project
ON environments (project_id)
WHERE is_default = TRUE;
```

Khi người quản trị chọn một environment mới làm mặc định, hệ thống phải cập nhật environment mặc định cũ về `false` và environment mới về `true` trong cùng một transaction. Cách xử lý này tránh trạng thái trung gian trong đó project có hai environment mặc định hoặc không có environment mặc định ngoài ý muốn.

Ràng buộc environment trong project

Để bảo đảm tính nhất quán của dữ liệu, environment phải tuân theo các ràng buộc sau:

- Mỗi environment phải thuộc đúng một project thông qua khóa ngoại `projectId`.
- Cặp giá trị `projectId` và `key` phải là duy nhất.
- Một project chỉ có tối đa một environment có `isDefault = true`.
- Environment chỉ có thể được sử dụng để tạo `FlagConfig` cho các Feature Flag thuộc cùng project.
- Khóa environment phải tuân theo quy tắc định dạng khóa của hệ thống.
- Không được xóa environment nếu thao tác đó làm mất dữ liệu cấu hình runtime ngoài ý muốn.

Ràng buộc duy nhất trên `projectId` và `key` có thể được biểu diễn như sau:

Đoạn mã 2.7: Ràng buộc khóa environment duy nhất trong project

```
ALTER TABLE environments
ADD CONSTRAINT unique_environment_key_per_project
UNIQUE (project_id, key);
```

Ngoài các ràng buộc trực tiếp trong bảng `Environment`, hệ thống cần kiểm tra tính đồng nhất giữa project của environment và project của Feature Flag khi tạo `FlagConfig`. Một cấu hình không được phép liên kết flag thuộc project này với environment thuộc project khác.

Về mặt mô hình quan hệ, `FlagConfig` chứa hai khóa ngoại riêng biệt là `featureFlagId` và `environmentId`. Do cơ sở dữ liệu không thể suy ra trực tiếp hai bản ghi này phải thuộc cùng

project chỉ từ hai khóa ngoại đơn, kiểm tra được thực hiện trong tầng dịch vụ trước khi ghi dữ liệu. Tất cả thao tác tạo hoặc cập nhật cấu hình đều phải xác nhận:

$$\texttt{FeatureFlag.projectId} = \texttt{Environment.projectId}$$

Đồng thời, cặp `featureFlagId-environmentId` phải là duy nhất để mỗi Feature Flag chỉ có một cấu hình runtime trong một environment. Ràng buộc này ngăn việc tồn tại nhiều cấu hình cạnh tranh cho cùng một flag và giúp Evaluation Engine xác định duy nhất snapshot cần sử dụng.

Khi xóa một project, các environment thuộc project không còn ý nghĩa độc lập. Vì vậy, quan hệ từ `Project` đến `Environment` có thể sử dụng cơ chế cascade delete. Tuy nhiên, thao tác xóa project cần được giới hạn bởi phân quyền, xác nhận rõ ràng và ghi audit log trước khi dữ liệu liên quan bị loại bỏ.

2.5.3 Feature Flag và cấu hình theo environment

Trong mô hình dữ liệu của hệ thống, định nghĩa Feature Flag được tách khỏi cấu hình runtime theo từng environment. Thực thể `FeatureFlag` lưu thông tin định danh và trạng thái vòng đời của flag, trong khi `FlagConfig` lưu cách flag được phục vụ trong một environment cụ thể. Thiết kế này cho phép cùng một flag có trạng thái và chiến lược rollout khác nhau giữa development, staging và production.

Thực thể FeatureFlag

Thực thể `FeatureFlag` đại diện cho định nghĩa logic của một flag trong phạm vi project. Đây là thông tin tương đối ổn định, được sử dụng để nhận diện flag trong dashboard quản trị, Evaluation API, audit log và ứng dụng tích hợp.

Các thuộc tính chính của `FeatureFlag` được trình bày trong Bảng 2.13.

Bảng 2.13: Các thuộc tính của thực thể FeatureFlag

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
`id`	UUID	Khóa chính	Định danh nội bộ duy nhất của flag.
`projectId`	UUID	Khóa ngoại	Tham chiếu đến project sở hữu flag.
`key`	Chuỗi	Bắt buộc	Khóa ổn định dùng để định danh flag trong Evaluation API.
`name`	Chuỗi	Bắt buộc	Tên hiển thị của flag trên dashboard quản trị.
`description`	Chuỗi	Có thể rỗng	Mô tả mục đích, phạm vi và cách sử dụng flag.
`lifecycleStatus`	Enum	Bắt buộc	Trạng thái vòng đời của flag, gồm `ACTIVE` hoặc `ARCHIVED`.
`createdAt`	Timestamp	Bắt buộc	Thời điểm flag được tạo.
`updatedAt`	Timestamp	Bắt buộc	Thời điểm flag được cập nhật gần nhất.

Khóa flag chỉ cần duy nhất trong phạm vi project. Do đó, hai project khác nhau có thể cùng sử dụng một khóa như `new-checkout`, nhưng trong cùng một project không được tồn tại hai flag có cùng khóa.

Ràng buộc duy nhất được áp dụng trên cặp `projectId` và `key`:

Đoạn mã 2.8: Ràng buộc khóa Feature Flag duy nhất trong project

```
ALTER TABLE feature_flags
ADD CONSTRAINT unique_feature_flag_key_per_project
UNIQUE (project_id, key);
```

Khóa flag cần ổn định trong suốt vòng đời tích hợp vì được lưu trực tiếp trong mã nguồn hoặc cấu hình của ứng dụng khách. Việc thay đổi khóa có thể khiến request evaluation không tìm thấy flag và trả về kết quả fallback. Vì vậy, hệ thống chỉ nên cho phép sửa tên và mô tả, trong khi việc đổi khóa phải được hạn chế hoặc không được hỗ trợ sau khi flag đã được sử dụng.

Trạng thái vòng đời của flag

Trạng thái vòng đời mô tả việc flag còn được sử dụng hay đã được loại khỏi hoạt động quản trị thông thường. Trong phạm vi hệ thống, `FeatureFlag` sử dụng hai trạng thái:

- `ACTIVE`: flag đang được quản lý và có thể được cấu hình, đánh giá hoặc thay đổi rule.

- `ARCHIVED`: flag không còn được sử dụng trong luồng phát hành thông thường nhưng vẫn được giữ lại để truy vết lịch sử.

Trạng thái vòng đời khác với trạng thái bật hoặc tắt của flag. Một flag ở trạng thái `ACTIVE` có thể được bật trong staging nhưng bị tắt trong production. Ngược lại, khi flag chuyển sang `ARCHIVED`, Evaluation Engine phải ngừng phục vụ flag đó bất kể cấu hình runtime đang lưu trong các environment.

Việc archive thay cho xóa vật lý mang lại các lợi ích sau:

- giữ lại lịch sử cấu hình và audit log;
- tránh làm mất liên kết với các rule hoặc thống kê đã phát sinh;
- cho phép khôi phục flag khi archive nhầm;
- hỗ trợ nhận diện và xử lý feature flag debt;
- giảm nguy cơ ứng dụng khách gặp hành vi không xác định do flag bị xóa đột ngột.

Khi evaluation một flag đã archive, hệ thống trả về kết quả an toàn với `enabled = false` và reason code `FLAG_ARCHIVED`. Trạng thái này được kiểm tra trước trạng thái cấu hình, serving mode và các rule targeting.

Bảng 2.14: Ý nghĩa trạng thái vòng đời của Feature Flag

Trạng thái	**Ý nghĩa quản trị**	**Hành vi khi evaluation**
`ACTIVE`	Flag còn hiệu lực và có thể được cấu hình theo environment.	Tiếp tục kiểm tra `FlagConfig`, kill switch, serving mode và các rule.
`ARCHIVED`	Flag đã được đưa ra khỏi vòng đời sử dụng thông thường.	Trả về tắt với reason code `FLAG_ARCHIVED`.

Việc chuyển trạng thái giữa `ACTIVE` và `ARCHIVED` phải được ghi audit log, bao gồm actor, thời điểm, trạng thái trước và trạng thái sau. Hệ thống cũng cần giới hạn thao tác này cho người dùng có quyền quản trị phù hợp.

Thực thể FlagConfig

Thực thể `FlagConfig` lưu cấu hình runtime của một Feature Flag trong một environment cụ thể. Đây là thực thể liên kết giữa `FeatureFlag` và `Environment`, đồng thời là nguồn dữ liệu trung tâm của Evaluation Engine.

Các thuộc tính chính của `FlagConfig` được trình bày trong Bảng 2.15.

Bảng 2.15: Các thuộc tính của thực thể FlagConfig

Thuộc tính	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa**
`id`	UUID	Khóa chính	Định danh nội bộ duy nhất của cấu hình.
`featureFlagId`	UUID	Khóa ngoại	Tham chiếu đến Feature Flag được cấu hình.
`environmentId`	UUID	Khóa ngoại	Tham chiếu đến environment áp dụng cấu hình.
`status`	Enum	Bắt buộc	Trạng thái cấu hình, gồm `ENABLED` hoặc `DISABLED`.
`servingMode`	Enum	Bắt buộc	Chế độ phục vụ flag, gồm `GLOBAL_ON` hoặc `TARGETED`.
`killSwitch`	Boolean	Mặc định là `false`	Cho phép tắt khẩn cấp flag trong environment.
`version`	Số nguyên	Bắt buộc	Phiên bản cấu hình phục vụ versioning và cache invalidation.
`createdAt`	Timestamp	Bắt buộc	Thời điểm cấu hình được tạo.
`updatedAt`	Timestamp	Bắt buộc	Thời điểm cấu hình được cập nhật gần nhất.

Mỗi cặp `featureFlagId` và `environmentId` chỉ được phép có một bản ghi `FlagConfig`. Ràng buộc này bảo đảm Evaluation Engine luôn xác định được duy nhất một cấu hình runtime cho một flag trong một environment.

Đoạn mã 2.9: Ràng buộc một cấu hình cho mỗi flag trong một environment

```
ALTER TABLE flag_configs
ADD CONSTRAINT unique_flag_config_per_environment
UNIQUE (feature_flag_id, environment_id);
```

Khi một Feature Flag mới được tạo, hệ thống có thể tự động khởi tạo một `FlagConfig` cho từng environment hiện có. Cấu hình mặc định nên sử dụng `DISABLED`, `TARGETED` và `killSwitch = false` để tránh vô tình phát hành tính năng khi chưa có cấu hình rõ ràng.

Tương tự, khi một environment mới được tạo, hệ thống có thể tạo cấu hình mặc định cho tất cả Feature Flag đang hoạt động trong project. Thao tác này nên được thực hiện trong transaction để tránh trạng thái environment chỉ có cấu hình cho một phần các flag.

Trạng thái cấu hình và serving mode

Thuộc tính `status` và `servingMode` biểu diễn hai khía cạnh khác nhau của cấu hình runtime.

`status` xác định flag có được phép tham gia quá trình evaluation trong environment hay không:

- `ENABLED`: cấu hình đang hoạt động; Evaluation Engine tiếp tục xét serving mode và rule.
- `DISABLED`: flag bị tắt trong environment; hệ thống trả về `enabled = false` với reason code `FLAG_DISABLED`.

`servingMode` xác định cách flag được bật khi cấu hình ở trạng thái `ENABLED`:

- `GLOBAL_ON`: bật flag cho toàn bộ request hợp lệ trong environment mà không cần xét các rule targeting.
- `TARGETED`: chỉ bật flag khi evaluation context thỏa mãn ít nhất một rule.

Bảng 2.16 trình bày các tổ hợp chính giữa trạng thái cấu hình và serving mode.

Bảng 2.16: Tổ hợp trạng thái cấu hình và serving mode

Status	**Serving mode**	**Hành vi evaluation**
`DISABLED`	Bất kỳ	Flag bị tắt ngay; serving mode và các rule không được xét.
`ENABLED`	`GLOBAL_ON`	Flag được bật cho toàn bộ request, trừ khi kill switch đang hoạt động.
`ENABLED`	`TARGETED`	Evaluation Engine đánh giá các rule theo thứ tự ưu tiên.

Kill switch có mức ưu tiên cao hơn serving mode. Khi `killSwitch = true`, flag bị tắt ngay cả khi cấu hình đang `ENABLED` và serving mode là `GLOBAL_ON`. Thứ tự kiểm tra tương ứng là:

1. trạng thái vòng đời `ARCHIVED`;
2. trạng thái cấu hình `DISABLED`;
3. kill switch;
4. serving mode `GLOBAL_ON`;
5. các rule trong chế độ `TARGETED`;
6. giá trị mặc định tắt.

Việc phân tách `status` và `servingMode` giúp tránh sử dụng một trường duy nhất để biểu diễn quá nhiều trạng thái. Người quản trị có thể tạm thời disable flag nhưng vẫn giữ nguyên các rule và serving mode để khôi phục sau đó mà không cần cấu hình lại.

Cấu hình flag theo từng environment

Mỗi Feature Flag có thể có một cấu hình riêng trong từng environment. Cơ chế này cho phép đội phát triển kiểm soát quá trình phát hành theo từng giai đoạn và giới hạn phạm vi ảnh hưởng của thay đổi.

Ví dụ, flag `new-checkout` có thể được cấu hình như trong Bảng 2.17.

Bảng 2.17: Ví dụ cấu hình một Feature Flag theo environment

Environment	**Status**	**Serving mode**	**Kill switch**	**Chiến lược áp dụng**
Development	`ENABLED`	`GLOBAL_ON`	`false`	Bật cho toàn bộ đội phát triển.
Staging	`ENABLED`	`TARGETED`	`false`	Bật cho role kiểm thử hoặc người dùng được chọn.
Production	`ENABLED`	`TARGETED`	`false`	Rollout cho một tỷ lệ người dùng ổn định.

Mỗi thay đổi cấu hình chỉ tác động đến environment được lựa chọn. Chẳng hạn, bật kill switch trong production không làm thay đổi cấu hình của development và staging. Điều này yêu cầu mọi API cập nhật cấu hình phải xác định rõ cả Feature Flag và environment đích.

Khi Evaluation API nhận request, hệ thống sử dụng bộ ba khóa:

(`projectKey`, `environmentKey`, `flagKey`)

để truy xuất duy nhất `FlagConfig` tương ứng. Sau đó, hệ thống nạp các `FlagRule` thuộc cấu hình này và tạo evaluation snapshot phục vụ Rule Evaluation Engine.

Việc lưu cấu hình theo environment mang lại các lợi ích chính sau:

- cho phép thử nghiệm tính năng ở development trước khi đưa sang staging;
- hỗ trợ kiểm thử có kiểm soát trước khi phát hành production;

- cho phép rollout dần hoặc targeting riêng trên production;
- cô lập sự cố cấu hình giữa các môi trường;
- hỗ trợ rollback nhanh bằng cách thay đổi riêng cấu hình production;
- duy trì một định nghĩa flag thống nhất trong toàn bộ project.

Mỗi thao tác thay đổi `status`, `servingMode`, `killSwitch` hoặc rule liên quan phải tăng phiên bản cấu hình và ghi một audit log. Phiên bản mới được sử dụng để làm mất hiệu lực cache cũ, trong khi audit log cho phép xác định ai đã thay đổi cấu hình, vào thời điểm nào và nội dung trước, sau thay đổi.

2.5.4 Rule và tham số rule

Rule là thành phần mô tả điều kiện để một Feature Flag được bật trong chế độ `TARGETED`. Mỗi rule thuộc về một `FlagConfig`, có loại xác định, mức ưu tiên và tập tham số tương ứng. Khi evaluation, các rule được xét theo thứ tự ưu tiên cho đến khi tìm thấy rule đầu tiên khớp với evaluation context.

Thực thể FlagRule

Thực thể `FlagRule` biểu diễn một điều kiện targeting cụ thể của Feature Flag trong một environment. Rule không gắn trực tiếp với `FeatureFlag` mà gắn với `FlagConfig`, vì cùng một flag có thể sử dụng tập rule khác nhau giữa các environment.

Các thuộc tính chính của `FlagRule` được trình bày trong Bảng 2.18.

Bảng 2.18: Các thuộc tính của thực thể FlagRule

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
`id`	UUID	Khóa chính	Định danh nội bộ duy nhất của rule.
`flagConfigId`	UUID	Khóa ngoại	Tham chiếu đến cấu hình flag chứa rule.
`type`	Enum	Bắt buộc	Xác định loại rule và cấu trúc tham số tương ứng.
`priority`	Số nguyên	Bắt buộc	Xác định thứ tự đánh giá rule trong cùng một cấu hình.
`parameters`	JSONB	Bắt buộc	Lưu các tham số phụ thuộc vào loại rule.
`createdAt`	Timestamp	Bắt buộc	Thời điểm rule được tạo.
`updatedAt`	Timestamp	Bắt buộc	Thời điểm rule được cập nhật gần nhất.

Mỗi `FlagRule` chỉ thuộc một `FlagConfig`. Khi cấu hình bị xóa, các rule liên quan không còn ý nghĩa độc lập và có thể được xóa theo cơ chế cascade. Mọi thao tác tạo, sửa hoặc xóa rule phải được ghi audit log để truy vết thay đổi cấu hình.

Kiểu rule

Trong phạm vi hệ thống, ba loại rule được hỗ trợ:

- `USER_ALLOWLIST`: bật flag cho một tập targeting key cụ thể.
- `ROLE_TARGETING`: bật flag nếu evaluation context chứa ít nhất một role được cấu hình.
- `PERCENTAGE_ROLLOUT`: bật flag cho một tỷ lệ ổn định người dùng dựa trên hàm băm.

Bảng 2.19 mô tả ý nghĩa của từng loại rule.

Bảng 2.19: Các loại rule được hỗ trợ

Loại rule	Điều kiện khớp	Trường hợp sử dụng
`USER_ALLOWLIST`	`targetingKey` nằm trong danh sách được cấu hình.	Bật tính năng cho người dùng nội bộ, tester hoặc khách hàng cụ thể.
`ROLE_TARGETING`	Có giao giữa danh sách role của request và danh sách role trong rule.	Bật tính năng cho nhóm người dùng theo vai trò nghiệp vụ.
`PERCENTAGE_ROLLOUT`	Bucket ổn định của `targetingKey` nhỏ hơn tỷ lệ rollout.	Phát hành dần tính năng cho một phần người dùng.

Các loại rule được mô hình hóa bằng enum thay vì chuỗi tự do nhằm tránh sai chính tả, đơn giản hóa validation và giúp Evaluation Engine xử lý rõ ràng từng trường hợp.

Thứ tự ưu tiên của rule

Thuộc tính `priority` xác định thứ tự đánh giá các rule trong cùng một `FlagConfig`. Rule có giá trị priority nhỏ hơn được đánh giá trước. Evaluation Engine dừng tại rule đầu tiên khớp và trả về kết quả tương ứng.

Ví dụ, một cấu hình có thể sử dụng thứ tự sau:

1. `USER_ALLOWLIST` với priority 10;
2. `ROLE_TARGETING` với priority 20;
3. `PERCENTAGE_ROLLOUT` với priority 30.

Cách sắp xếp này cho phép các đối tượng cụ thể được ưu tiên trước các chiến lược rộng hơn. Người dùng nằm trong allowlist sẽ được bật tính năng ngay cả khi không có role phù hợp hoặc không thuộc bucket rollout.

Để tránh hai rule có cùng vị trí đánh giá trong một cấu hình, hệ thống áp dụng ràng buộc duy nhất trên cặp `flagConfigId` và `priority`:

Đoạn mã 2.10: Ràng buộc priority duy nhất trong một cấu hình flag

```
ALTER TABLE flag_rules
ADD CONSTRAINT unique_rule_priority_per_config
UNIQUE (flag_config_id, priority);
```

Khi truy xuất dữ liệu phục vụ evaluation, các rule phải được sắp xếp tăng dần theo `priority`. Không nên phụ thuộc vào thứ tự vật lý của bản ghi trong cơ sở dữ liệu.

Nếu không có rule nào khớp, hệ thống trả về `enabled = false` với reason code `DEFAULT_OFF`. Cơ chế này bảo đảm hành vi fail-closed trong chế độ targeted.

Lưu trữ tham số bằng JSON

Các loại rule có cấu trúc tham số khác nhau. Vì vậy, thuộc tính `parameters` được lưu bằng kiểu `JSONB` của PostgreSQL thay vì tạo nhiều cột chỉ áp dụng cho từng loại rule [15].

Ví dụ tham số của rule `USER_ALLOWLIST`:

Đoạn mã 2.11: Tham số của rule USER_ALLOWLIST

```
{
  "targetingKeys": [
    "user-001",
    "user-017",
    "user-103"
  ]
}
```

Ví dụ tham số của rule `ROLE_TARGETING`:

Đoạn mã 2.12: Tham số của rule ROLE_TARGETING

```
{
  "roles": [
    "ADMIN",
    "BETA_CUSTOMER"
  ]
}
```

Ví dụ tham số của rule `PERCENTAGE_ROLLOUT`:

Đoạn mã 2.13: Tham số của rule PERCENTAGE_ROLLOUT

```
{
  "percentage": 25
}
```

Việc sử dụng `JSONB` mang lại các lợi ích sau:

- cho phép mỗi loại rule có cấu trúc tham số riêng;
- dễ bổ sung loại rule hoặc thuộc tính mới mà không phải thay đổi nhiều cột;
- tránh các trường rỗng không liên quan trong bảng;
- hỗ trợ truy vấn và lập chỉ mục trên nội dung JSON khi cần;

- phù hợp với việc lưu snapshot trước và sau trong audit log.

Tuy nhiên, `JSONB` không tự bảo đảm tính đúng đắn của cấu trúc nghiệp vụ. Vì vậy, validation tham số phải được thực hiện ở tầng API và tầng dịch vụ trước khi dữ liệu được lưu.

Validation tham số theo loại rule

Hệ thống thực hiện validation dựa trên giá trị của `type`. Mỗi loại rule có một schema tham số riêng và không được phép chứa cấu trúc không phù hợp.

Đối với `USER_ALLOWLIST`, tham số phải thỏa mãn:

- có trường `targetingKeys`;
- `targetingKeys` là một mảng chuỗi;
- mảng không rỗng;
- mỗi giá trị không phải chuỗi rỗng;
- các targeting key không bị trùng lặp.

Đối với `ROLE_TARGETING`, tham số phải thỏa mãn:

- có trường `roles`;
- `roles` là một mảng chuỗi;
- mảng không rỗng;
- mỗi role không phải chuỗi rỗng;
- các role không bị trùng lặp.

Đối với `PERCENTAGE_ROLLOUT`, tham số phải thỏa mãn:

- có trường `percentage`;
- `percentage` là số;
- giá trị nằm trong khoảng từ 0 đến 100;
- không chứa các trường không được hỗ trợ.

Bảng 2.20 tóm tắt các quy tắc validation.

Bảng 2.20: Validation tham số theo loại rule

Loại rule	Tham số bắt buộc	Ràng buộc chính
`USER_ALLOWLIST`	`targetingKeys`	Mảng chuỗi không rỗng, không có giá trị rỗng hoặc trùng lặp.
`ROLE_TARGETING`	`roles`	Mảng chuỗi không rỗng, không có role rỗng hoặc trùng lặp.
`PERCENTAGE_ROLLOUT`	`percentage`	Giá trị số trong khoảng từ 0 đến 100.

Nếu tham số không hợp lệ, API trả về lỗi `VALIDATION_ERROR` và không ghi dữ liệu vào cơ sở dữ liệu. Phản hồi lỗi cần chỉ rõ trường vi phạm để dashboard có thể hiển thị thông báo phù hợp.

Ngoài validation khi tạo và cập nhật rule, Evaluation Engine vẫn phải xử lý phòng vệ đối với dữ liệu không hợp lệ. Nếu không thể đọc hoặc diễn giải tham số, rule được xem là không khớp và lỗi cần được ghi log phục vụ chẩn đoán. Cách xử lý này tránh việc một rule lỗi làm bật tính năng ngoài ý muốn.

Sau mỗi thao tác thay đổi rule, hệ thống phải tăng phiên bản của `FlagConfig`, làm mất hiệu lực cache liên quan và ghi audit log chứa snapshot trước và sau thay đổi.

2.5.5 User context mẫu

Hệ thống lưu một tập dữ liệu user context mẫu nhằm phục vụ demo, kiểm thử Evaluation API và minh họa các chiến lược targeting. Dữ liệu này không thay thế cơ sở dữ liệu người dùng của ứng dụng khách và không được xem là nguồn danh tính chính thức trong môi trường thực tế. Khi tích hợp, ứng dụng khách tự xây dựng evaluation context từ dữ liệu nghiệp vụ của mình và gửi đến nền tảng Feature Flag.

Evaluation context có thể bao gồm targeting key, user ID, organization ID, danh sách role và các thuộc tính mở rộng. Trong đó, targeting key là định danh chính được sử dụng để đánh giá allowlist và percentage rollout, còn các trường còn lại cung cấp thêm thông tin cho các rule targeting hiện tại hoặc khả năng mở rộng trong tương lai [6].

Targeting key

`targetingKey` là định danh ổn định đại diện cho đối tượng được evaluation. Đối tượng này thường là một người dùng, nhưng cũng có thể là một organization, thiết bị, tenant hoặc thực thể nghiệp vụ khác tùy theo cách ứng dụng khách xây dựng context.

Targeting key có các vai trò chính sau:

- đối chiếu với danh sách định danh trong rule `USER_ALLOWLIST`;
- làm đầu vào cho thuật toán băm ổn định của rule `PERCENTAGE_ROLLOUT`;
- giúp cùng một đối tượng nhận kết quả nhất quán giữa nhiều lần evaluation;
- tránh phụ thuộc trực tiếp vào khóa chính nội bộ của nền tảng Feature Flag.

Trong mô hình dữ liệu mẫu, targeting key phải duy nhất trong phạm vi project. Hai project có thể sử dụng cùng một targeting key vì dữ liệu người dùng của chúng được quản lý độc lập.

Các thuộc tính chính của thực thể `SampleUser` được trình bày trong Bảng 2.21.

Bảng 2.21: Các thuộc tính của thực thể SampleUser

Thuộc tính	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa**
`id`	UUID	Khóa chính	Định danh nội bộ của bản ghi người dùng mẫu.
`projectId`	UUID	Khóa ngoại	Tham chiếu đến project sở hữu dữ liệu mẫu.
`targetingKey`	Chuỗi	Bắt buộc	Định danh ổn định dùng trong quá trình evaluation.
`userId`	Chuỗi	Có thể rỗng	Định danh người dùng trong hệ thống của ứng dụng khách.
`organizationId`	Chuỗi	Có thể rỗng	Định danh organization hoặc tenant của người dùng.
`roles`	Mảng chuỗi	Bắt buộc	Danh sách role nghiệp vụ của người dùng.
`attributes`	JSONB	Bắt buộc	Tập thuộc tính mở rộng phục vụ demo và targeting.
`createdAt`	Timestamp	Bắt buộc	Thời điểm dữ liệu mẫu được tạo.
`updatedAt`	Timestamp	Bắt buộc	Thời điểm dữ liệu mẫu được cập nhật gần nhất.

Ràng buộc duy nhất trên cặp `projectId` và `targetingKey` có thể được biểu diễn như sau:

Đoạn mã 2.14: Ràng buộc targeting key duy nhất trong project

```
ALTER TABLE sample_users
ADD CONSTRAINT unique_targeting_key_per_project
UNIQUE (project_id, targeting_key);
```

Targeting key phải ổn định trong suốt thời gian đối tượng tham gia rollout. Nếu giá trị này thay đổi, kết quả băm phần trăm có thể thay đổi và người dùng có thể bị chuyển sang cohort

khác.

User ID và organization ID

`userId` là định danh người dùng trong hệ thống của ứng dụng khách. Giá trị này có thể trùng với targeting key, nhưng không bắt buộc. Việc tách hai trường cho phép ứng dụng lựa chọn một định danh phù hợp hơn cho evaluation mà không làm thay đổi khóa nghiệp vụ gốc.

Ví dụ, ứng dụng có thể sử dụng:

- `userId = 18452` làm định danh nội bộ trong cơ sở dữ liệu;
- `targetingKey = customer-18452` làm định danh ổn định cho Feature Flag.

`organizationId` đại diện cho organization, tenant, doanh nghiệp hoặc nhóm mà người dùng thuộc về. Trường này hỗ trợ các trường hợp cần bật tính năng cho toàn bộ thành viên của một organization thay vì cấu hình riêng từng người dùng.

Ví dụ một evaluation context có thể được biểu diễn như sau:

Đoạn mã 2.15: Evaluation context chứa user ID và organization ID

```
{
  "targetingKey": "customer-18452",
  "userId": "18452",
  "organizationId": "org-viettel-001"
}
```

Trong phiên bản hiện tại, `userId` và `organizationId` chủ yếu được lưu để mô phỏng context và chuẩn bị cho khả năng mở rộng rule. Các rule bắt buộc của hệ thống chưa đánh giá trực tiếp hai trường này nếu chúng không được chọn làm targeting key.

Nền tảng Feature Flag không tự suy luận hoặc truy vấn thông tin organization từ user ID. Ứng dụng khách chịu trách nhiệm lấy dữ liệu từ cơ sở dữ liệu nghiệp vụ và truyền context phù hợp trong mỗi request evaluation.

Role của người dùng

Thuộc tính `roles` lưu danh sách vai trò nghiệp vụ của người dùng trong ứng dụng khách. Đây không phải role quản trị của nền tảng Feature Flag mà là role dùng để quyết định việc phát

hành tính năng trong demo app.

Ví dụ, ứng dụng thương mại điện tử có thể sử dụng các role:

- `ADMIN`;
- `BETA_CUSTOMER`;
- `REGULAR_CUSTOMER`.

Một người dùng có thể có một hoặc nhiều role:

Đoạn mã 2.16: Danh sách role trong user context mẫu

```
{
  "roles": [
    "BETA_CUSTOMER",
    "REGULAR_CUSTOMER"
  ]
}
```

Khi đánh giá rule `ROLE_TARGETING`, hệ thống kiểm tra giao giữa danh sách role trong evaluation context và danh sách role được cấu hình trong rule. Rule được xem là khớp nếu tồn tại ít nhất một role chung.

Biểu diễn điều kiện khớp như sau:

$$\texttt{context.roles} \cap \texttt{rule.parameters.roles} \neq \varnothing$$

Role phải được chuẩn hóa về cách viết, chẳng hạn sử dụng chữ hoa và dấu gạch dưới. Việc chuẩn hóa giúp tránh các trường hợp cùng một role nhưng được biểu diễn khác nhau như `beta-customer`, `BetaCustomer` hoặc `BETA_CUSTOMER`.

Danh sách role của dữ liệu mẫu phải thỏa mãn các điều kiện:

- là một mảng chuỗi;
- không chứa chuỗi rỗng;
- không chứa giá trị trùng lặp;

- mỗi giá trị tuân theo quy ước đặt tên của ứng dụng.

Thuộc tính mở rộng

Thuộc tính `attributes` lưu các thông tin bổ sung của người dùng dưới dạng `JSONB`. Cấu trúc linh hoạt này cho phép dữ liệu mẫu biểu diễn nhiều loại context mà không cần thay đổi schema mỗi khi bổ sung một thuộc tính mới [15].

Ví dụ:

Đoạn mã 2.17: Thuộc tính mở rộng của user context mẫu

```
{
  "country": "VN",
  "city": "Ho Chi Minh City",
  "deviceType": "mobile",
  "subscriptionPlan": "premium",
  "accountAgeDays": 420
}
```

Các thuộc tính này có thể được sử dụng cho những loại rule mở rộng trong tương lai, chẳng hạn:

- targeting theo quốc gia hoặc khu vực;
- targeting theo loại thiết bị;
- targeting theo gói dịch vụ;
- targeting theo thời gian sử dụng tài khoản;
- targeting theo thuộc tính nghiệp vụ tùy chỉnh.

Mặc dù sử dụng `JSONB`, hệ thống vẫn cần validation cơ bản để bảo đảm `attributes` là một object JSON hợp lệ. Các giá trị nên giới hạn ở kiểu dữ liệu đơn giản như chuỗi, số, boolean hoặc mảng giá trị nguyên thủy để Evaluation Engine có thể xử lý nhất quán.

Dữ liệu nhạy cảm như mật khẩu, token xác thực, số giấy tờ, thông tin thanh toán hoặc dữ liệu cá nhân không cần thiết không được lưu trong attributes. Evaluation context chỉ nên chứa những thuộc tính tối thiểu cần thiết cho việc targeting, qua đó giảm rủi ro lộ dữ liệu và hạn

chế việc ghi thông tin nhạy cảm vào log [8, 12].

Phạm vi sử dụng dữ liệu mẫu

Thực thể `SampleUser` được sử dụng trong các phạm vi sau:

- cung cấp dữ liệu có sẵn cho demo app;
- kiểm thử rule theo targeting key;
- kiểm thử rule theo role;
- kiểm tra tính ổn định của percentage rollout;
- minh họa user context trong dashboard quản trị;
- hỗ trợ seed data và các kịch bản trình diễn.

Dữ liệu mẫu không tham gia bắt buộc vào luồng Evaluation API. Evaluation Engine đánh giá trực tiếp context được gửi trong request và không cần truy vấn `SampleUser` để lấy thông tin người dùng. Cách thiết kế này giữ Evaluation API độc lập với cơ sở dữ liệu người dùng của demo app và giảm một lần truy vấn trong luồng runtime.

Luồng xử lý được phân tách như sau:

1. Demo app lựa chọn hoặc đọc một người dùng mẫu.
2. Demo app xây dựng evaluation context từ targeting key, user ID, organization ID, role và attributes.
3. Evaluation context được gửi đến Evaluation API.
4. Evaluation Engine sử dụng trực tiếp context để đánh giá rule.
5. Bảng `SampleUser` không được truy vấn lại trong quá trình evaluation.

Trong môi trường thực tế, dữ liệu người dùng được quản lý bởi ứng dụng khách. Nền tảng Feature Flag chỉ nhận context cần thiết tại thời điểm evaluation. Vì vậy, `SampleUser` là thành phần phục vụ minh họa và kiểm thử, không phải chức năng quản lý danh tính hoặc hồ sơ khách hàng.

Việc giới hạn phạm vi này giúp tránh biến nền tảng Feature Flag thành một hệ thống quản lý người dùng, đồng thời duy trì ranh giới trách nhiệm rõ ràng giữa ứng dụng khách và dịch vụ

evaluation.

2.5.6 Flag group và group kill switch

Flag group được sử dụng để tập hợp nhiều Feature Flag có liên quan về mặt nghiệp vụ hoặc vận hành. Mỗi group thuộc một project và có thể chứa nhiều flag. Bên cạnh mục đích tổ chức dữ liệu trên dashboard, group còn cung cấp cơ chế kill switch ở cấp nhóm, cho phép tắt nhanh toàn bộ các flag thành viên khi xảy ra sự cố.

Thực thể FlagGroup

Thực thể `FlagGroup` đại diện cho một nhóm Feature Flag trong phạm vi project. Một group có thể tương ứng với một miền chức năng, một luồng nghiệp vụ, một đợt phát hành hoặc một tập tính năng cần được kiểm soát chung.

Các thuộc tính chính của `FlagGroup` được trình bày trong Bảng 2.22.

Bảng 2.22: Các thuộc tính của thực thể FlagGroup

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
`id`	UUID	Khóa chính	Định danh nội bộ duy nhất của group.
`projectId`	UUID	Khóa ngoại	Tham chiếu đến project sở hữu group.
`key`	Chuỗi	Bắt buộc	Khóa ổn định dùng để định danh group trong phạm vi project.
`name`	Chuỗi	Bắt buộc	Tên hiển thị của group trên dashboard quản trị.
`description`	Chuỗi	Có thể rỗng	Mô tả phạm vi nghiệp vụ hoặc mục đích của group.
`killSwitch`	Boolean	Mặc định là `false`	Xác định group kill switch có đang được kích hoạt hay không.
`createdAt`	Timestamp	Bắt buộc	Thời điểm group được tạo.
`updatedAt`	Timestamp	Bắt buộc	Thời điểm group được cập nhật gần nhất.

Khóa group chỉ cần duy nhất trong phạm vi project. Ràng buộc này có thể được biểu diễn như sau:

Đoạn mã 2.18: Ràng buộc khóa group duy nhất trong project

```
ALTER TABLE flag_groups
ADD CONSTRAINT unique_flag_group_key_per_project
```

```
UNIQUE (project_id, key);
```

Group không thay thế project hoặc environment. Project vẫn là phạm vi quản lý cấp cao nhất, còn environment vẫn xác định cấu hình runtime. Flag group chỉ bổ sung một lớp tổ chức và điều khiển vận hành đối với một tập flag có liên quan.

Ví dụ, project thương mại điện tử có thể định nghĩa các group như:

- `checkout-flow`: chứa các flag liên quan đến quy trình thanh toán;
- `recommendation-system`: chứa các flag liên quan đến gợi ý sản phẩm;
- `promotion-engine`: chứa các flag liên quan đến khuyến mãi;
- `experimental-ui`: chứa các flag giao diện đang thử nghiệm.

Quan hệ giữa group và flag

Trong phạm vi mini project, một `FeatureFlag` có thể thuộc tối đa một `FlagGroup`, trong khi một `FlagGroup` có thể chứa nhiều Feature Flag. Quan hệ này được mô hình hóa theo dạng một–nhiều:

$$\texttt{FlagGroup}\ 1 \longrightarrow N\ \texttt{FeatureFlag}$$

Thực thể `FeatureFlag` chứa khóa ngoại `groupId` có thể rỗng:

- nếu `groupId` có giá trị, flag thuộc group tương ứng;
- nếu `groupId` là `NULL`, flag không thuộc group nào;
- group và flag phải thuộc cùng một project.

Việc giới hạn mỗi flag chỉ thuộc một group giúp mô hình đơn giản, tránh xung đột khi nhiều group cùng bật kill switch và làm rõ phạm vi vận hành của từng flag. Thiết kế này phù hợp với mục tiêu của mini project, trong đó group chủ yếu được sử dụng để tổ chức flag và thực hiện thao tác tắt khẩn cấp theo nhóm.

Các ràng buộc cần được kiểm tra khi gán flag vào group gồm:

- group phải tồn tại;

- flag phải tồn tại;
- `FlagGroup.projectId` phải bằng `FeatureFlag.projectId`;
- flag không được gán đồng thời vào nhiều group;
- flag đã archive không nên được gán mới vào group.

Điều kiện đồng nhất project được biểu diễn như sau:

$$\texttt{FlagGroup.projectId} = \texttt{FeatureFlag.projectId}$$

Do khóa ngoại `groupId` chỉ xác nhận group tồn tại mà không tự kiểm tra group và flag thuộc cùng project, tầng dịch vụ phải thực hiện validation trước khi cập nhật quan hệ.

Khi một group bị xóa, hệ thống không nên xóa cascade các Feature Flag thành viên. Thay vào đó, trường `groupId` của các flag được đặt về `NULL`. Cách xử lý này bảo toàn định nghĩa flag và cấu hình runtime, vì group chỉ là lớp tổ chức bổ sung.

Trạng thái group kill switch

Thuộc tính `killSwitch` trong `FlagGroup` biểu diễn trạng thái tắt khẩn cấp của toàn bộ group. Trạng thái này gồm hai giá trị:

- `false`: group hoạt động bình thường; từng flag tiếp tục được đánh giá theo cấu hình riêng;
- `true`: toàn bộ flag thuộc group bị tắt ở bước evaluation.

Group kill switch có mức ưu tiên cao hơn trạng thái cấu hình, serving mode và các rule của từng flag. Khi group kill switch được bật, Evaluation Engine không cần tiếp tục đánh giá `FlagConfig` hoặc `FlagRule` của flag thành viên.

Thứ tự kiểm tra có thể được xác định như sau:

1. kiểm tra trạng thái vòng đời của flag;
2. kiểm tra group kill switch;
3. kiểm tra trạng thái cấu hình của flag;
4. kiểm tra flag-level kill switch;

5. kiểm tra serving mode;
6. đánh giá các rule targeting;
7. trả về giá trị mặc định tắt.

Nếu flag thuộc một group đang bật kill switch, Evaluation API trả về:

- `enabled = false`;
- variant tắt;
- reason code `GROUP_KILL_SWITCH`.

Việc sử dụng reason code riêng giúp phân biệt trường hợp flag bị tắt do group với trường hợp bị tắt trực tiếp tại cấu hình flag. Điều này hỗ trợ dashboard, audit log và quá trình chẩn đoán sự cố.

Mỗi thao tác bật hoặc tắt group kill switch phải được ghi audit log với các thông tin:

- actor thực hiện;
- project và group bị tác động;
- trạng thái trước và sau;
- thời điểm thay đổi;
- request ID;
- lý do hoặc ghi chú nếu có.

Phạm vi ảnh hưởng của group kill switch

Trong thiết kế hiện tại, group thuộc project nhưng không gắn trực tiếp với environment. Do đó, group kill switch có phạm vi toàn project và ảnh hưởng đến tất cả cấu hình environment của các flag thành viên.

Ví dụ, group `checkout-flow` chứa ba flag:

- `new-checkout`;
- `express-payment`;
- `smart-address-suggestion`.

Khi `checkout-flow.killSwitch = true`, cả ba flag đều bị tắt trong development, staging và production, bất kể cấu hình riêng của từng flag trong các environment này.

Phạm vi toàn project phù hợp với mục tiêu của kill switch cấp nhóm là phản ứng nhanh trước sự cố nghiêm trọng. Người quản trị chỉ cần thực hiện một thao tác để vô hiệu hóa toàn bộ nhóm tính năng có liên quan, thay vì cập nhật từng flag riêng lẻ.

Tuy nhiên, phạm vi này cũng đòi hỏi thao tác phải được kiểm soát chặt chẽ vì có thể ảnh hưởng đồng thời đến nhiều environment. Dashboard cần hiển thị rõ:

- số lượng flag thuộc group;
- danh sách environment chịu ảnh hưởng;
- cảnh báo trước khi bật kill switch;
- trạng thái group kill switch nổi bật;
- thời điểm và người thực hiện thay đổi gần nhất.

Group kill switch không làm thay đổi dữ liệu cấu hình riêng của các flag. Các giá trị `status`, `servingMode`, flag-level `killSwitch` và danh sách rule vẫn được giữ nguyên. Khi group kill switch được tắt, các flag quay lại hành vi evaluation theo cấu hình trước đó.

Điều này có thể được biểu diễn như sau:

$$\texttt{effectiveEnabled} = \neg\texttt{group.killSwitch} \wedge \texttt{flagEvaluationResult}$$

Trong đó, `flagEvaluationResult` là kết quả evaluation thông thường của flag sau khi xét trạng thái vòng đời, cấu hình, kill switch riêng và các rule.

Sau khi trạng thái group kill switch thay đổi, hệ thống phải làm mất hiệu lực các cache evaluation liên quan đến toàn bộ flag thành viên. Nếu cache được tổ chức theo từng flag và environment, quá trình invalidation cần xác định tất cả cặp `flagId-environmentId` thuộc group để xóa hoặc tăng phiên bản tương ứng.

2.5.7 Audit log và lịch sử cấu hình

Audit log được sử dụng để ghi nhận các thao tác làm thay đổi trạng thái hoặc cấu hình của hệ thống Feature Flag. Mỗi bản ghi audit mô tả ai đã thực hiện thay đổi, thay đổi được thực hiện

trong request nào, đối tượng nào bị tác động và dữ liệu trước, sau khi cập nhật. Cơ chế này hỗ trợ truy vết sự cố, kiểm tra trách nhiệm và khôi phục lại diễn biến của quá trình thay đổi cấu hình [12].

Bên cạnh audit log, hệ thống sử dụng số phiên bản cấu hình để xác định mỗi lần thay đổi trạng thái runtime hoặc rule của flag. Hai cơ chế có vai trò bổ sung cho nhau: audit log lưu thông tin chi tiết phục vụ truy vết, trong khi versioning cung cấp định danh gọn cho trạng thái cấu hình tại một thời điểm.

Cấu trúc AuditLogEntry

Thực thể `AuditLogEntry` biểu diễn một sự kiện thay đổi dữ liệu trong hệ thống quản trị. Mỗi bản ghi tương ứng với một thao tác đã hoàn tất, chẳng hạn như tạo Feature Flag, cập nhật cấu hình, thêm rule, archive flag hoặc bật group kill switch.

Các thuộc tính chính của `AuditLogEntry` được trình bày trong Bảng 2.23.

Bảng 2.23: Các thuộc tính của thực thể AuditLogEntry

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
`id`	UUID	Khóa chính	Định danh duy nhất của bản ghi audit.
`projectId`	UUID	Khóa ngoại	Project chứa tài nguyên bị thay đổi.
`actor`	Chuỗi	Bắt buộc	Định danh người dùng hoặc tác nhân thực hiện thao tác.
`requestId`	Chuỗi	Bắt buộc	Định danh request tạo ra thay đổi.
`targetType`	Enum	Bắt buộc	Loại đối tượng bị tác động.
`targetId`	UUID	Có thể rỗng	Định danh nội bộ của đối tượng bị tác động.
`targetKey`	Chuỗi	Có thể rỗng	Khóa nghiệp vụ của đối tượng để hỗ trợ tra cứu.
`action`	Enum	Bắt buộc	Loại thao tác đã được thực hiện.
`before`	JSONB	Có thể rỗng	Snapshot của đối tượng trước khi thay đổi.
`after`	JSONB	Có thể rỗng	Snapshot của đối tượng sau khi thay đổi.
`createdAt`	Timestamp	Bắt buộc	Thời điểm sự kiện audit được ghi nhận.

Thuộc tính `targetType` cho phép phân loại đối tượng bị tác động, gồm các giá trị như:

- `PROJECT`;

- ENVIRONMENT;
- FEATURE_FLAG;
- FLAG_CONFIG;
- FLAG_RULE;
- FLAG_GROUP;
- SAMPLE_USER.

Thuộc tính action mô tả cụ thể thao tác đã xảy ra. Một số action tiêu biểu gồm:

- PROJECT_CREATED, PROJECT_UPDATED;
- ENVIRONMENT_CREATED, ENVIRONMENT_UPDATED;
- FEATURE_FLAG_CREATED, FEATURE_FLAG_UPDATED;
- FEATURE_FLAG_ARCHIVED, FEATURE_FLAG_RESTORED;
- FLAG_CONFIG_UPDATED;
- FLAG_RULE_CREATED, FLAG_RULE_UPDATED, FLAG_RULE_DELETED;
- FLAG_GROUP_CREATED, FLAG_GROUP_UPDATED;
- GROUP_KILL_SWITCH_ENABLED, GROUP_KILL_SWITCH_DISABLED.

Việc sử dụng enum cho targetType và action giúp thống nhất dữ liệu audit, tránh sai chính tả và hỗ trợ lọc lịch sử theo loại tài nguyên hoặc loại thao tác.

Thông tin actor và request

Thuộc tính actor xác định tác nhân đã thực hiện thao tác thay đổi. Tác nhân có thể là tài khoản quản trị, developer, tiến trình seed dữ liệu hoặc một dịch vụ nội bộ.

Trong các API mutation, hệ thống yêu cầu thông tin actor trước khi cho phép thay đổi dữ liệu. Actor có thể được lấy từ thông tin xác thực của request hoặc từ header được chuẩn hóa trong phạm vi mini project. Giá trị actor sau đó được truyền xuyên suốt từ controller, service đến lớp ghi audit log.

Thông tin actor giúp trả lời các câu hỏi:

- ai đã bật hoặc tắt flag;
- ai đã thay đổi phần trăm rollout;
- ai đã thêm hoặc xóa rule;
- ai đã kích hoạt kill switch;
- ai đã archive hoặc khôi phục flag.

Thuộc tính `requestId` định danh request tạo ra thay đổi. Mỗi request được gán một giá trị duy nhất và giá trị này được sử dụng trong response lỗi, application log và audit log.

Việc duy trì cùng một request ID giữa các thành phần giúp liên kết:

- request nhận tại Backend API;
- log xử lý trong ứng dụng;
- lỗi phát sinh trong transaction;
- bản ghi audit được tạo;
- phản hồi trả về dashboard.

Ví dụ một bản ghi audit có thể chứa:

Đoạn mã 2.19: Thông tin actor và request trong audit log

```
{
  "actor": "admin@example.com",
  "requestId": "req-8a729f31",
  "targetType": "FLAG_CONFIG",
  "targetKey": "new-checkout",
  "action": "FLAG_CONFIG_UPDATED"
}
```

Actor và request ID phải được ghi nhận từ phía server. Client không được tự quyết định toàn bộ nội dung audit log vì có thể dẫn đến việc giả mạo thông tin truy vết.

Snapshot trước và sau thay đổi

Hai thuộc tính `before` và `after` lưu trạng thái của đối tượng trước và sau thao tác dưới dạng `JSONB`. Cách lưu này cho phép audit log biểu diễn nhiều loại tài nguyên có cấu trúc khác nhau trong cùng một bảng [15].

Ý nghĩa của snapshot phụ thuộc vào loại thao tác:

- thao tác tạo mới: `before = NULL`, `after` chứa dữ liệu vừa tạo;
- thao tác cập nhật: `before` chứa dữ liệu cũ, `after` chứa dữ liệu mới;
- thao tác xóa: `before` chứa dữ liệu trước khi xóa, `after = NULL`;
- thao tác archive hoặc restore: cả hai snapshot chứa trạng thái vòng đời tương ứng.

Ví dụ snapshot khi thay đổi percentage rollout từ 20% lên 40%:

Đoạn mã 2.20: Snapshot trước và sau khi cập nhật rule

```
{
  "before": {
    "id": "rule-001",
    "type": "PERCENTAGE_ROLLOUT",
    "priority": 30,
    "parameters": {
      "percentage": 20
    }
  },
  "after": {
    "id": "rule-001",
    "type": "PERCENTAGE_ROLLOUT",
    "priority": 30,
    "parameters": {
      "percentage": 40
    }
  }
}
```

Snapshot cần được chuẩn hóa trước khi ghi vào trường JSON. Các giá trị thời gian được chuyển sang chuỗi ISO 8601, giá trị `undefined` bị loại bỏ và các kiểu dữ liệu không tương thích với JSON phải được chuyển đổi phù hợp.

Audit snapshot không nên chứa thông tin nhạy cảm như mật khẩu, token xác thực, khóa bí mật hoặc toàn bộ dữ liệu cá nhân không liên quan. Việc ghi log phải bảo đảm đủ dữ liệu để truy vết nhưng không làm tăng nguy cơ lộ thông tin [12].

Các snapshot là bản mô tả lịch sử, không được sử dụng trực tiếp làm nguồn cấu hình runtime. Evaluation Engine luôn đọc dữ liệu hiện tại từ các bảng cấu hình hoặc từ cache tương ứng.

Tính chất append-only

Audit log có tính chất append-only, nghĩa là hệ thống chỉ được thêm bản ghi mới và không cho phép cập nhật hoặc xóa các bản ghi đã tồn tại. Tính chất này giúp duy trì tính toàn vẹn của lịch sử thay đổi và hạn chế việc chỉnh sửa dấu vết sau khi thao tác đã xảy ra.

Các nguyên tắc của audit log append-only gồm:

- mỗi thay đổi tạo một bản ghi audit mới;
- không cung cấp API cập nhật audit log;
- không cung cấp API xóa từng bản ghi audit;
- thao tác đọc chỉ hỗ trợ tìm kiếm, lọc và phân trang;
- tầng ứng dụng không sử dụng audit log như bảng dữ liệu có thể chỉnh sửa.

Ngoài việc giới hạn ở tầng API, hệ thống có thể sử dụng trigger trong PostgreSQL để ngăn thao tác `UPDATE` và `DELETE` trực tiếp trên bảng audit log.

Đoạn mã 2.21: Hàm ngăn cập nhật và xóa audit log

```
CREATE OR REPLACE FUNCTION prevent_audit_log_mutation()
RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION
        'Audit log entries are append-only';
END;
$$ LANGUAGE plpgsql;
```

Trigger bảo vệ bảng được định nghĩa như sau:

Đoạn mã 2.22: Trigger bảo đảm audit log có tính chất append-only

```
CREATE TRIGGER prevent_audit_log_update
BEFORE UPDATE ON audit_log_entries
FOR EACH ROW
EXECUTE FUNCTION prevent_audit_log_mutation();

CREATE TRIGGER prevent_audit_log_delete
BEFORE DELETE ON audit_log_entries
FOR EACH ROW
EXECUTE FUNCTION prevent_audit_log_mutation();
```

Bản ghi audit nên được tạo trong cùng transaction với thay đổi dữ liệu nghiệp vụ. Nếu thao tác cập nhật cấu hình thất bại, bản ghi audit tương ứng cũng không được lưu. Ngược lại, nếu transaction thành công, thay đổi và dấu vết audit phải cùng tồn tại.

Cách tổ chức này tránh hai trạng thái không nhất quán:

- cấu hình đã thay đổi nhưng không có audit log;
- audit log ghi nhận thay đổi nhưng dữ liệu nghiệp vụ thực tế không được cập nhật.

Tính chất append-only không đồng nghĩa với việc dữ liệu audit phải được lưu vô thời hạn. Trong hệ thống thực tế có thể áp dụng chính sách lưu trữ hoặc chuyển dữ liệu cũ sang kho lưu trữ riêng. Tuy nhiên, việc này nằm ngoài phạm vi của mini project.

Versioning cấu hình

Versioning được sử dụng để xác định phiên bản hiện tại của một `FlagConfig`. Thuộc tính `version` là số nguyên dương và được tăng sau mỗi thay đổi có thể ảnh hưởng đến kết quả evaluation.

Các thay đổi làm tăng phiên bản gồm:

- thay đổi `status`;
- thay đổi `servingMode`;

- bật hoặc tắt flag-level kill switch;
- tạo rule mới;
- cập nhật loại, priority hoặc tham số rule;
- xóa rule;
- thay đổi thứ tự ưu tiên của rule.

Giả sử cấu hình ban đầu có:

$$\texttt{version} = 1$$

Sau khi thay đổi serving mode, phiên bản được cập nhật thành:

$$\texttt{version} = 2$$

Nếu tiếp tục cập nhật percentage rollout, phiên bản mới là:

$$\texttt{version} = 3$$

Việc tăng phiên bản phải được thực hiện trong cùng transaction với thao tác cập nhật cấu hình hoặc rule. Điều này bảo đảm không tồn tại trường hợp dữ liệu đã thay đổi nhưng version vẫn giữ nguyên.

Ví dụ câu lệnh cập nhật phiên bản:

Đoạn mã 2.23: Tăng phiên bản cấu hình sau khi thay đổi

```
UPDATE flag_configs
SET
    version = version + 1,
    updated_at = CURRENT_TIMESTAMP
WHERE id = :flag_config_id;
```

Version có thể được sử dụng cho các mục đích:

- xác định cấu hình đã thay đổi kể từ lần đọc trước;
- tạo cache key hoặc kiểm tra tính hợp lệ của cache;
- hỗ trợ cache invalidation;
- hiển thị lịch sử thay đổi trên dashboard;
- phát hiện xung đột cập nhật đồng thời;
- liên kết một evaluation snapshot với phiên bản cấu hình cụ thể.

Một cache key có thể bao gồm phiên bản cấu hình:

```
projectKey:environmentKey:flagKey:version
```

Khi version tăng, cache cũ không còn được sử dụng cho lần evaluation tiếp theo. Cách tiếp cận này giảm nguy cơ trả về kết quả từ cấu hình đã lỗi thời.

Audit log và versioning không thay thế lẫn nhau. Version chỉ cho biết cấu hình đã thay đổi bao nhiêu lần hoặc trạng thái hiện tại thuộc phiên bản nào. Audit log cung cấp thông tin chi tiết về actor, request, action và snapshot dữ liệu.

Quan hệ giữa hai cơ chế được trình bày trong Bảng 2.24.

Bảng 2.24: Vai trò của audit log và versioning

Cơ chế	**Mục đích chính**	**Thông tin cung cấp**
Audit log	Truy vết và kiểm tra lịch sử thay đổi.	Actor, request ID, action, thời điểm, snapshot trước và sau.
Versioning	Xác định phiên bản hiện tại của cấu hình.	Số phiên bản dùng cho consistency, cache và kiểm soát cập nhật.

Trong phạm vi mini project, hệ thống lưu phiên bản hiện tại trên `FlagConfig` và sử dụng `AuditLogEntry` làm lịch sử chi tiết. Thiết kế này tránh phải sao chép toàn bộ cấu hình thành một bảng phiên bản riêng, đồng thời vẫn đáp ứng yêu cầu truy vết và quản lý thay đổi.

2.5.8 Ràng buộc toàn vẹn dữ liệu

Các ràng buộc toàn vẹn dữ liệu được áp dụng nhằm bảo đảm cấu hình Feature Flag luôn hợp lệ, nhất quán và có thể được Evaluation Engine xử lý theo cách xác định. Hệ thống kết hợp

ràng buộc tại cơ sở dữ liệu với validation ở tầng ứng dụng. Cơ sở dữ liệu chịu trách nhiệm bảo vệ các quy tắc cấu trúc như khóa chính, khóa ngoại và tính duy nhất, trong khi tầng ứng dụng kiểm tra các điều kiện nghiệp vụ phụ thuộc vào loại rule hoặc quan hệ giữa nhiều thực thể.

Ràng buộc khóa và tính duy nhất

Mỗi thực thể sử dụng trường `id` làm khóa chính. Khóa chính được sinh dưới dạng UUID nhằm cung cấp định danh duy nhất, không phụ thuộc vào thứ tự tạo bản ghi và phù hợp với việc trao đổi dữ liệu giữa các thành phần của hệ thống.

Bên cạnh khóa chính, các khóa nghiệp vụ cần được giới hạn tính duy nhất trong phạm vi tương ứng. Các ràng buộc chính được trình bày trong Bảng 2.25.

Bảng 2.25: Các ràng buộc duy nhất trong mô hình dữ liệu

Thực thể	Thuộc tính duy nhất	Ý nghĩa
`Project`	`key`	Không tồn tại hai project có cùng khóa.
`Environment`	`projectId, key`	Khóa environment là duy nhất trong từng project.
`FeatureFlag`	`projectId, key`	Khóa flag là duy nhất trong từng project.
`FlagGroup`	`projectId, key`	Khóa group là duy nhất trong từng project.
`FlagConfig`	`featureFlagId, environmentId`	Mỗi flag chỉ có một cấu hình trong một environment.
`FlagRule`	`flagConfigId, priority`	Không có hai rule cùng priority trong một cấu hình.
`SampleUser`	`projectId, targetingKey`	Targeting key là duy nhất trong phạm vi project.

Ví dụ, ràng buộc duy nhất cho Feature Flag được định nghĩa như sau:

Đoạn mã 2.24: Ràng buộc khóa Feature Flag duy nhất trong project

```
ALTER TABLE feature_flags
ADD CONSTRAINT unique_feature_flag_key_per_project
UNIQUE (project_id, key);
```

Các khóa nghiệp vụ như project key, environment key và flag key phải tuân theo quy tắc định dạng thống nhất. Trong hệ thống, khóa chỉ chứa chữ thường, chữ số và dấu gạch ngang, có độ dài từ 3 đến 64 ký tự:

Đoạn mã 2.25: Biểu thức kiểm tra định dạng khóa nghiệp vụ

```
^[a-z0-9][a-z0-9-]{2,63}$
```

Việc giới hạn định dạng giúp các khóa có thể được sử dụng ổn định trong URL, cache key, log và request evaluation.

Ràng buộc quan hệ tham chiếu

Khóa ngoại được sử dụng để bảo đảm các thực thể con luôn tham chiếu đến thực thể cha hợp lệ. Các quan hệ tham chiếu chính gồm:

- `Environment.projectId` tham chiếu đến `Project.id`;
- `FeatureFlag.projectId` tham chiếu đến `Project.id`;
- `FlagGroup.projectId` tham chiếu đến `Project.id`;
- `FeatureFlag.groupId` tham chiếu đến `FlagGroup.id`;
- `FlagConfig.featureFlagId` tham chiếu đến `FeatureFlag.id`;
- `FlagConfig.environmentId` tham chiếu đến `Environment.id`;
- `FlagRule.flagConfigId` tham chiếu đến `FlagConfig.id`;
- `SampleUser.projectId` tham chiếu đến `Project.id`;
- `AuditLogEntry.projectId` tham chiếu đến `Project.id`.

Một số quy tắc nghiệp vụ không thể được bảo đảm hoàn toàn chỉ bằng khóa ngoại đơn. Ví dụ, khi tạo `FlagConfig`, Feature Flag và environment phải thuộc cùng một project:

$$\texttt{FeatureFlag.projectId} = \texttt{Environment.projectId}$$

Tương tự, khi gán Feature Flag vào group, hai thực thể phải thuộc cùng project:

$$\texttt{FeatureFlag.projectId} = \texttt{FlagGroup.projectId}$$

Các điều kiện này được kiểm tra tại tầng dịch vụ trước khi ghi dữ liệu. Thao tác kiểm tra và tạo bản ghi phải nằm trong cùng transaction để hạn chế dữ liệu thay đổi giữa thời điểm validation

và thời điểm ghi.

Chính sách xử lý khi thực thể cha bị xóa phụ thuộc vào ý nghĩa của quan hệ:

- khi xóa `FlagConfig`, các `FlagRule` thuộc cấu hình có thể bị xóa cascade;
- khi xóa `FlagGroup`, trường `groupId` của các flag thành viên được đặt thành `NULL`;
- Feature Flag và project không nên bị xóa vật lý trong luồng quản trị thông thường;
- audit log không bị xóa cascade cùng tài nguyên nghiệp vụ.

Đối với environment mặc định, hệ thống sử dụng chỉ mục duy nhất có điều kiện để bảo đảm một project chỉ có tối đa một environment với `isDefault = true`:

Đoạn mã 2.26: Ràng buộc một environment mặc định trong mỗi project

```
CREATE UNIQUE INDEX unique_default_environment_per_project
ON environments (project_id)
WHERE is_default = TRUE;
```

Ràng buộc giá trị phần trăm rollout

Đối với rule `PERCENTAGE_ROLLOUT`, tham số `percentage` phải là một giá trị số nằm trong khoảng từ 0 đến 100:

$$0 \leq \texttt{percentage} \leq 100$$

Ý nghĩa của hai giá trị biên được xác định như sau:

- `percentage = 0`: không có đối tượng nào khớp rule;
- `percentage = 100`: mọi đối tượng có targeting key hợp lệ đều khớp rule.

Vì tham số rule được lưu trong trường `JSONB`, validation chính được thực hiện tại tầng ứng dụng trước khi lưu. Hệ thống phải kiểm tra:

- trường `percentage` tồn tại;
- giá trị có kiểu số;
- giá trị không phải `NaN` hoặc vô hạn;

- giá trị nằm trong khoảng từ 0 đến 100;
- object không chứa cấu trúc không được hỗ trợ.

Ví dụ payload hợp lệ:

Đoạn mã 2.27: Tham số percentage rollout hợp lệ

```
{
  "percentage": 25
}
```

Ví dụ các giá trị không hợp lệ gồm:

Đoạn mã 2.28: Các tham số percentage rollout không hợp lệ

```
{ "percentage": -10 }
{ "percentage": 120 }
{ "percentage": "25" }
{ "percentage": null }
```

Ngoài validation tại tầng ứng dụng, PostgreSQL có thể sử dụng `CHECK` constraint dựa trên nội dung JSONB để tăng mức bảo vệ dữ liệu:

Đoạn mã 2.29: Ràng buộc giá trị percentage trong tham số JSONB

```
ALTER TABLE flag_rules
ADD CONSTRAINT valid_percentage_rollout
CHECK (
    type <> 'PERCENTAGE_ROLLOUT'
    OR (
        parameters ? 'percentage'
        AND jsonb_typeof(parameters->'percentage') = 'number'
        AND (parameters->>'percentage')::numeric >= 0
        AND (parameters->>'percentage')::numeric <= 100
    )
);
```

Ràng buộc cơ sở dữ liệu đóng vai trò bảo vệ cuối cùng. Tầng API vẫn phải validation trước để trả về thông báo lỗi rõ ràng thay vì phụ thuộc hoàn toàn vào lỗi constraint của PostgreSQL.

Ràng buộc không sửa hoặc xóa audit log

Bảng `AuditLogEntry` có tính chất append-only. Sau khi bản ghi được tạo, hệ thống không cho phép sửa hoặc xóa thông qua API quản trị.

Các biện pháp bảo vệ gồm:

- repository audit log chỉ cung cấp thao tác tạo và đọc;
- API không định nghĩa endpoint update hoặc delete audit log;
- quyền truy cập cơ sở dữ liệu của ứng dụng được giới hạn phù hợp;
- trigger PostgreSQL từ chối mọi thao tác `UPDATE` và `DELETE`.

Hàm trigger được định nghĩa như sau:

Đoạn mã 2.30: Hàm ngăn thay đổi bản ghi audit log

```
CREATE OR REPLACE FUNCTION prevent_audit_log_mutation()
RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION
        'Audit log entries are append-only';
END;
$$ LANGUAGE plpgsql;
```

Trigger được áp dụng cho cả thao tác cập nhật và xóa:

Đoạn mã 2.31: Trigger bảo vệ bảng audit log

```
CREATE TRIGGER prevent_audit_log_update
BEFORE UPDATE ON audit_log_entries
FOR EACH ROW
EXECUTE FUNCTION prevent_audit_log_mutation();

CREATE TRIGGER prevent_audit_log_delete
BEFORE DELETE ON audit_log_entries
FOR EACH ROW
EXECUTE FUNCTION prevent_audit_log_mutation();
```

Audit log phải được ghi trong cùng transaction với thay đổi dữ liệu nghiệp vụ. Transaction chỉ được commit khi cả thao tác cập nhật và thao tác tạo audit log đều thành công.

Quy tắc này có thể biểu diễn như sau:

$$\texttt{Commit} \iff \texttt{BusinessChangeSucceeded} \land \texttt{AuditLogCreated}$$

Nếu không thể ghi audit log, toàn bộ transaction phải rollback để tránh tồn tại thay đổi cấu hình không có dấu vết truy vết.

Chiến lược archive thay cho xóa vật lý

Đối với Feature Flag, hệ thống ưu tiên archive thay cho xóa vật lý. Khi không còn sử dụng, trạng thái `lifecycleStatus` của flag được chuyển từ `ACTIVE` sang `ARCHIVED`.

Archive không xóa các dữ liệu liên quan gồm:

- định nghĩa Feature Flag;
- các `FlagConfig` theo environment;
- danh sách `FlagRule`;
- quan hệ với Flag Group;
- audit log;
- phiên bản cấu hình;
- dữ liệu thống kê đã phát sinh.

Khi flag ở trạng thái `ARCHIVED`, Evaluation Engine trả về:

- `enabled = false`;
- variant tắt;
- reason code `FLAG_ARCHIVED`.

Archive mang lại các lợi ích sau:

- bảo toàn lịch sử thay đổi và khả năng truy vết;
- tránh phá vỡ quan hệ khóa ngoại;

- hạn chế việc tạo lại flag có cùng khóa trong khi ứng dụng cũ vẫn tham chiếu;
- hỗ trợ khôi phục flag nếu thao tác archive được thực hiện nhầm;
- giúp nhận diện các flag không còn sử dụng để xử lý feature flag debt.

Thao tác archive và restore đều phải tạo audit log. Ví dụ:

- archive sử dụng action `FEATURE_FLAG_ARCHIVED`;
- khôi phục sử dụng action `FEATURE_FLAG_RESTORED`.

Không nên cho phép chỉnh sửa cấu hình hoặc tạo rule mới cho flag đã archive. Nếu cần sử dụng lại flag, người quản trị phải restore flag về trạng thái `ACTIVE` trước khi thực hiện các thay đổi tiếp theo.

Xóa vật lý chỉ nên được sử dụng trong các trường hợp giới hạn như dữ liệu seed, dữ liệu kiểm thử hoặc quy trình quản trị đặc biệt. Thao tác này không thuộc luồng sử dụng thông thường của dashboard và phải được kiểm soát chặt chẽ để tránh làm mất lịch sử cấu hình.

2.6 Thiết kế Rule Evaluation Engine

2.6.1 Evaluation request và context

Evaluation request là dữ liệu đầu vào được ứng dụng khách gửi đến Evaluation API để yêu cầu xác định trạng thái của một feature flag trong một ngữ cảnh cụ thể. Request phải cung cấp đủ thông tin để hệ thống định danh project, môi trường, feature flag và đối tượng đang được đánh giá. Phần thông tin mô tả đối tượng đánh giá được gọi là *evaluation context*.

Evaluation context có thể bao gồm targeting key, mã người dùng, vai trò, mã tổ chức và các thuộc tính mở rộng. Các thông tin này không trực tiếp quyết định kết quả mà được Rule Evaluation Engine sử dụng để đối chiếu với các rule đã cấu hình. Cách tổ chức này giúp tách biệt dữ liệu của ứng dụng khách khỏi logic đánh giá của nền tảng feature flag, đồng thời cho phép bổ sung các chiến lược targeting mới mà không cần thay đổi cấu trúc tổng thể của Evaluation API [6].

Trong hệ thống đề xuất, evaluation request có cấu trúc khái quát như sau:

Đoạn mã 2.32: Cấu trúc evaluation request

```
{
```

```
  "projectKey": "ecommerce-platform",
  "environmentKey": "production",
  "flagKey": "new-checkout",
  "context": {
    "targetingKey": "user-001",
    "userId": "user-001",
    "roles": ["beta-customer"],
    "organizationId": "organization-a",
    "attributes": {
      "country": "VN",
      "deviceType": "mobile"
    }
  }
}
```

Project key, environment key và flag key

Ba trường `projectKey`, `environmentKey` và `flagKey` được sử dụng để xác định chính xác cấu hình cần đánh giá.

`projectKey` định danh project chứa feature flag. Việc đặt feature flag trong phạm vi project giúp phân tách cấu hình giữa các ứng dụng hoặc miền nghiệp vụ khác nhau. Hai project có thể sử dụng cùng một `flagKey` mà không gây xung đột, do feature flag luôn được tra cứu trong phạm vi của một project cụ thể.

`environmentKey` xác định môi trường mà ứng dụng khách đang hoạt động, chẳng hạn như `development`, `staging` hoặc `production`. Một feature flag có thể có cấu hình khác nhau giữa các môi trường. Ví dụ, tính năng có thể được bật toàn cục trong môi trường staging nhưng chỉ được rollout cho một phần người dùng trong môi trường production. Mô hình cấu hình flag theo môi trường là cách tiếp cận phổ biến trong các nền tảng feature flag nhằm hỗ trợ kiểm thử và phát hành độc lập giữa các môi trường [7].

Trong trường hợp request không cung cấp `environmentKey`, hệ thống sử dụng môi trường mặc định của project. Mỗi project chỉ được phép có tối đa một môi trường mặc định nhằm bảo đảm kết quả tra cứu có tính xác định.

`flagKey` định danh feature flag cần đánh giá. Đây là khóa ổn định được ứng dụng khách sử dụng trong mã nguồn và không nên phụ thuộc vào tên hiển thị của flag. Việc sử dụng khóa định danh ổn định giúp tránh phải thay đổi mã tích hợp khi tên mô tả của feature flag được cập nhật.

Bộ ba khóa được tra cứu theo thứ tự:

1. Tìm project theo `projectKey`.
2. Tìm environment theo `environmentKey` hoặc lấy environment mặc định.
3. Tìm feature flag theo `flagKey` trong project.
4. Tải cấu hình và tập rule tương ứng với feature flag trong environment.

Nếu project, environment hoặc feature flag không tồn tại, Evaluation API trả về kết quả fail-closed với `enabled = false` và mã lý do phù hợp, thay vì tự động bật tính năng.

Targeting key

`targetingKey` là định danh ổn định đại diện cho đối tượng đang được đánh giá. Đối tượng này thường là người dùng, nhưng cũng có thể là thiết bị, phiên làm việc, tài khoản doanh nghiệp hoặc tổ chức tùy theo mô hình của ứng dụng khách.

Targeting key có hai vai trò chính. Thứ nhất, nó có thể được sử dụng để đối chiếu trực tiếp với rule cho phép một tập đối tượng cụ thể. Thứ hai, targeting key là dữ liệu đầu vào của thuật toán băm ổn định khi thực hiện percentage rollout.

Đối với percentage rollout, hệ thống không lựa chọn ngẫu nhiên lại ở mỗi request. Thay vào đó, targeting key được kết hợp với project key và flag key để tạo đầu vào cho hàm băm. Kết quả băm được ánh xạ vào một bucket phần trăm cố định. Nhờ đó, cùng một đối tượng thường nhận được cùng một kết quả trong các lần đánh giá liên tiếp, miễn là targeting key và cấu hình rollout không thay đổi. Cơ chế phân nhóm ổn định này giúp tránh hiện tượng một người dùng liên tục chuyển đổi giữa trạng thái bật và tắt [5].

Trong hệ thống đề xuất, giá trị băm được tạo từ chuỗi:

$$H = \mathrm{SHA256}\left(projectKey \parallel flagKey \parallel targetingKey\right) \tag{2.1}$$

Trong đó, ký hiệu || biểu diễn phép nối chuỗi. Một phần của kết quả băm được chuyển thành giá trị bucket trong khoảng từ 0 đến nhỏ hơn 100. Feature flag được bật theo percentage rollout nếu bucket nhỏ hơn tỷ lệ phần trăm đã cấu hình.

Targeting key phải ổn định trong phạm vi ứng dụng khách. Các giá trị thay đổi thường xuyên, chẳng hạn như access token, địa chỉ IP tạm thời hoặc mã phiên đăng nhập, không phù hợp để sử dụng làm targeting key vì có thể làm thay đổi bucket của cùng một người dùng.

Nếu request không cung cấp targeting key, các rule phụ thuộc vào định danh đối tượng hoặc percentage rollout không thể được đánh giá. Khi đó, engine bỏ qua các rule này và tiếp tục với rule tiếp theo hoặc trả về trạng thái mặc định.

User ID, role và organization ID

Bên cạnh targeting key tổng quát, evaluation context có thể chứa các trường nghiệp vụ cụ thể gồm `userId`, `roles` và `organizationId`.

`userId` là định danh người dùng trong hệ thống của ứng dụng khách. Trường này được sử dụng để hỗ trợ chiến lược bật tính năng cho một danh sách người dùng cụ thể. Ví dụ, đội phát triển có thể bật một giao diện mới cho tài khoản kiểm thử nội bộ hoặc một nhóm khách hàng beta trước khi phát hành rộng rãi.

`roles` là danh sách các vai trò nghiệp vụ của người dùng, chẳng hạn như `admin`, `beta-customer` hoặc `regular-customer`. Rule Evaluation Engine kiểm tra giao giữa danh sách role trong context và danh sách role được cấu hình trong rule. Rule được xem là khớp khi tồn tại ít nhất một role chung giữa hai tập hợp. Targeting theo role giúp kiểm soát tính năng theo nhóm quyền hoặc nhóm khách hàng mà không cần liệt kê từng user ID [4].

`organizationId` định danh tổ chức, doanh nghiệp hoặc tenant mà người dùng thuộc về. Thuộc tính này cho phép phát hành tính năng cho toàn bộ thành viên của một tổ chức cụ thể. Đây là trường hợp hữu ích đối với các ứng dụng đa tenant, trong đó một tính năng thường được kích hoạt theo hợp đồng hoặc theo nhóm khách hàng doanh nghiệp thay vì theo từng người dùng riêng lẻ.

Các định danh này thuộc miền dữ liệu của ứng dụng khách. Nền tảng feature flag không tự xác thực ý nghĩa nghiệp vụ của chúng mà chỉ sử dụng các giá trị được truyền vào để đánh giá rule. Do đó, ứng dụng tích hợp chịu trách nhiệm xây dựng context từ nguồn dữ liệu đáng tin cậy.

Đối với các quyết định có ảnh hưởng đến phân quyền hoặc dữ liệu nhạy cảm, không nên tin cậy context được gửi trực tiếp từ trình duyệt mà chưa được backend của ứng dụng xác minh.

Trong phiên bản hiện tại, `targetingKey` là trường định danh tổng quát dùng cho stable rollout, trong khi `userId`, `roles` và `organizationId` cung cấp thông tin rõ nghĩa cho từng loại targeting. Khi phù hợp, ứng dụng khách có thể sử dụng cùng một giá trị cho `targetingKey` và `userId`.

Thuộc tính mở rộng

Trường `attributes` chứa tập các thuộc tính tùy chỉnh dưới dạng các cặp khóa–giá trị. Các thuộc tính này cho phép evaluation context biểu diễn thêm thông tin nghiệp vụ mà không phải thay đổi contract của Evaluation API mỗi khi xuất hiện một yêu cầu targeting mới.

Một số thuộc tính mở rộng có thể được sử dụng gồm:

- `country`: quốc gia hoặc khu vực của người dùng;
- `deviceType`: loại thiết bị như desktop, mobile hoặc tablet;
- `appVersion`: phiên bản ứng dụng khách;
- `subscriptionPlan`: gói dịch vụ của người dùng;
- `accountAge`: thời gian tồn tại của tài khoản.

Việc sử dụng thuộc tính mở rộng tạo cơ sở để phát triển các rule tổng quát hơn trong tương lai, chẳng hạn như so sánh bằng, kiểm tra tập hợp, so sánh phiên bản hoặc so sánh giá trị số. Evaluation context theo mô hình mở rộng cũng phù hợp với cách tiếp cận của OpenFeature, trong đó context có thể chứa targeting key và các thuộc tính bổ sung phục vụ quá trình đánh giá [6].

Trong phạm vi mini project, thuộc tính mở rộng được lưu dưới dạng đối tượng JSON và được truyền nguyên vẹn đến engine. Engine chỉ đọc các thuộc tính cần thiết cho rule đang được đánh giá. Cách tổ chức này giúp duy trì API ổn định và cho phép mở rộng rule engine mà không cần bổ sung cột dữ liệu riêng cho mọi thuộc tính nghiệp vụ.

Tuy nhiên, context không nên chứa mật khẩu, access token, thông tin thanh toán hoặc dữ liệu cá nhân không cần thiết. Chỉ những thuộc tính thực sự phục vụ quyết định feature flag mới nên được gửi đến Evaluation API. Việc giới hạn dữ liệu context giúp giảm rủi ro rò rỉ thông tin và tránh làm Evaluation API phụ thuộc quá mức vào dữ liệu nghiệp vụ của ứng dụng khách.

Validation evaluation request

Validation được thực hiện trước khi tải cấu hình và gọi Rule Evaluation Engine. Mục tiêu của bước này là từ chối sớm các request không hợp lệ, bảo đảm engine chỉ xử lý dữ liệu có cấu trúc xác định và tránh các truy vấn không cần thiết đến cơ sở dữ liệu.

Các trường `projectKey` và `flagKey` là bắt buộc. `environmentKey` có thể được bỏ qua khi project đã có môi trường mặc định. Các khóa định danh phải là chuỗi không rỗng và tuân theo quy tắc:

```
^[a-z0-9][a-z0-9-]{2,63}$
```
(2.2)

Quy tắc này yêu cầu khóa có độ dài từ 3 đến 64 ký tự, bắt đầu bằng chữ cái thường hoặc chữ số, đồng thời chỉ chứa chữ cái thường, chữ số và dấu gạch ngang. Việc chuẩn hóa định dạng khóa giúp tránh khác biệt chữ hoa–chữ thường, giảm lỗi tích hợp và đơn giản hóa quá trình tra cứu.

Đối tượng `context` là tùy chọn vì một số trường hợp như global on, flag disabled hoặc kill switch không cần thông tin người dùng. Tuy nhiên, khi context được cung cấp, các trường bên trong phải thỏa mãn các yêu cầu sau:

- `targetingKey`, `userId` và `organizationId`, nếu xuất hiện, phải là chuỗi không rỗng;
- `roles` phải là một mảng các chuỗi không rỗng và không chứa giá trị trùng lặp;
- `attributes` phải là một đối tượng JSON hợp lệ;
- kích thước và độ sâu của `attributes` phải được giới hạn để tránh request quá lớn;
- các trường không được khai báo trong API contract phải bị loại bỏ hoặc từ chối tùy theo chính sách validation.

Request không hợp lệ được trả về với mã lỗi `VALIDATION_ERROR`, thông điệp mô tả ngắn gọn và danh sách chi tiết các trường vi phạm. Cấu trúc lỗi thống nhất giúp SDK hoặc ứng dụng khách xử lý lỗi mà không phụ thuộc vào thông điệp dạng văn bản.

Evaluation API không sử dụng giá trị mặc định tùy ý để sửa một request sai cấu trúc. Ví dụ, một mảng role không hợp lệ không được tự động chuyển thành chuỗi hoặc ngược lại. Nguyên

tắc validation chặt chẽ giúp kết quả evaluation có tính xác định và làm rõ trách nhiệm giữa ứng dụng khách với nền tảng feature flag.

Sau khi request vượt qua validation, hệ thống chuẩn hóa dữ liệu bằng cách loại bỏ khoảng trắng không cần thiết, chuẩn hóa danh sách role và tạo evaluation context nội bộ. Context đã chuẩn hóa sau đó được truyền đến Rule Evaluation Engine cùng snapshot cấu hình của feature flag. Evaluation API cần trả về kết quả có cấu trúc ổn định, bao gồm trạng thái flag, variant, mã lý do và rule đã khớp nếu có [10].

2.6.2 Evaluation snapshot

Evaluation snapshot là tập dữ liệu cấu hình đầy đủ và nhất quán được Rule Evaluation Engine sử dụng để đánh giá một feature flag tại một thời điểm xác định. Snapshot được xây dựng sau khi evaluation request đã vượt qua bước validation và trước khi engine thực hiện các kiểm tra về trạng thái flag, kill switch, serving mode và targeting rule.

Việc tạo snapshot giúp tách quá trình truy xuất dữ liệu khỏi quá trình đánh giá. Repository chịu trách nhiệm đọc và chuẩn hóa cấu hình từ cơ sở dữ liệu, trong khi Rule Evaluation Engine chỉ xử lý một cấu trúc dữ liệu đầu vào đã hoàn chỉnh. Cách tổ chức này làm giảm sự phụ thuộc của engine vào tầng lưu trữ, hỗ trợ kiểm thử đơn vị và giúp kết quả evaluation có tính xác định.

Khái niệm evaluation snapshot

Một evaluation snapshot có thể được xem là ảnh chụp logic của trạng thái cấu hình liên quan đến một lần đánh giá. Snapshot không phải là bản sao vật lý của toàn bộ cơ sở dữ liệu mà chỉ bao gồm những dữ liệu cần thiết cho feature flag được yêu cầu trong project và environment tương ứng.

Trong hệ thống đề xuất, snapshot được tạo theo bộ khóa gồm:

$$S = f(projectKey, environmentKey, flagKey) \tag{2.3}$$

Trong đó, S là evaluation snapshot và f là quá trình truy xuất, liên kết và chuẩn hóa dữ liệu cấu hình.

Snapshot phải cung cấp đủ thông tin để engine có thể đưa ra kết quả mà không cần thực hiện thêm truy vấn trong quá trình evaluation. Điều này đặc biệt quan trọng vì một lần đánh giá có

thể phải kiểm tra nhiều tầng điều kiện, từ trạng thái vòng đời của flag đến các rule targeting theo thứ tự ưu tiên.

Cấu trúc khái quát của evaluation snapshot trong hệ thống được biểu diễn như sau:

Đoạn mã 2.33: Cấu trúc khái quát của evaluation snapshot

```
{
  "project": {
    "id": "project-id",
    "key": "ecommerce-platform"
  },
  "environment": {
    "id": "environment-id",
    "key": "production",
    "isDefault": false
  },
  "flag": {
    "id": "flag-id",
    "key": "new-checkout",
    "lifecycleStatus": "ACTIVE"
  },
  "config": {
    "status": "ENABLED",
    "servingMode": "TARGETED",
    "killSwitch": false,
    "variant": "on",
    "version": 12
  },
  "group": {
    "id": "group-id",
    "killSwitch": false
  },
  "rules": [
```

```
    {
      "id": "rule-1",
      "type": "USER_ALLOWLIST",
      "priority": 10,
      "parameters": {
        "userIds": ["user-001", "user-002"]
      }
    },
    {
      "id": "rule-2",
      "type": "ROLE_TARGETING",
      "priority": 20,
      "parameters": {
        "roles": ["beta-customer"]
      }
    }
  ]
}
```

Snapshot là cấu trúc chỉ đọc trong phạm vi một lần evaluation. Rule Evaluation Engine không được phép thay đổi dữ liệu trong snapshot hoặc ghi ngược thay đổi vào cơ sở dữ liệu. Các side effect như ghi thống kê số lần evaluation được xử lý ở bước riêng, sau khi engine đã tạo kết quả.

Dữ liệu cấu hình cần truy xuất

Để xây dựng evaluation snapshot, hệ thống cần truy xuất các nhóm dữ liệu sau:

- thông tin project được xác định bởi `projectKey`;
- thông tin environment được xác định bởi `environmentKey` hoặc environment mặc định;
- thông tin feature flag được xác định bởi `flagKey`;
- cấu hình của feature flag trong environment tương ứng;
- nhóm flag mà feature flag thuộc về, nếu có;

- danh sách rule đang hoạt động của cấu hình;
- phiên bản cấu hình phục vụ kiểm soát consistency và cache invalidation.

Thông tin project và environment được sử dụng để xác lập đúng phạm vi đánh giá. Feature flag cung cấp trạng thái vòng đời, chẳng hạn như `ACTIVE` hoặc `ARCHIVED`. Cấu hình theo environment cung cấp trạng thái runtime gồm `ENABLED`, `DISABLED`, `GLOBAL_ON`, `TARGETED` và kill switch.

Nếu hệ thống hỗ trợ group kill switch, snapshot phải bao gồm trạng thái của flag group. Group kill switch được đánh giá trước các rule targeting nhằm cho phép tắt nhanh toàn bộ nhóm tính năng khi có sự cố.

Mỗi rule trong snapshot cần chứa tối thiểu các trường sau:

- `id`: định danh rule phục vụ truy vết;
- `type`: loại rule như user allowlist, role targeting hoặc percentage rollout;
- `priority`: thứ tự đánh giá;
- `parameters`: tập tham số phụ thuộc loại rule;
- trạng thái hoạt động của rule, nếu mô hình dữ liệu có hỗ trợ.

Trường `parameters` được biểu diễn dưới dạng JSON nhằm hỗ trợ nhiều cấu trúc rule khác nhau mà không cần tạo một bảng riêng cho từng loại. PostgreSQL cung cấp kiểu dữ liệu JSONB phù hợp cho việc lưu trữ, kiểm tra và truy vấn các cấu trúc bán linh hoạt như rule parameters [15].

Repository phải kiểm tra tính đầy đủ của các quan hệ khi xây dựng snapshot. Nếu feature flag tồn tại nhưng không có cấu hình tương ứng trong environment, hệ thống không được tự động suy diễn trạng thái bật. Thay vào đó, evaluation phải trả về trạng thái tắt an toàn hoặc lỗi cấu hình phù hợp.

Những dữ liệu không ảnh hưởng đến quyết định evaluation, chẳng hạn như mô tả dài, thông tin giao diện hoặc toàn bộ audit log, không cần được đưa vào snapshot. Việc giới hạn dữ liệu giúp giảm chi phí truy xuất, giảm kích thước cache và đơn giản hóa Rule Evaluation Engine.

Sắp xếp rule trước khi evaluation

Các rule phải được sắp xếp trước khi truyền vào Rule Evaluation Engine. Trong hệ thống đề xuất, thứ tự được xác định chủ yếu bởi trường `priority`. Rule có giá trị priority nhỏ hơn được

đánh giá trước.

Với tập rule $R = \{r_1, r_2, \ldots, r_n\}$, thứ tự đánh giá phải thỏa mãn:

$$priority(r_i) \leq priority(r_{i+1}) \tag{2.4}$$

Việc sắp xếp rule trước evaluation bảo đảm rằng engine xử lý các rule theo đúng ý định của người quản trị. Khi nhiều rule cùng có khả năng khớp với một context, rule xuất hiện trước sẽ quyết định kết quả theo nguyên tắc *first match wins*. Cách tiếp cận đánh giá rule theo thứ tự là mô hình phổ biến trong các nền tảng feature flag có hỗ trợ targeting rule [4].

Repository có thể yêu cầu cơ sở dữ liệu trả rule theo thứ tự:

Đoạn mã 2.34: Truy xuất rule theo thứ tự ưu tiên

```
ORDER BY priority ASC, created_at ASC, id ASC
```

Ngoài `priority`, hệ thống cần một tiêu chí phụ ổn định như thời điểm tạo hoặc định danh rule. Tiêu chí phụ này xử lý trường hợp nhiều rule có cùng priority và ngăn thứ tự kết quả thay đổi giữa các lần truy vấn.

Tuy nhiên, tốt hơn hết hệ thống nên ràng buộc priority là duy nhất trong phạm vi một flag config. Khi đó, mỗi rule có một vị trí xác định và người quản trị có thể thay đổi thứ tự thông qua thao tác cập nhật priority.

Việc sắp xếp rule trong repository thay vì bên trong từng hàm đánh giá giúp Rule Evaluation Engine nhận được dữ liệu đã chuẩn hóa. Engine chỉ cần duyệt tuần tự qua danh sách rule và dừng tại rule đầu tiên khớp.

Thứ tự giữa các trạng thái hệ thống và rule targeting không được quyết định bằng trường priority. Các điều kiện như archived, disabled, group kill switch, flag kill switch và global on thuộc precedence toàn cục của engine và luôn được kiểm tra trước danh sách rule.

Tính nhất quán của snapshot

Tính nhất quán của snapshot yêu cầu toàn bộ dữ liệu được sử dụng trong một lần evaluation phải đại diện cho một trạng thái cấu hình hợp lệ và không bị trộn lẫn giữa các phiên bản khác nhau.

Ví dụ, nếu flag config được cập nhật đồng thời với quá trình evaluation, hệ thống không nên tạo snapshot chứa trạng thái `servingMode` của phiên bản cũ nhưng danh sách rule của phiên bản mới. Snapshot như vậy có thể dẫn đến kết quả không phản ánh bất kỳ cấu hình hợp lệ nào đã từng tồn tại.

Trong phạm vi mini project, tính nhất quán được bảo đảm thông qua các biện pháp sau:

- truy xuất flag config và rule trong cùng một thao tác repository;
- sử dụng quan hệ khóa ngoại để bảo đảm rule thuộc đúng config;
- gắn trường version hoặc `updatedAt` vào cấu hình;
- chỉ trả snapshot sau khi mọi thành phần cần thiết đã được tải thành công;
- coi snapshot là bất biến trong suốt quá trình evaluation.

Nếu repository sử dụng nhiều truy vấn riêng biệt, các truy vấn nên được thực hiện trong cùng một database transaction với mức cô lập phù hợp. Điều này giảm khả năng đọc phải dữ liệu trung gian trong khi cấu hình đang được cập nhật.

Một phương án đơn giản hơn là sử dụng một truy vấn quan hệ duy nhất để tải project, environment, feature flag, config, group và rule. ORM có thể ánh xạ kết quả truy vấn này thành một snapshot nội bộ trước khi truyền cho engine.

Khi sử dụng cache, cache entry phải đại diện cho toàn bộ snapshot hoặc cho một phiên bản cấu hình xác định. Cache key có thể được xây dựng theo cấu trúc:

$$K = projectKey \parallel environmentKey \parallel flagKey \parallel version \tag{2.5}$$

Khi cấu hình hoặc rule thay đổi, version phải được tăng hoặc cache entry tương ứng phải bị invalidation. Nếu không có cơ chế invalidation phù hợp, Evaluation API có thể tiếp tục trả về kết quả từ cấu hình cũ. Các hệ thống feature flag thường phải cân bằng giữa tính nhất quán, độ trễ và khả năng sẵn sàng khi thiết kế cơ chế phân phối cấu hình và cache [8].

Trong trường hợp không thể tạo snapshot đầy đủ hoặc phát hiện dữ liệu cấu hình không nhất quán, hệ thống áp dụng nguyên tắc fail-closed. Evaluation API trả về `enabled = false` với mã lý do `ERROR` hoặc mã lỗi cấu hình tương ứng, thay vì tiếp tục đánh giá trên dữ liệu không đáng tin cậy.

2.6.3 Thứ tự ưu tiên khi đánh giá

Rule Evaluation Engine đánh giá feature flag theo một thứ tự ưu tiên cố định nhằm bảo đảm kết quả có tính xác định. Các điều kiện kiểm soát an toàn và trạng thái hệ thống được xử lý trước các rule targeting. Khi một điều kiện ở mức ưu tiên cao hơn đã quyết định kết quả, engine dừng ngay quá trình đánh giá và không tiếp tục kiểm tra các điều kiện phía sau.

Thứ tự ưu tiên được áp dụng trong hệ thống như sau:

1. flag đã được archive;
2. flag bị disabled;
3. group kill switch;
4. flag kill switch;
5. global on;
6. user allowlist;
7. role targeting;
8. percentage rollout;
9. default off.

Thứ tự này thể hiện nguyên tắc các trạng thái vô hiệu hóa và cơ chế ngắt khẩn cấp luôn có quyền ưu tiên cao hơn các chiến lược phát hành. Nhờ đó, hệ thống có thể dừng cung cấp tính năng ngay cả khi một hoặc nhiều rule targeting vẫn đang khớp với evaluation context.

Cấu trúc khái quát của quá trình đánh giá được biểu diễn như sau:

Đoạn mã 2.35: Thứ tự ưu tiên của Rule Evaluation Engine

```
if (flag.lifecycleStatus === "ARCHIVED") {
  return disabled("FLAG_ARCHIVED");
}

if (config.status === "DISABLED") {
  return disabled("FLAG_DISABLED");
}
```

```
if (group?.killSwitch === true) {
  return disabled("GROUP_KILL_SWITCH");
}

if (config.killSwitch === true) {
  return disabled("KILL_SWITCH");
}

if (config.servingMode === "GLOBAL_ON") {
  return enabled("GLOBAL_ON");
}

if (matchesUserAllowlist(context, rules)) {
  return enabled("USER_MATCH");
}

if (matchesRoleTargeting(context, rules)) {
  return enabled("ROLE_MATCH");
}

if (matchesPercentageRollout(context, rules)) {
  return enabled("PERCENTAGE_MATCH");
}

return disabled("DEFAULT_OFF");
```

Flag đã được archive

Trạng thái archive có mức ưu tiên cao nhất trong quá trình evaluation. Một feature flag đã được archive được xem là không còn tham gia vào hoạt động phát hành thông thường và không được phép trả về trạng thái bật.

Khi `lifecycleStatus` của flag bằng `ARCHIVED`, engine trả về:

Đoạn mã 2.36: Kết quả khi feature flag đã được archive

```
{
  "enabled": false,
  "variant": "off",
  "reason": "FLAG_ARCHIVED"
}
```

Engine không tiếp tục kiểm tra trạng thái disabled, kill switch, serving mode hoặc targeting rule. Điều này ngăn một flag đã bị loại khỏi vòng đời sử dụng vô tình được kích hoạt lại bởi cấu hình cũ còn tồn tại trong cơ sở dữ liệu.

Archive được ưu tiên thay cho xóa vật lý để bảo toàn lịch sử cấu hình, audit log và các quan hệ tham chiếu. Nếu cần sử dụng lại flag, người quản trị phải thực hiện thao tác restore rõ ràng trước khi flag có thể tiếp tục được đánh giá.

Flag bị disabled

Sau khi xác nhận flag chưa được archive, engine kiểm tra trạng thái cấu hình của flag trong environment hiện tại. Nếu `status` bằng `DISABLED`, feature flag bị tắt hoàn toàn trong environment đó.

Kết quả evaluation được trả về với:

Đoạn mã 2.37: Kết quả khi cấu hình flag bị disabled

```
{
  "enabled": false,
  "variant": "off",
  "reason": "FLAG_DISABLED"
}
```

Trạng thái disabled có hiệu lực cao hơn group kill switch, flag kill switch, global on và toàn bộ targeting rule. Mặc dù kill switch cũng dẫn đến kết quả tắt, mã lý do `FLAG_DISABLED` được ưu tiên vì đây là trạng thái cấu hình trực tiếp đã được xác định trước khi xét các cơ chế ngắt khẩn cấp.

Việc tách biệt `DISABLED` và kill switch giúp phản ánh đúng nguyên nhân vận hành. Disabled thường thể hiện quyết định cấu hình thông thường, trong khi kill switch được sử dụng để phản ứng nhanh khi xảy ra sự cố.

Group kill switch

Group kill switch cho phép tắt đồng thời toàn bộ feature flag thuộc cùng một nhóm. Cơ chế này hữu ích khi nhiều flag cùng phục vụ một chức năng lớn hoặc cùng phụ thuộc vào một thành phần đang gặp lỗi.

Nếu flag thuộc một group có `killSwitch = true`, engine trả về:

Đoạn mã 2.38: Kết quả khi group kill switch được kích hoạt

```
{
  "enabled": false,
  "variant": "off",
  "reason": "GROUP_KILL_SWITCH"
}
```

Group kill switch được đánh giá trước flag kill switch và các rule riêng của từng flag. Nhờ đó, một thao tác ở cấp nhóm có thể vô hiệu hóa toàn bộ phạm vi tính năng mà không cần cập nhật từng cấu hình riêng lẻ.

Nếu flag không thuộc group hoặc group không kích hoạt kill switch, engine tiếp tục đánh giá flag kill switch. Group kill switch không làm thay đổi dữ liệu rule, serving mode hoặc trạng thái cấu hình; nó chỉ ghi đè kết quả evaluation tại thời điểm chạy.

Flag kill switch

Flag kill switch là cơ chế ngắt khẩn cấp ở cấp một feature flag cụ thể. Khi `killSwitch = true`, engine phải trả về trạng thái tắt bất kể flag đang ở chế độ global on hay có rule targeting nào khớp.

Kết quả evaluation có dạng:

Đoạn mã 2.39: Kết quả khi flag kill switch được kích hoạt

```
{
```

```
  "enabled": false,
  "variant": "off",
  "reason": "KILL_SWITCH"
}
```

Kill switch được thiết kế để hỗ trợ rollback vận hành nhanh mà không cần deploy lại ứng dụng. Đây là một trong những use case quan trọng của feature flag, đặc biệt khi một tính năng mới gây lỗi trên production hoặc tạo ảnh hưởng ngoài dự kiến [1].

Kill switch không thay thế trạng thái disabled. Disabled phù hợp với việc tắt tính năng theo cấu hình thông thường, trong khi kill switch biểu diễn một biện pháp ưu tiên cao, thường được sử dụng trong tình huống khẩn cấp.

Global on

Nếu flag đang active, config đang enabled và không có kill switch nào được kích hoạt, engine kiểm tra serving mode. Khi `servingMode = GLOBAL_ON`, feature flag được bật cho mọi evaluation request trong environment hiện tại.

Kết quả trả về là:

Đoạn mã 2.40: Kết quả khi feature flag được bật toàn cục

```
{
  "enabled": true,
  "variant": "on",
  "reason": "GLOBAL_ON"
}
```

Ở chế độ global on, evaluation context không bắt buộc phải chứa targeting key, user ID hoặc role. Engine không tiếp tục kiểm tra user allowlist, role targeting hoặc percentage rollout vì trạng thái bật đã được quyết định ở mức toàn cục.

Việc đặt global on trước các targeting rule giúp tránh tình trạng một rule cũ vô tình làm thay đổi kết quả khi người quản trị đã chủ động phát hành tính năng cho toàn bộ người dùng.

User allowlist

Nếu flag sử dụng chế độ targeted, engine bắt đầu đánh giá danh sách rule. User allowlist được kiểm tra trước role targeting và percentage rollout vì đây là chiến lược chỉ định đối tượng cụ thể nhất.

Rule user allowlist khớp khi `userId` hoặc targeting key trong evaluation context thuộc danh sách định danh được cấu hình trong rule. Ví dụ:

Đoạn mã 2.41: Ví dụ tham số của user allowlist rule

```
{
  "type": "USER_ALLOWLIST",
  "priority": 10,
  "parameters": {
    "userIds": ["user-001", "user-002"]
  }
}
```

Khi rule khớp, engine trả về:

Đoạn mã 2.42: Kết quả khi user allowlist khớp

```
{
  "enabled": true,
  "variant": "on",
  "reason": "USER_MATCH",
  "matchedRuleId": "rule-id"
}
```

User allowlist phù hợp cho tài khoản kiểm thử nội bộ, người dùng beta hoặc một tập khách hàng được lựa chọn trước. Nếu request không cung cấp định danh cần thiết, rule được xem là không khớp và engine tiếp tục với rule tiếp theo.

Role targeting

Role targeting được đánh giá sau user allowlist. Rule này cho phép bật tính năng cho một nhóm người dùng có cùng vai trò nghiệp vụ mà không cần liệt kê từng định danh riêng lẻ.

Rule khớp khi tồn tại ít nhất một phần tử chung giữa danh sách role trong evaluation context và danh sách role được cấu hình:

$$context.roles \cap rule.roles \neq \varnothing \tag{2.6}$$

Ví dụ, nếu context chứa role `beta-customer` và rule cho phép role tương ứng, engine trả về:

Đoạn mã 2.43: Kết quả khi role targeting khớp

```
{
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-id"
}
```

Các targeting rule thường được đánh giá theo thứ tự và dừng tại rule đầu tiên khớp [4]. Trong hệ thống đề xuất, nếu không có role nào khớp, engine tiếp tục kiểm tra percentage rollout.

Percentage rollout

Percentage rollout được đánh giá sau các rule targeting trực tiếp. Cách sắp xếp này bảo đảm người dùng được chỉ định rõ bằng user allowlist hoặc role targeting không bị phụ thuộc vào bucket phần trăm.

Để đánh giá percentage rollout, request phải cung cấp targeting key ổn định. Engine kết hợp project key, flag key và targeting key để tạo giá trị băm, sau đó ánh xạ giá trị này vào bucket trong khoảng từ 0 đến nhỏ hơn 100.

Giả sử b là bucket của đối tượng và p là tỷ lệ rollout được cấu hình, rule khớp khi:

$$b < p \tag{2.7}$$

Ví dụ, với rollout 25%, các đối tượng có bucket nhỏ hơn 25 được bật tính năng. Cơ chế stable hashing giúp cùng một targeting key tiếp tục thuộc cùng một bucket giữa các lần evaluation [5].

Khi percentage rollout khớp, kết quả có dạng:

Đoạn mã 2.44: Kết quả khi percentage rollout khớp

```
{
  "enabled": true,
  "variant": "on",
  "reason": "PERCENTAGE_MATCH",
  "matchedRuleId": "rule-id"
}
```

Nếu request thiếu targeting key, percentage rule không thể được đánh giá và được xem là không khớp. Engine không sử dụng giá trị ngẫu nhiên thay thế vì điều đó sẽ làm mất tính ổn định của kết quả.

Default off

Default off là kết quả cuối cùng khi không có điều kiện nào trước đó bật feature flag. Trạng thái này được áp dụng khi flag đang active và enabled, không có kill switch, không ở chế độ global on và không có targeting rule nào khớp.

Kết quả evaluation được trả về như sau:

Đoạn mã 2.45: Kết quả mặc định khi không có rule nào khớp

```
{
  "enabled": false,
  "variant": "off",
  "reason": "DEFAULT_OFF"
}
```

Default off thể hiện nguyên tắc fail-closed của hệ thống. Feature flag chỉ được bật khi có một điều kiện rõ ràng cho phép. Cách tiếp cận này giảm nguy cơ phát hành ngoài ý muốn khi context thiếu dữ liệu, rule bị cấu hình sai hoặc chưa có rule phù hợp.

Mã lý do `DEFAULT_OFF` khác với `FLAG_DISABLED`, `FLAG_ARCHIVED` và `KILL_SWITCH`. Nó cho biết flag vẫn hợp lệ và có thể được đánh giá, nhưng evaluation context hiện tại không thỏa mãn điều kiện bật nào.

2.6.4 Đánh giá trạng thái toàn cục

Trước khi đánh giá các rule targeting, Rule Evaluation Engine phải xác định trạng thái toàn cục của feature flag trong environment hiện tại. Bước này dựa trên hai nhóm thông tin chính gồm trạng thái cấu hình và serving mode.

Trạng thái cấu hình xác định feature flag có được phép tham gia evaluation hay không. Serving mode xác định cách feature flag được cung cấp khi cấu hình đang hoạt động. Việc tách hai khái niệm này giúp phân biệt rõ giữa quyết định tắt hoàn toàn một flag và quyết định bật flag theo toàn bộ người dùng hoặc theo điều kiện targeting.

Trong hệ thống đề xuất, cấu hình của một feature flag có thể được mô hình hóa như sau:

Đoạn mã 2.46: Cấu trúc trạng thái toàn cục của flag config

```
{
  "status": "ENABLED",
  "servingMode": "TARGETED",
  "killSwitch": false
}
```

Quá trình đánh giá trạng thái toàn cục được thực hiện sau khi kiểm tra trạng thái archive và các kill switch có mức ưu tiên cao hơn. Nếu trạng thái toàn cục đã xác định được kết quả, engine không tiếp tục đánh giá các rule phía sau.

Trạng thái disabled

Trạng thái `DISABLED` biểu thị rằng cấu hình feature flag đã bị vô hiệu hóa trong một environment cụ thể. Khi trạng thái này được thiết lập, feature flag phải trả về kết quả tắt bất kể serving mode hoặc các rule targeting đang được cấu hình như thế nào.

Điều kiện đánh giá được biểu diễn như sau:

$$config.status = \texttt{DISABLED} \Rightarrow enabled = false \tag{2.8}$$

Kết quả trả về có cấu trúc:

Đoạn mã 2.47: Kết quả khi flag config ở trạng thái disabled

```
{
  "enabled": false,
  "variant": "off",
  "reason": "FLAG_DISABLED"
}
```

Khi gặp trạng thái disabled, engine không cần kiểm tra serving mode. Một cấu hình có `status = DISABLED` và `servingMode = GLOBAL_ON` vẫn phải trả về trạng thái tắt vì status có mức ưu tiên cao hơn serving mode.

Trạng thái disabled thường được sử dụng trong các trường hợp sau:

- feature flag chưa sẵn sàng để được sử dụng trong environment;
- đội phát triển muốn tạm dừng một chiến lược rollout;
- cấu hình cần được giữ lại để chỉnh sửa nhưng chưa được phép phục vụ;
- tính năng đã kết thúc giai đoạn thử nghiệm nhưng chưa được archive.

Disabled không làm mất các rule đã cấu hình. Khi flag được chuyển lại sang trạng thái `ENABLED`, các rule và serving mode trước đó có thể tiếp tục được sử dụng. Việc này khác với archive, vốn thể hiện thay đổi ở cấp vòng đời của feature flag.

Serving mode global on

Serving mode `GLOBAL_ON` biểu thị rằng feature flag được bật cho mọi đối tượng trong environment hiện tại. Chế độ này chỉ có hiệu lực khi config đang ở trạng thái `ENABLED` và không có kill switch nào được kích hoạt.

Điều kiện đánh giá có thể biểu diễn như sau:

$$config.status = \texttt{ENABLED} \land config.servingMode = \texttt{GLOBAL_ON} \Rightarrow enabled = true \tag{2.9}$$

Kết quả evaluation được trả về với mã lý do `GLOBAL_ON`:

Đoạn mã 2.48: Kết quả khi serving mode là global on

```
{
  "enabled": true,
  "variant": "on",
  "reason": "GLOBAL_ON"
}
```

Trong chế độ global on, evaluation context không ảnh hưởng đến kết quả. Request không bắt buộc phải chứa user ID, role, organization ID hoặc targeting key. Engine cũng không cần duyệt danh sách rule vì tính năng đã được phát hành cho toàn bộ phạm vi environment.

Global on phù hợp với các trường hợp:

- tính năng đã hoàn tất rollout và được mở cho toàn bộ người dùng;
- tính năng cần được bật mặc định trong một environment nội bộ;
- đội phát triển muốn bỏ qua tạm thời toàn bộ rule targeting;
- feature flag được sử dụng như một công tắc bật hoặc tắt toàn cục.

Việc hỗ trợ trạng thái bật toàn cục là một trong những khả năng cơ bản của hệ thống feature flag, cho phép thay đổi hành vi runtime mà không cần triển khai lại mã nguồn [1].

Serving mode targeted

Serving mode `TARGETED` biểu thị rằng feature flag không được bật mặc định cho toàn bộ người dùng. Thay vào đó, kết quả phụ thuộc vào evaluation context và các rule đã được cấu hình.

Điều kiện để engine chuyển sang bước đánh giá rule là:

$$config.status = \texttt{ENABLED} \land config.servingMode = \texttt{TARGETED} \tag{2.10}$$

Khi đó, engine lần lượt kiểm tra các rule như user allowlist, role targeting và percentage rollout theo thứ tự ưu tiên. Nếu một rule khớp, feature flag được bật. Nếu không có rule nào khớp, engine trả về default off.

Luồng xử lý được mô tả như sau:

Đoạn mã 2.49: Xử lý serving mode targeted

```
if (config.status === "ENABLED" &&
    config.servingMode === "TARGETED") {
  return evaluateRules(snapshot.rules, context);
}
```

Chế độ targeted hỗ trợ các chiến lược phát hành có kiểm soát, chẳng hạn như:

- bật tính năng cho một nhóm người dùng thử nghiệm;
- bật theo role hoặc organization;
- rollout dần theo tỷ lệ phần trăm;
- kết hợp nhiều nhóm đối tượng theo thứ tự rule.

Cách tiếp cận targeting theo context cho phép hệ thống kiểm soát phạm vi phát hành mà không cần thay đổi logic ứng dụng khách. Các nền tảng feature flag thường sử dụng evaluation context và targeting rule để đưa ra quyết định runtime cho từng đối tượng cụ thể [6, 4].

Trong chế độ targeted, việc thiếu context không được xem là lỗi trong mọi trường hợp. Engine chỉ coi các rule phụ thuộc vào trường bị thiếu là không khớp. Nếu không có rule nào có thể quyết định trạng thái bật, kết quả cuối cùng là `DEFAULT_OFF`.

Tương tác giữa trạng thái và serving mode

Trạng thái và serving mode có vai trò khác nhau trong quá trình evaluation. Trạng thái xác định cấu hình có đang hoạt động hay không, trong khi serving mode xác định cách tính năng được cung cấp khi cấu hình đang hoạt động.

Quan hệ giữa hai thuộc tính được tổng hợp trong Bảng 2.26.

Bảng 2.26: Tương tác giữa trạng thái và serving mode

Status	**Serving mode**	**Kết quả trực tiếp**	**Hành vi của engine**
`DISABLED`	`GLOBAL_ON`	Tắt	Bỏ qua serving mode và toàn bộ rule, trả về `FLAG_DISABLED`.
`DISABLED`	`TARGETED`	Tắt	Không đánh giá targeting rule, trả về `FLAG_DISABLED`.
`ENABLED`	`GLOBAL_ON`	Bật	Trả về `GLOBAL_ON`, không yêu cầu evaluation context.
`ENABLED`	`TARGETED`	Chưa xác định	Tiếp tục đánh giá user allowlist, role targeting và percentage rollout.

Từ bảng trên, có thể xác định quy tắc ưu tiên tổng quát:

$$status > servingMode > targetingRules \tag{2.11}$$

Điều này có nghĩa là status được đánh giá trước serving mode, còn serving mode được đánh giá trước các rule targeting. Một cấu hình disabled không thể được bật bởi global on hoặc bởi rule. Ngược lại, một cấu hình enabled và global on không cần dựa vào bất kỳ rule nào.

Ngoài status và serving mode, các kill switch vẫn có mức ưu tiên cao hơn global on và targeted. Do đó, điều kiện đầy đủ để global on trả về trạng thái bật là:

$$\begin{aligned} & lifecycleStatus = \texttt{ACTIVE} \\ \wedge\ & config.status = \texttt{ENABLED} \\ \wedge\ & groupKillSwitch = false \\ \wedge\ & flagKillSwitch = false \\ \wedge\ & servingMode = \texttt{GLOBAL_ON} \end{aligned} \tag{2.12}$$

Việc mô hình hóa riêng status và serving mode giúp giao diện quản trị thể hiện rõ ý nghĩa của từng thao tác. Người quản trị có thể vô hiệu hóa tạm thời toàn bộ cấu hình bằng status hoặc giữ config ở trạng thái enabled và chuyển đổi giữa global on với targeted tùy theo giai đoạn

phát hành.

2.6.5 Targeting theo người dùng

Targeting theo người dùng cho phép feature flag được bật cho một tập đối tượng cụ thể thay vì áp dụng cho toàn bộ người dùng. Trong hệ thống đề xuất, cơ chế này được triển khai thông qua rule `USER_ALLOWLIST`. Rule chứa danh sách các định danh được cho phép và được đánh giá dựa trên `targetingKey` hoặc `userId` trong evaluation context.

User targeting được áp dụng sau khi engine đã kiểm tra trạng thái archive, disabled, group kill switch, flag kill switch và global on. Nếu flag sử dụng serving mode `TARGETED`, engine duyệt các rule theo thứ tự ưu tiên và trả về trạng thái bật ngay khi tìm thấy user allowlist rule khớp.

User allowlist

User allowlist là danh sách các định danh người dùng được phép truy cập tính năng. Cơ chế này phù hợp với các trường hợp cần phát hành cho một nhóm nhỏ đã biết trước, chẳng hạn như tài khoản kiểm thử nội bộ, người dùng beta, khách hàng ưu tiên hoặc các tài khoản đang tham gia chương trình thử nghiệm.

Một user allowlist rule có cấu trúc khái quát như sau:

Đoạn mã 2.50: Cấu trúc user allowlist rule

```
{
  "id": "rule-user-allowlist",
  "type": "USER_ALLOWLIST",
  "priority": 10,
  "parameters": {
    "targetingKeys": [
      "customer-001",
      "customer-002"
    ],
    "userIds": [
      "user-001",
      "user-002"
    ]
```

```
  }
}
```

Tùy theo API contract, hệ thống có thể lưu một danh sách định danh chung hoặc tách riêng `targetingKeys` và `userIds`. Việc tách riêng giúp biểu diễn rõ ý nghĩa của từng loại định danh và tránh nhầm lẫn khi targeting key không trùng với user ID.

Rule được xem là khớp nếu ít nhất một định danh trong evaluation context xuất hiện trong danh sách tương ứng của rule:

$$\begin{aligned} matched = context.targetingKey \in rule.targetingKeys \\ \vee \ context.userId \in rule.userIds \end{aligned} \tag{2.13}$$

Khi rule khớp, engine dừng đánh giá và trả về:

Đoạn mã 2.51: Kết quả khi user allowlist rule khớp

```
{
  "enabled": true,
  "variant": "on",
  "reason": "USER_MATCH",
  "matchedRuleId": "rule-user-allowlist"
}
```

User allowlist có mức ưu tiên cao hơn role targeting và percentage rollout vì đây là cách chỉ định đối tượng cụ thể hơn. Một người dùng nằm trong allowlist phải được bật tính năng ngay cả khi role của họ không khớp hoặc bucket phần trăm nằm ngoài phạm vi rollout.

Các hệ thống feature flag thường hỗ trợ targeting trực tiếp theo định danh người dùng để phát hành có chọn lọc và kiểm soát chính xác phạm vi ảnh hưởng [4].

So khớp targeting key

`targetingKey` là định danh tổng quát và ổn định của đối tượng đang được đánh giá. Trong nhiều trường hợp, đối tượng này là người dùng, nhưng cũng có thể là thiết bị, tài khoản, phiên thuê bao hoặc một thực thể nghiệp vụ khác.

Khi đánh giá user allowlist, engine thực hiện các bước sau:

1. Dọc `targetingKey` từ evaluation context.
2. Kiểm tra giá trị có tồn tại và không phải chuỗi rỗng.
3. Chuẩn hóa giá trị theo quy tắc của API.
4. So sánh chính xác với danh sách targeting key trong rule.
5. Trả về trạng thái khớp nếu tìm thấy giá trị tương ứng.

Phép so khớp targeting key trong hệ thống đề xuất là phép so sánh bằng chính xác:

$$normalize(context.targetingKey) = normalize(ruleTargetingKey) \tag{2.14}$$

Hệ thống không sử dụng phép so khớp gần đúng, tìm kiếm chuỗi con hoặc biểu thức chính quy cho user allowlist. Điều này giúp tránh trường hợp các định danh gần giống nhau bị coi là cùng một đối tượng.

Ví dụ, các giá trị `user-01`, `user-001` và `USER-001` được xem là khác nhau nếu API không có quy tắc chuẩn hóa chữ hoa và chữ thường. Do đó, ứng dụng khách phải sử dụng targeting key ổn định và nhất quán giữa các lần evaluation.

Đoạn mã giả cho quá trình so khớp có thể được biểu diễn như sau:

Đoạn mã 2.52: So khớp targeting key với user allowlist

```
function matchesTargetingKey(rule, context) {
  if (!context.targetingKey) {
    return false;
  }

  return rule.parameters.targetingKeys.includes(
    context.targetingKey
  );
}
```

Targeting key cũng được sử dụng trong percentage rollout. Tuy nhiên, hai cơ chế có ý nghĩa

khác nhau. User allowlist thực hiện so khớp trực tiếp, trong khi percentage rollout sử dụng targeting key làm đầu vào của hàm băm ổn định.

Theo mô hình evaluation context, targeting key nên định danh duy nhất đối tượng được đánh giá và phải ổn định trong phạm vi hệ thống tích hợp [6].

So khớp user ID

`userId` là định danh người dùng trong miền dữ liệu của ứng dụng khách. Trường này cung cấp ý nghĩa nghiệp vụ rõ ràng hơn targeting key khi rule được thiết kế riêng cho người dùng.

Engine chỉ thực hiện so khớp user ID đối với danh sách `userIds` trong user allowlist rule. Điều kiện khớp là:

$$context.userId \in rule.parameters.userIds \tag{2.15}$$

Đoạn mã giả được biểu diễn như sau:

Đoạn mã 2.53: So khớp user ID với user allowlist

```
function matchesUserId(rule, context) {
  if (!context.userId) {
    return false;
  }

  return rule.parameters.userIds.includes(
    context.userId
  );
}
```

Trong nhiều ứng dụng, `targetingKey` và `userId` có thể mang cùng một giá trị. Tuy nhiên, hệ thống vẫn tách hai trường vì targeting key có tính tổng quát hơn và có thể đại diện cho một thực thể không phải người dùng.

Khi cả hai trường cùng xuất hiện, engine có thể áp dụng phép OR:

$$matched = matchesTargetingKey(rule, context) \\ \vee\ matchesUserId(rule, context) \tag{2.16}$$

Cách xử lý này bảo đảm rule có thể khớp nếu một trong hai định danh đã được cấu hình. Tuy nhiên, hệ thống phải tránh việc sử dụng một danh sách định danh không phân loại khi targeting key và user ID thuộc các namespace khác nhau nhưng có thể trùng giá trị.

Ví dụ, targeting key `12345` có thể đại diện cho một organization, trong khi user ID `12345` đại diện cho một người dùng. Nếu hai loại định danh được lưu chung mà không phân biệt, engine có thể bật tính năng cho sai đối tượng.

Vì vậy, cấu trúc rule nên lưu rõ loại định danh hoặc sử dụng các trường riêng biệt. Điều này cũng giúp giao diện quản trị mô tả chính xác đối tượng đang được thêm vào allowlist.

Xử lý khi thiếu định danh người dùng

Evaluation context có thể không chứa `targetingKey`, `userId` hoặc cả hai. Trường hợp này không nhất thiết làm toàn bộ evaluation request trở thành không hợp lệ vì một số trạng thái như global on, disabled hoặc kill switch không cần thông tin người dùng.

Khi đánh giá user allowlist, engine áp dụng các quy tắc sau:

- nếu có `targetingKey`, engine kiểm tra danh sách targeting key;
- nếu có `userId`, engine kiểm tra danh sách user ID;
- nếu chỉ có một trong hai trường, engine chỉ thực hiện phép so khớp tương ứng;
- nếu không có cả hai trường, user allowlist rule được xem là không khớp;
- engine tiếp tục đánh giá role targeting hoặc percentage rollout nếu các rule đó có đủ dữ liệu đầu vào.

Đoạn mã giả tổng quát có thể được biểu diễn như sau:

Đoạn mã 2.54: Xử lý user allowlist khi thiếu định danh

```
function matchesUserAllowlist(rule, context) {
  const targetingKeyMatched =
```

```
    context.targetingKey !== undefined &&
    rule.parameters.targetingKeys.includes(
      context.targetingKey
    );

  const userIdMatched =
    context.userId !== undefined &&
    rule.parameters.userIds.includes(
      context.userId
    );

  return targetingKeyMatched || userIdMatched;
}
```

Hệ thống không tự sinh định danh ngẫu nhiên và không sử dụng các thuộc tính khác thay thế cho user ID. Việc tự suy diễn định danh có thể tạo ra kết quả không ổn định hoặc làm một người dùng được nhận diện khác nhau giữa các request.

Nếu không có rule nào khác khớp, kết quả cuối cùng là:

Đoạn mã 2.55: Kết quả khi thiếu định danh và không có rule nào khớp

```
{
  "enabled": false,
  "variant": "off",
  "reason": "DEFAULT_OFF"
}
```

Thiếu định danh chỉ nên được trả về dưới dạng validation error nếu API contract của một endpoint hoặc một use case cụ thể quy định trường đó là bắt buộc. Trong thiết kế tổng quát của Evaluation API, context có thể thiếu một số thuộc tính và engine phải coi các rule phụ thuộc vào thuộc tính đó là không khớp, thay vì làm thất bại toàn bộ quá trình evaluation.

Cách xử lý này phù hợp với nguyên tắc evaluation context chỉ cung cấp dữ liệu đầu vào cho từng rule. Mỗi rule chịu trách nhiệm xác định liệu context hiện tại có đủ thông tin để đánh giá hay không [10].

2.6.6 Targeting theo vai trò

Targeting theo vai trò cho phép hệ thống bật feature flag cho một hoặc nhiều nhóm người dùng có cùng vai trò nghiệp vụ. Thay vì phải liệt kê từng user ID trong allowlist, người quản trị có thể cấu hình một tập role như `admin`, `beta-customer` hoặc `regular-customer`. Rule Evaluation Engine sẽ đối chiếu tập role trong evaluation context với tập role được cấu hình trong rule để xác định kết quả.

Role targeting chỉ được đánh giá khi feature flag đang ở trạng thái hoạt động, config đang enabled, không có kill switch, serving mode là `TARGETED` và không có user allowlist rule có mức ưu tiên cao hơn đã khớp. Cơ chế này phù hợp với các trường hợp phát hành tính năng theo nhóm quyền, nhóm khách hàng hoặc phân khúc người dùng [4].

Cấu trúc role trong evaluation context

Trong evaluation context, role được biểu diễn dưới dạng một mảng chuỗi thay vì một giá trị đơn. Cách biểu diễn này cho phép một người dùng đồng thời mang nhiều vai trò trong ứng dụng khách.

Ví dụ:

Đoạn mã 2.56: Role trong evaluation context

```
{
  "targetingKey": "user-001",
  "userId": "user-001",
  "roles": [
    "beta-customer",
    "regular-customer"
  ]
}
```

Mỗi phần tử trong `roles` phải là một chuỗi không rỗng. Danh sách role nên được chuẩn hóa trước khi truyền vào engine bằng cách loại bỏ khoảng trắng ở đầu và cuối, loại bỏ giá trị trùng lặp và áp dụng quy tắc nhất quán về chữ hoa, chữ thường.

Một role targeting rule có cấu trúc khái quát như sau:

Đoạn mã 2.57: Cấu trúc role targeting rule

```
{
  "id": "rule-role-targeting",
  "type": "ROLE_TARGETING",
  "priority": 20,
  "parameters": {
    "roles": [
      "admin",
      "beta-customer"
    ]
  }
}
```

Tập role trong rule biểu diễn các vai trò được phép truy cập tính năng. Evaluation context do ứng dụng khách cung cấp, trong khi rule được quản lý bởi nền tảng feature flag. Engine chỉ thực hiện phép đối chiếu giữa hai tập dữ liệu này và không tự suy diễn quyền hạn từ tên role.

Role trong evaluation context thuộc miền nghiệp vụ của ứng dụng tích hợp, không phải role quản trị của nền tảng feature flag. Ví dụ, role `beta-customer` trong demo app dùng để phân nhóm khách hàng, trong khi role `admin` của dashboard dùng để kiểm soát quyền thay đổi cấu hình. Hai hệ role này cần được tách biệt về ngữ nghĩa và phạm vi sử dụng.

So khớp một hoặc nhiều role

Trong hệ thống đề xuất, role targeting sử dụng phép so khớp theo nguyên tắc OR. Rule được xem là khớp nếu tồn tại ít nhất một role xuất hiện đồng thời trong evaluation context và trong tập role của rule.

Gọi C là tập role trong context và R là tập role trong rule. Điều kiện khớp được biểu diễn như sau:

$$C \cap R \neq \varnothing \tag{2.17}$$

Ví dụ, với:

$$C = \{\texttt{regular-customer}, \texttt{beta-customer}\} \tag{2.18}$$

và:

$$R = \{\texttt{admin}, \texttt{beta-customer}\} \tag{2.19}$$

hai tập có phần tử chung là `beta-customer`, do đó rule khớp.

Đoạn mã giả cho phép so khớp được biểu diễn như sau:

Đoạn mã 2.58: So khớp role theo nguyên tắc OR

```
function matchesRoleTargeting(rule, context) {
  if (!Array.isArray(context.roles) ||
      context.roles.length === 0) {
    return false;
  }

  return context.roles.some((contextRole) =>
    rule.parameters.roles.includes(contextRole)
  );
}
```

Khi rule khớp, engine trả về:

Đoạn mã 2.59: Kết quả khi role targeting khớp

```
{
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-role-targeting"
}
```

Nguyên tắc OR phù hợp với mục tiêu bật tính năng cho bất kỳ người dùng nào thuộc một trong các nhóm được chỉ định. Nếu cần yêu cầu người dùng đồng thời có nhiều role, hệ thống có thể

mở rộng rule bằng thuộc tính `matchMode` với các giá trị như `ANY` hoặc `ALL`. Tuy nhiên, trong phạm vi mini project, cơ chế `ANY` được lựa chọn để giữ logic đơn giản, rõ ràng và dễ kiểm thử.

Phép so sánh role phải là phép so sánh chính xác. Engine không nên coi `admin` là khớp với `super-admin` hoặc `administrator`. Việc sử dụng tìm kiếm chuỗi con có thể dẫn đến bật tính năng sai đối tượng.

Quan hệ giữa role targeting và user targeting

User targeting và role targeting đều thuộc nhóm rule dựa trên evaluation context, nhưng có mức độ cụ thể khác nhau. User targeting chỉ định trực tiếp một đối tượng cụ thể, trong khi role targeting áp dụng cho một nhóm người dùng có cùng vai trò.

Trong thứ tự ưu tiên của hệ thống, user allowlist được đánh giá trước role targeting:

$$USER_ALLOWLIST > ROLE_TARGETING \tag{2.20}$$

Thứ tự này bảo đảm một người dùng được chỉ định trực tiếp có thể nhận tính năng ngay cả khi role của họ không thuộc nhóm được cấu hình. Ví dụ, một người dùng thử nghiệm có role `regular-customer` vẫn có thể được bật tính năng nếu user ID của họ nằm trong allowlist.

Luồng xử lý được biểu diễn như sau:

Đoạn mã 2.60: Quan hệ ưu tiên giữa user targeting và role targeting

```
const userRule = findMatchingUserRule(rules, context);

if (userRule) {
  return enabled("USER_MATCH", userRule.id);
}

const roleRule = findMatchingRoleRule(rules, context);

if (roleRule) {
  return enabled("ROLE_MATCH", roleRule.id);
}
```

Nếu user targeting không khớp, engine mới tiếp tục đánh giá role targeting. Việc user rule không khớp không đồng nghĩa feature flag bị tắt ngay, vì người dùng vẫn có thể được bật thông qua role hoặc percentage rollout.

Trong trường hợp nhiều rule cùng loại tồn tại, thứ tự giữa chúng được xác định bởi trường `priority`. Engine duyệt theo thứ tự đã sắp xếp và dừng tại rule đầu tiên khớp. Cách tiếp cận *first match wins* giúp kết quả có tính xác định và phản ánh đúng thứ tự cấu hình [4].

User targeting và role targeting đều chỉ có chức năng bật flag trong mô hình hiện tại. Nếu không có rule nào khớp, engine tiếp tục đánh giá percentage rollout hoặc trả về default off. Hệ thống không sử dụng rule phủ định để tắt riêng một người dùng đã được bật bởi rule có mức ưu tiên cao hơn.

Xử lý khi context không chứa role

Evaluation context có thể không chứa trường `roles`, hoặc trường này có thể là một mảng rỗng. Trường hợp đó không làm toàn bộ evaluation request thất bại vì các trạng thái và rule khác có thể không cần dữ liệu role.

Khi context không có role, role targeting rule được xem là không khớp:

$$context.roles = \varnothing \Rightarrow roleMatched = false \tag{2.21}$$

Đoạn mã xử lý có thể được biểu diễn như sau:

Đoạn mã 2.61: Xử lý khi evaluation context không chứa role

```
function matchesRoleTargeting(rule, context) {
  const roles = context.roles;

  if (!Array.isArray(roles) || roles.length === 0) {
    return false;
  }

  return roles.some((role) =>
    rule.parameters.roles.includes(role)
```

```
  );
}
```

Engine không tự gán role mặc định và không suy diễn role từ user ID, organization ID hoặc targeting key. Việc suy diễn như vậy có thể làm kết quả phụ thuộc vào dữ liệu ngoài snapshot và phá vỡ tính thuần túy của Rule Evaluation Engine.

Sau khi role targeting không khớp do thiếu role, engine tiếp tục theo thứ tự ưu tiên:

1. kiểm tra percentage rollout nếu có targeting key;
2. trả về `DEFAULT_OFF` nếu không có rule nào khác khớp.

Ví dụ, context sau vẫn là một request hợp lệ:

Đoạn mã 2.62: Evaluation context không chứa role

```
{
  "targetingKey": "user-100",
  "userId": "user-100"
}
```

Trong trường hợp này, user allowlist và percentage rollout vẫn có thể được đánh giá. Chỉ các role targeting rule bị bỏ qua do thiếu dữ liệu đầu vào.

Nếu trường `roles` được cung cấp nhưng không phải mảng chuỗi hợp lệ, request phải bị từ chối ở bước validation thay vì được coi là không có role. Cần phân biệt rõ giữa dữ liệu hợp lệ nhưng bị thiếu và dữ liệu được cung cấp sai cấu trúc. Evaluation context có thể chứa tập thuộc tính linh hoạt, nhưng mỗi thuộc tính đã xuất hiện phải tuân thủ đúng kiểu dữ liệu được định nghĩa trong API contract [6, 10].

2.6.7 Rollout theo phần trăm người dùng

Percentage rollout là chiến lược phát hành cho phép bật feature flag đối với một tỷ lệ xác định trong tập người dùng thay vì phát hành đồng thời cho toàn bộ đối tượng. Cơ chế này hỗ trợ mở rộng phạm vi sử dụng theo từng giai đoạn, theo dõi ảnh hưởng của tính năng và giảm phạm vi sự cố nếu phiên bản mới phát sinh lỗi.

Trong hệ thống đề xuất, percentage rollout chỉ được đánh giá khi feature flag đang ở trạng thái

hoạt động, config đang enabled, không có kill switch, serving mode là `TARGETED` và không có user allowlist hoặc role targeting rule có mức ưu tiên cao hơn đã khớp. Engine sử dụng stable hashing để ánh xạ mỗi targeting key vào một bucket ổn định, sau đó so sánh bucket với tỷ lệ rollout được cấu hình.

Mục tiêu của percentage rollout

Mục tiêu chính của percentage rollout là kiểm soát phạm vi phát hành theo tỷ lệ mà vẫn duy trì kết quả ổn định đối với từng người dùng. Ví dụ, khi một feature flag được cấu hình rollout ở mức 20%, hệ thống cần bật tính năng cho xấp xỉ 20% tập targeting key hợp lệ và tắt đối với phần còn lại.

Percentage rollout hỗ trợ các trường hợp sau:

- phát hành canary cho một nhóm nhỏ người dùng trước khi mở rộng;
- tăng dần tỷ lệ từ 5%, 10%, 25% đến 100%;
- giới hạn phạm vi ảnh hưởng khi tính năng mới chưa được kiểm chứng đầy đủ;
- thu thập số liệu vận hành trước khi phát hành toàn cục;
- rollback bằng cách giảm tỷ lệ rollout hoặc kích hoạt kill switch.

Khác với lựa chọn ngẫu nhiên độc lập ở từng request, percentage rollout phải bảo đảm cùng một đối tượng nhận được kết quả nhất quán trong các lần đánh giá liên tiếp. Vì vậy, hệ thống không sử dụng hàm sinh số ngẫu nhiên tại runtime mà sử dụng hàm băm xác định trên targeting key [5].

Một percentage rollout rule có cấu trúc khái quát như sau:

Đoạn mã 2.63: Cấu trúc percentage rollout rule

```
{
  "id": "rule-percentage-rollout",
  "type": "PERCENTAGE_ROLLOUT",
  "priority": 30,
  "parameters": {
    "percentage": 25
  }
```

```
}
```

Giá trị `percentage` phải nằm trong khoảng từ 0 đến 100. Với 0%, rule không khớp với bất kỳ targeting key nào. Với 100%, mọi targeting key hợp lệ đều khớp.

Stable hashing

Stable hashing là cơ chế sử dụng một hàm băm xác định để ánh xạ cùng một dữ liệu đầu vào thành cùng một kết quả đầu ra. Nếu chuỗi đầu vào không thay đổi, giá trị băm và bucket tương ứng cũng không thay đổi.

Trong hệ thống đề xuất, SHA-256 được sử dụng làm hàm băm:

$$D = \mathrm{SHA256}(I) \tag{2.22}$$

Trong đó:

- I là chuỗi đầu vào đã được chuẩn hóa;
- D là giá trị băm 256 bit.

SHA-256 được lựa chọn vì có tính xác định, phân phối đầu ra tương đối đồng đều và được hỗ trợ sẵn trong môi trường Node.js. Mục tiêu của việc sử dụng SHA-256 trong percentage rollout không phải để mã hóa dữ liệu mà để tạo phân phối bucket ổn định.

Đoạn mã khái quát được biểu diễn như sau:

Đoạn mã 2.64: Tạo giá trị băm ổn định bằng SHA-256

```
import { createHash } from "node:crypto";

function stableHash(input) {
  return createHash("sha256")
    .update(input, "utf8")
    .digest();
}
```

Engine không được sử dụng các hàm băm phụ thuộc vào tiến trình hoặc runtime nếu kết quả của chúng có thể thay đổi giữa các lần khởi động. Hàm băm phải tạo cùng một kết quả trên

mọi instance backend để bảo đảm tính nhất quán khi hệ thống được triển khai theo mô hình nhiều replica.

Thành phần đầu vào của hàm băm

Đầu vào của hàm băm không chỉ bao gồm targeting key mà còn bao gồm các khóa xác định phạm vi của feature flag. Trong hệ thống đề xuất, chuỗi đầu vào được tạo theo cấu trúc:

$$I = projectKey \parallel : \parallel flagKey \parallel : \parallel targetingKey \tag{2.23}$$

Ví dụ:

Đoạn mã 2.65: Chuỗi đầu vào của stable hash

```
const input =
  `${projectKey}:${flagKey}:${targetingKey}`;
```

Việc đưa `projectKey` vào chuỗi băm giúp tách biệt cùng một targeting key giữa các project. Việc đưa `flagKey` vào chuỗi băm giúp một người dùng có thể thuộc các bucket khác nhau đối với các feature flag khác nhau.

Nếu chỉ băm targeting key, cùng một người dùng sẽ luôn rơi vào cùng một bucket trên mọi flag. Điều này có thể tạo ra nhóm người dùng cố định luôn được nhận các tính năng rollout sớm, làm giảm tính phân tán giữa các flag.

`environmentKey` không được đưa vào đầu vào trong thiết kế hiện tại. Nhờ đó, cùng một người dùng có thể giữ cùng vị trí bucket giữa staging và production đối với cùng project và flag. Tuy nhiên, nếu cần phân phối độc lập theo environment, hệ thống có thể mở rộng đầu vào thành:

$$\begin{aligned} I = projectKey \parallel environmentKey \\ \parallel flagKey \parallel targetingKey \end{aligned} \tag{2.24}$$

Cấu trúc chuỗi đầu vào phải được định nghĩa rõ và không thay đổi tùy tiện. Việc thay đổi thứ tự thành phần, ký tự phân cách hoặc quy tắc chuẩn hóa sẽ làm thay đổi bucket của toàn bộ người dùng.

Chuyển đổi kết quả băm thành bucket

Sau khi tạo digest SHA-256, engine chuyển một phần kết quả băm thành số nguyên không âm. Trong hệ thống đề xuất, 64 bit đầu tiên của digest được sử dụng để tạo giá trị số:

$$N = \text{uint64}(D[0..7]) \tag{2.25}$$

Giá trị này được ánh xạ thành bucket trong khoảng từ 0 đến nhỏ hơn 100:

$$bucket = \frac{N \bmod 10000}{100} \tag{2.26}$$

Cách tính trên tạo 10000 bucket nhỏ, tương ứng độ chính xác 0.01%. Giá trị bucket thuộc miền:

$$0 \leq bucket < 100 \tag{2.27}$$

Đoạn mã khái quát như sau:

Đoạn mã 2.66: Chuyển đổi SHA-256 digest thành bucket phần trăm

```
function calculateBucket(
  projectKey,
  flagKey,
  targetingKey
) {
  const input =
    `${projectKey}:${flagKey}:${targetingKey}`;

  const digest = createHash("sha256")
    .update(input, "utf8")
    .digest();

  const hashValue = digest.readBigUInt64BE(0);
  const bucketNumber = Number(hashValue % 10000n);
```

```
  return bucketNumber / 100;
}
```

Rule khớp nếu bucket nhỏ hơn tỷ lệ rollout:

$$matched \Leftrightarrow bucket < percentage \tag{2.28}$$

Ví dụ, nếu bucket bằng 18.42 và percentage bằng 25, người dùng được bật tính năng. Nếu bucket bằng 72.15, rule không khớp.

Sử dụng phép so sánh nhỏ hơn thay vì nhỏ hơn hoặc bằng giúp 0% không khớp với bucket nào và 100% khớp với toàn bộ bucket trong khoảng $[0, 100)$.

Persistent assignment

Persistent assignment là tính chất một targeting key tiếp tục nhận cùng kết quả giữa nhiều lần evaluation, miễn là đầu vào hàm băm và ngưỡng rollout không thay đổi.

Giả sử một người dùng được ánh xạ vào bucket b. Với tỷ lệ rollout p, kết quả được xác định bởi:

$$enabled = \begin{cases} true, & b < p, \\ false, & b \geq p. \end{cases} \tag{2.29}$$

Vì b không thay đổi giữa các request, người dùng không bị chuyển ngẫu nhiên giữa trạng thái bật và tắt. Tính chất này đặc biệt quan trọng đối với trải nghiệm người dùng, vì việc thay đổi giao diện hoặc hành vi liên tục giữa các lần truy cập có thể gây nhầm lẫn và làm sai lệch số liệu thử nghiệm.

Khi tỷ lệ rollout được tăng từ p_1 lên p_2 với $p_2 > p_1$, mọi người dùng đã được bật ở mức p_1 tiếp tục được bật ở mức p_2:

$$b < p_1 \Rightarrow b < p_2 \tag{2.30}$$

Do đó, tăng rollout tạo ra tập người dùng mở rộng theo tính chất lồng nhau. Ví dụ, nhóm 10%

đầu tiên là tập con của nhóm 25%, và nhóm 25% là tập con của nhóm 50%.

Ngược lại, khi giảm rollout, những người dùng có bucket nằm ngoài ngưỡng mới sẽ bị tắt tính năng. Đây là hành vi mong đợi khi cần thu hẹp phạm vi phát hành, nhưng không bảo đảm mọi người dùng từng được bật sẽ tiếp tục được bật.

Persistent assignment trong thiết kế này không yêu cầu lưu bucket của từng người dùng vào cơ sở dữ liệu. Assignment được tính lại một cách xác định từ cùng chuỗi đầu vào. Cách tiếp cận này giảm chi phí lưu trữ và tránh phải đồng bộ bảng phân nhóm người dùng [5].

Ảnh hưởng khi thay đổi targeting key

Targeting key là một thành phần trực tiếp của chuỗi đầu vào hàm băm. Vì vậy, khi targeting key thay đổi, digest và bucket cũng có thể thay đổi hoàn toàn:

$$\begin{aligned} targetingKey_1 \neq targetingKey_2 &\Rightarrow hash_1 \neq hash_2 \\ &\Rightarrow bucket_1 \neq bucket_2 \quad \text{trong phần lớn trường hợp} \end{aligned} \tag{2.31}$$

Một người dùng trước đó nằm trong phạm vi rollout có thể bị chuyển ra ngoài phạm vi nếu ứng dụng khách sử dụng targeting key mới. Ngược lại, người dùng đang bị tắt có thể được chuyển vào nhóm được bật.

Các trường hợp có thể làm targeting key thay đổi gồm:

- sử dụng session ID thay cho user ID ổn định;
- tạo targeting key mới sau mỗi lần đăng nhập;
- thay đổi định dạng từ số sang UUID;
- bổ sung hoặc loại bỏ prefix trong định danh;
- chuyển từ user ID sang email;
- thay đổi normalization giữa các phiên bản client.

Do đó, targeting key dùng cho percentage rollout phải đáp ứng các yêu cầu sau:

- ổn định theo thời gian;

- duy nhất trong phạm vi cần đánh giá;
- được tạo theo cùng một quy tắc trên mọi client;
- không phụ thuộc vào session hoặc access token;
- không thay đổi chỉ vì người dùng cập nhật thông tin hiển thị.

Khi người dùng chưa đăng nhập, ứng dụng có thể sử dụng một anonymous ID được lưu ổn định trên thiết bị. Tuy nhiên, khi chuyển từ anonymous ID sang user ID sau đăng nhập, bucket có thể thay đổi. Ứng dụng tích hợp cần chấp nhận hành vi này hoặc thiết kế cơ chế ánh xạ định danh phù hợp.

Nếu evaluation request không chứa targeting key, percentage rollout rule không thể được đánh giá. Engine phải coi rule là không khớp và tiếp tục đến rule tiếp theo hoặc trả về `DEFAULT_OFF`. Hệ thống không tự sinh targeting key ngẫu nhiên vì điều đó sẽ phá vỡ persistent assignment.

2.6.8 Thực thi rule theo độ ưu tiên

Sau khi evaluation snapshot được xây dựng và các trạng thái toàn cục đã được kiểm tra, Rule Evaluation Engine thực thi danh sách rule theo thứ tự ưu tiên. Mục tiêu của bước này là bảo đảm kết quả có tính xác định khi nhiều rule cùng tồn tại và có khả năng khớp với một evaluation context.

Trong hệ thống đề xuất, mỗi rule có trường `priority`. Giá trị priority nhỏ hơn biểu thị mức ưu tiên cao hơn. Engine duyệt danh sách rule đã được sắp xếp và dừng ngay tại rule đầu tiên khớp theo nguyên tắc *first match wins* [4].

Sắp xếp rule

Trước khi thực thi, các rule phải được sắp xếp tăng dần theo trường `priority`. Với tập rule:

$$R = \{r_1, r_2, \ldots, r_n\} \tag{2.32}$$

thứ tự thực thi phải thỏa mãn:

$$priority(r_i) \leq priority(r_{i+1}) \tag{2.33}$$

Ví dụ, tập rule sau:

Đoạn mã 2.67: Danh sách rule trước khi sắp xếp

```
[
  {
    "id": "rule-percentage",
    "type": "PERCENTAGE_ROLLOUT",
    "priority": 30
  },
  {
    "id": "rule-user",
    "type": "USER_ALLOWLIST",
    "priority": 10
  },
  {
    "id": "rule-role",
    "type": "ROLE_TARGETING",
    "priority": 20
  }
]
```

sau khi sắp xếp sẽ có thứ tự:

Đoạn mã 2.68: Danh sách rule sau khi sắp xếp

```
[
  {
    "id": "rule-user",
    "type": "USER_ALLOWLIST",
    "priority": 10
  },
  {
    "id": "rule-role",
    "type": "ROLE_TARGETING",
```

```
    "priority": 20
  },
  {
    "id": "rule-percentage",
    "type": "PERCENTAGE_ROLLOUT",
    "priority": 30
  }
]
```

Việc sắp xếp có thể được thực hiện tại tầng repository thông qua câu lệnh `ORDER BY`, hoặc tại tầng service trước khi truyền snapshot vào engine. Tuy nhiên, engine không nên phụ thuộc hoàn toàn vào thứ tự ngẫu nhiên của dữ liệu trả về từ cơ sở dữ liệu.

Để tăng độ an toàn, engine có thể thực hiện lại thao tác sắp xếp trên bản sao của danh sách rule:

Đoạn mã 2.69: Sắp xếp rule trước khi thực thi

```
const orderedRules = [...snapshot.rules].sort(
  (left, right) => {
    if (left.priority !== right.priority) {
      return left.priority - right.priority;
    }

    return left.id.localeCompare(right.id);
  }
);
```

Việc sử dụng bản sao giúp tránh thay đổi trực tiếp evaluation snapshot, vốn được xem là bất biến trong phạm vi một lần evaluation.

Priority giữa các rule không thay thế precedence toàn cục của engine. Các điều kiện như archived, disabled, group kill switch, flag kill switch và global on luôn được xử lý trước toàn bộ danh sách rule, bất kể priority được cấu hình như thế nào.

Dừng tại rule đầu tiên khớp

Sau khi danh sách rule được sắp xếp, engine duyệt tuần tự từ rule có priority cao nhất đến thấp nhất. Khi một rule khớp với evaluation context, engine tạo kết quả và kết thúc ngay quá trình đánh giá.

Luồng thực thi có thể được biểu diễn như sau:

Đoạn mã 2.70: Dừng tại rule đầu tiên khớp

```
for (const rule of orderedRules) {
  const matchResult = evaluateRule(
    rule,
    snapshot,
    context
  );

  if (matchResult.matched) {
    return {
      enabled: true,
      variant: "on",
      reason: matchResult.reason,
      matchedRuleId: rule.id
    };
  }
}

return {
  enabled: false,
  variant: "off",
  reason: "DEFAULT_OFF"
};
```

Nguyên tắc *first match wins* giúp kết quả có tính xác định khi một context có thể đồng thời thỏa mãn nhiều rule. Ví dụ, một người dùng vừa nằm trong user allowlist, vừa có role `beta-customer`,

đồng thời thuộc bucket percentage rollout. Nếu user allowlist có priority cao nhất, kết quả phải được xác định bởi rule này và mã lý do được trả về là `USER_MATCH`.

Khi engine đã tìm thấy rule khớp, các rule còn lại không được thực thi. Điều này có các lợi ích sau:

- giảm chi phí xử lý không cần thiết;
- tránh nhiều rule cùng tạo ra kết quả cạnh tranh;
- bảo toàn ý nghĩa của trường priority;
- giúp mã lý do và `matchedRuleId` phản ánh chính xác rule quyết định kết quả.

Trong mô hình hiện tại, các rule targeting chỉ tạo kết quả bật. Nếu không rule nào khớp, engine trả về `DEFAULT_OFF`. Nếu hệ thống được mở rộng để hỗ trợ cả kết quả bật và tắt theo rule, nguyên tắc dừng tại rule đầu tiên khớp vẫn có thể được giữ nguyên, nhưng mỗi rule phải chứa outcome hoặc variant rõ ràng.

Xử lý các rule cùng độ ưu tiên

Hai hoặc nhiều rule có cùng priority có thể làm thứ tự thực thi trở nên không rõ ràng nếu hệ thống không có tiêu chí phân xử bổ sung. Thứ tự bản ghi trong cơ sở dữ liệu không được bảo đảm khi truy vấn chỉ sắp xếp theo một cột có giá trị trùng nhau.

Ví dụ:

Đoạn mã 2.71: Hai rule có cùng độ ưu tiên

```
[
  {
    "id": "rule-a",
    "type": "USER_ALLOWLIST",
    "priority": 10
  },
  {
    "id": "rule-b",
    "type": "ROLE_TARGETING",
    "priority": 10
```

```
  }
]
```

Nếu cả hai rule cùng khớp, kết quả có thể thay đổi giữa các lần evaluation nếu thứ tự trả về không ổn định. Vì vậy, hệ thống cần lựa chọn một trong hai chiến lược.

Chiến lược được ưu tiên là không cho phép trùng priority trong cùng một flag config. Có thể áp dụng ràng buộc duy nhất:

$$UNIQUE(flagConfigId, priority) \tag{2.34}$$

Ràng buộc này giúp mỗi rule có một vị trí rõ ràng trong chuỗi thực thi. Khi người quản trị thay đổi thứ tự rule, backend phải cập nhật priority trong một transaction để tránh tạo trạng thái trùng lặp tạm thời.

Trong trường hợp mô hình dữ liệu vẫn cho phép trùng priority, hệ thống phải sử dụng tiêu chí phụ ổn định, chẳng hạn như:

1. `priority` tăng dần;
2. `createdAt` tăng dần;
3. `id` tăng dần theo thứ tự từ điển.

Thứ tự truy vấn có thể được định nghĩa như sau:

Đoạn mã 2.72: Sắp xếp ổn định khi rule có cùng priority

```
ORDER BY
  priority ASC,
  created_at ASC,
  id ASC
```

Tiêu chí phụ chỉ có vai trò tạo kết quả xác định, không nên được xem là một cơ chế priority ngầm. Giao diện quản trị cần cảnh báo hoặc ngăn người dùng cấu hình nhiều rule cùng độ ưu tiên để tránh khó hiểu trong quá trình vận hành.

Rule không hợp lệ hoặc không đủ context

Trong quá trình thực thi, engine có thể gặp rule không hợp lệ hoặc rule không có đủ dữ liệu context để đánh giá. Hai trường hợp này cần được xử lý khác nhau.

Rule không đủ context là rule có cấu trúc hợp lệ nhưng evaluation context thiếu thuộc tính cần thiết. Ví dụ:

- user allowlist rule nhưng context không có `targetingKey` và `userId`;
- role targeting rule nhưng context không có `roles`;
- percentage rollout rule nhưng context không có `targetingKey`.

Trong trường hợp này, rule được xem là không khớp và engine tiếp tục với rule tiếp theo:

Đoạn mã 2.73: Bỏ qua rule khi không đủ context

```
if (!hasRequiredContext(rule, context)) {
  continue;
}
```

Thiếu context không làm toàn bộ evaluation thất bại vì các rule phía sau vẫn có thể được đánh giá. Ví dụ, context không có role nhưng vẫn có targeting key thì percentage rollout vẫn có thể khớp.

Ngược lại, rule không hợp lệ là rule có cấu trúc hoặc tham số vi phạm contract. Một số trường hợp gồm:

- loại rule không được hỗ trợ;
- `parameters` không phải đối tượng hợp lệ;
- user allowlist không chứa danh sách định danh;
- role targeting chứa giá trị role không phải chuỗi;
- percentage rollout có tỷ lệ nhỏ hơn 0 hoặc lớn hơn 100;
- rule thiếu `id`, `type` hoặc `priority`.

Rule không hợp lệ cần được ngăn chặn chủ yếu tại thời điểm tạo hoặc cập nhật cấu hình. Backend phải validation tham số theo từng loại rule trước khi ghi vào cơ sở dữ liệu. Tuy nhiên,

engine vẫn cần kiểm tra phòng vệ vì dữ liệu có thể bị lỗi do migration, chỉnh sửa trực tiếp cơ sở dữ liệu hoặc phiên bản cấu hình cũ.

Trong hệ thống đề xuất, engine áp dụng cách xử lý fail-closed. Nếu phát hiện rule không hợp lệ, engine không được xem rule đó là khớp. Tùy mức độ lỗi, hệ thống có thể:

1. bỏ qua rule không hợp lệ và ghi log cảnh báo;
2. trả về `ERROR` nếu rule lỗi làm snapshot không còn đáng tin cậy;
3. ghi metric để hỗ trợ phát hiện cấu hình sai.

Đối với mini project, phương án phù hợp là bỏ qua rule không hợp lệ, tiếp tục đánh giá các rule còn lại và ghi log có cấu trúc. Nếu không có rule hợp lệ nào khớp, engine trả về `DEFAULT_OFF`. Cách xử lý này duy trì khả năng sẵn sàng nhưng không bật tính năng dựa trên dữ liệu sai.

Đoạn mã khái quát được biểu diễn như sau:

Đoạn mã 2.74: Xử lý rule không hợp lệ trong quá trình thực thi

```
for (const rule of orderedRules) {
  if (!isValidRule(rule)) {
    logger.warn({
      event: "INVALID_EVALUATION_RULE",
      ruleId: rule.id,
      flagKey: snapshot.flag.key
    });

    continue;
  }

  if (!hasRequiredContext(rule, context)) {
    continue;
  }

  const result = evaluateRule(
    rule,
    snapshot,
```

```
      context
    );

    if (result.matched) {
      return createEnabledResult(rule, result.reason);
    }
  }

  return createDefaultOffResult();
```

Việc phân biệt rule không hợp lệ với rule không đủ context giúp engine phản ánh đúng bản chất của lỗi. Thiếu context là tình huống runtime hợp lệ, còn rule không hợp lệ là vấn đề cấu hình cần được theo dõi và khắc phục. Evaluation API nên trả về kết quả ổn định và fail-closed khi không thể xác định trạng thái bật một cách đáng tin cậy [10, 8].

2.6.9 Variant và kết quả đánh giá

Sau khi hoàn tất quá trình kiểm tra trạng thái toàn cục và thực thi các rule theo độ ưu tiên, Rule Evaluation Engine tạo một đối tượng kết quả thống nhất để trả về cho Evaluation API. Kết quả không chỉ cho biết feature flag đang bật hay tắt mà còn cung cấp variant, mã lý do và metadata phục vụ truy vết quyết định.

Trong phạm vi hệ thống đề xuất, feature flag sử dụng hai variant cơ bản là `on` và `off`. Thiết kế này phù hợp với mô hình boolean feature flag, trong đó ứng dụng khách lựa chọn giữa nhánh tính năng mới và nhánh mặc định. Cấu trúc kết quả vẫn được thiết kế theo hướng có thể mở rộng để hỗ trợ nhiều variant trong tương lai.

Variant bật và variant tắt

Variant là giá trị biểu diễn nhánh cấu hình mà ứng dụng khách nên sử dụng sau quá trình evaluation. Đối với boolean feature flag, hệ thống định nghĩa hai variant:

- `on`: đại diện cho nhánh tính năng được bật;
- `off`: đại diện cho nhánh tính năng bị tắt hoặc hành vi cũ.

Quan hệ giữa trạng thái boolean và variant được xác định như sau:

$$variant = \begin{cases} \texttt{on}, & enabled = true, \\ \texttt{off}, & enabled = false. \end{cases} \tag{2.35}$$

Ví dụ, khi user allowlist rule khớp, engine trả về:

Đoạn mã 2.75: Variant bật khi user allowlist khớp

```
{
  "enabled": true,
  "variant": "on",
  "reason": "USER_MATCH",
  "matchedRuleId": "rule-user-allowlist"
}
```

Ngược lại, khi flag bị disabled, kết quả có variant tắt:

Đoạn mã 2.76: Variant tắt khi flag bị disabled

```
{
  "enabled": false,
  "variant": "off",
  "reason": "FLAG_DISABLED"
}
```

Việc trả về đồng thời `enabled` và `variant` giúp ứng dụng khách tích hợp thuận tiện với boolean flag, đồng thời tạo nền tảng cho việc mở rộng sang multivariate flag. Trong mô hình mở rộng, variant có thể nhận các giá trị như `control`, `blue-layout` hoặc `compact-layout`, trong khi trạng thái `enabled` vẫn biểu thị feature flag có đang phục vụ một nhánh bật hay không.

Các nền tảng feature flag thường sử dụng variant để phân biệt nhiều giá trị hoặc nhiều trải nghiệm có thể được trả về từ cùng một flag [7].

Variant mặc định

Variant mặc định là giá trị được trả về khi engine không tìm thấy điều kiện hợp lệ để bật feature flag hoặc không thể hoàn tất quá trình evaluation một cách đáng tin cậy. Trong hệ thống đề xuất, variant mặc định là `off`.

Quy tắc này được biểu diễn như sau:

$$defaultVariant = \texttt{off} \tag{2.36}$$

Variant `off` được sử dụng trong các trường hợp:

- feature flag không tồn tại;
- feature flag đã được archive;
- flag config đang disabled;
- group kill switch hoặc flag kill switch đang hoạt động;
- không có targeting rule nào khớp;
- evaluation context thiếu dữ liệu cần thiết;
- xảy ra lỗi trong quá trình tải snapshot hoặc evaluation.

Ví dụ, khi không có rule nào khớp:

Đoạn mã 2.77: Variant mặc định khi không có rule nào khớp

```
{
  "enabled": false,
  "variant": "off",
  "reason": "DEFAULT_OFF"
}
```

Khi xảy ra lỗi nội bộ, service cũng trả về variant mặc định theo nguyên tắc fail-closed:

Đoạn mã 2.78: Variant mặc định khi evaluation xảy ra lỗi

```
{
  "enabled": false,
  "variant": "off",
  "reason": "ERROR"
}
```

Variant mặc định không được hiểu là kết quả của một rule targeting. Đây là giá trị an toàn được sử dụng khi không có bằng chứng rõ ràng để bật tính năng. Evaluation API cần cung cấp default value và kết quả có cấu trúc ổn định để ứng dụng khách có thể xử lý lỗi mà không làm gián đoạn luồng chính [10].

Cấu trúc EvaluationResult

`EvaluationResult` là cấu trúc dữ liệu chuẩn được Rule Evaluation Engine trả về sau mỗi lần đánh giá. Trong hệ thống đề xuất, kết quả bao gồm các trường chính sau:

Đoạn mã 2.79: Cấu trúc EvaluationResult

```
{
  "projectKey": "ecommerce-platform",
  "flagKey": "new-checkout",
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-role-targeting"
}
```

Ý nghĩa của từng trường được mô tả như sau:

- `projectKey`: project chứa feature flag được đánh giá;
- `flagKey`: khóa của feature flag;
- `enabled`: trạng thái boolean cuối cùng;
- `variant`: nhánh cấu hình được trả về;
- `reason`: mã lý do giải thích quyết định;
- `matchedRuleId`: định danh rule đã quyết định kết quả, nếu có.

Trong mã nguồn TypeScript, kiểu dữ liệu có thể được định nghĩa như sau:

Đoạn mã 2.80: Định nghĩa kiểu EvaluationResult

```
interface EvaluationResult {
  projectKey: string;
```

```
  flagKey: string;
  enabled: boolean;
  variant: string;
  reason: EvaluationReason;
  matchedRuleId?: string;
}
```

Trường `reason` sử dụng tập mã cố định thay vì thông điệp tự do. Các mã lý do chính gồm:

- `GLOBAL_ON`;
- `USER_MATCH`;
- `ROLE_MATCH`;
- `PERCENTAGE_MATCH`;
- `DEFAULT_OFF`;
- `FLAG_DISABLED`;
- `FLAG_ARCHIVED`;
- `GROUP_KILL_SWITCH`;
- `KILL_SWITCH`;
- `NOT_FOUND`;
- `ERROR`.

Việc sử dụng mã lý do chuẩn hóa giúp frontend, SDK và hệ thống logging xử lý kết quả mà không cần phân tích thông điệp văn bản. Mã lý do cũng hỗ trợ kiểm thử unit test vì mỗi nhánh evaluation có thể được xác nhận bằng một giá trị xác định.

Engine nên trả về một đối tượng mới thay vì sửa đổi snapshot hoặc context. Điều này giúp giữ Rule Evaluation Engine gần với mô hình hàm thuần túy:

$$EvaluationResult = evaluate(EvaluationSnapshot, EvaluationContext) \tag{2.37}$$

Với cùng snapshot và context, engine phải tạo cùng một `EvaluationResult`. Tính chất này

giúp quá trình đánh giá dễ kiểm thử, dễ cache và dễ tái hiện khi điều tra sự cố.

Matched rule và metadata

Trường `matchedRuleId` xác định rule đã trực tiếp quyết định trạng thái bật của feature flag. Trường này chỉ xuất hiện khi kết quả được tạo từ một targeting rule cụ thể.

Quan hệ giữa reason và matched rule được tổng hợp trong Bảng 2.27.

Bảng 2.27: Quan hệ giữa mã lý do và matched rule

Reason	**Có matchedRuleId**	**Nguồn quyết định**
`USER_MATCH`	Có	User allowlist rule đầu tiên khớp.
`ROLE_MATCH`	Có	Role targeting rule đầu tiên khớp.
`PERCENTAGE_MATCH`	Có	Percentage rollout rule đầu tiên khớp.
`GLOBAL_ON`	Không	Serving mode của flag config.
`FLAG_DISABLED`	Không	Trạng thái của flag config.
`FLAG_ARCHIVED`	Không	Trạng thái vòng đời của feature flag.
`GROUP_KILL_SWITCH`	Không	Kill switch của flag group.
`KILL_SWITCH`	Không	Kill switch của flag config.
`DEFAULT_OFF`	Không	Không có rule nào khớp.
`NOT_FOUND` hoặc `ERROR`	Không	Không tạo được evaluation snapshot hợp lệ.

Ngoài `matchedRuleId`, hệ thống có thể bổ sung metadata nội bộ phục vụ logging, debugging hoặc thống kê, chẳng hạn như:

- version của flag config;
- environment key;
- loại rule đã khớp;
- priority của rule;
- bucket của percentage rollout;
- thời gian xử lý evaluation;
- request ID phục vụ truy vết.

Ví dụ về kết quả chi tiết nội bộ:

Đoạn mã 2.81: EvaluationResult với metadata nội bộ

```
{
  "projectKey": "ecommerce-platform",
  "environmentKey": "production",
  "flagKey": "new-checkout",
  "enabled": true,
  "variant": "on",
  "reason": "PERCENTAGE_MATCH",
  "matchedRuleId": "rule-percentage",
  "metadata": {
    "configVersion": 12,
    "ruleType": "PERCENTAGE_ROLLOUT",
    "rulePriority": 30,
    "bucket": 18.42,
    "requestId": "request-abc"
  }
}
```

Không phải toàn bộ metadata đều cần được trả về cho ứng dụng khách. Một số thông tin như bucket hoặc version cấu hình có thể chỉ được ghi trong log hoặc metric nội bộ để tránh làm API response phức tạp và để hạn chế lộ chi tiết triển khai.

Metadata không được chứa dữ liệu nhạy cảm từ evaluation context như access token, mật khẩu hoặc thông tin cá nhân không cần thiết. Khi ghi log, hệ thống chỉ nên lưu các định danh và thuộc tính cần thiết cho việc truy vết, đồng thời tuân thủ nguyên tắc giảm thiểu dữ liệu [12].

Việc cung cấp reason và matched rule giúp Evaluation API không chỉ trả về quyết định mà còn giải thích được nguồn gốc của quyết định đó. Đây là yếu tố quan trọng đối với khả năng kiểm thử, vận hành và điều tra khi kết quả feature flag không đúng với kỳ vọng.

2.6.10 Reason code

Reason code là mã định danh ngắn gọn mô tả nguyên nhân trực tiếp dẫn đến kết quả evaluation. Thay vì chỉ trả về giá trị `enabled`, Evaluation API cung cấp thêm trường `reason` để cho biết

feature flag được bật hoặc tắt bởi trạng thái hệ thống, rule targeting, kết quả mặc định hay cơ chế fallback.

Trong hệ thống đề xuất, reason code được biểu diễn bằng một tập giá trị cố định. Mỗi nhánh quyết định trong Rule Evaluation Engine phải ánh xạ đến đúng một reason code. Việc chuẩn hóa này giúp kết quả có tính nhất quán, dễ kiểm thử và không phụ thuộc vào thông điệp văn bản tự do.

Kiểu dữ liệu reason code có thể được định nghĩa như sau:

Đoạn mã 2.82: Tập reason code của Evaluation API

```
type EvaluationReason =
  | "FLAG_ARCHIVED"
  | "FLAG_DISABLED"
  | "GROUP_KILL_SWITCH"
  | "KILL_SWITCH"
  | "GLOBAL_ON"
  | "USER_MATCH"
  | "ROLE_MATCH"
  | "PERCENTAGE_MATCH"
  | "DEFAULT_OFF"
  | "NOT_FOUND"
  | "ERROR";
```

Reason code cho trạng thái hệ thống

Reason code cho trạng thái hệ thống phản ánh các quyết định được tạo ra trước khi engine thực thi targeting rule. Các mã này cho biết feature flag bị vô hiệu hóa bởi trạng thái vòng đời, trạng thái cấu hình, kill switch hoặc được bật trực tiếp bởi serving mode.

Các reason code thuộc nhóm này gồm:

- `FLAG_ARCHIVED`: feature flag đã được archive;
- `FLAG_DISABLED`: flag config trong environment đang disabled;
- `GROUP_KILL_SWITCH`: kill switch của flag group đang được kích hoạt;

- `KILL_SWITCH`: kill switch của flag config đang được kích hoạt;
- `GLOBAL_ON`: flag đang enabled và serving mode là global on.

Quan hệ giữa trạng thái hệ thống và kết quả được tổng hợp trong Bảng 2.28.

Bảng 2.28: Reason code cho trạng thái hệ thống

Reason code	**Enabled**	**Điều kiện phát sinh**
`FLAG_ARCHIVED`	`false`	Feature flag có trạng thái vòng đời `ARCHIVED`.
`FLAG_DISABLED`	`false`	Flag config trong environment có trạng thái `DISABLED`.
`GROUP_KILL_SWITCH`	`false`	Flag thuộc một group đang kích hoạt kill switch.
`KILL_SWITCH`	`false`	Kill switch riêng của flag config đang được kích hoạt.
`GLOBAL_ON`	`true`	Config đang enabled, không có kill switch và serving mode là `GLOBAL_ON`.

Các mã trạng thái phải tuân theo đúng thứ tự ưu tiên của engine. Ví dụ, nếu một flag vừa disabled vừa có kill switch đang bật, kết quả là `FLAG_DISABLED` vì trạng thái disabled được kiểm tra trước kill switch trong precedence đã xác định.

Mỗi lần evaluation chỉ trả về reason code của điều kiện đầu tiên quyết định kết quả. Engine không trả về đồng thời nhiều reason code vì điều đó có thể làm mất tính rõ ràng của precedence.

Reason code cho targeting

Reason code cho targeting phản ánh trường hợp feature flag được bật bởi một rule cụ thể trong serving mode `TARGETED`. Nhóm này gồm:

- `USER_MATCH`: user allowlist rule khớp;
- `ROLE_MATCH`: role targeting rule khớp;
- `PERCENTAGE_MATCH`: percentage rollout rule khớp.

Khi trả về một targeting reason code, kết quả nên kèm theo `matchedRuleId` để xác định chính xác rule đã quyết định trạng thái:

Đoạn mã 2.83: Kết quả evaluation với targeting reason code

```
{
```

```
  "projectKey": "ecommerce-platform",
  "flagKey": "new-checkout",
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-role-targeting"
}
```

Quan hệ giữa loại rule và reason code được xác định như sau:

$$reason(rule) = \begin{cases} \texttt{USER_MATCH}, & type = \texttt{USER_ALLOWLIST}, \\ \texttt{ROLE_MATCH}, & type = \texttt{ROLE_TARGETING}, \\ \texttt{PERCENTAGE_MATCH}, & type = \texttt{PERCENTAGE_ROLLOUT}. \end{cases} \tag{2.38}$$

Reason code phải tương ứng với rule đầu tiên khớp theo thứ tự priority. Nếu một người dùng đồng thời nằm trong allowlist, có role phù hợp và thuộc bucket rollout, reason code vẫn chỉ phản ánh rule đầu tiên đã quyết định kết quả.

Cách tổ chức này phù hợp với mô hình targeting rule có thứ tự, trong đó quá trình evaluation dừng tại điều kiện đầu tiên khớp [4].

Reason code cho default và fallback

Nhóm reason code cho default và fallback được sử dụng khi feature flag không được bật bởi trạng thái hệ thống hoặc targeting rule. Các mã chính gồm:

- `DEFAULT_OFF`: evaluation hoàn tất hợp lệ nhưng không có điều kiện bật nào khớp;
- `NOT_FOUND`: không tìm thấy project, environment, feature flag hoặc cấu hình cần đánh giá;
- `ERROR`: xảy ra lỗi trong quá trình truy xuất snapshot hoặc thực thi evaluation.

`DEFAULT_OFF` không biểu thị lỗi. Mã này cho biết hệ thống đã đánh giá thành công nhưng context hiện tại không thỏa mãn điều kiện bật nào:

Đoạn mã 2.84: Kết quả default off

```
{
  "enabled": false,
  "variant": "off",
  "reason": "DEFAULT_OFF"
}
```

`NOT_FOUND` được sử dụng khi không thể xác định cấu hình theo các khóa trong request. Trong thiết kế fail-closed, Evaluation API vẫn trả về trạng thái tắt:

Đoạn mã 2.85: Kết quả khi không tìm thấy feature flag

```
{
  "projectKey": "ecommerce-platform",
  "flagKey": "unknown-flag",
  "enabled": false,
  "variant": "off",
  "reason": "NOT_FOUND"
}
```

`ERROR` được sử dụng khi hệ thống gặp lỗi ngoài dự kiến, chẳng hạn như không truy cập được cơ sở dữ liệu, snapshot không nhất quán hoặc engine phát sinh exception:

Đoạn mã 2.86: Kết quả fallback khi evaluation xảy ra lỗi

```
{
  "projectKey": "ecommerce-platform",
  "flagKey": "new-checkout",
  "enabled": false,
  "variant": "off",
  "reason": "ERROR"
}
```

Cần phân biệt rõ `DEFAULT_OFF`, `NOT_FOUND` và `ERROR`. Cả ba đều có thể trả về `enabled = false`, nhưng nguyên nhân vận hành hoàn toàn khác nhau:

- `DEFAULT_OFF`: hành vi nghiệp vụ hợp lệ;
- `NOT_FOUND`: request tham chiếu đến cấu hình không tồn tại;

- `ERROR`: evaluation không thể hoàn tất bình thường.

Evaluation API nên sử dụng default value an toàn khi không thể tạo ra kết quả đáng tin cậy, đồng thời cung cấp reason code để client hoặc hệ thống giám sát phân biệt fallback với kết quả nghiệp vụ thông thường [10].

Vai trò của reason code trong debug

Reason code giúp giải thích vì sao hai evaluation request cho cùng một flag có thể nhận kết quả khác nhau. Thay vì chỉ quan sát `enabled = false`, người vận hành có thể xác định flag bị tắt do disabled, kill switch, không khớp rule, không tìm thấy cấu hình hay lỗi hệ thống.

Trong quá trình debug, reason code hỗ trợ các mục đích sau:

- xác định nhánh precedence đã kết thúc evaluation;
- kiểm tra rule nào đã khớp thông qua `matchedRuleId`;
- phân biệt lỗi cấu hình với hành vi default off;
- xây dựng unit test cho từng nhánh của engine;
- phân nhóm metric theo nguyên nhân evaluation;
- truy vết sự cố giữa ứng dụng khách, Evaluation API và audit log.

Ví dụ, hai kết quả đều có trạng thái tắt nhưng yêu cầu cách xử lý khác nhau:

Đoạn mã 2.87: Hai kết quả tắt với nguyên nhân khác nhau

```
// Configuration decision
{
  "enabled": false,
  "reason": "FLAG_DISABLED"
}

// No targeting rule matched
{
  "enabled": false,
  "reason": "DEFAULT_OFF"
}
```

Trong trường hợp thứ nhất, người quản trị cần kiểm tra trạng thái flag config. Trong trường hợp thứ hai, cần kiểm tra evaluation context, danh sách rule và priority.

Reason code cũng là cơ sở để viết unit test theo bảng quyết định. Ví dụ:

Đoạn mã 2.88: Kiểm thử reason code của Rule Evaluation Engine

```
expect(result).toEqual({
  enabled: false,
  variant: "off",
  reason: "KILL_SWITCH"
});
```

Khi ghi log, reason code nên được lưu cùng request ID, project key, environment key, flag key, config version và matched rule ID nếu có. Không nên ghi toàn bộ evaluation context nếu context chứa dữ liệu cá nhân hoặc thông tin nhạy cảm. Log cần cung cấp đủ dữ liệu để điều tra nhưng vẫn tuân thủ nguyên tắc giảm thiểu thông tin [12].

Reason code phải được xem là một phần ổn định của API contract. Việc đổi tên hoặc thay đổi ý nghĩa của một mã có thể ảnh hưởng đến SDK, frontend, test và hệ thống giám sát. Khi cần bổ sung mã mới, hệ thống nên duy trì khả năng tương thích và cập nhật tài liệu API tương ứng.

2.6.11 Default value và error fallback

Default value và error fallback xác định hành vi của Evaluation API khi hệ thống không thể bật feature flag dựa trên một điều kiện hợp lệ. Trong hệ thống đề xuất, giá trị mặc định là trạng thái tắt với variant `off`. Cách tiếp cận này giúp ứng dụng khách duy trì hành vi cũ hoặc hành vi ổn định khi cấu hình không tồn tại, context không đầy đủ hoặc quá trình evaluation phát sinh lỗi.

Kết quả fallback cơ bản có dạng:

Đoạn mã 2.89: Kết quả fallback mặc định

```
{
  "enabled": false,
  "variant": "off",
```

```
  "reason": "ERROR"
}
```

Default value không thay thế reason code. Trạng thái và variant có thể giống nhau trong nhiều trường hợp, nhưng trường `reason` phải phản ánh đúng nguyên nhân để hỗ trợ debug và giám sát.

Nguyên tắc fail-safe và fail-closed

Fail-safe là nguyên tắc đưa hệ thống về trạng thái ít rủi ro hơn khi xảy ra lỗi. Trong ngữ cảnh feature flag, trạng thái an toàn thường là giữ nguyên hành vi cũ thay vì bật một tính năng mới chưa được xác nhận.

Fail-closed là một cách cụ thể để thực hiện fail-safe, trong đó feature flag chỉ được bật khi engine có đủ dữ liệu và xác định rõ một điều kiện cho phép. Nếu không thể chứng minh rằng flag phải được bật, kết quả mặc định là tắt:

$$\neg confirmedEnabled \Rightarrow enabled = false \tag{2.39}$$

Trong hệ thống đề xuất, nguyên tắc fail-closed được áp dụng cho các trường hợp:

- không tìm thấy cấu hình;
- thiếu dữ liệu cần thiết để một rule khớp;
- cache không khả dụng và không thể đọc database;
- snapshot không nhất quán;
- engine phát sinh lỗi ngoài dự kiến.

Cách tiếp cận này đặc biệt phù hợp với release flag và experiment flag, vì việc vô tình bật một tính năng mới thường tạo rủi ro cao hơn việc tạm thời giữ hành vi cũ. Evaluation API cũng cần cung cấp default value ổn định để ứng dụng khách có thể tiếp tục hoạt động khi evaluation thất bại [10].

Tuy nhiên, fail-closed không đồng nghĩa với việc mọi loại flag đều luôn nên trả về `false`. Trong một hệ thống tổng quát, default value có thể phụ thuộc vào loại flag và yêu cầu nghiệp vụ. Ví dụ, một ops flag dùng để vô hiệu hóa cơ chế bảo vệ có thể cần chiến lược an toàn khác. Trong

phạm vi mini project, tất cả boolean flag sử dụng mặc định `off` để giữ thiết kế nhất quán và dễ kiểm thử.

Flag không tồn tại

Flag không tồn tại là trường hợp repository không tìm thấy project, environment, feature flag hoặc flag config tương ứng với evaluation request. Hệ thống không tự động tạo cấu hình hoặc giả định flag đang bật.

Kết quả trả về là:

Đoạn mã 2.90: Fallback khi không tìm thấy feature flag

```
{
  "projectKey": "ecommerce-platform",
  "flagKey": "unknown-feature",
  "enabled": false,
  "variant": "off",
  "reason": "NOT_FOUND"
}
```

`NOT_FOUND` có thể phát sinh trong các trường hợp:

- `projectKey` không tồn tại;
- `environmentKey` không tồn tại trong project;
- project không có environment mặc định khi request bỏ qua environment key;
- `flagKey` không tồn tại trong project;
- flag tồn tại nhưng chưa có config trong environment được yêu cầu.

Đối với Evaluation API, việc trả về kết quả tắt thường an toàn hơn việc trả HTTP error làm gián đoạn ứng dụng khách. Tuy nhiên, backend vẫn cần ghi log và metric cho `NOT_FOUND`, vì nguyên nhân có thể là lỗi tích hợp, lỗi chính tả trong flag key hoặc cấu hình chưa được khởi tạo.

API quản trị và Evaluation API có thể xử lý not found khác nhau. API quản trị nên trả về HTTP `404`, trong khi Evaluation API có thể trả về một `EvaluationResult` fail-closed để client

tiếp tục sử dụng hành vi mặc định.

Context không đầy đủ

Context không đầy đủ là trường hợp request hợp lệ về cấu trúc nhưng thiếu một hoặc nhiều thuộc tính cần thiết cho một rule cụ thể. Ví dụ:

- thiếu `userId` và `targetingKey` đối với user allowlist;
- thiếu `roles` đối với role targeting;
- thiếu `targetingKey` đối với percentage rollout;
- thiếu `organizationId` đối với rule targeting theo tổ chức trong phiên bản mở rộng.

Thiếu context không phải lỗi toàn cục của evaluation request. Engine chỉ xem rule phụ thuộc vào dữ liệu bị thiếu là không khớp và tiếp tục đánh giá rule tiếp theo:

Đoạn mã 2.91: Xử lý rule khi context không đầy đủ

```
for (const rule of orderedRules) {
  if (!hasRequiredContext(rule, context)) {
    continue;
  }

  const result = evaluateRule(rule, context);

  if (result.matched) {
    return createEnabledResult(rule, result.reason);
  }
}
```

Ví dụ, context không chứa role nhưng vẫn có targeting key thì engine có thể bỏ qua role targeting và tiếp tục đánh giá percentage rollout.

Nếu không rule nào còn lại khớp, kết quả là:

Đoạn mã 2.92: Kết quả khi context không đầy đủ và không có rule khớp

```
{
```

```
  "enabled": false,
  "variant": "off",
  "reason": "DEFAULT_OFF"
}
```

Cần phân biệt context không đầy đủ với context không hợp lệ. Nếu trường `roles` bị thiếu, engine có thể tiếp tục evaluation. Nếu `roles` được cung cấp dưới dạng chuỗi thay vì mảng, request phải bị từ chối ở bước validation.

Cách xử lý này cho phép Evaluation API linh hoạt với nhiều loại flag và context khác nhau, đồng thời vẫn giữ nguyên tắc chỉ bật khi có điều kiện hợp lệ [6].

Lỗi database hoặc cache

Evaluation snapshot có thể được truy xuất từ cache hoặc database. Khi một trong hai thành phần gặp lỗi, service cần áp dụng chiến lược fallback rõ ràng để cân bằng giữa tính sẵn sàng và tính nhất quán.

Luồng truy xuất có thể được thiết kế như sau:

1. kiểm tra cache;
2. nếu cache hit, trả về snapshot đã lưu;
3. nếu cache miss hoặc cache lỗi, truy xuất database;
4. nếu database thành công, tạo snapshot và cập nhật cache;
5. nếu cả cache và database đều không cung cấp được snapshot đáng tin cậy, trả về fallback tắt.

Doạn mã khái quát:

Doạn mã 2.93: Fallback giữa cache và database

```
async function loadSnapshot(keys) {
  try {
    const cached = await cache.get(keys);

    if (cached) {
```

```
      return cached;
    }
  } catch (error) {
    logger.warn({
      event: "EVALUATION_CACHE_ERROR",
      error
    });
  }

  return database.loadSnapshot(keys);
}
```

Lỗi cache không nhất thiết làm evaluation thất bại nếu database vẫn hoạt động. Cache chỉ là lớp tối ưu hiệu năng, không phải nguồn dữ liệu duy nhất. Vì vậy, khi cache lỗi, hệ thống nên bỏ qua cache và đọc trực tiếp từ database.

Ngược lại, nếu database lỗi nhưng cache còn chứa snapshot hợp lệ, hệ thống có thể sử dụng cache để duy trì khả năng phục vụ. Tuy nhiên, snapshot cache phải có version, thời gian hết hạn hoặc cơ chế invalidation rõ ràng để hạn chế nguy cơ sử dụng cấu hình quá cũ. Thiết kế caching cần cân bằng giữa độ trễ, khả năng sẵn sàng và tính nhất quán [8, 13].

Nếu database lỗi và không có cache hợp lệ, service trả về:

Đoạn mã 2.94: Fallback khi không thể truy xuất snapshot

```
{
  "enabled": false,
  "variant": "off",
  "reason": "ERROR"
}
```

Lỗi hạ tầng cần được ghi log với request ID, project key, environment key và flag key. Không nên trả chi tiết lỗi database hoặc cache trực tiếp cho ứng dụng khách vì có thể làm lộ thông tin triển khai nội bộ.

Lỗi nội bộ của Evaluation Engine

Lỗi nội bộ của Evaluation Engine là exception phát sinh sau khi snapshot đã được tải nhưng engine không thể hoàn tất quá trình đánh giá. Một số nguyên nhân có thể gồm:

- rule có kiểu không được hỗ trợ;
- tham số rule không đúng cấu trúc;
- lỗi khi chuyển đổi giá trị băm;
- dữ liệu snapshot vi phạm giả định nội bộ;
- lỗi lập trình trong hàm đánh giá.

Evaluation service phải bao bọc lời gọi engine bằng cơ chế xử lý exception:

Đoạn mã 2.95: Xử lý lỗi nội bộ của Evaluation Engine

```
async function evaluate(request) {
  try {
    const snapshot =
      await repository.findSnapshot(request);

    return evaluationEngine.evaluate(
      snapshot,
      request.context
    );
  } catch (error) {
    logger.error({
      event: "EVALUATION_ENGINE_ERROR",
      projectKey: request.projectKey,
      environmentKey: request.environmentKey,
      flagKey: request.flagKey,
      error
    });

    return {
```

```
      projectKey: request.projectKey,
      flagKey: request.flagKey,
      enabled: false,
      variant: "off",
      reason: "ERROR"
    };
  }
}
```

Engine không được trả về trạng thái bật nếu quá trình đánh giá bị gián đoạn trước khi xác định một nhánh hợp lệ. Kết quả fallback phải có `enabled = false`, `variant = off` và `reason = ERROR`.

Lỗi nội bộ khác với rule không đủ context. Thiếu context là trường hợp nghiệp vụ có thể dự đoán và dẫn đến rule không khớp. Exception nội bộ là lỗi kỹ thuật cần được ghi nhận, giám sát và khắc phục.

Để giảm khả năng lỗi runtime, hệ thống cần validation rule khi tạo hoặc cập nhật, sử dụng kiểu dữ liệu chặt chẽ, kiểm thử đơn vị cho từng nhánh evaluation và giữ engine gần với mô hình hàm thuần túy. Với cùng snapshot và context hợp lệ, engine phải trả về cùng một kết quả và không tạo side effect.

Thông tin lỗi chi tiết chỉ nên được ghi trong log nội bộ. Evaluation response không được chứa stack trace, câu lệnh SQL hoặc dữ liệu nhạy cảm. Việc xử lý lỗi REST cần giới hạn thông tin trả về và duy trì response contract ổn định [11, 12].

2.6.12 Khả năng mở rộng Rule Evaluation Engine

Rule Evaluation Engine trong hệ thống đề xuất được thiết kế theo hướng tách biệt giữa dữ liệu cấu hình, evaluation context và logic thực thi từng loại rule. Cách tổ chức này cho phép bổ sung chiến lược targeting mới mà không phải thay đổi toàn bộ luồng evaluation hoặc phá vỡ API contract hiện có.

Các hướng mở rộng chính gồm rule dựa trên thuộc tính, targeting theo organization, rule theo thời gian và vị trí, hỗ trợ nhiều variant và tách logic đánh giá theo Strategy Pattern. Những mở rộng này phù hợp với mô hình evaluation context linh hoạt, trong đó context có thể chứa

targeting key và các thuộc tính tùy chỉnh phục vụ quyết định runtime [6].

Rule dựa trên thuộc tính

Rule dựa trên thuộc tính cho phép engine đánh giá feature flag dựa trên các giá trị trong trường `attributes` của evaluation context. Các thuộc tính có thể mô tả quốc gia, loại thiết bị, phiên bản ứng dụng, gói dịch vụ, ngôn ngữ hoặc các thông tin nghiệp vụ khác.

Một rule dựa trên thuộc tính có cấu trúc khái quát như sau:

Đoạn mã 2.96: Cấu trúc rule dựa trên thuộc tính

```
{
  "id": "rule-device-type",
  "type": "ATTRIBUTE_TARGETING",
  "priority": 40,
  "parameters": {
    "attribute": "deviceType",
    "operator": "EQUALS",
    "value": "mobile"
  }
}
```

Để hỗ trợ nhiều kiểu điều kiện, hệ thống có thể định nghĩa các toán tử như:

- `EQUALS`: bằng chính xác;
- `NOT_EQUALS`: khác giá trị;
- `IN`: thuộc một tập giá trị;
- `NOT_IN`: không thuộc một tập giá trị;
- `GREATER_THAN`: lớn hơn;
- `LESS_THAN`: nhỏ hơn;
- `CONTAINS`: chuỗi hoặc mảng chứa giá trị;
- `STARTS_WITH`: chuỗi bắt đầu bằng tiền tố;
- `VERSION_GREATER_THAN`: so sánh phiên bản.

Quá trình đánh giá được biểu diễn như sau:

Đoạn mã 2.97: Đánh giá rule dựa trên thuộc tính

```
function evaluateAttributeRule(rule, context) {
  const attributeName =
    rule.parameters.attribute;

  const actualValue =
    context.attributes?.[attributeName];

  if (actualValue === undefined) {
    return false;
  }

  return compare(
    actualValue,
    rule.parameters.operator,
    rule.parameters.value
  );
}
```

Khi thuộc tính cần thiết không tồn tại, rule được xem là không khớp. Engine không tự suy diễn giá trị mặc định trừ khi rule contract quy định rõ.

Rule dựa trên thuộc tính làm tăng đáng kể khả năng biểu đạt của engine, nhưng cũng yêu cầu validation chặt chẽ về kiểu dữ liệu. Ví dụ, toán tử so sánh số không được áp dụng cho chuỗi, còn toán tử version phải sử dụng quy tắc so sánh phiên bản thay vì so sánh từ điển.

Các nền tảng feature flag thường sử dụng targeting rule dựa trên thuộc tính để kiểm soát tính năng theo nhiều đặc điểm của đối tượng đánh giá [4].

Rule theo organization

Rule theo organization cho phép bật feature flag cho toàn bộ người dùng thuộc một tổ chức, doanh nghiệp hoặc tenant cụ thể. Cơ chế này đặc biệt phù hợp với ứng dụng B2B hoặc hệ thống

đa tenant, trong đó một tính năng thường được phát hành theo khách hàng doanh nghiệp thay vì theo từng người dùng.

Evaluation context có thể chứa:

Đoạn mã 2.98: Organization ID trong evaluation context

```
{
  "targetingKey": "user-001",
  "userId": "user-001",
  "organizationId": "organization-a"
}
```

Rule targeting theo organization có thể được biểu diễn như sau:

Đoạn mã 2.99: Cấu trúc organization targeting rule

```
{
  "id": "rule-organization",
  "type": "ORGANIZATION_TARGETING",
  "priority": 25,
  "parameters": {
    "organizationIds": [
      "organization-a",
      "organization-b"
    ]
  }
}
```

Điều kiện khớp là:

$$context.organizationId \in rule.parameters.organizationIds \tag{2.40}$$

Đoạn mã đánh giá có thể được biểu diễn như sau:

Đoạn mã 2.100: So khớp organization ID

```
function evaluateOrganizationRule(rule, context) {
```

```
  if (!context.organizationId) {
    return false;
  }

  return rule.parameters.organizationIds.includes(
    context.organizationId
  );
}
```

Về thứ tự ưu tiên, organization targeting có thể được đặt sau user allowlist nhưng trước role targeting hoặc percentage rollout:

$$USER_ALLOWLIST > ORGANIZATION_TARGETING > ROLE_TARGETING > PERCENTAGE_ROLLOUT \tag{2.41}$$

Thứ tự này phản ánh mức độ cụ thể giảm dần từ một người dùng, một tổ chức, một nhóm role đến một tập phần trăm tổng quát.

Organization ID phải được lấy từ nguồn đáng tin cậy của ứng dụng khách. Nếu giá trị được gửi trực tiếp từ trình duyệt mà không được xác minh, người dùng có thể giả mạo organization để nhận tính năng không được phép. Vì vậy, các quyết định có ảnh hưởng đến quyền hoặc hợp đồng nên được thực hiện ở backend của ứng dụng tích hợp.

Rule theo thời gian và vị trí

Rule theo thời gian cho phép feature flag chỉ hoạt động trong một khoảng thời gian xác định. Ví dụ, hệ thống có thể bật tính năng từ thời điểm bắt đầu chiến dịch và tự động ngừng sau thời điểm kết thúc.

Một time window rule có thể được cấu hình như sau:

Đoạn mã 2.101: Cấu trúc rule theo thời gian

```
{
```

```
  "id": "rule-time-window",
  "type": "TIME_WINDOW",
  "priority": 35,
  "parameters": {
    "startAt": "2026-07-01T00:00:00Z",
    "endAt": "2026-07-31T23:59:59Z"
  }
}
```

Rule khớp khi thời điểm evaluation nằm trong khoảng:

$$startAt \leq now < endAt \tag{2.42}$$

Đoạn mã khái quát:

Đoạn mã 2.102: Đánh giá rule theo khoảng thời gian

```
function evaluateTimeWindow(rule, now) {
  const startAt =
    new Date(rule.parameters.startAt);

  const endAt =
    new Date(rule.parameters.endAt);

  return startAt <= now && now < endAt;
}
```

Hệ thống nên sử dụng UTC để lưu trữ và đánh giá thời gian nhằm tránh sai lệch do múi giờ. Thời điểm `now` nên được truyền vào engine như một dependency thay vì gọi trực tiếp đồng hồ hệ thống trong từng evaluator. Cách này giúp unit test có thể kiểm soát thời gian đầu vào.

Rule theo vị trí cho phép bật tính năng dựa trên quốc gia, khu vực hoặc thị trường. Vị trí có thể được truyền qua thuộc tính như `country` hoặc `region`:

Đoạn mã 2.103: Cấu trúc rule theo vị trí

```
{
  "id": "rule-country",
  "type": "LOCATION_TARGETING",
  "priority": 35,
  "parameters": {
    "countries": ["VN", "SG"]
  }
}
```

Điều kiện khớp là:

$$context.attributes.country \in rule.parameters.countries \tag{2.43}$$

Vị trí không nên được suy diễn trực tiếp trong Rule Evaluation Engine từ địa chỉ IP. Việc xác định vị trí nên được thực hiện bởi tầng ứng dụng hoặc một dịch vụ chuyên biệt, sau đó truyền giá trị đã chuẩn hóa vào evaluation context. Điều này giữ engine độc lập với network I/O và giúp kết quả dễ kiểm thử hơn.

Hỗ trợ nhiều variant

Hệ thống hiện tại sử dụng hai variant `on` và `off`. Trong tương lai, engine có thể được mở rộng để hỗ trợ nhiều variant nhằm phục vụ A/B testing, thử nghiệm giao diện hoặc phân phối nhiều cấu hình khác nhau.

Ví dụ, một feature flag có thể định nghĩa ba variant:

Đoạn mã 2.104: Cấu hình feature flag nhiều variant

```
{
  "variants": [
    {
      "key": "control",
      "weight": 50
    },
    {
```

```
      "key": "blue-layout",
      "weight": 30
    },
    {
      "key": "compact-layout",
      "weight": 20
    }
  ]
}
```

Tổng trọng số phải bằng 100:

$$\sum_{i=1}^{n} weight_i = 100 \tag{2.44}$$

Stable hashing tiếp tục được sử dụng để tạo bucket. Bucket sau đó được ánh xạ vào các khoảng trọng số tích lũy:

$$variant(b) = \begin{cases} v_1, & 0 \le b < w_1, \\ v_2, & w_1 \le b < w_1 + w_2, \\ \dots & \\ v_n, & \sum_{i=1}^{n-1} w_i \le b < 100. \end{cases} \tag{2.45}$$

Ví dụ:

- bucket từ 0 đến nhỏ hơn 50 nhận variant `control`;
- bucket từ 50 đến nhỏ hơn 80 nhận variant `blue-layout`;
- bucket từ 80 đến nhỏ hơn 100 nhận variant `compact-layout`.

EvaluationResult có thể được mở rộng như sau:

Đoạn mã 2.105: EvaluationResult cho flag nhiều variant

```
{
  "enabled": true,
```

```
  "variant": "blue-layout",
  "reason": "PERCENTAGE_MATCH",
  "matchedRuleId": "rule-experiment"
}
```

Hỗ trợ nhiều variant giúp cùng một feature flag trả về nhiều cấu hình khác nhau thay vì chỉ quyết định bật hoặc tắt. Tuy nhiên, hệ thống cần bổ sung validation tổng trọng số, stable assignment theo variant và metadata phục vụ phân tích thử nghiệm [7, 5].

Tách rule evaluator theo Strategy Pattern

Khi số lượng loại rule tăng lên, việc xử lý tất cả rule trong một hàm lớn với nhiều câu lệnh `if` hoặc `switch` sẽ làm engine khó bảo trì và khó kiểm thử. Strategy Pattern cho phép tách logic của từng loại rule thành một evaluator độc lập có cùng interface.

Interface chung có thể được định nghĩa như sau:

Đoạn mã 2.106: Interface của rule evaluator

```
interface RuleEvaluator {
  supports(ruleType: RuleType): boolean;

  evaluate(
    rule: EvaluationRule,
    context: EvaluationContext,
    runtime: EvaluationRuntime
  ): RuleMatchResult;
}
```

Mỗi loại rule có một implementation riêng:

Đoạn mã 2.107: Các strategy đánh giá rule

```
class UserAllowlistEvaluator
  implements RuleEvaluator {
  // ...
}
```

```
class RoleTargetingEvaluator
  implements RuleEvaluator {
  // ...
}

class PercentageRolloutEvaluator
  implements RuleEvaluator {
  // ...
}

class AttributeTargetingEvaluator
  implements RuleEvaluator {
  // ...
}
```

Engine duy trì một registry của các evaluator:

Đoạn mã 2.108: Registry của các rule evaluator

```
class RuleEvaluatorRegistry {
  constructor(
    private readonly evaluators:
      RuleEvaluator[]
  ) {}

  resolve(ruleType) {
    return this.evaluators.find(
      (evaluator) =>
        evaluator.supports(ruleType)
    );
  }
}
```

Luồng thực thi rule sau khi áp dụng Strategy Pattern:

Đoạn mã 2.109: Thực thi rule thông qua evaluator strategy

```
for (const rule of orderedRules) {
  const evaluator =
    registry.resolve(rule.type);

  if (!evaluator) {
    logger.warn({
      event: "UNSUPPORTED_RULE_TYPE",
      ruleId: rule.id,
      ruleType: rule.type
    });

    continue;
  }

  const result = evaluator.evaluate(
    rule,
    context,
    runtime
  );

  if (result.matched) {
    return createEnabledResult(
      rule,
      result.reason,
      result.variant
    );
  }
}
```

Strategy Pattern mang lại các lợi ích sau:

- mỗi evaluator chỉ chịu trách nhiệm cho một loại rule;
- dễ bổ sung rule mới mà không sửa logic của các evaluator hiện có;
- unit test có thể kiểm tra từng strategy độc lập;
- giảm độ phức tạp của hàm evaluation chính;
- hỗ trợ dependency injection cho các thành phần như clock hoặc hashing service.

Thiết kế này tuân theo nguyên tắc mở rộng mà hạn chế sửa đổi. Khi bổ sung rule theo organization, thời gian hoặc thuộc tính, hệ thống chỉ cần triển khai evaluator mới, đăng ký vào registry và cập nhật validation contract.

Rule Evaluation Engine chính vẫn chịu trách nhiệm cho các bước chung gồm kiểm tra precedence toàn cục, sắp xếp rule, tìm evaluator phù hợp, dừng tại rule đầu tiên khớp và tạo EvaluationResult. Logic đặc thù của từng loại rule được đóng gói trong các strategy riêng biệt, giúp cấu trúc hệ thống rõ ràng và dễ mở rộng.

2.7 Thiết kế API

2.7.1 Nguyên tắc thiết kế API

API của hệ thống được thiết kế nhằm cung cấp giao diện thống nhất cho dashboard quản trị, demo application và các ứng dụng khách tích hợp Feature Flag. Thiết kế ưu tiên tính rõ ràng, khả năng dự đoán, khả năng mở rộng và xử lý lỗi nhất quán. Các endpoint được tổ chức theo tài nguyên nghiệp vụ, sử dụng định dạng JSON và tuân theo các nguyên tắc phổ biến của kiến trúc REST.

Kiến trúc REST

Hệ thống sử dụng kiến trúc REST, trong đó project, environment, feature flag, flag configuration, rule và audit log được biểu diễn dưới dạng các resource. Mỗi resource được định danh bằng URL và được thao tác thông qua các HTTP method phù hợp.

Các phương thức HTTP chính gồm:

- `GET`: truy vấn một resource hoặc danh sách resource;
- `POST`: tạo resource mới hoặc thực hiện thao tác evaluation;

- `PATCH`: cập nhật một phần trạng thái hoặc thuộc tính của resource;
- `DELETE`: xóa resource khi việc xóa vật lý được cho phép.

Ví dụ, endpoint `GET /v1/projects` trả về danh sách project, trong khi `POST /v1/projects` tạo một project mới. Việc cập nhật trạng thái của feature flag được thực hiện bằng `PATCH` vì request chỉ thay đổi một phần dữ liệu hiện có.

Đối với feature flag, thao tác archive được ưu tiên thay cho xóa vật lý nhằm bảo toàn lịch sử cấu hình và audit log. Do đó, feature flag được archive thông qua endpoint cập nhật trạng thái thay vì bị loại bỏ hoàn toàn khỏi cơ sở dữ liệu.

Evaluation API được thiết kế dưới dạng `POST /v1/evaluate`. Mặc dù thao tác evaluation không làm thay đổi cấu hình flag, request sử dụng `POST` vì evaluation context được truyền trong request body và có thể chứa nhiều trường như targeting key, user ID, role và custom attributes. Thiết kế này cũng tránh việc đưa thông tin ngữ cảnh hoặc dữ liệu có khả năng nhạy cảm vào query string [6, 10].

API sử dụng JSON cho cả request và response. Mỗi response trả về HTTP status code phù hợp với kết quả xử lý, đồng thời sử dụng cấu trúc dữ liệu thống nhất để client có thể xử lý một cách dự đoán được.

Versioning đường dẫn API

Các endpoint được đặt dưới tiền tố phiên bản `/v1`. Ví dụ:

```
POST /v1/evaluate
GET  /v1/projects
GET  /v1/projects/{projectId}/flags
```

Versioning trên đường dẫn giúp phân biệt rõ ràng giữa các phiên bản API và cho phép hệ thống phát triển mà không làm gián đoạn các ứng dụng khách đang sử dụng phiên bản cũ. Khi xuất hiện thay đổi không tương thích ngược, chẳng hạn thay đổi cấu trúc request, response hoặc ý nghĩa của một trường dữ liệu, hệ thống có thể cung cấp phiên bản mới dưới tiền tố `/v2`.

Các thay đổi tương thích ngược, như bổ sung trường tùy chọn vào response hoặc thêm endpoint mới, không yêu cầu tăng phiên bản API. Ngược lại, việc xóa trường, đổi tên trường hoặc thay đổi kiểu dữ liệu được xem là breaking change và cần được triển khai trong phiên bản mới.

Trong phạm vi mini project, toàn bộ endpoint sử dụng phiên bản `v1`. Cách tổ chức này đủ đơn giản cho việc triển khai hiện tại nhưng vẫn thể hiện khả năng mở rộng và duy trì hợp đồng tích hợp trong tương lai.

Quy ước đặt tên resource

Tên resource trong URL sử dụng danh từ số nhiều, chữ thường và dấu gạch ngang khi cần phân tách nhiều từ. Các URL không sử dụng động từ để mô tả thao tác vì hành động đã được thể hiện bởi HTTP method.

Một số resource chính gồm:

```
GET /v1/projects
GET /v1/environments
GET /v1/feature-flags
GET /v1/flag-configs
GET /v1/flag-rules
GET /v1/audit-logs
```

Đối với tài nguyên phụ thuộc vào resource cha, API sử dụng URL lồng nhau để thể hiện phạm vi. Ví dụ:

```
GET  /v1/projects/{projectId}/environments
GET  /v1/projects/{projectId}/flags
POST /v1/flags/{flagId}/rules
GET  /v1/flags/{flagId}/audit-logs
```

Các tham số định danh trong URL sử dụng hậu tố `Id`, chẳng hạn `projectId`, `flagId` và `ruleId`. Các khóa nghiệp vụ như `projectKey`, `environmentKey` và `flagKey` được sử dụng trong request body hoặc query parameter khi cần tìm kiếm resource theo khóa có ý nghĩa đối với ứng dụng khách.

Tên trường JSON sử dụng quy ước `camelCase`, ví dụ:

```
{
  "projectKey": "ecommerce-demo",
  "environmentKey": "production",
```

```
  "flagKey": "new-checkout",
  "targetingKey": "user-001"
}
```

Quy ước thống nhất giữa URL, DTO và response giúp giảm nhầm lẫn trong quá trình phát triển frontend, backend và SDK.

Phân trang và lọc dữ liệu

Các endpoint trả về danh sách resource hỗ trợ phân trang bằng hai query parameter là `limit` và `offset`. Trong đó, `limit` xác định số bản ghi tối đa được trả về và `offset` xác định số bản ghi cần bỏ qua từ đầu tập kết quả.

Ví dụ:

```
GET /v1/projects?limit=20&offset=0
GET /v1/audit-logs?limit=50&offset=100
```

Giá trị mặc định của `limit` là 20, giá trị mặc định của `offset` là 0 và giá trị tối đa của `limit` là 100. Việc giới hạn số lượng bản ghi trong mỗi request giúp tránh tải quá nhiều dữ liệu vào bộ nhớ và giảm thời gian phản hồi của API.

Response danh sách bao gồm dữ liệu và metadata phân trang:

```
{
  "items": [],
  "pagination": {
    "limit": 20,
    "offset": 0,
    "total": 125
  }
}
```

Các endpoint danh sách có thể hỗ trợ thêm query parameter để lọc dữ liệu theo trạng thái, project, environment hoặc từ khóa tìm kiếm. Ví dụ:

```
GET /v1/feature-flags?projectId=...&lifecycleStatus=ACTIVE
GET /v1/audit-logs?targetType=FEATURE_FLAG&actorId=...
```

```
GET /v1/projects?search=ecommerce
```

Các điều kiện lọc được kết hợp theo phép hội. Những query parameter không được cung cấp sẽ không tham gia vào điều kiện truy vấn. Backend thực hiện validation giá trị query parameter trước khi truy vấn cơ sở dữ liệu nhằm tránh các điều kiện không hợp lệ hoặc gây tải không cần thiết.

Kết quả danh sách phải có thứ tự xác định. Nếu client không cung cấp tiêu chí sắp xếp, hệ thống sử dụng thứ tự mặc định theo thời điểm tạo giảm dần hoặc theo khóa nghiệp vụ tăng dần, tùy loại resource. Việc xác định rõ thứ tự giúp kết quả phân trang ổn định giữa các request liên tiếp.

Idempotency và tính nhất quán

Idempotency thể hiện việc gửi cùng một request nhiều lần nhưng không làm thay đổi kết quả cuối cùng của hệ thống sau lần xử lý đầu tiên. Các endpoint sử dụng `GET`, `PUT` và `DELETE` về nguyên tắc cần có tính idempotent, trong khi `POST` thường không bảo đảm tính chất này.

Trong hệ thống, các thao tác cập nhật trạng thái bằng `PATCH` được thiết kế theo hướng idempotent. Ví dụ, việc gửi nhiều lần request đặt trạng thái flag thành `DISABLED` vẫn tạo ra cùng một trạng thái cuối cùng:

```
PATCH /v1/flag-configs/{configId}

{
  "status": "DISABLED"
}
```

Đối với endpoint tạo resource bằng `POST`, backend áp dụng ràng buộc duy nhất trên các khóa nghiệp vụ như project key, environment key và flag key. Nếu client gửi lại request tạo resource với cùng khóa, hệ thống trả về lỗi `409 Conflict` thay vì tạo bản ghi trùng lặp.

Evaluation API có tính idempotent về mặt kết quả khi cấu hình flag và evaluation context không thay đổi. Cùng một project key, environment key, flag key và targeting key phải tạo ra cùng một kết quả evaluation. Đặc biệt, percentage rollout sử dụng hàm băm xác định thay vì giá trị ngẫu nhiên để bảo đảm một người dùng được gán ổn định vào cùng một bucket qua nhiều lần evaluation [5].

Các thao tác thay đổi nhiều resource liên quan, chẳng hạn cập nhật flag configuration và ghi audit log, được thực hiện trong cùng một database transaction. Nếu một bước thất bại, toàn bộ transaction được rollback nhằm tránh trạng thái cấu hình đã thay đổi nhưng audit log chưa được ghi nhận.

Đối với các request cập nhật cấu hình đồng thời, hệ thống sử dụng timestamp cập nhật hoặc version cấu hình để phát hiện dữ liệu cũ. Client không được ghi đè một phiên bản mới hơn bằng dữ liệu được đọc từ phiên bản cũ mà không có kiểm tra phù hợp. Nguyên tắc này giúp hạn chế lost update và duy trì tính nhất quán của cấu hình Feature Flag.

Sau khi mutation thành công, cache liên quan đến project, environment hoặc flag phải được invalidation trước khi request tiếp theo sử dụng dữ liệu mới. Cơ sở dữ liệu được xem là nguồn dữ liệu chính, trong khi cache chỉ đóng vai trò tăng hiệu năng và không được phép trở thành nguồn cấu hình độc lập.

2.7.2 API quản lý project và environment

Project là phạm vi quản lý cấp cao nhất của hệ thống Feature Flag. Mỗi project đại diện cho một ứng dụng hoặc một sản phẩm độc lập và chứa các environment, feature flag, user context mẫu và audit log tương ứng. Environment biểu diễn các môi trường triển khai như development, staging và production, cho phép cùng một feature flag có cấu hình khác nhau theo từng môi trường.

Tạo và truy vấn project

Project được tạo thông qua endpoint:

```
POST /v1/projects
```

Request body gồm tên hiển thị, khóa định danh và mô tả tùy chọn:

```
{
  "key": "ecommerce-demo",
  "name": "E-commerce Demo",
  "description": "Project used by the demonstration application."
}
```

Trường `key` là khóa nghiệp vụ được sử dụng khi tích hợp Evaluation API. Giá trị này phải tuân theo biểu thức chính quy:

```
^[a-z0-9][a-z0-9-]{2,63}$
```

Theo quy ước trên, project key chỉ gồm chữ thường, chữ số và dấu gạch ngang, có độ dài từ 3 đến 64 ký tự và không bắt đầu bằng dấu gạch ngang. Project key phải duy nhất trong toàn hệ thống và không được thay đổi sau khi project được tạo nhằm duy trì tính ổn định của các tích hợp bên ngoài.

Response thành công sử dụng mã trạng thái `201 Created`:

```
{
  "id": "project-id",
  "key": "ecommerce-demo",
  "name": "E-commerce Demo",
  "description": "Project used by the demonstration application.",
  "lifecycleStatus": "ACTIVE",
  "createdAt": "2026-07-03T10:00:00.000Z",
  "updatedAt": "2026-07-03T10:00:00.000Z"
}
```

Nếu project key đã tồn tại, hệ thống trả về `409 Conflict`. Nếu request không thỏa mãn các ràng buộc validation, hệ thống trả về `400 Bad Request` cùng danh sách trường không hợp lệ.

Danh sách project được truy vấn thông qua endpoint:

```
GET /v1/projects
```

Endpoint hỗ trợ phân trang, tìm kiếm và lọc theo trạng thái vòng đời:

```
GET /v1/projects?limit=20&offset=0
GET /v1/projects?search=ecommerce
GET /v1/projects?lifecycleStatus=ACTIVE
```

Kết quả trả về bao gồm danh sách project và metadata phân trang:

```
{
  "items": [
```

```
    {
      "id": "project-id",
      "key": "ecommerce-demo",
      "name": "E-commerce Demo",
      "lifecycleStatus": "ACTIVE",
      "environmentCount": 3,
      "flagCount": 12,
      "createdAt": "2026-07-03T10:00:00.000Z",
      "updatedAt": "2026-07-03T10:00:00.000Z"
    }
  ],
  "pagination": {
    "limit": 20,
    "offset": 0,
    "total": 1
  }
}
```

Thông tin chi tiết của một project được truy vấn bằng định danh nội bộ:

```
GET /v1/projects/{projectId}
```

Response chi tiết có thể bao gồm environment mặc định, số lượng feature flag và các metadata cần thiết cho dashboard. Nếu project không tồn tại, hệ thống trả về `404 Not Found`.

Cập nhật và archive project

Thông tin có thể chỉnh sửa của project được cập nhật thông qua endpoint:

```
PATCH /v1/projects/{projectId}
```

Request chỉ cần chứa các trường cần thay đổi:

```
{
  "name": "E-commerce Platform",
  "description": "Feature flags for the e-commerce application."
}
```

Project key không được phép thay đổi sau khi khởi tạo vì khóa này có thể đã được sử dụng trong Evaluation API, SDK và cấu hình của các ứng dụng khách. Việc giữ nguyên project key giúp tránh phá vỡ hợp đồng tích hợp và hạn chế yêu cầu cập nhật đồng thời trên nhiều hệ thống.

Mỗi lần cập nhật thành công, backend ghi một audit log chứa actor, request ID, dữ liệu trước thay đổi và dữ liệu sau thay đổi. Việc cập nhật project và ghi audit log được thực hiện trong cùng một transaction để bảo đảm tính nhất quán.

Thay vì xóa vật lý, project được archive thông qua endpoint:

```
PATCH /v1/projects/{projectId}/archive
```

Khi archive, trạng thái vòng đời của project được chuyển từ `ACTIVE` sang `ARCHIVED`. Thao tác này không xóa environment, feature flag, rule hoặc audit log thuộc project. Nhờ đó, hệ thống vẫn bảo toàn được lịch sử cấu hình và khả năng truy vết.

Project đã archive không cho phép tạo mới hoặc sửa đổi environment, feature flag và rule. Các endpoint mutation đối với resource thuộc project sẽ trả về `409 Conflict` hoặc lỗi nghiệp vụ tương ứng. Evaluation API đối với project đã archive trả về kết quả an toàn với trạng thái tắt và reason code phù hợp.

Project có thể được khôi phục thông qua endpoint:

```
PATCH /v1/projects/{projectId}/restore
```

Thao tác archive và restore được thiết kế idempotent. Việc archive một project đã ở trạng thái `ARCHIVED` hoặc restore một project đang ở trạng thái `ACTIVE` không tạo ra trạng thái trung gian không hợp lệ.

Quản lý environment

Environment được quản lý trong phạm vi một project. Endpoint tạo environment có dạng:

```
POST /v1/projects/{projectId}/environments
```

Request body gồm khóa, tên hiển thị, mô tả và trạng thái environment mặc định:

```
{
  "key": "production",
```

```
  "name": "Production",
  "description": "Production environment.",
  "isDefault": false
}
```

Environment key phải duy nhất trong phạm vi project. Hai project khác nhau có thể cùng sử dụng các environment key phổ biến như `development`, `staging` và `production`. Tuy nhiên, trong cùng một project không được tồn tại hai environment có cùng key.

Response thành công sử dụng mã trạng thái `201 Created`:

```
{
  "id": "environment-id",
  "projectId": "project-id",
  "key": "production",
  "name": "Production",
  "description": "Production environment.",
  "isDefault": false,
  "createdAt": "2026-07-03T10:10:00.000Z",
  "updatedAt": "2026-07-03T10:10:00.000Z"
}
```

Danh sách environment của project được truy vấn bằng endpoint:

```
GET /v1/projects/{projectId}/environments
```

Thông tin chi tiết của một environment được truy vấn bằng:

```
GET /v1/environments/{environmentId}
```

Environment được cập nhật thông qua endpoint:

```
PATCH /v1/environments/{environmentId}
```

Các trường có thể cập nhật gồm `name` và `description`. Environment key được giữ ổn định sau khi tạo vì giá trị này có thể được ứng dụng khách truyền trong Evaluation API.

Trong phạm vi hệ thống, environment không được xóa nếu đang được tham chiếu bởi các flag configuration. Trường hợp cần loại bỏ một environment, hệ thống chỉ cho phép thực hiện khi

environment không còn cấu hình liên quan và không phải environment mặc định. Quy tắc này giúp ngăn việc xóa ngoài ý muốn toàn bộ cấu hình Feature Flag của một môi trường đang hoạt động.

Mỗi thao tác tạo, cập nhật hoặc xóa environment đều tạo audit log. Audit log được liên kết với project và environment tương ứng để hỗ trợ truy vết lịch sử quản trị.

Thiết lập environment mặc định

Mỗi project phải có tối đa một environment mặc định. Environment mặc định được sử dụng khi Evaluation API không nhận được `environmentKey` từ ứng dụng khách.

Environment mặc định được thiết lập thông qua endpoint:

```
PATCH /v1/projects/{projectId}/environments/{environmentId}/default
```

Request không cần body vì hành động đã được xác định đầy đủ bởi URL. Khi endpoint được gọi, backend thực hiện các bước sau trong cùng một transaction:

1. Kiểm tra project và environment có tồn tại;
2. Kiểm tra environment thuộc đúng project;
3. Đặt `isDefault = false` cho environment mặc định hiện tại;
4. Đặt `isDefault = true` cho environment được lựa chọn;
5. Ghi audit log cho thay đổi.

Cơ sở dữ liệu sử dụng partial unique index để bảo đảm chỉ có một environment có `isDefault = true` trong mỗi project. Ràng buộc ở tầng cơ sở dữ liệu giúp duy trì tính đúng đắn ngay cả khi có nhiều request cập nhật đồng thời.

Ví dụ response:

```
{
  "id": "environment-id",
  "projectId": "project-id",
  "key": "production",
  "name": "Production",
  "isDefault": true,
```

```
  "updatedAt": "2026-07-03T10:15:00.000Z"
}
```

Nếu environment không thuộc project được chỉ định, hệ thống trả về `404 Not Found` nhằm tránh làm lộ sự tồn tại của resource ngoài phạm vi truy cập. Nếu project đã archive, thao tác thiết lập environment mặc định bị từ chối.

Khi project chưa có environment mặc định, environment đầu tiên có thể được tự động đặt làm mặc định. Đối với các environment được tạo sau đó, trường `isDefault` mặc định là `false`, trừ khi client yêu cầu thiết lập làm mặc định và thao tác được thực hiện thành công trong cùng transaction.

Việc lựa chọn environment mặc định giúp Evaluation API đơn giản hơn đối với các ứng dụng chỉ sử dụng một môi trường. Tuy nhiên, ứng dụng khách triển khai trên nhiều môi trường nên truyền rõ `environmentKey` để tránh đánh giá nhầm cấu hình.

2.7.3 API quản lý Feature Flag

Feature Flag là tài nguyên trung tâm của hệ thống, đại diện cho một điểm kiểm soát hành vi trong ứng dụng khách. Mỗi flag thuộc về một project, được định danh bằng khóa nghiệp vụ ổn định và có thể sở hữu cấu hình riêng cho từng environment. API quản lý Feature Flag hỗ trợ tạo mới, truy vấn, cập nhật metadata, thay đổi trạng thái vòng đời và quản lý cấu hình runtime theo environment.

Tạo Feature Flag

Feature Flag được tạo trong phạm vi một project thông qua endpoint:

```
POST /v1/projects/{projectId}/feature-flags
```

Request body gồm khóa định danh, tên hiển thị, mô tả và các thông tin metadata cơ bản:

```
{
  "key": "new-checkout",
  "name": "New Checkout",
  "description": "Controls access to the redesigned checkout flow."
}
```

Trường `key` là khóa nghiệp vụ được ứng dụng khách sử dụng khi gọi Evaluation API. Giá trị này phải tuân theo biểu thức chính quy:

```
^[a-z0-9][a-z0-9-]{2,63}$
```

Flag key phải duy nhất trong phạm vi project. Hai project khác nhau có thể sử dụng cùng một flag key, nhưng trong cùng một project không được tồn tại hai flag có cùng khóa. Sau khi được tạo, flag key không được phép thay đổi nhằm tránh phá vỡ các đoạn mã tích hợp, SDK và request evaluation đang sử dụng khóa này.

Khi tạo flag, backend kiểm tra project tồn tại và đang ở trạng thái `ACTIVE`. Nếu project đã archive, request bị từ chối vì các resource thuộc project không còn được phép thay đổi.

Response thành công sử dụng mã trạng thái `201 Created`:

```
{
  "id": "flag-id",
  "projectId": "project-id",
  "key": "new-checkout",
  "name": "New Checkout",
  "description": "Controls access to the redesigned checkout flow.",
  "lifecycleStatus": "ACTIVE",
  "createdAt": "2026-07-03T11:00:00.000Z",
  "updatedAt": "2026-07-03T11:00:00.000Z"
}
```

Nếu flag key đã tồn tại trong project, hệ thống trả về `409 Conflict`. Nếu dữ liệu đầu vào không hợp lệ, hệ thống trả về `400 Bad Request` cùng thông tin chi tiết về các trường vi phạm validation.

Sau khi flag được tạo, hệ thống có thể tự động tạo các bản ghi cấu hình mặc định cho những environment hiện có của project. Mỗi cấu hình ban đầu sử dụng trạng thái an toàn:

- `status = DISABLED`;
- `servingMode = TARGETED`;
- `killSwitch = false`;

- variant mặc định là `off`.

Cách khởi tạo này bảo đảm tính năng mới không được bật ngoài ý muốn ngay sau khi flag được tạo. Nguyên tắc mặc định tắt giúp giảm rủi ro phát hành nhầm và phù hợp với chiến lược fail-safe của hệ thống Feature Flag [8].

Thao tác tạo flag và tạo các cấu hình ban đầu được thực hiện trong cùng một database transaction. Nếu một bước thất bại, toàn bộ dữ liệu được rollback để tránh tồn tại flag không có cấu hình môi trường tương ứng.

Truy vấn danh sách và chi tiết flag

Danh sách Feature Flag của một project được truy vấn thông qua endpoint:

```
GET /v1/projects/{projectId}/feature-flags
```

Endpoint hỗ trợ phân trang, tìm kiếm và lọc theo trạng thái vòng đời:

```
GET /v1/projects/{projectId}/feature-flags?limit=20&offset=0
GET /v1/projects/{projectId}/feature-flags?search=checkout
GET /v1/projects/{projectId}/feature-flags?lifecycleStatus=ACTIVE
```

Kết quả trả về gồm danh sách flag và metadata phân trang:

```
{
  "items": [
    {
      "id": "flag-id",
      "projectId": "project-id",
      "key": "new-checkout",
      "name": "New Checkout",
      "description": "Controls access to the redesigned checkout flow.",
      "lifecycleStatus": "ACTIVE",
      "configurationCount": 3,
      "createdAt": "2026-07-03T11:00:00.000Z",
      "updatedAt": "2026-07-03T11:00:00.000Z"
    }
```

```
  ],
  "pagination": {
    "limit": 20,
    "offset": 0,
    "total": 1
  }
}
```

Kết quả mặc định được sắp xếp theo thời điểm cập nhật giảm dần để các flag vừa thay đổi xuất hiện trước trên dashboard. Client có thể sử dụng các query parameter bổ sung để lọc flag theo environment, trạng thái cấu hình hoặc group khi các chức năng này được hỗ trợ.

Thông tin chi tiết của một flag được truy vấn bằng endpoint:

```
GET /v1/feature-flags/{flagId}
```

Response chi tiết bao gồm metadata của flag và danh sách cấu hình theo environment:

```
{
  "id": "flag-id",
  "projectId": "project-id",
  "key": "new-checkout",
  "name": "New Checkout",
  "description": "Controls access to the redesigned checkout flow.",
  "lifecycleStatus": "ACTIVE",
  "configurations": [
    {
      "id": "config-id",
      "environment": {
        "id": "environment-id",
        "key": "production",
        "name": "Production"
      },
      "status": "ENABLED",
      "servingMode": "TARGETED",
```

```
      "killSwitch": false,
      "version": 4
    }
  ],
  "createdAt": "2026-07-03T11:00:00.000Z",
  "updatedAt": "2026-07-03T11:30:00.000Z"
}
```

Danh sách chi tiết không nhất thiết trả về toàn bộ rule parameter nếu dashboard chưa yêu cầu. Các rule có thể được tải thông qua endpoint riêng nhằm giảm kích thước response và tránh truy vấn dữ liệu không cần thiết.

Nếu flag không tồn tại, hệ thống trả về `404 Not Found`. Flag đã archive vẫn có thể được truy vấn để phục vụ kiểm tra lịch sử, nhưng response phải thể hiện rõ `lifecycleStatus = ARCHIVED`.

Cập nhật thông tin flag

Metadata của Feature Flag được cập nhật thông qua endpoint:

```
PATCH /v1/feature-flags/{flagId}
```

Request chỉ cần chứa các trường cần thay đổi:

```
{
  "name": "Redesigned Checkout",
  "description": "Controls the new multi-step checkout experience."
}
```

Các trường có thể cập nhật gồm `name` và `description`. Trường `key` không được phép thay đổi sau khi flag được tạo vì đây là định danh được sử dụng trực tiếp trong mã nguồn và Evaluation API.

Backend kiểm tra flag tồn tại, thuộc project đang hoạt động và chưa bị archive. Nếu flag đã archive, thao tác cập nhật metadata bị từ chối cho đến khi flag được restore.

Response thành công trả về trạng thái mới nhất của flag:

```
{
```

```
  "id": "flag-id",
  "projectId": "project-id",
  "key": "new-checkout",
  "name": "Redesigned Checkout",
  "description": "Controls the new multi-step checkout experience.",
  "lifecycleStatus": "ACTIVE",
  "createdAt": "2026-07-03T11:00:00.000Z",
  "updatedAt": "2026-07-03T11:45:00.000Z"
}
```

Mỗi lần cập nhật tạo một audit log với action tương ứng, thông tin actor, request ID, snapshot trước thay đổi và snapshot sau thay đổi. Việc cập nhật flag và ghi audit log được thực hiện trong cùng một transaction.

Metadata của flag không trực tiếp làm thay đổi kết quả evaluation. Tuy nhiên, sau khi mutation thành công, cache liên quan vẫn có thể được invalidation để bảo đảm các response quản trị và SDK không tiếp tục sử dụng dữ liệu cũ.

Archive và restore flag

Feature Flag không được xóa vật lý trong luồng quản trị thông thường. Khi không còn sử dụng, flag được archive thông qua endpoint:

```
PATCH /v1/feature-flags/{flagId}/archive
```

Thao tác archive chuyển `lifecycleStatus` từ `ACTIVE` sang `ARCHIVED`. Các cấu hình, rule, thống kê evaluation và audit log liên quan vẫn được giữ nguyên trong cơ sở dữ liệu.

Trước khi archive, backend kiểm tra trạng thái hiện tại của flag. Nếu flag đang được bật trong một hoặc nhiều environment, hệ thống có thể yêu cầu người quản trị xác nhận hoặc tự động chuyển các cấu hình về trạng thái tắt trước khi hoàn tất thao tác. Trong phạm vi mini project, quá trình archive được thực hiện trong transaction gồm:

1. Chuyển trạng thái các cấu hình của flag thành `DISABLED`;
2. Chuyển vòng đời flag thành `ARCHIVED`;
3. Tăng version của các cấu hình bị thay đổi;

4. Ghi audit log cho toàn bộ thao tác;
5. Invalidate các cache liên quan đến flag.

Evaluation API khi gặp flag đã archive luôn trả về trạng thái tắt:

```
{
  "projectKey": "ecommerce-demo",
  "flagKey": "new-checkout",
  "enabled": false,
  "variant": "off",
  "reason": "FLAG_ARCHIVED"
}
```

Flag được khôi phục thông qua endpoint:

```
PATCH /v1/feature-flags/{flagId}/restore
```

Thao tác restore chỉ chuyển vòng đời của flag về `ACTIVE`. Các cấu hình theo environment vẫn giữ trạng thái `DISABLED` để tránh tự động bật lại tính năng trên production. Người quản trị phải chủ động kích hoạt từng cấu hình sau khi kiểm tra.

Archive và restore được thiết kế theo hướng idempotent. Gửi lại request archive đối với flag đã archive hoặc restore đối với flag đang active không làm phát sinh trạng thái không nhất quán. Hệ thống có thể trả về trạng thái hiện tại hoặc lỗi nghiệp vụ rõ ràng mà không tạo thêm thay đổi dữ liệu.

Việc archive thay cho xóa vật lý giúp duy trì khả năng truy vết, bảo toàn các tham chiếu audit log và hạn chế nguy cơ tái sử dụng nhầm một flag key cũ. Quản lý vòng đời và loại bỏ các flag không còn sử dụng cũng giúp giảm feature flag debt trong hệ thống [9].

Quản lý cấu hình theo environment

Một Feature Flag có thể có nhiều cấu hình, mỗi cấu hình tương ứng với một environment. Cách tổ chức này cho phép cùng một flag được bật trên development, rollout giới hạn trên staging và tắt hoàn toàn trên production.

Cấu hình của flag trong một environment được truy vấn bằng endpoint:

```
GET /v1/feature-flags/{flagId}/configurations/{environmentId}
```

Response gồm trạng thái phục vụ, kill switch, version và danh sách rule:

```
{
  "id": "config-id",
  "flagId": "flag-id",
  "environmentId": "environment-id",
  "status": "ENABLED",
  "servingMode": "TARGETED",
  "killSwitch": false,
  "defaultVariant": "off",
  "version": 4,
  "rules": [
    {
      "id": "rule-id",
      "type": "ROLE_TARGETING",
      "priority": 10,
      "parameters": {
        "roles": ["beta-customer"]
      }
    }
  ],
  "createdAt": "2026-07-03T11:00:00.000Z",
  "updatedAt": "2026-07-03T12:00:00.000Z"
}
```

Cấu hình được cập nhật thông qua endpoint:

```
PATCH /v1/feature-flags/{flagId}/configurations/{environmentId}
```

Ví dụ request bật flag theo chế độ toàn cục:

```
{
  "status": "ENABLED",
  "servingMode": "GLOBAL_ON",
  "killSwitch": false,
```

```
  "expectedVersion": 4
}
```

Ví dụ request chuyển flag sang chế độ đánh giá theo rule:

```
{
  "status": "ENABLED",
  "servingMode": "TARGETED",
  "killSwitch": false,
  "expectedVersion": 5
}
```

Trường `status` xác định cấu hình có tham gia evaluation hay không. Khi `status = DISABLED`, Evaluation Engine trả về trạng thái tắt trước khi xét serving mode và các rule.

Trường `servingMode` có hai giá trị chính:

- `GLOBAL_ON`: bật tính năng cho toàn bộ context hợp lệ trong environment;
- `TARGETED`: đánh giá tuần tự các rule targeting và percentage rollout.

Trường `killSwitch` có độ ưu tiên cao hơn serving mode và các rule. Khi giá trị này bằng `true`, flag bị tắt ngay lập tức trong environment tương ứng và Evaluation API trả về reason code `KILL_SWITCH`.

Trường `expectedVersion` được sử dụng để kiểm soát cập nhật đồng thời. Backend chỉ chấp nhận mutation khi version hiện tại của cấu hình trùng với version mà client đã đọc. Nếu không trùng khớp, hệ thống trả về `409 Conflict` nhằm ngăn một request cũ ghi đè cấu hình mới hơn.

Sau mỗi lần cập nhật thành công, version của cấu hình được tăng lên:

```
{
  "id": "config-id",
  "flagId": "flag-id",
  "environmentId": "environment-id",
  "status": "ENABLED",
  "servingMode": "TARGETED",
  "killSwitch": false,
```

```
  "version": 6,
  "updatedAt": "2026-07-03T12:15:00.000Z"
}
```

Nếu flag chưa có cấu hình cho environment được chỉ định, hệ thống có thể tạo cấu hình thông qua endpoint:

```
POST /v1/feature-flags/{flagId}/configurations
```

Request body gồm environment và trạng thái ban đầu:

```
{
  "environmentId": "environment-id",
  "status": "DISABLED",
  "servingMode": "TARGETED",
  "killSwitch": false
}
```

Backend phải kiểm tra environment và flag cùng thuộc một project. Mỗi cặp `flagId` và `environmentId` chỉ được tồn tại một cấu hình duy nhất.

Mọi mutation đối với cấu hình đều được ghi audit log và thực hiện trong transaction. Sau khi transaction hoàn tất, hệ thống invalidates cache theo project, environment và flag để các evaluation tiếp theo sử dụng cấu hình mới nhất. Việc phân tách cấu hình theo environment phù hợp với mô hình quản lý Feature Flag phổ biến, trong đó một flag có thể có trạng thái và targeting rule độc lập trên từng môi trường triển khai [7, 2].

2.7.4 API quản lý rule

Rule xác định điều kiện để một Feature Flag được bật đối với từng evaluation context. Mỗi rule thuộc về một flag configuration của một environment và được đánh giá theo thứ tự ưu tiên đã cấu hình. API quản lý rule cho phép tạo, cập nhật, xóa và sắp xếp thứ tự thực thi của các rule, đồng thời thực hiện kiểm tra tính hợp lệ của từng loại rule trước khi lưu vào cơ sở dữ liệu.

Tạo rule

Rule được tạo thông qua endpoint:

```
POST /v1/flag-configurations/{configurationId}/rules
```

Request body gồm loại rule, độ ưu tiên và các tham số tương ứng:

```
{
  "type": "ROLE_TARGETING",
  "priority": 10,
  "parameters": {
    "roles": [
      "beta-customer",
      "admin"
    ]
  }
}
```

Hệ thống hỗ trợ ba loại rule chính:

- `USER_ALLOWLIST`;
- `ROLE_TARGETING`;
- `PERCENTAGE_ROLLOUT`.

Các tham số của từng loại rule được lưu trong trường `parameters` dưới dạng JSONB để cho phép mở rộng mà không phải thay đổi lược đồ cơ sở dữ liệu [15].

Khi nhận request tạo rule, backend thực hiện các bước:

1. Kiểm tra flag configuration tồn tại;
2. Kiểm tra feature flag chưa bị archive;
3. Kiểm tra project đang ở trạng thái hoạt động;
4. Validation dữ liệu theo từng loại rule;
5. Gán version mới cho cấu hình;
6. Ghi audit log.

Response thành công:

```
{
  "id": "rule-id",
  "configurationId": "configuration-id",
  "type": "ROLE_TARGETING",
  "priority": 10,
  "parameters": {
    "roles": [
      "beta-customer",
      "admin"
    ]
  },
  "createdAt": "2026-07-03T13:00:00.000Z",
  "updatedAt": "2026-07-03T13:00:00.000Z"
}
```

Rule mới không được phép có độ ưu tiên trùng với rule khác trong cùng một flag configuration. Nếu xảy ra xung đột, backend có thể tự động dịch chuyển các rule phía sau hoặc từ chối request bằng mã lỗi `409 Conflict`. Trong mini project, hệ thống tự động điều chỉnh thứ tự để giảm thao tác thủ công của người quản trị.

Cập nhật rule

Rule được cập nhật bằng endpoint:

```
PATCH /v1/rules/{ruleId}
```

Request chỉ bao gồm các trường cần thay đổi:

```
{
  "parameters": {
    "roles": [
      "beta-customer",
      "premium-customer"
    ]
  }
```

```
}
```

Hoặc:

```
{
  "parameters": {
    "percentage": 25
  }
}
```

Backend không cho phép thay đổi loại của một rule đã tồn tại. Nếu muốn chuyển từ `ROLE_TARGETING` sang `PERCENTAGE_ROLLOUT`, người quản trị phải tạo rule mới và xóa rule cũ. Quy tắc này giúp đơn giản hóa quá trình validation và bảo toàn lịch sử thay đổi.

Sau khi cập nhật thành công, backend:

1. Kiểm tra dữ liệu mới;
2. Tăng version của flag configuration;
3. Ghi audit log;
4. Invalidate cache evaluation.

Response trả về dữ liệu rule mới nhất:

```
{
  "id": "rule-id",
  "type": "ROLE_TARGETING",
  "priority": 10,
  "parameters": {
    "roles": [
      "beta-customer",
      "premium-customer"
    ]
  },
  "updatedAt": "2026-07-03T13:15:00.000Z"
}
```

Việc cập nhật rule không thay đổi định danh của rule và không ảnh hưởng đến các rule khác nếu độ ưu tiên vẫn được giữ nguyên.

Xóa rule

Rule được xóa thông qua endpoint:

```
DELETE /v1/rules/{ruleId}
```

Sau khi xác nhận rule tồn tại, backend thực hiện:

1. Xóa rule khỏi cơ sở dữ liệu;
2. Sắp xếp lại thứ tự ưu tiên của các rule còn lại;
3. Tăng version của flag configuration;
4. Ghi audit log;
5. Invalidate cache evaluation.

Response thành công sử dụng mã trạng thái:

```
204 No Content
```

Nếu rule không tồn tại, hệ thống trả về:

```
404 Not Found
```

Việc xóa rule chỉ ảnh hưởng đến một environment cụ thể vì mỗi environment sở hữu tập rule độc lập.

Thay đổi độ ưu tiên

Rule Evaluation Engine đánh giá các rule theo thứ tự ưu tiên từ nhỏ đến lớn. Rule đầu tiên thỏa mãn điều kiện sẽ quyết định kết quả evaluation và các rule phía sau không còn được kiểm tra.

Độ ưu tiên được cập nhật thông qua endpoint:

```
PATCH /v1/rules/{ruleId}/priority
```

Ví dụ:

```
{
  "priority": 5
}
```

Hoặc endpoint hỗ trợ thay đổi trực tiếp vị trí của rule:

```
POST /v1/rules/reorder
```

Ví dụ request:

```
{
  "configurationId": "configuration-id",
  "orderedRuleIds": [
    "rule-3",
    "rule-1",
    "rule-2"
  ]
}
```

Backend cập nhật toàn bộ thứ tự trong cùng một transaction để tránh trạng thái trung gian.

Sau khi cập nhật, các rule có thứ tự như sau:

Rule	Priority
Rule 3	1
Rule 1	2
Rule 2	3

Việc thay đổi thứ tự có ảnh hưởng trực tiếp đến kết quả evaluation. Ví dụ, một rule allowlist đặt trước percentage rollout sẽ luôn được áp dụng trước khi hệ thống thực hiện phân phối theo phần trăm. Do đó dashboard cần hiển thị rõ thứ tự thực thi để người quản trị tránh cấu hình sai [4].

Validation tham số rule

Mỗi loại rule có tập tham số và quy tắc validation riêng.

Đối với `USER_ALLOWLIST`, request phải chứa danh sách người dùng:

```
{
  "targetingKeys": [
    "user-001",
    "user-002"
  ]
}
```

Danh sách không được rỗng và không được chứa giá trị trùng lặp.

Đối với `ROLE_TARGETING`, request phải chứa ít nhất một role hợp lệ:

```
{
  "roles": [
    "admin",
    "beta-customer"
  ]
}
```

Các role không được để trống và mỗi role chỉ xuất hiện một lần.

Đối với `PERCENTAGE_ROLLOUT`, request có dạng:

```
{
  "percentage": 25
}
```

Giá trị `percentage` phải nằm trong khoảng từ 0 đến 100.

Ngoài validation theo từng loại rule, backend còn kiểm tra:

- kiểu dữ liệu của từng trường;
- số lượng phần tử tối đa trong danh sách;
- cấu trúc JSON hợp lệ;
- không tồn tại thuộc tính không được định nghĩa trong DTO.

Nếu request không hợp lệ, hệ thống trả về:

```
400 Bad Request
```

Ví dụ response:

```
{
  "code": "VALIDATION_ERROR",
  "message": "Invalid rule parameters.",
  "details": [
    {
      "field": "percentage",
      "message": "Value must be between 0 and 100."
    }
  ],
  "requestId": "request-id"
}
```

Validation được thực hiện trước khi ghi dữ liệu xuống cơ sở dữ liệu nhằm ngăn các cấu hình không hợp lệ tham gia vào quá trình evaluation. Cách tiếp cận này giúp Rule Evaluation Engine luôn làm việc trên tập cấu hình hợp lệ, giảm khả năng phát sinh lỗi trong thời gian chạy và bảo đảm kết quả đánh giá nhất quán.

2.7.5 Evaluation API

Evaluation API là thành phần quan trọng nhất của nền tảng Feature Flag. API này được các ứng dụng khách gọi tại thời điểm chạy để xác định một feature có được bật hay không đối với một evaluation context cụ thể. Thiết kế API ưu tiên tính xác định, độ trễ thấp, khả năng mở rộng và hợp đồng dữ liệu ổn định nhằm đáp ứng yêu cầu tích hợp giữa nhiều ứng dụng khác nhau [10, 6].

Evaluation request contract

Evaluation được thực hiện thông qua endpoint:

```
POST /v1/evaluate
```

Request body bao gồm khóa project, khóa feature flag và evaluation context:

```
{
  "projectKey": "ecommerce-demo",
```

```
  "environmentKey": "production",
  "flagKey": "new-checkout",
  "context": {
    "targetingKey": "user-001",
    "userId": "1001",
    "roles": [
      "beta-customer"
    ],
    "attributes": {
      "country": "VN",
      "membership": "gold"
    }
  }
}
```

Các trường có ý nghĩa như sau:

- `projectKey`: xác định project chứa Feature Flag.
- `environmentKey`: xác định environment cần đánh giá.
- `flagKey`: khóa nghiệp vụ của Feature Flag.
- `context`: tập thông tin mô tả người dùng hoặc request hiện tại.

Trong evaluation context:

- `targetingKey` là khóa định danh ổn định được sử dụng cho percentage rollout.
- `userId` phục vụ các rule theo người dùng.
- `roles` phục vụ role targeting.
- `attributes` là tập thuộc tính mở rộng để hỗ trợ các rule trong tương lai.

Trong ba loại rule được hỗ trợ của mini project:

- `USER_ALLOWLIST` sử dụng `userId`;
- `ROLE_TARGETING` sử dụng `roles`;

- `PERCENTAGE_ROLLOUT` sử dụng `targetingKey`.

Evaluation request không chứa thông tin cấu hình của Feature Flag. Toàn bộ cấu hình được truy xuất từ cơ sở dữ liệu dựa trên project, environment và flag tương ứng. Cách tách biệt này giúp ứng dụng khách chỉ cung cấp evaluation context, trong khi toàn bộ logic quyết định được quản lý tập trung trên nền tảng Feature Flag.

Backend thực hiện validation request trước khi đánh giá:

- các trường bắt buộc không được để trống;
- project và flag key phải đúng định dạng;
- context phải là đối tượng JSON hợp lệ;
- `roles` phải là mảng chuỗi;
- `attributes` phải là object JSON.

Sau khi validation thành công, Evaluation Engine tải snapshot cấu hình tương ứng và thực hiện đánh giá theo thứ tự ưu tiên đã thiết kế.

Evaluation response contract

Response của Evaluation API trả về toàn bộ thông tin cần thiết để ứng dụng khách quyết định hành vi của tính năng.

Ví dụ:

```
{
  "projectKey": "ecommerce-demo",
  "flagKey": "new-checkout",
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-id"
}
```

Các trường của response gồm:

- `projectKey`: project đã được đánh giá.

- `flagKey`: feature flag tương ứng.
- `enabled`: kết quả cuối cùng của evaluation.
- `variant`: variant được chọn.
- `reason`: nguyên nhân tạo ra kết quả.
- `matchedRuleId`: rule đầu tiên khớp với evaluation context.

Trong phạm vi mini project, hệ thống sử dụng hai variant mặc định:

- `on`;
- `off`.

Thiết kế response vẫn giữ trường `variant` để hỗ trợ mở rộng sang nhiều biến thể trong tương lai mà không cần thay đổi hợp đồng API.

Evaluation API luôn trả về một kết quả xác định đối với cùng một snapshot cấu hình và cùng một evaluation context. Điều này đặc biệt quan trọng đối với percentage rollout nhằm bảo đảm người dùng luôn nhận cùng một kết quả qua nhiều lần evaluation.

Xử lý environment mặc định

Trường `environmentKey` là tùy chọn trong request.

Nếu client truyền environment:

```
{
  "environmentKey": "production"
}
```

Evaluation Engine sử dụng đúng cấu hình của environment đó.

Nếu client không truyền:

```
{
  "projectKey": "ecommerce-demo",
  "flagKey": "new-checkout",
  "context": {
    "targetingKey": "user-001"
```

```
  }
}
```

Backend tự động tìm environment mặc định của project và sử dụng cấu hình tương ứng.

Quá trình xử lý gồm:

1. tìm project theo project key;
2. kiểm tra request có environment hay không;
3. nếu không có, truy vấn environment mặc định;
4. tải flag configuration của environment đó;
5. thực hiện evaluation.

Nếu project chưa cấu hình environment mặc định hoặc environment mặc định không tồn tại, Evaluation API trả về kết quả an toàn với trạng thái tắt thay vì cố gắng lựa chọn ngẫu nhiên một environment khác.

Thiết kế này giúp các ứng dụng chỉ sử dụng một môi trường triển khai có thể gọi API đơn giản hơn, trong khi các hệ thống nhiều môi trường vẫn có thể chỉ định rõ environment mong muốn.

Reason code và matched rule

Ngoài kết quả bật hoặc tắt, Evaluation API trả về reason code để giải thích vì sao quyết định đó được đưa ra.

Một số reason code chính gồm:

Reason	Ý nghĩa
GLOBAL_ON	Bật toàn cục
USER_MATCH	Khớp allowlist người dùng
ROLE_MATCH	Khớp rule theo role
PERCENTAGE_MATCH	Khớp rollout phần trăm
DEFAULT_OFF	Không có rule phù hợp
FLAG_DISABLED	Flag bị disable
FLAG_ARCHIVED	Flag đã archive
GROUP_KILL_SWITCH	Group kill switch kích hoạt
KILL_SWITCH	Kill switch của flag kích hoạt
ERROR	Lỗi nội bộ hệ thống

Nếu evaluation được quyết định bởi một rule cụ thể, response trả về định danh của rule:

```
{
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-id"
}
```

Nếu evaluation kết thúc trước khi xét rule, ví dụ:

- flag bị disable;
- kill switch được bật;
- flag đã archive;

thì trường:

```
"matchedRuleId": null
```

Việc trả về reason code giúp dashboard, SDK và log dễ dàng giải thích nguyên nhân của kết quả evaluation, đồng thời hỗ trợ quá trình kiểm thử và gỡ lỗi.

Xử lý lỗi evaluation

Evaluation API được thiết kế theo nguyên tắc fail-safe. Khi không thể xác định chắc chắn trạng thái của Feature Flag, hệ thống luôn ưu tiên trả về kết quả tắt.

Nếu project hoặc feature flag không tồn tại:

```
{
  "projectKey": "ecommerce-demo",
  "flagKey": "new-checkout",
  "enabled": false,
  "variant": "off",
  "reason": "NOT_FOUND"
}
```

Nếu evaluation context không đủ thông tin cho một loại rule, Evaluation Engine bỏ qua rule đó và tiếp tục đánh giá các rule phía sau. Ví dụ, nếu request không có `roles`, các rule `ROLE_TARGETING` sẽ không khớp.

Khi xảy ra lỗi nội bộ như:

- lỗi cơ sở dữ liệu;
- lỗi cache;
- lỗi Evaluation Engine;
- exception chưa được xử lý;

API trả về:

```
{
  "projectKey": "ecommerce-demo",
  "flagKey": "new-checkout",
  "enabled": false,
  "variant": "off",
  "reason": "ERROR"
}
```

Đối với các lỗi do request không hợp lệ, API sử dụng HTTP status code phù hợp.

Ví dụ:

```
400 Bad Request
```

Response:

```
{
  "code": "VALIDATION_ERROR",
  "message": "Invalid evaluation request.",
  "details": [
    {
      "field": "projectKey",
      "message": "Project key is required."
    }
  ],
  "requestId": "request-id"
}
```

Thiết kế này giúp phân biệt rõ hai nhóm lỗi:

- lỗi request của client sử dụng HTTP error;
- lỗi trong quá trình đánh giá sử dụng response evaluation với trạng thái an toàn.

Cách xử lý trên giúp ứng dụng khách luôn nhận được kết quả có thể dự đoán được, đồng thời hạn chế nguy cơ bật nhầm tính năng khi xảy ra sự cố hệ thống [10, 11].

2.7.6 API audit log

Audit Log API cung cấp khả năng truy vết toàn bộ các thay đổi cấu hình trong hệ thống Feature Flag. Mỗi thao tác tạo, cập nhật, archive, restore hoặc xóa dữ liệu đều sinh ra một bản ghi audit độc lập. Các bản ghi này có tính chất append-only, không được phép chỉnh sửa hoặc xóa nhằm bảo đảm khả năng kiểm toán và truy xuất lịch sử cấu hình [12].

Mỗi bản ghi audit lưu các thông tin chính gồm:

- actor thực hiện thao tác;
- thời điểm thực hiện;

- loại tài nguyên bị thay đổi;
- định danh tài nguyên;
- hành động thực hiện;
- snapshot trước thay đổi;
- snapshot sau thay đổi;
- request ID phục vụ truy vết.

Audit Log API chỉ hỗ trợ truy vấn dữ liệu, không cung cấp endpoint chỉnh sửa hoặc xóa audit log.

Truy vấn audit log

Danh sách audit log được truy vấn thông qua endpoint:

```
GET /v1/audit-logs
```

Endpoint hỗ trợ phân trang:

```
GET /v1/audit-logs?limit=20&offset=0
```

Response:

```
{
  "items": [
    {
      "id": "audit-id",
      "timestamp": "2026-07-03T14:00:00.000Z",
      "actorId": "admin-001",
      "targetType": "FEATURE_FLAG",
      "targetId": "flag-id",
      "action": "FEATURE_FLAG_UPDATED",
      "requestId": "request-id"
    }
  ],
  "pagination": {
```

```
    "limit": 20,
    "offset": 0,
    "total": 235
  }
}
```

Danh sách mặc định được sắp xếp theo thời gian giảm dần để các thay đổi mới nhất xuất hiện trước.

Thông tin chi tiết của một bản ghi được truy vấn bằng:

```
GET /v1/audit-logs/{auditId}
```

Response bao gồm đầy đủ metadata cùng snapshot trước và sau thay đổi.

Nếu audit log không tồn tại, hệ thống trả về:

```
404 Not Found
```

Lọc theo actor và action

Audit log hỗ trợ lọc theo người thực hiện và loại hành động nhằm hỗ trợ kiểm tra lịch sử thao tác của từng quản trị viên.

Ví dụ:

```
GET /v1/audit-logs?actorId=admin-001
```

Hoặc:

```
GET /v1/audit-logs?action=FEATURE_FLAG_UPDATED
```

Hai điều kiện có thể kết hợp:

```
GET /v1/audit-logs?actorId=admin-001
    &action=FLAG_RULE_CREATED
```

Một số action phổ biến gồm:

Action	Ý nghĩa
PROJECT_CREATED	Tạo project
PROJECT_UPDATED	Cập nhật project
FEATURE_FLAG_CREATED	Tạo Feature Flag
FEATURE_FLAG_UPDATED	Cập nhật Feature Flag
FEATURE_FLAG_ARCHIVED	Archive Feature Flag
FEATURE_FLAG_RESTORED	Restore Feature Flag
FLAG_CONFIG_UPDATED	Cập nhật cấu hình
FLAG_RULE_CREATED	Tạo rule
FLAG_RULE_UPDATED	Cập nhật rule
FLAG_RULE_DELETED	Xóa rule

Việc lọc theo actor giúp xác định ai đã thực hiện một thay đổi cụ thể, trong khi lọc theo action hỗ trợ thống kê hoặc kiểm tra các loại thao tác trong hệ thống.

Lọc theo target và thời gian

Audit log cũng hỗ trợ truy vấn theo tài nguyên bị thay đổi.

Ví dụ:

```
GET /v1/audit-logs?targetType=FEATURE_FLAG
```

Hoặc:

```
GET /v1/audit-logs?targetId=flag-id
```

Hai điều kiện có thể được kết hợp:

```
GET /v1/audit-logs?targetType=FEATURE_FLAG
    &targetId=flag-id
```

API còn hỗ trợ lọc theo khoảng thời gian:

```
GET /v1/audit-logs?
    from=2026-07-01T00:00:00Z&
    to=2026-07-03T23:59:59Z
```

Các điều kiện có thể sử dụng đồng thời:

```
GET /v1/audit-logs?
    actorId=admin-001&
    action=FLAG_RULE_UPDATED&
    targetType=FEATURE_FLAG&
    from=2026-07-01T00:00:00Z&
    to=2026-07-03T23:59:59Z
```

Backend chỉ áp dụng các điều kiện được cung cấp. Những tham số không xuất hiện trong request sẽ không tham gia vào truy vấn.

Việc hỗ trợ nhiều điều kiện lọc giúp dashboard nhanh chóng tìm được bản ghi cần thiết ngay cả khi số lượng audit log tăng lên theo thời gian.

Truy vấn snapshot thay đổi

Một bản ghi audit không chỉ lưu metadata mà còn lưu trạng thái của dữ liệu trước và sau khi thay đổi.

Ví dụ response:

```
{
  "id": "audit-id",
  "timestamp": "2026-07-03T14:00:00.000Z",
  "actorId": "admin-001",
  "targetType": "FEATURE_FLAG",
  "targetId": "flag-id",
  "action": "FEATURE_FLAG_UPDATED",
  "before": {
    "name": "New Checkout",
    "description": "Old description"
  },
  "after": {
    "name": "Redesigned Checkout",
    "description": "New description"
  },
```

```
  "requestId": "request-id"
}
```

Snapshot được lưu dưới dạng JSONB để có thể ghi nhận đầy đủ trạng thái của từng loại resource mà không phụ thuộc vào cấu trúc bảng dữ liệu [15].

Đối với thao tác tạo mới, trường `before` bằng `null`.

Ví dụ:

```
{
  "before": null,
  "after": {
    "key": "new-checkout",
    "name": "New Checkout"
  }
}
```

Đối với thao tác xóa hoặc archive, trường `after` phản ánh trạng thái cuối cùng sau mutation, hoặc bằng `null` nếu tài nguyên bị loại bỏ khỏi cơ sở dữ liệu.

Audit snapshot cho phép:

- xác định chính xác dữ liệu đã thay đổi;
- so sánh hai phiên bản cấu hình;
- hỗ trợ điều tra sự cố;
- phục vụ kiểm toán và truy vết lịch sử.

Các snapshot được ghi cùng transaction với mutation tương ứng. Nếu transaction thất bại, cả thay đổi cấu hình và audit log đều được rollback nhằm bảo đảm dữ liệu và lịch sử thay đổi luôn nhất quán. Theo nguyên tắc bảo mật, audit log chỉ lưu những thông tin cần thiết để truy vết và tránh ghi nhận dữ liệu nhạy cảm không cần thiết [12].

2.7.7 API thống kê evaluation

Evaluation Statistics API cung cấp khả năng thống kê số lần Feature Flag được đánh giá trong quá trình hoạt động của hệ thống. Đây là chức năng thuộc mức khuyến khích của mini project,

giúp người quản trị quan sát mức độ sử dụng của từng Feature Flag, đánh giá hiệu quả rollout và hỗ trợ phân tích sau triển khai.

Trong phạm vi hệ thống, việc thống kê được thực hiện theo nguyên tắc best-effort. Quá trình ghi nhận thống kê không ảnh hưởng đến kết quả của Evaluation API. Nếu việc ghi thống kê thất bại, Evaluation Engine vẫn trả về kết quả evaluation bình thường nhằm bảo đảm độ sẵn sàng của hệ thống.

Ghi nhận evaluation event

Sau mỗi lần Evaluation API hoàn thành, backend tạo một evaluation event phục vụ thống kê.

Thông tin được ghi nhận gồm:

- project;
- environment;
- feature flag;
- thời điểm evaluation;
- trạng thái bật hoặc tắt;
- reason code;
- matched rule (nếu có).

Ví dụ một evaluation event:

```
{
  "projectId": "project-id",
  "environmentId": "environment-id",
  "flagId": "flag-id",
  "enabled": true,
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-id",
  "evaluatedAt": "2026-07-03T15:00:00.000Z"
}
```

Evaluation event chỉ phục vụ mục đích thống kê nên không lưu toàn bộ evaluation context nhằm hạn chế lưu trữ dữ liệu không cần thiết và giảm nguy cơ ghi nhận thông tin nhạy cảm [12].

Việc ghi nhận thống kê được thực hiện sau khi Evaluation Engine đã trả kết quả. Nếu quá trình ghi thống kê gặp lỗi, backend chỉ ghi log nội bộ và không thay đổi response của Evaluation API.

Thống kê theo Feature Flag

Dashboard có thể truy vấn số lần evaluation của một Feature Flag thông qua endpoint:

```
GET /v1/statistics/feature-flags/{flagId}
```

Ví dụ response:

```
{
  "flagId": "flag-id",
  "totalEvaluations": 15842,
  "enabledCount": 12430,
  "disabledCount": 3412,
  "lastEvaluatedAt": "2026-07-03T15:05:18.000Z"
}
```

Ngoài số lần evaluation, hệ thống có thể thống kê theo reason code:

```
{
  "reasonStatistics": {
    "GLOBAL_ON": 420,
    "ROLE_MATCH": 6812,
    "USER_MATCH": 315,
    "PERCENTAGE_MATCH": 4883,
    "DEFAULT_OFF": 3412
  }
}
```

Các thống kê này giúp người quản trị đánh giá hiệu quả của từng chiến lược rollout và phát hiện những Feature Flag hầu như không còn được sử dụng.

Thống kê theo environment

Evaluation statistics cũng được tổng hợp theo từng environment.

Endpoint:

```
GET /v1/statistics/environments/{environmentId}
```

Ví dụ response:

```
{
  "environmentId": "environment-id",
  "totalEvaluations": 86452,
  "featureFlags": 18,
  "enabledCount": 65420,
  "disabledCount": 21032
}
```

Trong trường hợp cần phân tích một Feature Flag trên một environment cụ thể, client có thể sử dụng:

```
GET /v1/statistics/feature-flags/{flagId}
    ?environmentId=environment-id
```

Thống kê theo environment giúp người quản trị so sánh mức độ sử dụng giữa development, staging và production, đồng thời phát hiện những cấu hình ít được truy cập hoặc không còn cần thiết.

Tổng hợp theo khoảng thời gian

API hỗ trợ tổng hợp thống kê theo khoảng thời gian nhằm phục vụ dashboard và biểu đồ.

Ví dụ:

```
GET /v1/statistics/feature-flags/{flagId}
    ?from=2026-07-01T00:00:00Z
    &to=2026-07-03T23:59:59Z
```

Response:

```
{
  "flagId": "flag-id",
  "from": "2026-07-01T00:00:00Z",
  "to": "2026-07-03T23:59:59Z",
  "totalEvaluations": 8421,
  "enabledCount": 6210,
  "disabledCount": 2211
}
```


Hệ thống cũng có thể tổng hợp dữ liệu theo ngày để hiển thị biểu đồ xu hướng:


```
{
  "dailyStatistics": [
    {
      "date": "2026-07-01",
      "evaluations": 2531
    },
    {
      "date": "2026-07-02",
      "evaluations": 2768
    },
    {
      "date": "2026-07-03",
      "evaluations": 3122
    }
  ]
}
```


Các thống kê được xây dựng từ dữ liệu evaluation event đã ghi nhận trước đó. Đối với mini project, việc tổng hợp có thể được thực hiện trực tiếp từ cơ sở dữ liệu. Trong các hệ thống quy mô lớn, thống kê thường được tính toán định kỳ hoặc sử dụng dữ liệu đã tổng hợp trước để giảm tải cho Evaluation API và cơ sở dữ liệu chính.

Evaluation statistics chỉ mang tính chất tham khảo và không tham gia vào quá trình quyết định

kết quả evaluation. Việc tách biệt thống kê khỏi Evaluation Engine giúp duy trì độ trễ thấp, giảm phụ thuộc giữa các thành phần và bảo đảm rằng lỗi thống kê không ảnh hưởng đến hoạt động của Feature Flag.

2.7.8 Validation dữ liệu đầu vào

Validation dữ liệu đầu vào là tuyến phòng thủ đầu tiên của hệ thống nhằm ngăn chặn các request không hợp lệ trước khi truy cập cơ sở dữ liệu hoặc thực hiện các thao tác nghiệp vụ. Toàn bộ request đều được kiểm tra ở tầng API thông qua DTO và các ràng buộc validation. Chỉ những request hợp lệ mới được chuyển tới service layer để xử lý tiếp theo [11].

Nếu validation thất bại, hệ thống trả về mã trạng thái:

```
400 Bad Request
```

Response có cấu trúc thống nhất:

```
{
  "code": "VALIDATION_ERROR",
  "message": "Validation failed.",
  "details": [
    {
      "field": "projectKey",
      "message": "Project key is required."
    }
  ],
  "requestId": "request-id"
}
```

Validation key

Các khóa nghiệp vụ như `projectKey`, `environmentKey` và `flagKey` được sử dụng trực tiếp trong Evaluation API và phải bảo đảm tính ổn định trong toàn bộ vòng đời của hệ thống.

Các khóa này phải thỏa mãn biểu thức chính quy:

```
^[a-z0-9][a-z0-9-]{2,63}$
```

Quy tắc trên bảo đảm:

- chỉ sử dụng chữ thường;
- chỉ cho phép chữ số và dấu gạch ngang;
- không bắt đầu bằng dấu gạch ngang;
- độ dài từ 3 đến 64 ký tự.

Ví dụ hợp lệ:

```
ecommerce-demo
checkout-v2
production
```

Ví dụ không hợp lệ:

```
Checkout
new_checkout
-demo
ab
```

Ngoài định dạng, backend còn kiểm tra tính duy nhất của project key trong toàn hệ thống, đồng thời kiểm tra environment key và flag key là duy nhất trong phạm vi project tương ứng.

Validation enum và trạng thái

Các trường biểu diễn trạng thái hoặc kiểu dữ liệu nghiệp vụ chỉ được phép nhận các giá trị thuộc tập enum đã định nghĩa.

Ví dụ:

```
status:
  ENABLED
  DISABLED

servingMode:
  GLOBAL_ON
  TARGETED
```

```
lifecycleStatus:
  ACTIVE
  ARCHIVED

ruleType:
  USER_ALLOWLIST
  ROLE_TARGETING
  PERCENTAGE_ROLLOUT
```

Nếu client gửi giá trị ngoài tập enum:

```
{
  "status": "ACTIVE"
}
```

backend trả về lỗi validation thay vì tự động ánh xạ sang giá trị gần đúng.

Ngoài kiểm tra enum, hệ thống còn kiểm tra tính hợp lệ của trạng thái nghiệp vụ. Ví dụ, không cho phép cập nhật cấu hình của một Feature Flag đã archive hoặc thêm rule vào một project không còn hoạt động.

Validation percentage rollout

Đối với rule `PERCENTAGE_ROLLOUT`, trường:

```
percentage
```

phải là số thực hoặc số nguyên nằm trong khoảng:

```
0 <= percentage <= 100
```

Ví dụ hợp lệ:

```
{
  "percentage": 0
}

{
  "percentage": 25
}
```

```
{
  "percentage": 100
}
```

Ví dụ không hợp lệ:

```
{
  "percentage": -5
}

{
  "percentage": 135
}

{
  "percentage": "fifty"
}
```

Backend cũng kiểm tra kiểu dữ liệu của trường này trước khi lưu xuống cơ sở dữ liệu. Giá trị rollout được sử dụng trực tiếp trong Rule Evaluation Engine, do đó mọi giá trị ngoài phạm vi cho phép đều bị từ chối ngay từ tầng API.

Validation evaluation context

Evaluation context là dữ liệu do ứng dụng khách gửi tới Evaluation API.

Ví dụ:

```
{
  "context": {
    "targetingKey": "user-001",
    "userId": "1001",
    "roles": [
      "beta-customer"
    ],
    "attributes": {
      "country": "VN"
```

```
    }
  }
}
```

Backend kiểm tra:

- `context` phải là object JSON;
- `targetingKey` là chuỗi nếu được cung cấp;
- `userId` là chuỗi;
- `roles` là mảng chuỗi;
- mỗi role không được rỗng;
- `attributes` là object JSON hợp lệ.

Các trường trong evaluation context đều là tùy chọn. Nếu một trường không được cung cấp, Rule Evaluation Engine chỉ bỏ qua các rule cần trường đó và tiếp tục đánh giá các rule còn lại. Ví dụ, nếu request không có `roles`, các rule `ROLE_TARGETING` sẽ không khớp nhưng percentage rollout vẫn có thể được thực hiện nếu tồn tại `targetingKey`.

Việc kiểm tra kiểu dữ liệu ngay từ tầng API giúp giảm khả năng phát sinh lỗi trong quá trình evaluation và bảo đảm Evaluation Engine luôn làm việc với cấu trúc dữ liệu xác định [6].

Validation tham số phân trang

Các endpoint trả về danh sách resource hỗ trợ hai tham số phân trang:

```
limit
offset
```

Ví dụ:

```
GET /v1/projects?limit=20&offset=40
```

Backend kiểm tra:

- `limit` là số nguyên;
- `offset` là số nguyên;

- `limit` lớn hơn 0;
- `offset` lớn hơn hoặc bằng 0;
- `limit` không vượt quá 100.

Nếu client không truyền tham số, hệ thống sử dụng:

```
limit = 20
offset = 0
```

Ví dụ không hợp lệ:

```
GET /v1/projects?limit=-1

GET /v1/projects?offset=-5

GET /v1/projects?limit=1000
```

Trong các trường hợp trên, API trả về lỗi validation thay vì tự động điều chỉnh giá trị. Việc kiểm tra giới hạn phân trang giúp ngăn các request tải quá nhiều dữ liệu, giảm tải cho cơ sở dữ liệu và duy trì hiệu năng ổn định của các endpoint truy vấn.

2.7.9 Mô hình phản hồi lỗi

Hệ thống sử dụng một mô hình phản hồi lỗi thống nhất cho toàn bộ REST API nhằm giúp ứng dụng khách xử lý lỗi một cách nhất quán. Mỗi phản hồi lỗi đều bao gồm mã lỗi nghiệp vụ, thông báo mô tả, danh sách chi tiết (nếu có) và request ID để hỗ trợ truy vết. Việc chuẩn hóa cấu trúc phản hồi giúp giảm sự phụ thuộc vào từng endpoint cụ thể và đơn giản hóa việc tích hợp SDK hoặc frontend dashboard [11].

Cấu trúc lỗi thống nhất

Mọi lỗi đều sử dụng cùng một cấu trúc JSON.

Ví dụ:

```
{
  "code": "VALIDATION_ERROR",
  "message": "Validation failed.",
  "details": [
```

```
    {
      "field": "projectKey",
      "message": "Project key is required."
    }
  ],
  "requestId": "request-id"
}
```

Ý nghĩa các trường:

- `code`: mã lỗi nghiệp vụ.
- `message`: mô tả ngắn gọn của lỗi.
- `details`: danh sách lỗi chi tiết.
- `requestId`: định danh duy nhất của request.

Các mã lỗi nghiệp vụ phổ biến gồm:

Code	Ý nghĩa
VALIDATION_ERROR	Dữ liệu đầu vào không hợp lệ
NOT_FOUND	Không tìm thấy tài nguyên
CONFLICT	Xung đột dữ liệu
INTERNAL_ERROR	Lỗi nội bộ hệ thống

HTTP status code được sử dụng để biểu diễn loại lỗi ở mức giao thức, trong khi trường `code` giúp client phân biệt chi tiết nguyên nhân xử lý.

Validation error

Nếu request không thỏa mãn các ràng buộc validation, API trả về:

```
400 Bad Request
```

Ví dụ:

```
POST /v1/projects
```

Request:

```
{
  "key": "",
  "name": ""
}
```

Response:

```
{
  "code": "VALIDATION_ERROR",
  "message": "Validation failed.",
  "details": [
    {
      "field": "key",
      "message": "Project key is required."
    },
    {
      "field": "name",
      "message": "Project name is required."
    }
  ],
  "requestId": "request-id"
}
```

Trường `details` có thể chứa nhiều lỗi cùng lúc để client có thể hiển thị toàn bộ các trường cần sửa mà không phải gửi lại nhiều request liên tiếp.

Validation được thực hiện trước khi truy cập cơ sở dữ liệu nhằm giảm tải cho hệ thống và ngăn chặn các thao tác nghiệp vụ trên dữ liệu không hợp lệ.

Not found và conflict

Nếu tài nguyên được yêu cầu không tồn tại, hệ thống trả về:

```
404 Not Found
```

Ví dụ:

```
GET /v1/feature-flags/unknown-id
```

Response:

```
{
  "code": "NOT_FOUND",
  "message": "Feature flag not found.",
  "details": [],
  "requestId": "request-id"
}
```

Đối với các trường hợp xung đột dữ liệu hoặc vi phạm ràng buộc duy nhất, hệ thống trả về:

```
409 Conflict
```

Ví dụ khi tạo project với project key đã tồn tại:

```
{
  "code": "CONFLICT",
  "message": "Project key already exists.",
  "details": [],
  "requestId": "request-id"
}
```

Các tình huống thường dẫn đến lỗi `409 Conflict` gồm:

- project key đã tồn tại;
- environment key trùng trong cùng project;
- flag key trùng trong cùng project;
- cập nhật cấu hình với version cũ;
- thao tác không phù hợp với trạng thái hiện tại của tài nguyên.

Việc phân biệt rõ giữa `404` và `409` giúp client có thể đưa ra hành vi xử lý phù hợp đối với từng loại lỗi.

Internal error

Các lỗi ngoài dự kiến trong quá trình xử lý được ánh xạ thành:

```
500 Internal Server Error
```

Ví dụ:

```
{
  "code": "INTERNAL_ERROR",
  "message": "An unexpected error occurred.",
  "details": [],
  "requestId": "request-id"
}
```

Các nguyên nhân có thể gồm:

- lỗi kết nối cơ sở dữ liệu;
- lỗi cache;
- exception chưa được xử lý;
- lỗi trong quá trình thực thi transaction.

Response không trả về stack trace, câu lệnh SQL hoặc các thông tin nội bộ của hệ thống nhằm tránh làm lộ chi tiết triển khai. Thông tin kỹ thuật đầy đủ chỉ được ghi vào log phía máy chủ để phục vụ việc điều tra và khắc phục sự cố [11, 12].

Đối với Evaluation API, lỗi nội bộ không sử dụng phản hồi HTTP 500 nếu request đã hợp lệ. Thay vào đó, Evaluation Engine trả về kết quả an toàn với `enabled = false` và reason code `ERROR` như đã trình bày ở Mục 2.7.5.

Request ID và khả năng truy vết

Mỗi HTTP request được gán một định danh duy nhất gọi là request ID. Giá trị này được tạo ngay khi request đi vào hệ thống và được truyền xuyên suốt toàn bộ quá trình xử lý.

Request ID được:

- trả về trong mọi response lỗi;

- ghi vào audit log;
- ghi vào application log;
- liên kết với các thao tác mutation trong cùng request.

Ví dụ:

```
{
  "code": "NOT_FOUND",
  "message": "Feature flag not found.",
  "details": [],
  "requestId": "7c8f9c5c-5d3b-4d3c-a50b-b4cf8b31ef52"
}
```

Khi người dùng báo cáo lỗi, quản trị viên chỉ cần sử dụng request ID để tìm toàn bộ log liên quan trên máy chủ, bao gồm thông tin request, exception phát sinh và các audit log tương ứng. Cơ chế này giúp rút ngắn thời gian điều tra sự cố và tăng khả năng truy vết trong quá trình vận hành.

Request ID được xem là metadata của request và không mang ý nghĩa nghiệp vụ. Giá trị này không được sử dụng để xác thực người dùng hoặc xác định tài nguyên trong hệ thống, mà chỉ phục vụ mục đích quan sát và chẩn đoán lỗi.

2.7.10 Tài liệu hóa API

Bên cạnh việc thiết kế API rõ ràng, hệ thống cung cấp tài liệu API tự động nhằm hỗ trợ phát triển, kiểm thử và tích hợp với các ứng dụng khách. Tài liệu được sinh trực tiếp từ mã nguồn backend thông qua chuẩn OpenAPI, giúp bảo đảm tính đồng nhất giữa tài liệu và triển khai thực tế.

Trong quá trình phát triển, mọi thay đổi đối với DTO, endpoint hoặc response đều được phản ánh tự động trong tài liệu API, giảm nguy cơ tài liệu bị lỗi thời so với hệ thống đang triển khai.

OpenAPI và Swagger

Hệ thống sử dụng chuẩn OpenAPI để mô tả toàn bộ REST API. Trên nền tảng này, Swagger UI được tích hợp nhằm cung cấp giao diện trực quan để xem tài liệu và thử nghiệm các endpoint.

Sau khi backend khởi động, tài liệu API có thể được truy cập thông qua:

`/api/docs`

Swagger UI hiển thị:

- danh sách endpoint;
- HTTP method;
- request body;
- response schema;
- HTTP status code;
- ví dụ request và response.

Các endpoint được nhóm theo chức năng, ví dụ:

- Projects;
- Environments;
- Feature Flags;
- Flag Configurations;
- Rules;
- Evaluation;
- Audit Logs;
- Statistics.

Việc nhóm endpoint theo tài nguyên giúp người phát triển dễ dàng tìm kiếm và hiểu cấu trúc API.

OpenAPI cũng cho phép sinh SDK hoặc client code tự động cho nhiều ngôn ngữ lập trình, giảm công sức tích hợp giữa nền tảng Feature Flag và các ứng dụng khách.

Mô tả request và response schema

Mỗi endpoint đều định nghĩa rõ request schema và response schema.

Ví dụ đối với Evaluation API:

```
POST /v1/evaluate
```

Request schema:

```
{
  "projectKey": "string",
  "environmentKey": "string",
  "flagKey": "string",
  "context": {
    "targetingKey": "string",
    "userId": "string",
    "roles": [
      "string"
    ],
    "attributes": {
      "...": "..."
    }
  }
}
```

Response schema:

```
{
  "projectKey": "string",
  "flagKey": "string",
  "enabled": "boolean",
  "variant": "string",
  "reason": "string",
  "matchedRuleId": "string | null"
}
```

Các schema đều mô tả:

- kiểu dữ liệu;
- trường bắt buộc;

- trường tùy chọn;
- giá trị enum nếu có;
- mô tả ý nghĩa của từng trường.

Nhờ vậy, frontend, SDK và các ứng dụng tích hợp có thể hiểu chính xác hợp đồng dữ liệu mà không cần tham khảo mã nguồn backend.

Ví dụ request

Tài liệu API cung cấp ví dụ request cho từng endpoint.

Ví dụ tạo Feature Flag:

```
POST /v1/projects/{projectId}/feature-flags
```

Request:

```
{
  "key": "new-checkout",
  "name": "New Checkout",
  "description": "Controls the redesigned checkout."
}
```

Ví dụ Evaluation API:

```
POST /v1/evaluate
```

Request:

```
{
  "projectKey": "ecommerce-demo",
  "environmentKey": "production",
  "flagKey": "new-checkout",
  "context": {
    "targetingKey": "user-001",
    "userId": "1001",
    "roles": [
      "beta-customer"
```

```
    ],
    "attributes": {
      "country": "VN"
    }
  }
}
```

Việc cung cấp request mẫu giúp người phát triển nhanh chóng hiểu cách sử dụng API và giảm sai sót trong quá trình tích hợp.

Ví dụ response và error

Bên cạnh request, tài liệu API cũng cung cấp response mẫu cho cả trường hợp thành công và thất bại.

Ví dụ evaluation thành công:

```
200 OK

{
  "projectKey": "ecommerce-demo",
  "flagKey": "new-checkout",
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule-id"
}
```

Ví dụ lỗi validation:

```
400 Bad Request

{
  "code": "VALIDATION_ERROR",
  "message": "Validation failed.",
  "details": [
    {
```

```
      "field": "projectKey",
      "message": "Project key is required."
    }
  ],
  "requestId": "request-id"
}
```

Ví dụ không tìm thấy tài nguyên:

```
404 Not Found

{
  "code": "NOT_FOUND",
  "message": "Feature flag not found.",
  "details": [],
  "requestId": "request-id"
}
```

Ví dụ lỗi nội bộ:

```
500 Internal Server Error

{
  "code": "INTERNAL_ERROR",
  "message": "An unexpected error occurred.",
  "details": [],
  "requestId": "request-id"
}
```

Việc cung cấp đầy đủ các ví dụ request và response giúp giảm thời gian tích hợp, hỗ trợ kiểm thử API bằng Swagger UI hoặc các công cụ như Postman, đồng thời tạo ra một hợp đồng giao tiếp rõ ràng giữa backend và các ứng dụng khách. Các ví dụ được sinh trực tiếp từ DTO và metadata của OpenAPI nên luôn đồng bộ với phiên bản API đang triển khai.

2.8 Hiệu năng, nhất quán và bảo mật

2.8.1 Caching kết quả evaluation

Trong hệ thống Feature Flag, Evaluation API có thể được gọi với tần suất rất lớn bởi nhiều ứng dụng khách trong quá trình xử lý request. Mỗi lần đánh giá đều cần truy xuất cấu hình feature flag, rule và environment trước khi thực hiện quá trình evaluation. Nếu mọi request đều truy cập trực tiếp cơ sở dữ liệu thì độ trễ sẽ tăng, đồng thời tạo áp lực không cần thiết lên hệ quản trị cơ sở dữ liệu. Vì vậy, caching là một kỹ thuật quan trọng nhằm giảm thời gian phản hồi, giảm số lần truy vấn dữ liệu và tăng khả năng mở rộng của hệ thống. Các nền tảng Feature Flag hiện đại đều khuyến nghị sử dụng cache để tối ưu quá trình evaluation trong khi vẫn đảm bảo tính đúng đắn của kết quả [8, 13].

Mục tiêu của caching

Mục tiêu chính của caching là giảm số lần truy cập cơ sở dữ liệu đối với các dữ liệu ít thay đổi nhưng được đọc rất thường xuyên. Trong hệ thống Feature Flag, cấu hình của một feature flag thường chỉ thay đổi khi người quản trị cập nhật trên dashboard, trong khi Evaluation API có thể được gọi hàng nghìn lần mỗi phút bởi các ứng dụng khách.

Việc sử dụng cache mang lại nhiều lợi ích như giảm độ trễ của Evaluation API, tăng throughput của hệ thống, giảm tải cho PostgreSQL và cải thiện khả năng đáp ứng khi số lượng request tăng cao. Đồng thời, cache cũng góp phần duy trì trải nghiệm ổn định cho ứng dụng khách ngay cả khi cơ sở dữ liệu đang chịu tải lớn hoặc có độ trễ tạm thời [8].

Tuy nhiên, cache chỉ nên được sử dụng đối với dữ liệu có thể tái sử dụng giữa nhiều request. Những dữ liệu phụ thuộc vào evaluation context của từng người dùng cần được cân nhắc kỹ nhằm tránh trả về kết quả không chính xác.

Cache configuration snapshot

Trong hệ thống được đề xuất, dữ liệu phù hợp nhất để cache là configuration snapshot của feature flag. Snapshot bao gồm toàn bộ thông tin cần thiết cho quá trình evaluation như trạng thái của flag, serving mode, kill switch, danh sách rule và thứ tự ưu tiên của các rule.

Khi nhận được request evaluation, hệ thống trước tiên tìm snapshot trong cache theo khóa gồm project, environment và feature flag. Nếu snapshot đã tồn tại, Evaluation Engine có thể thực

hiện đánh giá ngay mà không cần truy vấn cơ sở dữ liệu. Nếu cache không có dữ liệu (cache miss), hệ thống mới truy xuất PostgreSQL, xây dựng snapshot và lưu lại vào cache để phục vụ các request tiếp theo.

Do configuration chỉ thay đổi khi có thao tác quản trị, cache snapshot có tỷ lệ tái sử dụng rất cao. Khi người quản trị cập nhật cấu hình feature flag, hệ thống sẽ xóa hoặc làm mới cache tương ứng để đảm bảo các request evaluation tiếp theo luôn sử dụng cấu hình mới nhất. Cách tiếp cận này giúp giảm đáng kể số lần truy cập cơ sở dữ liệu trong khi vẫn duy trì tính nhất quán của cấu hình [8].

Cache kết quả evaluation

Ngoài configuration snapshot, một số hệ thống còn cache trực tiếp kết quả evaluation. Tuy nhiên, cách tiếp cận này cần được áp dụng thận trọng vì kết quả evaluation phụ thuộc vào evaluation context của từng request, bao gồm targeting key, vai trò người dùng và các thuộc tính khác.

Nếu cache kết quả evaluation, khóa cache phải bao gồm toàn bộ các thông tin ảnh hưởng đến kết quả đánh giá. Khi số lượng người dùng và thuộc tính lớn, số lượng khóa cache sẽ tăng nhanh, làm giảm hiệu quả sử dụng bộ nhớ và tăng chi phí invalidation khi cấu hình feature flag thay đổi.

Trong phạm vi mini project, giải pháp được lựa chọn là ưu tiên cache configuration snapshot thay vì cache kết quả evaluation. Evaluation Engine vẫn thực hiện tính toán trên snapshot đã được cache, do thuật toán evaluation có độ phức tạp thấp và chỉ bao gồm các phép so sánh điều kiện đơn giản. Cách tiếp cận này giúp cân bằng giữa hiệu năng, tính đúng đắn và độ đơn giản của hệ thống.

Lựa chọn in-memory cache và Redis

Có hai phương án phổ biến để triển khai cache là in-memory cache và Redis.

In-memory cache lưu dữ liệu trực tiếp trong bộ nhớ của tiến trình ứng dụng. Phương án này có ưu điểm là tốc độ truy cập rất nhanh, triển khai đơn giản và không yêu cầu thêm hạ tầng. Tuy nhiên, cache sẽ bị mất khi ứng dụng khởi động lại và không thể chia sẻ giữa nhiều instance của backend.

Redis là hệ quản trị dữ liệu trong bộ nhớ hoạt động độc lập với ứng dụng. Redis cho phép nhiều backend cùng sử dụng một cache chung, hỗ trợ thời gian sống (TTL), cơ chế invalidation linh hoạt và khả năng mở rộng tốt hơn. Đổi lại, hệ thống cần triển khai thêm một thành phần riêng và phát sinh chi phí giao tiếp qua mạng [13].

Trong phạm vi mini project, hệ thống ưu tiên sử dụng in-memory cache nhằm giảm độ phức tạp triển khai và đáp ứng đầy đủ yêu cầu về hiệu năng. Kiến trúc của lớp cache được thiết kế theo hướng trừu tượng hóa để có thể thay thế bằng Redis trong tương lai mà không cần thay đổi Evaluation Engine. Điều này phù hợp với định hướng mở rộng của hệ thống và cũng đáp ứng yêu cầu khuyến khích về caching trong đề tài.

2.8.2 Thiết kế cache key và thời gian tồn tại

Để cache hoạt động hiệu quả, mỗi đối tượng được lưu trong cache phải có khóa (cache key) duy nhất và phản ánh đầy đủ các yếu tố ảnh hưởng đến dữ liệu được lưu. Một cache key được thiết kế tốt giúp tăng tỷ lệ cache hit, giảm nguy cơ trả về dữ liệu sai và đơn giản hóa quá trình invalidation khi cấu hình feature flag thay đổi. Bên cạnh đó, thời gian tồn tại (Time To Live - TTL) của cache cũng cần được lựa chọn phù hợp nhằm cân bằng giữa hiệu năng và tính nhất quán của dữ liệu [13, 8].

Thành phần của cache key

Trong hệ thống được đề xuất, cache key được xây dựng từ các thành phần định danh duy nhất của configuration snapshot. Đối với mỗi feature flag, khóa cache bao gồm project key, environment key và feature flag key theo định dạng:

```
projectKey:environmentKey:flagKey
```

Cách tổ chức này đảm bảo mỗi cấu hình của feature flag trong từng environment được lưu thành một bản ghi độc lập. Khi Evaluation API nhận request, hệ thống có thể xác định chính xác snapshot cần sử dụng chỉ thông qua các khóa định danh mà không phải truy vấn cơ sở dữ liệu.

Việc sử dụng các khóa có ý nghĩa rõ ràng cũng giúp đơn giản hóa quá trình ghi log, giám sát cache và xóa cache khi cấu hình thay đổi.

Cache key theo environment

Một feature flag có thể được cấu hình khác nhau giữa các environment như Development, Staging và Production. Ví dụ, cùng một feature flag có thể được bật hoàn toàn trong môi trường Development nhưng chỉ rollout cho một tỷ lệ nhỏ người dùng trên Production.

Do đó, environment phải là một thành phần bắt buộc trong cache key. Nếu chỉ sử dụng project key và feature flag key, snapshot của các environment khác nhau có thể ghi đè lên nhau, dẫn đến Evaluation Engine sử dụng sai cấu hình.

Việc tách biệt cache theo environment cũng phù hợp với mô hình quản lý feature flag của các nền tảng hiện đại, trong đó mỗi environment được xem là một không gian cấu hình độc lập [7].

Cache key theo evaluation context

Đối với configuration snapshot, cache key không cần bao gồm evaluation context vì snapshot chỉ chứa cấu hình của feature flag và không phụ thuộc vào người dùng cụ thể. Sau khi snapshot được lấy từ cache, Evaluation Engine sẽ áp dụng các rule dựa trên evaluation context của request để tính toán kết quả cuối cùng.

Ngược lại, nếu lựa chọn cache trực tiếp kết quả evaluation thì evaluation context phải trở thành một phần của cache key. Khi đó, khóa cache cần phản ánh toàn bộ các thuộc tính có ảnh hưởng đến kết quả đánh giá, chẳng hạn như targeting key, vai trò người dùng hoặc các thuộc tính tùy chỉnh.

Tuy nhiên, số lượng tổ hợp evaluation context trong thực tế rất lớn, khiến cache key tăng nhanh và làm giảm hiệu quả của cache. Vì lý do đó, hệ thống trong mini project không cache kết quả evaluation mà chỉ cache configuration snapshot, giúp cache có tỷ lệ tái sử dụng cao hơn và đơn giản hóa cơ chế invalidation.

Lựa chọn thời gian tồn tại

Thời gian tồn tại của cache cần được lựa chọn sao cho vừa tận dụng được lợi ích của cache vừa hạn chế nguy cơ sử dụng dữ liệu cũ. Hai phương pháp phổ biến là sử dụng TTL cố định hoặc chủ động invalidation khi dữ liệu thay đổi.

Trong hệ thống được đề xuất, configuration snapshot được ưu tiên làm mới ngay sau mỗi lần người quản trị thay đổi cấu hình feature flag. Khi xảy ra thao tác tạo, cập nhật, bật, tắt hoặc

thay đổi rule, cache tương ứng sẽ bị xóa hoặc ghi đè để các request evaluation tiếp theo sử dụng cấu hình mới nhất.

Bên cạnh cơ chế invalidation chủ động, cache vẫn có thể được cấu hình TTL để tự động loại bỏ các bản ghi ít được sử dụng hoặc tồn tại quá lâu. TTL không đóng vai trò đảm bảo tính nhất quán mà chủ yếu hỗ trợ quản lý bộ nhớ và phục hồi trong các trường hợp invalidation không được thực hiện do lỗi hệ thống. Cách kết hợp giữa invalidation theo sự kiện và TTL giúp duy trì hiệu năng cao trong khi vẫn đảm bảo Evaluation Engine sử dụng cấu hình gần với trạng thái hiện tại của hệ thống [13, 8].

2.8.3 Cache invalidation

Một trong những thách thức lớn nhất của caching là đảm bảo dữ liệu trong cache luôn phản ánh đúng trạng thái hiện tại của hệ thống. Nếu cache không được cập nhật hoặc xóa kịp thời sau khi cấu hình feature flag thay đổi, Evaluation Engine có thể sử dụng dữ liệu cũ và trả về kết quả không chính xác. Vì vậy, cache invalidation cần được thực hiện ngay khi xảy ra các thay đổi ảnh hưởng đến quá trình evaluation, thay vì chỉ phụ thuộc vào thời gian sống (TTL) của cache [8, 13].

Invalidation khi cập nhật flag

Các thuộc tính của feature flag như trạng thái lifecycle, trạng thái bật hoặc tắt, serving mode hoặc kill switch đều ảnh hưởng trực tiếp đến kết quả evaluation. Khi một trong các thuộc tính này được cập nhật, configuration snapshot tương ứng không còn hợp lệ.

Trong hệ thống được đề xuất, sau khi thao tác cập nhật feature flag được thực hiện thành công và dữ liệu đã được lưu vào cơ sở dữ liệu, backend sẽ xóa cache của feature flag tương ứng. Request evaluation tiếp theo sẽ truy xuất dữ liệu mới từ cơ sở dữ liệu, xây dựng lại snapshot và lưu trở lại cache. Cơ chế này giúp tránh việc Evaluation Engine tiếp tục sử dụng cấu hình cũ sau khi người quản trị đã thay đổi trạng thái của feature flag.

Invalidation khi cập nhật rule

Danh sách rule và thứ tự ưu tiên của các rule là thành phần quan trọng của configuration snapshot. Việc thêm, sửa, xóa hoặc thay đổi priority của bất kỳ rule nào đều có thể làm thay đổi kết quả evaluation đối với một hoặc nhiều người dùng.

Do đó, mọi thao tác cập nhật rule đều phải thực hiện invalidation đối với cache snapshot của feature flag tương ứng. Sau khi cache bị xóa, snapshot mới sẽ được tạo lại từ dữ liệu mới nhất trong cơ sở dữ liệu khi có request evaluation tiếp theo. Việc invalidation theo feature flag thay vì toàn bộ hệ thống giúp giới hạn phạm vi ảnh hưởng và duy trì hiệu năng của cache.

Invalidation khi thay đổi group kill switch

Trong hệ thống hỗ trợ group kill switch, một nhóm feature flag có thể được bật hoặc tắt đồng thời thông qua một cấu hình chung. Khi trạng thái group kill switch thay đổi, kết quả evaluation của tất cả các feature flag thuộc nhóm đều có khả năng thay đổi mặc dù bản thân từng feature flag không được chỉnh sửa.

Vì vậy, hệ thống cần thực hiện invalidation đối với toàn bộ configuration snapshot của các feature flag thuộc group bị ảnh hưởng. Thay vì xóa toàn bộ cache của hệ thống, backend chỉ cần xác định danh sách feature flag trong group và xóa các cache key tương ứng. Cách tiếp cận này giúp giảm phạm vi invalidation và giữ nguyên cache của các feature flag không liên quan.

Invalidation theo environment

Feature flag thường được cấu hình độc lập theo từng environment như Development, Staging và Production. Một thay đổi trong môi trường Production không làm thay đổi cấu hình của cùng feature flag trên Staging hoặc Development.

Do cache key đã bao gồm environment, quá trình invalidation cũng được thực hiện theo từng environment riêng biệt. Khi cấu hình của một feature flag trong một environment được cập nhật, hệ thống chỉ xóa cache snapshot tương ứng với environment đó mà không ảnh hưởng đến cache của các environment khác.

Cơ chế invalidation theo environment giúp hạn chế số lượng cache bị xóa sau mỗi lần cập nhật, tăng tỷ lệ cache hit đối với các môi trường không thay đổi và phù hợp với mô hình quản lý cấu hình độc lập của các nền tảng Feature Flag hiện đại [7, 8].

2.8.4 Tính nhất quán của cấu hình

Trong hệ thống Feature Flag, tính nhất quán của cấu hình quyết định liệu Evaluation Engine có luôn sử dụng phiên bản cấu hình mới nhất hay không. Khi áp dụng caching nhằm cải thiện hiệu năng, hệ thống phải chấp nhận sự đánh đổi nhất định giữa tốc độ truy cập và độ mới của

dữ liệu. Do đó, việc lựa chọn mô hình nhất quán phù hợp là một yếu tố quan trọng trong thiết kế kiến trúc của nền tảng Feature Flag [8].

Strong consistency và eventual consistency

Strong consistency đảm bảo rằng ngay sau khi cấu hình feature flag được cập nhật thành công, mọi request evaluation tiếp theo đều quan sát được phiên bản mới nhất của cấu hình. Mô hình này mang lại kết quả chính xác tuyệt đối nhưng thường yêu cầu đồng bộ dữ liệu chặt chẽ, làm tăng độ trễ và giảm khả năng mở rộng của hệ thống.

Ngược lại, eventual consistency cho phép tồn tại một khoảng thời gian ngắn mà các request evaluation vẫn có thể sử dụng cấu hình cũ. Sau khi quá trình đồng bộ hoặc invalidation hoàn tất, toàn bộ hệ thống sẽ hội tụ về cùng một trạng thái cấu hình. Đây là mô hình phổ biến trong các nền tảng Feature Flag hiện đại do đáp ứng tốt yêu cầu hiệu năng và khả năng mở rộng [8].

Trong phạm vi mini project, hệ thống ưu tiên cập nhật cache ngay sau khi cấu hình thay đổi nhằm giảm tối đa thời gian tồn tại của dữ liệu cũ. Tuy nhiên, trong trường hợp triển khai nhiều backend hoặc sử dụng cache phân tán, hệ thống vẫn chấp nhận mô hình eventual consistency trong khoảng thời gian rất ngắn.

Stale configuration

Stale configuration là trạng thái Evaluation Engine sử dụng một phiên bản cấu hình đã lỗi thời so với dữ liệu hiện có trong cơ sở dữ liệu. Hiện tượng này có thể xảy ra khi cache chưa được invalidation, quá trình đồng bộ bị chậm hoặc nhiều instance backend chưa cập nhật cùng một thời điểm.

Việc sử dụng stale configuration có thể dẫn đến nhiều hậu quả như tiếp tục bật một feature đã bị tắt, áp dụng sai tỷ lệ rollout hoặc sử dụng danh sách rule không còn hợp lệ. Mặc dù khoảng thời gian tồn tại của dữ liệu cũ thường ngắn, hệ thống vẫn cần hạn chế tối đa hiện tượng này nhằm đảm bảo hành vi của ứng dụng phù hợp với cấu hình mà người quản trị đã thiết lập.

Trong hệ thống được đề xuất, stale configuration được giảm thiểu bằng cách thực hiện cache invalidation ngay sau khi cấu hình được cập nhật thành công và chỉ tạo lại configuration snapshot khi có request evaluation tiếp theo.

Giao dịch khi cập nhật cấu hình

Việc cập nhật feature flag thường liên quan đến nhiều bảng dữ liệu như feature flag, flag configuration, rule và audit log. Nếu từng thao tác được thực hiện độc lập, hệ thống có thể rơi vào trạng thái chỉ cập nhật một phần dữ liệu khi xảy ra lỗi, dẫn đến cấu hình không nhất quán.

Để đảm bảo tính toàn vẹn dữ liệu, các thay đổi liên quan đến cùng một thao tác quản trị nên được thực hiện trong một transaction của cơ sở dữ liệu. Chỉ khi transaction được commit thành công thì cache mới được invalidation hoặc làm mới. Nếu transaction bị rollback, cache hiện tại vẫn tiếp tục được sử dụng vì dữ liệu trong cơ sở dữ liệu chưa thay đổi.

Trình tự này giúp đảm bảo cache luôn phản ánh một trạng thái cấu hình hợp lệ, đồng thời tránh việc xóa cache quá sớm khiến Evaluation Engine phải tải lại dữ liệu chưa được cập nhật đầy đủ.

Cân bằng giữa hiệu năng và độ mới dữ liệu

Một hệ thống Feature Flag cần cân bằng giữa hai mục tiêu là giảm độ trễ của Evaluation API và đảm bảo dữ liệu cấu hình luôn đủ mới để phản ánh các thay đổi của người quản trị.

Nếu mọi request đều truy cập trực tiếp cơ sở dữ liệu để đảm bảo strong consistency, độ trễ của Evaluation API sẽ tăng và khả năng mở rộng của hệ thống bị hạn chế. Ngược lại, nếu phụ thuộc hoàn toàn vào cache với thời gian tồn tại dài, nguy cơ sử dụng stale configuration sẽ tăng lên.

Giải pháp được lựa chọn trong mini project là ưu tiên cache configuration snapshot kết hợp với invalidation theo sự kiện mỗi khi cấu hình thay đổi. Cơ chế này cho phép phần lớn request evaluation được xử lý trực tiếp từ cache với độ trễ thấp, đồng thời vẫn nhanh chóng phản ánh các thay đổi của feature flag sau khi người quản trị cập nhật cấu hình. Cách tiếp cận này phù hợp với các khuyến nghị của các nền tảng Feature Flag hiện đại về việc cân bằng giữa hiệu năng, khả năng mở rộng và tính nhất quán của hệ thống [8, 7].

2.8.5 Trust boundary của evaluation context

Evaluation context là tập hợp các thông tin được sử dụng để xác định trạng thái của một feature flag đối với từng request cụ thể. Các thông tin này có thể bao gồm targeting key, user ID, vai trò người dùng và các thuộc tính mở rộng như quốc gia hoặc loại thiết bị. Do Evaluation Engine hoàn toàn dựa trên evaluation context để thực hiện targeting, tính tin cậy của các dữ

liệu này ảnh hưởng trực tiếp đến tính đúng đắn của kết quả evaluation. Vì vậy, hệ thống cần xác định rõ trust boundary giữa dữ liệu đáng tin cậy và dữ liệu do ứng dụng khách cung cấp [6, 11].

Nguồn tạo evaluation context

Evaluation context có thể được tạo từ nhiều nguồn khác nhau tùy theo kiến trúc của ứng dụng. Trong các hệ thống phía server, context thường được xây dựng sau khi request đã trải qua quá trình xác thực người dùng. Thông tin như user ID, vai trò và organization được lấy từ session, access token hoặc cơ sở dữ liệu của ứng dụng thay vì từ dữ liệu người dùng tự nhập.

Đối với các ứng dụng phía client, một phần evaluation context có thể được lấy từ trạng thái hiện tại của ứng dụng hoặc thông tin đã được xác thực trước đó. Tuy nhiên, mọi dữ liệu nhận trực tiếp từ trình duyệt đều cần được xem là không hoàn toàn đáng tin cậy cho đến khi được phía server kiểm chứng [6].

Dữ liệu do client tự khai báo

Một số thuộc tính trong evaluation context có thể được client gửi trực tiếp thông qua Evaluation API, chẳng hạn như targeting key hoặc các thuộc tính phục vụ cá nhân hóa. Các dữ liệu này thuận tiện cho việc triển khai nhưng cũng làm tăng nguy cơ bị sửa đổi trước khi gửi tới server.

Trong hệ thống được đề xuất, Evaluation Engine chỉ sử dụng các thuộc tính do client cung cấp cho mục đích targeting và không xem chúng là bằng chứng xác thực danh tính của người dùng. Những thuộc tính có ý nghĩa bảo mật cần được lấy từ nguồn đã được xác thực hoặc được server tự xây dựng trong quá trình xử lý request. Cách tiếp cận này giúp hạn chế nguy cơ người dùng thay đổi dữ liệu nhằm tác động đến kết quả evaluation [11].

Nguy cơ giả mạo user ID và role

Nếu Evaluation API chấp nhận hoàn toàn user ID hoặc role do client gửi lên mà không kiểm chứng, người dùng có thể tự thay đổi các giá trị này để nhận được những tính năng không dành cho mình. Ví dụ, một người dùng thông thường có thể tự khai báo vai trò quản trị hoặc sử dụng user ID của tài khoản khác nhằm vượt qua các rule targeting.

Để giảm thiểu rủi ro này, server nên lấy user ID và vai trò từ thông tin đã được xác thực, chẳng hạn như access token hoặc session của ứng dụng. Evaluation context chỉ nên chứa các giá trị

đã được kiểm chứng thay vì dữ liệu do client tự khai báo. Đây cũng là nguyên tắc chung trong thiết kế các REST API an toàn nhằm tránh việc tin tưởng dữ liệu đầu vào từ phía người dùng [11].

Evaluation phía client và phía server

Feature flag có thể được đánh giá ở phía client hoặc phía server tùy theo yêu cầu của ứng dụng.

Evaluation phía client giúp giảm số lần gọi API và phản hồi nhanh hơn đối với các thay đổi giao diện. Tuy nhiên, evaluation context và kết quả evaluation đều có thể bị người dùng quan sát hoặc sửa đổi, do đó chỉ phù hợp với các tính năng không liên quan đến bảo mật hoặc dữ liệu nhạy cảm.

Ngược lại, evaluation phía server diễn ra trong môi trường đáng tin cậy hơn vì server có thể tự xây dựng evaluation context từ dữ liệu đã xác thực và kiểm soát toàn bộ quá trình evaluation. Trong phạm vi mini project, Evaluation Engine được triển khai trên backend, còn ứng dụng khách chỉ gửi request tới Evaluation API để nhận kết quả. Cách tiếp cận này giúp giảm nguy cơ giả mạo evaluation context và đơn giản hóa việc quản lý cấu hình feature flag.

Giới hạn sử dụng Feature Flag cho phân quyền

Feature flag được thiết kế nhằm kiểm soát quá trình phát hành và triển khai tính năng, không phải để thay thế cơ chế xác thực hoặc phân quyền của hệ thống. Mặc dù rule targeting có thể sử dụng thuộc tính vai trò để bật hoặc tắt một tính năng đối với từng nhóm người dùng, việc này chỉ quyết định khả năng hiển thị hoặc kích hoạt chức năng, không quyết định quyền truy cập tài nguyên.

Mọi thao tác liên quan đến dữ liệu quan trọng hoặc chức năng quản trị vẫn phải được kiểm tra bằng cơ chế authorization độc lập. Nếu chỉ dựa vào feature flag để bảo vệ các API nhạy cảm, người dùng có thể tìm cách bỏ qua giao diện và gọi trực tiếp endpoint của hệ thống. Do đó, feature flag nên được xem là lớp điều khiển hành vi của ứng dụng, trong khi việc bảo vệ tài nguyên vẫn thuộc trách nhiệm của cơ chế xác thực và phân quyền [11, 7].

2.8.6 Bảo vệ Management API

Khác với Evaluation API chủ yếu phục vụ truy vấn trạng thái feature flag, Management API cho phép tạo, sửa và quản lý cấu hình của hệ thống. Các endpoint này có thể thay đổi trạng

thái feature flag, cập nhật rule, bật hoặc tắt kill switch và xem lịch sử thay đổi. Nếu không được bảo vệ thích hợp, kẻ tấn công có thể thay đổi cấu hình feature flag hoặc che giấu các hành vi bất thường. Do đó, Management API cần được thiết kế theo nguyên tắc xác thực đầy đủ, phân quyền rõ ràng và ghi nhận đầy đủ các thao tác quản trị [11, 12].

Xác thực người dùng quản trị

Mọi request gửi tới Management API phải được thực hiện bởi người dùng đã được xác thực. Trước khi cho phép truy cập các chức năng quản trị, hệ thống cần kiểm tra danh tính của người gửi thông qua cơ chế xác thực phù hợp như session hoặc access token.

Sau khi xác thực thành công, thông tin người dùng được gắn vào request để phục vụ quá trình phân quyền và ghi nhận audit log. Các request không hợp lệ hoặc không có thông tin xác thực phải bị từ chối trước khi thực hiện bất kỳ thao tác nào trên dữ liệu của hệ thống. Việc tách riêng quá trình xác thực khỏi logic nghiệp vụ giúp giảm nguy cơ truy cập trái phép và đơn giản hóa việc mở rộng hệ thống trong tương lai.

Bảo vệ mutation endpoint

Các endpoint thực hiện thay đổi dữ liệu như tạo project, tạo feature flag, cập nhật rule, bật hoặc tắt kill switch và lưu cấu hình mới được gọi là mutation endpoint. Đây là nhóm endpoint có mức độ rủi ro cao vì có thể làm thay đổi hành vi của toàn bộ hệ thống.

Đối với các endpoint này, hệ thống cần kiểm tra quyền truy cập trước khi thực hiện thao tác cập nhật dữ liệu. Ngoài việc xác thực người dùng, backend phải kiểm tra vai trò hoặc quyền tương ứng đối với từng loại thao tác nhằm đảm bảo chỉ những người được ủy quyền mới có thể thay đổi cấu hình. Đồng thời, toàn bộ thao tác cập nhật cần được thực hiện thông qua transaction và chỉ ghi audit log sau khi transaction được commit thành công.

Validation actor

Mỗi thao tác quản trị cần xác định rõ actor là người thực hiện thay đổi cấu hình. Actor thường bao gồm định danh người dùng, thời điểm thực hiện và các thông tin phục vụ truy vết như request ID hoặc địa chỉ IP nếu cần thiết.

Trong hệ thống được đề xuất, mọi mutation endpoint đều yêu cầu actor hợp lệ trước khi thực hiện thay đổi dữ liệu. Nếu request không xác định được actor hoặc thông tin actor không hợp lệ,

hệ thống sẽ từ chối xử lý thay vì tạo bản ghi audit log không đầy đủ. Điều này giúp đảm bảo mọi thay đổi cấu hình đều có thể truy vết về người thực hiện và đáp ứng yêu cầu về accountability của hệ thống [12].

Giới hạn truy cập audit log

Audit log chứa lịch sử thay đổi cấu hình feature flag, bao gồm actor, thời gian thực hiện, hành động và dữ liệu trước và sau khi thay đổi. Đây là nguồn thông tin quan trọng phục vụ kiểm tra, khắc phục sự cố và điều tra các thay đổi bất thường.

Do audit log có thể chứa thông tin nhạy cảm về cấu hình hệ thống và hoạt động của người quản trị, không phải mọi người dùng đều được phép truy cập. Trong phạm vi mini project, chỉ người dùng có quyền quản trị mới được xem audit log thông qua Management API. Hệ thống không cho phép chỉnh sửa hoặc xóa các bản ghi audit log, đảm bảo lịch sử thay đổi luôn mang tính append-only và có giá trị phục vụ truy vết sau này [12, 11].

2.8.7 Phân quyền quản trị

Hệ thống quản lý Feature Flag bao gồm nhiều chức năng với mức độ ảnh hưởng khác nhau, từ xem cấu hình, đánh giá trạng thái đến thay đổi trực tiếp hành vi của ứng dụng đang chạy. Vì vậy, không phải mọi người dùng đều nên có cùng quyền truy cập đối với Management API và Dashboard. Việc áp dụng mô hình phân quyền theo vai trò (Role-Based Access Control – RBAC) giúp giới hạn phạm vi thao tác của từng nhóm người dùng, giảm nguy cơ thay đổi cấu hình ngoài ý muốn và tăng khả năng truy vết khi xảy ra sự cố. Trong phạm vi mini project, hệ thống sử dụng ba vai trò gồm *admin*, *developer* và *viewer* để minh họa cơ chế phân quyền [11].

Quyền của admin

Admin là vai trò có quyền cao nhất trong hệ thống. Người dùng thuộc vai trò này có thể thực hiện toàn bộ các chức năng quản trị như tạo project, tạo và cập nhật feature flag, cấu hình rule, bật hoặc tắt kill switch, quản lý group, xem audit log và quản lý tài khoản người dùng.

Do các thao tác của admin có thể ảnh hưởng trực tiếp đến hành vi của hệ thống đang vận hành, mọi thay đổi đều được ghi nhận vào audit log cùng với thông tin actor và thời điểm thực hiện. Đây là vai trò duy nhất được phép thực hiện các thao tác có khả năng thay đổi cấu hình của toàn bộ hệ thống.

Quyền của developer

Developer là vai trò phục vụ quá trình phát triển và kiểm thử tính năng. Người dùng thuộc nhóm này có thể xem danh sách project, feature flag, rule và trạng thái hiện tại của hệ thống. Tùy theo chính sách triển khai, developer cũng có thể được phép cập nhật một số cấu hình trong môi trường phát triển hoặc thử nghiệm.

Trong phạm vi mini project, developer được phép quản lý feature flag và rule nhưng không được thực hiện các thao tác quản trị hệ thống như quản lý tài khoản hoặc thay đổi quyền của người dùng khác. Việc tách biệt quyền của developer giúp giảm rủi ro thay đổi ngoài ý muốn đối với các chức năng quản trị quan trọng.

Quyền của viewer

Viewer là vai trò chỉ có quyền đọc dữ liệu. Người dùng thuộc nhóm này có thể xem danh sách project, feature flag, trạng thái runtime và audit log nếu được cấp quyền, nhưng không được phép thực hiện bất kỳ thao tác thay đổi cấu hình nào.

Mọi endpoint thực hiện tạo, cập nhật hoặc xóa dữ liệu đều phải từ chối request đến từ viewer. Vai trò này phù hợp với các đối tượng cần theo dõi trạng thái hệ thống hoặc kiểm tra cấu hình mà không tham gia vào quá trình quản trị.

Kiểm tra quyền tại Backend API

Việc kiểm tra quyền phải được thực hiện tại Backend API thay vì chỉ dựa trên giao diện người dùng. Mỗi request gửi tới Management API trước tiên được xác thực để xác định danh tính người dùng, sau đó backend kiểm tra vai trò tương ứng trước khi thực hiện logic nghiệp vụ.

Nếu người dùng không có quyền thực hiện thao tác yêu cầu, hệ thống trả về mã lỗi phù hợp và không thực hiện bất kỳ thay đổi nào đối với dữ liệu. Cách tiếp cận này đảm bảo các endpoint vẫn được bảo vệ ngay cả khi người dùng bỏ qua Dashboard và gửi request trực tiếp tới API [11].

Hiển thị quyền trên Dashboard

Bên cạnh việc kiểm tra quyền tại backend, Dashboard cũng cần phản ánh rõ quyền hạn của từng người dùng nhằm cải thiện trải nghiệm sử dụng. Các chức năng không được phép thực

hiện có thể được ẩn hoàn toàn hoặc hiển thị ở trạng thái vô hiệu hóa, đồng thời cung cấp thông báo giải thích khi cần thiết.

Tuy nhiên, việc ẩn nút chức năng trên giao diện chỉ mang ý nghĩa hỗ trợ người dùng và không thay thế cho cơ chế kiểm tra quyền tại Backend API. Mọi thao tác thay đổi cấu hình vẫn phải được xác thực và phân quyền ở phía server để đảm bảo an toàn cho hệ thống.

2.8.8 Auditability và truy vết thay đổi

Một trong những yêu cầu quan trọng của hệ thống Feature Flag là mọi thay đổi cấu hình đều phải có khả năng truy vết. Khi xảy ra sự cố trên môi trường thực tế, đội phát triển cần xác định ai đã thực hiện thay đổi, thay đổi những gì và thay đổi vào thời điểm nào. Vì vậy, hệ thống cần xây dựng cơ chế audit log đầy đủ, đảm bảo mọi thao tác quản trị đều được ghi nhận và không thể bị chỉnh sửa sau khi lưu trữ. Điều này không chỉ hỗ trợ điều tra sự cố mà còn tăng tính minh bạch và trách nhiệm trong quá trình quản trị hệ thống [12].

Ghi nhận actor

Mỗi bản ghi audit log cần xác định rõ actor là người thực hiện thao tác. Actor thường bao gồm định danh người dùng đã được xác thực và có thể kèm theo các thông tin bổ sung như tên hiển thị hoặc vai trò tại thời điểm thực hiện thao tác.

Việc ghi nhận actor giúp xác định trách nhiệm đối với từng thay đổi cấu hình và hỗ trợ quá trình kiểm tra khi có sự cố hoặc thay đổi ngoài mong muốn. Trong hệ thống được đề xuất, mọi mutation endpoint đều yêu cầu actor hợp lệ trước khi thực hiện thay đổi dữ liệu. Nếu không xác định được actor, request sẽ bị từ chối thay vì tạo bản ghi audit log không đầy đủ.

Ghi nhận request ID

Bên cạnh actor, mỗi thao tác quản trị cũng được gắn với một request ID duy nhất. Request ID được tạo khi request đi vào hệ thống và được truyền xuyên suốt qua các thành phần của backend như controller, service, repository và audit log.

Việc lưu request ID giúp liên kết các bản ghi log của cùng một request, hỗ trợ quá trình truy vết khi phân tích lỗi hoặc điều tra sự cố. Khi kết hợp với log của ứng dụng và cơ sở dữ liệu, request ID giúp tái hiện toàn bộ luồng xử lý của một thao tác quản trị mà không cần suy luận từ thời gian thực hiện.

Snapshot trước và sau thay đổi

Để biết chính xác nội dung đã được chỉnh sửa, audit log không chỉ ghi nhận loại thao tác mà còn lưu snapshot của dữ liệu trước và sau khi thay đổi. Snapshot có thể bao gồm các thông tin như trạng thái feature flag, serving mode, danh sách rule hoặc các thuộc tính cấu hình khác tùy theo đối tượng được cập nhật.

Việc lưu cả hai phiên bản dữ liệu giúp dễ dàng xác định các trường đã thay đổi, hỗ trợ khôi phục cấu hình khi cần thiết và phục vụ quá trình kiểm tra lịch sử thay đổi. Trong hệ thống được đề xuất, snapshot được lưu dưới dạng dữ liệu JSON nhằm biểu diễn đầy đủ cấu trúc của từng đối tượng mà không phụ thuộc vào số lượng thuộc tính của từng bảng dữ liệu [15].

Bảo vệ tính toàn vẹn audit log

Audit log chỉ có giá trị khi các bản ghi không thể bị chỉnh sửa hoặc xóa sau khi đã được tạo. Nếu lịch sử thay đổi có thể bị sửa đổi, hệ thống sẽ mất khả năng truy vết và không còn đáp ứng yêu cầu về tính minh bạch của quá trình quản trị.

Trong phạm vi mini project, audit log được thiết kế theo mô hình append-only. Sau khi một bản ghi được tạo, hệ thống không cung cấp API cập nhật hoặc xóa dữ liệu. Đồng thời, cơ sở dữ liệu có thể áp dụng các ràng buộc hoặc trigger để ngăn chặn thao tác `UPDATE` và `DELETE` trên bảng audit log. Cách tiếp cận này giúp bảo vệ tính toàn vẹn của lịch sử thay đổi và đảm bảo mọi thao tác quản trị đều có thể được truy vết về sau [12].

2.9 Phương pháp kiểm thử và đánh giá

2.9.1 Unit test cho Rule Evaluation Engine

Rule Evaluation Engine là thành phần quyết định trạng thái cuối cùng của mỗi feature flag dựa trên cấu hình và evaluation context. Do đây là thành phần chứa phần lớn logic nghiệp vụ của hệ thống, việc xây dựng các bài kiểm thử đơn vị (unit test) giúp xác minh tính đúng đắn của thuật toán, đảm bảo mỗi quy tắc được thực thi theo đúng thứ tự ưu tiên đã thiết kế và hạn chế lỗi khi thay đổi mã nguồn trong tương lai. Các bài kiểm thử được thực hiện độc lập với cơ sở dữ liệu và API, chỉ tập trung đánh giá đầu vào và đầu ra của Evaluation Engine. Cách tiếp cận này phù hợp với khuyến nghị tách biệt logic đánh giá feature flag khỏi các thành phần lưu trữ và giao tiếp mạng để tăng khả năng kiểm thử và bảo trì [10, 8].

Kiểm thử trạng thái archived và disabled

Nhóm kiểm thử đầu tiên xác minh các trạng thái đặc biệt của feature flag. Khi một flag ở trạng thái `ARCHIVED`, hệ thống phải trả về kết quả tắt ngay lập tức và không tiếp tục đánh giá các rule phía sau. Tương tự, khi cấu hình của flag ở trạng thái `DISABLED`, Evaluation Engine cũng phải kết thúc quá trình đánh giá và trả về reason tương ứng.

Các trường hợp kiểm thử bao gồm:

- Flag ở trạng thái `ARCHIVED`.
- Flag ở trạng thái `DISABLED`.
- Flag ở trạng thái hoạt động bình thường để xác nhận Evaluation Engine tiếp tục đánh giá các rule còn lại.

Các bài kiểm thử này nhằm đảm bảo thứ tự ưu tiên của các trạng thái hệ thống luôn được duy trì, tránh việc một flag đã bị vô hiệu hóa vẫn có thể được bật bởi các rule targeting phía sau [7, 1].

Kiểm thử kill switch

Kill switch là cơ chế cho phép vô hiệu hóa ngay lập tức một feature mà không cần triển khai lại ứng dụng. Vì vậy, Evaluation Engine phải ưu tiên kiểm tra kill switch trước các rule targeting.

Các trường hợp kiểm thử bao gồm:

- Kill switch bật, kết quả evaluation luôn trả về `enabled = false`.
- Kill switch tắt, Evaluation Engine tiếp tục đánh giá các rule tiếp theo.
- Kill switch bật đồng thời tồn tại rule user hoặc role phù hợp, kết quả vẫn phải ưu tiên kill switch.

Việc kiểm thử bảo đảm hệ thống luôn có khả năng rollback nhanh khi phát sinh sự cố trên môi trường production, đúng với mục tiêu của Feature Flag trong quy trình phát hành phần mềm hiện đại [1, 8].

Kiểm thử user và role targeting

Nhóm kiểm thử này xác minh khả năng đánh giá các rule dựa trên thuộc tính của người dùng trong evaluation context. Evaluation Engine phải xác định đúng khi người dùng thuộc danh sách cho phép hoặc sở hữu vai trò phù hợp với cấu hình của feature flag.

Các trường hợp kiểm thử gồm:

- User ID nằm trong danh sách cho phép.
- User ID không nằm trong danh sách.
- Role của người dùng khớp với rule.
- Role không khớp với bất kỳ rule nào.
- Đồng thời tồn tại nhiều rule với mức ưu tiên khác nhau.

Các bài kiểm thử cũng xác nhận Evaluation Engine áp dụng đúng thứ tự ưu tiên giữa các rule, đảm bảo kết quả evaluation luôn xác định và nhất quán [4, 6].

Kiểm thử percentage rollout

Percentage rollout sử dụng giá trị băm ổn định của `targetingKey` để phân bổ người dùng vào các bucket phần trăm. Do đó, các bài kiểm thử không chỉ kiểm tra việc so sánh với ngưỡng rollout mà còn phải bảo đảm tính ổn định của thuật toán.

Các trường hợp kiểm thử gồm:

- Rollout bằng 0%.
- Rollout bằng 100%.
- Rollout với các giá trị trung gian như 10%, 25%, 50% và 75%.
- Cùng một `targetingKey` luôn cho kết quả giống nhau qua nhiều lần evaluation.
- Hai `targetingKey` khác nhau có thể thuộc các bucket khác nhau.

Những bài kiểm thử này giúp bảo đảm việc phân phối người dùng ổn định giữa các lần đánh giá, tránh hiện tượng người dùng liên tục thay đổi trạng thái feature sau mỗi request [5].

Kiểm thử default và error fallback

Ngoài các trường hợp thành công, Evaluation Engine cũng cần xử lý an toàn khi không tìm được rule phù hợp hoặc xảy ra lỗi trong quá trình đánh giá. Theo nguyên tắc fail-safe, hệ thống phải trả về giá trị mặc định thay vì phát sinh hành vi không xác định.

Các trường hợp kiểm thử gồm:

- Không có rule nào khớp, trả về giá trị mặc định.
- Evaluation context thiếu thông tin bắt buộc.
- Flag không tồn tại.
- Xuất hiện ngoại lệ trong quá trình evaluation.

Kết quả mong đợi là Evaluation Engine luôn trả về phản hồi hợp lệ với reason code phù hợp thay vì làm gián đoạn ứng dụng sử dụng Feature Flag. Điều này giúp tăng tính ổn định và khả năng chịu lỗi của hệ thống trong môi trường thực tế [10, 11].

2.9.2 Integration test cho API

Bên cạnh các bài kiểm thử đơn vị đối với Rule Evaluation Engine, hệ thống còn được kiểm thử tích hợp (integration test) nhằm xác minh sự phối hợp giữa các thành phần như Controller, Service, Repository, cơ sở dữ liệu và các middleware. Các bài kiểm thử được thực hiện thông qua việc gửi yêu cầu HTTP tới các endpoint của Backend API và kiểm tra toàn bộ phản hồi, dữ liệu được lưu trong cơ sở dữ liệu cũng như các tác động phát sinh sau mỗi thao tác. Cách tiếp cận này giúp đánh giá tính đúng đắn của API từ góc nhìn của ứng dụng khách, đồng thời bảo đảm các endpoint tuân thủ hợp đồng API đã thiết kế [10, 11].

Kiểm thử API quản trị

Nhóm kiểm thử đầu tiên tập trung vào các API quản trị phục vụ việc quản lý project, feature flag, cấu hình rule và group. Mỗi endpoint được kiểm tra cả trường hợp thành công lẫn các tình huống dữ liệu không hợp lệ.

Các trường hợp kiểm thử bao gồm:

- Tạo mới project và feature flag thành công.
- Cập nhật thông tin feature flag.

- Thêm, sửa và xóa rule.
- Thay đổi trạng thái enabled, disabled hoặc archived.
- Gửi dữ liệu không hợp lệ và kiểm tra phản hồi validation.
- Kiểm tra các trường hợp conflict hoặc tài nguyên không tồn tại.

Kết quả mong đợi là API trả về đúng mã trạng thái HTTP, dữ liệu được lưu chính xác trong cơ sở dữ liệu và phản hồi tuân theo cấu trúc thống nhất đã thiết kế [11].

Kiểm thử Evaluation API

Evaluation API là endpoint quan trọng nhất của hệ thống vì được các ứng dụng khách sử dụng tại thời điểm chạy. Các bài kiểm thử xác minh toàn bộ luồng từ việc nhận evaluation context, truy xuất cấu hình flag đến trả về kết quả cuối cùng.

Các trường hợp kiểm thử gồm:

- Đánh giá feature được bật toàn cục.
- Đánh giá theo user targeting.
- Đánh giá theo role targeting.
- Đánh giá theo percentage rollout.
- Flag không tồn tại.
- Evaluation context thiếu dữ liệu.
- Kiểm tra reason code và variant trả về.

Các bài kiểm thử bảo đảm Evaluation API luôn trả về kết quả xác định với cùng một evaluation context, đồng thời phản hồi đúng giá trị mặc định khi không có rule phù hợp hoặc xảy ra lỗi trong quá trình đánh giá [10, 6].

Kiểm thử audit log

Audit log được sử dụng để ghi nhận toàn bộ thay đổi cấu hình nhằm phục vụ truy vết và kiểm toán sau này. Vì vậy, mỗi thao tác thay đổi dữ liệu đều phải tạo ra một bản ghi audit tương ứng.

Các trường hợp kiểm thử gồm:

- Tạo mới feature flag.
- Cập nhật cấu hình flag.
- Thay đổi rule.
- Thay đổi trạng thái archived hoặc restored.
- Kiểm tra thông tin actor, request ID, thời gian và snapshot trước/sau thay đổi.
- Xác minh các bản ghi audit không bị cập nhật hoặc xóa sau khi tạo.

Kết quả mong đợi là mỗi thao tác quản trị tạo đúng một bản ghi audit với đầy đủ thông tin phục vụ truy vết, đồng thời bảo đảm tính toàn vẹn của lịch sử thay đổi [12].

Kiểm thử quyền truy cập

Hệ thống áp dụng cơ chế phân quyền theo vai trò nhằm giới hạn các thao tác quản trị đối với từng nhóm người dùng. Các bài kiểm thử tích hợp xác minh việc kiểm tra quyền được thực hiện chính xác tại Backend API.

Các trường hợp kiểm thử gồm:

- Người dùng có quyền quản trị thực hiện thành công các thao tác tạo, sửa và lưu trữ feature flag.
- Người dùng không đủ quyền bị từ chối truy cập các mutation endpoint.
- Người dùng chỉ có quyền xem vẫn có thể truy cập các API chỉ đọc.
- Yêu cầu không có thông tin xác thực bị từ chối.
- Kiểm tra mã trạng thái HTTP và thông báo lỗi khi truy cập trái phép.

Các bài kiểm thử này giúp bảo đảm chỉ những người dùng được cấp quyền mới có thể thay đổi cấu hình feature flag, giảm nguy cơ thay đổi trái phép và phù hợp với các khuyến nghị về bảo vệ REST API cũng như kiểm soát truy cập [11].

2.9.3 Kiểm thử validation và xử lý lỗi

Ngoài việc kiểm thử các luồng xử lý thành công, hệ thống còn được kiểm thử đối với các tình huống ngoại lệ nhằm đánh giá khả năng validation dữ liệu và xử lý lỗi của Backend API. Mục tiêu của nhóm kiểm thử này là bảo đảm mọi yêu cầu không hợp lệ đều được phát hiện sớm, trả

về thông báo lỗi rõ ràng và không làm ảnh hưởng đến tính nhất quán của dữ liệu. Đồng thời, API phải duy trì cấu trúc phản hồi lỗi thống nhất để ứng dụng khách có thể xử lý một cách nhất quán [11, 10].

Dữ liệu đầu vào không hợp lệ

Nhóm kiểm thử đầu tiên xác minh cơ chế validation đối với các tham số được gửi từ ứng dụng khách. Các ràng buộc về định dạng, kiểu dữ liệu, phạm vi giá trị và các trường bắt buộc đều được kiểm tra trước khi yêu cầu được xử lý.

Các trường hợp kiểm thử bao gồm:

- Thiếu các trường bắt buộc trong request.
- Feature flag key không đúng định dạng.
- Giá trị percentage rollout nằm ngoài khoảng từ 0 đến 100.
- Enum hoặc trạng thái không hợp lệ.
- Kiểu dữ liệu của các trường không đúng với API specification.

Kết quả mong đợi là API trả về mã lỗi phù hợp cùng danh sách các trường vi phạm để ứng dụng khách có thể dễ dàng xác định nguyên nhân và thực hiện sửa lỗi.

Resource không tồn tại

Nhóm kiểm thử này xác minh khả năng xử lý khi tài nguyên được yêu cầu không còn tồn tại hoặc chưa từng được tạo trong hệ thống.

Các trường hợp kiểm thử gồm:

- Project không tồn tại.
- Feature flag không tồn tại.
- Rule không tồn tại.
- Environment không tồn tại.
- Evaluation API yêu cầu đánh giá một flag không tồn tại.

Trong các trường hợp trên, hệ thống phải trả về mã trạng thái HTTP phù hợp cùng thông báo lỗi rõ ràng. Đối với Evaluation API, hệ thống trả về giá trị mặc định theo nguyên tắc fail-safe

thay vì làm gián đoạn ứng dụng khách [10].

Xung đột dữ liệu

Các bài kiểm thử xung đột dữ liệu được sử dụng để xác minh việc áp dụng các ràng buộc toàn vẹn trong cơ sở dữ liệu cũng như logic nghiệp vụ của hệ thống.

Các trường hợp kiểm thử bao gồm:

- Tạo feature flag với key đã tồn tại trong cùng project.
- Tạo environment mặc định khi project đã có environment mặc định khác.
- Tạo dữ liệu vi phạm các ràng buộc khóa duy nhất.
- Thực hiện thao tác không hợp lệ đối với tài nguyên đã được lưu trữ.

Kết quả mong đợi là hệ thống từ chối thao tác, không làm thay đổi dữ liệu hiện có và trả về thông tin lỗi thể hiện rõ nguyên nhân xảy ra xung đột.

Lỗi nội bộ hệ thống

Bên cạnh các lỗi do dữ liệu đầu vào, hệ thống cũng được kiểm thử đối với các lỗi phát sinh trong quá trình xử lý nhằm đánh giá khả năng chịu lỗi và duy trì tính ổn định của dịch vụ.

Các trường hợp kiểm thử gồm:

- Không thể kết nối cơ sở dữ liệu.
- Phát sinh ngoại lệ trong Rule Evaluation Engine.
- Lỗi khi ghi audit log.
- Lỗi không xác định trong quá trình xử lý request.

Trong các trường hợp này, API phải trả về phản hồi lỗi theo cấu trúc thống nhất, bao gồm mã lỗi, thông báo mô tả và request ID để phục vụ việc truy vết. Hệ thống không được để lộ thông tin nội bộ như stack trace hoặc chi tiết triển khai nhằm giảm thiểu nguy cơ khai thác lỗ hổng bảo mật, đồng thời vẫn bảo đảm khả năng chẩn đoán sự cố thông qua cơ chế ghi log [11, 12].

2.9.4 Kiểm thử stable percentage rollout

Percentage rollout là cơ chế cho phép phát hành tính năng dần dần tới một tỷ lệ người dùng xác định. Để bảo đảm trải nghiệm nhất quán, hệ thống sử dụng thuật toán băm ổn định (stable hashing) dựa trên `targetingKey` nhằm ánh xạ mỗi người dùng vào một bucket cố định. Do đó, việc kiểm thử không chỉ đánh giá tỷ lệ phân phối mà còn phải xác minh tính ổn định của kết quả khi thực hiện nhiều lần evaluation cũng như khi thay đổi giá trị rollout. Đây là một đặc điểm quan trọng của các hệ thống Feature Flag hiện đại nhằm tránh hiện tượng người dùng liên tục thay đổi trạng thái của cùng một feature giữa các lần truy cập [5, 4].

Tính ổn định của bucket

Nhóm kiểm thử đầu tiên xác minh rằng cùng một `targetingKey` luôn được ánh xạ tới cùng một bucket khi cấu hình feature flag không thay đổi.

Các trường hợp kiểm thử bao gồm:

- Thực hiện evaluation nhiều lần với cùng một `targetingKey`.
- Khởi động lại ứng dụng và thực hiện evaluation lại.
- Đánh giá cùng một người dùng trên nhiều request khác nhau.
- So sánh kết quả trước và sau khi làm mới cache.

Kết quả mong đợi là giá trị bucket và trạng thái feature luôn giữ nguyên trong mọi lần đánh giá. Điều này bảo đảm người dùng không bị thay đổi trạng thái feature một cách ngẫu nhiên do thuật toán phân phối.

Phân bố người dùng theo phần trăm

Nhóm kiểm thử này đánh giá khả năng phân phối người dùng của thuật toán stable hashing trên tập dữ liệu mẫu với số lượng lớn người dùng.

Các trường hợp kiểm thử gồm:

- Rollout ở mức 10%.
- Rollout ở mức 25%.
- Rollout ở mức 50%.

- Rollout ở mức 75%.
- Rollout ở mức 100%.

Đối với mỗi mức rollout, hệ thống thực hiện evaluation trên tập người dùng mẫu và thống kê số lượng người dùng được bật feature. Kết quả mong đợi là tỷ lệ phân bố xấp xỉ giá trị cấu hình, đồng thời không xuất hiện hiện tượng thiên lệch rõ rệt giữa các bucket [5].

Ảnh hưởng khi tăng tỷ lệ rollout

Khi tăng giá trị rollout, hệ thống cần bảo đảm những người dùng đã được bật feature trước đó vẫn tiếp tục được bật, đồng thời chỉ bổ sung thêm người dùng mới vào tập được phục vụ.

Các trường hợp kiểm thử bao gồm:

- Tăng rollout từ 10% lên 25%.
- Tăng rollout từ 25% lên 50%.
- Tăng rollout từ 50% lên 100%.

Kết quả mong đợi là toàn bộ người dùng thuộc rollout cũ vẫn giữ nguyên trạng thái được bật feature, trong khi chỉ các bucket mới được bổ sung trở thành người dùng được phục vụ. Điều này giúp quá trình phát hành diễn ra liên tục và hạn chế ảnh hưởng tới người dùng đã tham gia rollout trước đó [5].

Ảnh hưởng khi giảm tỷ lệ rollout

Ngược lại, khi giảm tỷ lệ rollout, hệ thống phải loại bỏ những người dùng nằm ngoài ngưỡng rollout mới nhưng vẫn duy trì tính ổn định đối với các bucket còn lại.

Các trường hợp kiểm thử gồm:

- Giảm rollout từ 100% xuống 50%.
- Giảm rollout từ 50% xuống 25%.
- Giảm rollout từ 25% xuống 10%.

Kết quả mong đợi là những người dùng vẫn nằm trong khoảng rollout mới tiếp tục được bật feature, trong khi chỉ các bucket vượt quá ngưỡng mới bị tắt. Việc kiểm thử này xác nhận rằng

quá trình thu hẹp phạm vi phát hành được thực hiện có kiểm soát, không làm thay đổi trạng thái của toàn bộ người dùng và vẫn bảo đảm tính xác định của thuật toán stable hashing [5, 8].

2.9.5 Kiểm thử cache và invalidation

Để giảm độ trễ của Evaluation API và hạn chế số lần truy cập cơ sở dữ liệu, hệ thống sử dụng cơ chế cache đối với cấu hình feature flag. Tuy nhiên, việc sử dụng cache phải đi kèm cơ chế invalidation phù hợp nhằm bảo đảm dữ liệu được sử dụng trong quá trình evaluation luôn phản ánh cấu hình mới nhất. Do đó, nhóm kiểm thử này tập trung đánh giá tính đúng đắn của cache trong các tình huống truy cập thông thường, cập nhật cấu hình và xử lý sự cố. Các nguyên tắc này cũng phù hợp với khuyến nghị về quản lý cache trong các hệ thống Feature Flag hiện đại [13, 8].

Cache hit và cache miss

Nhóm kiểm thử đầu tiên xác minh hành vi của hệ thống khi dữ liệu đã tồn tại hoặc chưa tồn tại trong cache.

Các trường hợp kiểm thử bao gồm:

- Lần đánh giá đầu tiên khi cache chưa có dữ liệu (cache miss).
- Các lần đánh giá tiếp theo đối với cùng một feature flag (cache hit).
- Đánh giá nhiều feature flag khác nhau để xác nhận mỗi cấu hình được lưu độc lập.
- So sánh kết quả evaluation giữa cache hit và cache miss.

Kết quả mong đợi là cache miss sẽ tải dữ liệu từ cơ sở dữ liệu và lưu vào cache, trong khi các lần truy cập tiếp theo sử dụng dữ liệu đã lưu mà vẫn trả về kết quả evaluation giống nhau.

Invalidation sau mutation

Khi cấu hình feature flag thay đổi, dữ liệu cũ trong cache phải được loại bỏ hoặc cập nhật để tránh sử dụng cấu hình không còn hợp lệ.

Các trường hợp kiểm thử gồm:

- Thay đổi trạng thái enabled hoặc disabled của feature flag.
- Thay đổi serving mode hoặc kill switch.

- Thêm, sửa hoặc xóa rule targeting.
- Lưu trữ hoặc khôi phục feature flag.

Sau mỗi thao tác mutation, yêu cầu evaluation tiếp theo phải đọc cấu hình mới thay vì sử dụng dữ liệu cũ trong cache. Điều này bảo đảm cấu hình được cập nhật nhất quán giữa Dashboard và Evaluation API [13].

Isolation giữa các environment

Một feature flag có thể được cấu hình khác nhau giữa các environment như Development, Staging và Production. Vì vậy, cache phải được tách biệt theo từng environment để tránh sử dụng nhầm cấu hình.

Các trường hợp kiểm thử bao gồm:

- Hai environment có cấu hình khác nhau cho cùng một feature flag.
- Cập nhật cấu hình tại một environment.
- Kiểm tra dữ liệu cache của environment còn lại không bị ảnh hưởng.
- Thực hiện evaluation đồng thời trên nhiều environment.

Kết quả mong đợi là mỗi environment sử dụng vùng cache riêng và việc invalidation chỉ tác động đến environment có thay đổi cấu hình, bảo đảm tính độc lập giữa các môi trường triển khai.

Hành vi khi cache không khả dụng

Cache chỉ đóng vai trò tối ưu hiệu năng và không được trở thành điểm gây gián đoạn dịch vụ. Khi cache gặp sự cố, hệ thống vẫn phải tiếp tục phục vụ yêu cầu bằng cách truy xuất trực tiếp cơ sở dữ liệu.

Các trường hợp kiểm thử gồm:

- Cache service không khả dụng.
- Lỗi khi đọc dữ liệu từ cache.
- Lỗi khi ghi dữ liệu vào cache.
- Cache bị xóa toàn bộ trong khi hệ thống đang hoạt động.

Kết quả mong đợi là Evaluation API vẫn trả về kết quả chính xác bằng cách sử dụng dữ liệu từ cơ sở dữ liệu. Mặc dù thời gian phản hồi có thể tăng do không còn cache hỗ trợ, hệ thống vẫn duy trì tính đúng đắn của kết quả và không làm gián đoạn dịch vụ [13, 8].

2.9.6 Đánh giá Dashboard

Bên cạnh các tiêu chí về tính đúng đắn của Backend API, hệ thống còn được đánh giá thông qua Dashboard quản trị nhằm xác định mức độ hỗ trợ người quản trị trong quá trình cấu hình và theo dõi feature flag. Việc đánh giá tập trung vào khả năng hiển thị trạng thái, cấu hình rule, truy vết thay đổi và hạn chế các thao tác gây ảnh hưởng ngoài ý muốn. Các tiêu chí này phù hợp với mục tiêu xây dựng một hệ thống Feature Flag dễ sử dụng, dễ quản trị và giảm thiểu rủi ro do lỗi cấu hình [7, 8].

Khả năng nhận biết trạng thái flag

Dashboard cần thể hiện trực quan trạng thái hiện tại của mỗi feature flag để người quản trị có thể nhanh chóng xác định khả năng phục vụ của hệ thống mà không cần kiểm tra từng cấu hình chi tiết.

Các tiêu chí đánh giá bao gồm:

- Hiển thị rõ trạng thái enabled, disabled và archived.
- Hiển thị serving mode hiện tại của feature flag.
- Phân biệt trực quan các trạng thái bằng màu sắc hoặc badge.
- Cho phép tìm kiếm và lọc feature flag theo trạng thái.

Kết quả mong đợi là người quản trị có thể nhanh chóng xác định trạng thái hoạt động của từng feature flag, từ đó giảm thời gian thao tác và hạn chế nhầm lẫn trong quá trình vận hành.

Khả năng cấu hình rule

Dashboard cần hỗ trợ việc tạo và chỉnh sửa các rule targeting một cách trực quan, đồng thời phản ánh đúng mô hình dữ liệu của hệ thống.

Các tiêu chí đánh giá gồm:

- Tạo mới và chỉnh sửa các rule theo user, role và percentage rollout.

- Thay đổi thứ tự ưu tiên của các rule.
- Cập nhật trạng thái hoặc serving mode của feature flag.
- Validation dữ liệu ngay trên giao diện trước khi gửi yêu cầu đến Backend API.

Kết quả mong đợi là người quản trị có thể hoàn thành việc cấu hình feature flag với số bước thao tác tối thiểu, đồng thời giảm nguy cơ tạo ra cấu hình không hợp lệ.

Khả năng theo dõi thay đổi

Dashboard cần hỗ trợ việc theo dõi lịch sử thay đổi cấu hình nhằm phục vụ kiểm toán và truy vết khi xảy ra sự cố.

Các tiêu chí đánh giá bao gồm:

- Hiển thị danh sách các bản ghi audit log.
- Hiển thị actor thực hiện thay đổi.
- Hiển thị thời gian và hành động tương ứng.
- Hiển thị thông tin before và after của mỗi lần cập nhật.
- Hỗ trợ tìm kiếm hoặc lọc lịch sử thay đổi.

Kết quả mong đợi là người quản trị có thể nhanh chóng xác định ai đã thực hiện thay đổi, thời điểm thực hiện và nội dung đã được cập nhật, từ đó nâng cao khả năng truy vết và kiểm toán của hệ thống [12].

Khả năng hạn chế thao tác sai

Một Dashboard quản trị không chỉ hỗ trợ thao tác nhanh mà còn cần giảm thiểu các lỗi do người dùng gây ra thông qua các cơ chế xác nhận và kiểm tra dữ liệu.

Các tiêu chí đánh giá gồm:

- Xác nhận trước các thao tác quan trọng như archived hoặc kill switch.
- Vô hiệu hóa các chức năng mà người dùng không có quyền thực hiện.
- Hiển thị thông báo lỗi khi validation không thành công.
- Hiển thị thông báo kết quả sau khi thao tác thành công hoặc thất bại.

Kết quả mong đợi là Dashboard hỗ trợ người quản trị thực hiện cấu hình một cách an toàn, giảm thiểu thao tác ngoài ý muốn và bảo đảm chỉ những người dùng được cấp quyền mới có thể thay đổi cấu hình của hệ thống [11, 7].

2.9.7 Thiết kế kịch bản demo

Bên cạnh các bài kiểm thử tự động, hệ thống được đánh giá thông qua các kịch bản trình diễn (demo scenario) nhằm minh họa trực quan giá trị của Feature Flag trong quá trình phát triển và vận hành phần mềm. Mỗi kịch bản được xây dựng để phản ánh một tình huống sử dụng phổ biến trong thực tế, đồng thời chứng minh sự phối hợp giữa Dashboard quản trị, Backend API, Rule Evaluation Engine và ứng dụng khách. Việc thiết kế các kịch bản này cũng bám sát các chức năng bắt buộc và khuyến khích của mini project, giúp đánh giá toàn diện khả năng triển khai của hệ thống [1, 7].

Kịch bản bật và tắt toàn cục

Kịch bản đầu tiên minh họa khả năng phát hành hoặc thu hồi một tính năng cho toàn bộ người dùng mà không cần triển khai lại ứng dụng.

Các bước thực hiện gồm:

- Tạo một feature flag trên Dashboard.
- Chuyển serving mode sang chế độ bật toàn cục.
- Truy cập Demo App để xác nhận tính năng được hiển thị cho tất cả người dùng.
- Chuyển trạng thái về disabled hoặc tắt chế độ bật toàn cục.
- Thực hiện evaluation lại và xác nhận tính năng không còn được hiển thị.

Kịch bản này chứng minh khả năng tách biệt giữa quá trình triển khai và phát hành tính năng, cho phép thay đổi hành vi của hệ thống ngay tại thời điểm chạy mà không cần triển khai lại ứng dụng [1, 3].

Kịch bản targeting theo user và role

Kịch bản thứ hai minh họa khả năng phát hành tính năng cho một nhóm người dùng xác định dựa trên user hoặc role.

Các bước thực hiện gồm:

- Cấu hình rule theo user ID hoặc role trên Dashboard.
- Đăng nhập hoặc lựa chọn người dùng phù hợp trong Demo App.
- Thực hiện evaluation và xác nhận tính năng được bật.
- Thay đổi sang người dùng không thỏa điều kiện.
- Thực hiện evaluation lại và xác nhận tính năng không được bật.

Kịch bản này chứng minh khả năng triển khai từng phần cho các nhóm người dùng cụ thể, hỗ trợ thử nghiệm nội bộ hoặc phát hành giới hạn trước khi triển khai rộng rãi [4, 6].

Kịch bản percentage rollout

Kịch bản này minh họa cơ chế phát hành theo tỷ lệ phần trăm người dùng dựa trên stable hashing.

Các bước thực hiện gồm:

- Cấu hình rollout ở một tỷ lệ xác định, ví dụ 20% hoặc 50%.
- Thực hiện evaluation trên nhiều người dùng mẫu.
- Ghi nhận số lượng người dùng được bật feature.
- Tăng hoặc giảm tỷ lệ rollout.
- Thực hiện evaluation lại để quan sát sự thay đổi của nhóm người dùng được phục vụ.

Kết quả mong đợi là cùng một người dùng luôn nhận được kết quả nhất quán trong cùng một mức rollout, đồng thời việc tăng hoặc giảm tỷ lệ chỉ ảnh hưởng đến các bucket mới được thêm hoặc loại bỏ [5].

Kịch bản kill switch

Kịch bản này mô phỏng tình huống phát hiện lỗi trên môi trường production và cần vô hiệu hóa tính năng ngay lập tức.

Các bước thực hiện gồm:

- Cấu hình feature đang được bật cho một nhóm người dùng.

- Kích hoạt kill switch trên Dashboard.
- Thực hiện evaluation lại từ Demo App.
- Xác nhận toàn bộ người dùng đều nhận kết quả tắt feature.
- Hủy kích hoạt kill switch và xác nhận hệ thống quay lại trạng thái ban đầu.

Kịch bản chứng minh khả năng rollback nhanh của hệ thống mà không cần triển khai phiên bản mới, góp phần giảm phạm vi ảnh hưởng khi xảy ra sự cố trên môi trường thực tế [1, 8].

Kịch bản audit log và thống kê

Kịch bản cuối cùng minh họa khả năng theo dõi lịch sử thay đổi và quan sát hoạt động của hệ thống sau mỗi thao tác quản trị.

Các bước thực hiện gồm:

- Thực hiện tạo mới hoặc cập nhật feature flag.
- Thay đổi cấu hình rule hoặc trạng thái của feature.
- Kiểm tra Dashboard để xác nhận bản ghi audit được tạo.
- Thực hiện nhiều yêu cầu evaluation từ Demo App.
- Quan sát số liệu thống kê và lịch sử thay đổi của feature flag.

Kết quả mong đợi là mỗi thao tác quản trị đều được ghi nhận đầy đủ với thông tin actor, thời gian và nội dung thay đổi, đồng thời các số liệu thống kê phản ánh hoạt động của Evaluation API. Kịch bản này giúp chứng minh khả năng truy vết, kiểm toán và giám sát của hệ thống trong quá trình vận hành [12, 7].

Chương 3

Kết quả thực hiện và đánh giá

3.1 Môi trường và cấu trúc triển khai

3.1.1 Cấu trúc mã nguồn

Nhằm tách biệt các thành phần của hệ thống nhưng vẫn thuận tiện cho việc phát triển, kiểm thử và triển khai, toàn bộ mã nguồn được tổ chức theo mô hình monorepo. Kiến trúc này cho phép Backend API, Admin Dashboard, Demo Application và JavaScript SDK cùng sử dụng chung quy trình quản lý phiên bản, phụ thuộc và môi trường phát triển. Đồng thời, việc đặt các thành phần trong cùng một kho mã nguồn giúp giảm chi phí đồng bộ API contract, chia sẻ kiểu dữ liệu và triển khai Docker Compose cho toàn bộ hệ thống [14].

Tổ chức monorepo

Dự án được tổ chức theo mô hình workspace với hai thư mục chính là `apps` và `packages`. Thư mục `apps` chứa các ứng dụng có thể triển khai độc lập, bao gồm Backend API, Admin Dashboard và Demo Application. Thư mục `packages` chứa JavaScript SDK dùng chung để các ứng dụng khách gọi Evaluation API.

Cấu trúc này giúp giảm trùng lặp mã nguồn, dễ dàng chia sẻ các kiểu dữ liệu giữa các thành phần và hỗ trợ mở rộng thêm ứng dụng mới mà không ảnh hưởng đến kiến trúc hiện có. Việc phát triển theo monorepo cũng phù hợp với quy trình tích hợp và triển khai liên tục khi toàn bộ hệ thống được quản lý trong cùng một repository [3, 14].

Cấu trúc Backend API

Backend API được xây dựng bằng NestJS theo kiến trúc module nhằm tách biệt các miền chức năng của hệ thống. Mỗi module đảm nhiệm một nghiệp vụ riêng như quản lý project, environment, feature flag, flag group, rule evaluation, audit log, authentication và thống kê evaluation.

Tầng controller chịu trách nhiệm tiếp nhận và xác thực request, tầng service xử lý nghiệp vụ, còn tầng repository thực hiện truy cập cơ sở dữ liệu thông qua Prisma ORM. Các DTO được sử dụng để kiểm tra dữ liệu đầu vào, trong khi middleware, guard, interceptor và exception filter đảm nhiệm các chức năng cắt ngang như xác thực, phân quyền, ghi nhận request ID, logging và xử lý lỗi thống nhất. Kiến trúc phân lớp này giúp tăng khả năng bảo trì, kiểm thử và mở rộng hệ thống khi bổ sung các loại rule hoặc endpoint mới.

Cấu trúc Admin Dashboard

Admin Dashboard được xây dựng bằng React theo hướng component-based. Giao diện được chia thành các nhóm thành phần độc lập như trang quản lý project, feature flag, flag group, environment, audit log, dashboard thống kê và các màn hình cấu hình rule.

Các thành phần dùng chung như layout, biểu mẫu, bảng dữ liệu, hộp thoại xác nhận, badge trạng thái và hook xử lý API được tách riêng để tái sử dụng trong toàn bộ ứng dụng. Dữ liệu được lấy từ Backend API thông qua các service riêng biệt, giúp tách logic giao diện khỏi logic giao tiếp mạng và đơn giản hóa việc bảo trì khi API thay đổi.

Cấu trúc Demo Application và SDK

Demo Application là một ứng dụng React độc lập dùng để minh họa việc tích hợp Feature Flag vào ứng dụng khách. Thay vì gọi trực tiếp Evaluation API, ứng dụng sử dụng JavaScript SDK được phát triển trong thư mục `packages/js-sdk`. SDK chịu trách nhiệm xây dựng request, gửi Evaluation API, kiểm tra tính hợp lệ của response và trả về kết quả dưới dạng giao diện lập trình thống nhất cho ứng dụng.

Việc tách SDK thành một package riêng giúp giảm sự phụ thuộc giữa ứng dụng khách và Backend API, đồng thời tạo điều kiện tái sử dụng cho nhiều ứng dụng khác nhau. Cách tổ chức này cũng phù hợp với mô hình Feature Flag Platform trong thực tế, nơi client chỉ cần tích hợp SDK mà không cần quan tâm đến chi tiết triển khai của Evaluation Engine [7, 10].

3.1.2 Môi trường phát triển

Để bảo đảm tính ổn định, khả năng tái lập môi trường và thuận tiện trong quá trình phát triển, hệ thống được xây dựng trên nền tảng Node.js với kiến trúc monorepo. Backend API, Admin Dashboard, Demo Application và JavaScript SDK cùng sử dụng chung hệ sinh thái công cụ, đồng thời cơ sở dữ liệu được quản lý thông qua Prisma ORM và PostgreSQL. Ngoài ra, dự án tích hợp các công cụ kiểm thử, kiểm tra chất lượng mã nguồn và tài liệu hóa API nhằm nâng cao khả năng bảo trì và mở rộng hệ thống.

Runtime và package manager

Toàn bộ hệ thống được phát triển bằng TypeScript trên nền tảng Node.js. Backend sử dụng NestJS, trong khi Admin Dashboard và Demo Application được xây dựng bằng React. Các thành phần trong monorepo được quản lý bằng npm workspaces, cho phép chia sẻ package nội bộ, đồng bộ phiên bản phụ thuộc và đơn giản hóa quá trình cài đặt.

Việc sử dụng cùng một runtime giúp giảm sự khác biệt giữa các thành phần của hệ thống, đồng thời tạo điều kiện thuận lợi cho việc xây dựng quy trình build, kiểm thử và triển khai thống nhất. Đây cũng là mô hình thường được áp dụng trong các hệ thống web hiện đại theo quy trình DevOps [3].

Database và công cụ ORM

Hệ thống sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu chính. PostgreSQL đáp ứng tốt các yêu cầu về tính toàn vẹn dữ liệu, hỗ trợ transaction, khóa ngoại, chỉ mục và kiểu dữ liệu JSONB phục vụ lưu trữ linh hoạt các tham số rule, thuộc tính người dùng và snapshot của audit log [15].

Lớp truy cập dữ liệu được xây dựng bằng Prisma ORM. Prisma cung cấp cơ chế mô hình hóa dữ liệu thông qua schema, hỗ trợ migration, sinh mã truy cập cơ sở dữ liệu theo kiểu dữ liệu tĩnh và giảm đáng kể lượng mã SQL thủ công. Việc sử dụng Prisma cũng giúp tăng tính an toàn kiểu dữ liệu, đơn giản hóa thao tác CRUD và thuận tiện khi mở rộng mô hình dữ liệu trong tương lai.

Công cụ kiểm thử và kiểm tra mã nguồn

Dự án sử dụng Jest để xây dựng các bài kiểm thử đơn vị cho các thành phần nghiệp vụ quan trọng, đặc biệt là Rule Evaluation Engine và các hàm tiện ích. Các trường hợp kiểm thử bao gồm đánh giá theo user, role, percentage rollout, kill switch, default value và các tình huống ngoại lệ nhằm bảo đảm tính đúng đắn của thuật toán evaluation.

Bên cạnh đó, ESLint được sử dụng để kiểm tra chất lượng mã nguồn và phát hiện các lỗi phổ biến trong quá trình phát triển. Việc kết hợp TypeScript với ESLint giúp chuẩn hóa phong cách lập trình, hạn chế lỗi tiềm ẩn và nâng cao khả năng bảo trì của toàn bộ hệ thống.

Công cụ tài liệu hóa API

Backend API tích hợp OpenAPI thông qua Swagger để tự động sinh tài liệu REST API. Các endpoint được mô tả đầy đủ về request, response, tham số đầu vào, mã lỗi và kiểu dữ liệu, giúp lập trình viên dễ dàng kiểm thử và tích hợp với các ứng dụng khách.

Việc sử dụng Swagger giúp tài liệu API luôn đồng bộ với mã nguồn, giảm công sức cập nhật tài liệu thủ công và hỗ trợ kiểm thử trực tiếp ngay trên giao diện trình duyệt. Cách tiếp cận này phù hợp với các hệ thống Feature Flag hiện đại, nơi Evaluation API đóng vai trò là điểm tích hợp giữa nền tảng quản trị và các ứng dụng khách [10].

3.1.3 Database migration và seed data

Để bảo đảm cơ sở dữ liệu luôn đồng bộ với mã nguồn và có thể tái tạo trên nhiều môi trường khác nhau, hệ thống sử dụng Prisma Migrate để quản lý schema và lịch sử thay đổi cơ sở dữ liệu. Bên cạnh đó, các tập dữ liệu mẫu được chuẩn bị sẵn nhằm hỗ trợ phát triển, kiểm thử và trình diễn các kịch bản Feature Flag mà không cần nhập dữ liệu thủ công.

Khởi tạo database schema

Toàn bộ mô hình dữ liệu của hệ thống được định nghĩa trong Prisma Schema và sinh tự động thành các bảng trong PostgreSQL thông qua Prisma Migrate. Schema bao gồm các thực thể chính như Project, Environment, Feature Flag, Flag Group, Flag Configuration, Flag Rule, Sample User, Organization, Audit Log và các bảng phục vụ thống kê evaluation.

Các ràng buộc toàn vẹn như khóa chính, khóa ngoại, chỉ mục, giá trị duy nhất và các quan hệ một-nhiều được khai báo trực tiếp trong schema. Ngoài các kiểu dữ liệu cơ bản, PostgreSQL

còn sử dụng kiểu dữ liệu JSONB để lưu trữ linh hoạt các tham số rule, thuộc tính người dùng và snapshot của audit log, giúp hệ thống dễ dàng mở rộng khi bổ sung các loại rule mới [15].

Quản lý migration

Mỗi thay đổi đối với mô hình dữ liệu đều được quản lý dưới dạng migration độc lập. Prisma tự động sinh các tập lệnh SQL tương ứng và lưu lại lịch sử migration trong cơ sở dữ liệu, bảo đảm mọi môi trường phát triển và triển khai đều sử dụng cùng một phiên bản schema.

Cách tiếp cận này giúp kiểm soát sự thay đổi của cơ sở dữ liệu theo thời gian, hỗ trợ cập nhật hoặc khôi phục schema khi cần thiết, đồng thời giảm rủi ro sai khác giữa môi trường phát triển, kiểm thử và production.

Seed project, environment và Feature Flag

Sau khi hoàn tất migration, hệ thống cung cấp các script seed để tạo dữ liệu mẫu phục vụ quá trình phát triển và trình diễn. Dữ liệu khởi tạo bao gồm một hoặc nhiều project, các environment mặc định như `development`, `staging` và `production`, các flag group cùng nhiều feature flag đại diện cho các kịch bản phổ biến.

Mỗi feature flag được cấu hình với trạng thái, serving mode, kill switch và các rule phù hợp để minh họa các chiến lược rollout như bật hoặc tắt toàn cục, targeting theo user, targeting theo role, percentage rollout và group kill switch. Nhờ đó, các chức năng của Evaluation Engine và Dashboard có thể được kiểm thử ngay sau khi khởi tạo dữ liệu.

Seed user và organization mẫu

Hệ thống đồng thời tạo tập dữ liệu người dùng và organization mẫu nhằm phục vụ các kịch bản đánh giá Feature Flag. Mỗi người dùng được gán một `targetingKey` duy nhất cùng với các thuộc tính như vai trò, organization và các metadata cần thiết cho quá trình evaluation.

Việc chuẩn bị sẵn dữ liệu mẫu giúp dễ dàng kiểm thử các trường hợp targeting theo user, role và organization, đồng thời bảo đảm các kết quả percentage rollout luôn ổn định nhờ sử dụng cùng một định danh người dùng trong quá trình đánh giá. Các tập dữ liệu này cũng được sử dụng để minh họa các kịch bản demo và hỗ trợ kiểm thử tự động của hệ thống.

3.1.4 Triển khai bằng Docker Compose

Để đơn giản hóa quá trình cài đặt và bảo đảm tính nhất quán giữa các môi trường phát triển, hệ thống được triển khai bằng Docker Compose. Cách tiếp cận này cho phép khởi tạo đồng thời các thành phần của nền tảng Feature Flag thông qua một cấu hình thống nhất, giảm sự phụ thuộc vào môi trường cài đặt của từng máy và hỗ trợ tái tạo hệ thống một cách nhanh chóng [14].

Các service trong Docker Compose

Docker Compose được sử dụng để quản lý các service chính của hệ thống. Trong phiên bản hiện tại, các service bao gồm cơ sở dữ liệu PostgreSQL và các ứng dụng chạy trong môi trường Node.js như Backend API, Admin Dashboard và Demo Application. Khi cần mở rộng, các service này có thể được bổ sung thêm Redis để cache kết quả evaluation hoặc các thành phần giám sát mà không làm thay đổi kiến trúc tổng thể.

Mỗi service được khai báo độc lập với image, cổng truy cập, biến môi trường, volume và các phụ thuộc cần thiết. Việc quản lý tập trung giúp quá trình khởi động, dừng và cập nhật toàn bộ hệ thống chỉ cần thực hiện bằng một số lệnh Docker Compose thống nhất.

Network giữa các service

Các service được kết nối thông qua mạng nội bộ do Docker Compose tự động tạo. Nhờ đó, Backend API có thể truy cập PostgreSQL thông qua tên service thay vì địa chỉ IP cố định, trong khi Admin Dashboard và Demo Application giao tiếp với Backend API thông qua các endpoint được cấu hình trong biến môi trường.

Việc sử dụng mạng nội bộ giúp cô lập các thành phần của hệ thống với môi trường bên ngoài, giảm sự phụ thuộc vào cấu hình mạng của máy chủ và đơn giản hóa quá trình triển khai trên nhiều thiết bị khác nhau [14].

Volume lưu trữ dữ liệu

Để bảo đảm dữ liệu không bị mất khi container được khởi động lại hoặc cập nhật phiên bản, PostgreSQL sử dụng Docker Volume để lưu trữ dữ liệu lâu dài. Toàn bộ schema, dữ liệu nghiệp vụ, audit log và các thông tin cấu hình Feature Flag được lưu trên volume thay vì bên trong container.

Việc tách dữ liệu khỏi vòng đời của container giúp hệ thống dễ dàng sao lưu, phục hồi và nâng cấp mà không ảnh hưởng đến dữ liệu đã tồn tại. Đây cũng là phương pháp triển khai được khuyến nghị đối với các dịch vụ sử dụng cơ sở dữ liệu [14].

Biến môi trường và cấu hình runtime

Các thông số cấu hình của hệ thống được quản lý thông qua biến môi trường nhằm tách biệt mã nguồn với thông tin triển khai. Các biến quan trọng bao gồm chuỗi kết nối PostgreSQL, cổng dịch vụ, khóa bí mật phục vụ xác thực, cấu hình CORS, địa chỉ Backend API và các tham số runtime khác.

Việc sử dụng biến môi trường giúp cùng một mã nguồn có thể triển khai trên nhiều môi trường như development, staging và production mà không cần chỉnh sửa chương trình. Đồng thời, các thông tin nhạy cảm không được ghi trực tiếp trong mã nguồn, góp phần nâng cao tính bảo mật và khả năng quản lý cấu hình của hệ thống [11, 14].

3.1.5 Quy trình khởi chạy hệ thống

Hệ thống được thiết kế để có thể khởi tạo nhanh trên một môi trường mới thông qua quy trình gồm chuẩn bị môi trường, khởi tạo cơ sở dữ liệu, sinh dữ liệu mẫu và khởi chạy các dịch vụ. Các bước này bảo đảm toàn bộ thành phần của nền tảng Feature Flag được đồng bộ trước khi đưa vào sử dụng hoặc trình diễn.

Khởi tạo môi trường

Bước đầu tiên là cài đặt các công cụ cần thiết như Node.js, npm và Docker Desktop. Sau khi tải mã nguồn, toàn bộ các package trong monorepo được cài đặt thông qua npm workspaces để đồng bộ các phụ thuộc giữa Backend API, Admin Dashboard, Demo Application và JavaScript SDK.

Tiếp theo, các biến môi trường được cấu hình thông qua tệp cấu hình tương ứng. Các tham số như chuỗi kết nối PostgreSQL, cổng dịch vụ, khóa bí mật và địa chỉ Backend API được thiết lập trước khi khởi động hệ thống nhằm bảo đảm các thành phần có thể giao tiếp với nhau một cách chính xác.

Chạy migration và seed

Sau khi cơ sở dữ liệu PostgreSQL được khởi động, Prisma Migrate được sử dụng để tạo hoặc cập nhật schema theo phiên bản mới nhất của hệ thống. Quá trình migration đồng thời thiết lập các bảng dữ liệu, khóa ngoại, chỉ mục và các ràng buộc toàn vẹn cần thiết.

Khi migration hoàn tất, các script seed được thực thi để tạo dữ liệu mẫu phục vụ phát triển và kiểm thử. Dữ liệu khởi tạo bao gồm project, environment, feature flag, flag group, rule, organization và người dùng mẫu, cho phép hệ thống có thể sử dụng ngay sau khi cài đặt mà không cần cấu hình thủ công.

Khởi chạy các service

Các service của hệ thống được khởi động thông qua Docker Compose hoặc chạy trực tiếp trong môi trường phát triển. PostgreSQL được khởi động trước, tiếp theo là Backend API, Admin Dashboard và Demo Application. Trong quá trình khởi động, Backend API tự động kết nối đến cơ sở dữ liệu và sẵn sàng phục vụ các yêu cầu từ Dashboard cũng như Demo Application.

Việc khởi chạy theo thứ tự giúp bảo đảm các dịch vụ phụ thuộc luôn sẵn sàng trước khi ứng dụng bắt đầu xử lý yêu cầu, đồng thời giảm khả năng phát sinh lỗi kết nối trong giai đoạn khởi động [14].

Kiểm tra trạng thái hệ thống

Sau khi toàn bộ service được khởi động, trạng thái hệ thống được kiểm tra bằng cách truy cập giao diện Admin Dashboard, Demo Application và tài liệu Swagger của Backend API. Các endpoint quản lý project, feature flag và evaluation được sử dụng để xác nhận Backend API đã kết nối thành công với cơ sở dữ liệu.

Cuối cùng, một số kịch bản đánh giá Feature Flag như bật hoặc tắt toàn cục, targeting theo role và percentage rollout được thực hiện trên Demo Application nhằm xác nhận Rule Evaluation Engine, Dashboard và cơ sở dữ liệu hoạt động đúng theo thiết kế trước khi tiến hành kiểm thử hoặc trình diễn hệ thống.

3.2 Kết quả xây dựng Backend API

3.2.1 Quản lý project và environment

Nhóm chức năng quản lý project và environment đóng vai trò là nền tảng của toàn bộ hệ thống Feature Flag. Mỗi feature flag được quản lý trong phạm vi một project và được cấu hình riêng cho từng environment, qua đó cho phép tách biệt cấu hình giữa các môi trường như Production hoặc các môi trường mở rộng trong tương lai. Thiết kế này phù hợp với kiến trúc của các nền tảng Feature Flag hiện đại như LaunchDarkly và Unleash, trong đó project và environment là hai thực thể cốt lõi để tổ chức cấu hình và triển khai tính năng [7, 2].

Kết quả triển khai API project

Backend triển khai đầy đủ nhóm REST API quản lý project theo kiến trúc NestJS. Các endpoint bao gồm lấy danh sách project, lấy thông tin chi tiết theo khóa định danh (`projectKey`), tạo project mới, cập nhật thông tin và xóa project. Các thao tác thay đổi dữ liệu được bảo vệ bằng cơ chế phân quyền, trong khi các thao tác truy vấn chỉ yêu cầu quyền đọc tương ứng.

Project được định danh bằng trường `key` duy nhất trong toàn hệ thống thay vì sử dụng khóa chính nội bộ. Cách tiếp cận này giúp API ổn định hơn khi tích hợp với các ứng dụng bên ngoài, đồng thời phù hợp với thông lệ xây dựng Feature Flag Platform [7].

Quá trình tạo project được thực hiện bên trong transaction nhằm đảm bảo tính toàn vẹn dữ liệu. Sau khi project được tạo thành công, hệ thống đồng thời khởi tạo environment mặc định và ghi một bản ghi audit log cho sự kiện tạo project. Nếu bất kỳ thao tác nào thất bại, toàn bộ transaction sẽ được rollback để tránh phát sinh trạng thái không nhất quán.

Kết quả triển khai API environment

Mỗi project quản lý tập các environment độc lập. Trong phạm vi mini project, backend xây dựng lớp repository để quản lý environment, bao gồm các thao tác tìm environment mặc định của project, tìm environment theo khóa định danh và truy vấn toàn bộ environment thuộc một project.

Environment được sử dụng trong quá trình evaluation nhằm xác định bộ cấu hình feature flag tương ứng với từng môi trường triển khai. Thiết kế này cho phép cùng một feature flag có thể sở hữu các cấu hình khác nhau giữa các environment mà không làm thay đổi khóa định danh

của feature flag, phù hợp với mô hình quản lý của các nền tảng Feature Flag hiện nay [7, 2].

Danh sách environment được sắp xếp theo trường `sortOrder` trước, sau đó theo khóa định danh, giúp giao diện quản trị luôn hiển thị các môi trường theo thứ tự xác định.

Environment mặc định

Khi tạo mới một project, hệ thống tự động sinh environment mặc định có khóa định danh là `production`. Environment này được đánh dấu bằng thuộc tính `isDefault = true` và được sử dụng khi Evaluation API không chỉ định environment trong yêu cầu đánh giá.

Việc luôn tồn tại một environment mặc định giúp đơn giản hóa quá trình tích hợp với ứng dụng khách, đồng thời đảm bảo Evaluation Engine luôn có thể xác định được cấu hình cần sử dụng. Cách tổ chức này tương tự mô hình project–environment được áp dụng trong các hệ thống Feature Flag thương mại [7, 2].

Ngoài việc tạo environment mặc định, backend còn lưu thông tin này trong audit log của sự kiện tạo project để phục vụ truy vết lịch sử cấu hình sau này.

Validation và ràng buộc dữ liệu

Backend sử dụng DTO kết hợp thư viện `class-validator` để kiểm tra dữ liệu đầu vào trước khi xử lý nghiệp vụ. Các trường bắt buộc như `key` và `name` đều được kiểm tra kiểu dữ liệu, giới hạn độ dài và định dạng nhằm đảm bảo dữ liệu hợp lệ ngay từ tầng API.

Ở mức cơ sở dữ liệu, bảng `Project` áp dụng ràng buộc duy nhất đối với trường `key`, trong khi bảng `Environment` sử dụng khóa duy nhất kết hợp `(projectId, key)` để bảo đảm không tồn tại hai environment trùng khóa trong cùng một project. Quan hệ giữa project và environment được thiết lập bằng khóa ngoại, đồng thời sử dụng chỉ mục trên các trường thường xuyên truy vấn nhằm tối ưu hiệu năng.

Trước khi tạo project mới, backend kiểm tra sự tồn tại của khóa định danh để phát hiện xung đột và trả về lỗi phù hợp. Các lỗi không tìm thấy tài nguyên, dữ liệu không hợp lệ hoặc vi phạm ràng buộc đều được chuẩn hóa theo một cấu trúc phản hồi thống nhất, giúp client dễ dàng xử lý và truy vết lỗi. Việc kết hợp validation ở tầng ứng dụng với các ràng buộc toàn vẹn ở tầng cơ sở dữ liệu góp phần nâng cao tính đúng đắn và độ tin cậy của hệ thống [11].

3.2.2 Quản lý Feature Flag

Nhóm chức năng quản lý Feature Flag là thành phần trung tâm của Backend API, cho phép tạo mới, cập nhật, lưu trữ và quản lý trạng thái của các feature flag trong từng project. Thay vì xóa trực tiếp khỏi hệ thống, feature flag được quản lý theo vòng đời (lifecycle), giúp bảo toàn lịch sử cấu hình, hỗ trợ audit log và tránh ảnh hưởng đến các ứng dụng đang tích hợp. Cách tiếp cận này phù hợp với khuyến nghị của các nền tảng Feature Flag hiện đại nhằm giảm rủi ro trong quá trình phát hành phần mềm và hạn chế phát sinh technical debt [1, 8].

Tạo và truy vấn Feature Flag

Backend triển khai đầy đủ nhóm REST API phục vụ việc tạo mới và truy vấn feature flag. Khi tạo một feature flag, người dùng cung cấp các thông tin cơ bản như khóa định danh (`key`), tên hiển thị, mô tả và project tương ứng. Sau khi kiểm tra tính hợp lệ của dữ liệu, hệ thống tạo bản ghi feature flag đồng thời khởi tạo cấu hình mặc định cho environment mặc định của project.

Mỗi feature flag được định danh duy nhất bởi cặp `(projectId, key)`, cho phép các project khác nhau sử dụng cùng một khóa mà không xảy ra xung đột. API hỗ trợ truy vấn danh sách feature flag theo project, tìm kiếm theo khóa định danh hoặc tên hiển thị, đồng thời hỗ trợ phân trang nhằm giảm tải cho cơ sở dữ liệu khi số lượng feature flag tăng lên.

Các API truy vấn chỉ trả về những feature flag còn hoạt động theo mặc định, trong khi các feature flag đã archive chỉ được trả về khi người quản trị yêu cầu, giúp giao diện quản trị luôn hiển thị danh sách ngắn gọn và dễ theo dõi.

Cập nhật thông tin flag

Backend cho phép cập nhật các thông tin mô tả của feature flag như tên hiển thị, mô tả hoặc metadata mà không làm thay đổi khóa định danh. Việc giữ nguyên trường `key` giúp các ứng dụng khách tiếp tục sử dụng cùng một identifier trong Evaluation API mà không cần thay đổi mã nguồn.

Mỗi yêu cầu cập nhật đều trải qua quá trình kiểm tra sự tồn tại của project và feature flag trước khi thực hiện. Sau khi cập nhật thành công, hệ thống ghi nhận đầy đủ trạng thái trước và sau thay đổi vào audit log để phục vụ truy vết. Quá trình này được thực hiện trong transaction nhằm đảm bảo dữ liệu luôn nhất quán nếu xảy ra lỗi trong quá trình ghi log.

Archive và restore flag

Thay vì xóa vật lý khỏi cơ sở dữ liệu, backend áp dụng cơ chế archive đối với feature flag. Khi archive, trường `lifecycleStatus` được chuyển từ `ACTIVE` sang `ARCHIVED`. Các feature flag ở trạng thái này sẽ không còn được Evaluation Engine sử dụng và trả về kết quả với lý do tương ứng nếu được yêu cầu đánh giá.

Việc archive thay cho delete giúp bảo toàn toàn bộ lịch sử cấu hình, audit log và các quan hệ dữ liệu liên quan, đồng thời giảm nguy cơ ảnh hưởng đến những ứng dụng vẫn đang tham chiếu tới feature flag đó. Khi cần sử dụng lại, người quản trị có thể thực hiện thao tác restore để chuyển trạng thái trở lại `ACTIVE` mà không cần tạo mới feature flag. Cách quản lý vòng đời này phù hợp với khuyến nghị của các hệ thống Feature Flag trong thực tế [1, 8].

Mỗi thao tác archive hoặc restore đều được ghi nhận vào audit log cùng thông tin actor, thời gian thực hiện và snapshot trước, sau khi thay đổi nhằm đảm bảo khả năng truy vết.

Quản lý cấu hình theo environment

Backend tách riêng thông tin định danh của feature flag và cấu hình triển khai theo từng environment. Mỗi feature flag có thể sở hữu nhiều bản cấu hình (`FlagConfig`), trong đó mỗi cấu hình tương ứng với một environment khác nhau.

Thông qua mô hình này, cùng một feature flag có thể được bật hoàn toàn trên môi trường Production nhưng vẫn ở trạng thái tắt hoặc rollout theo tỷ lệ trên các environment khác mà không cần tạo thêm feature flag mới. Mỗi cấu hình quản lý các thuộc tính như trạng thái kích hoạt, chế độ phục vụ (`GLOBAL_ON` hoặc `TARGETED`), kill switch và tập luật đánh giá tương ứng.

Evaluation Engine luôn xác định environment trước khi lấy cấu hình feature flag. Nếu yêu cầu đánh giá không chỉ định environment, hệ thống tự động sử dụng environment mặc định của project. Thiết kế này giúp Backend API hỗ trợ triển khai nhiều môi trường độc lập, đồng thời vẫn giữ được cùng một khóa định danh feature flag xuyên suốt toàn bộ vòng đời phát triển phần mềm, phù hợp với mô hình của LaunchDarkly và Unleash [7, 2].

3.2.3 Quản lý rule và flag group

Backend API hỗ trợ cấu hình các luật (rule) quyết định trạng thái của feature flag tại thời điểm đánh giá. Mỗi rule được lưu độc lập trong cơ sở dữ liệu, liên kết với một cấu hình feature flag

theo từng environment và được đánh giá theo thứ tự ưu tiên xác định trước. Trong phạm vi mini project, hệ thống hỗ trợ ba loại rule chính gồm user allowlist, role targeting và percentage rollout. Ngoài ra, backend còn triển khai cơ chế flag group kết hợp group kill switch nhằm cho phép vô hiệu hóa đồng thời nhiều feature flag khi cần thiết. Thiết kế này phù hợp với các chiến lược targeting và rollout được áp dụng trong các nền tảng Feature Flag hiện đại [4, 5].

Rule user allowlist

Rule user allowlist cho phép chỉ định danh sách các người dùng được phép sử dụng một tính năng cụ thể. Backend lưu danh sách các targeting key trong phần tham số của rule và thực hiện so sánh trực tiếp với thông tin được gửi trong evaluation context.

Khi evaluation context chứa targeting key trùng với một phần tử trong allowlist, Evaluation Engine ngay lập tức trả về kết quả bật feature flag mà không cần tiếp tục đánh giá các rule có mức ưu tiên thấp hơn. Cơ chế này phù hợp với các kịch bản cấp quyền cho một nhóm người dùng thử nghiệm hoặc xử lý các trường hợp ngoại lệ trong quá trình phát hành tính năng.

Rule role targeting

Rule role targeting được sử dụng để bật feature flag cho những người dùng thuộc một hoặc nhiều vai trò xác định. Backend lưu danh sách role trong tham số của rule và so sánh với tập role được truyền trong evaluation context.

Trong quá trình đánh giá, nếu tồn tại ít nhất một role của người dùng trùng với danh sách role được cấu hình, rule được xem là thỏa mãn và Evaluation Engine trả về kết quả tương ứng. Thiết kế này hỗ trợ các kịch bản phổ biến như chỉ kích hoạt tính năng cho nhóm quản trị viên, beta customer hoặc regular customer mà không cần cấu hình riêng cho từng người dùng [4].

Rule percentage rollout

Rule percentage rollout cho phép phát hành tính năng theo một tỷ lệ phần trăm người dùng xác định. Backend sử dụng thuật toán băm ổn định dựa trên `projectKey`, `flagKey` và `targetingKey` để tạo giá trị bucket cố định cho từng người dùng.

Giá trị bucket được so sánh với tỷ lệ rollout đã cấu hình để quyết định người dùng có được bật feature hay không. Do sử dụng hàm băm ổn định, cùng một người dùng luôn nhận được kết quả nhất quán giữa các lần evaluation khi cấu hình không thay đổi. Khi điều chỉnh tỷ lệ

rollout, chỉ những người dùng nằm trong phần bucket mới được bổ sung hoặc loại bỏ, giúp hạn chế biến động không cần thiết trong quá trình phát hành tính năng [5].

Quản lý độ ưu tiên rule

Mỗi rule được gán một trường `priority` để xác định thứ tự đánh giá. Backend luôn truy vấn và sắp xếp danh sách rule theo độ ưu tiên tăng dần trước khi chuyển cho Evaluation Engine xử lý.

Evaluation Engine áp dụng nguyên tắc first-match-wins, nghĩa là ngay khi một rule thỏa mãn điều kiện, quá trình đánh giá kết thúc và trả về kết quả tương ứng. Cơ chế này giúp quá trình đánh giá có tính xác định, tránh xung đột giữa nhiều rule cùng áp dụng cho một người dùng, đồng thời tạo điều kiện mở rộng thêm các loại rule mới trong tương lai mà không ảnh hưởng đến kiến trúc hiện tại [4].

Group kill switch

Bên cạnh kill switch của từng feature flag, backend còn triển khai cơ chế flag group nhằm quản lý nhiều feature flag theo từng nhóm chức năng. Mỗi feature flag có thể được gán vào một flag group và kế thừa trạng thái group kill switch trong quá trình evaluation.

Khi group kill switch được kích hoạt, toàn bộ feature flag thuộc nhóm tương ứng đều bị vô hiệu hóa mà không cần thay đổi cấu hình của từng feature flag riêng lẻ. Cơ chế này đặc biệt hữu ích khi cần phản ứng nhanh với sự cố trên production hoặc tạm thời vô hiệu hóa một nhóm chức năng có liên quan.

Mọi thao tác tạo, cập nhật, đổi tên hoặc thay đổi trạng thái group kill switch đều được ghi nhận vào audit log nhằm đảm bảo khả năng truy vết và phục vụ công tác quản trị hệ thống. Việc bổ sung group kill switch cũng đáp ứng yêu cầu mở rộng được đề xuất trong mini project, đồng thời phù hợp với thực tiễn triển khai các hệ thống Feature Flag quy mô lớn [1, 8].

3.2.4 Evaluation API

Evaluation API là thành phần quan trọng nhất của Backend API, chịu trách nhiệm xác định trạng thái cuối cùng của một feature flag tại thời điểm chạy dựa trên cấu hình hiện có và evaluation context do ứng dụng khách cung cấp. API được thiết kế theo mô hình đồng bộ (synchronous request–response), trong đó backend truy vấn toàn bộ cấu hình cần thiết, thực

hiện đánh giá theo thứ tự ưu tiên xác định và trả về kết quả thống nhất gồm trạng thái bật/tắt, variant và reason code. Thiết kế này phù hợp với đặc tả Evaluation API của OpenFeature cũng như kiến trúc của các nền tảng Feature Flag hiện đại [10, 6].

Tiếp nhận evaluation context

Evaluation API tiếp nhận yêu cầu đánh giá thông qua phương thức `POST`, trong đó client gửi các thông tin gồm `projectKey`, `flagKey`, `environmentKey` (tùy chọn) và evaluation context.

Evaluation context bao gồm `targetingKey`, `userId`, danh sách `roles` và tập thuộc tính mở rộng (`attributes`). Backend thực hiện validation dữ liệu đầu vào trước khi xử lý nhằm đảm bảo các trường bắt buộc có định dạng hợp lệ và loại bỏ các yêu cầu không hợp lệ ngay tại tầng API.

Việc sử dụng evaluation context giúp Evaluation Engine không phụ thuộc vào mô hình dữ liệu của từng ứng dụng khách, đồng thời tạo điều kiện mở rộng thêm các loại targeting mới trong tương lai mà không cần thay đổi giao diện API [6].

Truy vấn evaluation snapshot

Sau khi hoàn thành bước validation, backend truy vấn toàn bộ dữ liệu cần thiết để đánh giá feature flag và xây dựng một evaluation snapshot thống nhất.

Snapshot bao gồm thông tin project, environment, feature flag, cấu hình của environment tương ứng, trạng thái group kill switch và danh sách rule đã được sắp xếp theo độ ưu tiên. Việc gom toàn bộ dữ liệu cần thiết trước khi bắt đầu đánh giá giúp Evaluation Engine chỉ làm việc trên một tập dữ liệu bất biến trong suốt quá trình xử lý, tránh phát sinh nhiều lần truy vấn cơ sở dữ liệu và giảm nguy cơ xuất hiện trạng thái không nhất quán.

Nếu client không truyền `environmentKey`, backend tự động sử dụng environment mặc định của project để truy vấn snapshot.

Thực thi evaluation precedence

Sau khi có evaluation snapshot, Evaluation Engine tiến hành đánh giá theo thứ tự ưu tiên đã được xác định trong quá trình thiết kế hệ thống.

Trước tiên, engine kiểm tra các trạng thái có mức ưu tiên cao nhất như feature flag đã archive,

cấu hình bị disable, group kill switch hoặc kill switch của chính feature flag. Nếu xuất hiện một trong các điều kiện này, quá trình evaluation kết thúc ngay và trả về kết quả tương ứng.

Nếu feature flag vẫn được phép phục vụ, hệ thống tiếp tục xử lý theo serving mode. Đối với chế độ `GLOBAL_ON`, feature được bật ngay mà không cần đánh giá các rule. Đối với chế độ `TARGETED`, Evaluation Engine lần lượt đánh giá các rule theo thứ tự ưu tiên gồm user allowlist, role targeting và percentage rollout. Engine áp dụng nguyên tắc *first-match-wins*, nghĩa là rule đầu tiên thỏa mãn điều kiện sẽ quyết định kết quả cuối cùng [4, 5].

Nếu không có rule nào phù hợp, hệ thống trả về giá trị mặc định đã cấu hình cho feature flag.

Trả về variant và reason code

Sau khi hoàn thành evaluation, backend trả về một đối tượng phản hồi thống nhất bao gồm khóa project, khóa feature flag, trạng thái bật hoặc tắt, variant, reason code và thông tin rule được áp dụng nếu có.

Reason code phản ánh nguyên nhân trực tiếp dẫn đến kết quả evaluation, chẳng hạn như `GLOBAL_ON`, `USER_ALLOWLIST`, `ROLE_MATCH`, `PERCENTAGE_ROLLOUT`, `DEFAULT_OFF`, `FLAG_DISABLED`, `FLAG_ARCHIVED`, `GROUP_KILL_SWITCH` hoặc `KILL_SWITCH`. Việc trả về reason code giúp client, dashboard và hệ thống audit có thể giải thích rõ vì sao một feature flag được bật hoặc tắt, đồng thời hỗ trợ hiệu quả cho quá trình kiểm thử và xử lý sự cố [10].

Error fallback

Evaluation API được thiết kế theo nguyên tắc fail-safe nhằm đảm bảo hệ thống luôn trả về kết quả xác định ngay cả khi xảy ra lỗi trong quá trình xử lý.

Khi feature flag không tồn tại, environment không hợp lệ hoặc phát sinh lỗi nội bộ trong quá trình evaluation, backend không trả về trạng thái không xác định mà chuyển sang giá trị fallback an toàn. Trong các trường hợp này, API trả về trạng thái tắt cùng reason code phù hợp như `NOT_FOUND` hoặc `ERROR`. Cách tiếp cận này giúp ứng dụng khách luôn nhận được phản hồi có cấu trúc thống nhất, hạn chế nguy cơ kích hoạt nhầm các tính năng chưa sẵn sàng trên môi trường production [10, 11].

3.2.5 Audit log và lịch sử cấu hình

Để đảm bảo khả năng truy vết và kiểm toán toàn bộ quá trình quản trị Feature Flag, backend triển khai cơ chế audit log cho tất cả các thao tác thay đổi dữ liệu (mutation). Mỗi sự kiện đều ghi nhận đầy đủ thông tin về đối tượng bị tác động, hành động thực hiện, người thao tác, thời điểm thực hiện, request identifier và trạng thái dữ liệu trước và sau khi thay đổi. Thiết kế này đáp ứng yêu cầu về accountability và auditability của hệ thống, đồng thời phù hợp với các khuyến nghị của OWASP về ghi nhật ký bảo mật và quản lý thay đổi cấu hình [12, 11].

Ghi audit log cho mutation

Backend tự động tạo bản ghi audit log sau mỗi thao tác mutation như tạo, cập nhật, archive, restore feature flag, thay đổi cấu hình environment, quản lý rule, quản lý flag group và các thao tác quản trị khác.

Mỗi bản ghi chứa các thông tin như loại đối tượng (`targetType`), khóa định danh đối tượng, loại hành động (`action`), actor thực hiện, thời gian thực hiện và request identifier. Việc ghi nhận audit log được thực hiện cùng transaction với thao tác nghiệp vụ nhằm đảm bảo dữ liệu nghiệp vụ và lịch sử thay đổi luôn nhất quán.

Thiết kế này cho phép truy vết đầy đủ mọi thay đổi phát sinh trong hệ thống mà không phụ thuộc vào ứng dụng khách.

Lưu snapshot trước và sau

Đối với mỗi thao tác cập nhật, backend lưu đồng thời hai bản snapshot gồm trạng thái trước khi thay đổi (`before`) và trạng thái sau khi thay đổi (`after`). Hai snapshot được lưu dưới dạng dữ liệu JSON nhằm bảo toàn toàn bộ cấu trúc của đối tượng tại thời điểm ghi nhận.

Việc lưu snapshot giúp người quản trị dễ dàng xác định chính xác những trường dữ liệu đã thay đổi mà không cần so sánh thủ công giữa nhiều phiên bản cấu hình. Đồng thời, đây cũng là cơ sở để hỗ trợ các chức năng mở rộng như so sánh phiên bản hoặc khôi phục cấu hình trong tương lai. Việc sử dụng kiểu dữ liệu JSONB của PostgreSQL giúp lưu trữ linh hoạt các snapshot có cấu trúc thay đổi theo từng loại đối tượng [15].

Truy vấn và lọc audit log

Backend cung cấp API truy vấn audit log phục vụ Dashboard quản trị. Người dùng có thể truy vấn theo nhiều tiêu chí như project, loại đối tượng, loại hành động, actor hoặc khoảng thời gian thực hiện.

Kết quả được sắp xếp theo thời gian giảm dần nhằm ưu tiên hiển thị các thay đổi mới nhất. API đồng thời hỗ trợ phân trang để giảm tải cho cơ sở dữ liệu khi số lượng bản ghi tăng lên theo thời gian.

Nhờ cơ chế lọc linh hoạt, người quản trị có thể nhanh chóng xác định nguyên nhân của các thay đổi cấu hình hoặc truy vết các thao tác phát sinh trong quá trình vận hành hệ thống.

Bảo vệ audit log khỏi sửa và xóa

Audit log được thiết kế theo mô hình append-only, nghĩa là chỉ cho phép thêm bản ghi mới và không cho phép cập nhật hoặc xóa các bản ghi đã tồn tại.

Ở tầng cơ sở dữ liệu, backend sử dụng trigger để ngăn chặn các thao tác `UPDATE` và `DELETE` trên bảng audit log. Mọi yêu cầu sửa hoặc xóa đều bị từ chối nhằm đảm bảo tính toàn vẹn của lịch sử thay đổi.

Cơ chế này giúp ngăn ngừa việc chỉnh sửa trái phép các bản ghi kiểm toán, đồng thời đáp ứng yêu cầu về tính tin cậy và khả năng truy vết của hệ thống theo khuyến nghị của OWASP [12].

Theo dõi version cấu hình

Bên cạnh việc lưu snapshot, backend còn theo dõi sự thay đổi cấu hình thông qua chuỗi các bản ghi audit log được tạo theo trình tự thời gian. Mỗi lần thay đổi cấu hình đều tạo ra một phiên bản logic mới của đối tượng mà không ghi đè lên dữ liệu lịch sử.

Thông qua các snapshot trước và sau cùng thông tin thời gian thực hiện, người quản trị có thể theo dõi toàn bộ quá trình tiến hóa của một feature flag, rule hoặc flag group trong suốt vòng đời hoạt động. Cách tiếp cận này đáp ứng yêu cầu versioning ở mức lịch sử cấu hình, đồng thời tạo nền tảng cho việc mở rộng các chức năng như xem lịch sử thay đổi hoặc phục hồi cấu hình trong các phiên bản tiếp theo của hệ thống [8, 9].

3.2.6 Validation và xử lý lỗi

Để đảm bảo tính đúng đắn và ổn định của Backend API, hệ thống triển khai cơ chế validation dữ liệu đầu vào kết hợp với mô hình xử lý lỗi thống nhất trên toàn bộ ứng dụng. Việc kiểm tra dữ liệu được thực hiện ngay tại tầng API trước khi chuyển đến tầng nghiệp vụ, trong khi mọi ngoại lệ phát sinh đều được chuẩn hóa thành một định dạng phản hồi thống nhất. Thiết kế này giúp giảm thiểu dữ liệu không hợp lệ, nâng cao khả năng tích hợp với ứng dụng khách và phù hợp với các khuyến nghị về bảo mật REST API của OWASP [11].

Validation DTO

Backend sử dụng các lớp Data Transfer Object (DTO) kết hợp với thư viện `class-validator` và `class-transformer` để kiểm tra dữ liệu đầu vào của tất cả các endpoint.

Các trường dữ liệu được kiểm tra về kiểu dữ liệu, độ dài, giá trị bắt buộc, định dạng chuỗi và giới hạn miền giá trị trước khi chuyển tới tầng nghiệp vụ. Đối với các API quản lý Feature Flag, hệ thống kiểm tra tính hợp lệ của các trường như `projectKey`, `flagKey`, `environmentKey`, trạng thái cấu hình, tỷ lệ rollout và các tham số của rule.

Validation được thực hiện thông qua `ValidationPipe` của NestJS, giúp loại bỏ dữ liệu không hợp lệ ngay từ tầng controller, giảm tải cho tầng nghiệp vụ và hạn chế phát sinh lỗi trong quá trình xử lý.

Chuẩn hóa phản hồi lỗi

Backend chuẩn hóa toàn bộ phản hồi lỗi theo một cấu trúc thống nhất để các ứng dụng khách có thể xử lý một cách nhất quán.

Mỗi phản hồi lỗi bao gồm mã lỗi, thông điệp mô tả, danh sách chi tiết lỗi (nếu có) và request identifier phục vụ truy vết. Hệ thống phân biệt rõ các nhóm lỗi như validation error, không tìm thấy tài nguyên, xung đột dữ liệu và lỗi nội bộ máy chủ.

Việc chuẩn hóa cấu trúc phản hồi giúp Dashboard, Demo App và các SDK có thể xử lý lỗi theo cùng một cơ chế mà không cần xây dựng logic riêng cho từng endpoint.

Request ID

Mỗi yêu cầu gửi tới Backend API đều được gắn một request identifier duy nhất trong suốt vòng đời xử lý.

Request ID được truyền xuyên suốt giữa middleware, controller, service và audit log, đồng thời xuất hiện trong phản hồi lỗi khi có ngoại lệ xảy ra. Nhờ đó, người quản trị có thể nhanh chóng đối chiếu giữa yêu cầu từ phía client với bản ghi audit log hoặc log hệ thống để xác định nguyên nhân của sự cố.

Việc bổ sung request ID giúp nâng cao khả năng quan sát (observability), hỗ trợ hiệu quả cho quá trình giám sát và gỡ lỗi trong môi trường vận hành.

Xử lý lỗi toàn cục

Backend triển khai Global Exception Filter để xử lý tập trung mọi ngoại lệ phát sinh trong quá trình thực thi API.

Các ngoại lệ nghiệp vụ, lỗi validation và các lỗi không mong muốn đều được chuyển đổi thành cấu trúc phản hồi thống nhất trước khi gửi về client. Đối với các lỗi nội bộ, hệ thống không trả về thông tin nhạy cảm như stack trace hoặc chi tiết cài đặt của máy chủ, qua đó giảm nguy cơ rò rỉ thông tin hệ thống.

Mô hình xử lý lỗi tập trung giúp toàn bộ Backend API duy trì cách phản hồi nhất quán, giảm trùng lặp mã nguồn giữa các controller và đáp ứng các khuyến nghị về bảo mật và thiết kế REST API của OWASP [11].

3.2.7 Phân quyền người dùng quản trị

Để đảm bảo an toàn cho các thao tác quản trị và đáp ứng yêu cầu mở rộng của mini project, Backend API triển khai cơ chế phân quyền theo vai trò (Role-Based Access Control – RBAC). Mỗi yêu cầu quản trị đều được xác định danh tính của người thực hiện và kiểm tra quyền trước khi xử lý nghiệp vụ. Thiết kế này giúp giới hạn phạm vi thao tác của từng nhóm người dùng, giảm nguy cơ thay đổi cấu hình ngoài ý muốn và phù hợp với các khuyến nghị về kiểm soát truy cập của OWASP [11].

Mô hình role

Hệ thống định nghĩa ba vai trò quản trị gồm `ADMIN`, `DEVELOPER` và `VIEWER`. Mỗi vai trò được gán một tập quyền riêng nhằm phục vụ các nhu cầu quản trị khác nhau.

Người dùng có vai trò `ADMIN` được phép thực hiện toàn bộ chức năng của hệ thống, bao gồm quản lý project, environment, feature flag, rule, flag group và audit log. Vai trò `DEVELOPER` được phép quản lý cấu hình feature flag nhưng bị giới hạn đối với một số thao tác quản trị hệ thống. Trong khi đó, `VIEWER` chỉ có quyền truy cập dữ liệu để theo dõi trạng thái và lịch sử thay đổi mà không được phép thực hiện các thao tác làm thay đổi dữ liệu.

Việc tách biệt quyền theo vai trò giúp Dashboard phản ánh đúng trách nhiệm của từng nhóm người dùng và tạo nền tảng để mở rộng các chính sách phân quyền chi tiết hơn trong tương lai.

Kiểm tra quyền mutation

Các API thực hiện thao tác mutation như tạo, cập nhật, archive, restore feature flag, quản lý rule, thay đổi group kill switch hoặc cấu hình environment đều được bảo vệ bằng cơ chế kiểm tra quyền trước khi thực hiện nghiệp vụ.

Backend sử dụng guard kết hợp thông tin actor trong request để xác định vai trò của người dùng. Chỉ khi người dùng sở hữu quyền phù hợp, yêu cầu mới được chuyển đến tầng service để xử lý. Nếu không đáp ứng điều kiện phân quyền, yêu cầu bị từ chối ngay tại tầng bảo vệ mà không truy cập vào logic nghiệp vụ hoặc cơ sở dữ liệu.

Việc kiểm tra quyền tập trung giúp toàn bộ các endpoint mutation áp dụng cùng một chính sách bảo mật và giảm nguy cơ bỏ sót quyền kiểm tra tại từng controller.

Kiểm tra quyền truy vấn

Đối với các API chỉ đọc dữ liệu như truy vấn project, feature flag, environment hoặc audit log, backend vẫn thực hiện kiểm tra quyền truy cập trước khi trả về dữ liệu.

Những người dùng có vai trò `VIEWER`, `DEVELOPER` và `ADMIN` đều được phép sử dụng các endpoint truy vấn trong phạm vi quyền được cấp. Việc phân tách rõ quyền đọc và quyền ghi giúp hạn chế các thao tác thay đổi ngoài ý muốn, đồng thời vẫn đảm bảo các nhóm người dùng có thể theo dõi trạng thái hệ thống và lịch sử cấu hình khi cần thiết.

Cách tiếp cận này tuân theo nguyên tắc *least privilege*, trong đó mỗi người dùng chỉ được cấp những quyền cần thiết để thực hiện công việc của mình [11].

Xử lý truy cập không hợp lệ

Khi người dùng chưa được xác thực hoặc không có quyền thực hiện thao tác yêu cầu, backend từ chối truy cập và trả về mã trạng thái HTTP phù hợp cùng cấu trúc phản hồi lỗi thống nhất.

Các phản hồi lỗi không tiết lộ thông tin nội bộ của hệ thống hoặc chi tiết triển khai cơ chế phân quyền nhằm hạn chế nguy cơ khai thác từ phía đối tượng tấn công. Đồng thời, các yêu cầu bị từ chối vẫn được gắn request ID để phục vụ quá trình truy vết và hỗ trợ phân tích sự cố khi cần thiết.

Việc kết hợp xác thực actor, kiểm tra quyền tập trung và chuẩn hóa phản hồi lỗi giúp Backend API duy trì tính nhất quán trong toàn bộ cơ chế kiểm soát truy cập, đồng thời đáp ứng các yêu cầu bảo mật đối với hệ thống quản trị Feature Flag [11].

3.2.8 Cache và evaluation metrics

Để giảm độ trễ của Evaluation API và hỗ trợ theo dõi quá trình vận hành hệ thống, backend triển khai cơ chế cache cho evaluation snapshot kết hợp với việc ghi nhận thống kê số lần evaluation của từng feature flag. Việc sử dụng cache giúp hạn chế truy vấn lặp lại tới cơ sở dữ liệu đối với các cấu hình ít thay đổi, trong khi các evaluation metrics cung cấp dữ liệu phục vụ Dashboard quản trị và đánh giá mức độ sử dụng của từng feature flag. Thiết kế này phù hợp với các khuyến nghị về caching trong hệ thống Feature Flag hiện đại [8, 13].

Cơ chế caching

Backend triển khai lớp cache tại tầng Evaluation Repository nhằm lưu trữ evaluation snapshot đã được xây dựng từ cơ sở dữ liệu. Mỗi snapshot bao gồm thông tin project, environment, feature flag, cấu hình của environment tương ứng, trạng thái group kill switch và danh sách rule đã được sắp xếp theo thứ tự ưu tiên.

Khi Evaluation API nhận yêu cầu, hệ thống trước tiên kiểm tra cache. Nếu snapshot đã tồn tại và còn hiệu lực, backend sử dụng trực tiếp dữ liệu trong cache mà không cần truy vấn lại cơ sở dữ liệu. Trong trường hợp cache không chứa dữ liệu hoặc dữ liệu đã bị loại bỏ, backend xây dựng lại snapshot từ cơ sở dữ liệu và cập nhật cache cho các lần evaluation tiếp theo.

Việc cache ở mức evaluation snapshot giúp giảm đáng kể số lượng truy vấn cơ sở dữ liệu trong khi vẫn đảm bảo Evaluation Engine luôn làm việc trên một tập dữ liệu đầy đủ và nhất quán.

Cache invalidation

Để đảm bảo dữ liệu cache luôn phản ánh đúng trạng thái hiện tại của hệ thống, backend thực hiện cache invalidation ngay sau mỗi thao tác mutation làm thay đổi cấu hình feature flag.

Các thao tác như cập nhật project, environment, feature flag, flag configuration, rule hoặc group kill switch đều kích hoạt việc xóa các snapshot liên quan khỏi cache. Lần evaluation tiếp theo sẽ tự động xây dựng lại snapshot mới từ cơ sở dữ liệu trước khi lưu trở lại cache.

Chiến lược invalidation theo sự kiện giúp tránh sử dụng cấu hình lỗi thời, đồng thời vẫn tận dụng được lợi ích về hiệu năng của cơ chế cache [13].

Ghi nhận số lần evaluation

Sau mỗi lần Evaluation Engine hoàn thành việc đánh giá feature flag, backend thực hiện ghi nhận thống kê số lần evaluation theo cơ chế best-effort mà không ảnh hưởng đến kết quả trả về cho ứng dụng khách.

Thông tin thống kê được tổng hợp theo từng feature flag và environment, đồng thời lưu thêm thời điểm cập nhật gần nhất để phục vụ quá trình phân tích. Việc ghi nhận metrics được tách khỏi luồng xử lý chính nhằm đảm bảo Evaluation API vẫn duy trì độ trễ thấp ngay cả khi việc cập nhật thống kê gặp sự cố.

Dữ liệu thống kê giúp người quản trị xác định các feature flag được sử dụng nhiều nhất, theo dõi mức độ khai thác của từng tính năng và hỗ trợ đánh giá hiệu quả của các chiến lược rollout.

Truy vấn dữ liệu thống kê

Backend cung cấp API phục vụ Dashboard truy vấn dữ liệu thống kê evaluation theo từng feature flag. Kết quả trả về bao gồm tổng số lần evaluation và các thông tin tổng hợp cần thiết để trực quan hóa dưới dạng biểu đồ.

Các API thống kê chỉ thực hiện truy vấn dữ liệu tổng hợp thay vì phân tích trực tiếp audit log, giúp giảm chi phí xử lý và cải thiện thời gian phản hồi. Nhờ đó, Dashboard có thể hiển thị nhanh các chỉ số sử dụng của feature flag, hỗ trợ người quản trị theo dõi hiệu quả vận hành và

phát hiện các tính năng ít được sử dụng để phục vụ quá trình tối ưu hoặc dọn dẹp technical debt [9, 8].

3.3 Kết quả xây dựng Frontend Dashboard

3.3.1 Màn hình quản lý project

Màn hình quản lý project là điểm truy cập đầu tiên của người quản trị sau khi đăng nhập vào hệ thống. Giao diện này cho phép tạo, cập nhật và lựa chọn project cần quản lý trước khi thực hiện các thao tác cấu hình Feature Flag. Thiết kế theo mô hình phân cấp giúp cô lập cấu hình giữa các dự án, đồng thời tạo nền tảng để mở rộng nhiều project và nhiều môi trường triển khai trong tương lai [7, 2].

Danh sách project

Danh sách project được hiển thị dưới dạng các thẻ (card), mỗi thẻ thể hiện các thông tin cơ bản gồm tên project, project key, số lượng environment và thời gian cập nhật gần nhất. Việc sử dụng card thay vì bảng dữ liệu giúp người dùng nhanh chóng nhận biết từng project và phù hợp với số lượng project không quá lớn trong phạm vi mini project.

Người dùng có thể quan sát toàn bộ project hiện có, lựa chọn project cần thao tác hoặc thực hiện các hành động tạo và chỉnh sửa ngay trên giao diện. Mỗi project được định danh bằng một *project key* duy nhất nhằm phục vụ cho Evaluation API khi ứng dụng khách gửi yêu cầu đánh giá Feature Flag.

Tạo và chỉnh sửa project

Dashboard cung cấp biểu mẫu để tạo mới hoặc cập nhật thông tin project. Các trường dữ liệu bao gồm tên project, project key và mô tả ngắn. Dữ liệu đầu vào được kiểm tra trước khi gửi tới Backend API nhằm đảm bảo các trường bắt buộc được nhập đầy đủ và project key tuân thủ định dạng quy định.

Khi người dùng lưu thay đổi, giao diện gửi yêu cầu tới Management API và cập nhật lại danh sách project nếu thao tác thành công. Trường hợp xảy ra lỗi validation hoặc trùng project key, hệ thống hiển thị thông báo lỗi tương ứng để người dùng điều chỉnh. Thiết kế này giúp giảm thao tác sai và duy trì tính nhất quán của dữ liệu quản lý.

Truy cập chi tiết project

Mỗi project trong danh sách đều có thể được mở để chuyển sang màn hình quản trị chi tiết. Sau khi lựa chọn project, Dashboard tải toàn bộ dữ liệu liên quan như Feature Flag, Flag Group, Environment, Audit Log và các thông tin thống kê tương ứng.

Việc tổ chức giao diện theo từng project giúp toàn bộ cấu hình được cô lập giữa các ứng dụng khác nhau. Đây cũng là mô hình được sử dụng phổ biến trong các nền tảng Feature Flag nhằm cho phép nhiều dự án cùng sử dụng chung một hệ thống quản lý nhưng vẫn đảm bảo tính độc lập về cấu hình [7, 2].

Hiển thị environment

Trong mỗi project, Dashboard hiển thị danh sách các environment đã được cấu hình, bao gồm tối thiểu *Staging* và *Production*. Environment đang được lựa chọn được hiển thị nổi bật để người dùng nhận biết phạm vi cấu hình hiện tại.

Việc tách cấu hình theo environment cho phép cùng một Feature Flag có các trạng thái hoặc rule khác nhau giữa môi trường kiểm thử và môi trường triển khai thực tế. Nhờ đó nhóm phát triển có thể kiểm thử trên Staging trước khi áp dụng cấu hình tương tự cho Production mà không cần triển khai lại ứng dụng, phù hợp với nguyên tắc tách biệt giữa deploy và release của Feature Flag [3, 7].

3.3.2 Màn hình quản lý Feature Flag

Màn hình quản lý Feature Flag là thành phần trung tâm của Dashboard, cho phép người quản trị theo dõi toàn bộ các Feature Flag thuộc một project và thực hiện các thao tác quản lý trong suốt vòng đời của chúng. Giao diện được thiết kế nhằm giúp người dùng nhanh chóng nhận biết trạng thái hiện tại của từng flag, tìm kiếm theo nhiều tiêu chí và thực hiện các thao tác như archive hoặc restore mà không ảnh hưởng đến dữ liệu lịch sử. Thiết kế này phù hợp với các nguyên tắc quản lý Feature Flag hiện đại và hạn chế phát sinh *feature flag debt* thông qua việc quản lý vòng đời của flag [7, 8].

Danh sách Feature Flag

Danh sách Feature Flag được trình bày dưới dạng bảng dữ liệu, trong đó mỗi dòng tương ứng với một Feature Flag. Các cột thông tin bao gồm tên flag, flag key, trạng thái lifecycle, trạng

thái runtime, environment đang áp dụng, nhóm flag và thời điểm cập nhật gần nhất.

Bảng dữ liệu cho phép người quản trị quan sát đồng thời nhiều Feature Flag và thực hiện nhanh các thao tác như xem chi tiết, chỉnh sửa cấu hình hoặc chuyển sang màn hình cấu hình rule. Việc hiển thị đầy đủ các thuộc tính quan trọng giúp giảm số lần chuyển màn hình và hỗ trợ quá trình quản trị hiệu quả hơn.

Tìm kiếm và lọc flag

Dashboard cung cấp chức năng tìm kiếm theo tên hoặc flag key nhằm hỗ trợ người dùng nhanh chóng xác định Feature Flag cần quản lý khi số lượng flag tăng lên.

Bên cạnh tìm kiếm, giao diện còn hỗ trợ lọc theo các tiêu chí như environment, lifecycle status, runtime status hoặc nhóm Feature Flag. Các điều kiện lọc có thể được kết hợp để thu hẹp phạm vi kết quả, giúp người quản trị dễ dàng xác định các Feature Flag đang hoạt động, đã archive hoặc thuộc một nhóm chức năng cụ thể.

Việc tìm kiếm và lọc trực tiếp trên Dashboard giúp giảm thời gian thao tác và phù hợp với mô hình quản lý số lượng lớn Feature Flag trong các hệ thống thực tế [7].

Hiển thị trạng thái runtime

Mỗi Feature Flag đều hiển thị trạng thái runtime dưới dạng nhãn trực quan nhằm phản ánh khả năng phục vụ của flag tại thời điểm hiện tại. Trạng thái này được tổng hợp từ lifecycle status, trạng thái cấu hình, serving mode và kill switch của environment tương ứng.

Thông qua các nhãn hiển thị, người quản trị có thể nhanh chóng phân biệt các trường hợp như flag đang hoạt động, đã bị vô hiệu hóa, đang phục vụ theo chế độ Global On, đang áp dụng Targeted Rollout hoặc đã bị Kill Switch ngắt hoàn toàn. Việc hiển thị rõ trạng thái runtime giúp giảm nguy cơ cấu hình sai và hỗ trợ quá trình kiểm tra trước khi phát hành tính năng.

Archive và restore flag

Thay vì xóa vật lý khỏi cơ sở dữ liệu, Dashboard hỗ trợ chuyển Feature Flag sang trạng thái *Archived*. Khi archive, flag sẽ không còn được sử dụng trong quá trình evaluation nhưng toàn bộ cấu hình, rule và lịch sử audit vẫn được bảo toàn.

Trong trường hợp cần sử dụng lại, người quản trị có thể thực hiện thao tác *Restore* để đưa

Feature Flag trở về trạng thái hoạt động mà không cần tạo mới. Cách tiếp cận này giúp bảo toàn lịch sử thay đổi, tránh mất dữ liệu và phù hợp với khuyến nghị quản lý vòng đời Feature Flag của các nền tảng hiện đại [1, 8].

3.3.3 Màn hình tạo và chỉnh sửa flag

Màn hình tạo và chỉnh sửa Feature Flag cho phép người quản trị khởi tạo mới hoặc cập nhật thông tin của một Feature Flag trong project. Giao diện được thiết kế dưới dạng biểu mẫu, trong đó các trường dữ liệu được nhóm theo chức năng nhằm giảm thao tác cấu hình và hạn chế sai sót. Toàn bộ thay đổi được gửi tới Management API thông qua các yêu cầu REST và chỉ được lưu khi vượt qua các bước kiểm tra hợp lệ ở cả phía Frontend và Backend [7, 11].

Nhập thông tin định danh

Biểu mẫu cho phép người dùng nhập các thông tin định danh cơ bản của Feature Flag, bao gồm tên hiển thị, flag key, mô tả và nhóm Feature Flag nếu có. Trong đó, flag key đóng vai trò là định danh duy nhất để Demo App và Evaluation API truy cập tới Feature Flag tương ứng.

Các trường thông tin được bố trí theo thứ tự từ định danh đến mô tả nhằm phù hợp với quy trình tạo mới. Sau khi hoàn tất, người dùng có thể lưu cấu hình hoặc hủy thao tác để quay lại màn hình danh sách Feature Flag.

Chọn trạng thái và serving mode

Dashboard cho phép cấu hình các thuộc tính điều khiển chính của Feature Flag, bao gồm trạng thái kích hoạt và chế độ phục vụ (*Serving Mode*). Trạng thái kích hoạt xác định Feature Flag có được phép tham gia quá trình evaluation hay không, trong khi Serving Mode quyết định cơ chế đánh giá được áp dụng.

Hệ thống hỗ trợ hai chế độ phục vụ là *Global On* và *Targeted*. Với Global On, Feature Flag được bật cho toàn bộ người dùng của environment hiện tại. Với Targeted, kết quả evaluation được xác định dựa trên các rule đã cấu hình như user targeting, role targeting hoặc percentage rollout. Cách phân tách này giúp người quản trị chuyển đổi nhanh giữa phát hành toàn cục và phát hành có kiểm soát mà không cần thay đổi mã nguồn ứng dụng [7, 4].

Validation biểu mẫu

Trước khi gửi dữ liệu tới Backend API, Dashboard thực hiện kiểm tra hợp lệ đối với các trường đầu vào. Các trường bắt buộc phải được nhập đầy đủ, trong khi flag key phải tuân thủ định dạng quy định và không chứa các ký tự không hợp lệ.

Ngoài các kiểm tra phía Frontend nhằm nâng cao trải nghiệm người dùng, toàn bộ dữ liệu vẫn được Backend xác thực lại trước khi ghi vào cơ sở dữ liệu. Cơ chế kiểm tra hai lớp giúp giảm số lượng yêu cầu không hợp lệ, đồng thời bảo đảm tính toàn vẹn của dữ liệu ngay cả khi yêu cầu được gửi trực tiếp tới API [11].

Hiển thị lỗi từ Backend API

Trong trường hợp Backend API trả về lỗi, Dashboard hiển thị thông báo tương ứng ngay trên giao diện biểu mẫu. Các lỗi phổ biến bao gồm dữ liệu không hợp lệ, trùng flag key, không tìm thấy project hoặc các lỗi xử lý nội bộ của hệ thống.

Thông tin lỗi được hiển thị gần trường dữ liệu liên quan hoặc dưới dạng thông báo tổng quát tùy theo loại lỗi. Cách trình bày này giúp người dùng nhanh chóng xác định nguyên nhân và thực hiện chỉnh sửa mà không cần tải lại trang. Việc phản hồi lỗi rõ ràng cũng phù hợp với mô hình API thống nhất được áp dụng trong toàn bộ hệ thống [11].

3.3.4 Màn hình cấu hình rule

Màn hình cấu hình rule cho phép người quản trị xác định tập người dùng được phép sử dụng một Feature Flag khi Serving Mode được đặt ở chế độ *Targeted*. Dashboard hỗ trợ trực quan hóa từng loại rule, chỉnh sửa tham số tương ứng và thay đổi thứ tự ưu tiên giữa các rule. Cách tổ chức này giúp quá trình cấu hình phù hợp với cơ chế đánh giá tuần tự của Rule Evaluation Engine, đồng thời phản ánh rõ phạm vi ảnh hưởng của từng chiến lược rollout [4, 5].

Cấu hình user allowlist

Dashboard hỗ trợ tạo các rule thuộc loại *User Allowlist*, cho phép chỉ định trực tiếp các người dùng sẽ được bật Feature Flag. Người quản trị chỉ cần lựa chọn hoặc nhập các *targeting key* tương ứng với những người dùng cần cấp quyền truy cập.

User Allowlist thường được sử dụng trong các tình huống kiểm thử nội bộ, triển khai cho khách hàng chiến lược hoặc xác minh tính năng trước khi mở rộng phạm vi phát hành. Do chỉ áp

dụng cho một tập nhỏ người dùng, đây là loại rule có mức ưu tiên cao nhằm đảm bảo các đối tượng được chỉ định luôn nhận được kết quả mong muốn [4].

Cấu hình role targeting

Dashboard cho phép cấu hình Feature Flag dựa trên vai trò của người dùng thông qua các rule *Role Targeting*. Người quản trị lựa chọn một hoặc nhiều role đã được định nghĩa trong hệ thống, chẳng hạn như *Admin*, *Beta Customer* hoặc *Regular Customer*.

Trong quá trình evaluation, nếu vai trò của người dùng khớp với một trong các role được cấu hình, Feature Flag sẽ được bật theo kết quả của rule đó. Cách tiếp cận này phù hợp với các kịch bản phát hành theo nhóm người dùng hoặc phân quyền sử dụng tính năng mà không cần liệt kê từng người dùng riêng lẻ [4].

Cấu hình percentage rollout

Đối với các trường hợp cần phát hành dần tính năng, Dashboard hỗ trợ cấu hình *Percentage Rollout*. Người quản trị chỉ định tỷ lệ phần trăm người dùng sẽ được bật Feature Flag thông qua thanh trượt hoặc trường nhập giá trị.

Hệ thống sử dụng thuật toán băm ổn định để ánh xạ mỗi *targeting key* vào một bucket cố định. Nhờ đó, cùng một người dùng luôn nhận được kết quả nhất quán giữa nhiều lần evaluation, ngay cả khi hệ thống được khởi động lại. Khi thay đổi tỷ lệ rollout, chỉ những người dùng thuộc phần bucket mới được bổ sung hoặc loại bỏ khỏi tập được phục vụ, giúp giảm sự biến động trong trải nghiệm người dùng [5].

Thay đổi độ ưu tiên rule

Dashboard cho phép thay đổi thứ tự ưu tiên của các rule thông qua các thao tác trực quan. Sau khi điều chỉnh, hệ thống cập nhật lại giá trị ưu tiên và lưu cấu hình mới vào Backend API.

Thứ tự ưu tiên quyết định trình tự đánh giá của Rule Evaluation Engine. Các rule có mức ưu tiên cao hơn sẽ được kiểm tra trước và quá trình evaluation sẽ kết thúc ngay khi tìm thấy rule đầu tiên thỏa mãn điều kiện. Cơ chế này giúp hành vi của hệ thống luôn xác định và dễ dự đoán khi tồn tại nhiều rule đồng thời [4].

Hiển thị tác động của rule

Sau khi cấu hình hoặc chỉnh sửa rule, Dashboard hiển thị các thông tin giúp người quản trị nhận biết phạm vi tác động của từng rule, bao gồm loại rule, điều kiện áp dụng, mức ưu tiên và trạng thái hiện tại.

Việc trực quan hóa cấu hình giúp người dùng dễ dàng kiểm tra lại logic rollout trước khi lưu thay đổi, đồng thời giảm nguy cơ cấu hình sai dẫn đến phát hành ngoài mong muốn. Thiết kế này cũng hỗ trợ quá trình quản trị và bảo trì Feature Flag khi số lượng rule tăng lên theo thời gian [8, 4].

3.3.5 Màn hình quản lý flag group

Màn hình quản lý Flag Group được xây dựng nhằm hỗ trợ quản lý nhiều Feature Flag có cùng chức năng hoặc cùng phạm vi nghiệp vụ. Thay vì cấu hình từng Feature Flag riêng lẻ, người quản trị có thể tổ chức chúng thành các nhóm và thực hiện các thao tác tập trung như bật hoặc tắt nhanh toàn bộ nhóm thông qua Group Kill Switch. Cách tiếp cận này giúp giảm số lượng thao tác quản trị và phù hợp với các hệ thống có số lượng Feature Flag lớn [1, 8].

Danh sách flag group

Dashboard hiển thị danh sách toàn bộ Flag Group thuộc project hiện tại dưới dạng các thẻ thông tin. Mỗi nhóm thể hiện tên nhóm, mô tả, số lượng Feature Flag đang quản lý và trạng thái của Group Kill Switch.

Từ màn hình này, người quản trị có thể tạo mới, đổi tên hoặc lựa chọn một Flag Group để xem danh sách các Feature Flag thuộc nhóm. Việc tổng hợp thông tin theo từng nhóm giúp dễ dàng quản lý các tính năng liên quan đến cùng một nghiệp vụ hoặc cùng một đợt phát hành.

Quản lý thành viên của group

Dashboard hỗ trợ thêm hoặc loại bỏ Feature Flag khỏi từng Flag Group thông qua giao diện trực quan. Khi cập nhật thành viên, hệ thống đồng thời hiển thị danh sách các Feature Flag hiện có để người quản trị dễ dàng lựa chọn.

Một Feature Flag chỉ thuộc về một Flag Group trong phạm vi project, nhờ đó cấu trúc quản lý luôn rõ ràng và tránh phát sinh xung đột khi áp dụng Group Kill Switch. Sau khi lưu thay

đổi, danh sách thành viên được cập nhật ngay trên giao diện để phản ánh trạng thái mới của nhóm.

Kích hoạt group kill switch

Đối với mỗi Flag Group, Dashboard cung cấp công tắc bật hoặc tắt Group Kill Switch. Khi Kill Switch được kích hoạt, toàn bộ Feature Flag thuộc nhóm sẽ bị vô hiệu hóa trong quá trình evaluation mà không cần chỉnh sửa từng Feature Flag riêng lẻ.

Cơ chế này đặc biệt hữu ích khi cần xử lý nhanh sự cố trên production. Thay vì thực hiện nhiều thao tác cấu hình độc lập, người quản trị chỉ cần kích hoạt Group Kill Switch để ngắt đồng thời toàn bộ các tính năng liên quan, từ đó rút ngắn thời gian phản ứng khi xảy ra lỗi [1, 8].

Xác nhận thao tác nguy hiểm

Các thao tác có khả năng ảnh hưởng tới nhiều Feature Flag như kích hoạt hoặc hủy Group Kill Switch đều yêu cầu người dùng xác nhận trước khi thực hiện. Dashboard hiển thị hộp thoại xác nhận kèm theo thông tin về Flag Group và số lượng Feature Flag sẽ bị tác động.

Sau khi người dùng xác nhận, yêu cầu mới được gửi tới Backend API để cập nhật cấu hình. Thiết kế này giúp hạn chế các thao tác nhầm lẫn, giảm nguy cơ vô hiệu hóa ngoài ý muốn nhiều Feature Flag cùng lúc và góp phần bảo đảm an toàn trong quá trình vận hành hệ thống.

3.3.6 Màn hình audit log

Màn hình Audit Log cho phép theo dõi toàn bộ các thay đổi cấu hình được thực hiện trên hệ thống quản lý Feature Flag. Mỗi thao tác quản trị như tạo, cập nhật, archive, restore Feature Flag, thay đổi rule hoặc cấu hình Group Kill Switch đều được ghi nhận dưới dạng một bản ghi bất biến. Giao diện này hỗ trợ truy vết lịch sử thay đổi, xác định nguyên nhân của các sự cố cấu hình và tăng khả năng kiểm toán trong quá trình vận hành hệ thống [12, 8].

Danh sách thay đổi

Dashboard hiển thị Audit Log dưới dạng bảng dữ liệu theo thứ tự thời gian giảm dần, giúp người quản trị nhanh chóng quan sát các thay đổi mới nhất. Mỗi bản ghi bao gồm thời điểm thực hiện, actor, loại đối tượng bị tác động, hành động thực hiện và request ID tương ứng.

Các hành động được hiển thị dưới dạng nhãn trực quan như tạo mới, cập nhật, archive, restore

hoặc thay đổi cấu hình rule. Nhờ đó người dùng có thể nhanh chóng nhận biết lịch sử hoạt động của hệ thống mà không cần truy cập trực tiếp vào cơ sở dữ liệu.

Lọc theo actor và action

Khi số lượng Audit Log tăng lên, Dashboard hỗ trợ lọc dữ liệu theo nhiều tiêu chí nhằm rút ngắn thời gian tìm kiếm. Người quản trị có thể lọc theo actor thực hiện thao tác hoặc theo loại hành động cần kiểm tra.

Việc kết hợp nhiều điều kiện lọc giúp nhanh chóng xác định các thay đổi liên quan đến một người dùng hoặc một loại thao tác cụ thể, từ đó hỗ trợ quá trình điều tra sự cố và kiểm tra lịch sử cấu hình trong các hệ thống có nhiều người quản trị [12].

Hiển thị before và after snapshot

Đối với các thao tác cập nhật cấu hình, Dashboard hiển thị đồng thời trạng thái trước và sau khi thay đổi thông qua hai bản snapshot. Các trường dữ liệu được trình bày song song để người quản trị dễ dàng so sánh và xác định chính xác nội dung đã được chỉnh sửa.

Việc lưu trữ snapshot trước và sau thay đổi giúp hỗ trợ truy vết, kiểm tra lịch sử cấu hình và phục vụ quá trình phân tích khi xảy ra sự cố. Đây cũng là một trong những thông tin quan trọng của hệ thống Audit Log nhằm bảo đảm khả năng kiểm toán và minh bạch trong quản trị [12].

Truy vết theo request ID

Mỗi yêu cầu gửi tới Management API đều được gắn một request ID duy nhất và được lưu cùng bản ghi Audit Log. Dashboard hiển thị request ID để người quản trị có thể liên kết nhiều thao tác phát sinh từ cùng một yêu cầu hoặc đối chiếu với log của Backend API.

Cơ chế truy vết theo request ID giúp rút ngắn thời gian xác định nguyên nhân khi xảy ra lỗi, hỗ trợ quá trình gỡ lỗi giữa Frontend và Backend, đồng thời nâng cao khả năng quan sát và vận hành hệ thống theo các khuyến nghị về ghi log và truy vết trong các ứng dụng web hiện đại [12, 11].

3.3.7 Màn hình thống kê evaluation

Màn hình thống kê evaluation cung cấp cái nhìn tổng quan về hoạt động của Rule Evaluation Engine thông qua các số liệu được thu thập trong quá trình xử lý yêu cầu từ Demo App hoặc các ứng dụng khách. Các thống kê được trình bày dưới dạng số liệu tổng hợp và biểu đồ trực quan, giúp người quản trị đánh giá mức độ sử dụng của từng Feature Flag cũng như theo dõi hiệu quả của các chiến lược rollout. Việc theo dõi các chỉ số này hỗ trợ quá trình vận hành và tối ưu cấu hình Feature Flag trong suốt vòng đời của hệ thống [7, 8].

Tổng số lần evaluation

Dashboard hiển thị tổng số yêu cầu evaluation đã được hệ thống xử lý trong khoảng thời gian được lựa chọn. Chỉ số này phản ánh mức độ sử dụng của Evaluation API và cho phép người quản trị nhanh chóng đánh giá lưu lượng truy cập của hệ thống.

Ngoài giá trị tổng, giao diện còn cập nhật thời điểm thống kê gần nhất nhằm bảo đảm người dùng luôn quan sát được dữ liệu mới nhất. Việc theo dõi tổng số lần evaluation cũng giúp đánh giá hiệu quả của cơ chế cache và hỗ trợ lập kế hoạch mở rộng hệ thống khi lưu lượng truy cập tăng.

Thống kê theo Feature Flag

Dashboard cung cấp bảng thống kê số lần evaluation của từng Feature Flag trong project. Mỗi dòng thể hiện tên Feature Flag cùng với số lượng yêu cầu evaluation tương ứng, cho phép người quản trị xác định những Feature Flag được sử dụng nhiều nhất.

Thông tin này giúp nhận biết các Feature Flag đóng vai trò quan trọng trong hệ thống, từ đó ưu tiên theo dõi, kiểm thử hoặc tối ưu cấu hình. Đồng thời, các Feature Flag ít hoặc không còn được sử dụng cũng có thể được xem xét archive nhằm giảm chi phí quản lý và hạn chế phát sinh *feature flag debt* [1, 8].

Thống kê theo kết quả bật và tắt

Bên cạnh số lượng evaluation, Dashboard còn thống kê số lần Feature Flag được đánh giá ở trạng thái bật (*Enabled*) và tắt (*Disabled*). Các số liệu được trình bày dưới dạng biểu đồ hoặc tỷ lệ phần trăm để người quản trị dễ dàng quan sát.

Thông qua thống kê này, người quản trị có thể đánh giá mức độ ảnh hưởng của các chiến lược

rollout, xác định Feature Flag đang được phục vụ cho bao nhiêu người dùng và kiểm tra xem kết quả evaluation có phù hợp với cấu hình mong muốn hay không.

Biểu đồ theo thời gian

Dashboard hiển thị biểu đồ thể hiện số lượng evaluation theo từng khoảng thời gian như giờ hoặc ngày. Dữ liệu được tổng hợp liên tục từ các yêu cầu evaluation và cập nhật khi người dùng thay đổi khoảng thời gian thống kê.

Biểu đồ theo thời gian giúp nhận biết xu hướng sử dụng của hệ thống, phát hiện các khoảng thời gian có lưu lượng truy cập cao hoặc các biến động bất thường trong quá trình vận hành. Các thông tin này hỗ trợ người quản trị đánh giá tác động của việc phát hành tính năng mới, điều chỉnh chiến lược rollout và chuẩn bị tài nguyên phù hợp cho hệ thống.

3.3.8 Hiển thị quyền và trạng thái

Dashboard không chỉ cung cấp các chức năng quản trị Feature Flag mà còn phản ánh rõ quyền của người dùng và trạng thái hiện tại của từng đối tượng trong hệ thống. Giao diện được thiết kế để chỉ hiển thị các thao tác phù hợp với quyền được cấp, đồng thời trực quan hóa trạng thái lifecycle và runtime của Feature Flag nhằm giảm nguy cơ thao tác sai. Cách tiếp cận này giúp tăng tính an toàn trong quản trị và phù hợp với mô hình phân quyền của hệ thống [11, 7].

Ẩn hoặc vô hiệu hóa thao tác

Dashboard kiểm tra quyền của người dùng trước khi hiển thị các chức năng quản trị. Đối với những thao tác mà người dùng không được phép thực hiện, giao diện sẽ ẩn hoàn toàn hoặc chuyển nút chức năng sang trạng thái vô hiệu hóa (*disabled*).

Các thao tác như tạo Feature Flag, chỉnh sửa cấu hình, archive, restore hoặc kích hoạt Group Kill Switch chỉ khả dụng đối với các vai trò có quyền tương ứng. Việc kiểm soát ngay trên giao diện giúp giảm số lượng yêu cầu không hợp lệ gửi tới Backend API và cải thiện trải nghiệm sử dụng, mặc dù toàn bộ quyền truy cập vẫn được xác thực lại ở phía máy chủ [11].

Hiển thị role hiện tại

Dashboard hiển thị vai trò của người dùng đang đăng nhập tại khu vực thông tin tài khoản, giúp người dùng nhận biết phạm vi thao tác của mình trong hệ thống. Các vai trò như *Administrator*,

Developer hoặc *Viewer* được biểu diễn bằng nhãn trực quan để dễ phân biệt.

Việc hiển thị role ngay trên giao diện giúp người dùng hiểu lý do một số chức năng không khả dụng, đồng thời giảm nhầm lẫn khi nhiều tài khoản có các mức quyền khác nhau cùng sử dụng hệ thống quản trị.

Hiển thị trạng thái lifecycle

Mỗi Feature Flag đều hiển thị trạng thái vòng đời (*Lifecycle Status*) thông qua các nhãn trực quan như *Active* hoặc *Archived*. Trạng thái này phản ánh khả năng tồn tại của Feature Flag trong hệ thống và được cập nhật ngay sau mỗi thao tác archive hoặc restore.

Việc phân biệt rõ lifecycle status giúp người quản trị dễ dàng nhận biết các Feature Flag đã ngừng sử dụng nhưng vẫn được lưu trữ để phục vụ truy vết hoặc khôi phục sau này. Đây cũng là cơ sở để hạn chế việc cấu hình hoặc đánh giá các Feature Flag đã được archive [8].

Hiển thị trạng thái runtime

Bên cạnh lifecycle status, Dashboard còn hiển thị trạng thái runtime của Feature Flag nhằm phản ánh khả năng phục vụ thực tế trong quá trình evaluation. Trạng thái này được xác định từ nhiều yếu tố như trạng thái kích hoạt, Serving Mode, Kill Switch và cấu hình rule hiện hành.

Các trạng thái runtime được biểu diễn bằng màu sắc và nhãn trực quan để người quản trị nhanh chóng nhận biết Feature Flag đang được bật toàn cục, đang phục vụ theo rule targeting, đã bị vô hiệu hóa hoặc đang chịu tác động của Kill Switch. Việc hiển thị đồng thời lifecycle và runtime giúp người dùng phân biệt rõ giữa trạng thái quản lý của Feature Flag và trạng thái hoạt động thực tế trong hệ thống [7, 8].

3.4 Kết quả xây dựng Demo Application và SDK

3.4.1 Cấu trúc Demo Application

Demo Application được xây dựng dưới dạng một ứng dụng thương mại điện tử (e-commerce) đơn giản nhằm minh họa cách một ứng dụng khách tích hợp với nền tảng Feature Flag thông qua JavaScript SDK và Evaluation API. Toàn bộ mã nguồn của ứng dụng được triển khai độc lập với Dashboard quản trị và Backend API, trong khi hành vi của các tính năng được quyết định hoàn toàn tại thời điểm chạy dựa trên kết quả evaluation. Cách tiếp cận này phản ánh

đúng mục tiêu tách biệt giữa quá trình triển khai (deploy) và phát hành (release) của Feature Flag [1, 3].

Mô hình ứng dụng thương mại điện tử mẫu

Demo Application mô phỏng một website thương mại điện tử với các chức năng quen thuộc như duyệt sản phẩm, quản lý giỏ hàng và thanh toán. Mục tiêu của ứng dụng không phải xây dựng đầy đủ nghiệp vụ bán hàng mà tập trung tạo ra các tình huống thực tế để minh họa việc bật hoặc tắt tính năng bằng Feature Flag.

Ứng dụng được chia thành hai thành phần chính gồm giao diện người dùng và lớp tích hợp Feature Flag. Khi người dùng lựa chọn tài khoản mẫu, ứng dụng tạo *Evaluation Context* tương ứng và sử dụng JavaScript SDK để gửi yêu cầu đánh giá đến Evaluation API. Kết quả trả về quyết định giao diện nào sẽ được hiển thị, chẳng hạn sử dụng quy trình thanh toán cũ hay giao diện thanh toán mới. Kiến trúc này phù hợp với mô hình đánh giá Feature Flag được đề xuất trong OpenFeature [10, 6].

Dữ liệu người dùng và organization

Để phục vụ các kịch bản demo, hệ thống chuẩn bị sẵn tập dữ liệu người dùng mẫu (seed data). Mỗi tài khoản bao gồm các thông tin như `userId`, `targetingId`, vai trò (*role*) và thông tin `organization`. Trong đó, `targetingId` được sử dụng làm khóa ổn định cho quá trình percentage rollout nhằm đảm bảo cùng một người dùng luôn nhận được kết quả đánh giá giống nhau giữa các lần truy cập [5].

Các tài khoản được chia thành nhiều nhóm như quản trị viên cửa hàng, khách hàng beta và khách hàng thông thường. Việc phân chia này giúp minh họa đầy đủ các chiến lược targeting theo vai trò, theo người dùng và theo phần trăm người dùng mà không cần xây dựng cơ chế đăng ký tài khoản hoặc xác thực phức tạp.

Các tính năng được kiểm soát bằng flag

Demo Application tích hợp nhiều Feature Flag đại diện cho các tình huống phát hành phổ biến trong thực tế. Một số flag được sử dụng để minh họa giao diện thanh toán mới (`new-checkout`), công cụ áp dụng mã giảm giá (`coupon-engine`), bảng điều khiển beta (`beta-dashboard`) và cửa sổ hỗ trợ trực tuyến (`live-support-widget`).

Các Feature Flag này được cấu hình từ Dashboard quản trị và có thể thay đổi ngay trong quá trình chạy mà không cần triển khai lại ứng dụng. Tùy theo cấu hình của từng môi trường, người dùng hoặc vai trò, cùng một phiên bản ứng dụng có thể hiển thị các chức năng khác nhau. Điều này thể hiện đúng đặc điểm quản lý phát hành động của Feature Flag Platform [7, 8].

Môi trường staging và production

Demo Application được triển khai đồng thời trên hai môi trường là *staging* và *production*. Hai môi trường sử dụng cùng một mã nguồn nhưng đánh giá Feature Flag dựa trên cấu hình riêng của từng environment.

Trong quá trình trình diễn, hai phiên bản của ứng dụng được mở song song để chứng minh rằng cùng một người dùng và cùng một Feature Flag có thể nhận các kết quả evaluation khác nhau khi chuyển đổi giữa hai môi trường. Cách tổ chức này phản ánh quy trình phát hành phổ biến trong các hệ thống hiện đại, cho phép kiểm thử trên staging trước khi áp dụng cấu hình tương ứng cho production mà không cần triển khai lại phần mềm [3, 7].

3.4.2 JavaScript SDK

Để đơn giản hóa quá trình tích hợp Feature Flag vào ứng dụng khách, hệ thống xây dựng một JavaScript SDK đóng vai trò trung gian giữa Demo Application và Evaluation API. Thay vì tự xây dựng HTTP request cho mỗi lần kiểm tra trạng thái Feature Flag, ứng dụng chỉ cần khởi tạo SDK một lần và sử dụng các phương thức do SDK cung cấp. SDK chịu trách nhiệm tạo Evaluation Context, gửi yêu cầu đến Backend API, xử lý phản hồi và trả về kết quả dưới dạng thuận tiện cho ứng dụng sử dụng. Thiết kế này phù hợp với mô hình Feature Provider được đề xuất trong OpenFeature [10, 6].

Khởi tạo SDK client

SDK được khởi tạo khi ứng dụng bắt đầu hoạt động bằng cách cung cấp địa chỉ Evaluation API và các thông số cấu hình cần thiết. Sau khi được tạo, SDK được sử dụng như một singleton trong toàn bộ vòng đời của ứng dụng nhằm tránh khởi tạo lại nhiều lần.

Bên cạnh địa chỉ API, SDK cho phép cấu hình môi trường làm việc, thời gian timeout và các giá trị mặc định. Việc tập trung cấu hình tại một vị trí giúp đơn giản hóa quá trình bảo trì cũng như tạo điều kiện mở rộng khi cần bổ sung cơ chế cache hoặc xác thực trong tương lai.

Phương thức evaluate

Phương thức chính của SDK là `evaluate()`, nhận vào khóa Feature Flag cùng Evaluation Context của người dùng. SDK tạo yêu cầu HTTP đến Evaluation API và nhận phản hồi bao gồm trạng thái bật hoặc tắt của Feature Flag, biến thể (variant) và reason code giải thích nguyên nhân của kết quả evaluation.

Sau khi nhận được phản hồi, SDK chuyển đổi dữ liệu thành cấu trúc đơn giản để ứng dụng sử dụng trực tiếp. Nhờ đó, mã nguồn của Demo Application không cần quan tâm đến chi tiết giao thức truyền thông hay định dạng phản hồi của Backend API. Cách thiết kế này giúp giảm sự phụ thuộc giữa ứng dụng khách và nền tảng Feature Flag, đồng thời phù hợp với kiến trúc Feature Flag hiện đại [10].

Cấu hình default value

SDK hỗ trợ khai báo giá trị mặc định (*default value*) cho mỗi lần evaluation. Nếu Feature Flag không tồn tại hoặc Evaluation API không trả về kết quả hợp lệ, SDK sẽ trả về giá trị mặc định thay vì để ứng dụng xử lý ngoại lệ.

Cơ chế này bảo đảm ứng dụng luôn hoạt động ổn định ngay cả khi xảy ra lỗi ngoài mong muốn. Việc sử dụng default value cũng phù hợp với nguyên tắc fail-safe trong các hệ thống Feature Flag, giúp hạn chế ảnh hưởng của sự cố đến trải nghiệm người dùng [10, 8].

Xử lý timeout và lỗi mạng

Trong quá trình giao tiếp với Evaluation API, SDK thiết lập thời gian chờ tối đa cho mỗi yêu cầu. Nếu quá thời gian quy định hoặc xảy ra lỗi mạng, SDK sẽ hủy yêu cầu, ghi nhận lỗi và trả về giá trị mặc định đã cấu hình thay vì làm gián đoạn luồng xử lý của ứng dụng.

Ngoài timeout, SDK cũng xử lý các trường hợp như máy chủ không phản hồi, phản hồi không hợp lệ hoặc lỗi HTTP. Việc che giấu các chi tiết xử lý lỗi bên trong SDK giúp mã nguồn của Demo Application ngắn gọn hơn, đồng thời bảo đảm mọi Feature Flag đều được đánh giá theo cùng một quy trình thống nhất. Cách tiếp cận này phù hợp với khuyến nghị về xử lý lỗi và bảo vệ tính sẵn sàng của các dịch vụ REST [11, 10].

3.4.3 Tích hợp SDK với Demo Application

Demo Application sử dụng JavaScript SDK như lớp trung gian duy nhất để giao tiếp với Feature Flag Platform. Mỗi khi cần xác định trạng thái của một tính năng, ứng dụng tạo Evaluation Context từ thông tin người dùng hiện tại, gọi phương thức `evaluate()` của SDK và sử dụng kết quả trả về để quyết định giao diện hoặc hành vi tương ứng. Cách tích hợp này giúp tách biệt hoàn toàn logic nghiệp vụ của ứng dụng với cơ chế đánh giá Feature Flag, đồng thời tạo điều kiện mở rộng khi thay đổi nền tảng Feature Flag trong tương lai [10, 6].

Tạo evaluation context

Khi người dùng truy cập Demo Application hoặc chuyển đổi sang một tài khoản khác, ứng dụng thu thập các thông tin cần thiết để tạo Evaluation Context. Các thông tin này bao gồm `targetingKey`, `userId`, danh sách vai trò và các thuộc tính bổ sung của người dùng.

Evaluation Context được truyền nguyên vẹn đến SDK để gửi tới Evaluation API. Việc chuẩn hóa cấu trúc dữ liệu giúp tất cả các Feature Flag được đánh giá theo cùng một quy trình, đồng thời tạo khả năng mở rộng khi cần bổ sung thêm các thuộc tính targeting trong tương lai mà không ảnh hưởng đến mã nguồn của ứng dụng [6].

Gọi Evaluation API

Sau khi tạo Evaluation Context, Demo Application gọi phương thức `evaluate()` của SDK cho từng Feature Flag cần kiểm tra. SDK chịu trách nhiệm gửi yêu cầu đến Evaluation API và nhận kết quả đánh giá từ Rule Evaluation Engine.

Phản hồi từ Evaluation API bao gồm trạng thái bật hoặc tắt của Feature Flag, biến thể được lựa chọn và reason code giải thích kết quả evaluation. Nhờ đó, ứng dụng chỉ cần sử dụng dữ liệu trả về mà không phải tự cài đặt các thuật toán đánh giá rule hay percentage rollout, bảo đảm tính nhất quán giữa Dashboard quản trị và ứng dụng khách [10, 4].

Hiển thị hoặc ẩn tính năng

Kết quả evaluation được sử dụng trực tiếp để điều khiển giao diện của Demo Application. Nếu Feature Flag được bật, thành phần giao diện hoặc chức năng tương ứng sẽ được hiển thị; ngược lại, ứng dụng tiếp tục sử dụng hành vi mặc định hoặc ẩn hoàn toàn tính năng đó.

Cơ chế này cho phép cùng một phiên bản ứng dụng có thể cung cấp các trải nghiệm khác nhau

đối với từng người dùng, từng vai trò hoặc từng nhóm rollout mà không cần thay đổi mã nguồn. Đây cũng là đặc điểm cốt lõi của Feature Flag trong việc tách biệt giữa triển khai và phát hành tính năng [1, 7].

Làm mới trạng thái sau thay đổi cấu hình

Sau khi cấu hình Feature Flag được thay đổi trên Dashboard quản trị, Demo Application thực hiện đánh giá lại thông qua SDK để nhận trạng thái mới nhất từ Evaluation API. Việc đánh giá lại có thể được kích hoạt khi người dùng tải lại trang, chuyển đổi tài khoản hoặc thực hiện thao tác làm mới trạng thái trong ứng dụng.

Nhờ cơ chế này, các thay đổi về bật hoặc tắt Feature Flag, thay đổi rule hoặc điều chỉnh tỷ lệ rollout được phản ánh ngay trên giao diện mà không cần triển khai lại ứng dụng. Điều này minh họa rõ khả năng quản lý và phát hành tính năng theo thời gian thực của Feature Flag Platform [8, 3].

3.4.4 User context trong Demo Application

User context là tập hợp các thông tin mô tả người dùng tại thời điểm gửi yêu cầu evaluation đến Feature Flag Platform. Demo Application xây dựng user context từ dữ liệu của tài khoản hiện tại và truyền toàn bộ thông tin này cho JavaScript SDK. Evaluation Engine sử dụng user context để xác định người dùng có thuộc phạm vi áp dụng của Feature Flag hay không. Thiết kế này tuân theo mô hình Evaluation Context của OpenFeature, trong đó mọi quyết định đều được thực hiện dựa trên ngữ cảnh của người dùng thay vì trạng thái lưu cục bộ trong ứng dụng [6, 10].

Targeting key

`targetingKey` là định danh ổn định được sử dụng cho các cơ chế targeting và percentage rollout. Trong Demo Application, mỗi người dùng được gán một `targetingKey` duy nhất và giá trị này được giữ nguyên trong suốt vòng đời của tài khoản.

Evaluation Engine sử dụng `targetingKey` để tính toán giá trị băm phục vụ percentage rollout. Nhờ sử dụng cùng một khóa cho mọi lần đánh giá, người dùng luôn được gán vào cùng một bucket, tránh hiện tượng thay đổi kết quả ngẫu nhiên giữa các lần truy cập [5].

User ID

Bên cạnh `targetingKey`, Demo Application cũng truyền `userId` của người dùng đến Evaluation API. Thuộc tính này được sử dụng trong các rule targeting theo người dùng, cho phép bật hoặc tắt Feature Flag đối với các tài khoản cụ thể.

Việc tách biệt `userId` và `targetingKey` giúp hệ thống có thể sử dụng các chiến lược đánh giá khác nhau. Trong khi `userId` phù hợp với các danh sách allowlist hoặc denylist, `targetingKey` được tối ưu cho các thuật toán rollout ổn định theo phần trăm người dùng [4, 5].

Role

Thông tin vai trò của người dùng được truyền dưới dạng danh sách các giá trị `role`. Evaluation Engine kiểm tra các vai trò này khi đánh giá các rule targeting theo role để quyết định trạng thái của Feature Flag.

Trong Demo Application, các vai trò như khách hàng thông thường, khách hàng beta hoặc quản trị viên được sử dụng để minh họa việc phát hành tính năng theo từng nhóm người dùng. Nhờ đó, cùng một Feature Flag có thể được kích hoạt cho một số vai trò nhất định mà không ảnh hưởng đến các nhóm còn lại [4].

Organization ID

Mỗi người dùng trong Demo Application đều thuộc một `organizationId`. Thuộc tính này được đưa vào Evaluation Context nhằm minh họa khả năng mở rộng của hệ thống khi cần triển khai Feature Flag theo từng tổ chức hoặc khách hàng.

Trong phạm vi mini project, `organizationId` chủ yếu phục vụ mục đích trình diễn và chuẩn bị cho các chiến lược targeting nâng cao trong tương lai. Việc đưa thuộc tính này vào Evaluation Context ngay từ đầu giúp hạn chế thay đổi giao diện lập trình của SDK khi mở rộng hệ thống.

Thuộc tính mở rộng

Ngoài các trường dữ liệu chuẩn, Demo Application cho phép truyền thêm các thuộc tính mở rộng dưới dạng tập khóa–giá trị. Các thuộc tính này có thể mô tả đặc điểm của người dùng hoặc ngữ cảnh sử dụng, chẳng hạn khu vực, loại thiết bị hoặc nhóm khách hàng.

Mặc dù Rule Evaluation Engine hiện tại chưa khai thác toàn bộ các thuộc tính mở rộng, việc

thiết kế Evaluation Context theo hướng mở giúp hệ thống dễ dàng bổ sung các chiến lược targeting dựa trên thuộc tính mà không cần thay đổi giao diện của SDK hoặc Evaluation API. Dây cũng là định hướng được khuyến nghị trong đặc tả OpenFeature nhằm tăng khả năng mở rộng và khả năng tương thích giữa các Feature Flag Provider [6].

3.4.5 Hành vi fallback

Để bảo đảm Demo Application luôn hoạt động ổn định trong các tình huống bất thường, JavaScript SDK triển khai cơ chế fallback khi không thể nhận được kết quả evaluation hợp lệ từ Backend API. Thay vì làm gián đoạn luồng xử lý hoặc gây lỗi giao diện, SDK trả về giá trị mặc định đã được cấu hình và ghi nhận nguyên nhân xảy ra lỗi. Cách tiếp cận này phù hợp với nguyên tắc fail-safe trong các hệ thống Feature Flag, giúp ứng dụng vẫn duy trì khả năng phục vụ ngay cả khi dịch vụ đánh giá tạm thời không khả dụng [10, 8].

Evaluation API không khả dụng

Trong trường hợp Evaluation API không thể kết nối do máy chủ ngừng hoạt động hoặc gặp sự cố mạng, SDK không tiếp tục thử đánh giá Feature Flag tại phía ứng dụng. Thay vào đó, SDK trả về giá trị mặc định đã được cấu hình cho Feature Flag tương ứng.

Cơ chế này giúp Demo Application không phụ thuộc hoàn toàn vào trạng thái của Evaluation API và tránh tình trạng toàn bộ giao diện bị ảnh hưởng bởi lỗi của một dịch vụ bên ngoài. Đồng thời, việc xử lý tập trung trong SDK giúp ứng dụng khách không cần tự cài đặt các cơ chế phục hồi riêng.

Request timeout

Mỗi yêu cầu gửi đến Evaluation API đều được giới hạn bởi một khoảng thời gian chờ tối đa. Nếu máy chủ phản hồi chậm vượt quá thời gian này, SDK sẽ hủy yêu cầu và coi đây là một lần evaluation thất bại.

Sau khi timeout xảy ra, SDK trả về giá trị mặc định thay vì tiếp tục chờ phản hồi. Cách xử lý này giúp duy trì khả năng phản hồi của giao diện người dùng, đồng thời tránh hiện tượng treo ứng dụng khi dịch vụ phía máy chủ gặp vấn đề về hiệu năng hoặc kết nối mạng [11].

Flag không tồn tại

Nếu Evaluation API xác định Feature Flag không tồn tại hoặc đã bị loại bỏ khỏi hệ thống, phản hồi sẽ chứa reason code tương ứng và SDK trả về giá trị mặc định của ứng dụng.

Việc sử dụng giá trị mặc định trong trường hợp này giúp ứng dụng tiếp tục hoạt động bình thường ngay cả khi tồn tại sự khác biệt giữa phiên bản ứng dụng và cấu hình hiện có trên Feature Flag Platform. Đồng thời, cơ chế này hạn chế nguy cơ phát sinh lỗi do truy cập vào các Feature Flag không hợp lệ [10].

Context không đầy đủ

Trong một số trường hợp, Evaluation Context có thể thiếu các trường dữ liệu cần thiết như `targetingKey`, `userId` hoặc `role`. Khi đó, Evaluation API không thể áp dụng đầy đủ các rule targeting và sẽ thực hiện xử lý theo cơ chế mặc định của Rule Evaluation Engine.

Đối với Demo Application, SDK vẫn gửi toàn bộ thông tin hiện có trong Evaluation Context và sử dụng kết quả trả về từ Evaluation API. Nếu không thể xác định trạng thái của Feature Flag, SDK tiếp tục trả về giá trị mặc định đã cấu hình. Cách tiếp cận này giúp ứng dụng hoạt động ổn định, đồng thời cho phép mở rộng Evaluation Context trong tương lai mà không làm thay đổi giao diện lập trình của SDK [6, 10].

3.5 Các kịch bản minh họa

3.5.1 Bật và tắt tính năng toàn cục

Kịch bản đầu tiên minh họa khả năng bật hoặc tắt một tính năng cho toàn bộ người dùng mà không cần triển khai lại ứng dụng. Đây là trường hợp sử dụng phổ biến nhất của Feature Flag trong thực tế, cho phép tách quá trình triển khai mã nguồn khỏi quá trình phát hành tính năng, đồng thời hỗ trợ rollback nhanh khi phát hiện sự cố trên môi trường production [1, 3]. Trong hệ thống được xây dựng, chế độ này được thực hiện thông qua *Serving Mode = Global On* kết hợp với trạng thái cấu hình của Feature Flag.

Cấu hình ban đầu

Ban đầu, Feature Flag được tạo trong Dashboard với trạng thái *Enabled* và chế độ phục vụ (*Serving Mode*) được đặt là *Targeted*. Khi đó, Demo Application chỉ hiển thị tính năng nếu kết

quả evaluation thỏa mãn các rule đã cấu hình. Evaluation Engine thực hiện kiểm tra trạng thái của Feature Flag trước khi đánh giá các rule, bảo đảm thứ tự ưu tiên xử lý nhất quán trong toàn hệ thống.

Để minh họa việc phát hành tính năng cho toàn bộ người dùng, người quản trị chuyển chế độ phục vụ sang *Global On*. Khi đó, mọi yêu cầu evaluation hợp lệ đều nhận kết quả bật tính năng mà không cần xét đến user, role hoặc tỷ lệ rollout. Cơ chế này phù hợp với mô hình phát hành toàn cục được áp dụng trong nhiều nền tảng Feature Flag hiện đại [7].

Thao tác trên Dashboard

Từ màn hình quản lý Feature Flag, người quản trị chọn Feature Flag cần cấu hình và mở trang chỉnh sửa cấu hình. Tại đây, chế độ phục vụ được chuyển từ *Targeted* sang *Global On* và lưu thay đổi.

Sau khi cấu hình được cập nhật thành công, Backend API ghi nhận thay đổi vào cơ sở dữ liệu, đồng thời cập nhật cache nếu có sử dụng cơ chế lưu đệm. Những yêu cầu evaluation được gửi sau thời điểm này sẽ sử dụng cấu hình mới mà không cần khởi động lại Backend hoặc Demo Application. Việc thay đổi trạng thái thông qua Dashboard thể hiện đặc điểm cấu hình tại thời gian chạy (*runtime configuration*) của Feature Flag [8].

Kết quả trên Demo Application

Sau khi Dashboard hoàn tất cập nhật cấu hình, Demo Application gửi yêu cầu đến Evaluation API với Evaluation Context của người dùng hiện tại. Evaluation Engine kiểm tra trạng thái của Feature Flag và xác định chế độ phục vụ là *Global On*. Do đó, quá trình đánh giá kết thúc ngay mà không cần tiếp tục kiểm tra các rule targeting.

Evaluation API trả về kết quả với thuộc tính `enabled = true` cùng mã nguyên nhân (*reason*) tương ứng với chế độ bật toàn cục. Demo Application nhận phản hồi và hiển thị đầy đủ giao diện hoặc chức năng được bảo vệ bởi Feature Flag. Kết quả này giống nhau đối với mọi người dùng, bất kể user ID, role hoặc các thuộc tính khác trong Evaluation Context.

Để minh họa khả năng rollback tức thời, người quản trị tiếp tục chuyển Feature Flag từ *Global On* về *Targeted* hoặc *Disabled*. Ngay sau khi cấu hình được cập nhật, các lần evaluation tiếp theo phản ánh trạng thái mới mà không cần triển khai lại ứng dụng, đúng với mục tiêu tách biệt giữa deploy và release [1, 3].

Audit log phát sinh

Mỗi thao tác thay đổi trạng thái hoặc chế độ phục vụ của Feature Flag đều sinh một bản ghi Audit Log. Bản ghi lưu thông tin người thực hiện, thời điểm thay đổi, hành động tương ứng, Request ID cùng dữ liệu trước và sau khi cập nhật cấu hình.

Việc lưu trữ đầy đủ thông tin thay đổi giúp truy vết nguyên nhân khi phát sinh sự cố, hỗ trợ kiểm tra lịch sử cấu hình và tăng khả năng kiểm toán của hệ thống. Audit Log được lưu theo mô hình append-only, không cho phép chỉnh sửa trực tiếp nhằm bảo đảm tính toàn vẹn của dữ liệu [12].

3.5.2 Bật tính năng cho người dùng cụ thể

Kịch bản thứ hai minh họa khả năng phát hành tính năng cho một hoặc một nhóm nhỏ người dùng xác định thông qua *User Allowlist*. Đây là chiến lược thường được sử dụng để kiểm thử nội bộ, phát hành sớm cho khách hàng quan trọng hoặc xác minh tính năng trước khi mở rộng phạm vi triển khai. Trong hệ thống được xây dựng, Rule Evaluation Engine ưu tiên kiểm tra rule theo thứ tự đã thiết kế, trong đó User Allowlist được đánh giá trước các rule theo role và percentage rollout nhằm bảo đảm người dùng được chỉ định luôn nhận được kết quả mong muốn [4].

Cấu hình user allowlist

Từ Dashboard, người quản trị mở trang cấu hình Feature Flag và tạo một rule có kiểu *User Allowlist*. Rule này chứa danh sách các *targetingKey* hoặc *userId* được phép sử dụng tính năng. Sau khi lưu cấu hình, Backend API cập nhật dữ liệu vào cơ sở dữ liệu và sử dụng cấu hình mới cho các lần evaluation tiếp theo.

Evaluation Context do Demo Application gửi lên bao gồm thông tin định danh người dùng, trong đó *targetingKey* đóng vai trò là khóa chính để thực hiện so khớp với danh sách đã cấu hình. Việc sử dụng Evaluation Context giúp tách biệt thông tin người dùng khỏi logic đánh giá Feature Flag, phù hợp với mô hình được đề xuất trong OpenFeature [6].

Người dùng khớp rule

Khi người dùng có *targetingKey* nằm trong danh sách cho phép gửi yêu cầu đến Evaluation API, Evaluation Engine lần lượt kiểm tra trạng thái của Feature Flag và tiếp tục đánh giá rule

User Allowlist. Do điều kiện được thỏa mãn, quá trình evaluation kết thúc ngay mà không cần tiếp tục đánh giá các rule phía sau.

Evaluation API trả về kết quả với thuộc tính `enabled = true`. Demo Application nhận phản hồi và hiển thị đầy đủ tính năng tương ứng. Những người dùng khác không bị ảnh hưởng bởi cấu hình này, cho phép triển khai thử nghiệm trên phạm vi rất nhỏ trước khi mở rộng phát hành.

Người dùng không khớp rule

Đối với người dùng không nằm trong danh sách cho phép, Evaluation Engine tiếp tục xử lý theo các rule còn lại nếu có. Trong trường hợp Feature Flag chỉ cấu hình duy nhất User Allowlist, quá trình evaluation không tìm thấy rule phù hợp và hệ thống sử dụng giá trị mặc định của Feature Flag.

Evaluation API trả về kết quả `enabled = false`, do đó Demo Application tiếp tục ẩn chức năng được bảo vệ bởi Feature Flag. Cách xử lý này bảo đảm chỉ những người dùng được chỉ định mới có thể sử dụng tính năng trong giai đoạn thử nghiệm.

Reason code trả về

Ngoài trạng thái bật hoặc tắt, Evaluation API còn trả về *reason code* nhằm giải thích nguyên nhân của kết quả evaluation. Khi người dùng khớp với User Allowlist, hệ thống trả về mã nguyên nhân `USER_MATCH`. Nếu không có rule nào phù hợp, Evaluation Engine trả về mã nguyên nhân mặc định tương ứng với việc không tìm thấy điều kiện kích hoạt, chẳng hạn `DEFAULT_OFF`. Các mã nguyên nhân này giúp Demo Application, Dashboard và Audit Log dễ dàng theo dõi quá trình đánh giá, đồng thời hỗ trợ kiểm tra và xử lý sự cố trong quá trình vận hành [10, 4].

3.5.3 Bật tính năng theo vai trò

Kịch bản thứ ba minh họa khả năng phát hành tính năng dựa trên vai trò của người dùng (*Role Targeting*). Đây là một trong những chiến lược phổ biến của Feature Flag nhằm triển khai tính năng cho từng nhóm người dùng có cùng đặc điểm nghiệp vụ mà không cần cấu hình riêng cho từng tài khoản. Trong hệ thống được xây dựng, thông tin vai trò được truyền trong Evaluation Context và được Rule Evaluation Engine sử dụng để xác định quyền truy cập vào tính năng [4, 6].

Cấu hình role targeting

Người quản trị mở Dashboard và tạo một rule có kiểu *Role Targeting*. Rule này quy định danh sách các role được phép sử dụng Feature Flag, ví dụ `beta-customer`. Sau khi lưu cấu hình, Backend API cập nhật dữ liệu và áp dụng ngay cho các lần evaluation tiếp theo.

Khi Demo Application gửi yêu cầu đến Evaluation API, Evaluation Context bao gồm danh sách role hiện tại của người dùng. Rule Evaluation Engine sẽ kiểm tra các role này với danh sách đã cấu hình và quyết định trạng thái của Feature Flag.

Beta customer

Đối với người dùng mang role `beta-customer`, Evaluation Engine xác định có ít nhất một role khớp với rule cấu hình. Quá trình evaluation dừng tại rule này và trả về kết quả `enabled = true` cùng reason code tương ứng.

Demo Application nhận phản hồi từ Evaluation API và hiển thị tính năng mới cho nhóm người dùng beta. Kịch bản này phù hợp với giai đoạn phát hành thử nghiệm trước khi triển khai rộng rãi cho toàn bộ khách hàng.

Regular customer

Đối với người dùng chỉ có role `customer` hoặc các role không nằm trong danh sách được cấu hình, Evaluation Engine không tìm thấy điều kiện phù hợp tại bước Role Targeting. Nếu không còn rule nào khác có thể áp dụng, hệ thống sử dụng giá trị mặc định của Feature Flag.

Evaluation API trả về `enabled = false`, do đó Demo Application tiếp tục ẩn chức năng tương ứng. Việc giới hạn phạm vi phát hành theo role giúp giảm rủi ro khi đưa tính năng mới vào môi trường thực tế và cho phép kiểm soát chính xác nhóm người dùng được trải nghiệm tính năng.

Trường hợp người dùng có nhiều role

Trong nhiều hệ thống, một người dùng có thể đồng thời sở hữu nhiều role. Evaluation Context vì vậy truyền danh sách role thay vì một giá trị duy nhất. Rule Evaluation Engine thực hiện so khớp toàn bộ danh sách này với danh sách role được cấu hình trong Feature Flag.

Nếu tồn tại ít nhất một role khớp với rule, Feature Flag được đánh giá là bật và quá trình evaluation kết thúc ngay. Ngược lại, nếu không có role nào phù hợp, hệ thống tiếp tục đánh

giá các rule tiếp theo hoặc trả về giá trị mặc định. Cách xử lý này giúp hệ thống hỗ trợ linh hoạt các mô hình phân quyền trong thực tế mà vẫn duy trì quá trình evaluation đơn giản và xác định [4, 10].

3.5.4 Rollout theo phần trăm người dùng

Kịch bản thứ tư minh họa khả năng phát hành tính năng theo một tỷ lệ người dùng xác định (*Percentage Rollout*). Thay vì bật đồng thời cho toàn bộ hệ thống, Feature Flag chỉ được kích hoạt cho một phần người dùng dựa trên kết quả của thuật toán phân nhóm ổn định (*stable hashing*). Chiến lược này giúp triển khai tính năng từng bước, theo dõi phản hồi thực tế và giảm thiểu rủi ro khi phát hành phiên bản mới [1, 5].

Cấu hình tỷ lệ rollout

Người quản trị truy cập Dashboard và tạo một rule có kiểu *Percentage Rollout*. Tại đây, tỷ lệ rollout được cấu hình dưới dạng phần trăm, ví dụ 20%, 50% hoặc 100%. Sau khi lưu thay đổi, Backend API cập nhật cấu hình của Feature Flag và sử dụng giá trị mới cho các lần evaluation tiếp theo.

Trong Demo Application, mọi yêu cầu evaluation đều gửi kèm *targetingKey* trong Evaluation Context. Đây là khóa được sử dụng để xác định người dùng có thuộc nhóm được bật tính năng hay không, bảo đảm kết quả evaluation có tính nhất quán giữa các lần truy cập [6].

Phân nhóm người dùng bằng stable hashing

Khi Evaluation Engine xử lý rule Percentage Rollout, hệ thống tạo một giá trị băm ổn định từ tổ hợp *projectKey*, *flagKey* và *targetingKey*. Giá trị băm sau đó được ánh xạ vào một khoảng giá trị từ 0 đến dưới 100 để xác định bucket của người dùng.

Nếu bucket của người dùng nhỏ hơn tỷ lệ rollout đã cấu hình thì Feature Flag được bật; ngược lại, tính năng tiếp tục bị tắt. Do cùng một đầu vào luôn tạo ra cùng một giá trị băm, một người dùng sẽ luôn thuộc cùng một bucket miễn là các thông tin đầu vào không thay đổi. Điều này bảo đảm việc phân phối người dùng ổn định trong suốt quá trình rollout [5].

Kiểm tra tính ổn định

Để kiểm tra tính ổn định của thuật toán, cùng một người dùng thực hiện nhiều lần evaluation với cùng *targetingKey*. Kết quả cho thấy người dùng luôn nhận được cùng một trạng thái Feature Flag ở mọi lần yêu cầu, ngay cả khi ứng dụng được tải lại hoặc gửi nhiều request liên tiếp.

Trong khi đó, các người dùng khác với *targetingKey* khác nhau sẽ được phân bố vào các bucket khác nhau. Việc sử dụng stable hashing giúp tránh hiện tượng người dùng liên tục thay đổi giữa trạng thái bật và tắt, từ đó mang lại trải nghiệm nhất quán trong quá trình sử dụng hệ thống.

Tăng và giảm tỷ lệ rollout

Sau khi xác nhận tính năng hoạt động ổn định, người quản trị có thể tăng dần tỷ lệ rollout, chẳng hạn từ 20% lên 50%, sau đó lên 100%. Khi tỷ lệ được mở rộng, những người dùng đã nằm trong nhóm được bật trước đó vẫn tiếp tục sử dụng tính năng, đồng thời hệ thống bổ sung thêm các người dùng mới theo kết quả của thuật toán stable hashing.

Ngược lại, khi cần giảm phạm vi phát hành do phát hiện lỗi hoặc muốn hạn chế ảnh hưởng, người quản trị chỉ cần giảm tỷ lệ rollout trên Dashboard. Các lần evaluation tiếp theo sẽ phản ánh ngay cấu hình mới mà không cần triển khai lại ứng dụng. Kịch bản này minh họa khả năng kiểm soát quá trình phát hành theo từng giai đoạn và thực hiện rollback linh hoạt, đúng với mục tiêu của Feature Flag trong quy trình phát triển phần mềm hiện đại [1, 3, 5].

3.5.5 Cấu hình theo environment

Kịch bản thứ năm minh họa khả năng quản lý cấu hình Feature Flag độc lập theo từng môi trường triển khai (*Environment*). Trong hệ thống được xây dựng, mỗi Feature Flag có thể tồn tại nhiều cấu hình tương ứng với các environment khác nhau như *staging* và *production*. Nhờ đó, cùng một tính năng có thể được bật để kiểm thử trên môi trường staging nhưng vẫn giữ trạng thái tắt trên production, hỗ trợ quy trình phát hành từng bước và giảm thiểu rủi ro khi đưa tính năng mới vào vận hành [7, 3].

Cấu hình trên staging

Người quản trị lựa chọn environment *staging* trên Dashboard và cấu hình Feature Flag theo mục đích kiểm thử. Ví dụ, Feature Flag được đặt ở chế độ *Global On* hoặc cấu hình các rule targeting để toàn bộ nhóm kiểm thử có thể sử dụng tính năng mới.

Sau khi lưu thay đổi, Backend API chỉ cập nhật cấu hình của Feature Flag trong environment staging. Các yêu cầu evaluation gửi với environment này sẽ sử dụng cấu hình tương ứng mà không ảnh hưởng đến các environment khác.

Cấu hình trên production

Đối với environment *production*, người quản trị có thể giữ Feature Flag ở trạng thái *Disabled*, cấu hình rollout theo tỷ lệ nhỏ hoặc áp dụng các rule targeting khác với staging. Điều này cho phép tính năng tiếp tục được kiểm thử trên staging trong khi production vẫn duy trì trạng thái ổn định.

Việc quản lý cấu hình độc lập giúp nhóm phát triển chuẩn bị sẵn cấu hình cho production trước khi chính thức phát hành, đồng thời giảm nhu cầu chỉnh sửa cấu hình trong thời điểm triển khai phiên bản mới.

So sánh kết quả evaluation

Demo Application lần lượt gửi Evaluation API với cùng một Evaluation Context nhưng thay đổi giá trị environment trong yêu cầu. Khi sử dụng environment *staging*, Evaluation Engine tải cấu hình của staging và trả về kết quả tương ứng. Khi chuyển sang environment *production*, hệ thống sử dụng cấu hình của production để thực hiện evaluation.

Mặc dù cùng một Feature Flag và cùng một người dùng, kết quả evaluation có thể khác nhau do mỗi environment có bộ cấu hình, trạng thái và rule riêng. Điều này phản ánh đúng cách các nền tảng Feature Flag hiện đại quản lý nhiều môi trường triển khai trong cùng một dự án [7].

Isolation giữa các environment

Mỗi environment được lưu trữ và quản lý như một cấu hình độc lập. Các thao tác tạo, cập nhật hoặc thay đổi rule trong một environment không làm thay đổi dữ liệu của environment khác. Rule Evaluation Engine luôn truy xuất cấu hình theo cặp *project* và *environment* trước khi thực hiện evaluation, từ đó bảo đảm tính cô lập giữa các môi trường.

Cơ chế này cho phép nhóm phát triển kiểm thử, xác nhận và điều chỉnh Feature Flag trên staging trước khi áp dụng sang production. Việc tách biệt cấu hình theo environment góp phần nâng cao tính an toàn của quá trình phát hành và phù hợp với quy trình triển khai liên tục trong phát triển phần mềm hiện đại [8, 3].

3.5.6 Thứ tự ưu tiên của rule

Kịch bản thứ sáu minh họa cơ chế xử lý khi một Feature Flag đồng thời chứa nhiều rule khác nhau. Trong hệ thống được xây dựng, các rule được đánh giá theo thứ tự ưu tiên đã cấu hình. Rule Evaluation Engine lần lượt kiểm tra từng rule và dừng ngay khi gặp rule đầu tiên thỏa mãn điều kiện. Cơ chế này giúp quá trình evaluation có tính xác định, dễ dự đoán và phù hợp với cách triển khai của nhiều nền tảng Feature Flag hiện đại [4, 10].

Nhiều rule cùng tồn tại

Một Feature Flag có thể đồng thời chứa nhiều rule như User Allowlist, Role Targeting và Percentage Rollout. Mỗi rule được gán một mức ưu tiên để xác định thứ tự đánh giá. Ví dụ, hệ thống cấu hình User Allowlist ở mức ưu tiên cao nhất, tiếp theo là Role Targeting và cuối cùng là Percentage Rollout.

Khi Demo Application gửi yêu cầu evaluation, Rule Evaluation Engine tải toàn bộ danh sách rule của Feature Flag theo đúng thứ tự ưu tiên trước khi bắt đầu quá trình đánh giá.

Rule có độ ưu tiên cao hơn

Evaluation Engine luôn xử lý rule có mức ưu tiên cao hơn trước. Nếu người dùng đồng thời thỏa mãn nhiều rule khác nhau, kết quả của rule được đánh giá trước sẽ được sử dụng và các rule còn lại sẽ không được kiểm tra.

Ví dụ, một người dùng vừa nằm trong User Allowlist vừa có role `beta-customer`. Do User Allowlist có mức ưu tiên cao hơn, Evaluation Engine trả về kết quả của rule này mà không tiếp tục đánh giá Role Targeting hoặc Percentage Rollout.

Rule đầu tiên khớp

Nếu rule đầu tiên không thỏa mãn điều kiện, Evaluation Engine tiếp tục kiểm tra các rule tiếp theo theo đúng thứ tự đã cấu hình. Quá trình này lặp lại cho đến khi tìm được rule phù hợp hoặc toàn bộ danh sách rule đã được đánh giá.

Trong trường hợp không có rule nào khớp, hệ thống sử dụng giá trị mặc định của Feature Flag và trả về kết quả tương ứng. Việc dừng ngay khi gặp rule đầu tiên phù hợp giúp giảm số lượng phép kiểm tra, đồng thời bảo đảm mọi lần evaluation đều cho kết quả nhất quán với cùng một cấu hình.

Reason code và matched rule

Ngoài trạng thái bật hoặc tắt của Feature Flag, Evaluation API còn trả về *reason code* và thông tin về rule đã được áp dụng (*matched rule*). Reason code phản ánh nguyên nhân dẫn đến kết quả evaluation, chẳng hạn `USER_MATCH`, `ROLE_MATCH` hoặc `PERCENTAGE_MATCH`, trong khi thông tin matched rule cho biết chính xác rule nào đã quyết định kết quả.

Các thông tin này được sử dụng để hỗ trợ gỡ lỗi, kiểm tra cấu hình và đối chiếu với Audit Log trong quá trình vận hành. Đồng thời, Dashboard có thể hiển thị kết quả evaluation một cách trực quan, giúp người quản trị nhanh chóng xác định nguyên nhân khi Feature Flag hoạt động không như mong đợi [10, 4].

3.5.7 Flag kill switch

Kịch bản thứ bảy minh họa khả năng vô hiệu hóa ngay lập tức một Feature Flag thông qua *Kill Switch*. Đây là cơ chế quan trọng trong các nền tảng Feature Flag, cho phép đội vận hành nhanh chóng ngăn chặn một tính năng đang gây ra lỗi mà không cần rollback phiên bản hoặc triển khai lại hệ thống. Trong Rule Evaluation Engine của hệ thống, Kill Switch được kiểm tra trước các rule targeting nhằm bảo đảm mọi yêu cầu evaluation đều bị chặn ngay khi cơ chế này được kích hoạt [1, 8].

Tính năng đang được rollout

Ban đầu, Feature Flag đang hoạt động bình thường và được phát hành theo một trong các chiến lược như Global On, Role Targeting hoặc Percentage Rollout. Demo Application gửi Evaluation Context đến Evaluation API và nhận kết quả evaluation theo đúng cấu hình hiện tại.

Trong giai đoạn này, người dùng thuộc phạm vi rollout vẫn có thể sử dụng tính năng mới. Audit Log đồng thời ghi nhận các lần thay đổi cấu hình trước đó để phục vụ việc theo dõi và truy vết khi cần thiết.

Kích hoạt kill switch

Khi phát hiện sự cố trên môi trường vận hành, người quản trị truy cập Dashboard và bật tùy chọn *Kill Switch* của Feature Flag. Sau khi lưu thay đổi, Backend API cập nhật cấu hình tương ứng và áp dụng ngay cho các yêu cầu evaluation tiếp theo.

Do Kill Switch được kiểm tra ở giai đoạn đầu của quá trình evaluation, Rule Evaluation Engine

không tiếp tục đánh giá Global On, User Allowlist, Role Targeting hay Percentage Rollout. Mọi rule còn lại đều bị bỏ qua cho đến khi Kill Switch được tắt.

Kết quả tắt tức thời

Ngay sau khi Kill Switch được kích hoạt, tất cả yêu cầu gửi đến Evaluation API đều nhận kết quả `enabled = false`. Evaluation Engine trả về reason code tương ứng với trạng thái Kill Switch để giải thích nguyên nhân của kết quả evaluation.

Demo Application nhận phản hồi mới và ngay lập tức ẩn hoặc vô hiệu hóa tính năng đang được bảo vệ bởi Feature Flag. Quá trình này diễn ra hoàn toàn thông qua thay đổi cấu hình tại thời gian chạy, không yêu cầu triển khai lại Backend hoặc Demo Application. Đây là ưu điểm quan trọng của Feature Flag trong việc giảm thời gian xử lý sự cố trên môi trường production [1, 3].

Khôi phục sau sự cố

Sau khi nguyên nhân gây lỗi được xử lý, người quản trị chỉ cần tắt Kill Switch trên Dashboard. Rule Evaluation Engine tiếp tục sử dụng cấu hình ban đầu của Feature Flag và đánh giá lại các rule theo đúng thứ tự ưu tiên đã thiết kế.

Nếu Feature Flag trước đó đang được rollout theo tỷ lệ hoặc theo các rule targeting, người dùng sẽ tiếp tục nhận kết quả evaluation như trước khi kích hoạt Kill Switch. Toàn bộ quá trình bật, tắt và khôi phục đều được ghi nhận trong Audit Log với thông tin actor, thời điểm thực hiện, Request ID cùng dữ liệu trước và sau khi thay đổi, hỗ trợ kiểm toán và truy vết trong quá trình vận hành [12].

3.5.8 Group kill switch

Kịch bản thứ tám minh họa khả năng vô hiệu hóa đồng thời nhiều Feature Flag thông qua *Group Kill Switch*. Thay vì phải kích hoạt Kill Switch cho từng Feature Flag riêng lẻ, người quản trị chỉ cần thao tác trên một nhóm (*Feature Flag Group*) để nhanh chóng dừng toàn bộ các tính năng liên quan. Cơ chế này đặc biệt hữu ích khi nhiều Feature Flag cùng phục vụ một chức năng hoặc một phiên bản phát hành và cần được vô hiệu hóa đồng thời khi phát sinh sự cố [1, 8].

Nhóm nhiều Feature Flag

Trong Dashboard, nhiều Feature Flag có thể được tổ chức vào cùng một nhóm theo chức năng hoặc phạm vi nghiệp vụ. Ví dụ, các Feature Flag liên quan đến một tính năng thương mại điện tử mới đều được đưa vào cùng một Feature Flag Group.

Mỗi Feature Flag vẫn duy trì cấu hình, rule và trạng thái riêng. Tuy nhiên, tất cả đều kế thừa trạng thái Group Kill Switch của nhóm mà chúng thuộc về. Điều này giúp người quản trị quản lý nhiều Feature Flag một cách tập trung mà không làm thay đổi cấu hình chi tiết của từng Feature Flag.

Kích hoạt group kill switch

Khi phát hiện sự cố ảnh hưởng đến toàn bộ nhóm chức năng, người quản trị truy cập Dashboard và kích hoạt Group Kill Switch. Backend API cập nhật trạng thái của nhóm và thực hiện cơ chế đồng bộ để các Feature Flag trong cùng nhóm đều phản ánh trạng thái mới.

Trong quá trình evaluation, Rule Evaluation Engine kiểm tra trạng thái Group Kill Switch trước khi tiếp tục đánh giá cấu hình của từng Feature Flag. Nếu Group Kill Switch đang được bật, Evaluation Engine kết thúc ngay quá trình xử lý và trả về kết quả tắt tính năng mà không tiếp tục đánh giá các rule khác.

Phạm vi ảnh hưởng

Sau khi Group Kill Switch được kích hoạt, tất cả Feature Flag thuộc cùng nhóm đều bị vô hiệu hóa đồng thời. Những Feature Flag không thuộc nhóm này vẫn tiếp tục hoạt động bình thường theo cấu hình hiện có và không chịu ảnh hưởng từ thao tác vừa thực hiện.

Nhờ cơ chế quản lý theo nhóm, người quản trị có thể cô lập nhanh một tập hợp tính năng đang gặp sự cố mà không ảnh hưởng đến các chức năng khác của hệ thống. Điều này giúp giảm phạm vi tác động trong quá trình xử lý sự cố và tăng tính linh hoạt khi vận hành hệ thống Feature Flag.

Audit log và cache invalidation

Mỗi lần bật hoặc tắt Group Kill Switch đều được ghi nhận trong Audit Log với đầy đủ thông tin về actor, thời điểm thực hiện, hành động, Request ID cùng dữ liệu trước và sau khi thay

đổi. Các bản ghi này hỗ trợ truy vết lịch sử cấu hình và xác định nguyên nhân khi xảy ra sự cố trong quá trình vận hành [12].

Đồng thời, hệ thống thực hiện cơ chế *cache invalidation* đối với các Feature Flag thuộc nhóm vừa thay đổi. Việc làm mới cache bảo đảm các lần evaluation tiếp theo sử dụng cấu hình mới nhất thay vì dữ liệu đã được lưu trước đó, từ đó duy trì tính nhất quán giữa Dashboard, Backend API và Demo Application [13].

3.5.9 Audit log và versioning

Kịch bản thứ chín minh họa khả năng ghi nhận lịch sử thay đổi cấu hình của hệ thống thông qua Audit Log và cơ chế lưu phiên bản cấu hình (*versioning*). Mỗi thao tác thay đổi Feature Flag đều được lưu lại dưới dạng một bản ghi độc lập, bao gồm thông tin về đối tượng bị tác động, dữ liệu trước và sau khi thay đổi, người thực hiện và Request ID. Nhờ đó, hệ thống hỗ trợ truy vết, kiểm toán và phân tích nguyên nhân khi phát sinh sự cố trong quá trình vận hành [12].

Thay đổi trạng thái flag

Người quản trị thực hiện các thao tác như bật hoặc tắt Feature Flag, thay đổi Serving Mode hoặc kích hoạt Kill Switch thông qua Dashboard. Sau khi Backend API xử lý thành công yêu cầu, một bản ghi Audit Log mới được tạo để phản ánh trạng thái vừa thay đổi.

Mỗi lần cập nhật được xem như một phiên bản cấu hình mới của Feature Flag. Các bản ghi lịch sử được lưu bổ sung thay vì ghi đè lên dữ liệu cũ, giúp bảo toàn toàn bộ quá trình thay đổi cấu hình trong suốt vòng đời của Feature Flag.

Thay đổi rule

Ngoài trạng thái của Feature Flag, hệ thống còn ghi nhận các thao tác tạo mới, chỉnh sửa hoặc xóa rule như User Allowlist, Role Targeting và Percentage Rollout. Khi một rule được cập nhật, Audit Log lưu lại đầy đủ nội dung trước và sau khi thay đổi.

Việc lưu lịch sử cấu hình giúp người quản trị xác định chính xác thời điểm một rule được thay đổi, đồng thời hỗ trợ khôi phục hoặc đối chiếu cấu hình khi phát hiện kết quả evaluation không như mong đợi.

So sánh before và after snapshot

Mỗi bản ghi Audit Log đều chứa hai ảnh chụp dữ liệu (*snapshot*) gồm trạng thái trước (*before*) và sau (*after*) khi thực hiện thao tác. Dashboard hiển thị đồng thời hai snapshot để người quản trị dễ dàng quan sát những trường dữ liệu đã thay đổi.

Việc lưu snapshot đầy đủ thay vì chỉ ghi nhận tên hành động giúp việc phân tích lịch sử cấu hình trở nên trực quan hơn. Người quản trị có thể nhanh chóng xác định các thuộc tính đã được chỉnh sửa mà không cần truy vấn nhiều phiên bản dữ liệu khác nhau.

Truy vết actor và request ID

Mỗi bản ghi Audit Log đều lưu thông tin về actor thực hiện thao tác cùng Request ID tương ứng với yêu cầu gửi đến Backend API. Actor cho biết người dùng nào đã thực hiện thay đổi, trong khi Request ID hỗ trợ liên kết giữa Audit Log, nhật ký hệ thống và các yêu cầu xử lý của Backend.

Nhờ lưu đầy đủ các thông tin truy vết, hệ thống hỗ trợ kiểm toán, điều tra sự cố và xác định nguyên nhân của các thay đổi cấu hình một cách hiệu quả. Mô hình lưu trữ append-only kết hợp với before và after snapshot giúp bảo đảm tính toàn vẹn của lịch sử thay đổi và đáp ứng các yêu cầu về khả năng truy vết trong quá trình vận hành [12].

3.5.10 Evaluation metrics

Kịch bản thứ mười minh họa khả năng thu thập và thống kê dữ liệu evaluation của hệ thống. Mỗi lần Demo Application gửi yêu cầu đến Evaluation API đều phát sinh một sự kiện evaluation được ghi nhận bởi Backend. Dữ liệu này được tổng hợp theo từng Feature Flag và hiển thị trên Dashboard nhằm hỗ trợ theo dõi mức độ sử dụng, đánh giá hiệu quả rollout và phát hiện các bất thường trong quá trình vận hành.

Phát sinh evaluation event

Mỗi yêu cầu evaluation được xử lý thành công đều tạo ra một bản ghi thống kê. Bản ghi bao gồm các thông tin như Feature Flag được đánh giá, environment, thời điểm thực hiện, kết quả evaluation và reason code tương ứng.

Việc ghi nhận dữ liệu evaluation được thực hiện sau khi Rule Evaluation Engine hoàn thành quá trình đánh giá, bảo đảm số liệu phản ánh đúng kết quả thực tế trả về cho Demo Application.

Cơ chế này giúp hệ thống thu thập dữ liệu vận hành mà không làm thay đổi logic evaluation [10].

Tổng hợp theo flag

Các bản ghi evaluation được Backend API tổng hợp theo từng Feature Flag. Dashboard sử dụng dữ liệu này để hiển thị tổng số lần evaluation của mỗi Feature Flag trong khoảng thời gian tương ứng.

Thông tin thống kê giúp người quản trị nhận biết Feature Flag nào đang được sử dụng nhiều, Feature Flag nào ít được truy cập và hỗ trợ đánh giá mức độ ảnh hưởng của từng tính năng trong hệ thống. Đây cũng là cơ sở để xác định các Feature Flag không còn được sử dụng nhằm giảm thiểu *feature flag debt* [9].

Phân bố kết quả bật và tắt

Ngoài tổng số lần evaluation, hệ thống còn thống kê số lượng kết quả `enabled = true` và `enabled = false`. Tỷ lệ phân bố giữa hai trạng thái phản ánh phạm vi phát hành thực tế của Feature Flag tại thời điểm quan sát.

Đối với các Feature Flag sử dụng Percentage Rollout, dữ liệu thống kê cho phép người quản trị kiểm tra xem tỷ lệ người dùng được bật tính năng có phù hợp với cấu hình rollout hay không. Đồng thời, sự thay đổi đột ngột về tỷ lệ bật hoặc tắt cũng có thể là dấu hiệu của việc thay đổi cấu hình hoặc phát sinh sự cố trong hệ thống.

Hiển thị trên Dashboard

Dashboard cung cấp màn hình thống kê evaluation dưới dạng các chỉ số tổng hợp và biểu đồ theo thời gian. Người quản trị có thể lựa chọn Feature Flag cần quan sát để xem tổng số lần evaluation, số lượng kết quả bật và tắt cùng xu hướng thay đổi theo từng khoảng thời gian.

Thông tin thống kê được cập nhật từ dữ liệu do Backend API tổng hợp, giúp Dashboard phản ánh tình trạng vận hành của Feature Flag theo thời gian thực. Việc trực quan hóa dữ liệu evaluation hỗ trợ người quản trị đánh giá hiệu quả của các chiến lược rollout và đưa ra quyết định điều chỉnh cấu hình một cách phù hợp.

3.6 Kết quả kiểm thử

3.6.1 Kết quả unit test Rule Evaluation Engine

Để bảo đảm Rule Evaluation Engine hoạt động đúng theo thiết kế, đề tài xây dựng bộ unit test bằng Jest cho các thành phần cốt lõi của quá trình đánh giá Feature Flag. Các trường hợp kiểm thử tập trung vào thứ tự ưu tiên (precedence), các chiến lược targeting, cơ chế percentage rollout và các tình huống fallback khi không thể đưa ra kết quả theo luồng thông thường. Việc kiểm thử ở mức đơn vị giúp xác nhận từng nhánh xử lý độc lập trước khi tích hợp với Evaluation API và Dashboard.

Kiểm thử precedence

Thứ tự ưu tiên của Rule Evaluation Engine được kiểm thử nhằm bảo đảm kết quả evaluation luôn xác định và nhất quán khi nhiều điều kiện đồng thời tồn tại. Các trường hợp kiểm thử bao gồm Feature Flag đã bị archive, flag bị vô hiệu hóa, kill switch được kích hoạt, chế độ bật toàn cục (Global On), rule targeting và giá trị mặc định.

Kết quả kiểm thử cho thấy Evaluation Engine luôn áp dụng đúng thứ tự ưu tiên đã thiết kế. Khi một điều kiện có mức ưu tiên cao hơn được thỏa mãn, các rule phía sau sẽ không tiếp tục được đánh giá. Điều này giúp tránh phát sinh kết quả mâu thuẫn và phù hợp với nguyên tắc đánh giá Feature Flag trong các nền tảng hiện đại [1, 7].

Kiểm thử targeting rule

Các unit test tiếp theo kiểm tra khả năng đánh giá các rule targeting dựa trên thông tin trong Evaluation Context. Các trường hợp được kiểm thử gồm user allowlist, role targeting, nhiều role đồng thời và trường hợp không khớp với bất kỳ rule nào.

Kết quả cho thấy Evaluation Engine xác định chính xác rule đầu tiên thỏa mãn điều kiện và trả về trạng thái tương ứng cùng với reason code phù hợp. Khi không tồn tại rule phù hợp, hệ thống tiếp tục đánh giá các bước kế tiếp hoặc trả về giá trị mặc định. Cách xử lý này phù hợp với mô hình Evaluation Context và Rule Evaluation được đề xuất trong OpenFeature cũng như các nền tảng Feature Flag phổ biến [6, 10, 4].

Kiểm thử percentage rollout

Đối với chiến lược rollout theo phần trăm, bộ unit test kiểm tra tính ổn định của thuật toán băm (stable hashing), giới hạn phần trăm và khả năng phân phối người dùng vào các bucket.

Kết quả kiểm thử cho thấy cùng một `targetingKey` luôn được ánh xạ tới cùng một bucket trong các lần evaluation khác nhau, bảo đảm người dùng không thay đổi trạng thái ngẫu nhiên giữa các lần truy cập. Đồng thời, các giá trị rollout khác nhau như 0%, 25%, 50%, 75% và 100% đều tạo ra kết quả đúng với cấu hình mong muốn. Cơ chế này phù hợp với nguyên tắc stable assignment được sử dụng trong các hệ thống Feature Flag thực tế [5].

Kiểm thử fallback

Các trường hợp fallback được xây dựng nhằm kiểm tra khả năng xử lý khi Evaluation Engine không thể xác định kết quả theo luồng thông thường. Bộ kiểm thử bao gồm các tình huống như Feature Flag không tồn tại, Evaluation Context thiếu thông tin cần thiết, không có rule phù hợp hoặc phát sinh lỗi trong quá trình xử lý.

Kết quả cho thấy hệ thống luôn trả về phản hồi hợp lệ với giá trị mặc định an toàn (fail-safe), đồng thời sinh reason code tương ứng để hỗ trợ việc ghi log và truy vết. Cơ chế fallback này giúp ứng dụng khách duy trì hoạt động ổn định ngay cả khi gặp lỗi hoặc dữ liệu không đầy đủ, phù hợp với các khuyến nghị về Feature Flag và Evaluation API [1, 10, 8].

3.6.2 Kết quả kiểm thử API

Sau khi hoàn thành kiểm thử ở mức đơn vị, hệ thống tiếp tục được kiểm thử ở mức API nhằm đánh giá tính đúng đắn của các endpoint, khả năng xử lý dữ liệu đầu vào, phản hồi lỗi và sự phối hợp giữa các thành phần Backend API, cơ sở dữ liệu và Rule Evaluation Engine. Các kịch bản kiểm thử được thực hiện thông qua Swagger UI và các yêu cầu HTTP với dữ liệu mẫu đã được seed trước. Kết quả cho thấy các API hoạt động ổn định, trả về mã trạng thái phù hợp và đáp ứng đúng đặc tả đã thiết kế.

API quản trị

Nhóm API quản trị bao gồm các chức năng quản lý project, environment, feature flag, rule, flag group và sample user. Các trường hợp kiểm thử bao gồm tạo mới, cập nhật, truy vấn, archive, khôi phục và các thao tác thay đổi cấu hình.

Kết quả kiểm thử cho thấy dữ liệu được kiểm tra hợp lệ trước khi ghi xuống cơ sở dữ liệu. Các trường hợp khóa trùng, tham số không hợp lệ hoặc thiếu dữ liệu đều trả về phản hồi lỗi thống nhất cùng request ID để hỗ trợ truy vết. Đối với các thao tác thay đổi cấu hình, hệ thống đồng thời ghi nhận audit log và thực hiện cache invalidation khi cần thiết, bảo đảm dữ liệu phục vụ evaluation luôn được cập nhật.

Evaluation API

Evaluation API được kiểm thử với đầy đủ các kịch bản như bật toàn cục, user targeting, role targeting, percentage rollout, group kill switch, Feature Flag không tồn tại và các trường hợp fallback.

Kết quả kiểm thử cho thấy API luôn trả về đầy đủ thông tin gồm trạng thái bật hoặc tắt, variant, reason code và thông tin rule được áp dụng khi có. Các request có cùng `targetingKey` luôn nhận được kết quả nhất quán trong kịch bản percentage rollout, trong khi các trường hợp lỗi vẫn trả về giá trị mặc định an toàn thay vì phát sinh ngoại lệ cho ứng dụng khách. Kết quả này phù hợp với mô hình Evaluation API và Evaluation Context được đề xuất trong OpenFeature [10, 6].

Audit log API

API audit log được kiểm thử nhằm xác nhận khả năng lưu vết toàn bộ các thao tác thay đổi cấu hình trong hệ thống. Các trường hợp kiểm thử bao gồm tạo, cập nhật, archive, khôi phục Feature Flag, thay đổi rule, thay đổi trạng thái và thao tác trên group kill switch.

Kết quả kiểm thử cho thấy mỗi thao tác đều tạo một bản ghi audit mới với đầy đủ thông tin về actor, action, timestamp, request ID cùng before và after snapshot. Dữ liệu audit được lưu theo cơ chế append-only, cho phép truy vết lịch sử cấu hình và hỗ trợ phân tích nguyên nhân khi xảy ra sự cố. Cách triển khai này phù hợp với các khuyến nghị về logging và audit trong các hệ thống quản trị hiện đại [12].

Statistics API

Statistics API được kiểm thử nhằm đánh giá khả năng tổng hợp dữ liệu phục vụ Dashboard. Các trường hợp kiểm thử bao gồm thống kê tổng số lần evaluation, thống kê theo Feature Flag, phân bố kết quả bật hoặc tắt và dữ liệu biểu đồ theo thời gian.

Kết quả kiểm thử cho thấy dữ liệu thống kê được cập nhật chính xác sau mỗi lần evaluation và có thể truy vấn theo nhiều tiêu chí khác nhau. Các endpoint trả về dữ liệu đúng định dạng để Dashboard trực tiếp hiển thị biểu đồ và báo cáo mà không cần xử lý bổ sung ở phía giao diện. Điều này giúp hệ thống hỗ trợ theo dõi hiệu quả sử dụng Feature Flag cũng như đánh giá phạm vi tác động của từng cấu hình trong quá trình vận hành.

3.6.3 Kết quả kiểm thử validation và error handling

Bên cạnh việc kiểm tra chức năng nghiệp vụ, hệ thống còn được kiểm thử đối với các trường hợp dữ liệu không hợp lệ và các tình huống lỗi trong quá trình xử lý. Mục tiêu của các kịch bản này là bảo đảm Backend API luôn phản hồi theo định dạng thống nhất, không làm gián đoạn dịch vụ và cung cấp đầy đủ thông tin để hỗ trợ việc gỡ lỗi và truy vết. Kết quả kiểm thử cho thấy các cơ chế validation và xử lý lỗi hoạt động đúng theo thiết kế.

Validation key và enum

Các trường hợp kiểm thử bao gồm project key, environment key, feature flag key không đúng định dạng, giá trị enum không hợp lệ và các trường dữ liệu bắt buộc bị thiếu.

Kết quả kiểm thử cho thấy toàn bộ dữ liệu đầu vào đều được kiểm tra trước khi xử lý nghiệp vụ. Khi phát hiện dữ liệu không hợp lệ, API trả về mã lỗi tương ứng cùng danh sách các trường vi phạm và request ID để hỗ trợ truy vết. Việc kiểm tra dữ liệu đầu vào giúp hạn chế lỗi phát sinh trong các tầng xử lý phía sau và nâng cao tính ổn định của hệ thống.

Validation rule parameters

Các kịch bản kiểm thử tập trung vào các tham số cấu hình rule như danh sách user, role, giá trị percentage rollout và cấu trúc của các tham số cấu hình.

Kết quả kiểm thử cho thấy hệ thống từ chối các cấu hình không hợp lệ như tỷ lệ rollout ngoài khoảng cho phép, tham số thiếu dữ liệu hoặc kiểu dữ liệu không đúng. Chỉ những rule đáp ứng đầy đủ các ràng buộc mới được lưu vào cơ sở dữ liệu, qua đó bảo đảm Rule Evaluation Engine luôn hoạt động trên tập dữ liệu hợp lệ.

Not found và conflict

Nhóm kiểm thử này đánh giá khả năng xử lý khi tài nguyên không tồn tại hoặc xảy ra xung đột dữ liệu. Các trường hợp bao gồm truy vấn Feature Flag không tồn tại, cập nhật đối tượng đã bị archive, tạo mới với khóa đã tồn tại hoặc tham chiếu đến project và environment không hợp lệ.

Kết quả kiểm thử cho thấy API trả về đúng mã lỗi theo từng tình huống, đồng thời cung cấp thông báo rõ ràng để phía ứng dụng khách có thể xử lý phù hợp. Việc phân biệt giữa lỗi không tìm thấy và lỗi xung đột giúp quá trình tích hợp với Dashboard và Demo Application trở nên đơn giản và nhất quán.

Internal error

Các trường hợp kiểm thử internal error được thực hiện bằng cách mô phỏng lỗi trong quá trình truy cập cơ sở dữ liệu hoặc phát sinh ngoại lệ ngoài dự kiến trong tầng nghiệp vụ.

Kết quả kiểm thử cho thấy mọi ngoại lệ đều được bộ xử lý lỗi toàn cục tiếp nhận và chuyển đổi thành phản hồi thống nhất mà không làm lộ thông tin nội bộ của hệ thống. Dồng thời, request ID được trả về cho ứng dụng khách để đối chiếu với audit log và log của máy chủ khi cần điều tra nguyên nhân. Cách xử lý này phù hợp với các khuyến nghị về bảo mật API và quản lý lỗi trong các hệ thống dịch vụ hiện đại [11, 12].

3.6.4 Kết quả kiểm thử stable rollout

Một trong những yêu cầu quan trọng của Feature Flag là bảo đảm người dùng nhận được kết quả nhất quán trong các lần evaluation khi sử dụng chiến lược percentage rollout. Để đánh giá cơ chế này, hệ thống được kiểm thử với thuật toán stable hashing dựa trên `targetingKey`, đồng thời quan sát sự phân bố bucket và hành vi khi thay đổi tỷ lệ rollout. Kết quả kiểm thử cho thấy cơ chế stable rollout hoạt động đúng theo thiết kế và đáp ứng yêu cầu về tính ổn định trong môi trường thực tế.

Độ ổn định theo targeting key

Kịch bản kiểm thử được thực hiện bằng cách gửi nhiều request với cùng một `targetingKey` trong các thời điểm khác nhau nhưng giữ nguyên cấu hình Feature Flag.

Kết quả cho thấy cùng một `targetingKey` luôn được ánh xạ tới cùng một bucket và nhận được

cùng một kết quả evaluation trong tất cả các lần kiểm thử. Điều này bảo đảm người dùng không bị thay đổi trạng thái Feature Flag một cách ngẫu nhiên giữa các lần truy cập, phù hợp với nguyên tắc stable assignment của các nền tảng Feature Flag hiện đại [5].

Phân bố bucket

Để đánh giá tính đồng đều của thuật toán băm, hệ thống tiến hành kiểm thử với tập dữ liệu gồm nhiều `targetingKey` khác nhau và thống kê bucket tương ứng.

Kết quả cho thấy các bucket được phân bố tương đối đồng đều trên toàn bộ miền giá trị, không xuất hiện hiện tượng tập trung bất thường vào một khoảng nhất định. Nhờ đó, tỷ lệ người dùng được bật Feature Flag gần với tỷ lệ rollout đã cấu hình, giúp các đợt phát hành từng phần phản ánh đúng quy mô mong muốn.

Tăng tỷ lệ rollout

Kịch bản kiểm thử được thực hiện bằng cách tăng dần giá trị rollout, ví dụ từ 10% lên 30%, 50%, 75% và 100%, đồng thời theo dõi tập người dùng được bật Feature Flag.

Kết quả kiểm thử cho thấy tất cả người dùng đã được bật ở mức rollout thấp hơn vẫn tiếp tục được bật khi tăng tỷ lệ rollout. Hệ thống chỉ bổ sung thêm các bucket mới thuộc phạm vi mở rộng mà không thay đổi trạng thái của các bucket đã được chọn trước đó. Điều này giúp quá trình phát hành diễn ra liên tục và hạn chế ảnh hưởng đến trải nghiệm của người dùng [5].

Giảm tỷ lệ rollout

Đối với trường hợp giảm tỷ lệ rollout, hệ thống lần lượt giảm giá trị cấu hình và quan sát các bucket còn nằm trong phạm vi được phục vụ.

Kết quả kiểm thử cho thấy chỉ các bucket nằm ngoài ngưỡng rollout mới bị loại bỏ, trong khi các bucket vẫn thuộc phạm vi cấu hình tiếp tục giữ nguyên trạng thái. Cơ chế này bảo đảm việc thu hẹp phạm vi phát hành diễn ra có kiểm soát, đồng thời duy trì tính nhất quán của kết quả evaluation đối với những người dùng vẫn còn trong tập rollout.

3.6.5 Kết quả kiểm thử cache

Nhằm giảm độ trễ của Evaluation API và hạn chế số lần truy cập cơ sở dữ liệu, hệ thống triển khai cơ chế cache cho dữ liệu phục vụ quá trình evaluation. Các kịch bản kiểm thử tập trung

đánh giá khả năng tận dụng cache, cơ chế invalidation khi cấu hình thay đổi và hành vi của hệ thống khi cache không khả dụng. Kết quả kiểm thử cho thấy cơ chế cache hoạt động đúng theo thiết kế và không làm ảnh hưởng đến tính chính xác của kết quả evaluation.

Cache hit và cache miss

Kịch bản kiểm thử được thực hiện bằng cách gửi liên tiếp nhiều request evaluation với cùng project, environment và Feature Flag. Lần truy cập đầu tiên được sử dụng để tạo dữ liệu trong cache, các lần tiếp theo kiểm tra khả năng tái sử dụng dữ liệu đã lưu.

Kết quả cho thấy request đầu tiên xảy ra cache miss và tải dữ liệu từ cơ sở dữ liệu trước khi ghi vào cache. Các request tiếp theo sử dụng dữ liệu đã được lưu, giảm số lần truy cập cơ sở dữ liệu trong khi vẫn trả về kết quả evaluation giống nhau. Điều này góp phần cải thiện thời gian phản hồi của Evaluation API và phù hợp với mục tiêu tối ưu hiệu năng của hệ thống [13].

Invalidation khi cập nhật flag

Sau khi thay đổi trạng thái hoặc cấu hình của Feature Flag, hệ thống tiến hành kiểm thử khả năng làm mới dữ liệu trong cache.

Kết quả kiểm thử cho thấy cache liên quan đến Feature Flag được xóa ngay sau khi thao tác cập nhật hoàn tất. Request evaluation kế tiếp thực hiện tải lại dữ liệu mới từ cơ sở dữ liệu và ghi trở lại cache, bảo đảm các thay đổi được phản ánh ngay trong kết quả evaluation mà không sử dụng dữ liệu cũ.

Invalidation khi cập nhật rule

Kịch bản kiểm thử tiếp tục được thực hiện đối với các thao tác thêm, sửa hoặc xóa rule của Feature Flag.

Kết quả cho thấy mỗi lần cấu hình rule thay đổi, dữ liệu cache tương ứng đều được invalidation trước khi phục vụ các request mới. Nhờ đó, Rule Evaluation Engine luôn đánh giá trên tập cấu hình mới nhất và tránh tình trạng sử dụng rule đã hết hiệu lực. Cơ chế này giúp duy trì tính nhất quán giữa dữ liệu trong cache và cơ sở dữ liệu.

Hành vi khi cache lỗi

Để đánh giá khả năng chịu lỗi, hệ thống mô phỏng các trường hợp cache không khả dụng hoặc phát sinh lỗi trong quá trình truy cập.

Kết quả kiểm thử cho thấy Evaluation API vẫn hoạt động bình thường bằng cách truy vấn trực tiếp cơ sở dữ liệu thay vì phụ thuộc hoàn toàn vào cache. Mặc dù thời gian phản hồi có thể tăng so với trường hợp cache hoạt động, kết quả evaluation vẫn chính xác và không ảnh hưởng đến chức năng của ứng dụng. Cách triển khai này bảo đảm cache chỉ đóng vai trò tối ưu hiệu năng, không trở thành điểm lỗi duy nhất của hệ thống [13, 8].

3.6.6 Kết quả kiểm thử phân quyền

Hệ thống được kiểm thử cơ chế phân quyền nhằm bảo đảm mỗi nhóm người dùng chỉ được thực hiện các thao tác phù hợp với vai trò được cấp. Các kịch bản kiểm thử bao gồm quyền của `admin`, `developer`, `viewer` và các trường hợp truy cập trái phép. Kết quả cho thấy Backend API và Dashboard phối hợp thực hiện kiểm soát quyền một cách nhất quán, đồng thời ngăn chặn các thao tác không hợp lệ ngay từ tầng giao diện và tầng dịch vụ.

Quyền admin

Vai trò `admin` được kiểm thử với đầy đủ các chức năng quản trị như tạo mới, cập nhật, archive và khôi phục Feature Flag, cấu hình rule, quản lý group, kích hoạt group kill switch, xem audit log và truy vấn dữ liệu thống kê.

Kết quả kiểm thử cho thấy người dùng có vai trò `admin` được phép thực hiện toàn bộ các thao tác quản trị theo thiết kế. Sau mỗi thao tác thay đổi cấu hình, hệ thống đồng thời ghi nhận audit log, thực hiện cache invalidation khi cần thiết và cập nhật dữ liệu phục vụ Dashboard, bảo đảm tính nhất quán của toàn bộ hệ thống.

Quyền developer

Vai trò `developer` được kiểm thử với các chức năng theo phạm vi được cấp quyền như truy vấn thông tin Feature Flag, xem cấu hình, thực hiện evaluation và sử dụng các chức năng phục vụ phát triển, đồng thời kiểm tra khả năng bị từ chối đối với các thao tác chỉ dành cho quản trị viên.

Kết quả kiểm thử cho thấy các thao tác vượt quá quyền hạn đều bị Backend API từ chối và

Dashboard đồng thời ẩn hoặc vô hiệu hóa các nút chức năng tương ứng. Cơ chế này giúp hạn chế thao tác sai và bảo đảm chỉ người dùng có quyền phù hợp mới có thể thay đổi cấu hình của hệ thống.

Quyền viewer

Vai trò `viewer` được kiểm thử với các chức năng chỉ đọc như xem danh sách project, Feature Flag, rule, audit log và dữ liệu thống kê.

Kết quả kiểm thử cho thấy người dùng chỉ có thể truy cập các API phục vụ truy vấn thông tin, trong khi toàn bộ các endpoint thay đổi dữ liệu đều bị từ chối. Trên Dashboard, các chức năng tạo mới, chỉnh sửa, archive và cấu hình rule đều được ẩn hoặc chuyển sang trạng thái vô hiệu hóa, giúp giao diện phản ánh đúng quyền của người sử dụng.

Truy cập trái phép

Nhóm kiểm thử cuối cùng đánh giá khả năng xử lý các yêu cầu không hợp lệ như người dùng chưa xác thực, người dùng không có quyền hoặc cố gắng gọi trực tiếp các endpoint quản trị thông qua HTTP client.

Kết quả kiểm thử cho thấy Backend API luôn kiểm tra quyền trước khi thực hiện nghiệp vụ. Các request không đáp ứng yêu cầu xác thực hoặc phân quyền đều bị từ chối với mã trạng thái và thông báo lỗi phù hợp, đồng thời không làm thay đổi dữ liệu trong cơ sở dữ liệu cũng như không tạo ra các thao tác quản trị ngoài ý muốn. Cách triển khai này phù hợp với các khuyến nghị về bảo vệ REST API và kiểm soát truy cập của OWASP [11].

3.6.7 Kết quả kiểm thử triển khai

Sau khi hoàn thành quá trình phát triển và kiểm thử chức năng, hệ thống tiếp tục được kiểm thử ở mức triển khai nhằm đánh giá khả năng xây dựng, khởi chạy và vận hành đồng thời các thành phần của nền tảng Feature Flag. Các kịch bản kiểm thử bao gồm quá trình build các service, triển khai bằng Docker Compose, kết nối giữa các thành phần và khởi tạo dữ liệu mẫu. Kết quả cho thấy hệ thống có thể được triển khai thành công trong môi trường mới với quy trình cài đặt đơn giản và ổn định.

Build các service

Các service của hệ thống bao gồm Backend API, Dashboard và Demo Application được kiểm thử thông qua quá trình build từ mã nguồn.

Kết quả kiểm thử cho thấy toàn bộ các service đều được biên dịch thành công, không phát sinh lỗi phụ thuộc hoặc lỗi cấu hình trong quá trình build. Các artifact được tạo đúng theo cấu hình triển khai và sẵn sàng sử dụng trong môi trường Docker, đáp ứng yêu cầu đóng gói hệ thống để triển khai trên nhiều môi trường khác nhau.

Khởi chạy Docker Compose

Kịch bản kiểm thử tiếp theo sử dụng Docker Compose để khởi tạo đồng thời các container của hệ thống, bao gồm cơ sở dữ liệu PostgreSQL, Backend API, Dashboard và Demo Application.

Kết quả kiểm thử cho thấy các container được tạo và khởi động thành công theo đúng thứ tự phụ thuộc. Docker Compose tự động thiết lập mạng nội bộ, khởi tạo các service cần thiết và duy trì trạng thái hoạt động ổn định trong suốt quá trình kiểm thử. Việc triển khai bằng Docker Compose giúp đơn giản hóa quá trình cài đặt và bảo đảm môi trường thực thi nhất quán giữa các lần triển khai [14].

Kết nối giữa các service

Sau khi các container được khởi chạy, hệ thống tiếp tục được kiểm thử khả năng giao tiếp giữa các thành phần thông qua mạng Docker.

Kết quả kiểm thử cho thấy Backend API kết nối thành công với PostgreSQL để truy cập dữ liệu, Dashboard có thể gọi các API quản trị và Demo Application có thể gửi request tới Evaluation API để lấy kết quả Feature Flag. Quá trình trao đổi dữ liệu giữa các service diễn ra ổn định và không xuất hiện lỗi kết nối trong các kịch bản kiểm thử.

Khởi tạo dữ liệu mẫu

Kịch bản kiểm thử cuối cùng đánh giá quá trình khởi tạo dữ liệu mẫu phục vụ việc trình diễn và kiểm thử hệ thống.

Kết quả kiểm thử cho thấy các migration được thực thi thành công trước khi seed dữ liệu. Hệ thống tạo đầy đủ project, environment, Feature Flag, rule, group, sample user và các dữ liệu

liên quan theo đúng cấu trúc đã thiết kế. Sau khi khởi tạo, Dashboard và Demo Application có thể sử dụng ngay các dữ liệu này để thực hiện các kịch bản bật hoặc tắt toàn cục, user targeting, role targeting, percentage rollout, group kill switch và theo dõi audit log mà không cần cấu hình thủ công.

3.7 Đánh giá kết quả thực hiện

3.7.1 Mức độ đáp ứng yêu cầu MVP

Đề tài được xây dựng theo các yêu cầu bắt buộc của Mini Project Feature Flag Platform, bao gồm báo cáo nghiên cứu, Backend API, Dashboard quản trị, Demo Application và các yêu cầu kỹ thuật tối thiểu. Kết quả thực hiện cho thấy hệ thống đáp ứng đầy đủ các chức năng cốt lõi của một nền tảng Feature Flag, đồng thời triển khai thêm nhiều chức năng thuộc mức khuyến khích như caching, JavaScript SDK, phân quyền, thống kê evaluation, group kill switch và triển khai bằng Docker Compose. Nhìn chung, các mục tiêu đặt ra ở giai đoạn MVP đều đã được hoàn thành.

Báo cáo nghiên cứu

Báo cáo trình bày đầy đủ cơ sở lý thuyết về Feature Flag, bao gồm khái niệm, vai trò trong quy trình phát triển phần mềm, sự khác biệt giữa deploy và release, các loại Feature Flag phổ biến, kiến trúc hệ thống, mô hình dữ liệu, Rule Evaluation Engine, thiết kế API, Dashboard quản trị, Demo Application và các phương pháp kiểm thử.

Ngoài việc mô tả quá trình thiết kế và triển khai, báo cáo còn phân tích các vấn đề như rollout theo phần trăm, kill switch, audit log, caching, bảo mật và khả năng mở rộng của hệ thống dựa trên các tài liệu tham khảo uy tín như Martin Fowler, Unleash, LaunchDarkly, OpenFeature và Microsoft [1, 8, 7, 3].

Backend API

Backend API đáp ứng đầy đủ các chức năng quản lý project, environment, Feature Flag, rule, group, sample user, audit log và Evaluation API theo yêu cầu của đề tài. Rule Evaluation Engine hỗ trợ các chiến lược bật hoặc tắt toàn cục, user targeting, role targeting, percentage rollout và group kill switch với thứ tự ưu tiên rõ ràng.

Bên cạnh các chức năng nghiệp vụ, hệ thống còn triển khai validation dữ liệu đầu vào, xử lý lỗi thống nhất, request ID, audit log, cache, thống kê evaluation và cơ chế phân quyền quản trị. Kiến trúc Backend được tổ chức theo hướng module hóa, giúp thuận lợi cho việc bảo trì và mở rộng trong tương lai.

Frontend Dashboard

Dashboard cung cấp đầy đủ các màn hình phục vụ quản trị Feature Flag, bao gồm quản lý project, environment, Feature Flag, rule, group, sample user, audit log và thống kê evaluation. Giao diện thể hiện rõ trạng thái lifecycle, runtime, serving mode, quyền của người dùng và các thao tác quản trị tương ứng.

Việc tích hợp trực tiếp với Backend API giúp Dashboard phản ánh ngay các thay đổi cấu hình, đồng thời hỗ trợ theo dõi lịch sử thay đổi và thống kê số lần evaluation thông qua các biểu đồ trực quan.

Demo Application

Demo Application tích hợp trực tiếp với Evaluation API thông qua JavaScript SDK và minh họa rõ cơ chế hoạt động của Feature Flag trong ứng dụng thực tế. Hệ thống hỗ trợ các kịch bản bật hoặc tắt toàn cục, user targeting, role targeting, percentage rollout, group kill switch và các tình huống fallback khi evaluation không thành công.

Các kịch bản trình diễn cho thấy việc thay đổi cấu hình Feature Flag trên Dashboard được phản ánh ngay trong ứng dụng mà không cần triển khai lại phiên bản mới, phù hợp với mục tiêu tách biệt giữa quá trình deploy và release của Feature Flag [1, 3].

Yêu cầu kỹ thuật tối thiểu

Hệ thống đáp ứng đầy đủ các yêu cầu kỹ thuật tối thiểu của đề tài. Dữ liệu được lưu trữ trong PostgreSQL thông qua Prisma ORM; toàn bộ API đều có validation và xử lý lỗi thống nhất; dữ liệu mẫu được khởi tạo bằng migration và seed; tài liệu thiết kế, README và API specification được xây dựng đầy đủ.

Ngoài các yêu cầu bắt buộc, hệ thống còn triển khai nhiều chức năng mở rộng như caching, JavaScript SDK, unit test cho Rule Evaluation Engine, phân quyền người dùng, thống kê evaluation, group kill switch và Docker Compose. Các chức năng này góp phần nâng cao tính hoàn

thiện của nền tảng và tiệm cận hơn với mô hình Feature Flag được áp dụng trong các hệ thống thực tế [8, 7].

3.7.2 Mức độ đáp ứng yêu cầu khuyến khích

Bên cạnh việc hoàn thành toàn bộ các yêu cầu bắt buộc, hệ thống còn triển khai đầy đủ các chức năng thuộc mức khuyến khích của Mini Project. Các chức năng này góp phần nâng cao hiệu năng, khả năng mở rộng, tính bảo trì và trải nghiệm quản trị của nền tảng Feature Flag. Kết quả thực hiện cho thấy hệ thống không chỉ đáp ứng yêu cầu của đề tài mà còn tiệm cận nhiều đặc điểm của các nền tảng Feature Flag hiện đại.

Cache và SDK

Hệ thống triển khai cơ chế cache nhằm giảm số lần truy cập cơ sở dữ liệu trong quá trình evaluation. Dữ liệu cache được tự động invalidation sau các thao tác thay đổi Feature Flag hoặc Rule, bảo đảm Evaluation Engine luôn sử dụng cấu hình mới nhất. Đồng thời, hệ thống vẫn duy trì khả năng hoạt động khi cache không khả dụng bằng cách truy vấn trực tiếp cơ sở dữ liệu.

Bên cạnh đó, một JavaScript SDK đơn giản được xây dựng để hỗ trợ Demo Application gọi Evaluation API và xử lý kết quả một cách thuận tiện. Việc bổ sung SDK giúp giảm sự phụ thuộc của ứng dụng khách vào chi tiết giao thức HTTP và phù hợp với cách các nền tảng Feature Flag thực tế cung cấp SDK cho nhiều môi trường phát triển khác nhau [8, 7, 13].

Unit test và versioning

Rule Evaluation Engine được kiểm thử bằng các bộ unit test bao phủ các kịch bản quan trọng như precedence, user targeting, role targeting, percentage rollout và fallback. Việc kiểm thử ở mức đơn vị giúp phát hiện sớm các lỗi logic và bảo đảm tính đúng đắn của thuật toán evaluation trước khi tích hợp với các thành phần khác.

Ngoài ra, hệ thống còn triển khai cơ chế lưu vết thay đổi cấu hình thông qua audit log với before và after snapshot, actor, request ID và thời điểm thực hiện. Mặc dù không xây dựng cơ chế khôi phục theo từng phiên bản cấu hình, lịch sử thay đổi vẫn được lưu đầy đủ để phục vụ truy vết và kiểm tra các thay đổi trong quá trình vận hành.

Phân quyền và thống kê

Hệ thống triển khai mô hình phân quyền với ba vai trò gồm `admin`, `developer` và `viewer`. Quyền truy cập được kiểm soát đồng thời tại Backend API và Dashboard nhằm ngăn chặn các thao tác không hợp lệ cũng như bảo đảm mỗi người dùng chỉ thực hiện các chức năng phù hợp với vai trò của mình.

Song song với đó, hệ thống ghi nhận các sự kiện evaluation và cung cấp các API thống kê phục vụ Dashboard. Người quản trị có thể theo dõi tổng số lần evaluation, thống kê theo Feature Flag, phân bố kết quả bật hoặc tắt và biểu đồ theo thời gian để đánh giá mức độ sử dụng của từng Feature Flag trong quá trình vận hành.

Group kill switch và Docker Compose

Một trong những chức năng mở rộng nổi bật của hệ thống là group kill switch, cho phép quản trị viên nhanh chóng vô hiệu hóa nhiều Feature Flag thuộc cùng một nhóm chỉ bằng một thao tác. Cơ chế này đặc biệt hữu ích khi cần giảm nhanh phạm vi ảnh hưởng của sự cố trên môi trường production và phù hợp với các khuyến nghị về vận hành Feature Flag trong thực tế [1, 8].

Ngoài ra, toàn bộ hệ thống được đóng gói và triển khai bằng Docker Compose, cho phép khởi tạo đồng thời PostgreSQL, Backend API, Dashboard và Demo Application trong một môi trường thống nhất. Việc sử dụng Docker Compose giúp đơn giản hóa quá trình cài đặt, giảm khác biệt giữa các môi trường phát triển và tạo thuận lợi cho việc trình diễn cũng như mở rộng hệ thống trong tương lai [14].

3.7.3 Đánh giá Rule Evaluation Engine

Rule Evaluation Engine là thành phần trung tâm của nền tảng Feature Flag, chịu trách nhiệm xác định trạng thái của từng Feature Flag dựa trên cấu hình và Evaluation Context. Kết quả triển khai cho thấy Engine đáp ứng các yêu cầu về tính đúng đắn, tính xác định, khả năng giải thích kết quả và khả năng mở rộng. Thiết kế theo hướng tách biệt giữa dữ liệu cấu hình và thuật toán đánh giá giúp hệ thống dễ bảo trì, đồng thời tạo nền tảng cho việc bổ sung các chiến lược rollout mới trong tương lai.

Tính đúng đắn

Rule Evaluation Engine được xây dựng với thứ tự ưu tiên rõ ràng giữa các trạng thái runtime, bao gồm lifecycle, trạng thái kích hoạt, group kill switch, Global On, các rule targeting và giá trị mặc định. Mỗi request evaluation đều được xử lý theo cùng một quy trình, bảo đảm kết quả phản ánh chính xác cấu hình hiện tại của Feature Flag.

Kết quả kiểm thử cho thấy các chiến lược user targeting, role targeting và percentage rollout đều hoạt động đúng theo thiết kế. Các trường hợp Feature Flag không tồn tại, cấu hình không hợp lệ hoặc không có rule phù hợp đều được xử lý bằng cơ chế fallback an toàn, phù hợp với nguyên tắc triển khai Feature Flag trong các hệ thống thực tế [1, 10].

Tính xác định

Một yêu cầu quan trọng của Rule Evaluation Engine là cùng một Evaluation Context phải luôn tạo ra cùng một kết quả khi cấu hình Feature Flag không thay đổi. Hệ thống đáp ứng yêu cầu này thông qua thuật toán stable hashing sử dụng `targetingKey` để xác định bucket trong các kịch bản percentage rollout.

Nhờ cơ chế trên, cùng một người dùng luôn nhận được kết quả nhất quán giữa các lần evaluation, đồng thời việc tăng hoặc giảm tỷ lệ rollout chỉ mở rộng hoặc thu hẹp tập bucket mà không làm thay đổi ngẫu nhiên trạng thái của các bucket đã được xác định trước. Điều này bảo đảm tính ổn định của quá trình phát hành từng phần và phù hợp với các nền tảng Feature Flag hiện đại [5].

Khả năng giải thích kết quả

Ngoài việc trả về trạng thái bật hoặc tắt, Rule Evaluation Engine còn cung cấp các thông tin như `reason`, `variant` và rule được áp dụng trong quá trình evaluation. Các thông tin này giúp Dashboard, Demo Application và nhà phát triển xác định nhanh nguyên nhân dẫn đến kết quả evaluation.

Khả năng giải thích kết quả đặc biệt hữu ích trong quá trình kiểm thử, vận hành và xử lý sự cố, khi người quản trị cần xác định Feature Flag được kích hoạt bởi Global On, targeting rule, percentage rollout hay cơ chế fallback. Thiết kế này phù hợp với mô hình Evaluation API của OpenFeature và nâng cao khả năng quan sát của toàn bộ hệ thống [10, 6].

Khả năng mở rộng

Rule Evaluation Engine được thiết kế theo hướng tách biệt giữa dữ liệu cấu hình, các loại rule và thuật toán đánh giá. Nhờ đó, việc bổ sung các chiến lược mới như targeting theo thuộc tính, organization, thời gian, vị trí hoặc hỗ trợ nhiều variant có thể được thực hiện mà không làm thay đổi đáng kể kiến trúc hiện có.

Bên cạnh đó, việc kết hợp giữa audit log, cache, phân quyền và Evaluation API tạo nên một nền tảng thống nhất, thuận lợi cho việc mở rộng quy mô hệ thống cũng như tích hợp với nhiều ứng dụng khách trong tương lai. Cách tiếp cận này phù hợp với các khuyến nghị về thiết kế và phát triển hệ thống Feature Flag có khả năng mở rộng [8, 7].

3.7.4 Đánh giá Backend API

Backend API là thành phần trung tâm kết nối Dashboard, Rule Evaluation Engine và Demo Application. Kết quả triển khai cho thấy hệ thống được tổ chức theo kiến trúc module rõ ràng, cung cấp các endpoint phục vụ quản trị và evaluation với giao diện thống nhất. Bên cạnh việc đáp ứng các yêu cầu chức năng, Backend API còn chú trọng đến validation, xử lý lỗi, tài liệu hóa và khả năng mở rộng nhằm hỗ trợ quá trình phát triển và bảo trì lâu dài.

Cấu trúc API

Các endpoint được tổ chức theo từng nhóm chức năng như project, environment, Feature Flag, rule, group, sample user, evaluation, audit log và statistics. Cách phân chia này giúp tách biệt rõ trách nhiệm của từng module, đồng thời thuận lợi cho việc mở rộng khi bổ sung các chức năng mới.

Định dạng request và response được chuẩn hóa trên toàn bộ hệ thống, giúp Dashboard và Demo Application dễ dàng tích hợp với Backend API. Việc tách riêng Evaluation API khỏi các API quản trị cũng phù hợp với mô hình triển khai Feature Flag trong thực tế, nơi việc đánh giá Feature Flag và quản lý cấu hình thường được thực hiện bởi các thành phần độc lập [7, 10].

Validation và xử lý lỗi

Backend API áp dụng cơ chế validation cho toàn bộ dữ liệu đầu vào nhằm kiểm tra định dạng key, giá trị enum, tham số của rule và Evaluation Context trước khi xử lý nghiệp vụ. Điều này giúp phát hiện sớm các lỗi và ngăn chặn dữ liệu không hợp lệ được lưu vào cơ sở dữ liệu.

Đối với các trường hợp lỗi, hệ thống sử dụng cơ chế xử lý ngoại lệ tập trung để trả về phản hồi thống nhất, bao gồm mã lỗi, thông báo, chi tiết lỗi và request ID phục vụ truy vết. Cách tiếp cận này giúp giảm sự phụ thuộc giữa các module, đồng thời phù hợp với các khuyến nghị về thiết kế REST API và xử lý lỗi an toàn [11].

Tài liệu hóa

Backend API được tài liệu hóa thông qua OpenAPI và Swagger, cho phép mô tả đầy đủ các endpoint, tham số đầu vào, cấu trúc dữ liệu, phản hồi thành công và các trường hợp lỗi.

Việc cung cấp tài liệu API giúp giảm thời gian tích hợp giữa Backend với Dashboard và Demo Application, đồng thời hỗ trợ quá trình kiểm thử và phát triển các ứng dụng khách trong tương lai. Đây cũng là nền tảng thuận lợi để mở rộng hệ thống hoặc tích hợp với các dịch vụ bên ngoài mà không cần phụ thuộc vào việc đọc trực tiếp mã nguồn.

Khả năng bảo trì

Backend API được xây dựng theo kiến trúc module hóa với sự tách biệt giữa controller, service, repository, DTO và Rule Evaluation Engine. Mỗi thành phần đảm nhận một trách nhiệm riêng, giúp giảm mức độ phụ thuộc giữa các module và nâng cao khả năng bảo trì.

Bên cạnh đó, việc kết hợp validation, audit log, cache, phân quyền và unit test giúp hệ thống dễ dàng kiểm soát thay đổi trong quá trình phát triển. Kiến trúc hiện tại cũng tạo điều kiện thuận lợi cho việc bổ sung các loại rule mới, mở rộng Evaluation API hoặc thay thế các thành phần hạ tầng mà không ảnh hưởng đáng kể đến các module còn lại.

3.7.5 Đánh giá Dashboard

Dashboard là giao diện quản trị trực tiếp của nền tảng Feature Flag, hỗ trợ người dùng quản lý toàn bộ vòng đời của Feature Flag từ cấu hình đến theo dõi trạng thái và lịch sử thay đổi. Kết quả triển khai cho thấy Dashboard đáp ứng tốt các yêu cầu về khả năng sử dụng, khả năng quan sát trạng thái, cấu hình rule và hỗ trợ người dùng hạn chế các thao tác không mong muốn. Giao diện được thiết kế theo hướng trực quan, giúp đơn giản hóa quá trình quản trị ngay cả khi số lượng Feature Flag tăng lên.

Khả năng sử dụng

Dashboard tổ chức các chức năng theo từng nhóm như project, environment, Feature Flag, rule, group, sample user, audit log và statistics. Cách bố trí này giúp người quản trị dễ dàng tìm kiếm và thực hiện các thao tác cần thiết mà không phải chuyển đổi qua nhiều giao diện khác nhau.

Các thao tác như tạo mới, chỉnh sửa, archive, khôi phục, bật hoặc tắt Feature Flag và cấu hình rule đều được thực hiện trực tiếp trên giao diện với quy trình thống nhất. Việc tích hợp chặt chẽ với Backend API giúp các thay đổi được cập nhật ngay sau khi thao tác hoàn tất.

Khả năng nhận biết trạng thái

Dashboard hiển thị đồng thời nhiều trạng thái quan trọng của Feature Flag như lifecycle, runtime status, serving mode, group kill switch và quyền của người dùng hiện tại. Các trạng thái này được thể hiện trực quan thông qua nhãn và các thành phần giao diện, giúp người quản trị nhanh chóng xác định tình trạng hoạt động của từng Feature Flag.

Ngoài ra, Dashboard còn cung cấp audit log và thống kê evaluation, hỗ trợ người dùng theo dõi lịch sử thay đổi cũng như mức độ sử dụng của từng Feature Flag trong quá trình vận hành. Khả năng quan sát này phù hợp với các nguyên tắc quản lý Feature Flag được đề xuất trong các nền tảng hiện đại [7, 8].

Khả năng cấu hình rule

Dashboard hỗ trợ cấu hình các chiến lược bật hoặc tắt toàn cục, user targeting, role targeting, percentage rollout và group kill switch thông qua các biểu mẫu trực quan. Người quản trị có thể thêm, sửa hoặc xóa rule mà không cần thao tác trực tiếp với cơ sở dữ liệu hoặc API.

Các tham số cấu hình được kiểm tra ngay trên giao diện trước khi gửi tới Backend API, đồng thời phản hồi lỗi được hiển thị rõ ràng khi dữ liệu không hợp lệ. Cách tiếp cận này giúp giảm số lượng thao tác sai và nâng cao hiệu quả quản trị Feature Flag.

Khả năng hạn chế thao tác sai

Dashboard kết hợp nhiều cơ chế nhằm giảm thiểu các thao tác không mong muốn trong quá trình quản trị. Các chức năng chỉ hiển thị khi người dùng có quyền phù hợp, các nút thao tác

được vô hiệu hóa đối với những vai trò không được phép và các thao tác quan trọng như archive hoặc kích hoạt group kill switch đều yêu cầu xác nhận trước khi thực hiện.

Bên cạnh đó, toàn bộ thay đổi cấu hình đều được ghi nhận trong audit log và các thay đổi liên quan đến Feature Flag hoặc Rule sẽ kích hoạt cơ chế cache invalidation để bảo đảm dữ liệu hiển thị luôn phản ánh đúng trạng thái hiện tại của hệ thống. Những cơ chế này góp phần nâng cao tính an toàn và giảm nguy cơ phát sinh lỗi trong quá trình vận hành.

3.7.6 Đánh giá Demo Application

Demo Application được xây dựng nhằm minh họa trực quan cách một ứng dụng khách tích hợp với nền tảng Feature Flag thông qua JavaScript SDK và Evaluation API. Thay vì chỉ trình diễn các chức năng của Dashboard, ứng dụng cho phép quan sát trực tiếp sự thay đổi của giao diện khi cấu hình Feature Flag được cập nhật. Kết quả thực hiện cho thấy Demo Application đáp ứng tốt mục tiêu chứng minh giá trị của Feature Flag trong quá trình phát triển và phát hành phần mềm.

Khả năng chứng minh deploy tách khỏi release

Một trong những mục tiêu quan trọng nhất của Demo Application là chứng minh việc triển khai (deploy) và phát hành tính năng (release) có thể được tách biệt. Sau khi ứng dụng được triển khai, người quản trị có thể thay đổi trạng thái của Feature Flag trên Dashboard mà không cần build hoặc triển khai lại ứng dụng.

Kết quả trình diễn cho thấy các thay đổi được phản ánh ngay trong Demo Application thông qua Evaluation API. Điều này minh họa rõ lợi ích của Feature Flag trong việc giảm rủi ro phát hành và phù hợp với cách tiếp cận được khuyến nghị trong các quy trình DevOps hiện đại [1, 3].

Khả năng minh họa targeting

Demo Application hỗ trợ các kịch bản user targeting và role targeting bằng cách xây dựng Evaluation Context từ thông tin người dùng hiện tại trước khi gửi request tới Evaluation API.

Kết quả trình diễn cho thấy các người dùng khác nhau có thể nhận được trạng thái Feature Flag khác nhau mặc dù sử dụng cùng một phiên bản ứng dụng. Điều này giúp minh họa rõ cơ chế phát hành theo từng nhóm người dùng và khả năng kiểm soát phạm vi ảnh hưởng của

Feature Flag, phù hợp với mô hình targeting được áp dụng trong các nền tảng Feature Flag hiện đại [4, 6].

Khả năng minh họa rollout

Đối với chiến lược percentage rollout, Demo Application cho phép quan sát trực tiếp sự khác biệt giữa các người dùng dựa trên `targetingKey`. Khi thay đổi tỷ lệ rollout trên Dashboard, chỉ một phần người dùng được kích hoạt tính năng mới, trong khi các người dùng còn lại vẫn giữ nguyên hành vi của ứng dụng.

Kết quả trình diễn đồng thời cho thấy cùng một người dùng luôn nhận được kết quả nhất quán giữa các lần truy cập nhờ cơ chế stable hashing. Điều này giúp minh họa trực quan quá trình phát hành từng phần và phù hợp với nguyên tắc stable rollout được sử dụng trong các hệ thống Feature Flag thực tế [5].

Khả năng minh họa rollback nhanh

Demo Application cũng chứng minh khả năng rollback nhanh của Feature Flag thông qua các thao tác tắt Feature Flag, kích hoạt group kill switch hoặc thay đổi cấu hình rule trên Dashboard.

Kết quả trình diễn cho thấy sau khi cấu hình được cập nhật, tính năng mới lập tức bị vô hiệu hóa đối với các người dùng thuộc phạm vi ảnh hưởng mà không cần triển khai lại ứng dụng hoặc khởi động lại hệ thống. Khả năng phản ứng nhanh này là một trong những ưu điểm quan trọng của Feature Flag trong việc giảm thiểu tác động của sự cố trên môi trường production và phù hợp với các khuyến nghị của Martin Fowler và Unleash [1, 8].

3.7.7 So sánh với các nền tảng hiện có

Các nền tảng Feature Flag phổ biến như LaunchDarkly và Unleash cung cấp hệ sinh thái hoàn chỉnh phục vụ quản lý và phát hành tính năng trong các hệ thống quy mô lớn. Trong khi đó, đề tài tập trung xây dựng một nền tảng Feature Flag ở quy mô mini project nhằm làm rõ các thành phần cốt lõi của hệ thống, đồng thời vẫn giữ được khả năng minh họa các kịch bản triển khai và phát hành tính năng trong thực tế. Kết quả đánh giá cho thấy hệ thống đã kế thừa nhiều đặc điểm quan trọng của các nền tảng hiện có, đồng thời chủ động đơn giản hóa một số chức năng để phù hợp với phạm vi nghiên cứu.

Những khả năng tương đồng

Hệ thống triển khai đầy đủ các thành phần cốt lõi của một nền tảng Feature Flag, bao gồm Dashboard quản trị, Backend API, Rule Evaluation Engine, Evaluation API, Demo Application và cơ chế audit log. Các chiến lược bật hoặc tắt toàn cục, user targeting, role targeting, percentage rollout và group kill switch đều được hỗ trợ theo nguyên tắc tương tự các nền tảng Feature Flag hiện đại.

Ngoài ra, hệ thống còn áp dụng nhiều thực hành phổ biến như tách biệt deploy khỏi release, sử dụng Evaluation Context, stable hashing cho percentage rollout, ghi nhận lịch sử thay đổi, cache dữ liệu phục vụ evaluation và phân quyền người dùng quản trị. Những đặc điểm này phù hợp với các mô hình được đề xuất bởi Martin Fowler, Unleash, LaunchDarkly và OpenFeature [1, 8, 7, 10].

Những chức năng đã được đơn giản hóa

Để phù hợp với phạm vi của mini project, một số chức năng được triển khai ở mức đơn giản hơn so với các sản phẩm thương mại. Hệ thống chỉ hỗ trợ một tập chiến lược targeting cơ bản, sử dụng JavaScript SDK cho Demo Application và triển khai cache theo quy mô nhỏ thay vì hạ tầng phân tán.

Bên cạnh đó, cơ chế versioning được thực hiện thông qua audit log và before/after snapshot thay vì quản lý nhiều phiên bản cấu hình hoặc hỗ trợ khôi phục tự động. Dashboard cũng tập trung vào các chức năng quản trị cốt lõi thay vì cung cấp các tính năng cộng tác hoặc quản lý tổ chức ở quy mô doanh nghiệp.

Những chức năng chưa hỗ trợ

So với các nền tảng Feature Flag thương mại, hệ thống chưa hỗ trợ một số chức năng nâng cao như multi-variant experimentation, A/B testing hoàn chỉnh, prerequisite flag, phân đoạn người dùng động, streaming configuration, SDK cho nhiều ngôn ngữ lập trình, đồng bộ cấu hình theo thời gian thực hoặc triển khai trên nhiều khu vực địa lý.

Ngoài ra, hệ thống cũng chưa tích hợp với các nền tảng giám sát, hệ thống phân tích dữ liệu, dịch vụ nhận dạng người dùng hoặc các công cụ CI/CD để tự động hóa toàn bộ quy trình phát hành tính năng. Đây đều là các hướng mở rộng phù hợp nếu tiếp tục phát triển đề tài ở quy mô lớn hơn.

Mức độ phù hợp với phạm vi mini project

Xét theo mục tiêu của Mini Project, hệ thống đã đáp ứng đầy đủ các yêu cầu về nghiên cứu, thiết kế, triển khai và trình diễn nền tảng Feature Flag. Các chức năng cốt lõi đều được hiện thực hóa, đồng thời nhiều yêu cầu khuyến khích như cache, JavaScript SDK, unit test, phân quyền, thống kê evaluation, group kill switch và Docker Compose cũng đã được triển khai.

Mặc dù chưa hướng tới quy mô của các nền tảng thương mại, hệ thống vẫn thể hiện đầy đủ các nguyên lý quan trọng của Feature Flag, giúp minh họa rõ quy trình quản lý và phát hành tính năng trong các hệ thống hiện đại. Vì vậy, phạm vi triển khai được đánh giá là phù hợp với mục tiêu nghiên cứu, yêu cầu của mini project và tiêu chí đánh giá đã đặt ra.

3.7.8 Giá trị và tính mới của sản phẩm

Đề tài không chỉ hiện thực hóa các chức năng cốt lõi của một nền tảng Feature Flag mà còn xây dựng một hệ thống hoàn chỉnh gồm Backend API, Dashboard, Rule Evaluation Engine, Demo Application và cơ chế kiểm thử. Sản phẩm đồng thời đáp ứng mục tiêu nghiên cứu, hỗ trợ học tập và minh họa trực quan quy trình phát hành phần mềm hiện đại dựa trên Feature Flag. Bên cạnh việc kế thừa các nguyên lý từ các nền tảng hiện có, hệ thống cũng có những điểm mở rộng phù hợp với phạm vi của mini project.

Giá trị về mặt học thuật

Đề tài hệ thống hóa tương đối đầy đủ các kiến thức liên quan đến Feature Flag, bao gồm sự khác biệt giữa deploy và release, các loại Feature Flag, Rule Evaluation Engine, Evaluation Context, percentage rollout, kill switch, audit log, caching, phân quyền và quy trình quản lý vòng đời của Feature Flag.

Ngoài việc trình bày cơ sở lý thuyết, đề tài còn chuyển hóa các khái niệm này thành một hệ thống có thể triển khai và kiểm thử thực tế. Điều này giúp thu hẹp khoảng cách giữa lý thuyết và ứng dụng, đồng thời tạo nguồn tài liệu tham khảo cho việc nghiên cứu các hệ thống quản lý Feature Flag trong tương lai [1, 8, 10].

Giá trị về mặt kỹ thuật

Về mặt kỹ thuật, hệ thống được xây dựng theo kiến trúc phân tách rõ ràng giữa Backend API, Rule Evaluation Engine, Dashboard và Demo Application. Các thành phần được kết nối

thông qua các API thống nhất, kết hợp với validation, audit log, cache, phân quyền, thống kê evaluation và triển khai bằng Docker Compose.

Đề tài đồng thời áp dụng nhiều kỹ thuật thường gặp trong các hệ thống hiện đại như stable hashing cho percentage rollout, Evaluation Context, request ID, before và after snapshot, JavaScript SDK và cơ chế cache invalidation. Những thành phần này giúp sản phẩm có tính hoàn chỉnh cao hơn so với một ứng dụng minh họa thông thường.

Giá trị trong release management

Sản phẩm minh họa rõ vai trò của Feature Flag trong quy trình release management thông qua khả năng bật hoặc tắt tính năng tại thời điểm chạy mà không cần triển khai lại ứng dụng. Các chiến lược như user targeting, role targeting, percentage rollout và group kill switch giúp kiểm soát phạm vi phát hành, giảm rủi ro khi đưa tính năng mới vào môi trường production và hỗ trợ rollback nhanh khi phát sinh sự cố.

Nhờ đó, hệ thống thể hiện được giá trị thực tiễn của Feature Flag trong các quy trình phát triển phần mềm hiện đại, phù hợp với cách tiếp cận DevOps và Continuous Delivery được nhiều tổ chức áp dụng [3, 1].

Khả năng tái sử dụng và mở rộng

Kiến trúc hiện tại được xây dựng theo hướng module hóa, giúp các thành phần như Rule Evaluation Engine, Evaluation API, Dashboard và JavaScript SDK có thể được tái sử dụng trong các dự án khác hoặc tiếp tục phát triển thành một nền tảng Feature Flag hoàn chỉnh hơn.

Trong tương lai, hệ thống có thể mở rộng để hỗ trợ nhiều variant, nhiều loại SDK, targeting theo thuộc tính hoặc phân đoạn người dùng, prerequisite flag, realtime configuration, nhiều môi trường triển khai và tích hợp với các nền tảng CI/CD hoặc hệ thống giám sát. Những hướng mở rộng này có thể được thực hiện mà không làm thay đổi đáng kể kiến trúc tổng thể, cho thấy sản phẩm có khả năng phát triển lâu dài và phù hợp làm nền tảng cho các nghiên cứu tiếp theo.

3.7.9 Hạn chế của hệ thống

Mặc dù đã đáp ứng đầy đủ các yêu cầu của mini project và hiện thực hóa các chức năng cốt lõi của một nền tảng Feature Flag, hệ thống vẫn còn một số hạn chế về quy mô triển khai, phạm

vi hỗ trợ rule, khả năng tích hợp với ứng dụng khách và các chức năng phục vụ vận hành trong môi trường production. Những hạn chế này xuất phát từ phạm vi nghiên cứu cũng như mục tiêu xây dựng một nền tảng minh họa thay vì một sản phẩm thương mại hoàn chỉnh.

Hạn chế về quy mô triển khai

Hệ thống được thiết kế và triển khai cho mục đích nghiên cứu và trình diễn nên chưa hướng tới việc phục vụ số lượng lớn người dùng hoặc nhiều ứng dụng khách cùng lúc. Cơ sở dữ liệu, Backend API và Dashboard đều được triển khai theo mô hình đơn giản, phù hợp với môi trường phát triển và kiểm thử.

Ngoài ra, hệ thống chưa hỗ trợ các cơ chế như cân bằng tải, triển khai nhiều phiên bản dịch vụ, sao lưu dữ liệu, phục hồi sau sự cố hoặc vận hành trên hạ tầng phân tán. Do đó, khả năng mở rộng theo chiều ngang vẫn còn là một hướng phát triển trong tương lai.

Hạn chế về rule targeting

Rule Evaluation Engine hiện hỗ trợ các chiến lược targeting cơ bản gồm bật hoặc tắt toàn cục, user targeting, role targeting, percentage rollout và group kill switch. Các chức năng này đáp ứng đầy đủ yêu cầu của đề tài nhưng vẫn chưa bao phủ toàn bộ các chiến lược được sử dụng trong các nền tảng Feature Flag thương mại.

Hệ thống hiện chưa hỗ trợ các rule phức tạp như targeting theo nhiều thuộc tính kết hợp, phân đoạn người dùng động, prerequisite flag, thời gian hiệu lực, vị trí địa lý hoặc các chiến lược nhiều biến thể (multi-variant). Vì vậy, khả năng biểu diễn các kịch bản phát hành phức tạp vẫn còn hạn chế so với các nền tảng hiện đại [4, 7].

Hạn chế về SDK và realtime update

Đề tài hiện chỉ xây dựng JavaScript SDK phục vụ Demo Application. Hệ thống chưa hỗ trợ SDK cho các nền tảng và ngôn ngữ lập trình khác như Java, .NET, Go, Python hoặc Mobile, đồng thời chưa cung cấp các cơ chế đồng bộ cấu hình theo thời gian thực.

Việc cập nhật trạng thái Feature Flag hiện vẫn dựa trên các request Evaluation API hoặc làm mới dữ liệu từ phía ứng dụng khách. Các kỹ thuật như streaming configuration, Server-Sent Events hoặc WebSocket để đồng bộ thay đổi theo thời gian thực chưa được triển khai.

Hạn chế về bảo mật và vận hành

Hệ thống đã triển khai xác thực, phân quyền, validation, audit log và request ID, tuy nhiên vẫn còn thiếu một số chức năng thường xuất hiện trong môi trường production. Chẳng hạn, hệ thống chưa hỗ trợ rate limiting, API key dành cho Evaluation API, quản lý secret, mã hóa dữ liệu nhạy cảm, giám sát tập trung, cảnh báo tự động hoặc tích hợp với các nền tảng quan sát và phân tích log.

Bên cạnh đó, việc triển khai hiện mới dừng ở Docker Compose và chưa hỗ trợ các giải pháp orchestration như Kubernetes hoặc các quy trình CI/CD hoàn chỉnh. Đây là những hướng phát triển phù hợp nếu hệ thống được mở rộng để phục vụ các môi trường triển khai thực tế với quy mô lớn hơn [11, 12, 14].

Chương 4

Kết luận

4.1 Tổng kết kết quả đạt được

4.1.1 Kết quả nghiên cứu

Quá trình thực hiện đề tài không chỉ tập trung vào việc xây dựng một hệ thống Feature Flag hoàn chỉnh mà còn giúp nghiên cứu và hệ thống hóa các kiến thức quan trọng liên quan đến release management, thiết kế hệ thống, kiến trúc phần mềm và vận hành ứng dụng. Thông qua việc khảo sát các nền tảng Feature Flag phổ biến như LaunchDarkly và Unleash cùng với các tài liệu của Martin Fowler, Microsoft và OpenFeature, đề tài đã tổng hợp được các nguyên lý cốt lõi và vận dụng vào quá trình thiết kế hệ thống phù hợp với phạm vi của mini project [1, 8, 7, 3].

Nhận thức về deploy và release

Một trong những kết quả quan trọng nhất của quá trình nghiên cứu là làm rõ sự khác biệt giữa *deploy* và *release*. Thay vì xem hai khái niệm này là một bước duy nhất, đề tài đã chỉ ra rằng deploy chỉ là quá trình đưa phiên bản mới lên môi trường vận hành, trong khi release là quyết định cho phép người dùng sử dụng tính năng mới. Feature Flag đóng vai trò tách biệt hai hoạt động này, giúp doanh nghiệp có thể triển khai mã nguồn trước và kích hoạt tính năng vào thời điểm phù hợp mà không cần triển khai lại hệ thống. Cách tiếp cận này góp phần giảm rủi ro khi phát hành, hỗ trợ Continuous Delivery và nâng cao khả năng phản ứng khi xảy ra sự cố trong môi trường production [1, 3].

Các chiến lược Feature Flag

Đề tài đã nghiên cứu và phân tích các chiến lược triển khai Feature Flag được sử dụng phổ biến trong thực tế, bao gồm bật hoặc tắt toàn cục, targeting theo người dùng, targeting theo vai trò và rollout theo phần trăm người dùng. Đồng thời, các loại Feature Flag như release flag, experiment flag, operational flag (kill switch) và permission flag cũng được khảo sát nhằm làm rõ mục đích sử dụng của từng loại trong các giai đoạn khác nhau của vòng đời phát triển phần mềm. Những kiến thức này là cơ sở để xây dựng Rule Evaluation Engine với thứ tự đánh giá rõ ràng, bảo đảm tính xác định của kết quả và khả năng mở rộng trong tương lai [1, 4, 5, 2].

Các vấn đề về caching, consistency và bảo mật

Bên cạnh việc nghiên cứu cơ chế đánh giá Feature Flag, đề tài cũng khảo sát các vấn đề kỹ thuật quan trọng ảnh hưởng trực tiếp đến khả năng vận hành của hệ thống. Về hiệu năng, việc sử dụng cache giúp giảm độ trễ của quá trình evaluation và hạn chế số lần truy cập cơ sở dữ liệu, tuy nhiên cần kết hợp với cơ chế cache invalidation phù hợp để bảo đảm tính nhất quán của cấu hình. Về bảo mật, hệ thống cần áp dụng xác thực và phân quyền đối với Management API, hạn chế lộ thông tin nhạy cảm trong evaluation context và xây dựng audit log đầy đủ nhằm hỗ trợ truy vết các thay đổi cấu hình. Những nội dung này đã được nghiên cứu và tích hợp vào thiết kế tổng thể của hệ thống theo các khuyến nghị từ OpenFeature, OWASP và Unleash [6, 10, 11, 12, 13, 8].

4.1.2 Kết quả xây dựng hệ thống

Dựa trên các yêu cầu đặt ra của mini project, đề tài đã xây dựng thành công một nền tảng Feature Flag hoàn chỉnh gồm Backend API, Rule Evaluation Engine, Admin Dashboard, Demo Application và JavaScript SDK. Bên cạnh các chức năng bắt buộc, hệ thống còn hiện thực nhiều chức năng thuộc mức khuyến khích như caching, phân quyền, thống kê evaluation, versioning, group kill switch và triển khai bằng Docker Compose. Các thành phần được thiết kế theo kiến trúc tách biệt, giúp dễ dàng bảo trì và mở rộng trong tương lai.

Backend API và Rule Evaluation Engine

Backend API được xây dựng theo kiến trúc RESTful, cung cấp đầy đủ các API phục vụ quản lý project, environment, feature flag, rule, audit log và evaluation. Hệ thống sử dụng Rule Evaluation Engine để đánh giá trạng thái của Feature Flag dựa trên thứ tự ưu tiên xác định,

bao gồm kiểm tra trạng thái hệ thống, global enable, user targeting, role targeting và percentage rollout. Ngoài việc trả về kết quả bật hoặc tắt, Evaluation API còn cung cấp reason code và thông tin rule được áp dụng nhằm tăng khả năng giải thích và hỗ trợ quá trình theo dõi. Thiết kế này phù hợp với các nguyên tắc của OpenFeature và kế thừa các chiến lược Feature Flag được sử dụng trong các nền tảng hiện đại [10, 6, 4, 5].

Admin Dashboard

Đề tài đã xây dựng giao diện quản trị trên nền tảng web cho phép người quản trị thực hiện toàn bộ các thao tác quản lý Feature Flag. Dashboard hỗ trợ tạo và cập nhật project, environment, feature flag và rule; thay đổi trạng thái runtime; theo dõi audit log; xem thống kê evaluation; đồng thời hiển thị quyền của từng người dùng theo mô hình phân quyền. Giao diện được thiết kế trực quan nhằm giúp người quản trị dễ dàng nhận biết trạng thái của từng Feature Flag và giảm nguy cơ thao tác nhầm trong quá trình phát hành tính năng.

Demo Application và SDK

Để minh họa khả năng ứng dụng thực tế của Feature Flag, đề tài xây dựng một ứng dụng web mẫu tích hợp với Evaluation API thông qua JavaScript SDK. Demo Application sử dụng evaluation context để gửi thông tin người dùng và hiển thị hoặc ẩn các tính năng dựa trên kết quả đánh giá từ hệ thống. Các kịch bản như bật hoặc tắt toàn cục, targeting theo người dùng, targeting theo vai trò, percentage rollout và fallback khi Evaluation API không khả dụng đều được mô phỏng thành công, qua đó chứng minh khả năng tách biệt giữa deploy và release trong thực tế [3, 1].

Các chức năng khuyến khích

Ngoài các chức năng thuộc phạm vi MVP, hệ thống còn hiện thực nhiều nội dung khuyến khích nhằm nâng cao giá trị thực tiễn của sản phẩm. Hệ thống hỗ trợ caching kết quả evaluation để cải thiện hiệu năng, lưu trữ lịch sử thay đổi cấu hình thông qua audit log và versioning, thống kê số lần evaluation theo Feature Flag, phân quyền quản trị với các vai trò khác nhau, triển khai group kill switch để vô hiệu hóa nhanh nhiều Feature Flag cùng lúc và sử dụng Docker Compose để triển khai đồng bộ toàn bộ các thành phần của hệ thống. Những chức năng này giúp nền tảng tiệm cận hơn với cách triển khai của các hệ thống Feature Flag hiện đại và tạo tiền đề cho việc mở rộng trong các giai đoạn phát triển tiếp theo [8, 13, 14, 12].

4.1.3 Kết quả kiểm thử và demo

Sau khi hoàn thành quá trình phát triển, hệ thống được kiểm thử trên toàn bộ các chức năng chính nhằm đánh giá tính đúng đắn của nghiệp vụ, khả năng hoạt động của Rule Evaluation Engine và hiệu quả của các kịch bản demo. Các trường hợp kiểm thử được xây dựng dựa trên yêu cầu của đề tài, bao phủ cả các chức năng bắt buộc và các chức năng khuyến khích. Kết quả cho thấy hệ thống hoạt động ổn định, đáp ứng đúng các yêu cầu đặt ra và thể hiện được giá trị của Feature Flag trong việc tách biệt quá trình deploy khỏi release.

Kết quả kiểm thử chức năng

Các API quản lý project, environment, feature flag, rule, audit log và evaluation đều được kiểm thử với các trường hợp dữ liệu hợp lệ và không hợp lệ. Kết quả kiểm thử cho thấy các chức năng tạo, cập nhật, truy vấn và lưu trữ dữ liệu đều hoạt động đúng theo thiết kế. Dashboard phản ánh chính xác trạng thái của hệ thống sau mỗi thao tác, đồng thời các cơ chế validation, xử lý lỗi và phân quyền được thực hiện đúng như mong đợi. Audit log ghi nhận đầy đủ các thay đổi cấu hình, hỗ trợ truy vết quá trình quản trị hệ thống.

Kết quả kiểm thử evaluation

Rule Evaluation Engine được kiểm thử với nhiều bộ evaluation context khác nhau nhằm xác minh tính đúng đắn của thuật toán đánh giá. Các trường hợp kiểm thử bao gồm bật hoặc tắt toàn cục, user targeting, role targeting, percentage rollout, kill switch, feature flag không tồn tại, context không đầy đủ và các tình huống fallback khi xảy ra lỗi. Kết quả cho thấy hệ thống luôn trả về trạng thái Feature Flag phù hợp với cấu hình hiện tại, reason code phản ánh chính xác nguyên nhân của kết quả evaluation và cơ chế stable hashing bảo đảm cùng một người dùng luôn được gán vào cùng một bucket trong các lần đánh giá liên tiếp. Thiết kế này phù hợp với các nguyên tắc đánh giá Feature Flag được đề xuất bởi OpenFeature và các nền tảng Feature Flag hiện đại [10, 4, 5].

Kết quả các kịch bản demo

Demo Application đã minh họa thành công các kịch bản sử dụng Feature Flag trong môi trường thực tế. Người quản trị có thể thay đổi cấu hình Feature Flag trên Dashboard và quan sát ngay sự thay đổi của giao diện ứng dụng thông qua Evaluation API mà không cần triển khai lại hệ thống. Các kịch bản demo bao gồm bật hoặc tắt toàn cục, targeting theo người dùng, targeting

theo vai trò, rollout theo phần trăm người dùng, group kill switch, theo dõi audit log và thống kê evaluation. Kết quả đạt được cho thấy hệ thống có khả năng hỗ trợ quá trình phát hành tính năng linh hoạt, giảm rủi ro khi triển khai và minh họa rõ giá trị của Feature Flag trong quy trình phát triển phần mềm hiện đại [1, 3, 8].

4.2 Các đóng góp chính

4.2.1 Đóng góp về nghiên cứu

Hệ thống hóa kiến thức Feature Flag

Đề tài đã hệ thống hóa các khái niệm nền tảng của Feature Flag và vai trò của kỹ thuật này trong quy trình phát triển phần mềm hiện đại. Báo cáo làm rõ sự khác biệt giữa *deploy* và *release*, chỉ ra rằng Feature Flag cho phép tách biệt hai quá trình này nhằm giảm rủi ro khi phát hành tính năng mới, đồng thời tạo điều kiện cho phát triển liên tục và triển khai liên tục (Continuous Integration/Continuous Delivery). Nội dung nghiên cứu cũng tổng hợp các loại Feature Flag phổ biến như release flag, experiment flag, operational (kill switch) flag và permission flag cùng các trường hợp sử dụng điển hình trong thực tế [1, 2, 7, 3].

Bên cạnh việc trình bày các khái niệm lý thuyết, báo cáo còn xây dựng bức tranh tổng thể về kiến trúc của một nền tảng Feature Flag, bao gồm Dashboard quản trị, Backend Management API, Rule Evaluation Engine, Evaluation API và ứng dụng khách. Việc liên kết giữa các thành phần giúp làm rõ luồng hoạt động của hệ thống từ quá trình cấu hình Feature Flag đến khi ứng dụng khách thực hiện đánh giá và áp dụng kết quả trong thời gian chạy.

Phân tích các chiến lược rollout

Một đóng góp quan trọng của đề tài là phân tích và tổng hợp các chiến lược rollout được sử dụng phổ biến trong các nền tảng Feature Flag hiện nay. Báo cáo trình bày nguyên lý hoạt động, ưu điểm và phạm vi áp dụng của các chiến lược như bật hoặc tắt toàn cục (global on/off), targeting theo người dùng, targeting theo vai trò và rollout theo tỷ lệ phần trăm người dùng. Đồng thời, thứ tự ưu tiên khi đánh giá các rule cũng được phân tích nhằm đảm bảo kết quả evaluation xác định và nhất quán [4, 5, 10].

Ngoài ra, đề tài nghiên cứu cơ chế stable hashing để bảo đảm cùng một người dùng luôn nhận được kết quả rollout giống nhau giữa nhiều lần đánh giá, đồng thời phân tích vai trò của

Evaluation Context trong việc truyền thông tin người dùng và thuộc tính phục vụ quá trình đánh giá Feature Flag [6, 5]. Các kết quả nghiên cứu này được sử dụng làm cơ sở cho việc thiết kế Rule Evaluation Engine và Evaluation API của hệ thống.

Phân tích rủi ro và yêu cầu vận hành

Đề tài không chỉ tập trung vào chức năng của Feature Flag mà còn phân tích các vấn đề phát sinh trong quá trình vận hành hệ thống. Báo cáo chỉ ra rằng việc sử dụng Feature Flag trong thời gian dài có thể tạo ra *feature flag debt*, làm tăng độ phức tạp của mã nguồn và gây khó khăn cho quá trình bảo trì nếu các flag không được quản lý vòng đời phù hợp [1, 9].

Bên cạnh đó, các yêu cầu về vận hành như cơ chế caching để giảm độ trễ evaluation, đảm bảo tính nhất quán sau khi cập nhật cấu hình, quản lý audit log phục vụ truy vết thay đổi, bảo vệ Management API và Evaluation API, cũng như phân quyền người dùng quản trị đã được tổng hợp và phân tích dựa trên các tài liệu thực tiễn của Unleash, OWASP và Docker [8, 11, 12, 13, 14]. Những nội dung này tạo nền tảng cho việc thiết kế hệ thống Feature Flag có khả năng vận hành ổn định, an toàn và dễ mở rộng trong phạm vi của mini project.

4.2.2 Đóng góp về thiết kế hệ thống

Thiết kế kiến trúc control plane và data plane

Đề tài đề xuất kiến trúc hệ thống theo hướng tách biệt giữa *control plane* và *data plane*, phù hợp với cách tổ chức của nhiều nền tảng Feature Flag hiện đại [7, 8]. Trong đó, control plane bao gồm Dashboard quản trị và Management API, chịu trách nhiệm quản lý project, environment, feature flag, rule và audit log. Data plane bao gồm Evaluation API, Rule Evaluation Engine và lớp caching, chịu trách nhiệm đánh giá Feature Flag với độ trễ thấp trong thời gian chạy.

Việc phân tách hai mặt phẳng giúp giảm sự phụ thuộc giữa quá trình quản trị cấu hình và quá trình đánh giá Feature Flag, cho phép các thay đổi cấu hình được thực hiện độc lập với lưu lượng evaluation của ứng dụng khách. Kiến trúc này cũng tạo tiền đề cho việc mở rộng theo hướng triển khai nhiều instance của Evaluation API hoặc bổ sung cơ chế đồng bộ cấu hình trong tương lai.

Thiết kế Evaluation Engine có precedence rõ ràng

Một đóng góp quan trọng của hệ thống là thiết kế Rule Evaluation Engine với thứ tự ưu tiên (precedence) được xác định rõ ràng. Engine thực hiện đánh giá lần lượt các trạng thái runtime của Feature Flag như archived, disabled và kill switch trước khi chuyển sang các chiến lược rollout như global on, user targeting, role targeting và percentage rollout. Cách tổ chức này bảo đảm mỗi yêu cầu evaluation luôn tạo ra một kết quả duy nhất cùng với reason code tương ứng [10, 4, 5].

Ngoài việc xác định precedence, hệ thống còn áp dụng stable hashing cho percentage rollout nhằm bảo đảm cùng một người dùng luôn nhận được kết quả nhất quán giữa các lần đánh giá. Evaluation Context được thiết kế theo hướng mở rộng để có thể bổ sung thêm các thuộc tính mới mà không ảnh hưởng đến giao diện Evaluation API [6]. Thiết kế này giúp Rule Evaluation Engine dễ bảo trì và thuận lợi khi bổ sung thêm các loại rule trong tương lai.

Thiết kế dữ liệu và audit log

Đề tài xây dựng mô hình dữ liệu bao gồm các thực thể Project, Environment, Feature Flag, Flag Configuration, Flag Rule, Sample User và Audit Log, phản ánh đầy đủ mối quan hệ giữa các thành phần của nền tảng Feature Flag. Cấu hình rule và các thuộc tính mở rộng được lưu dưới dạng JSONB nhằm tăng tính linh hoạt khi bổ sung các loại targeting mới mà không phải thay đổi cấu trúc cơ sở dữ liệu [15].

Bên cạnh đó, hệ thống thiết kế cơ chế audit log theo mô hình append-only, lưu lại actor, thời gian thực hiện, hành động, request ID cùng snapshot trước và sau mỗi thay đổi cấu hình. Thiết kế này đáp ứng yêu cầu truy vết, hỗ trợ kiểm tra lịch sử thay đổi và nâng cao khả năng vận hành hệ thống theo các khuyến nghị về logging và auditability [12, 11].

Thiết kế API có khả năng mở rộng

Hệ thống xây dựng bộ Management API và Evaluation API theo kiến trúc REST với cấu trúc request và response thống nhất, hỗ trợ validation dữ liệu đầu vào, chuẩn hóa mã lỗi và sử dụng request ID để phục vụ truy vết. Evaluation API được thiết kế tách biệt khỏi Management API nhằm đáp ứng yêu cầu tối ưu hóa cho lưu lượng đánh giá Feature Flag trong thời gian chạy [10].

Ngoài các chức năng bắt buộc của mini project, API còn được thiết kế theo hướng dễ mở rộng

thông qua việc sử dụng Evaluation Context, reason code và mô hình rule độc lập. Cách tiếp cận này cho phép bổ sung các chiến lược targeting theo thuộc tính, organization, thời gian hoặc nhiều biến thể (variant) mà không làm thay đổi giao diện của Evaluation API, đồng thời tạo điều kiện thuận lợi để tích hợp SDK hoặc các dịch vụ khác trong tương lai [6, 8, 7].

4.2.3 Đóng góp về triển khai sản phẩm

Nền tảng quản lý Feature Flag hoàn chỉnh

Đề tài đã xây dựng một nền tảng quản lý Feature Flag hoàn chỉnh đáp ứng đầy đủ các yêu cầu bắt buộc của mini project. Hệ thống bao gồm Dashboard quản trị, Backend Management API, Rule Evaluation Engine, Evaluation API, cơ sở dữ liệu lưu trữ cấu hình và Demo Application minh họa việc sử dụng Feature Flag trong thực tế. Các thành phần được tổ chức theo kiến trúc thống nhất, cho phép quản lý toàn bộ vòng đời của Feature Flag từ khởi tạo, cấu hình, đánh giá đến lưu vết thay đổi.

Sản phẩm hỗ trợ quản lý project, environment, feature flag, rule targeting, audit log và dữ liệu mẫu, đồng thời cung cấp giao diện trực quan để cấu hình và theo dõi trạng thái của từng Feature Flag. Việc triển khai đầy đủ các thành phần giúp minh họa tương đối đầy đủ quy trình quản trị Feature Flag trong một hệ thống phần mềm hiện đại [7, 8].

Demo Application chứng minh giá trị thực tế

Đề tài xây dựng một Demo Application đóng vai trò là ứng dụng khách tích hợp với Evaluation API nhằm minh họa giá trị của Feature Flag trong quá trình phát hành phần mềm. Ứng dụng sử dụng Evaluation Context để gửi thông tin người dùng, nhận kết quả evaluation và quyết định hiển thị hoặc ẩn các tính năng ngay trong thời gian chạy mà không cần triển khai lại ứng dụng.

Thông qua các kịch bản như bật hoặc tắt toàn cục, targeting theo người dùng, targeting theo vai trò và percentage rollout, Demo Application cho thấy khả năng tách biệt giữa deploy và release, đồng thời minh họa cách Feature Flag hỗ trợ kiểm soát phạm vi phát hành và giảm rủi ro khi đưa tính năng mới vào môi trường production [1, 3, 10].

Các chức năng vượt mức MVP

Ngoài các chức năng bắt buộc theo yêu cầu của mini project, hệ thống còn triển khai nhiều chức năng thuộc mức khuyến khích nhằm nâng cao giá trị sử dụng và khả năng mở rộng của sản phẩm. Các chức năng này bao gồm cơ chế caching kết quả evaluation, JavaScript SDK hỗ trợ tích hợp với ứng dụng khách, phân quyền người dùng quản trị theo vai trò, thống kê số lần evaluation, versioning và audit log, group kill switch để vô hiệu hóa đồng thời nhiều Feature Flag, cùng với các bài kiểm thử cho Rule Evaluation Engine.

Việc triển khai các chức năng trên giúp hệ thống tiệm cận hơn với cách xây dựng của các nền tảng Feature Flag hiện đại, đồng thời chứng minh khả năng mở rộng vượt ngoài phạm vi MVP ban đầu. Các chức năng này cũng góp phần nâng cao hiệu năng, khả năng quản trị và tính an toàn trong quá trình vận hành hệ thống [8, 7, 12, 13].

Khả năng triển khai bằng Docker Compose

Đề tài xây dựng môi trường triển khai sử dụng Docker Compose nhằm đơn giản hóa quá trình cài đặt và khởi chạy toàn bộ hệ thống. Các thành phần như Backend API, Dashboard, Demo Application, PostgreSQL và các dịch vụ hỗ trợ được cấu hình hoạt động thông qua một tệp Docker Compose thống nhất, giúp giảm sự phụ thuộc vào môi trường cài đặt của từng máy phát triển [14].

Việc đóng gói hệ thống bằng container giúp rút ngắn thời gian thiết lập môi trường, tăng tính nhất quán giữa các môi trường phát triển và trình diễn, đồng thời tạo nền tảng để mở rộng theo hướng triển khai trên các hạ tầng container hóa trong tương lai. Đây cũng là một trong những chức năng khuyến khích được hoàn thành trong phạm vi của mini project.

4.3 Bài học kinh nghiệm

4.3.1 Bài học về phân tích bài toán

Xác định phạm vi MVP

Một trong những bài học quan trọng nhất của đề tài là cần xác định rõ phạm vi của sản phẩm tối thiểu khả thi (Minimum Viable Product – MVP) ngay từ giai đoạn phân tích. Feature Flag là một lĩnh vực có phạm vi rất rộng, bao gồm nhiều chức năng như đa biến thể (multi-variant), realtime streaming, SDK cho nhiều ngôn ngữ, đồng bộ phân tán, experiment platform hay phân

tích hành vi người dùng. Nếu cố gắng triển khai toàn bộ các chức năng này trong phạm vi mini project sẽ dẫn đến hệ thống quá phức tạp, khó hoàn thành đúng tiến độ và khó kiểm chứng.

Vì vậy, đề tài tập trung vào các chức năng cốt lõi phản ánh đúng bản chất của Feature Flag, bao gồm quản lý Feature Flag, cấu hình rule theo người dùng, vai trò và phần trăm rollout, Evaluation API, dashboard quản trị, audit log và ứng dụng minh họa. Việc giới hạn phạm vi giúp nhóm tập trung vào chất lượng thiết kế của Rule Evaluation Engine, tính đúng đắn của kết quả evaluation và khả năng minh họa rõ giá trị của Feature Flag trong quá trình phát hành phần mềm [1, 8].

Phân tách yêu cầu bắt buộc và mở rộng

Trong quá trình phân tích yêu cầu, đề tài nhận thấy việc phân chia giữa các chức năng bắt buộc và các chức năng mở rộng có ý nghĩa quan trọng đối với việc lập kế hoạch phát triển. Các yêu cầu MVP được ưu tiên triển khai trước nhằm đảm bảo hệ thống có thể hoạt động đầy đủ và đáp ứng các mục tiêu chính của đề tài. Sau khi hoàn thành các chức năng nền tảng, các yêu cầu mở rộng như caching, JavaScript SDK, phân quyền quản trị, versioning, evaluation metrics, group kill switch và Docker Compose mới được bổ sung từng bước.

Cách tiếp cận này giúp giảm rủi ro trong quá trình phát triển, đồng thời tạo điều kiện đánh giá tiến độ một cách rõ ràng. Ngoài ra, việc xây dựng các chức năng mở rộng trên nền tảng kiến trúc đã ổn định cũng giúp hạn chế việc phải thay đổi thiết kế ban đầu và tăng khả năng bảo trì của hệ thống trong tương lai [8, 3].

Chuyển yêu cầu thành use case và thiết kế

Một bài học quan trọng khác là yêu cầu nghiệp vụ cần được chuyển đổi thành các use case cụ thể trước khi tiến hành thiết kế hệ thống. Thay vì thiết kế trực tiếp từ danh sách chức năng, đề tài xây dựng các luồng sử dụng như quản trị Feature Flag, cấu hình rule, đánh giá trạng thái flag, ghi nhận audit log và tích hợp với ứng dụng khách. Từ các use case này, kiến trúc hệ thống, cơ sở dữ liệu, API và Rule Evaluation Engine được thiết kế theo hướng phục vụ trực tiếp các nghiệp vụ thực tế.

Quá trình này cũng giúp xác định rõ các thực thể dữ liệu, luồng xử lý, thứ tự ưu tiên của các rule và các trường hợp ngoại lệ cần xử lý trong Evaluation Engine. Nhờ đó, hệ thống có cấu trúc rõ ràng, dễ kiểm thử, dễ mở rộng và phù hợp với các nguyên tắc thiết kế của các nền tảng

Feature Flag hiện đại cũng như đặc tả Evaluation API và Evaluation Context của OpenFeature [10, 6].

4.3.2 Bài học về thiết kế hệ thống

Tầm quan trọng của evaluation precedence

Trong quá trình thiết kế Rule Evaluation Engine, bài học quan trọng nhất là cần xác định thứ tự ưu tiên (evaluation precedence) ngay từ đầu. Khi nhiều điều kiện có thể đồng thời áp dụng cho một Feature Flag, hệ thống phải luôn đưa ra cùng một kết quả đối với cùng một Evaluation Context. Nếu không có precedence rõ ràng, các lần đánh giá có thể cho kết quả khác nhau tùy theo thứ tự thực thi hoặc cách cài đặt, làm giảm tính xác định của hệ thống.

Đề tài xây dựng một thứ tự đánh giá cố định, trong đó các trạng thái như archived, disabled và kill switch được kiểm tra trước, tiếp theo là các rule targeting theo người dùng, vai trò và percentage rollout, cuối cùng mới sử dụng giá trị mặc định nếu không có rule nào phù hợp. Thiết kế này giúp Evaluation Engine hoạt động nhất quán, dễ kiểm thử và thuận lợi khi bổ sung thêm các loại rule mới mà không làm thay đổi hành vi của các rule hiện có [10, 4].

Phân tách định nghĩa flag và cấu hình runtime

Một bài học khác là cần tách biệt giữa định nghĩa của Feature Flag và cấu hình runtime. Ban đầu có thể lựa chọn lưu toàn bộ thông tin trong một thực thể duy nhất để đơn giản hóa thiết kế, tuy nhiên cách tiếp cận này nhanh chóng bộc lộ hạn chế khi cần quản lý nhiều môi trường triển khai hoặc thay đổi cấu hình mà vẫn giữ nguyên định nghĩa của flag.

Trong hệ thống được xây dựng, phần định nghĩa Feature Flag chỉ chứa các thông tin mang tính ổn định như tên, khóa định danh và mô tả, trong khi trạng thái hoạt động, serving mode, kill switch và tập các rule được quản lý thông qua cấu hình runtime. Việc phân tách này giúp dữ liệu có tính mô-đun hơn, giảm sự phụ thuộc giữa các thành phần và tạo nền tảng để mở rộng sang nhiều environment hoặc nhiều phiên bản cấu hình trong tương lai [7, 2].

Thiết kế auditability từ đầu

Quá trình phát triển cho thấy khả năng truy vết (auditability) cần được xem là một yêu cầu kiến trúc thay vì một chức năng bổ sung ở giai đoạn cuối. Nếu chỉ ghi nhận các thay đổi sau khi hệ thống đã hoàn thiện, nhiều thông tin quan trọng như actor, request ID hoặc trạng thái

trước và sau khi thay đổi sẽ khó được bổ sung đầy đủ.

Vì vậy, hệ thống được thiết kế với cơ chế audit log append-only ngay từ đầu, ghi nhận toàn bộ các thao tác thay đổi cấu hình cùng thông tin người thực hiện, thời điểm thực hiện và snapshot của dữ liệu trước và sau khi cập nhật. Thiết kế này không chỉ hỗ trợ việc kiểm tra và khắc phục sự cố mà còn giúp tăng tính minh bạch, khả năng truy vết và đáp ứng các nguyên tắc ghi nhật ký an toàn được khuyến nghị bởi OWASP [12, 11].

Cân bằng giữa tính đơn giản và khả năng mở rộng

Feature Flag là một lĩnh vực có nhiều chức năng nâng cao như realtime update, multi-variant, experiment platform hay distributed configuration. Tuy nhiên, không phải tất cả các khả năng này đều cần thiết trong phạm vi của một mini project. Một bài học quan trọng là cần cân bằng giữa tính đơn giản của thiết kế và khả năng mở rộng trong tương lai.

Đề tài lựa chọn chỉ triển khai các chức năng cốt lõi nhưng vẫn xây dựng kiến trúc theo hướng mô-đun. Rule Evaluation Engine được tách biệt khỏi tầng truy cập dữ liệu, API được thiết kế theo nguyên tắc REST, dữ liệu rule được lưu dưới dạng có thể mở rộng và các thành phần như cache, SDK hoặc các loại targeting mới có thể được bổ sung mà không phải thay đổi đáng kể kiến trúc hiện có. Cách tiếp cận này giúp hệ thống vừa đáp ứng đầy đủ yêu cầu của mini project, vừa tạo nền tảng thuận lợi cho các hướng phát triển tiếp theo [8, 1, 9].

4.3.3 Bài học về triển khai và kiểm thử

Validation tại ranh giới hệ thống

Trong quá trình triển khai, đề tài nhận thấy việc kiểm tra dữ liệu đầu vào ngay tại ranh giới của hệ thống có vai trò quan trọng đối với tính ổn định và an toàn của toàn bộ nền tảng. Các yêu cầu như định dạng khóa định danh, giá trị phần trăm rollout, trạng thái Feature Flag, Evaluation Context hay quyền truy cập cần được kiểm tra trước khi dữ liệu đi vào các tầng nghiệp vụ. Điều này giúp hạn chế dữ liệu không hợp lệ lan truyền vào cơ sở dữ liệu hoặc Rule Evaluation Engine, đồng thời đơn giản hóa việc xử lý lỗi ở các tầng phía sau.

Bên cạnh đó, việc chuẩn hóa cấu trúc phản hồi lỗi cùng request ID cũng giúp client dễ dàng xác định nguyên nhân sự cố và hỗ trợ quá trình theo dõi, gỡ lỗi trong quá trình vận hành. Đây cũng là cách tiếp cận phù hợp với các khuyến nghị về bảo vệ REST API và validation đầu vào của OWASP [11].

Kiểm thử các trường hợp precedence

Một bài học quan trọng trong kiểm thử là không chỉ xác nhận từng rule hoạt động đúng mà còn phải kiểm thử các trường hợp nhiều điều kiện cùng tồn tại. Các kịch bản như flag vừa bị disabled vừa được bật kill switch, người dùng đồng thời thuộc allowlist và role được cấp quyền, hoặc nhiều rule targeting có khả năng cùng khớp đều cần được kiểm thử để xác nhận Evaluation Engine luôn tuân thủ đúng evaluation precedence đã thiết kế.

Việc xây dựng các bộ kiểm thử theo từng nhóm precedence giúp phát hiện sớm các trường hợp xung đột trong logic đánh giá, đồng thời đảm bảo kết quả evaluation luôn mang tính xác định và có thể dự đoán được. Đây là một yêu cầu quan trọng đối với các hệ thống Feature Flag nhằm duy trì hành vi nhất quán trong môi trường production [4, 10].

Quản lý cache invalidation

Quá trình bổ sung cơ chế cache cho Evaluation Engine cho thấy việc lưu trữ dữ liệu chỉ là một phần của bài toán, trong khi cache invalidation mới là yếu tố quyết định tính đúng đắn của kết quả evaluation. Nếu cache không được làm mới sau khi thay đổi trạng thái Feature Flag hoặc cập nhật rule, ứng dụng có thể tiếp tục sử dụng cấu hình cũ và dẫn đến kết quả không phản ánh đúng trạng thái thực tế của hệ thống.

Do đó, mọi thao tác thay đổi cấu hình đều cần kích hoạt cơ chế invalidation tương ứng để đảm bảo các lần evaluation tiếp theo sử dụng dữ liệu mới nhất. Thiết kế này giúp cân bằng giữa hiệu năng truy vấn và tính nhất quán của cấu hình, đồng thời phù hợp với các nguyên tắc triển khai Feature Flag ở quy mô lớn [8, 13].

Xây dựng dữ liệu và kịch bản demo

Một bài học thực tiễn khác là chất lượng của dữ liệu mẫu và kịch bản demo ảnh hưởng trực tiếp đến khả năng đánh giá giá trị của hệ thống. Thay vì chỉ chuẩn bị dữ liệu tối thiểu để minh họa giao diện, đề tài xây dựng nhiều người dùng, vai trò, tổ chức và Feature Flag với các cấu hình khác nhau nhằm tái hiện các tình huống thường gặp trong thực tế.

Các kịch bản demo bao gồm bật hoặc tắt toàn cục, targeting theo người dùng, targeting theo vai trò, percentage rollout, kill switch, audit log và thống kê evaluation. Nhờ đó, hệ thống có thể minh họa đầy đủ quá trình quản trị Feature Flag từ cấu hình đến evaluation, đồng thời giúp việc kiểm thử tích hợp và trình diễn sản phẩm trở nên trực quan, thuyết phục và phản

ánh đúng mục tiêu của đề tài [1, 3, 5].

4.4 Hạn chế

4.4.1 Hạn chế về kiến trúc

Mặc dù hệ thống đã đáp ứng đầy đủ các chức năng của một nền tảng Feature Flag trong phạm vi mini project, kiến trúc hiện tại vẫn còn một số hạn chế khi xét tới khả năng triển khai ở quy mô lớn hoặc trong môi trường phân tán. Các hạn chế này chủ yếu liên quan đến khả năng mở rộng, đồng bộ cấu hình và đánh giá hiệu năng dưới tải cao, đồng thời cũng là những hướng phát triển phổ biến của các nền tảng Feature Flag thương mại như LaunchDarkly hay Unleash [7, 8].

Chưa hỗ trợ triển khai phân tán

Kiến trúc của hệ thống được thiết kế để triển khai trên một cụm dịch vụ đơn giản phục vụ mục tiêu học tập và trình diễn. Backend API, Evaluation Engine và Dashboard sử dụng chung cơ sở dữ liệu, trong khi cơ chế cache chủ yếu hoạt động trên từng instance của ứng dụng. Mô hình này giúp giảm độ phức tạp trong quá trình phát triển nhưng chưa giải quyết các bài toán thường gặp trong môi trường nhiều node như đồng bộ cache, cân bằng tải hay đảm bảo tính nhất quán giữa các instance.

Trong các hệ thống Feature Flag thực tế, Evaluation Service thường được triển khai trên nhiều máy chủ kết hợp với cơ chế cache phân tán và đồng bộ cấu hình nhằm duy trì độ trễ thấp và khả năng chịu lỗi cao [8, 7]. Đây là hướng mở rộng chưa được triển khai trong phạm vi của đề tài.

Chưa có cơ chế đồng bộ cấu hình realtime

Sau khi cấu hình Feature Flag được thay đổi trên Dashboard, các instance của hệ thống hiện dựa vào cơ chế cache invalidation và tải lại dữ liệu khi cần thiết. Hệ thống chưa hỗ trợ các cơ chế truyền sự kiện thời gian thực như WebSocket, Server-Sent Events hoặc message broker để thông báo ngay lập tức tới các Evaluation Service hoặc SDK đang hoạt động.

Do đó, trong trường hợp triển khai nhiều instance hoặc sử dụng cache với thời gian sống dài, có thể tồn tại khoảng thời gian ngắn mà các instance chưa đồng bộ hoàn toàn cấu hình mới.

Trong các nền tảng thương mại, việc đồng bộ cấu hình theo thời gian thực là một thành phần quan trọng nhằm giảm độ trễ giữa thao tác quản trị và thời điểm cấu hình có hiệu lực trên toàn hệ thống [7, 8].

Chưa đánh giá ở lưu lượng lớn

Các thử nghiệm của đề tài chủ yếu tập trung vào việc kiểm chứng tính đúng đắn của Rule Evaluation Engine, các kịch bản rollout và khả năng hoạt động của toàn bộ hệ thống trong môi trường demo. Chưa thực hiện các bài kiểm thử hiệu năng với số lượng lớn request đồng thời, nhiều instance hoặc khối lượng dữ liệu lớn.

Vì vậy, báo cáo chưa đưa ra các chỉ số định lượng về throughput, độ trễ, khả năng mở rộng theo chiều ngang hay mức sử dụng tài nguyên của hệ thống. Việc xây dựng bộ kiểm thử tải bằng các công cụ chuyên dụng và đánh giá khả năng mở rộng trong nhiều điều kiện triển khai khác nhau sẽ là hướng nghiên cứu và hoàn thiện tiếp theo.

4.4.2 Hạn chế về chức năng

Bên cạnh việc đáp ứng đầy đủ các chức năng của mức MVP và hầu hết các chức năng khuyến khích trong đề bài, hệ thống vẫn còn một số hạn chế về mặt chức năng so với các nền tảng Feature Flag thương mại. Các hạn chế này chủ yếu nhằm giữ phạm vi mini project ở mức phù hợp, giảm độ phức tạp trong thiết kế và tập trung vào việc làm rõ nguyên lý hoạt động của Feature Flag thay vì xây dựng một nền tảng hoàn chỉnh [7, 8].

Số loại rule còn giới hạn

Rule Evaluation Engine hiện hỗ trợ ba loại rule chính gồm targeting theo người dùng, targeting theo vai trò và percentage rollout. Đây là các chiến lược phổ biến, đủ để minh họa nguyên lý tách biệt giữa deploy và release cũng như đáp ứng các yêu cầu của đề tài.

Tuy nhiên, hệ thống chưa hỗ trợ các rule phức tạp như điều kiện theo nhiều thuộc tính kết hợp, biểu thức logic AND/OR lồng nhau, targeting theo thời gian, vị trí địa lý, phiên bản ứng dụng hoặc các phân khúc người dùng được tạo động. Trong các nền tảng Feature Flag hiện đại, những khả năng này giúp đáp ứng nhiều kịch bản phát hành sản phẩm và cá nhân hóa hơn [4, 6].

Chưa hỗ trợ nhiều variant đầy đủ

Hệ thống hiện tập trung vào mô hình Feature Flag dạng bật hoặc tắt, đồng thời hỗ trợ trả về giá trị variant cơ bản phục vụ quá trình evaluation. Tuy nhiên, chưa xây dựng đầy đủ cơ chế quản lý nhiều variant với trọng số phân phối, cấu hình riêng cho từng variant hoặc rollout độc lập theo từng biến thể.

Việc mở rộng sang mô hình đa variant sẽ cho phép triển khai các kịch bản như A/B testing, multivariate testing hoặc phát hành nhiều phiên bản giao diện đồng thời. Đây là một trong những khả năng quan trọng của các nền tảng Feature Flag thương mại nhưng nằm ngoài phạm vi của mini project [7, 1].

Chưa hỗ trợ approval workflow

Mọi thay đổi cấu hình Feature Flag trong hệ thống đều được áp dụng ngay sau khi người dùng có đủ quyền thực hiện thao tác. Hệ thống đã ghi nhận đầy đủ audit log và lịch sử thay đổi nhưng chưa hỗ trợ quy trình phê duyệt trước khi cấu hình được áp dụng.

Trong môi trường doanh nghiệp, các thay đổi quan trọng thường phải trải qua quy trình nhiều bước như tạo yêu cầu, xem xét, phê duyệt và mới được phát hành nhằm giảm nguy cơ cấu hình sai hoặc thay đổi ngoài ý muốn. Chức năng này chưa được triển khai do vượt quá phạm vi của đề tài.

Chưa hỗ trợ stale flag detection

Hệ thống chưa cung cấp cơ chế tự động phát hiện các Feature Flag không còn được sử dụng hoặc đã tồn tại quá lâu trong mã nguồn. Việc quản lý vòng đời của Feature Flag hiện vẫn dựa trên thao tác thủ công của người quản trị.

Trong thực tế, việc tồn tại quá nhiều Feature Flag không còn giá trị sẽ làm tăng độ phức tạp của mã nguồn, gây khó khăn cho bảo trì và hình thành *feature flag debt*. Nhiều nền tảng hiện nay cung cấp khả năng theo dõi mức độ sử dụng, cảnh báo stale flag hoặc hỗ trợ dọn dẹp các flag không còn cần thiết nhằm duy trì chất lượng hệ thống [9, 8].

4.4.3 Hạn chế về bảo mật và vận hành

Hệ thống đã áp dụng các cơ chế xác thực, phân quyền, validation dữ liệu, audit log và cache invalidation nhằm đáp ứng yêu cầu của đề tài. Tuy nhiên, nếu triển khai trong môi trường sản

xuất với nhiều người dùng và lưu lượng truy cập lớn, hệ thống vẫn cần được bổ sung nhiều cơ chế về bảo mật và vận hành để đáp ứng các yêu cầu về tính sẵn sàng, khả năng giám sát và an toàn dữ liệu [11, 12].

Mô hình xác thực còn đơn giản

Hệ thống sử dụng cơ chế xác thực và phân quyền phù hợp với phạm vi của mini project, cho phép phân biệt các vai trò quản trị như administrator, developer và viewer. Tuy nhiên, mô hình xác thực hiện vẫn còn đơn giản và chưa tích hợp với các hệ thống quản lý danh tính phổ biến như OAuth 2.0, OpenID Connect hoặc Single Sign-On (SSO).

Ngoài ra, hệ thống cũng chưa hỗ trợ các cơ chế như xác thực đa yếu tố (Multi-Factor Authentication), quản lý phiên đăng nhập tập trung hay chính sách quản lý khóa truy cập dành cho các hệ thống tích hợp. Đây đều là những yêu cầu thường gặp đối với các nền tảng quản trị trong môi trường doanh nghiệp [11].

Chưa có rate limiting đầy đủ

Các API của hệ thống hiện chưa triển khai cơ chế giới hạn tốc độ truy cập đối với từng người dùng hoặc từng địa chỉ IP. Trong điều kiện bình thường, điều này không ảnh hưởng đến việc trình diễn và đánh giá chức năng của hệ thống. Tuy nhiên, khi triển khai thực tế, việc thiếu rate limiting có thể làm tăng nguy cơ bị khai thác thông qua các cuộc tấn công gửi số lượng lớn request hoặc gây quá tải cho dịch vụ.

Việc bổ sung rate limiting, quota theo API, giới hạn số lần evaluation trong một khoảng thời gian và cơ chế phát hiện bất thường sẽ góp phần nâng cao khả năng bảo vệ hệ thống trước các hành vi truy cập không hợp lệ [11].

Chưa có hệ thống monitoring chuyên biệt

Hệ thống hiện ghi nhận audit log và thống kê số lần evaluation phục vụ việc quản trị Feature Flag, nhưng chưa tích hợp các nền tảng monitoring chuyên biệt để theo dõi trạng thái vận hành của toàn bộ hệ thống.

Các chỉ số như thời gian phản hồi của Evaluation API, tỷ lệ lỗi, mức sử dụng CPU, bộ nhớ, trạng thái cache hay cảnh báo khi dịch vụ gặp sự cố vẫn chưa được thu thập và trực quan hóa. Trong môi trường sản xuất, việc kết hợp các công cụ monitoring và alerting sẽ giúp phát hiện

sớm sự cố, rút ngắn thời gian xử lý và nâng cao độ tin cậy của hệ thống.

Chưa triển khai cơ chế backup và recovery

Hệ thống chưa xây dựng quy trình sao lưu và khôi phục dữ liệu cho cơ sở dữ liệu cũng như các thành phần cấu hình của Feature Flag. Mặc dù toàn bộ thay đổi đều được lưu trong audit log và dữ liệu được quản lý tập trung, nhưng chưa có cơ chế tự động tạo bản sao lưu định kỳ hoặc phục hồi sau sự cố.

Trong môi trường triển khai thực tế, việc xây dựng chiến lược backup và disaster recovery là cần thiết nhằm giảm thiểu nguy cơ mất dữ liệu do lỗi phần cứng, thao tác nhầm hoặc sự cố hệ thống. Đây là một thành phần quan trọng để đảm bảo tính sẵn sàng và khả năng vận hành liên tục của nền tảng.

4.5 Hướng phát triển

4.6 Mở rộng Rule Evaluation Engine

Mặc dù Rule Evaluation Engine đã đáp ứng đầy đủ các yêu cầu của mini project với các chiến lược bật hoặc tắt toàn cục, targeting theo người dùng, targeting theo vai trò và percentage rollout, hệ thống vẫn còn nhiều hướng mở rộng nhằm đáp ứng các nhu cầu triển khai trong môi trường sản xuất quy mô lớn. Việc mở rộng engine theo hướng mô-đun sẽ giúp bổ sung các loại rule mới mà không ảnh hưởng đến các rule hiện có, đồng thời phù hợp với định hướng phát triển của các nền tảng Feature Flag hiện đại [7, 8].

4.6.1 Targeting theo thuộc tính

Hiện tại hệ thống chủ yếu đánh giá dựa trên `userId`, `role` và `targetingKey`. Trong tương lai, Evaluation Engine có thể hỗ trợ nhiều thuộc tính hơn như loại thiết bị, hệ điều hành, phiên bản ứng dụng, gói dịch vụ, quốc gia hoặc các thuộc tính nghiệp vụ do ứng dụng khách cung cấp thông qua Evaluation Context.

Các điều kiện này có thể được biểu diễn dưới dạng tập luật với các toán tử so sánh như bằng, khác, thuộc tập giá trị, so khớp chuỗi hoặc biểu thức chính quy. Cách tiếp cận này giúp hệ thống linh hoạt hơn trong việc triển khai tính năng cho các nhóm người dùng cụ thể mà không cần thay đổi mã nguồn của ứng dụng khách. Hướng phát triển này cũng phù hợp với mô hình

Evaluation Context được đề xuất trong OpenFeature [6].

4.6.2 Targeting theo organization

Trong nhiều hệ thống SaaS, Feature Flag thường được quản lý theo từng khách hàng hoặc từng tổ chức thay vì từng người dùng riêng lẻ. Do đó, một hướng phát triển quan trọng là bổ sung khả năng targeting dựa trên `organizationId` hoặc các thuộc tính của tổ chức.

Khi đó, Evaluation Engine có thể kích hoạt hoặc vô hiệu hóa tính năng cho toàn bộ doanh nghiệp, nhóm khách hàng hoặc tenant cụ thể. Việc này giúp đơn giản hóa quá trình quản trị khi số lượng người dùng lớn, đồng thời hỗ trợ triển khai các gói tính năng khác nhau cho từng khách hàng doanh nghiệp mà vẫn sử dụng chung một hệ thống.

4.6.3 Targeting theo thời gian và vị trí

Một hướng phát triển khác là bổ sung các rule phụ thuộc vào thời gian hoặc vị trí địa lý. Ví dụ, Feature Flag chỉ được kích hoạt trong một khoảng thời gian xác định để phục vụ chiến dịch marketing, sự kiện đặc biệt hoặc quá trình phát hành theo từng giai đoạn.

Ngoài ra, hệ thống có thể hỗ trợ triển khai tính năng theo khu vực hoặc quốc gia nhằm đáp ứng yêu cầu pháp lý hoặc chiến lược phát hành riêng của từng thị trường. Để thực hiện điều này, Evaluation Context cần được mở rộng với các thông tin về thời gian, múi giờ hoặc vị trí, đồng thời Evaluation Engine cần bổ sung các bộ đánh giá tương ứng mà vẫn đảm bảo thứ tự ưu tiên giữa các loại rule.

4.6.4 Hỗ trợ nhiều variant và A/B testing

Phiên bản hiện tại chủ yếu trả về kết quả bật hoặc tắt cùng một giá trị `variant` đơn giản. Trong tương lai, hệ thống có thể mở rộng để hỗ trợ nhiều biến thể của cùng một Feature Flag, cho phép phân phối nhiều phiên bản giao diện hoặc thuật toán khác nhau đến các nhóm người dùng.

Trên cơ sở đó, nền tảng có thể hỗ trợ các kịch bản A/B testing hoặc multivariate testing, trong đó người dùng được phân bổ ổn định vào từng biến thể thông qua cơ chế băm nhất quán. Kết hợp với hệ thống thống kê evaluation và các chỉ số nghiệp vụ, kết quả thử nghiệm có thể được sử dụng để đánh giá hiệu quả của từng biến thể trước khi triển khai rộng rãi. Đây là một trong những khả năng quan trọng của các nền tảng Feature Flag hiện đại và là hướng phát triển phù

hợp cho hệ thống trong tương lai [5, 1].

4.7 Mở rộng khả năng tích hợp

Trong phạm vi mini project, hệ thống đã cung cấp REST API và JavaScript SDK để các ứng dụng web có thể tích hợp và sử dụng Feature Flag. Tuy nhiên, nhằm đáp ứng nhu cầu triển khai trong nhiều môi trường và ngôn ngữ lập trình khác nhau, nền tảng vẫn còn nhiều hướng phát triển về khả năng tích hợp với các hệ thống bên ngoài, đồng thời nâng cao khả năng đồng bộ cấu hình và tính tương thích với các tiêu chuẩn mở của ngành [10, 7].

4.7.1 SDK cho nhiều ngôn ngữ

Hiện nay SDK mới được xây dựng cho ứng dụng JavaScript nhằm minh họa quá trình gọi Evaluation API. Trong tương lai, hệ thống có thể phát triển thêm SDK cho các ngôn ngữ phổ biến như Java, Go, Python, C#, PHP hoặc Kotlin để hỗ trợ nhiều loại ứng dụng khác nhau.

Các SDK nên cung cấp giao diện lập trình thống nhất, hỗ trợ quản lý Evaluation Context, xử lý lỗi, caching cục bộ và trả về kết quả đánh giá theo cùng một định dạng. Điều này giúp giảm đáng kể khối lượng mã nguồn tích hợp phía ứng dụng khách, đồng thời đảm bảo hành vi đánh giá Feature Flag nhất quán trên nhiều nền tảng.

4.7.2 Webhook khi cấu hình thay đổi

Một hướng phát triển khác là bổ sung cơ chế webhook để thông báo cho các hệ thống bên ngoài khi cấu hình Feature Flag thay đổi. Mỗi khi Feature Flag, Rule hoặc Environment được cập nhật, hệ thống có thể gửi sự kiện đến các URL đã đăng ký trước.

Webhook giúp các hệ thống tích hợp chủ động thực hiện các tác vụ như làm mới cache, đồng bộ dữ liệu, ghi nhận sự kiện hoặc kích hoạt quy trình CI/CD mà không cần liên tục truy vấn Management API. Cơ chế này cũng góp phần giảm lưu lượng truy cập và tăng khả năng mở rộng của toàn hệ thống.

4.7.3 Đồng bộ cấu hình theo thời gian thực

Phiên bản hiện tại chủ yếu sử dụng cơ chế cache kết hợp với invalidation sau khi có thay đổi cấu hình. Trong tương lai, hệ thống có thể mở rộng theo hướng đồng bộ cấu hình theo thời gian thực thông qua các công nghệ như WebSocket hoặc Server-Sent Events.

Khi đó, các SDK hoặc Evaluation Service có thể nhận ngay thông tin cập nhật mà không cần polling định kỳ. Việc đồng bộ theo thời gian thực giúp giảm độ trễ giữa thời điểm thay đổi cấu hình và thời điểm áp dụng trong ứng dụng khách, đồng thời cải thiện tính nhất quán của dữ liệu trong môi trường có nhiều instance triển khai [8].

4.7.4 Tương thích OpenFeature

OpenFeature đang dần trở thành tiêu chuẩn mở cho việc tích hợp Feature Flag giữa nhiều nền tảng khác nhau. Do đó, một hướng phát triển quan trọng là điều chỉnh SDK và Evaluation API để tương thích với các đặc tả của OpenFeature.

Việc hỗ trợ đầy đủ các khái niệm như Provider, Evaluation Context, Resolution Details và Evaluation API sẽ giúp ứng dụng khách có thể chuyển đổi giữa nhiều nhà cung cấp Feature Flag mà không cần thay đổi đáng kể mã nguồn. Điều này không chỉ nâng cao khả năng tái sử dụng của hệ thống mà còn tạo điều kiện để nền tảng dễ dàng tích hợp với hệ sinh thái Feature Flag hiện đại trong tương lai [6, 10].

4.8 Mở rộng quản trị vòng đời flag

Trong phiên bản hiện tại, hệ thống đã hỗ trợ các trạng thái cơ bản của Feature Flag cùng khả năng lưu vết thay đổi thông qua audit log và versioning. Tuy nhiên, việc quản trị vòng đời của Feature Flag trong môi trường sản xuất còn bao gồm nhiều quy trình liên quan đến phê duyệt, lập lịch triển khai và quản lý các flag không còn sử dụng. Đây là những chức năng thường xuất hiện trong các nền tảng Feature Flag quy mô lớn nhằm giảm rủi ro vận hành và hạn chế phát sinh technical debt [1, 8].

4.8.1 Approval workflow

Một hướng phát triển quan trọng là xây dựng quy trình phê duyệt trước khi các thay đổi cấu hình được áp dụng vào môi trường production. Thay vì cho phép cập nhật trực tiếp, các thao tác quan trọng như bật hoặc tắt Feature Flag, thay đổi rule hoặc điều chỉnh rollout có thể được chuyển sang trạng thái chờ phê duyệt.

Quy trình này có thể phân tách vai trò giữa người đề xuất và người phê duyệt, đồng thời lưu đầy đủ lịch sử của từng bước xử lý trong audit log. Cách tiếp cận này giúp giảm nguy cơ cấu

hình sai, tăng khả năng kiểm soát thay đổi và đáp ứng tốt hơn các yêu cầu về quản trị trong doanh nghiệp.

4.8.2 Lập lịch bật và tắt flag

Hiện tại mọi thay đổi đều được thực hiện thủ công thông qua Dashboard hoặc API. Trong tương lai, hệ thống có thể hỗ trợ lập lịch tự động kích hoạt hoặc vô hiệu hóa Feature Flag tại một thời điểm xác định.

Chức năng này đặc biệt hữu ích đối với các chiến dịch marketing, sự kiện theo mùa hoặc kế hoạch phát hành đã được chuẩn bị trước. Việc tự động thực thi theo lịch giúp giảm thao tác thủ công, hạn chế sai sót của con người và đảm bảo các thay đổi được áp dụng đúng thời điểm dự kiến.

4.8.3 Phát hiện stale flag

Sau khi một tính năng đã được triển khai ổn định, nhiều Feature Flag không còn giá trị sử dụng nhưng vẫn tồn tại trong hệ thống và trong mã nguồn ứng dụng. Điều này làm tăng độ phức tạp của logic xử lý và gây khó khăn cho quá trình bảo trì.

Do đó, hệ thống có thể được mở rộng để phát hiện các stale flag dựa trên nhiều tiêu chí như thời gian không cập nhật, thời gian không được evaluate, trạng thái luôn bật hoặc luôn tắt trong thời gian dài hoặc không còn được tham chiếu trong ứng dụng. Việc nhận diện sớm các stale flag giúp giảm technical debt và duy trì hệ thống ở trạng thái dễ quản lý hơn [1, 9].

4.8.4 Nhắc nhở và tự động dọn dẹp flag

Sau khi phát hiện stale flag, nền tảng có thể tiếp tục hỗ trợ cơ chế nhắc nhở hoặc tự động xử lý theo các chính sách được cấu hình trước. Ví dụ, hệ thống có thể gửi thông báo cho quản trị viên khi Feature Flag vượt quá thời gian tồn tại dự kiến hoặc đề xuất chuyển sang trạng thái lưu trữ.

Trong các phiên bản nâng cao, hệ thống còn có thể hỗ trợ quy trình tự động archive hoặc đánh dấu các Feature Flag cần được loại bỏ sau khi đã hoàn tất quá trình kiểm tra. Việc quản lý vòng đời theo hướng chủ động giúp hạn chế sự tích lũy của Feature Flag không còn sử dụng, giảm chi phí bảo trì và phù hợp với các thực tiễn được khuyến nghị trong các nền tảng Feature Flag hiện đại [8, 9].

4.9 Mở rộng khả năng vận hành

Phiên bản hiện tại đáp ứng tốt mục tiêu nghiên cứu và triển khai trong phạm vi mini project với quy mô nhỏ. Tuy nhiên, khi số lượng Feature Flag, người dùng và yêu cầu truy cập đồng thời tăng lên, hệ thống cần được mở rộng về khả năng vận hành nhằm đảm bảo tính sẵn sàng, hiệu năng và độ tin cậy. Các hướng phát triển dưới đây tập trung vào việc nâng cao khả năng mở rộng của hạ tầng mà vẫn giữ nguyên kiến trúc tổng thể của nền tảng [8, 14].

4.9.1 High availability

Một hướng phát triển quan trọng là triển khai nhiều instance cho Backend API và Dashboard nhằm loại bỏ điểm lỗi đơn (Single Point of Failure). Các instance có thể được đặt sau bộ cân bằng tải để phân phối lưu lượng truy cập và tự động chuyển hướng khi một nút gặp sự cố.

Bên cạnh đó, cơ sở dữ liệu cũng có thể được mở rộng theo mô hình primary–replica nhằm tăng khả năng chịu lỗi và cải thiện hiệu năng đọc. Cách tiếp cận này giúp Evaluation API duy trì khả năng phục vụ ngay cả khi một phần hạ tầng gặp sự cố, từ đó nâng cao độ sẵn sàng của toàn hệ thống.

4.9.2 Distributed cache

Hiện tại hệ thống sử dụng cơ chế cache đơn giản phục vụ quá trình evaluation. Trong môi trường có nhiều Backend API hoạt động đồng thời, mỗi instance sẽ có bộ nhớ đệm riêng, dẫn đến nguy cơ dữ liệu không đồng bộ sau khi cấu hình thay đổi.

Để khắc phục hạn chế này, hệ thống có thể sử dụng distributed cache dựa trên Redis hoặc các giải pháp tương tự làm tầng cache dùng chung giữa các instance. Kết hợp với cơ chế cache invalidation hoặc publish/subscribe, mọi thay đổi cấu hình có thể được đồng bộ nhanh chóng đến toàn bộ hệ thống, góp phần giảm độ trễ và duy trì tính nhất quán của dữ liệu [13].

4.9.3 Monitoring và alerting

Một hướng phát triển khác là xây dựng hệ thống giám sát và cảnh báo chuyên biệt cho nền tảng Feature Flag. Các chỉ số như số lượng evaluation, thời gian phản hồi API, tỷ lệ lỗi, số lần cache hit hoặc cache miss và mức sử dụng tài nguyên có thể được thu thập và trực quan hóa thông qua các công cụ giám sát.

Kết hợp với cơ chế cảnh báo tự động, hệ thống có thể nhanh chóng phát hiện các bất thường như tỷ lệ lỗi tăng cao, Evaluation API không khả dụng hoặc số lượng request vượt ngưỡng cho phép. Điều này giúp đội ngũ vận hành phản ứng kịp thời và giảm thiểu ảnh hưởng đến các ứng dụng đang sử dụng Feature Flag.

4.9.4 Triển khai trên Kubernetes

Phiên bản hiện tại sử dụng Docker Compose để đơn giản hóa quá trình triển khai trong môi trường phát triển và trình diễn. Trong tương lai, hệ thống có thể được mở rộng để triển khai trên Kubernetes nhằm đáp ứng yêu cầu vận hành ở quy mô lớn.

Việc sử dụng Kubernetes cho phép tự động mở rộng số lượng instance theo tải hệ thống, hỗ trợ rolling update, tự động khởi động lại các dịch vụ gặp lỗi và quản lý tài nguyên hiệu quả hơn. Đồng thời, nền tảng cũng có thể tận dụng các khả năng như Service Discovery, ConfigMap, Secret và Horizontal Pod Autoscaler để nâng cao tính linh hoạt và khả năng vận hành trong môi trường production [14].

Tài liệu tham khảo

[1] Pete Hodgson. *Feature Toggles (aka Feature Flags)*. Martin Fowler. 9 **october** 2017. URL: https://martinfowler.com/articles/feature-toggles.html.

[2] Unleash. *Feature Flags*. Unleash Documentation. URL: https://docs.getunleash.io/concepts/feature-flags.

[3] Microsoft. *How Microsoft Develops with DevOps*. Microsoft Learn. 28 **november** 2022. URL: https://learn.microsoft.com/en-us/devops/develop/how-microsoft-develops-devops.

[4] LaunchDarkly. *Targeting Rules*. LaunchDarkly Documentation. URL: https://launchdarkly.com/docs/home/flags/target-rules.

[5] LaunchDarkly. *Percentage Rollouts*. LaunchDarkly Documentation. URL: https://launchdarkly.com/docs/home/releases/percentage-rollouts.

[6] OpenFeature. *Evaluation Context*. OpenFeature Specification. URL: https://openfeature.dev/specification/sections/evaluation-context/.

[7] LaunchDarkly. *Feature Flags*. LaunchDarkly Documentation. URL: https://docs.launchdarkly.com/home/flags.

[8] Unleash. *11 Best Practices for Building and Scaling Feature Flag Systems*. Unleash Documentation. URL: https://docs.getunleash.io/topics/feature-flags/feature-flag-best-practices.

[9] Rezvan Mahdavi-Hezaveh, Jacob Dremann **and** Laurie Williams. **?**Software Development with Feature Toggles: Practices Used by Practitioners**? in***Empirical Software Engineering*:

26, 1 (8 **january** 2021). DOI: 10.1007/s10664-020-09901-z. URL: https://link.springer.com/article/10.1007/s10664-020-09901-z.

[10] OpenFeature. *Flag Evaluation API*. OpenFeature Specification. URL: https://openfeature.dev/specification/sections/flag-evaluation/.

[11] OWASP Foundation. *REST Security Cheat Sheet*. OWASP Cheat Sheet Series. URL: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html.

[12] OWASP Foundation. *Logging Cheat Sheet*. OWASP Cheat Sheet Series. URL: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html.

[13] Redis. *Client-Side Caching Introduction*. Redis Documentation. URL: https://redis.io/docs/latest/develop/clients/client-side-caching.

[14] Docker. *Docker Compose*. Docker Documentation. URL: https://docs.docker.com/compose/.

[15] PostgreSQL Global Development Group. *JSON Types*. PostgreSQL Documentation. URL: https://www.postgresql.org/docs/current/datatype-json.html.